DEVOPS ENGINEERING MASTERY

GIT, SCRIPTING & AUTOMATION ENGINEERING

Git, Bash, Python & Software Automation

VOLUME 03 • FIRST EDITION

```
nahid@ubuntu-server:~$ git log --oneline --graph
* a1b2c3d (HEAD -> main) Add health-check script
* 9f8e7d6 Merge feature/backup-automation
nahid@ubuntu-server:~$ ./health-check.sh
OK: disk 42% • memory 61% • nginx running
nahid@ubuntu-server:~$ python3 -m pytest tests/ -q
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

Git 2.43 • Bash 5.2 • Python 3.12 • pytest 9.2

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Git, Scripting & Automation Engineering

DEVOPS ENGINEERING MASTERY — Volume 03
Git, Bash, Python ও Software Automation

First Edition — Published 8 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS (Noble Numbat, released April 2024, supported until April 2029). Versions verified at publication on the build machine: Git 2.43.0, GNU Bash 5.2.21, Python 3.12.3, pytest 9.2.0, PyYAML 6.0.3, requests 2.34.2. All commands in this book were written for Ubuntu 24.04 LTS unless a section explicitly says otherwise. Sample terminal outputs are illustrative — your output will differ in dates, usernames, commit hashes, PIDs and sizes, and that is expected.

How to use this book. You have completed foundational Linux and networking study (Volumes 01–02) and know basic terminal commands and some programming. Read the parts in order: Git first, because every later automation tool lives in a repository; then Bash, then Python, then scheduling, SSH and production practice. Do every hands-on exercise and every lab in Appendix A before moving on. Do not copy commands blindly; read the explanation under each command first. Destructive commands are always flagged with a WARNING box and a safe alternative. Practise them only in a virtual machine, never on production.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches version control, scripting and automation judgment — but the judgment itself comes from doing the labs, breaking a test VM, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work. Code blocks contain no explanatory comments unless the example specifically teaches comments.

Security. No passwords, tokens or private credentials appear anywhere in the examples. Secrets are always read from environment variables. The book never recommends disabling TLS verification or host-key checking as a normal solution, and explains command-injection risks wherever user input meets a shell or a subprocess.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Commands that change a system should first be practised in a virtual machine or a disposable test server, never directly on a production machine. The author and publisher accept no liability for data loss caused by running commands without a backup.

Who this book is for

A learner who has finished foundational Linux and networking study, understands basic terminal commands, has some web development and programming experience, and wants to become a professional DevOps

and infrastructure engineer — with about three hours a day for study. No prior knowledge of Git internals, shell scripting or production automation is assumed.

Learning outcomes

By the end of this book you will be able to: explain version control and Git architecture (commits, trees, blobs, refs, history); manage branches, merges, rebases and conflicts; use GitHub professionally, including pull requests and code review; write safe, reusable Bash scripts; use Python for system and infrastructure automation; automate files, processes, backups and system checks; work with HTTP APIs, JSON, YAML and environment variables; implement logging, exception handling and exit codes; schedule automation with cron; run remote administration over SSH; and build, test, document and maintain reusable automation tools.

সূচিপত্র — Contents

PART I — Git Fundamentals & Version Control

Chapter 01 — Introduction to Version Control Systems 7

Chapter 02 — Git Architecture and Internal Working 14

Chapter 03 — Git Repositories, Commits and History 21

Chapter 04 — Branching, Merging and Rebasing 28

Chapter 05 — Remote Repositories and GitHub 35

Chapter 06 — Professional Git Workflow and Collaboration 42

PART II — Bash Scripting Engineering

Chapter 07 — Introduction to Shell Scripting 50

Chapter 08 — Variables, Data Types and User Input 59

Chapter 09 — Conditions, Loops and Control Flow 68

Chapter 10 — Functions, Arguments and Script Structure 76

Chapter 11 — Files, Text Processing and Command Pipelines 84

Chapter 12 — Error Handling, Exit Codes and Debugging 91

Chapter 13 — Advanced Bash Automation and Best Practices 99

PART III — Python for DevOps

Chapter 14 — Python Fundamentals for Automation 108

Chapter 15 — Files, Directories and System Operations 115

Chapter 16 — Processes, Subprocesses and Command Execution 125

Chapter 17 — APIs, HTTP Requests and JSON Processing 136

Chapter 18 — YAML, Configuration and Environment Management 145

Chapter 19 — Logging, Exceptions and Reliable Automation 151

PART IV — Automation Engineering

Chapter 20 — Scheduling, Cron and Background Automation 161

Chapter 21 — SSH Automation and Remote System Management 170

Chapter 22 — Building Reusable Automation Tools 178

PART V — Professional Engineering Workflow

Chapter 23 — Code Quality, Testing and Documentation 187

Chapter 24 — Automation Architecture and Production Practices 194

Appendices

Appendix A — Guided Labs (24 Labs) .

Appendix B — Independent Assignments (15) .

Appendix C — Major Projects (1, 2, 3 — TechNova) .

Appendix D — Quiz & Final Examination Answer Guide (120 MCQs) 265

Appendix E — Glossary (197 Terms) 288

Appendix F — References 297

Chapter-end quizzes (120 MCQs), short-answer and code-reading exercises, debugging exercises and the troubleshooting case studies are inside each chapter and Appendix D. Their answers — with explanations, not just option letters — are in the separate companion volume, *Quiz & Answer Guide*. The hands-on material is also collected in the separate *Practical Lab & Assignment Workbook*.

PART I • DEVOPS ENGINEERING MASTERY

Git Fundamentals & Version Control

ভার্সন কন্ট্রোলের প্রথম নীতি থেকে — Git-এর ভেতরের গঠন, কমিট ও ইতিহাস, ব্রাঞ্চ ও মার্জ, GitHub ও দলগত কাজ।

অধ্যায় ১ • Part I — Git Fundamentals & Version Control

Chapter 01: Introduction to Version Control Systems — Version Control ব্যবস্থার পরিচয়

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একটি ছোট software company, TechNova-তে চারজন developer একসঙ্গে একটি web application বানাচ্ছিল। একদিন সকালে জুনিয়র developer তানভীর একটি ফাইলে নতুন feature যোগ করল। দুপুরে সিনিয়র developer মেহজাবিন একই ফাইলে অন্য একটি bug ঠিক করল। বিকেলে কেউ একজন ভুল করে পুরনো একটি কপি দিয়ে নতুন ফাইলটি চাপা দিয়ে দিল। সন্ধ্যায় গিয়ে দেখা গেল দুজনের কাজই হারিয়ে গেছে, কে কী বদলেছিল তার কোনো হিসাব নেই, আর client-কে দেওয়ার মতো কাজ করা version-টি কোথায় আছে কেউ জানে না। দলটি সারা রাত জেগে কোড আবার লিখল। এই ঘটনা কোনো ব্যতিক্রম নয় — version control ছাড়া প্রায় প্রতিটি দলই একদিন এই অভিজ্ঞতার মুখোমুখি হয়। এই অধ্যায়ে আমরা শিখব কেন এই সমস্যা ঘটে, আর version control কীভাবে তাকে স্থায়ীভাবে সমাধান করে।

একজন একা developer-এর কাছেও সমস্যাটি একই রকম, শুধু ছোট আকারে। আপনি একটি project-এ কাজ করছেন, একটি নতুন পরীক্ষা করলেন, সেটি ভেঙে গেল, আর আগের ভালো অবস্থায় ফিরতে পারলেন না — কারণ আগের অবস্থাটি কোথাও সংরক্ষিত ছিল না। Version control হলো সেই সংরক্ষণ ও হিসাবের ব্যবস্থা, যা প্রতিটি পরিবর্তনকে নাম, তারিখ, লেখক ও কারণসহ জমা রাখে।

> **CONCEPT:** Version control মানে শুধু ফাইলের backup নয়। Backup বলে "গতকাল ফাইলটি কেমন ছিল", version control বলে "গতকাল কে, কেন, ঠিক কী বদলেছিল, আর সেই বদলটি কোন বদলের উপর ভিত্তি করে এসেছিল"। পার্থক্যটি একটি ছবির অ্যালবাম আর একটি ইতিহাস বইয়ের পার্থক্যের মতো।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Version control কী এবং কেন এটি software engineering-এর ভিত্তি, তা নিজের ভাষায় ব্যাখ্যা করতে।
- Manual backup, ফাইলের নামে তারিখ লেখা ও version control-এর পার্থক্য বিশ্লেষণ করতে।
- Local, centralized ও distributed version control system-এর মধ্যে পার্থক্য বলতে ও প্রতিটির সুবিধা-অসুবিধা মূল্যায়ন করতে।
- Git-এর ইতিহাস ও দর্শন — কেন Git distributed হলো, তা ব্যাখ্যা করতে।
- Git ও GitHub-এর পার্থক্য স্পষ্টভাবে বলতে।
- একটি বাস্তব দলের collaboration পরিস্থিতিতে version control কীভাবে সাহায্য করে, তা বর্ণনা করতে।

Prerequisites — পূর্বজ্ঞান

- Terminal-এ ফাইল তৈরি, কপি ও নাম বদলের প্রাথমিক অভিজ্ঞতা (Volume 01-এর Linux অধ্যায়গুলো)।
- একটি প্রোগ্রামিং ভাষায় ছোট program লেখার অভিজ্ঞতা।
- কোনো project-এ একাধিক ফাইল নিয়ে কাজ করার অভিজ্ঞতা থাকলে ভালো, না থাকলেও চলবে।

১.১ Version control আসলে কী?

Version Control System (VCS) হলো এমন একটি ব্যবস্থা যা একটি project-এর ফাইলগুলোর প্রতিটি পরিবর্তনকে সময়ের ক্রমে ধরে রাখে। প্রতিটি সংরক্ষিত অবস্থাকে বলে **version** বা **revision**। আপনি যখন কোনো পরিবর্তন জমা দেন, VCS তা একটি **commit** হিসেবে সংরক্ষণ করে — সঙ্গে থাকে কে বদলেছে, কখন বদলেছে, কী বদলেছে, আর কেন বদলেছে তার সংক্ষিপ্ত বর্ণনা।

একটি উপমা ভাবুন: একটি উপন্যাসের পাণ্ডুলিপি। লেখক প্রতিদিন কিছু বদলান। যদি তিনি প্রতিদিনের পাণ্ডুলিপি আলাদা খামে রেখে তারিখ ও সংক্ষিপ্ত নোট লিখে রাখেন — "অধ্যায় ৩-এর শেষটি নতুন করে লেখা হলো" — তাহলে যেকোনো দিনের পাণ্ডুলিপি ফিরে পাওয়া যায়, কোন বদল কখন এসেছিল জানা যায়, আর দুজন সম্পাদক একসঙ্গে কাজ করলেও কার বদল কোনটি তা মেলানো যায়। Version control ঠিক এই কাজটি করে, তবে স্বয়ংক্রিয়ভাবে, প্রতিটি লাইনের স্তরে।

Git-এ এই সংরক্ষণকে বলে **repository** বা সংক্ষেপে **repo**। একটি repository হলো একটি project-এর ফাইলগুলো ও তাদের পুরো ইতিহাস একসঙ্গে। ইতিহাস ছাড়া শুধু ফাইল থাকলে সেটি একটি folder, repository নয়।

১.২ কেন developer-দের version control দরকার?

প্রথম কারণ: **ভুল থেকে ফিরে আসা।** একটি বদল ভেঙে গেলে আগের ভালো version-এ ফেরা যায় — ঘণ্টার কাজ আবার লিখতে হয় না। দ্বিতীয় কারণ: **একসঙ্গে কাজ করা।** একই project-এ পাঁচজন developer কাজ করলে তাদের বদলগুলোকে সুশৃঙ্খলভাবে একত্র করা যায়, কার কাজ কার উপর চাপা পড়ে না। তৃতীয় কারণ: **জবাবদিহি ও ইতিহাস।** ছয় মাস পরে কেউ জিজ্ঞাসা করলে "এই লাইনটি কেন এভাবে লেখা?" — উত্তরের জন্য ইতিহাসে খুঁজলেই লেখক, তারিখ ও কারণ পাওয়া যায়। চতুর্থ কারণ: **পরীক্ষা-নিরীক্ষা।** নতুন ধারণা পরীক্ষা করার জন্য মূল কাজ থেকে আলাদা একটি শাখা (branch) বানিয়ে নিরাপদে পরীক্ষা করা যায়; পরীক্ষা ব্যর্থ হলে শাখাটি ফেলে দেওয়া যায়, মূল কাজ অক্ষত থাকে।

DevOps engineer-এর জন্য আরও একটি কারণ আছে: **deployment ও automation-এর ভিত্তি।** আধুনিক CI/CD pipeline Git repository থেকেই কোড নিয়ে test করে server-এ পাঠায়। কোন version production-এ চলছে, তা জানা না গেলে কোনো সমস্যার কারণ খুঁজে পাওয়া যায় না। এই বইয়ের Volume 04-এ CI/CD শেখার সময় এই সংযোগটি বারবার আসবে।

> **REAL WORLD:** সিলেটের একটি startup-এ একবার production server-এ একটি ভুল ফাইল upload করা হয়েছিল, আর ওয়েবসাইট বন্ধ হয়ে গিয়েছিল। Git থাকায় দলটি ঠিক আগের commit-টি চিহ্নিত করে দুই মিনিটে আগের version-এ ফিরে গিয়েছিল। Version control ছাড়া তাদের ভুল ফাইলটি হাতে খুঁজে বের করে ঠিক করতে ঘণ্টা লাগত, আর client-দের ক্ষতি হতো আরও বেশি।

১.৩ Manual backup বনাম version control

অনেক শিক্ষার্থী প্রথমে এইভাবে কাজ করে: `project.zip`, `project-final.zip`, `project-final-2.zip`, `project-final-sotti.zip`। এই পদ্ধতির সমস্যা চারটি। প্রথমত, কোন ফাইলটি আসলে চূড়ান্ত, তা বোঝা যায় না। দ্বিতীয়ত, দুটি কপির মধ্যে ঠিক কী বদলেছে, তা জানতে পুরো ফাইল মিলিয়ে দেখতে হয়। তৃতীয়ত, একই ফাইলে দুজন কাজ করলে তাদের বদল একত্র করার কোনো উপায় নেই — একজন আরেকজনের কাজ চাপা দিয়ে দেয়। চতুর্থত, প্রতিটি কপি পুরো project-এর পূর্ণ নকল, তাই disk নষ্ট হয়, আর ইতিহাস খুঁজে পাওয়া কঠিন।

বিষয়	Manual backup (zip / কপি)	Version control (Git)
ইতিহাস	শুধু কয়েকটি snapshot, নামে তারিখ	প্রতিটি পরিবর্তন, লেখক ও কারণসহ পূর্ণ ইতিহাস

পার্থক্য দেখা	হাতে মিলিয়ে দেখতে হয়	`git diff` দিয়ে লাইন ধরে পার্থক্য দেখা যায়
একসঙ্গে কাজ	একজন আরেকজনের কাজ চাপা দেয়	Branch ও merge দিয়ে বদল একত্র করা যায়
ফিরে যাওয়া	পুরনো zip খুঁজে বের করতে হয়	যেকোনো commit-এ এক কমান্ডে ফেরা যায়
জায়গা	প্রতিটি কপি পূর্ণ project	শুধু বদলটুকু জমা হয়, তাই অনেক কম জায়গা

মনে রাখবেন: backup আর version control পরস্পরের বিকল্প নয়। Git ইতিহাস রাখে, কিন্তু আপনার repository-র একটি remote কপি (যেমন GitHub) থাকা নিজেই এক ধরনের backup। Chapter 05-এ remote-এর কথা আসবে।

১.৪ Local, centralized ও distributed VCS

Version control ব্যবস্থাকে তিন ভাগে ভাগ করা যায়। **Local VCS**-এ ইতিহাস থাকে শুধু আপনার নিজের কম্পিউটারে, একটি database-এ। অন্য কেউ সেই ইতিহাস দেখতে পায় না, আর কম্পিউটার নষ্ট হলে সব হারায়। এটি শেখার জন্য যথেষ্ট, দলের কাজের জন্য নয়। **Centralized VCS** (যেমন Subversion বা SVN)-এ ইতিহাস থাকে একটি central server-এ। সবাই server থেকে কোড নেয়, বদলে আবার server-এ জমা দেয়। সুবিধা: সবাই একই ইতিহাস দেখে। অসুবিধা: server বন্ধ থাকলে বা নেটওয়ার্ক না থাকলে ইতিহাস দেখা যায় না, commit করা যায় না; আর server-টি একটিমাত্র ব্যর্থতার বিন্দু (single point of failure)।

Distributed VCS (DVCS)-এ প্রত্যেকের কম্পিউটারেই পুরো ইতিহাসের একটি পূর্ণ কপি থাকে। Git এই ধরনের। আপনি offline-এও commit করতে পারেন, ইতিহাস দেখতে পারেন, branch বানাতে পারেন — কারণ সবকিছু আপনার নিজের মেশিনেই আছে। অন্য কারও সঙ্গে বদল ভাগ করতে চাইলে তখনই শুধু নেটওয়ার্ক দরকার। কোনো একজন developer-এর মেশিন নষ্ট হলেও অন্যদের কাছে পুরো ইতিহাস থাকে, তাই কিছু হারায় না। এই কারণেই Git আজ শিল্পের মান (industry standard)।

তিন ধরনের VCS

Local: [আপনার PC — ইতিহাস শুধু এখানে]

Centralized: [সবাই] → Central Server (একটিমাত্র ইতিহাস) ← [সবাই]

Distributed (Git): [আপনার PC — পূর্ণ ইতিহাস] ⇄ [সহকর্মীর PC — পূর্ণ ইতিহাস] ⇄ [GitHub — পূর্ণ ইতিহাস]

Distributed-এ প্রতিটি কপিই সম্পূর্ণ, তাই কোনো একটিমাত্র ব্যর্থতার বিন্দু নেই।

১.৫ Git-এর ইতিহাস ও দর্শন

Git তৈরি হয়েছিল ২০০৫ সালে, Linux kernel-এর প্রয়োজনে। Linux kernel-এ হাজার হাজার developer বিশ্বজুড়ে কাজ করতেন, আর তখনকার ব্যবস্থা তাদের গতি ও স্বাধীনতা দিতে পারছিল না। Linus Torvalds নতুন একটি ব্যবস্থা বানালেন যার মূলনীতি ছিল: **গতি, distributed কাজ, ডেটার অখণ্ডতা, আর শাখা বানানো যেন সস্তা ও সহজ হয়।** Git-এর নামটি নিয়ে অনেক গল্প আছে; আনুষ্ঠানিকভাবে এটি কোনো শব্দের সংক্ষেপ নয়।

Git-এর দর্শনের তিনটি স্তম্ভ মনে রাখুন। প্রথম, **প্রতিটি commit একটি snapshot** — Git ফাইলের পার্থক্য (delta) জমায় না, সে প্রতিটি commit-এ পুরো project-এর একটি ছবি (snapshot) জমা রাখে, তবে অপরিবর্তিত ফাইলের জন্য আগের ছবির দিকে ইশারা করে, তাই জায়গা নষ্ট হয় না। দ্বিতীয়, **প্রায় সব কাজ local** — ইতিহাস দেখা, commit করা, branch

বানানো — সবই আপনার মেশিনে, তাই দ্রুত। তৃতীয়, **অখণ্ডতা (integrity)** — Git প্রতিটি তথ্যের জন্য SHA-1/SHA-256 hash গণনা করে; কোনো ফাইল নীরবে বদলে গেলে Git তা ধরে ফেলে। এই নকশা Chapter 02-এ বিস্তারিত আসবে।

১.৬ Git বনাম GitHub

এই দুটির গুলিয়ে ফেলা শিক্ষার্থীদের সবচেয়ে সাধারণ ভুল। **Git** হলো একটি software — আপনার কম্পিউটারে চলা একটি command-line tool, যা version control করে। **GitHub** হলো একটি ওয়েবসাইট ও সেবা — যা Git repository-গুলোকে ইন্টারনেটে রাখে, দলকে একসঙ্গে কাজ করার জায়গা দেয়, pull request, issue, code review-এর মতো সহযোগিতার সুবিধা দেয়। Git ছাড়া GitHub-এর অস্তিত্ব নেই, কিন্তু GitHub ছাড়াও Git পুরোপুরি কাজ করে — আপনার নিজের কম্পিউটারে, offline-এ।

বিষয়	Git	GitHub
কী	Version control software (tool)	Repository hosting ও collaboration platform (সেবা)
কোথায় চলে	আপনার কম্পিউটারে	ইন্টারনেটে, GitHub-এর server-এ
ইন্টারনেট দরকার?	না — local কাজ offline-এ হয়	হ্যাঁ
মূল কাজ	Commit, branch, merge, ইতিহাস	Sharing, pull request, issue, review, hosting
বিকল্প	Mercurial (অন্য VCS)	GitLab, Bitbucket, Codeberg (অন্য hosting)

একটি উপমা: Git হলো আপনার লেখার খাতা ও কলম, GitHub হলো লাইব্রেরি যেখানে খাতাটি রাখা হয় যাতে অন্যরা পড়তে ও মন্তব্য করতে পারে। খাতা ছাড়া লাইব্রেরি অর্থহীন, আবার লাইব্রেরি ছাড়াও খাতায় লেখা যায়।

১.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Version Control System (VCS)	ফাইলের পরিবর্তনের ইতিহাস ধরে রাখার ব্যবস্থা
Repository (repo)	Project-এর ফাইল ও পূর্ণ ইতিহাসের সংগ্রহ
Commit	ইতিহাসে জমা দেওয়া একটি snapshot, বার্তাসহ
Snapshot	কোনো মুহূর্তে project-এর পুরো অবস্থার ছবি
Branch	ইতিহাসের একটি স্বতন্ত্র ধারা, মূল ধারা থেকে আলাদা কাজের জন্য
Merge	দুটি শাখার বদল একত্র করা
Distributed VCS	প্রত্যেকের কাছে পূর্ণ ইতিহাস থাকা VCS, যেমন Git
Centralized VCS	ইতিহাস একটিমাত্র central server-এ থাকা VCS, যেমন SVN
Working Directory	আপনার কম্পিউটারে ফাইলগুলো যেখানে সম্পাদনা করেন
Remote	ইন্টারনেটে থাকা repository-র কপি, যেমন GitHub-এর repo

১.৮ বাস্তব প্রকৌশল পরিস্থিতি — TechNova-র দল

TechNova, ঢাকার একটি ছোট team, একটি inventory web app বানাচ্ছে। দলে আছেন রাফি (frontend), নুসরাত (backend), আর সাকিব (DevOps)। তারা GitHub-এ একটি repository বানিয়ে প্রত্যেকে নিজের feature-এর জন্য আলাদা branch-এ কাজ করে। রাফি login page-এর নকশা বদলাচ্ছে, নুসরাত API-তে নতুন endpoint যোগ করছে — দুজনে একই ফাইল ছোঁয় না, তাই কোনো সংঘর্ষ হয় না। যেদিন নুসরাতের API তৈরি, সেদিন সে pull request খোলে, রাফি কোড পড়ে মন্তব্য করে, তারপর সাকিব তা main branch-এ merge করে। Client হঠাৎ পুরনো একটি feature ফেরত চাইলে দলটি Git ইতিহাস থেকে ঠিক সেই commit-টি দেখিয়ে দেয় কবে তা সরানো হয়েছিল আর কেন। এই পুরো কর্মপ্রবাহের প্রতিটি ধাপ এই বইয়ের Part I-এ শেখা হবে — Chapter 04-এ branch, Chapter 05-এ GitHub, Chapter 06-এ দলের workflow।

১.৯ কমান্ডের প্রথম দেখা

এই অধ্যায়ে এখনো repository বানানো শেখা হয়নি — সেটি Chapter 03-এর কাজ। তবু Git আপনার মেশিনে আছে কি না, তা এখনই যাচাই করা যায়। নিচের কমান্ডটি terminal-এ চালান:

```
git --version
git config --global user.name "Nahid Hasan"
git config --global user.email "nahid@example.com"
```

প্রথম লাইনটি Git-এর version দেখায়। দ্বিতীয় ও তৃতীয় লাইন আপনার পরিচয় Git-কে জানায় — প্রতিটি commit-এ এই নাম ও ইমেইল জমা হবে, যাতে ইতিহাসে দেখা যায় কে বদলেছে। এখানে `example.com` একটি উদাহরণ মাত্র — আপনার আসল ইমেইল ব্যবহার করুন। `--global` মানে এই সেটিং আপনার ব্যবহারকারী account-এর সব repository-র জন্য প্রযোজ্য হবে। সেটিং ঠিক আছে কি না, তা দেখতে `git config --list` চালান।

উদাহরণ আউটপুট (illustrative):

```
$ git --version
git version 2.43.0
```

এই আউটপুটটি উদাহরণ হিসেবে দেখানো হয়েছে; আপনার মেশিনে version নম্বরটি কিছুটা আলাদা হতে পারে। এই বইয়ের পরিবেশে পরীক্ষিত version হলো Git 2.43, Ubuntu Server 24.04 LTS-এ।

> **WARNING:** `git config`-এ ভুল ইমেইল দিলে আপনার commit-গুলো ভুল পরিচয়ে জমা হবে, আর GitHub-এ আপনার প্রোফাইলের সঙ্গে তা যুক্ত হবে না। প্রথম দিনেই সঠিক নাম ও ইমেইল দিন। কখনোই অন্য কারও নাম-ইমেইল ব্যবহার করে commit করবেন না — এটি অন্যের পরিচয়ে কাজ করার শামিল।

১.১০ হাতে-কলমে অনুশীলন

> **LAB:** আপনার নিজের প্রথম চিন্তা-পরীক্ষা — কোনো repository না বানিয়ে শুধু ধারণাটি যাচাই করুন।

1. আপনার কম্পিউটারে একটি ফোল্ডার বানান `my-notes`, ভেতরে `notes.txt` নামে একটি ফাইল লিখুন।
2. ফাইলটির তিনটি কপি বানান: `notes-v1.txt`, `notes-v2.txt`, `notes-v3.txt` — প্রতিটিতে কিছু লাইন যোগ করুন।

3. এবার প্রশ্ন করুন: v2 আর v3-এর মধ্যে ঠিক কী বদলেছে? উত্তর দিতে আপনাকে দুটি ফাইল পাশাপাশি পড়তে হলো — এটিই manual পদ্ধতির কষ্ট।
4. Git install আছে কি না যাচাই করুন: `git --version`। না থাকলে Ubuntu-তে `sudo apt update && sudo apt install git` চালান (Volume 01-এর package অধ্যায় দেখুন)।
5. আপনার নাম ও ইমেইল দিয়ে `git config --global` সেট করুন, আর `git config --list` দিয়ে যাচাই করুন।

প্রত্যাশিত ফলাফল ও যাচাই: `git --version` একটি version নম্বর দেখাবে, আর `git config --list`-এ আপনার নাম ও ইমেইল দেখা যাবে। v1–v3 অনুশীলনটি আপনাকে বোঝাবে কেন হাতে version রাখা টেকসই নয় — Chapter 03 থেকে এই কাজটি Git-ই করবে।

১.১১ Common Mistakes ও Debugging

- **ভুল:** "Git মানেই GitHub" ভাবা। GitHub বন্ধ থাকলেও Git local-এ কাজ করে; GitHub শুধু একটি hosting সেবা।
- **ভুল:** ফাইলের নামে `final`, `final2` লিখে version বোঝানো। এক সপ্তাহ পরে কেউ জানে না কোনটি আসল চূড়ান্ত।
- **ভুল:** Commit বার্তায় "fix", "update", "asdf" লেখা। তিন মাস পরে এই বার্তা দিয়ে বোঝা যায় না কী ঠিক করা হয়েছিল। ভালো বার্তার নিয়ম Chapter 03-এ আসবে।
- **সমস্যা:** `git: command not found` — Git install হয়নি। Ubuntu-তে `sudo apt install git` দিয়ে install করুন।
- **সমস্যা:** Commit-এ নাম দেখাচ্ছে না — `git config --global user.name` সেট করা হয়নি।

১.১২ Security Considerations

Version control ইতিহাসে একবার যা জমা হয়, তা মুছে ফেলা কঠিন — বিশেষ করে remote-এ push করার পর। তাই কখনোই password, API token বা private key commit করবেন না। ভুল করে commit হয়ে গেলে শুধু ফাইল মুছে আবার commit করলেই হয় না — পুরনো commit-এর ইতিহাসে secret-টি থেকে যায়। এই বইয়ে secret রাখার নিরাপদ উপায় (environment variable, `.gitignore`, secret manager) Chapter 03 ও Chapter 18-এ শেখা হবে। আরেকটি নীতি: public repository-তে কখনো client-এর ব্যক্তিগত তথ্য, ঠিকানা বা ফোন নম্বর রাখবেন না।

১.১৩ Best Practices

- প্রতিটি project-এর প্রথম দিনেই Git repository বানান — project বড় হওয়ার অপেক্ষা করবেন না।
- ছোট ছোট যৌক্তিক commit করুন; এক commit-এ অনেক unrelated বদল মেশাবেন না।
- Commit বার্তা এমন লিখুন যাতে ছয় মাস পরে পড়লেও বোঝা যায় কেন বদলটি করা হয়েছিল।
- নিয়মিত remote-এ push করুন — শুধু local commit backup নয়।
- কোনো ফাইল commit করার আগে `git status` দিয়ে দেখুন ঠিক কী জমা হচ্ছে (Chapter 03)।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Version control-এর প্রধান উদ্দেশ্য কোনটি?
   ক) ফাইলের রং বদলানো খ) ফাইলের পরিবর্তনের ইতিহাস ধরে রাখা ও একসঙ্গে কাজ করা গ) কম্পিউটার দ্রুত করা ঘ) ইন্টারনেটের গতি বাড়ানো

2. নিচের কোনটি distributed version control system?
   ক) Subversion (SVN) খ) Git গ) শুধু zip backup ঘ) Google Drive-এর কপি
3. Git ও GitHub-এর পার্থক্য কোনটি সঠিক?
   ক) দুটি একই জিনিস খ) Git একটি tool, GitHub একটি hosting ও collaboration সেবা গ) GitHub ছাড়া Git চলে না ঘ) Git শুধু Windows-এ চলে
4. Manual backup পদ্ধতির সবচেয়ে বড় সমস্যা কোনটি?
   ক) ফাইল বড় হয় খ) দুজনের বদল একত্র করা যায় না ও ইতিহাসের কারণ জানা যায় না গ) ফাইলের নাম সুন্দর হয় না ঘ) ইন্টারনেট লাগে
5. Git কেন প্রতিটি commit-এ snapshot জমা রাখে?
   ক) জায়গা নষ্ট করতে খ) যেকোনো মুহূর্তের পুরো অবস্থায় ফিরে যাওয়া যায় বলে গ) ফাইল মুছে ফেলতে ঘ) ইন্টারনেট ছাড়া কাজ বন্ধ করতে

১.১৪ Practical Assignment

Assignment 1.1 (Beginner): আপনার একটি পুরনো project (বা নতুন একটি ছোট web page project) বাছুন। কাগজে বা একটি text ফাইলে লিখুন: (ক) গত এক মাসে সেই project-এ আপনি কী কী বদলেছেন, (খ) কোন বদলগুলো ভেঙে গিয়েছিল আর আপনি কীভাবে ফিরেছিলেন, (গ) আরেকজন developer আপনার সঙ্গে কাজ করলে কোন কোন জায়গায় সংঘর্ষ হতো। শেষে এক প্যারাগ্রাফে লিখুন version control থাকলে প্রতিটি সমস্যা কীভাবে সমাধান হতো। পূর্ণ সমাধান Answer Guide-এ আছে।

১.১৫ Chapter Summary

Version control হলো project-এর ফাইলের পরিবর্তনের ইতিহাস, লেখক ও কারণসহ ধরে রাখার ব্যবস্থা। Manual backup-এ শুধু কয়েকটি কপি থাকে, version control-এ থাকে পূর্ণ ইতিহাস ও বদল একত্র করার ক্ষমতা। Local VCS শুধু এক মেশিনে, centralized VCS এক central server-এ, আর distributed VCS — যেমন Git — প্রত্যেকের কাছে পূর্ণ ইতিহাস রাখে। Git ২০০৫ সালে Linux kernel-এর প্রয়োজনে তৈরি, তার মূলনীতি snapshot, local গতি ও অখণ্ডতা। Git একটি tool, GitHub একটি hosting সেবা — দুটি এক নয়। পরের অধ্যায়ে আমরা Git-এর ভেতরের স্থাপত্য — blob, tree, commit ও staging area — খুলে দেখব।

Revision Checklist

- ☐ Version control, repository, commit ও snapshot নিজের ভাষায় ব্যাখ্যা করতে পারি।
- ☐ Manual backup বনাম VCS-এর পার্থক্য পাঁচটি পয়েন্টে বলতে পারি।
- ☐ Local, centralized ও distributed VCS-এর পার্থক্য বলতে পারি।
- ☐ Git-এর ইতিহাস ও তিনটি মূলনীতি বলতে পারি।
- ☐ Git বনাম GitHub-এর পার্থক্য স্পষ্ট করে বলতে পারি।
- ☐ আমার মেশিনে Git install করে নাম ও ইমেইল সেট করেছি।

অধ্যায় ২ • Part I — Git Fundamentals & Version Control

Chapter 02: Git Architecture and Internal Working — Git-এর স্থাপত্য ও ভেতরের কাজ

শুরুতে: একটি বাস্তব দৃশ্য

চট্টগ্রামের TechNova অফিসে নুসরাত একটি অদ্ভুত সমস্যায় পড়ল। সে একটি ফাইল সম্পাদনা করে `git add` করল, কিন্তু `git status` বলল ফাইলটি এখনো commit হয়নি। সে ভাবল `add` মানেই সংরক্ষণ। তারপর সে `git commit` করল, কিন্তু GitHub-এ কিছু দেখা গেল না — কারণ commit শুধু তার নিজের কম্পিউটারেই জমা হয়েছে, push করা হয়নি। একই দিনে তিনটি ভিন্ন জায়গা — working directory, staging area, local repository — তাকে তিনবার বিভ্রান্ত করল। আসলে Git-এর পুরো নকশাই এই তিনটি জায়গার উপর দাঁড়িয়ে। এই অধ্যায়ে আমরা Git-এর ভেতরটা খুলে দেখব: ফাইল কোথায় থাকে, commit আসলে কী দিয়ে তৈরি, আর `HEAD` বলে জিনিসটি কী।

> **CONCEPT:** Git-কে বুঝতে হলে একটি ছবি মনে রাখুন — তিনটি বাক্স। প্রথম বাক্স Working Directory: যেখানে আপনি ফাইল সম্পাদনা করেন। দ্বিতীয় বাক্স Staging Area (index): যেখানে আপনি পরের commit-এর জন্য বদল সাজান। তৃতীয় বাক্স Local Repository: যেখানে commit স্থায়ীভাবে জমা হয়। ফাইল এক বাক্স থেকে আরেক বাক্সে যায় নির্দিষ্ট কমান্ডে, নিজে থেকে নয়।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Working directory, staging area ও local repository-র পার্থক্য ও ফাইলের চলাচল ব্যাখ্যা করতে।
- Blob, tree ও commit object কী, আর তারা কীভাবে একে অপরের সঙ্গে যুক্ত, তা বর্ণনা করতে।
- Git object database ও SHA hash-এর ভূমিকা ব্যাখ্যা করতে।
- Reference, branch ও HEAD-এর সম্পর্ক আঁকতে ও বলতে।
- `git config`-এর তিনটি স্তর (system, global, local) ব্যবহার করতে।
- একটি commit-এর ভেতরে কী থাকে, তা `git cat-file` দিয়ে পরীক্ষা করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 01: version control, repository, commit ও snapshot-এর ধারণা।
- Terminal-এ ফাইল তৈরি ও `git --version` চালানোর অভিজ্ঞতা।
- Hash বলতে একটি মান থেকে নির্দিষ্ট দৈর্ঘ্যের আঙুলের ছাপ বের করার ধারণা থাকলে ভালো।

২.১ তিনটি জায়গা: Working Directory, Staging Area, Repository

Git-এ প্রতিটি ফাইল যেকোনো মুহূর্তে তিনটি অবস্থার একটিতে থাকে। **Working directory** হলো আপনার project folder — যেখানে ফাইলগুলো সাধারণভাবে থাকে, আপনি editor দিয়ে সেগুলো বদলান। **Staging area** (যাকে **index**-ও বলে) হলো পরের commit-এ কী কী যাবে, তার একটি তালিকা — একটি মঞ্চের মতো, যেখানে অভিনয়ের আগে অভিনেতারা সাজেন। **Local repository** হলো `.git` নামের লুকানো folder-টি, যেখানে commit-গুলো স্থায়ীভাবে জমা হয়।

ফাইলের যাত্রা: আপনি ফাইল বদলান (working directory) → `git add` দিয়ে বদলটি staging area-তে তোলেন → `git commit` দিয়ে staging area-র সবকিছু repository-তে স্থায়ী snapshot হিসেবে জমা দেন। এই মাঝের ধাপটি কেন? কারণ এতে আপনি ঠিক করতে পারেন কোন বদল এই commit-এ যাবে আর কোনটি পরের commit-এ থাকবে। একটি ফাইলে দুটি আলাদা কাজ করলে একটি commit-এ একটি কাজ, আরেক commit-এ আরেকটি কাজ জমা দেওয়া যায় — ইতিহাস পরিষ্কার থাকে।

Git-এর তিনটি জায়গা ও ফাইলের চলাচল

Working Directory —(git add)→ Staging Area (index) —(git commit)→ Local Repository (.git)

Working Directory ←(git checkout / restore)— Local Repository

Local Repository —(git push)→ Remote (GitHub) | Remote —(git fetch / pull)→ Local Repository

মনে রাখুন: add মানে সংরক্ষণ নয়, commit মানে GitHub-এ পাঠানো নয়।

২.২ Git object database: .git-এর ভেতরে কী আছে?

একটি repository বানালে তার ভেতরে `.git` নামে একটি লুকানো directory তৈরি হয়। এটিই Git-এর মস্তিষ্ক। এর ভেতরে `objects/` হলো object database — এখানে Git তার সব তথ্য object হিসেবে জমা রাখে। `refs/`-এ থাকে branch ও tag-এর ইশারা। `HEAD` নামের ছোট ফাইলটি বলে এখন কোন branch-এ আছেন। `config` ফাইলে থাকে এই repository-র নিজস্ব সেটিং।

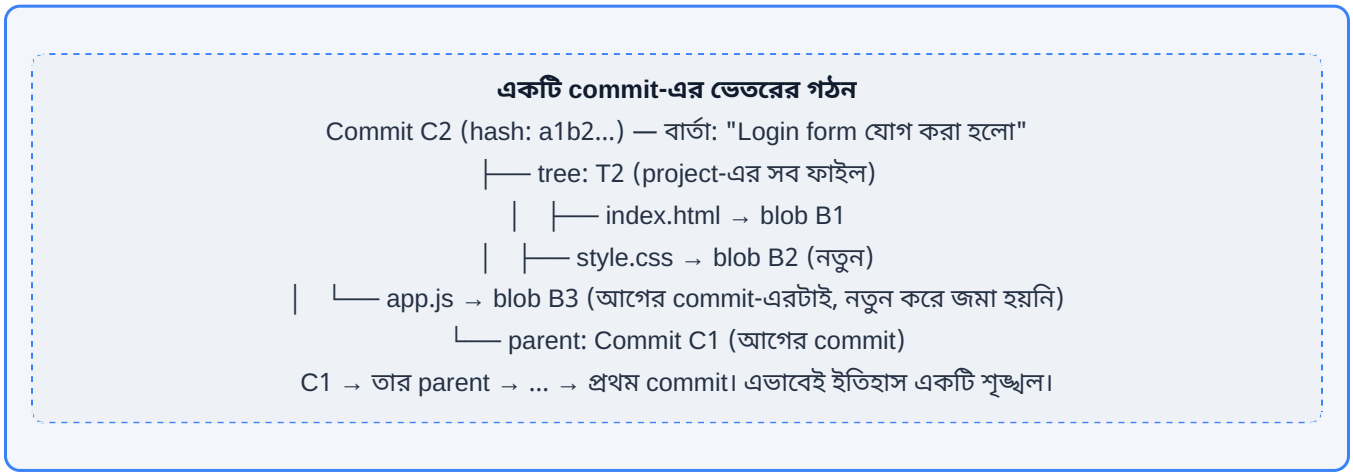
Git-এর সবচেয়ে গুরুত্বপূর্ণ নীতি: **সবকিছু content দিয়ে চেনা হয়, নাম দিয়ে নয়।** একটি ফাইলের বিষয়বস্তু থেকে Git একটি hash গণনা করে (SHA-1 বা নতুন repository-তে SHA-256)। একই বিষয়বস্তুর hash সবসময় একই হয়, আর বিষয়বস্তু এক অক্ষর বদলালেও hash সম্পূর্ণ বদলে যায়। এই hash-ই object-টির ঠিকানা। তাই Git কোনো তথ্য নীরবে বদলে গেলে ধরে ফেলতে পারে — hash আর মিলবে না।

২.৩ চার ধরনের object: Blob, Tree, Commit, Tag

Git-এর ভেতরে মাত্র চার ধরনের object আছে। **Blob** (binary large object) হলো একটি ফাইলের বিষয়বস্তু — শুধু বিষয়বস্তু, ফাইলের নাম নয়। একই বিষয়বস্তুর দুটি ফাইল থাকলে একটি blob-ই যথেষ্ট। **Tree** হলো একটি directory-র তালিকা: কোন নামের ফাইল কোন blob-কে দেখায়, আর কোন sub-directory কোন tree-কে দেখায়। **Commit** হলো একটি snapshot-এর পরিচয়: কোন tree-টি এই snapshot, কে লিখেছে, কখন, কী বার্তা, আর আগের commit কোনটি (parent)। **Tag** object হলো একটি commit-এর উপর একটি স্থায়ী নামের লেবেল, যেমন `v1.0` — বিস্তারিত Chapter 05-এ।

Object	কী জমা রাখে	উদাহরণ
Blob	একটি ফাইলের বিষয়বস্তু	`index.html`-এর ভেতরের লেখা
Tree	ফাইল ও directory-র নাম ও তাদের object-এর তালিকা	project-এর root directory
Commit	Tree, লেখক, তারিখ, বার্তা, parent commit	"Login page যোগ করা হলো"
Tag	একটি commit-এর নামযুক্ত ইশারা	`v1.0`

এই কাঠামোর ফলে ইতিহাস একটি শৃঙ্খল তৈরি করে: প্রতিটি commit তার parent commit-কে দেখায়, parent আবার তার parent-কে — এভাবেই পেছন দিকে পুরো ইতিহাস। আর প্রতিটি commit একটি tree-কে দেখায়, tree তার blob-গুলোকে দেখায়। কোনো ফাইল না বদলালে নতুন commit নতুন blob বানায় না — পুরনো blob-এর দিকেই ইশারা করে। তাই snapshot পদ্ধতি সত্ত্বেও Git জায়গা বাঁচায়।



২.৪ References, Branch ও HEAD

Reference (ref) হলো একটি commit-এর hash-এর উপর একটি মানুষের পড়ার মতো নাম। **Branch** হলো একটি চলমান reference — প্রতিটি নতুন commit-এ branch-টি স্বয়ংক্রিয়ভাবে নতুন commit-এর দিকে এগিয়ে যায়। **HEAD** হলো একটি বিশেষ pointer, যা বলে আপনি এখন কোন branch-এ (বা কোন commit-এ) আছেন। সাধারণত HEAD branch-কে দেখায়, branch commit-কে দেখায়।

একটি উদাহরণ: `main` branch C2-কে দেখাচ্ছে, HEAD `main`-কে দেখাচ্ছে। আপনি নতুন commit C3 করলেন — Git C3 বানিয়ে `main`-কে C3-র দিকে সরিয়ে দিল, HEAD এখনো `main`-কে দেখায়। তাই `git log`-এ আপনি C3-কে সবার উপরে দেখেন। এই সহজ যন্ত্রটিই Chapter 04-এর branching-এর ভিত্তি। **Detached HEAD** মানে HEAD সরাসরি একটি commit-কে দেখাচ্ছে, কোনো branch-কে নয় — তখন করা commit কোনো branch-এ জমা হয় না, সহজে হারিয়ে যেতে পারে। বিস্তারিত Chapter 04-এ।

২.৫ Git configuration-এর তিনটি স্তর

Git সেটিং পড়ে তিনটি স্তর থেকে, আর পরের স্তর আগেরটিকে ছাপিয়ে যায় (override করে)। **System** স্তর (`/etc/gitconfig`) মেশিনের সব ব্যবহারকারীর জন্য। **Global** স্তর (`~/.gitconfig`) আপনার ব্যবহারকারী account-এর জন্য — নাম ও ইমেইল এখানেই সেট করা হয়। **Local** স্তর (`.git/config`) শুধু একটি repository-র জন্য — যেমন একটি কাজের project-এ অফিসের ইমেইল, আর ব্যক্তিগত project-এ অন্য ইমেইল।

```
git config --list
git config --global user.name "Nahid Hasan"
git config --global user.email "nahid@example.com"
git config --local user.email "nahid@technova.example"
git config user.name
```

প্রথম লাইনটি সব স্তরের সেটিং একসঙ্গে দেখায়। দ্বিতীয় ও তৃতীয় লাইন global পরিচয় সেট করে। চতুর্থ লাইনটি শুধু এই repository-র জন্য ইমেইল বদলায় — global সেটিং অন্য repository-তে অপরিবর্তিত থাকে। শেষ লাইনটি এখন কার্যকর

মানটি দেখায়। `git config --list --show-origin` চালালে প্রতিটি সেটিং কোন ফাইল থেকে এসেছে, তাও দেখা যায় — সেটিং নিয়ে বিভ্রান্তি হলে এটি প্রথমে চালান।

এই অধ্যায়ের সবচেয়ে দরকারি পরীক্ষাটি হলো `git cat-file` — Git-এর object-গুলোকে সরাসরি দেখা। একটি ছোট repository বানিয়ে commit করুন, তারপর `git log` থেকে commit-এর hash নিয়ে নিচের কমান্ডগুলো চালান। এতে blob/tree/commit আর বিমূর্ত ধারণা থাকবে না।

```
git cat-file -p HEAD
git cat-file -p HEAD^{tree}
git cat-file -t HEAD
```

প্রথম কমান্ডটি বর্তমান commit-এর ভেতরের তথ্য দেখায় — tree-র hash, parent, লেখক, তারিখ ও বার্তা। দ্বিতীয় কমান্ডটি সেই commit-এর tree-র ভেতরের ফাইল তালিকা দেখায়। তৃতীয় কমান্ডটি শুধু বলে object-টির ধরন — এখানে `commit`। `-p` মানে pretty-print, অর্থাৎ মানুষের পড়ার মতো করে দেখানো।

২.৬ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Working Directory	ফাইল সম্পাদনার জায়গা, project folder
Staging Area / Index	পরের commit-এর জন্য সাজানো বদলের তালিকা
Local Repository	.git folder, যেখানে commit স্থায়ীভাবে জমা হয়
Blob	একটি ফাইলের বিষয়বস্তু ধারণকারী object
Tree	Directory-র তালিকা ধারণকারী object
Commit object	Snapshot-এর পরিচয়: tree, লেখক, তারিখ, বার্তা, parent
HEAD	বর্তমান branch বা commit-কে দেখানো pointer
Reference (ref)	Commit hash-এর উপর মানুষের পড়ার মতো নাম
SHA hash	বিষয়বস্তু থেকে গণনা করা নির্দিষ্ট দৈর্ঘ্যের আঙুলের ছাপ
Object database	.git/objects-এ Git-এর সব object-এর সংগ্রহ

২.৭ বাস্তব প্রকৌশল পরিস্থিতি — হারিয়ে যাওয়া ফাইল

TechNova-র সাকিব ভুল করে `config.py` ফাইলটি মুছে ফেলল। সে আতঙ্কিত হলো না — কারণ ফাইলটি commit করা ছিল। `git status` দেখাল ফাইলটি মুছে গেছে (working directory-তে নেই, repository-তে আছে)। `git restore config.py` চালাতেই ফাইলটি শেষ commit-এর অবস্থায় ফিরে এলো। এই পুনরুদ্ধার সম্ভব হয়েছে কারণ commit করা তথ্য .git-এ নিরাপদে আছে, আর working directory শুধু তার একটি প্রতিফলন। উল্টোদিকে, যে ফাইল কখনো `git add` বা commit করা হয়নি, তা মুছে গেলে Git-এর কিছু করার নেই — Git শুধু যা তাকে দেওয়া হয়েছে, তাই জানে।

> **PRO TIP:** `.git` folder-টি কখনো হাতে মুছবেন না বা সম্পাদনা করবেন না। এটি মুছে গেলে পুরো ইতিহাস হারিয়ে যাবে — ফাইলগুলো থাকবে, কিন্তু কোনটি কবে বদলেছে তার কোনো হিসাব থাকবে না। Backup নিতে হলে পুরো project folder-সহ `.git` কপি করুন, বা remote-এ push করুন।

২.৮ হাতে-কলমে অনুশীলন

> **LAB:** .git-এর ভেতরে হাত দিয়ে দেখা — object-গুলো নিজে পরীক্ষা করুন।

1. একটি নতুন ফোল্ডার `git-inside` বানিয়ে ভেতরে ঢুকুন, `git init` চালান (বিস্তারিত Chapter 03-এ)।
2. `hello.txt` ফাইলে "Hello Git" লিখুন, `git add hello.txt` ও `git commit -m "Add hello file"` চালান।
3. `ls -la .git/` দিয়ে ভেতরের ফাইলগুলো দেখুন। `cat .git/HEAD` চালান — কী দেখাচ্ছে?
4. `git cat-file -p HEAD` ও `git cat-file -p HEAD^{tree}` চালিয়ে commit ও tree-র ভেতর দেখুন।
5. `git config --list --show-origin` চালিয়ে আপনার নাম-ইমেইল কোন ফাইল থেকে এসেছে দেখুন।

প্রত্যাশিত ফলাফল ও যাচাই: `cat .git/HEAD` দেখাবে `ref: refs/heads/main` (বা `master`)। `git cat-file -p HEAD`-এ tree, author, committer ও আপনার বার্তা থাকবে। tree-র আউটপুটে `hello.txt` ও তার blob hash থাকবে। এগুলো দেখতে পেলে আপনি নিশ্চিত হতে পারেন: commit আসলে একটি tree ও বার্তার সমষ্টি, কোনো যাদু নয়।

উদাহরণ আউটপুট (illustrative):

```
$ cat .git/HEAD
ref: refs/heads/main
$ git cat-file -t HEAD
commit
```

২.৯ Common Mistakes ও Debugging

- **ভুল:** `git add`-কেই সংরক্ষণ ভাবা। add শুধু staging-এ তোলে; commit না করলে ইতিহাসে কিছু জমা হয় না, আর কম্পিউটার বন্ধ হলে staging-এর বদল হারাতে পারে।
- **ভুল:** Commit-কে GitHub-এ পাঠানো ভাবা। commit শুধু local; GitHub-এ যায় `git push`-এ (Chapter 05)।
- **ভুল:** `.git` folder-কে project-এর অপ্রয়োজনীয় অংশ ভেবে মুছে ফেলা। এতে পুরো ইতিহাস হারায়।
- **সমস্যা:** `git config user.name` কিছু দেখাচ্ছে না — global বা local কোনো স্তরেই নাম সেট হয়নি। `git config --global user.name "আপনার নাম"` দিন।
- **সমস্যা:** দুটি ইমেইল দেখাচ্ছে — global ও local আলাদা। `git config --list --show-origin` দিয়ে কোনটি কোথা থেকে এসেছে দেখুন।

২.১০ Security Considerations

Hash-এর কারণে Git তথ্যের অখণ্ডতা রক্ষা করে, কিন্তু এটি গোপনীয়তা দেয় না। Repository-র ভেতরে যা আছে, তা যার কাছে repository-র কপি আছে, সে পড়তে পারে — এমনকি পুরনো commit-এর ভেতরের মুছে ফেলা ফাইলও। তাই secret কখনো commit করবেন না, এমনকি "পরে মুছে ফেলব" ভেবেও নয়। আর `.git` folder-সহ project কাউকে zip করে পাঠানোর আগে ভাবুন: zip-এর ভেতরে পুরো ইতিহাস যাচ্ছে।

২.১১ Best Practices

- প্রতিটি commit-এর আগে `git status` দিয়ে দেখুন কোন ফাইল কোন অবস্থায় আছে — working, staged, না committed।
- Staging area-কে ইচ্ছাকৃতভাবে ব্যবহার করুন: একটি commit-এ একটি যৌক্তিক বদল রাখুন।
- কাজের ও ব্যক্তিগত project-এ আলাদা ইমেইল দরকার হলে local config ব্যবহার করুন, global বদলাবেন না।
- নতুন কিছু শিখলে `git cat-file` দিয়ে ভেতরটা দেখুন — Git-এর আচরণ তখন আর রহস্য থাকবে না।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Staging area-র কাজ কোনটি?
   ক) ফাইল মুছে ফেলা খ) পরের commit-এ কী যাবে তা সাজানো গ) GitHub-এ ফাইল পাঠানো ঘ) ফাইল সম্পাদনা করা
2. Blob object কী জমা রাখে?
   ক) ফাইলের নাম খ) একটি ফাইলের বিষয়বস্তু গ) commit-এর বার্তা ঘ) branch-এর নাম
3. HEAD কী?
   ক) একটি ফাইলের নাম খ) বর্তমান branch বা commit-কে দেখানো pointer গ) GitHub-এর server ঘ) একটি commit-এর বার্তা
4. কোন কমান্ডে commit-এর ভেতরের তথ্য দেখা যায়?
   ক) `git status` খ) `git cat-file -p HEAD` গ) `git push` ঘ) `git init`
5. একই বিষয়বস্তুর দুটি ফাইল থাকলে Git কী করে?
   ক) দুটি আলাদা blob বানায় খ) একটি blob-ই ব্যবহার করে গ) একটি ফাইল মুছে দেয় ঘ) ভুল বার্তা দেখায়

২.১২ Practical Assignment

Assignment 2.1 (Beginner): একটি ছোট repository বানিয়ে তিনটি commit করুন (তিনটি আলাদা ফাইল বা একই ফাইলে তিনটি ধাপ)। প্রতিটি commit-এর পর `git cat-file -p HEAD` চালিয়ে লিখুন: (ক) প্রতিটি commit-এর parent কোনটি, (খ) কোন commit-এ tree বদলেছে আর কোনটিতে শুধু বার্তা বদলেছে (এমন commit কি সম্ভব?), (গ) দ্বিতীয় commit-এর tree-তে প্রথম commit-এর blob-টি আছে কি না। এক পৃষ্ঠার প্রতিবেদন লিখুন। পূর্ণ সমাধান Answer Guide-এ আছে।

২.১৩ Chapter Summary

Git-এর কাজ তিনটি জায়গায় ভাগ করা: working directory-তে সম্পাদনা, staging area-তে সাজানো, local repository-তে স্থায়ী commit। .git folder-এর object database-এ চার ধরনের object থাকে — blob (ফাইলের বিষয়বস্তু), tree (directory তালিকা), commit (snapshot-এর পরিচয় ও parent) ও tag। প্রতিটি object তার বিষয়বস্তুর hash দিয়ে চেনা হয়, তাই তথ্যের অখণ্ডতা রক্ষা হয়। Branch হলো চলমান reference, HEAD বলে আপনি এখন কোথায় আছেন। Configuration-এর তিনটি স্তর — system, global, local — পরেরটি আগেরটিকে ছাপিয়ে যায়। পরের অধ্যায়ে এই জ্ঞান কাজে লাগিয়ে আমরা আসল repository বানিয়ে commit করা ও ইতিহাস পড়া শিখব।

Revision Checklist

- ☐ Working directory, staging area ও repository-র পার্থক্য ও ফাইলের চলাচল বলতে পারি।
- ☐ Blob, tree, commit ও tag-এর কাজ ব্যাখ্যা করতে পারি।
- ☐ Commit কীভাবে parent-এর সঙ্গে শৃঙ্খল তৈরি করে, তা আঁকতে পারি।
- ☐ HEAD ও branch-এর সম্পর্ক বলতে পারি।
- ☐ `git config`-এর তিনটি স্তর ব্যবহার করতে পারি।
- ☐ `git cat-file` দিয়ে একটি commit-এর ভেতর দেখতে পারি।

অধ্যায় ৩ • Part I — Git Fundamentals & Version Control

Chapter 03: Git Repositories, Commits and History — Repository, Commit ও ইতিহাস

শুরুতে: একটি বাস্তব দৃশ্য

সিলেটের একটি ছোট team-এ নতুন developer রাফি প্রথম দিনেই একটি ভুল করল: সে `git commit` করার আগে `git add` করতে ভুলে গেল, আর commit-টি খালি গেল। তারপর সে ভুল ফাইল commit করে ফেলল, আর ভাবল সব হারিয়ে গেছে। আসলে কিছুই হারায়নি — Git-এর প্রতিটি ভুলেরই ফেরার পথ আছে, যদি আপনি জানেন কোন কমান্ড কী করে। এই অধ্যায়ে আমরা দৈনন্দিন Git-এর মূল কমান্ডগুলো — init, clone, status, add, commit, log, diff, show — হাতে-কলমে শিখব, আর শিখব কীভাবে ভালো commit বার্তা লিখতে হয় ও সাধারণ ভুল থেকে ফিরতে হয়।

> **CONCEPT:** একটি ভালো commit হলো একটি ছোট, সম্পূর্ণ, বোধগম্য গল্পের একক। "কী বদলেছে" নয়, "কেন বদলেছে" — এটাই commit বার্তার প্রাণ। ছয় মাস পরে আপনার নিজের লেখা বার্তা পড়ে যদি বুঝতে না পারেন কেন বদলটি করেছিলেন, তবে বার্তাটি ব্যর্থ হয়েছে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- `git init` দিয়ে নতুন repository ও `git clone` দিয়ে বিদ্যমান repository-র কপি তৈরি করতে।
- `git status`, `git add` ও `git commit`-এর সঠিক ব্যবহার ব্যাখ্যা করতে।
- `git log`, `git diff` ও `git show` দিয়ে ইতিহাস পড়তে ও বদল পরীক্ষা করতে।
- `.gitignore` দিয়ে অপ্রয়োজনীয় ফাইল ইতিহাসের বাইরে রাখতে।
- ভালো commit বার্তা লেখার নিয়ম প্রয়োগ করতে।
- ভুল commit, ভুল ফাইল ও হারানো বদল থেকে নিরাপদে ফিরতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 01 ও 02: repository, commit, working directory, staging area, blob/tree/commit-এর ধারণা।
- Git install করা ও নাম-ইমেইল সেট করা (Chapter 01-এর অনুশীলন)।
- Terminal-এ directory বদল (`cd`) ও ফাইল তালিকা (`ls`) জানা।

৩.১ Repository তৈরি: git init ও git clone

নতুন project শুরু করতে চাইলে project folder-এ গিয়ে `git init` চালান। Git সেখানে `.git` folder বানিয়ে ফেলে — এখন থেকে এই folder একটি repository। আগে থেকে থাকা কোনো project-এও `git init` চালানো যায়; Git বিদ্যমান ফাইলগুলো মুছবে না, শুধু সেগুলোকে untracked হিসেবে দেখাবে।

অন্য কারও repository-র কপি নিতে চাইলে `git clone` ব্যবহার করুন — উদাহরণস্বরূপ GitHub থেকে। Clone করলে শুধু ফাইল আসে না, পুরো ইতিহাসও আসে, আর remote-এর ঠিকানাও মনে রাখা হয়। প্রথমবার clone করার সময় নেটওয়ার্ক লাগে; তারপর local কাজ offline-এ চলে।

```
mkdir technova-notes
cd technova-notes
git init
git clone https://github.com/example/technova-demo.git
```

প্রথম লাইনটি একটি ফোল্ডার বানায়। দ্বিতীয় লাইনটি তার ভেতরে ঢোকে। তৃতীয় লাইনটি সেখানে repository শুরু করে। চতুর্থ লাইনটি আলাদা একটি উদাহরণ — এটি GitHub থেকে একটি repository-র কপি আনে; এখানে `example/technova-demo` একটি উদাহরণ ঠিকানা, আপনার আসল project-এর ঠিকানা দিয়ে বদলে নিন। মনে রাখুন: `git init` একটি খালি repository বানায়, `git clone` ভরা repository-র কপি আনে।

> **WARNING:** ভুল folder-এ `git init` চালাবেন না — বিশেষ করে home directory (`~`)-তে নয়। Home-তে init করলে আপনার সব ফাইল, ছবি, ডকুমেন্ট এক repository-র ভেতরে চলে আসবে, আর `git status` হাজার হাজার ফাইল দেখাবে। ভুল হয়ে গেলে শুধু সেই folder-এর `.git` মুছে ফেললেই repository-টি আর থাকবে না — কিন্তু এটি করার আগে নিশ্চিত হোন কোন folder-এ আছেন (`pwd` চালান)।

৩.২ git status: Git-এর সঙ্গে আপনার কথোপকথন

`git status` হলো সবচেয়ে বেশি চালানো Git কমান্ড, আর সবচেয়ে উপকারী। এটি বলে: কোন ফাইল বদলেছে, কোনটি staged, কোনটি untracked, আপনি কোন branch-এ আছেন, আর remote-এর তুলনায় আপনি এগিয়ে না পিছিয়ে আছেন। কোনো কমান্ড চালানোর আগে ও পরে `git status` চালানোর অভ্যাস করুন — এটি অনেক ভুল আগেই ধরে দেয়।

Status-এর চারটি অবস্থা মনে রাখুন। **Untracked:** Git ফাইলটি চেনে না, এখনো কখনো add করা হয়নি। **Modified:** ফাইলটি commit করা আছে, কিন্তু তারপর বদলানো হয়েছে। **Staged:** বদলটি `git add` দিয়ে পরের commit-এর জন্য তোলা হয়েছে। **Committed / clean:** working directory ও repository একমত — কোনো বদল নেই। একটি ফাইলে একই সঙ্গে staged ও unstaged বদল থাকতে পারে — যেমন একটি লাইন add করা হয়েছে, তারপর আরেকটি লাইন বদলানো হয়েছে কিন্তু add করা হয়নি।

৩.৩ git add ও git commit: বদল জমা দেওয়ার দুটি ধাপ

`git add` ফাইলকে staging area-তে তোলে। `git add index.html` একটি ফাইল তোলে, `git add .` বর্তমান directory-র সব বদল তোলে। `git add -p` আরও সূক্ষ্ম — ফাইলের ভেতরের প্রতিটি বদল আলাদাভাবে বাছাই করা যায় (Chapter 06-এ বিস্তারিত)। `git commit -m "বার্তা"` staging area-র সবকিছু একটি commit হিসেবে জমা দেয়। বার্তা ছাড়া commit করা যায় না — Git editor খুলে বার্তা চাইবে।

```
git status
git add index.html style.css
git status
git commit -m "Add homepage layout and styles"
git log --oneline
```

প্রথম `git status` দেখায় কোন ফাইল বদলেছে। দ্বিতীয় লাইন দুটি ফাইল staging-এ তোলে। তৃতীয় `git status` দেখায় ফাইল দুটি এখন staged। চতুর্থ লাইন commit করে — বার্তায় বলা হয়েছে কী যোগ করা হলো। শেষ লাইনটি ইতিহাসের সংক্ষিপ্ত তালিকা দেখায়, যেখানে নতুন commit-টি সবার উপরে থাকবে। এই পাঁচ লাইনের ক্রমটিই দৈনন্দিন Git-এর হৃদয় — প্রথমে দেখুন, তারপর সাজান, তারপর জমা দিন, তারপর যাচাই করুন।

উদাহরণ আউটপুট (illustrative):

```
$ git log --oneline
a1b2c3d Add homepage layout and styles
e4f5g6h Initial commit
```

প্রতিটি লাইনে বামে commit-এর hash-এর সংক্ষিপ্ত অংশ, ডানে বার্তা। `--oneline` প্রতিটি commit-কে এক লাইনে দেখায়, তাই ইতিহাস দ্রুত চোখ বুলিয়ে নেওয়া যায়। Hash-টি উদাহরণ হিসেবে দেখানো হয়েছে; আপনার repository-তে hash আলাদা হবে।

৩.৪ ইতিহাস পড়া: git log, git diff, git show

`git log` ইতিহাস দেখায় — নতুন থেকে পুরনো। `git log --oneline --graph --all` branch-সহ ইতিহাসের ছবি দেখায় (Chapter 04-এ আরও কাজে লাগবে)। `git log --follow -- index.html` একটি ফাইলের পুরো ইতিহাস দেখায়, নাম বদলালেও। `git diff` দেখায় working directory-তে কী বদলেছে যা এখনো staged হয়নি। `git diff --staged` দেখায় staged বদল। `git show` একটি commit-এর পুরো বিবরণ দেখায় — কে, কখন, কী বার্তা, আর ঠিক কোন লাইন বদলেছে।

```
git log --oneline -5
git diff
git diff --staged
git show HEAD
git show a1b2c3d
```

প্রথম লাইন শেষ ৫টি commit দেখায়। দ্বিতীয় লাইন unstaged বদল দেখায় — commit করার আগে এটি দিয়ে নিজের বদল পরীক্ষা করুন। তৃতীয় লাইন staged বদল দেখায়। চতুর্থ লাইন বর্তমান commit-এর বিবরণ দেখায় — `HEAD` মানে বর্তমান অবস্থান। শেষ লাইন একটি নির্দিষ্ট commit-এর বিবরণ দেখায়, hash দিয়ে। এই কমান্ডগুলো শুধু পড়ে — কিছু বদলায় না — তাই এগুলো চালাতে ভয় নেই।

কমান্ড	কী দেখায়	কিছু বদলায়?
`git log`	Commit-এর তালিকা, নতুন থেকে পুরনো	না
`git diff`	Unstaged বদল (working বনাম staged)	না
`git diff --staged`	Staged বদল (staged বনাম শেষ commit)	না
`git show`	একটি commit-এর পূর্ণ বিবরণ ও বদল	না
`git status`	ফাইলগুলোর বর্তমান অবস্থা	না

একটি commit-এর শারীরস্থান (anatomy)

Commit a1b2c3d
├── hash: a1b2c3d... (বিষয়বস্তুর আঙুলের ছাপ)
├── author: Nahid Hasan <nahid@example.com>, 8 Nov 2026
├── committer: একই ব্যক্তি (সাধারণত author-ই committer)
├── parent: e4f5g6h (আগের commit)
├── message: "Add homepage layout and styles"
└── tree: এই মুহূর্তে project-এর সব ফাইলের snapshot

Author হলো যিনি বদলটি লিখেছেন, committer যিনি তা জমা দিয়েছেন — দলে cherry-pick বা rebase-এ দুজন আলাদা হতে পারে।

৩.৫ .gitignore: যা ইতিহাসে যাবে না

সব ফাইল commit করার মতো নয়। `node_modules/`, Python-এর `__pycache__/`, log ফাইল, editor-এর অস্থায়ী ফাইল, আর সবচেয়ে গুরুত্বপূর্ণ — secret ধারণকারী ফাইল (`.env`) — এগুলো repository-তে যাওয়া উচিত নয়। `.gitignore` নামের একটি ফাইলে প্রতি লাইনে একটি নিয়ম লিখুন; Git সেই ফাইলগুলোকে untracked হিসেবে দেখাবে না, আর `git add .` সেগুলো তুলবে না।

```
__pycache__/
*.log
.env
node_modules/
.DS_Store
```

প্রথম লাইন Python-এর cache folder বাদ দেয়। দ্বিতীয় লাইন সব `.log` ফাইল বাদ দেয়। তৃতীয় লাইন environment secret-এর ফাইল বাদ দেয় — এটি সবচেয়ে গুরুত্বপূর্ণ লাইন। চতুর্থ লাইন Node-এর dependency folder বাদ দেয়। শেষ লাইন macOS-এর system ফাইল বাদ দেয়। `.gitignore` ফাইলটি নিজে commit করা উচিত, যাতে দলের সবাই একই নিয়ম পায়। মনে রাখুন: ইতিমধ্যে commit হয়ে যাওয়া ফাইল `.gitignore`-এ লিখলেই ইতিহাস থেকে মুছে যায় না — তাকে `git rm --cached` দিয়ে tracking থেকে সরাতে হয়।

> **WARNING:** `.env` ফাইল ভুল করে commit হয়ে গেলে শুধু ফাইলটি মুছে আবার commit করলেই secret নিরাপদ হয় না — পুরনো commit-এর ইতিহাসে secret-টি থেকে যায়, আর কেউ ইতিহাস খুঁড়লে তা পেয়ে যাবে। তখন secret-টি বাতিল (rotate) করতে হয় — অর্থাৎ নতুন password/token বানাতে হয়। আর কখনো secret-কে শুধু `.gitignore`-এর উপর ভরসা করে রাখবেন না; commit করার আগে `git status` দিয়ে যাচাই করুন।

৩.৬ ভালো commit বার্তা

ভালো বার্তার নিয়ম সহজ। প্রথম লাইন ৫০ অক্ষরের মধ্যে, imperative mood-এ (আদেশের ভঙ্গিতে) লিখুন — "Added login" নয়, "Add login page"। কারণ commit বার্তা বলে এই commit কী করে, যেমন নির্দেশনা। প্রয়োজনে খালি লাইন দিয়ে বিস্তারিত লিখুন: কেন বদলটি দরকার, কী সমস্যা ঠিক হলো। খারাপ বার্তা: "fix", "update", "asdf", "final"। ভালো বার্তা: "Fix login redirect for expired sessions" — পড়লেই বোঝা যায় কী ঠিক হলো। দলে কাজ করলে Conventional Commits (যেমন `feat:`, `fix:`) ব্যবহার করা হয় — বিস্তারিত Chapter 06-এ।

৩.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
git init	নতুন repository শুরু করা
git clone	বিদ্যমান repository-র পূর্ণ কপি আনা
Untracked	Git এখনো চেনে না এমন ফাইল
Staged	পরের commit-এর জন্য তোলা বদল
Commit anatomy	একটি commit-এর উপাদান: hash, author, parent, tree, বার্তা
.gitignore	যেসব ফাইল commit হবে না, তার নিয়মের ফাইল
Author / Committer	বদলের লেখক / জমাদানকারী
git diff	দুটি অবস্থার মধ্যে লাইন ধরে পার্থক্য দেখানো

৩.৮ বাস্তব প্রকৌশল পরিস্থিতি — ভুল commit থেকে ফেরা

TechNova-র রাফি ভুল করে একটি debug ফাইল commit করে ফেলল। সে আতঙ্কিত হলো না। শেষ commit-টি এখনো push করা হয়নি, তাই `git reset --soft HEAD~1` দিয়ে commit-টি ফিরিয়ে staging-এ আনা গেল, ফাইলটি সরিয়ে আবার সঠিকভাবে commit করা গেল — working directory-র কোনো কাজ হারাল না। আরেকদিন সে একটি ফাইলে ভুল বদল করে ফেলল, কিন্তু add করেনি — `git restore index.html` দিয়ে ফাইলটি শেষ commit-এর অবস্থায় ফিরে এলো। এই দুটি কমান্ডের পার্থক্য মনে রাখুন: `restore` working directory-র বদল ফেরায়, `reset --soft` commit-কে ফিরিয়ে আনে কিন্তু কাজ রাখে। Push করা commit-এর ক্ষেত্রে এইসব করা বিপজ্জনক — তখন `git revert` ব্যবহার করতে হয় (Chapter 04)।

৩.৯ হাতে-কলমে অনুশীলন

> **LAB:** প্রথম repository — init থেকে log পর্যন্ত পুরো চক্র।

1. `mkdir my-first-repo && cd my-first-repo && git init` চালান।
2. `index.html` ফাইলে একটি ছোট heading লিখুন, `git status` দেখুন — ফাইলটি untracked দেখাবে।
3. `git add index.html` করে আবার `git status` দেখুন — এখন staged।
4. `git commit -m "Add homepage with heading"` চালান, তারপর `git log --oneline` দেখুন।
5. ফাইলে আরও একটি লাইন যোগ করুন, `git diff` দিয়ে বদলটি দেখুন, তারপর add ও commit করুন — এবার বার্তা লিখুন "Add welcome paragraph"।
6. `.gitignore` বানিয়ে তাতে `*.log` লিখুন, একটি `app.log` ফাইল বানান, আর `git status`-এ তা দেখা যায় কি না যাচাই করুন।

প্রত্যাশিত ফলাফল ও যাচাই: `git log --oneline`-এ দুটি commit থাকবে। `git status` শেষে clean দেখাবে (শুধু `.gitignore` untracked থাকতে পারে — সেটিও add ও commit করুন)। `app.log` status-এ দেখা যাবে না — এটিই `.gitignore`-এর প্রমাণ। `git show HEAD`-এ শেষ commit-এর বদল দেখা যাবে।

৩.১০ Common Mistakes ও Debugging

- **ভুল:** `git add` না করে commit করা — commit খালি যায় বা পুরনো অবস্থা জমা হয়। অভ্যাস: commit-এর আগে সবসময় `git status`।
- **ভুল:** `git add .` দিয়ে না দেখে সবকিছু তোলা — secret বা log ফাইল উঠে যেতে পারে। আগে `git status` দেখুন।
- **ভুল:** Commit বার্তায় "update" লেখা। পরে `git log` পড়ে কিছু বোঝা যায় না।
- **সমস্যা:** `git commit` করলে editor খুলে গেল — বার্তা না দিয়ে editor বন্ধ করবেন না; বার্তা লিখে save করে বের হন, বা `-m` ব্যবহার করুন।
- **সমস্যা:** ভুল ফাইল staged হয়ে গেছে — `git restore --staged <file>` দিয়ে staging থেকে সরান; ফাইলটি মুছবে না।
- **সমস্যা:** Working directory-র বদল ভুল করে ফেলেছেন — `git restore <file>` দিয়ে শেষ commit-এর অবস্থায় ফেরান; তবে এটি বদল চিরতরে মুছে দেয়, তাই নিশ্চিত হয়ে করুন।

৩.১১ Security Considerations

Commit করার আগে `git diff --staged` দিয়ে দেখুন ঠিক কী জমা হচ্ছে — এটি secret ফাঁসের শেষ প্রতিরক্ষা। `.env`, private key (`*.pem`), password-যুক্ত config ফাইল কখনো commit করবেন না। একটি project-এ `.gitignore` প্রথম দিনেই বানান, secret তৈরি হওয়ার পরে নয়। Public repository-তে commit করার আগে ভাবুন: এই ইতিহাস বিশ্বের যে কেউ পড়তে পারবে।

৩.১২ Best Practices

- ছোট ও যৌক্তিক commit করুন — একটি commit-এ একটি কাজ।
- Commit করার আগে `git diff` দিয়ে নিজের বদল নিজে পর্যালোচনা করুন — এটি code review-এর প্রথম ধাপ।
- প্রতিদিনের কাজ শেষে commit করুন; commit জমিয়ে রাখবেন না।
- `.gitignore` প্রথম commit-এর সঙ্গেই জমা দিন।
- বার্তা লিখুন যাতে `git log --oneline` পড়ে project-এর গল্প বোঝা যায়।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `git add`-এর কাজ কোনটি?
   ক) GitHub-এ ফাইল পাঠানো খ) বদলকে staging area-তে তোলা গ) ফাইল মুছে ফেলা ঘ) ইতিহাস দেখা
2. `git diff --staged` কী দেখায়?
   ক) Untracked ফাইল খ) Staged বদল গ) Remote-এর ইতিহাস ঘ) Commit বার্তা

3. ভালো commit বার্তার বৈশিষ্ট্য কোনটি?
   ক) "update" লেখা খ) ৫০ অক্ষরের মধ্যে imperative ভাষায় কী ও কেন বলা গ) যত লম্বা তত ভালো ঘ) ইমোজি দিয়ে ভরা
4. `.gitignore`-এর কাজ কী?
   ক) ফাইল মুছে ফেলা খ) নির্দিষ্ট ফাইলকে commit-এর বাইরে রাখা গ) GitHub-এ upload করা ঘ) Commit বার্তা লেখা
5. ভুল ফাইল staged হয়ে গেলে কী করবেন?
   ক) ফাইলটি মুছে ফেলবেন খ) `git restore --staged <file>` চালাবেন গ) Repository মুছে ফেলবেন ঘ) কম্পিউটার restart দেবেন

৩.১৩ Practical Assignment

Assignment 3.1 (Beginner): একটি ছোট HTML/CSS project-এর repository বানিয়ে অন্তত ৫টি অর্থবহ commit করুন — প্রতিটি commit-এ একটি যৌক্তিক বদল, ভালো বার্তাসহ। ইচ্ছাকৃতভাবে একটি ভুল করুন (ভুল ফাইল stage করা বা ভুল বার্তা), তারপর সঠিক কমান্ডে তা ঠিক করুন। শেষে `git log --oneline`-এর আউটপুটসহ এক পৃষ্ঠার প্রতিবেদন লিখুন: প্রতিটি commit কী করে, আর ভুলটি কীভাবে ঠিক করলেন। পূর্ণ সমাধান Answer Guide-এ আছে।

৩.১৪ Chapter Summary

`git init` নতুন repository বানায়, `git clone` বিদ্যমান repository-র কপি আনে। দৈনন্দিন চক্রটি হলো: `git status` দিয়ে দেখা, `git add` দিয়ে সাজানো, `git commit` দিয়ে জমা দেওয়া, `git log` / `diff` / `show` দিয়ে যাচাই করা। একটি commit-এর থাকে hash, author, committer, parent, tree ও বার্তা। `.gitignore` অপ্রয়োজনীয় ও গোপন ফাইলকে ইতিহাসের বাইরে রাখে। ভালো বার্তা সংক্ষিপ্ত, imperative ও কারণসহ হয়। আর সাধারণ ভুল — ভুল stage, ভুল commit — থেকে ফেরার জন্য `git restore` ও `git reset --soft` আছে, তবে push করা commit-এর ক্ষেত্রে `revert` ব্যবহার করতে হয়। পরের অধ্যায়ে আমরা branch দিয়ে একসঙ্গে অনেক ধারায় কাজ করা শিখব।

Revision Checklist

- ☐ `git init` ও `git clone`-এর পার্থক্য বলতে পারি।
- ☐ Status-এর চারটি অবস্থা (untracked, modified, staged, clean) চিনতে পারি।
- ☐ add → commit চক্রটি হাতে-কলমে করতে পারি।
- ☐ `git log`, `git diff` ও `git show` দিয়ে ইতিহাস পড়তে পারি।
- ☐ `.gitignore` লিখতে ও যাচাই করতে পারি।
- ☐ ভালো commit বার্তা লিখতে পারি ও ভুল stage থেকে ফিরতে পারি।

অধ্যায় ৪ • Part I — Git Fundamentals & Version Control

Chapter 04: Branching, Merging and Rebasing — Branch, Merge ও Rebase

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-তে নুসরাত একটি নতুন payment feature বানাচ্ছে, যাতে অন্তত এক সপ্তাহ লাগবে। এদিকে রাফির একটি ছোট bug ঠিক করতে হবে আজই, আর সাকিব production-এর একটি জরুরি hotfix দিতে চায়। তিনজন যদি একই main branch-এ সরাসরি কাজ করে, তবে নুসরাতের অসমাপ্ত কাজ production-এ চলে যাওয়ার ঝুঁকি থাকে, আর রাফির bugfix আটকে যায়। সমাধান: প্রত্যেকে আলাদা branch-এ কাজ করে, তৈরি হলে main-এ merge করে। Branch হলো Git-এর সবচেয়ে শক্তিশালী ধারণা — এটি একটি হালকা pointer মাত্র, তাই বানাতে এক সেকেন্ডও লাগে না, অথচ এটি দলকে একসঙ্গে অনেক কাজ নিরাপদে করতে দেয়। এই অধ্যায়ে আমরা branch, merge, conflict, rebase, cherry-pick ও নিরাপদ ইতিহাস ব্যবস্থাপনা শিখব।

> **CONCEPT:** Branch মানে ফাইলের কপি নয়। Branch মানে একটি commit-এর দিকে দেখানো একটি নাম। নতুন branch বানালে Git কোনো ফাইল কপি করে না — শুধু একটি ছোট pointer বানায়। তাই Git-এ branch বানানো সস্তা, আর দলে প্রতিটি কাজের জন্য আলাদা branch বানানোই স্বাভাবিক নিয়ম।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Branch কী ও HEAD কীভাবে branch-এর সঙ্গে চলে, তা ব্যাখ্যা করতে।
- Branch তৈরি, বদল, তালিকা দেখা ও মুছে ফেলা করতে।
- Fast-forward ও three-way merge-এর পার্থক্য বলতে ও কখন কোনটি ঘটে, তা চিনতে।
- Merge conflict বুঝে হাতে ঠিক করে merge সম্পূর্ণ করতে।
- Rebase কী, কখন তা উপকারী ও কখন তা বিপজ্জনক, তা ব্যাখ্যা করতে।
- Cherry-pick, revert ও reset-এর নিরাপদ ব্যবহার করতে ও detached HEAD চিনতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 02 ও 03: commit, parent, HEAD, reference ও add/commit/log-এর ব্যবহার।
- একটি repository-তে অন্তত কয়েকটি commit করার অভিজ্ঞতা।

৪.১ Branch-এর মূল ধারণা

Chapter 02-এ শিখেছি: branch হলো একটি commit-এর দিকে দেখানো চলমান pointer। `main` branch-টি এখন C2-কে দেখাচ্ছে। আপনি `git branch feature-login` চালালে Git একটি নতুন pointer বানায়, যা এই মুহূর্তে C2-কেই দেখায় — কোনো ফাইল কপি হয় না। তারপর `git switch feature-login` চালালে HEAD ওই branch-এর দিকে ঘোরে, আর আপনার working directory সেই commit-এর অবস্থায় আসে। এখন আপনি commit করলে শুধু `feature-login` এগোয়, `main` যেখানে ছিল সেখানেই থাকে।

এই স্বাধীনতাই branch-এর মূল্য। মূল ধারা (main) সবসময় কাজ করা অবস্থায় থাকে, আর পরীক্ষা-নিরীক্ষা আলাদা ধারায় চলে। পরীক্ষা সফল হলে merge করে মূল ধারায় আনা হয়; ব্যর্থ হলে branch-টি মুছে ফেলা হয়, মূল ধারার কিছু হয় না।

```
git branch
git branch feature-login
git switch feature-login
git branch
git log --oneline --graph --all
```

প্রথম লাইন সব branch-এর তালিকা দেখায়, বর্তমান branch-এর পাশে তারকা (*) থাকে। দ্বিতীয় লাইন নতুন branch বানায়, কিন্তু আপনাকে সেখানে নিয়ে যায় না। তৃতীয় লাইন সেই branch-এ যায় — `git switch` হলো branch বদলের আধুনিক কমান্ড (পুরনো `git checkout`-ও একই কাজ করে, তবে switch স্পষ্টতর)। চতুর্থ লাইন আবার তালিকা দেখায় — এবার তারকা `feature-login`-এর পাশে। শেষ লাইন ইতিহাসের গ্রাফ দেখায়, যেখানে দুটি branch একই commit-এ মিলে আছে। `git switch -c feature-login` দিয়ে এক কমান্ডে branch বানিয়ে সেখানে যাওয়া যায়।

Branch গ্রাফ: main থেকে feature শাখা

main: C1 ── C2

feature-login: C1 ── C2 ── C3 ── C4

HEAD এখন feature-login-এ। C3 ও C4 শুধু feature branch-এ আছে; main এখনো C2-তে।

merge-এর পর main C4-এর দিকে এগোবে (fast-forward) অথবা একটি merge commit তৈরি হবে।

৪.২ Fast-forward বনাম three-way merge

ধরুন `main` C2-তে আছে, আর `feature-login` C2 থেকে এগিয়ে C4-এ গেছে, কিন্তু main-এ এর মধ্যে কোনো নতুন commit আসেনি। তখন merge করতে Git-কে কিছু একত্র করতে হয় না — সে শুধু `main` pointer-টিকে C4-র দিকে সরিয়ে দেয়। একে বলে **fast-forward merge**। ইতিহাসে কোনো নতুন commit তৈরি হয় না, শুধু একটি সরল রেখা থাকে।

কিন্তু `main`-এও এর মধ্যে নতুন commit (C5) এসে গেলে পরিস্থিতি বদলায়। এখন দুটি ধারা C2 থেকে আলাদা পথে এগিয়েছে — একটিতে C3–C4, আরেকটিতে C5। Git তখন দুই ধারার সাধারণ পূর্বপুরুষ (C2) খুঁজে বের করে, দুই ধারার বদল একত্র করে একটি নতুন **merge commit** বানায়, যার দুটি parent থাকে। একে বলে **three-way merge**। Merge commit-টি প্রমাণ রাখে যে দুটি ধারা এখানে একত্র হয়েছে। অনেক দল `--no-ff` দিয়ে সবসময় merge commit বানাতে পছন্দ করে, যাতে feature-এর সীমানা ইতিহাসে দেখা যায়।

```
git switch main
git merge feature-login
git log --oneline --graph --all
```

প্রথম লাইন main branch-এ ফেরে। দ্বিতীয় লাইন feature branch-কে main-এ merge করে — fast-forward হলে শুধু pointer সরে, three-way হলে Git editor খুলে merge commit-এর বার্তা চাইতে পারে। তৃতীয় লাইন গ্রাফ দেখিয়ে যাচাই করে merge ঠিক হয়েছে কি না। Merge-এর আগে সবসময় `git status` দিয়ে নিশ্চিত হোন working directory clean — অসমাপ্ত বদল থাকলে merge জটিল হয়।

৪.৩ Merge conflict: যখন দুজন একই লাইন বদলায়

Conflict ঘটে যখন দুটি ধারা একই ফাইলের একই লাইনে আলাদা বদল করে, আর Git বুঝতে পারে না কোনটি রাখবে। এটি Git-এর ব্যর্থতা নয় — এটি মানুষের সিদ্ধান্তের জায়গা। Git ফাইলটিতে conflict চিহ্ন দিয়ে দুটি সংস্করণই রেখে দেয়:

```
<<<<<<< HEAD
Welcome to TechNova
=======
Welcome to TechNova Portal
>>>>>>> feature-login
```

উপরের অংশটি (`HEAD`, অর্থাৎ আপনার বর্তমান branch) বলছে "Welcome to TechNova", নিচের অংশটি (`feature-login`) বলছে "Welcome to TechNova Portal"। আপনার কাজ: ফাইলটি খুলে সিদ্ধান্ত নিন কোনটি রাখবেন — একটি, অপরটি, বা দুটির মিশ্রণ — তারপর চিহ্নগুলো (`<<<`, `===`, `>>>`) মুছে ফেলুন, ফাইলটি save করুন, `git add` করুন, আর `git commit` দিয়ে merge সম্পূর্ণ করুন। `git status` বলে দেবে কোন ফাইলে এখনো conflict আছে।

> **TROUBLESHOOTING:** Conflict দেখে আতঙ্কিত হবেন না, আর merge-এর মাঝখানে অন্য কমান্ড চালাবেন না। প্রথমে `git status` দেখুন, conflict-যুক্ত ফাইলগুলো একে একে ঠিক করুন, প্রতিটি ঠিক করার পর `git add` করুন। সব ঠিক হলে `git commit` করুন। যদি মনে হয় merge-টি ভুল পথে যাচ্ছে, `git merge --abort` দিয়ে merge বাতিল করে আগের অবস্থায় ফেরা যায় — যতক্ষণ commit হয়নি।

৪.৪ Rebase: ইতিহাসকে সরল রেখায় সাজানো

Rebase মানে একটি branch-এর commit-গুলোকে তুলে এনে আরেকটি commit-এর উপর আবার বসানো। ধরুন feature branch main-এর পুরনো C2 থেকে শুরু হয়েছিল, আর main এখন C5-এ। `git rebase main` চালালে Git feature-এর commit-গুলোকে (C3, C4) C5-এর উপর আবার প্রয়োগ করে — ফলে ইতিহাস একটি সরল রেখা হয়, কোনো merge commit থাকে না। ইতিহাস পরিষ্কার দেখায়, `git log` পড়তে সহজ হয়।

কিন্তু rebase-এর একটি কঠোর নিয়ম আছে: **যে commit ইতিমধ্যে অন্যদের সঙ্গে ভাগ করা হয়েছে (push করা হয়েছে), তাকে rebase করবেন না।** Rebase পুরনো commit মুছে নতুন hash-এর commit বানায় — অন্য কারও কাছে পুরনো commit থাকলে তাদের ইতিহাস আর আপনার ইতিহাস মিলবে না, আর সমস্যা তৈরি হবে। নিজের local branch-এ, push করার আগে rebase করা নিরাপদ ও সাধারণ অভ্যাস। Shared branch-এ (যেমন main) কখনো rebase বা force-push করবেন না।

```
git switch feature-login
git rebase main
git log --oneline --graph --all
```

প্রথম লাইন feature branch-এ যায়। দ্বিতীয় লাইন তার commit-গুলোকে main-এর বর্তমান মাথার উপর বসায়। তৃতীয় লাইন গ্রাফ দেখিয়ে যাচাই করে — এখন ইতিহাস একটি সরল রেখা হওয়ার কথা। Rebase-এর সময় conflict হলে Git থেমে যায়; conflict ঠিক করে `git add` করুন, তারপর `git rebase --continue` চালান। বাতিল করতে চাইলে `git rebase --abort`।

৪.৫ Cherry-pick, revert ও reset

Cherry-pick দিয়ে একটি নির্দিষ্ট commit-কে অন্য branch-এ তুলে আনা যায়, পুরো branch merge না করে। যেমন production-এর hotfix-টি একটি commit-এ আছে, সেটি main-এও দরকার — `git cherry-pick <hash>`। **Revert** দিয়ে একটি commit-এর বদলকে উল্টে দেওয়া হয় — কিন্তু ইতিহাস মুছে নয়, একটি নতুন commit বানিয়ে। তাই push করা commit-এর ভুল ঠিক করতে revert-ই নিরাপদ উপায়। **Reset** ইতিহাসের pointer-কে পেছনে সরায়: `--soft` কাজ রেখে commit ফেরায়, `--mixed` staging-ও খালি করে, `--hard` working directory-র বদলসহ সব মুছে দেয় — তাই `--hard` চালানোর আগে দুবার ভাবুন।

কমান্ড	কী করে	নিরাপদ কখন
`git revert <hash>`	বদল উল্টে নতুন commit বানায়	সবসময়, push করা commit-এও
`git reset --soft HEAD~1`	শেষ commit ফেরায়, কাজ রাখে	শুধু local, push-এর আগে
`git reset --hard HEAD~1`	Commit ও কাজ দুটোই মুছে ফেরায়	শুধু local, নিশ্চিত হয়ে
`git cherry-pick <hash>`	একটি commit অন্য branch-এ আনে	সাধারণত নিরাপদ

৪.৬ Detached HEAD

`git switch <commit-hash>` চালালে HEAD সরাসরি একটি commit-কে দেখায়, কোনো branch-কে নয় — একে বলে **detached HEAD**। এই অবস্থায় করা commit কোনো branch-এ জমা হয় না; আপনি অন্য branch-এ গেলে সেই commit খুঁজে পাওয়া কঠিন হয়ে যায় (তবে `git reflog` দিয়ে ফেরা যায়)। Detached HEAD পুরনো commit পরীক্ষা করার জন্য ভালো, কিন্তু সেখানে কাজ শুরু করার আগে একটি branch বানিয়ে নিন: `git switch -c fix-old-bug`।

৪.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Branch	একটি commit-কে দেখানো চলমান pointer, আলাদা কাজের ধারা
Fast-forward merge	শুধু pointer সরিয়ে merge, নতুন commit ছাড়া
Three-way merge	দুই ধারা ও সাধারণ পূর্বপুরুষ মিলিয়ে merge commit বানানো
Merge conflict	একই লাইনে আলাদা বদল, Git নিজে মেলাতে পারে না
Rebase	Commit-গুলোকে নতুন ভিত্তির উপর আবার বসানো
Cherry-pick	একটি নির্দিষ্ট commit অন্য branch-এ তুলে আনা
Revert	একটি commit-এর বদল উল্টে নতুন commit বানানো
Detached HEAD	HEAD কোনো branch নয়, সরাসরি একটি commit-কে দেখাচ্ছে

৪.৮ বাস্তব প্রকৌশল পরিস্থিতি — TechNova-র সপ্তাহ

এক সপ্তাহে TechNova-তে তিনটি কাজ একসঙ্গে চলল: নুসরাত `feature-payment` branch-এ payment, রাফি `fix-login-redirect` branch-এ bugfix, সাকিব `hotfix-prod` branch-এ production সমস্যা। রাফির bugfix তৈরি হলে সে main-এ merge করল (fast-forward)। সাকিবের hotfix-টি cherry-pick করে main-এও আনা হলো। নুসরাতের feature-টি এক সপ্তাহ পরে main-এর উপর rebase করে merge করা হলো — ইতিহাস সরল রইল। একদিন রাফি ও নুসরাত একই লাইন বদলে ফেলায় conflict হলো; তারা একসঙ্গে বসে সিদ্ধান্ত নিয়ে conflict ঠিক করল। কোনো কাজই অন্য কাজকে আটকাল না — এটাই branch-ভিত্তিক কাজের মূল্য।

৪.৯ হাতে-কলমে অনুশীলন

> **LAB:** Branch, conflict ও rebase — পুরো চক্রটি নিজে করুন।

1. Chapter 03-এর repository-তে (বা নতুন একটিতে) `git switch -c feature-greeting` চালান।
2. `index.html`-এ একটি লাইন যোগ করে commit করুন। `git switch main` করে একই ফাইলের একই লাইনে আলাদা কিছু লিখে commit করুন।
3. `git merge feature-greeting` চালান — conflict হবে। `git status` দেখুন, ফাইল খুলে সিদ্ধান্ত নিয়ে চিহ্ন মুছে commit করুন।
4. নতুন branch `feature-footer` বানিয়ে commit করুন, তারপর `git switch main`-এ একটি commit করে `git switch feature-footer && git rebase main` চালান। `git log --oneline --graph --all` দিয়ে গ্রাফ দেখুন।
5. একটি commit-কে `git revert HEAD` দিয়ে উল্টে দিন, আর `git log`-এ দেখুন ইতিহাস মুছে যায়নি, নতুন commit এসেছে।

প্রত্যাশিত ফলাফল ও যাচাই: Merge-এর পর গ্রাফে একটি merge commit দেখা যাবে যার দুটি parent আছে। Rebase-এর পর feature-footer-এর commit-গুলো main-এর উপরে সরল রেখায় থাকবে। Revert-এর পর `git log`-এ মূল commit ও revert commit দুটোই থাকবে। `git branch` দিয়ে সব branch-এর তালিকা দেখা যাবে।

৪.১০ Common Mistakes ও Debugging

- **ভুল:** অসমাপ্ত বদল রেখে branch বদলানো — Git বাধা দেবে বা বদলটি নতুন branch-এ নিয়ে যাবে। আগে commit করুন বা `git stash` ব্যবহার করুন (Chapter 06)।
- **ভুল:** Shared branch-এ rebase বা force-push করা। অন্যরা সেই branch pull করলে তাদের ইতিহাস ভেঙে যাবে।
- **ভুল:** Conflict-এর চিহ্ন (`<<<`) ফাইলে রেখে commit করা — কোড ভেঙে যাবে। Commit-এর আগে ফাইলটি খুলে দেখুন।
- **সমস্যা:** Merge-এর মাঝখানে আটকে গেছেন — `git merge --abort` দিয়ে বাতিল করুন।
- **সমস্যা:** Detached HEAD-এ commit করে ফেলেছেন — `git switch -c my-rescue-branch` দিয়ে সেটিকে branch-এ বাঁচান, তারপর অন্য branch-এ যান।
- **সমস্যা:** ভুল branch-এ commit করে ফেলেছেন — branch-টি মুছবেন না; নতুন branch বানিয়ে commit-টি সেখানে রাখুন, বা cherry-pick করুন।

৪.১১ Security Considerations

Branch ও merge নিজে নিরাপত্তার বিষয় নয়, তবে ইতিহাস ব্যবস্থাপনা নিরাপত্তার সঙ্গে জড়িত। Force-push (`git push --force`) shared branch-এ অন্যের commit মুছে দিতে পারে — production-এর গুরুত্বপূর্ণ fix হারিয়ে যেতে পারে। তাই shared branch-এ force-push নিষিদ্ধ রাখুন (Chapter 06-এ branch protection)। আর merge করার আগে `git diff` দিয়ে দেখুন কী merge হচ্ছে — ভুল branch merge করলে অসমাপ্ত বা ক্ষতিকর কোড production-এ যেতে পারে।

৪.১২ Best Practices

- প্রতিটি কাজের জন্য ছোট, নাম-স্পষ্ট branch বানান (যেমন `feature-login`, `fix-redirect`)।
- Branch ছোট রাখুন ও দ্রুত merge করুন — সপ্তাহের পর সপ্তাহ ধরে চলা branch conflict-এর জন্ম দেয়।
- নিজের local branch-এ push-এর আগে rebase করে ইতিহাস পরিষ্কার রাখুন; shared branch-এ rebase নয়।
- Merge-এর আগে `git diff main...feature` দিয়ে বদল পর্যালোচনা করুন।
- কাজ শেষ হলে branch মুছে ফেলুন (`git branch -d feature-login`) — জমে থাকা branch বিভ্রান্তি তৈরি করে।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Branch আসলে কী?
   ক) ফাইলের পূর্ণ কপি খ) একটি commit-কে দেখানো চলমান pointer গ) GitHub-এর একটি folder ঘ) একটি commit-এর বার্তা
2. Fast-forward merge কখন ঘটে?
   ক) দুই ধারাতেই নতুন commit থাকলে খ) main-এ merge-এর মধ্যে কোনো নতুন commit না থাকলে গ) Conflict থাকলে ঘ) কখনোই নয়
3. Merge conflict হলে প্রথমে কী করবেন?
   ক) Repository মুছে ফেলবেন খ) `git status` দেখে conflict-যুক্ত ফাইল হাতে ঠিক করবেন গ) Force-push করবেন ঘ) কম্পিউটার বন্ধ করবেন
4. Rebase কখন বিপজ্জনক?
   ক) নিজের local branch-এ খ) ইতিমধ্যে অন্যদের সঙ্গে ভাগ করা (push করা) commit-এ গ) নতুন branch-এ ঘ) কখনোই নয়
5. Push করা commit-এর ভুল ঠিক করতে নিরাপদ উপায় কোনটি?
   ক) `git reset --hard` খ) `git revert` গ) `.git` মুছে ফেলা ঘ) Force-push

৪.১৩ Practical Assignment

Assignment 4.1 (Intermediate): একটি repository-তে main থেকে তিনটি branch বানান — একটি feature, একটি bugfix, একটি পরীক্ষামূলক ধারণা। প্রতিটিতে অন্তত দুটি commit করুন, ইচ্ছাকৃতভাবে একটি conflict তৈরি করুন ও ঠিক করুন, একটি branch rebase করুন, আর একটি commit cherry-pick করুন। শেষে `git log --oneline --graph --all`-এর আউটপুটসহ এক পৃষ্ঠায় লিখুন কোন merge fast-forward ছিল, কোনটি three-way, আর কেন। পূর্ণ সমাধান Answer Guide-এ আছে।

৪.১৪ Chapter Summary

Branch হলো একটি commit-কে দেখানো হালকা pointer; নতুন branch বানাতে ফাইল কপি হয় না। Fast-forward merge-এ শুধু pointer সরে, three-way merge-এ দুই ধারার সাধারণ পূর্বপুরুষ থেকে একটি merge commit তৈরি হয়। একই লাইনে আলাদা বদল হলে conflict হয়, যা মানুষের সিদ্ধান্তে ঠিক করতে হয়। Rebase ইতিহাসকে সরল রেখায় সাজায়, তবে শুধু local, push-এর আগের commit-এ নিরাপদ। Cherry-pick একটি commit তুলে আনে, revert ইতিহাস না মুছে বদল উল্টায়, reset pointer সরায়। Detached HEAD-এ কাজ শুরু করার আগে branch বানান। পরের অধ্যায়ে আমরা এই branch-গুলোকে GitHub-এর মাধ্যমে দলের সঙ্গে ভাগ করা শিখব।

Revision Checklist

- ☐ Branch বানানো, বদলানো ও মুছে ফেলা করতে পারি।
- ☐ Fast-forward ও three-way merge-এর পার্থক্য বলতে পারি।
- ☐ Merge conflict হাতে ঠিক করে merge সম্পূর্ণ করতে পারি।
- ☐ Rebase কখন নিরাপদ আর কখন নয়, তা বলতে পারি।
- ☐ Cherry-pick, revert ও reset-এর পার্থক্য বলতে পারি।
- ☐ Detached HEAD চিনতে ও তা থেকে বাঁচতে পারি।

অধ্যায় ৫ • Part I — Git Fundamentals & Version Control

Chapter 05: Remote Repositories and GitHub — Remote Repository ও GitHub

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র রাফি ঢাকায়, নুসরাত সিলেটে, আর সাকিব চট্টগ্রামে — তিনজন তিন শহরে বসে একই project-এ কাজ করছে। তাদের প্রত্যেকের ল্যাপটপে repository-র পূর্ণ কপি আছে, কিন্তু তারা একে অপরের বদল দেখতে পায় না — যতক্ষণ না কেউ তা একটি shared জায়গায় পাঠায়। সেই shared জায়গাটি হলো GitHub-এর remote repository। একদিন নুসরাত একটি feature push করল, রাফি তা pull করে নিজের কাজের সঙ্গে মেলাল, আর সাকিব pull request-এ মন্তব্য করে একটি bug ধরে দিল — তিন শহরের দলটি একটিমাত্র সত্যের উৎস (single source of truth) থেকে কাজ করল। এই অধ্যায়ে আমরা remote, fetch/pull/push, SSH authentication, GitHub-এর pull request, issue, release ও নিরাপত্তা শিখব।

> **CONCEPT:** Remote মানে "আসল" repository নয় — Git-এ কোনো repository-ই অন্যটির চেয়ে আসল নয়। Remote শুধু একটি সুবিধাজনক shared কপি, যেখানে সবাই তাদের বদল পাঠায় ও সেখান থেকে নেয়। আপনার local repository-ই পূর্ণাঙ্গ; remote শুধু সহযোগিতার কেন্দ্রবিন্দু।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Remote repository-র ধারণা ও `origin` নামের অর্থ ব্যাখ্যা করতে।
- `git remote`, `fetch`, `pull` ও `push`-এর পার্থক্য ও সঠিক ব্যবহার করতে।
- Tracking branch কী ও কেন তা দরকার, তা ব্যাখ্যা করতে।
- SSH key তৈরি করে GitHub-এ যুক্ত করে password ছাড়া নিরাপদে কাজ করতে।
- GitHub-এ repository তৈরি, pull request খোলা ও পর্যালোচনা করতে।
- Issue, release, tag ও README দিয়ে project পরিচালনা করতে।
- GitHub নিরাপত্তার প্রাথমিক নিয়মগুলো প্রয়োগ করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 03 ও 04: commit, branch, merge ও conflict-এর অভিজ্ঞতা।
- একটি GitHub account (বিনামূল্যে তৈরি করা যায়) ও ইন্টারনেট সংযোগ।
- Terminal-এ SSH-এর প্রাথমিক ধারণা থাকলে ভালো; না থাকলে এই অধ্যায়েই শেখা হবে।

৫.১ Remote repository ও git remote

Remote হলো আপনার repository-র একটি কপি, যা অন্য কোথাও — সাধারণত GitHub, GitLab বা নিজের server-এ — রাখা আছে। `git remote -v` চালালে দেখা যায় কোন remote-গুলো যুক্ত আছে ও তাদের ঠিকানা কী। Clone করার সময়

Git স্বয়ংক্রিয়ভাবে মূল repository-কে `origin` নামে remote হিসেবে মনে রাখে। `origin` কোনো বিশেষ শব্দ নয় — এটি শুধু একটি প্রচলিত নাম, আপনি চাইলে `upstream` বা অন্য নামও দিতে পারেন।

```
git remote -v
git remote add origin git@github.com:technova/inventory-app.git
git remote add upstream git@github.com:example/inventory-app.git
```

প্রথম লাইন যুক্ত remote-গুলো দেখায়। দ্বিতীয় লাইন একটি remote যোগ করে — এখানে ঠিকানাটি SSH আকারে লেখা; HTTPS আকারেও লেখা যায় (`https://github.com/technova/inventory-app.git`)। তৃতীয় লাইন আরেকটি remote যোগ করে, যা fork করা project-এ মূল project-কে দেখাতে ব্যবহৃত হয়। মনে রাখুন: remote যোগ করা মানে এখনো কিছু পাঠানো বা আনা হয়নি — শুধু ঠিকানাটি মনে রাখা হলো।

৫.২ Fetch, pull ও push: তিনটি ভিন্ন কাজ

এই তিনটি কমান্ড গুলিয়ে ফেলা সবচেয়ে সাধারণ ভুল। **Fetch** remote থেকে নতুন commit ও branch-এর তথ্য আনে, কিন্তু আপনার working directory বা বর্তমান branch বদলায় না — শুধু remote-এর অবস্থা আপনার local-এ জানিয়ে দেয় (`origin/main` নামে)। **Pull** মানে fetch + merge — remote-এর বদল এনে আপনার বর্তমান branch-এর সঙ্গে মিলিয়ে দেয়। **Push** উল্টো দিকে কাজ করে — আপনার local commit-গুলোকে remote-এ পাঠায়।

কমান্ড	দিক	আপনার branch বদলায়?	কখন ব্যবহার করবেন
`git fetch`	Remote → local (শুধু তথ্য)	না	অন্যরা কী করেছে দেখতে, merge-এর আগে
`git pull`	Remote → local (এনে মেলানো)	হ্যাঁ	কাজ শুরুর আগে সর্বশেষ অবস্থা নিতে
`git push`	Local → remote	না (remote বদলায়)	নিজের commit অন্যদের সঙ্গে ভাগ করতে

```
git fetch origin
git status
git pull origin main
git push origin feature-login
```

প্রথম লাইন remote-এর নতুন তথ্য আনে। দ্বিতীয় লাইন দেখায় আপনি remote-এর তুলনায় কতটা পিছিয়ে আছেন। তৃতীয় লাইন remote-এর main branch-এর বদল এনে মেলায়। চতুর্থ লাইন আপনার feature branch-কে remote-এ পাঠায়। প্রতিদিন কাজ শুরুর আগে `git pull` করা ভালো অভ্যাস — তাহলে আপনি পুরনো কোডের উপর কাজ করবেন না।

Fetch বনাম Pull বনাম Push

আপনার PC: [main C2] GitHub (origin): [main C2 ── C3 ── C4]

git fetch origin → আপনার PC-তে origin/main = C4 জানা গেল, কিন্তু আপনার main এখনো C2-তে।

git pull origin main → আপনার main C4-এ এলো (merge বা fast-forward-এ)।

git push origin main → আপনার নতুন commit C5 GitHub-এ গেল, origin/main = C5 হলো।

Fetch শুধু খবর আনে, pull খবর এনে কাজে লাগায়, push খবর পাঠায়।

৫.৩ Tracking branch

যখন আপনি `git push -u origin feature-login` চালান, `-u` (upstream) আপনার local branch-কে remote-এর branch-এর সঙ্গে যুক্ত করে দেয় — একে বলে **tracking branch**। এরপর থেকে শুধু `git push` ও `git pull` লিখলেই Git জানে কোন remote branch-এর সঙ্গে কাজ করতে হবে। `git status` তখন বলে দেয় আপনি upstream-এর তুলনায় কত commit এগিয়ে বা পিছিয়ে আছেন — যেমন "Your branch is ahead of 'origin/main' by 2 commits"।

৫.৪ SSH authentication: password ছাড়া নিরাপদ পরিচয়

GitHub-এ push/pull করতে পরিচয় প্রমাণ করতে হয়। HTTPS পদ্ধতিতে প্রতিবার password বা personal access token লাগে। **SSH key** পদ্ধতিতে আপনি একবার একটি key জোড়া (public ও private) বানিয়ে public অংশটি GitHub-এ রাখেন; private অংশটি থাকে শুধু আপনার কম্পিউটারে। এরপর GitHub আপনার private key-এর প্রমাণ দেখে আপনাকে চিনে নেয় — password টাইপ করতে হয় না, আর password চুরি যাওয়ার ঝুঁকিও থাকে না।

```
ssh-keygen -t ed25519 -C "nahid@example.com"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
```

প্রথম লাইন একটি নতুন SSH key জোড়া বানায় — `ed25519` বর্তমানে সুপারিশকৃত অ্যালগরিদম। Key বানানোর সময় একটি passphrase দিতে বলা হবে — একটি শক্ত passphrase দিন, এটি আপনার private key-র অতিরিক্ত তালা। দ্বিতীয় লাইন public key-টি দেখায় — এই লেখাটি GitHub-এর Settings → SSH and GPG keys-এ যোগ করতে হয়। তৃতীয় লাইন সংযোগ পরীক্ষা করে — সফল হলে GitHub একটি স্বাগত বার্তা দেখায়।

> **WARNING:** Private key (`id_ed25519`) কখনো কাউকে পাঠাবেন না, GitHub-এ upload করবেন না, commit করবেন না, বা চ্যাটে শেয়ার করবেন না। শুধু `.pub` দিয়ে শেষ হওয়া public key-টি শেয়ার করা যায়। Private key ফাঁস হলে যে কেউ আপনার নামে GitHub-এ কাজ করতে পারবে — তখন GitHub থেকে সেই key মুছে নতুন key বানাতে হবে।

৫.৫ GitHub-এ repository, pull request ও issue

GitHub-এ repository তৈরি করুন, README যোগ করুন, আর local repository-কে তার সঙ্গে যুক্ত করুন। তারপর কাজের ধারা: একটি branch-এ কাজ করুন, push করুন, GitHub-এ **pull request (PR)** খুলুন — অর্থাৎ প্রস্তাব দিন "এই branch-টি main-এ merge করা হোক"। দলের অন্যরা PR-এ কোড পড়ে মন্তব্য করে, পরিবর্তন চায়, বা অনুমোদন দেয়। অনুমোদনের পর merge করা হয়। PR-ই হলো আধুনিক দলের code review-এর কেন্দ্র — Chapter 06-এ এর শিষ্টাচার বিস্তারিত আসবে।

Issue হলো কাজের তালিকা ও আলোচনার জায়গা — bug report, নতুন feature-এর প্রস্তাব, প্রশ্ন। ভালো issue-তে থাকে: সমস্যাটি কী, কীভাবে পুনরায় তৈরি করা যায়, আর প্রত্যাশিত আচরণ কী। Issue-কে PR-এর সঙ্গে যুক্ত করা যায় ("Fixes #12" লিখলে PR merge হলে issue স্বয়ংক্রিয়ভাবে বন্ধ হয়)।

৫.৬ Release, tag ও ডকুমেন্টেশন

একটি version প্রকাশ করতে চাইলে Git-এ **tag** দেওয়া হয় — যেমন `v1.0.0` — যা একটি নির্দিষ্ট commit-কে চিহ্নিত করে। GitHub-এ সেই tag থেকে **release** বানানো যায়, যেখানে লেখা থাকে এই version-এ কী নতুন, কী ঠিক হলো (release notes)। Semantic versioning অনুযায়ী `v1.2.3` মানে major.minor.patch — বিস্তারিত Chapter 06-এ।

Repository-র ডকুমেন্টেশন হলো তার মুখ। **README.md** ফাইলে থাকে: project-টি কী করে, কীভাবে install ও চালাতে হয়, কীভাবে অবদান রাখতে হয়। ভালো README ছাড়া একটি project অন্য কারও কাছে ব্যবহারের অযোগ্য। সঙ্গে থাকতে পারে `CONTRIBUTING.md` (অবদানের নিয়ম) ও `LICENSE` (ব্যবহারের অনুমতি)। এই ফাইলগুলো repository-র root-এ রাখুন ও commit করুন।

৫.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Remote	অন্য কোথাও রাখা repository-র কপি, যেমন GitHub
Origin	মূল remote-এর প্রচলিত নাম
Fetch	Remote-এর তথ্য আনা, branch না বদলে
Pull	Fetch করে বদল মিলিয়ে নেওয়া
Push	Local commit remote-এ পাঠানো
Tracking branch	Remote branch-এর সঙ্গে যুক্ত local branch
SSH key	Password ছাড়া পরিচয় প্রমাণের key জোড়া
Pull request (PR)	একটি branch merge করার প্রস্তাব ও পর্যালোচনার জায়গা
Issue	Bug, feature বা কাজের আলোচনা ও তালিকা
Tag / Release	একটি commit-এর নামযুক্ত চিহ্ন / প্রকাশিত version

৫.৮ বাস্তব প্রকৌশল পরিস্থিতি — TechNova-র প্রথম PR

নুসরাত `feature-payment` branch-এ কাজ শেষ করে push করল, আর GitHub-এ pull request খুলল: শিরোনাম "Add bKash payment option", বিবরণে লিখল কী বদলেছে ও কীভাবে পরীক্ষা করেছে। রাফি কোড পড়ে একটি জায়গায় মন্তব্য করল — একটি ভুলে amount দুবার যোগ হচ্ছে। নুসরাত ঠিক করে আবার push করল — PR-টি স্বয়ংক্রিয়ভাবে নতুন commit দেখাল, নতুন PR খুলতে হলো না। সাকিব অনুমোদন দিল, তারপর merge হলো। পুরো আলোচনাটি PR-এ সংরক্ষিত রইল — ছয় মাস পরে কেউ দেখতে পারবে কেন এই সিদ্ধান্ত নেওয়া হয়েছিল। এটি email বা চ্যাটে কোড পাঠানোর চেয়ে সম্পূর্ণ আলাদা — ইতিহাস, পর্যালোচনা ও সিদ্ধান্ত এক জায়গায় থাকে।

৫.৯ হাতে-কলমে অনুশীলন

> **LAB:** GitHub-এ প্রথম repository ও প্রথম pull request।

1. GitHub-এ একটি নতুন repository বানান (README-সহ), তার ঠিকানা কপি করুন।
2. আপনার local repository-তে `git remote add origin <ঠিকানা>` চালান, তারপর `git push -u origin main` দিয়ে push করুন।
3. SSH ব্যবহার করতে চাইলে `ssh-keygen -t ed25519` দিয়ে key বানিয়ে public key GitHub-এ যোগ করুন, আর `ssh -T git@github.com` দিয়ে পরীক্ষা করুন।
4. একটি নতুন branch-এ একটি ফাইল বদলে push করুন, GitHub-এ গিয়ে pull request খুলুন, নিজে পর্যালোচনা করে merge করুন।
5. GitHub-এ একটি issue খুলুন (যেমন "Add project documentation"), আর একটি tag বানান: `git tag v0.1.0 && git push origin v0.1.0`।

প্রত্যাশিত ফলাফল ও যাচাই: GitHub-এ আপনার commit-গুলো দেখা যাবে, PR-টি merged অবস্থায় থাকবে, issue-টি খোলা থাকবে, আর tag-টি Releases-এ দেখা যাবে। `git remote -v` ও `git status` দিয়ে local অবস্থা যাচাই করুন। `git fetch origin`-এর পর `git log --oneline --graph --all`-এ `origin/main` দেখা যাবে।

৫.১০ Common Mistakes ও Debugging

- **ভুল:** Push করার আগে pull না করা — remote-এ নতুন commit থাকলে push ব্যর্থ হবে ("rejected")। প্রথমে `git pull` করুন, conflict থাকলে ঠিক করুন, তারপর push করুন।
- **ভুল:** `git push --force` দিয়ে rejection জোর করে পার করা। এতে অন্যের commit মুছে যেতে পারে। Force-push শুধু নিজের একার branch-এ, সচেতনভাবে করুন।
- **ভুল:** ভুল remote ঠিকানা যোগ করা — `git remote -v` দিয়ে যাচাই করুন, ভুল হলে `git remote set-url origin <সঠিক ঠিকানা>`।
- **সমস্যা:** "Permission denied (publickey)" — SSH key GitHub-এ যোগ করা হয়নি, বা ভুল key ব্যবহার হচ্ছে। `ssh -T git@github.com` দিয়ে পরীক্ষা করুন।
- **সমস্যা:** Push-এ "src refspec main does not match" — local branch-এর নাম `master` হতে পারে, `git branch` দিয়ে দেখুন।
- **সমস্যা:** Clone করা repository-তে `git push` কাজ করছে না — হয়তো আপনার সেই repository-তে লেখার অনুমতি নেই (অন্যের public repo)। তখন fork করে নিজের কপিতে কাজ করুন।

৫.১১ Security Considerations

GitHub account-এ **two-factor authentication (2FA)** চালু করুন — password চুরি গেলেও account নিরাপদ থাকবে। Personal access token ব্যবহার করলে তাকে শুধু প্রয়োজনীয় অনুমতি (scope) দিন, আর কোথাও commit করবেন না। Public repository-তে কখনো secret, private key বা client-এর ব্যক্তিগত তথ্য রাখবেন না। Repository-র Settings-এ secret scanning চালু থাকলে GitHub নিজেই secret commit হলে সতর্ক করে। আর কোনো repository fork বা clone করার আগে তার উৎস যাচাই করুন — অচেনা repository-র script চালানোর আগে কোড পড়ুন।

৫.১২ Best Practices

- কাজ শুরুর আগে `git pull` করুন, কাজ শেষে নিয়মিত push করুন — local commit বেশিদিন আটকে রাখবেন না।
- সরাসরি main-এ push না করে branch ও pull request দিয়ে কাজ করুন।
- PR-এর বিবরণে লিখুন কী বদলেছে, কেন, আর কীভাবে পরীক্ষা করেছেন।
- README প্রথম দিনেই লিখুন, আর প্রতিটি বড় বদলের সঙ্গে তা হালনাগাদ করুন।
- প্রতিটি প্রকাশিত version-এ tag ও release notes দিন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `git fetch` ও `git pull`-এর পার্থক্য কোনটি?
   ক) দুটি একই খ) Fetch শুধু তথ্য আনে, pull এনে মিলিয়ে দেয় গ) Fetch পাঠায়, pull আনে ঘ) Fetch শুধু GitHub-এ চলে
2. SSH key-এর public অংশ কোথায় রাখতে হয়?
   ক) নিজের কম্পিউটারে গোপনে খ) GitHub-এ, Settings-এ গ) Repository-তে commit করে ঘ) চ্যাটে সবাইকে পাঠিয়ে
3. Pull request-এর প্রধান উদ্দেশ্য কী?
   ক) ফাইল download করা খ) একটি branch merge করার প্রস্তাব ও code review-এর জায়গা দেওয়া গ) Repository মুছে ফেলা ঘ) Password বদলানো
4. Push ব্যর্থ হলে ("rejected") সাধারণত কী করবেন?
   ক) Force-push করবেন খ) প্রথমে pull করে remote-এর বদল মিলিয়ে তারপর push করবেন গ) Repository মুছে ফেলবেন ঘ) নতুন account খুলবেন
5. Tag `v1.0.0`-এর কাজ কী?
   ক) একটি commit-কে নামযুক্ত চিহ্ন দিয়ে version চিহ্নিত করা খ) ফাইল মুছে ফেলা গ) Branch বানানো ঘ) Password সেট করা

৫.১৩ Practical Assignment

Assignment 5.1 (Intermediate): GitHub-এ একটি নতুন public repository বানান, তাতে README, `.gitignore` ও একটি ছোট project যোগ করুন। একটি feature branch-এ কাজ করে pull request খুলুন, নিজে একটি review মন্তব্য লিখুন (যেন আপনি পর্যালোচক), তারপর merge করুন। একটি issue খুলে তা PR-এর সঙ্গে যুক্ত করুন, আর `v0.1.0` tag দিয়ে একটি release বানান। শেষে repository-র লিংকসহ এক পৃষ্ঠার প্রতিবেদন লিখুন। পূর্ণ সমাধান Answer Guide-এ আছে।

৫.১৪ Chapter Summary

Remote হলো shared কপি, যার প্রচলিত নাম `origin`। Fetch শুধু remote-এর খবর আনে, pull এনে মিলিয়ে দেয়, push local commit remote-এ পাঠায়। Tracking branch (`-u`) local ও remote branch-কে যুক্ত করে। SSH key password ছাড়া নিরাপদ পরিচয় দেয় — private key কখনো শেয়ার করবেন না। GitHub-এ pull request হলো merge-এর প্রস্তাব ও code review-এর জায়গা, issue হলো কাজের তালিকা, tag ও release হলো version প্রকাশের উপায়, আর README

হলো project-এর মুখ। GitHub account-এ 2FA চালু করা ও secret commit না করা — এই দুটি নিয়ম সবসময় মনে রাখুন। পরের অধ্যায়ে আমরা এই সবকিছুকে একটি পেশাদার দলের workflow-তে সাজাব।

Revision Checklist

- ☐ Remote যোগ করা, দেখা ও ঠিকানা বদলাতে পারি।
- ☐ Fetch, pull ও push-এর পার্থক্য বলতে ও ব্যবহার করতে পারি।
- ☐ Tracking branch সেট করতে পারি।
- ☐ SSH key বানিয়ে GitHub-এ যোগ করে সংযোগ পরীক্ষা করতে পারি।
- ☐ Pull request খুলে, পর্যালোচনা করে merge করতে পারি।
- ☐ Issue, tag ও release বানাতে পারি।
- ☐ GitHub account-এ 2FA-র গুরুত্ব বলতে পারি।

অধ্যায় ৬ • Part I — Git Fundamentals & Version Control

Chapter 06: Professional Git Workflow and Collaboration — পেশাদার Git Workflow ও দলের কাজ

শুরুতে: একটি বাস্তব দৃশ্য

TechNova এখন আটজন developer-এর দল। একদিন সকালে দেখা গেল main branch-এ এমন কোড merge হয়েছে যা পরীক্ষা করা হয়নি, আর production বন্ধ। তদন্তে বের হলো: একজন সরাসরি main-এ push করেছিল, কোনো review হয়নি, commit বার্তা ছিল "fix stuff", আর কেউ জানে না কোন version-টি এখন production-এ চলছে। দলটি সিদ্ধান্ত নিল: আর কোনো সরাসরি push নয়, প্রতিটি বদল pull request দিয়ে যাবে, main branch সুরক্ষিত থাকবে, আর প্রতিটি release-এর একটি version নম্বর থাকবে। এই অধ্যায়ে আমরা সেই পেশাদার নিয়মগুলো শিখব — workflow, code review, commit convention, semantic versioning, release ব্যবস্থাপনা ও branch সুরক্ষা — যা একটি দলকে ছোট থেকে বড় হতে দেয় ভেঙে না পড়ে।

> **CONCEPT:** Workflow মানে শুধু কমান্ডের ক্রম নয় — এটি দলের একটি চুক্তি। চুক্তিটি বলে: কে কোন branch-এ কাজ করবে, কখন merge হবে, কে পর্যালোচনা করবে, আর কীভাবে একটি version প্রকাশিত হবে। ভালো workflow-তে নিয়মগুলো tool-এর মধ্যে বসানো থাকে (যেমন branch protection), মানুষের স্মৃতির উপর নির্ভর করে না।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Feature branch workflow, GitHub Flow ও trunk-based development-এর পার্থক্য বলতে ও পরিস্থিতি অনুযায়ী বাছতে।
- কার্যকর code review করতে ও pull request শিষ্টাচার প্রয়োগ করতে।
- Conventional Commits ও semantic versioning ব্যবহার করতে।
- Release ব্যবস্থাপনা ও conflict প্রতিরোধের কৌশল ব্যাখ্যা করতে।
- GitHub-এ main branch সুরক্ষিত (protected) করতে ও তার নিয়ম ব্যাখ্যা করতে।
- একটি বাস্তব দলের পরিস্থিতিতে সঠিক workflow সিদ্ধান্ত নিতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 03–05: commit, branch, merge, rebase, remote, pull request ও issue-র অভিজ্ঞতা।
- অন্তত একটি pull request খোলা ও merge করার অভিজ্ঞতা।

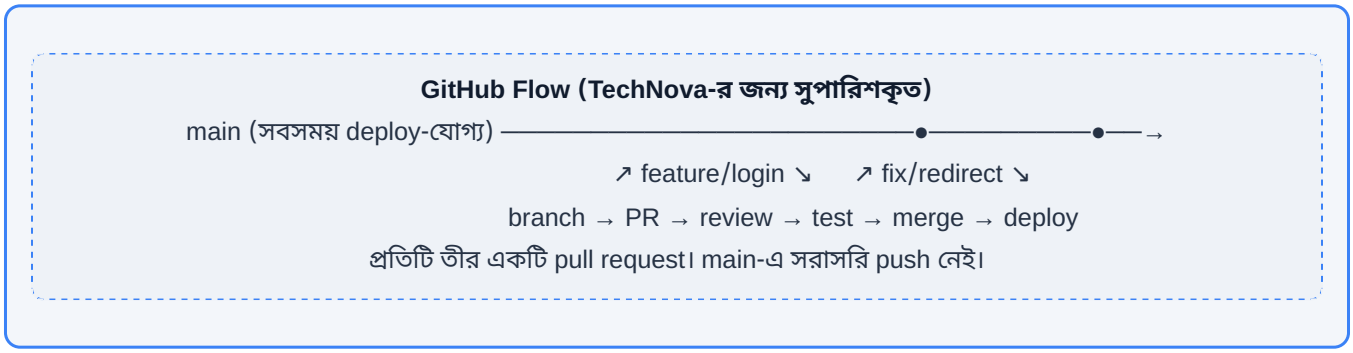
৬.১ তিনটি workflow: Feature branch, GitHub Flow, Trunk-based

Feature branch workflow-এ প্রতিটি feature বা fix-এর জন্য main থেকে একটি branch বানানো হয়, কাজ শেষে pull request দিয়ে main-এ merge করা হয়। ছোট ও মাঝারি দলের জন্য এটি সবচেয়ে পরিচিত ও নিরাপদ। **GitHub Flow** এর একটি সরল রূপ: main সবসময় deploy-যোগ্য থাকে, প্রতিটি বদল একটি branch ও pull request দিয়ে যায়, review ও পরীক্ষার পর merge হয়, আর merge হলেই deploy করা যায়। ছোট দল ও web project-এর জন্য এটি চমৎকার। **Trunk-**

based development-এ সবাই সরাসরি main (trunk)-এ বা খুব ছোটস্থায়ী branch-এ (এক-দুই দিনের) কাজ করে, আর অসমাপ্ত feature-কে feature flag দিয়ে লুকিয়ে রাখা হয়। বড় দল ও দ্রুত release-এর জন্য এটি কার্যকর, তবে শৃঙ্খলা ও automated test ছাড়া এটি বিপজ্জনক।

Workflow	Branch-এর আয়ু	ভালো যখন	ঝুঁকি
Feature branch	দিন থেকে সপ্তাহ	ছোট-মাঝারি দল, স্পষ্ট feature	দীর্ঘ branch-এ conflict বাড়ে
GitHub Flow	ঘণ্টা থেকে দিন	Web app, ঘন ঘন deploy	main-কে সবসময় deploy-যোগ্য রাখতে হয়
Trunk-based	ঘণ্টা থেকে ১–২ দিন	বড় দল, দ্রুত release, শক্তিশালী test	Test দুর্বল হলে main ভেঙে যায়

TechNova-র মতো আটজনের দলের জন্য GitHub Flow দিয়ে শুরু করা যুক্তিযুক্ত। দল বড় হলে ও automated test শক্তিশালী হলে trunk-based-এর দিকে যাওয়া যায়। কোন workflow-ই সব দলের জন্য সঠিক নয় — দলের আকার, release-এর গতি ও test-এর শক্তি দেখে বাছতে হয়।



৬.২ Code review: কীভাবে ভালো পর্যালোচনা করবেন

Code review-এর উদ্দেশ্য ভুল ধরা নয় — উদ্দেশ্য জ্ঞান ভাগ করা, মান ধরে রাখা ও bug আগেভাগে ধরা। ভালো পর্যালোচক কোড পড়ে প্রশ্ন করেন: এই বদলটি কি সমস্যাটি ঠিক করে? কোনো edge case বাদ পড়েছে কি? নামগুলো কি বোধগম্য? পরীক্ষা কি আছে? মন্তব্য লিখুন সুনির্দিষ্ট ও সম্মানজনকভাবে — "এই লাইনটি ভুল" নয়, "এখানে amount দুবার যোগ হচ্ছে বলে মনে হচ্ছে — ৪২ নম্বর লাইনটি দেখুন"। লেখক হিসেবে প্রতিটি মন্তব্যের উত্তর দিন — হয় ঠিক করে, নয় কারণসহ ব্যাখ্যা দিয়ে।

Pull request ছোট রাখুন — ৪০০ লাইনের বেশি বদল পর্যালোচনা করা কঠিন, ভুল চোখ এড়িয়ে যায়। PR-এর বিবরণে লিখুন: কী বদলেছে, কেন, কীভাবে পরীক্ষা করেছেন, আর কোনো ঝুঁকি আছে কি না। Draft PR ব্যবহার করুন যখন কাজ অসমাপ্ত কিন্তু মতামত চান।

> **PRO TIP:** Review-এর সময় প্রথমে PR-এর বিবরণ ও commit বার্তা পড়ুন, তারপর কোড। বিবরণ না বুঝলে কোড বোঝা কঠিন। আর নিজের PR merge করার আগে নিজেই একবার `git diff` দিয়ে দেখুন — নিজের ভুল নিজে ধরা সবচেয়ে সস্তা review।

৬.৩ Commit convention ও semantic versioning

দলে commit বার্তা একই নিয়মে লিখলে ইতিহাস থেকে স্বয়ংক্রিয়ভাবে release notes বানানো যায়। **Conventional Commits** নিয়মে বার্তা শুরু হয় একটি ধরন দিয়ে: `feat:` (নতুন feature), `fix:` (bug ঠিক), `docs:` (ডকুমেন্টেশন),

`refactor:` (আচরণ না বদলে কোড সাজানো), `test:` (পরীক্ষা), `chore:` (অন্যান্য কাজ)। উদাহরণ: `feat: add bKash payment option`, `fix: correct login redirect for expired sessions`।

Semantic versioning (SemVer) নিয়মে version নম্বর হয় `MAJOR.MINOR.PATCH` — যেমন `2.3.1`। MAJOR বাড়ে যখন পেছনের সঙ্গে অসামঞ্জস্যপূর্ণ (breaking) বদল আসে, MINOR বাড়ে যখন নতুন feature যোগ হয় কিন্তু পুরনো কাজ ভাঙে না, PATCH বাড়ে যখন শুধু bug ঠিক হয়। এই নিয়ম মানলে ব্যবহারকারী version নম্বর দেখেই বুঝতে পারে আপডেটটি নিরাপদ কি না।

```
git log --oneline
git tag v1.2.0
git push origin v1.2.0
```

প্রথম লাইন ইতিহাস দেখায় — Conventional Commits মানলে এই তালিকাটিই project-এর গল্প বলে। দ্বিতীয় লাইন বর্তমান commit-এ `v1.2.0` tag দেয়। তৃতীয় লাইন tag-টিকে remote-এ পাঠায়, যাতে GitHub-এ release বানানো যায়। Tag দেওয়ার আগে `git status` দিয়ে নিশ্চিত হোন আপনি ঠিক commit-এ আছেন।

৬.৪ Conflict প্রতিরোধ ও দলে কাজের নিয়ম

Conflict পুরোপুরি এড়ানো যায় না, তবে কমানো যায়। প্রথম নিয়ম: ঘন ঘন pull করুন ও ছোট branch-এ কাজ করুন — branch যত দীর্ঘ, conflict তত বেশি। দ্বিতীয় নিয়ম: কাজ শুরু করার আগে দলকে জানান কোন ফাইল ছুঁচ্ছেন — দুজন একই ফাইলের একই অংশে কাজ করলে conflict প্রায় নিশ্চিত। তৃতীয় নিয়ম: commit ছোট রাখুন, যাতে merge-এ বদল মেলানো সহজ হয়। চতুর্থ নিয়ম: merge-এর আগে `git fetch` করে দেখুন remote-এ কী নতুন এসেছে।

কাজের বণ্টনেও নিয়ম আছে: একটি feature-এর মালিক একজন — অন্যরা তার branch-এ সরাসরি push না করে pull request দিয়ে পরামর্শ দেয়। `git stash` দিয়ে অসমাপ্ত বদল সাময়িকভাবে সরিয়ে রাখা যায় — যেমন জরুরি hotfix-এর জন্য branch বদলাতে হলে: `git stash`, branch বদল, কাজ শেষে `git stash pop`।

```
git stash
git switch main
git pull origin main
git switch feature-login
git stash pop
```

প্রথম লাইন অসমাপ্ত বদল সাময়িকভাবে সরিয়ে রাখে। দ্বিতীয় ও তৃতীয় লাইন main-এ গিয়ে সর্বশেষ অবস্থা আনে। চতুর্থ লাইন আবার নিজের branch-এ ফেরে। শেষ লাইন সরিয়ে রাখা বদল ফিরিয়ে আনে। `git stash list` দিয়ে জমে থাকা stash দেখা যায় — ভুলে যাওয়া stash অনেকদিন পরে বিভ্রান্তি তৈরি করে, তাই দ্রুত ফিরিয়ে আনুন।

৬.৫ Main branch সুরক্ষা (branch protection)

GitHub-এ repository-র Settings → Branches-এ গিয়ে main branch-এর জন্য নিয়ম বসানো যায়: সরাসরি push নিষিদ্ধ, pull request ছাড়া merge নয়, অন্তত একজন পর্যালোচকের অনুমোদন লাগবে, automated test পাস না হলে merge নয়। এই নিয়মগুলো বসানো থাকলে ভুল করেও কেউ সরাসরি main-এ push করতে পারে না — tool-ই বাধা দেয়। ছোট দলেও অন্তত দুটি নিয়ম বসান: সরাসরি push বন্ধ, আর PR ছাড়া merge বন্ধ। Force-push-ও protected branch-এ বন্ধ রাখুন।

> **WARNING:** Branch protection ছাড়া একটি দল শুধু সবার সতর্কতার উপর ভরসা করে — আর মানুষ ভুল করবেই। TechNova-র production বন্ধ হওয়ার ঘটনাটি ঘটেছিল কারণ main সুরক্ষিত ছিল না। নিয়মটি বসাতে পাঁচ মিনিট লাগে, আর তা একটি রাতের production বিপর্যয় বাঁচাতে পারে।

৬.৬ Release ব্যবস্থাপনা

Release মানে একটি নির্দিষ্ট commit-কে "এই version-টি ব্যবহারকারীদের জন্য প্রস্তুত" বলে চিহ্নিত করা। ধাপগুলো: feature-গুলো main-এ merge করুন, পরীক্ষা চালান, tag দিন (`v1.2.0`), GitHub-এ release বানিয়ে release notes লিখুন — কী নতুন, কী ঠিক হলো, কোনো breaking change আছে কি না। Production-এ deploy করুন ঠিক সেই tag থেকে, main-এর সর্বশেষ অবস্থা থেকে নয় — তাহলে জানা থাকবে ঠিক কোন কোড চলছে। কোনো সমস্যা হলে আগের tag-এ ফিরে যাওয়া যায়।

৬.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
GitHub Flow	Branch → PR → review → merge → deploy, main সবসময় deploy-যোগ্য
Trunk-based development	সবাই মূল ধারায় বা খুব ছোট branch-এ কাজ করা
Code review	Merge-এর আগে অন্য developer-এর কোড পড়ে মতামত দেওয়া
Conventional Commits	feat/fix/docs ধরন দিয়ে শুরু করা commit বার্তার নিয়ম
Semantic versioning	MAJOR.MINOR.PATCH নিয়মে version নম্বর দেওয়া
Branch protection	সরাসরি push বন্ধ করে PR ও অনুমোদন বাধ্যতামূলক করা
Feature flag	অসমাপ্ত feature-কে কোডে লুকিয়ে রাখার সুইচ
git stash	অসমাপ্ত বদল সাময়িকভাবে সরিয়ে রাখা
Release	একটি version-কে প্রকাশিত হিসেবে চিহ্নিত করা

৬.৮ বাস্তব প্রকৌশল পরিস্থিতি — TechNova-র নতুন চুক্তি

TechNova-র নতুন চুক্তিটি লিখিত: (১) main-এ সরাসরি push নয় — সব বদল branch ও PR দিয়ে যাবে। (২) প্রতিটি PR-এ অন্তত একজন পর্যালোচকের অনুমোদন লাগবে। (৩) Commit বার্তা Conventional Commits নিয়মে লিখতে হবে। (৪) প্রতি শুক্রবার release — tag দিয়ে, release notes-সহ। (৫) কাজ শুরুর আগে pull, কাজ শেষে push — local commit দুই দিনের বেশি আটকে রাখা যাবে না। প্রথম মাসে দলটি দেখল PR-গুলো ছোট হয়েছে, review দ্রুত হয়েছে, আর main আর কখনো ভাঙেনি। নিয়মগুলো প্রথমে বাড়তি কাজ মনে হয়েছিল, কিন্তু সেগুলোই দলকে দ্রুত করেছে — কারণ ভাঙা main ঠিক করতে যে সময় যেত, তার চেয়ে review-এ সময় অনেক কম।

৬.৯ হাতে-কলমে অনুশীলন

> **LAB:** দলের workflow-টি নিজের repository-তে বসানো।

1. GitHub-এ আপনার repository-র Settings → Branches-এ গিয়ে main-এর জন্য branch protection চালু করুন: সরাসরি push বন্ধ, PR বাধ্যতামূলক।
2. একটি branch-এ Conventional Commits নিয়মে দুটি commit করুন (`feat:` ও `fix:`), push করে PR খুলুন।
3. PR-এর বিবরণে লিখুন কী বদলেছে, কেন, কীভাবে পরীক্ষা করেছেন। নিজে Files changed ট্যাবে গিয়ে একটি লাইনে মন্তব্য লিখুন।
4. সরাসরি main-এ push করার চেষ্টা করুন — GitHub বাধা দেবে; এটি নিশ্চিত করুন যে সুরক্ষা কাজ করছে।
5. PR merge করুন, তারপর `git tag v0.2.0` দিয়ে tag দিন ও GitHub-এ একটি release বানান।

প্রত্যাশিত ফলাফল ও যাচাই: সরাসরি push ব্যর্থ হবে (সুরক্ষার প্রমাণ)। PR merge-এর পর main-এ আপনার commit দেখা যাবে। Release পৃষ্ঠায় `v0.2.0` ও আপনার লেখা notes দেখা যাবে। `git log --oneline`-এ `feat:` ও `fix:` বার্তাগুলো ইতিহাসের গল্প বলবে।

৬.১০ Common Mistakes ও Debugging

- **ভুল:** PR-এ হাজার লাইনের বদল পাঠানো — কেউ ঠিকমতো review করতে পারে না। বড় কাজকে ছোট ছোট PR-এ ভাগ করুন।
- **ভুল:** Review-এর মন্তব্য ব্যক্তিগতভাবে নেওয়া। মন্তব্য কোডের উপর, মানুষের উপর নয় — আর পর্যালোচক হিসেবে ভাষা সম্মানজনক রাখুন।
- **ভুল:** Branch সপ্তাহের পর সপ্তাহ ধরে খোলা রাখা — merge-এর সময় conflict-এর পাহাড়। দুই-তিন দিনের মধ্যে merge করার লক্ষ্য রাখুন।
- **ভুল:** Commit বার্তায় "fix stuff" লেখা — release notes বানানোর সময় কেউ বুঝবে না কী ঠিক হয়েছিল।
- **সমস্যা:** PR-এ conflict দেখাচ্ছে — local-এ main pull করে নিজের branch-এ merge বা rebase করে conflict ঠিক করুন, তারপর push করুন।
- **সমস্যা:** Stash করা বদল ভুলে গেছেন — `git stash list` দিয়ে দেখুন, `git stash pop` দিয়ে ফিরিয়ে আনুন।

৬.১১ Security Considerations

Branch protection নিরাপত্তারও অংশ: এটি নিশ্চিত করে কোনো একজন ব্যক্তি একা production-এর কোড বদলাতে পারে না — অন্তত একজন পর্যালোচক লাগে। Repository-তে কে লিখতে পারে (collaborator) তা নিয়মিত পর্যালোচনা করুন; যিনি দল ছেড়েছেন, তার access সরিয়ে দিন। Secret কখনো PR-এর বিবরণে বা commit বার্তায় লিখবেন না — PR-এর ইতিহাসও public হতে পারে। আর fork থেকে আসা PR merge করার আগে কোড পড়ুন — অচেনা অবদানকারীর কোডে ক্ষতিকর কিছু থাকতে পারে।

৬.১২ Best Practices

- Main-কে সবসময় deploy-যোগ্য রাখুন — ভাঙা কোড main-এ merge করবেন না।
- PR ছোট রাখুন, বিবরণ সম্পূর্ণ লিখুন, আর review দ্রুত করুন — আটকে থাকা PR দলকে ধীর করে।

- Commit বার্তা ও version নম্বরে নিয়ম মানুন — ইতিহাসই আপনার ডকুমেন্টেশন।
- প্রতিটি release-এ tag ও notes দিন; production-এ কী চলছে, তা সবসময় জানা থাকতে হবে।
- নিয়মগুলো tool-এ বসান (branch protection, required review) — শুধু মুখের কথায় নয়।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. GitHub Flow-এর মূল নিয়ম কোনটি?
   ক) সরাসরি main-এ push করা খ) Main সবসময় deploy-যোগ্য, প্রতিটি বদল branch ও PR দিয়ে যায় গ) Branch কখনো merge না করা ঘ) Review ছাড়া deploy করা
2. Trunk-based development কখন উপযুক্ত?
   ক) কোনো test ছাড়া খ) বড় দল, শক্তিশালী automated test ও দ্রুত release থাকলে গ) একা কাজ করলে ঘ) কখনোই নয়
3. Conventional Commits-এ নতুন feature-এর বার্তা কীভাবে শুরু হয়?
   ক) `update:` খ) `feat:` গ) `fix:` ঘ) `asdf:`
4. Semantic versioning-এ `2.0.0`-তে MAJOR বাড়ার অর্থ কী?
   ক) শুধু bug ঠিক হয়েছে খ) পেছনের সঙ্গে অসামঞ্জস্যপূর্ণ (breaking) বদল এসেছে গ) শুধু ডকুমেন্টেশন বদলেছে ঘ) কিছুই বদলায়নি
5. Branch protection-এর কাজ কী?
   ক) Branch মুছে ফেলা খ) সরাসরি push বন্ধ করে PR ও অনুমোদন বাধ্যতামূলক করা গ) Repository private করা ঘ) Commit বার্তা মুছে ফেলা

৬.১৩ Practical Assignment

Assignment 6.1 (Intermediate): আপনার GitHub repository-র জন্য এক পৃষ্ঠার "Team Workflow Guide" লিখুন: কোন workflow বাছলেন ও কেন, branch-এর নামের নিয়ম, PR-এর বিবরণের নমুনা, commit বার্তার নিয়ম, review-এর নিয়ম, release-এর ধাপ, আর branch protection-এর সেটিং। তারপর নিজের repository-তে সেই নিয়মগুলো বসিয়ে একটি PR দিয়ে যাচাই করুন যে নিয়ম কাজ করছে। পূর্ণ সমাধান Answer Guide-এ আছে।

৬.১৪ Chapter Summary

Feature branch workflow প্রতিটি কাজের জন্য আলাদা branch ব্যবহার করে, GitHub Flow main-কে সবসময় deploy-যোগ্য রেখে প্রতিটি বদল PR দিয়ে নেয়, আর trunk-based development খুব ছোট branch-এ দ্রুত কাজ করে — বাছাই নির্ভর করে দলের আকার, release-এর গতি ও test-এর শক্তির উপর। ভালো code review সুনির্দিষ্ট ও সম্মানজনক, PR ছোট ও বিবরণসম্পন্ন হয়। Conventional Commits (`feat:` / `fix:`) ইতিহাসকে পাঠযোগ্য করে, semantic versioning (`MAJOR.MINOR.PATCH`) version-এর অর্থ বোঝায়। Conflict প্রতিরোধের উপায় ঘন ঘন pull, ছোট branch ও স্পষ্ট বণ্টন। Branch protection নিয়মগুলোকে tool-এর মধ্যে বসিয়ে দেয় — সরাসরি push বন্ধ, PR ও অনুমোদন বাধ্যতামূলক। এই ছয়টি অধ্যায়ে Git-এর ভিত্তি সম্পূর্ণ হলো — Part II-এ আমরা Bash scripting দিয়ে automation-এর দিকে যাব, যেখানে Git-এর এই শৃঙ্খলা script-গুলোকে নিরাপদে পরিচালনা করতে কাজে লাগবে।

Revision Checklist

- ☐ তিনটি workflow-এর পার্থক্য বলতে ও পরিস্থিতি অনুযায়ী বাছতে পারি।
- ☐ ভালো code review করতে ও PR শিষ্টাচার মানতে পারি।
- ☐ Conventional Commits ও semantic versioning ব্যবহার করতে পারি।
- ☐ Conflict প্রতিরোধের চারটি কৌশল বলতে পারি।
- ☐ `git stash` ব্যবহার করতে পারি।
- ☐ GitHub-এ main branch সুরক্ষিত করতে পারি ও release বানাতে পারি।

PART II • DEVOPS ENGINEERING MASTERY

Bash Scripting Engineering

Bash স্ক্রিপ্টিং — শেল, ভেরিয়েবল, শর্ত ও লুপ, ফাংশন, টেক্সট প্রসেসিং, এরর হ্যান্ডলিং ও নিরাপদ অটোমেশন।

অধ্যায় ৭ • Part II — Bash Scripting Engineering

Chapter 07: Introduction to Shell Scripting — শেল স্ক্রিপ্টিংয়ের পরিচয়

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকায় TechNova-র একজন junior DevOps engineer প্রতিদিন সকালে একই কাজ করতেন: পাঁচটি সার্ভারে লগ ইন করা, disk ভরা কি না দেখা, একটি service চলছে কি না পরীক্ষা করা, আর একটি লগ ফাইলের শেষ লাইনগুলো পড়া। প্রতিটি সার্ভারে পাঁচটি করে কমান্ড — মোট পঁচিশটি কমান্ড, প্রায় চল্লিশ মিনিট। একদিন তিনি ভুল করে একটি সার্ভারে ভুল কমান্ড চালিয়ে একটি লগ ফাইল মুছে ফেললেন। সেদিনই তিনি বুঝলেন, মানুষের হাতে বারবার একই কাজ করানো শুধু ধীর নয়, ভুলের ঝুঁকিও বহন করে। সমাধান ছিল একটি ছোট Bash script — একবার লিখলে তা প্রতিদিন একইভাবে, একই ক্রমে, কোনো ক্লান্তি ছাড়াই কাজটি করে। এই অধ্যায় সেই যাত্রার শুরু: shell কী, script কী, আর কেন একজন DevOps engineer-এর প্রথম automation ভাষা Bash।

এই বইয়ের Part I-এ আপনি Git শিখেছেন — কোডের ইতিহাস কীভাবে সংরক্ষণ করতে হয়। এখন শিখবেন কম্পিউটারকে নির্দেশনা দিয়ে কাজ করানো — অর্থাৎ automation-এর ভাষা। Git আপনাকে কোডের সংস্করণ দেয়, Bash আপনাকে সময় ফিরিয়ে দেয়।

> **CONCEPT:** Shell হলো মানুষ ও kernel-এর মধ্যে অনুবাদক। আপনি ইংরেজি-সদৃশ কমান্ড লেখেন, shell তা system call-এ বদলে kernel-কে দেয়। Script হলো সেই কমান্ডগুলোর লিখিত তালিকা — যেন একটি রেসিপি, যা কম্পিউটার বারবার একইভাবে অনুসরণ করে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Shell, terminal ও kernel-এর সম্পর্ক নিজের ভাষায় ব্যাখ্যা করতে।
- Bash ও অন্যান্য shell-এর পার্থক্য বলতে।
- Interactive ও non-interactive shell চিনতে ও ব্যাখ্যা করতে।
- Shebang লিখে একটি script executable করে চালাতে।
- File permission (read, write, execute) ব্যাখ্যা করে `chmod +x` ব্যবহার করতে।
- একটি সুগঠিত script-এর কাঠামো লিখতে।
- `bash -x` ও `set -x` দিয়ে script debug করতে।
- নিরাপদ scripting-এর মৌলিক নীতিগুলো প্রয়োগ করতে।

Prerequisites — পূর্বজ্ঞান

- Ubuntu terminal খোলা ও `ls`, `cd`, `pwd`, `cat`-এর মতো মৌলিক কমান্ড চালানোর অভিজ্ঞতা।
- Part I-এর Git অধ্যায়গুলো — কারণ script-ও Git repository-তে সংরক্ষণ করা হয়।
- যেকোনো একটি প্রোগ্রামিং ভাষায় variable ও if-এর ধারণা থাকলে ভালো, না থাকলেও চলবে — এই অধ্যায়ে ধাপে ধাপে শেখানো হবে।

৭.১ Shell আসলে কী?

কম্পিউটারের ভেতরে kernel হলো hardware পরিচালনার দায়িত্বে থাকা মূল প্রোগ্রাম। কিন্তু kernel সরাসরি মানুষের সঙ্গে কথা বলে না — তার ভাষা system call, যা মানুষের পক্ষে ব্যবহার করা কঠিন। **Shell** হলো kernel-এর চারপাশের খোলস (shell শব্দের অর্থই খোলস) — একটি প্রোগ্রাম যা আপনার লেখা কমান্ড পড়ে, বোঝে, আর kernel-কে দিয়ে কাজ করায়। আপনি যখন terminal-এ `ls` লেখেন, তখন shell সেই শব্দটি পড়ে, `/usr/bin/ls` প্রোগ্রামটি খুঁজে বের করে, চালায়, আর ফলাফল পর্দায় দেখায়।

একটি উপমা: একটি হোটেলের receptionist। আপনি receptionist-কে বলেন, "আমার ঘরে খাবার পাঠান।" Receptionist নিজে রান্না করেন না — তিনি রান্নাঘরকে (kernel) নির্দেশ দেন, আর খাবার এলে আপনার কাছে পৌঁছে দেন। Shell ঠিক এই receptionist-এর কাজ করে: সে নিজে ফাইল মুছে না, disk পড়ে না — সে kernel ও অন্যান্য প্রোগ্রামকে নির্দেশ দিয়ে ফলাফল আপনাকে ফেরত দেয়।

Terminal আর shell এক নয় — এটাই সবচেয়ে সাধারণ ভুল ধারণা। **Terminal** হলো জানালা — একটি প্রোগ্রাম (যেমন GNOME Terminal) যা কীবোর্ডের লেখা দেখায় আর পর্দায় ফলাফল আঁকে। **Shell** হলো জানালার ভেতরে বসা ব্যক্তি, যে আপনার লেখা বোঝে ও কাজ করে। একই terminal-এ Bash, sh, zsh বা fish — যেকোনো shell চালানো যায়। Terminal বদলালে shell বদলায় না, আর shell বদলালে terminal বদলায় না।

উপাদান	কাজ	উদাহরণ
Kernel	Hardware, memory, process ও ফাইল পরিচালনা	Linux kernel 6.8
Shell	কমান্ড পড়া, ব্যাখ্যা করা, প্রোগ্রাম চালানো	Bash, sh, zsh, fish
Terminal	লেখা দেখানো ও কীবোর্ডের input নেওয়া	GNOME Terminal, Terminal.app
Command	একটি নির্দিষ্ট কাজের নির্দেশনা	ls, grep, systemctl

৭.২ Bash বনাম অন্যান্য Shell

Bash (Bourne Again SHell) হলো সবচেয়ে প্রচলিত Linux shell। এর নামের "Again" অংশটি এসেছে পুরনো Bourne shell (sh)-এর উন্নত সংস্করণ হিসেবে — যেন "Bourne-again"। Ubuntu Server 24.04 LTS-এ Bash 5.2.21 হলো ডিফল্ট shell, আর এই বইয়ের পরিবেশে আমরা সেটিই ব্যবহার করব। অন্যান্য shell-ও আছে: `sh` (সবচেয়ে পুরনো, সবচেয়ে সীমিত, কিন্তু সব Unix-এ পাওয়া যায়), `zsh` (interactive ব্যবহারে জনপ্রিয়, macOS-এর ডিফল্ট), `fish` (বন্ধুত্বপূর্ণ, কিন্তু Bash-এর সঙ্গে syntax-এ অমিল আছে)।

কেন Bash? তিনটি কারণে। প্রথমত, প্রায় প্রতিটি Linux সার্ভারে Bash আগে থেকেই আছে — আলাদা করে install করতে হয় না। দ্বিতীয়ত, প্রায় সব DevOps টুল, Docker image আর CI/CD pipeline Bash ধরে নিয়েই লেখা। তৃতীয়ত, Bash-এর syntax একবার শিখলে sh-এর script-ও পড়া যায়, কারণ Bash sh-এর প্রায় পুরো syntax বোঝে। তবে একটি সতর্কতা: zsh বা fish-এর বিশেষ সুবিধা (যেমন fish-এর autosuggestion) Bash script-এ ব্যবহার করা যায় না — production script লিখতে হবে portable Bash-এ, যা Chapter 13-এ বিস্তারিত আসবে।

কোন shell চলছে তা জানতে terminal-এ লিখুন `echo $0` — উত্তর `-bash` বা `bash` হলে আপনি Bash-এ আছেন। আর `bash --version` লিখলে version দেখাবে। এই বইয়ে পরীক্ষিত পরিবেশ: Ubuntu Server 24.04 LTS, Bash 5.2.21, Git 2.43, Python 3.12। আপনার সার্ভারে version কিছুটা আলাদা হতে পারে — বড় পার্থক্য থাকলে বইয়ের কমান্ড চালানোর আগে version মিলিয়ে নিন।

৭.৩ Interactive বনাম Non-interactive Shell

Shell দুইভাবে চলতে পারে। **Interactive shell** হলো সেই shell যেখানে আপনি বসে একের পর এক কমান্ড লেখেন, আর shell সঙ্গে সঙ্গে উত্তর দেয় — যেমন terminal খুলে কাজ করা। এখানে prompt দেখা যায় (`$` চিহ্ন), history কাজ করে (উপরের তীর চাপলে আগের কমান্ড আসে), আর tab চাপলে নামের পরামর্শ আসে। **Non-interactive shell** হলো সেই shell যা কোনো script ফাইল পড়ে, তার কমান্ডগুলো একের পর এক চালিয়ে শেষ হয়ে যায় — মানুষের সঙ্গে কথা বলে না। cron job চললে, বা `bash backup.sh` লিখলে, একটি non-interactive shell তৈরি হয়।

পার্থক্যটি কেন গুরুত্বপূর্ণ? কারণ দুই ধরনের shell আলাদা startup ফাইল পড়ে। Interactive shell পড়ে `~/.bashrc` — যেখানে আপনার alias, prompt-এর রঙ, PATH-এর বাড়তি অংশ থাকে। Non-interactive shell তা পড়ে না (বা খুব কম পড়ে)। ফলে terminal-এ চলা একটি কমান্ড script-এ বা cron-এ চলতে গিয়ে "command not found" বলতে পারে — কারণ PATH আলাদা। এই সমস্যাটি Chapter 20-এ cron-এর প্রসঙ্গে আবার আসবে, তবে এখনই মনে রাখুন: script-এ কখনো terminal-এর alias বা পরিবেশের উপর নির্ভর করবেন না — পুরো পথ (absolute path) বা script-এর ভেতরেই PATH ঠিক করে নিন।

Shell-এর কাজের প্রবাহ

কীবোর্ড / Script ফাইল → Shell (কমান্ড পড়ে ও ব্যাখ্যা করে) → Kernel (system call) → Hardware (CPU, Disk, Network)

↑ ফলাফল ফেরত আসে একই পথে: Hardware → Kernel → Shell → Terminal-এ প্রদর্শন

Interactive: মানুষ লেখে, shell সঙ্গে সঙ্গে উত্তর দেয় (prompt $ দেখা যায়)
Non-interactive: script ফাইল পড়ে, শেষ হলে shell বন্ধ হয়ে যায় (কোনো prompt নেই)

৭.৪ Script Execution: কমান্ডের তালিকা থেকে প্রোগ্রাম

Script মানে একটি text ফাইলে সাজানো কমান্ডের তালিকা। আপনি terminal-এ যা যা লিখতেন, তা-ই ফাইলে লিখে রাখলে shell সেটি একটানা চালাতে পারে। পার্থক্য শুধু একটাই: terminal-এ ভুল হলে আপনি থেমে ভাবতে পারেন, script-এ ভুল হলে script নিজে থেকে থামে না — সে পরের লাইনে চলে যায়, যদি না আপনি তাকে থামতে শেখান (Chapter 12)।

Script চালানোর দুটি উপায় আছে। প্রথম: `bash script.sh` — এখানে আপনি Bash-কে বলছেন, "এই ফাইলটি পড়ে চালাও।" ফাইলটির executable permission না থাকলেও চলবে, কারণ Bash নিজেই ফাইলটি পড়ছে। দ্বিতীয়: `./script.sh` — এখানে আপনি সরাসরি ফাইলটিকে প্রোগ্রাম হিসেবে চালাচ্ছেন। এর জন্য দুটি জিনিস লাগে: ফাইলটিতে execute permission থাকতে হবে, আর ফাইলের প্রথম লাইনে shebang থাকতে হবে যাতে kernel জানে কোন interpreter দিয়ে ফাইলটি চালাতে হবে। Production-এ দ্বিতীয় উপায়টিই মান — কারণ তা script-কে একটি সত্যিকারের কমান্ডের মতো ব্যবহারযোগ্য করে।

একটি গুরুত্বপূর্ণ প্রশ্ন: `./script.sh` লিখতে হয় কেন, শুধু `script.sh` লিখলে হয় না কেন? কারণ shell কমান্ড খোঁজে PATH নামের directory-র তালিকায়, বর্তমান directory-তে নয়। এটি নিরাপত্তার জন্য ইচ্ছাকৃত: কেউ যদি একটি ক্ষতিকর ফাইলের নাম `ls` রেখে আপনার directory-তে রেখে দেয়, আর আপনি সেখানে `ls` লেখেন, তাহলে ক্ষতিকর ফাইলটি চলার ঝুঁকি থাকে। `./` দিয়ে আপনি স্পষ্ট করে বলছেন, "এই directory-র এই ফাইলটিই চালাও।" বর্তমান directory-কে PATH-এ যোগ করা নিরাপত্তার ঝুঁকি — কখনো করবেন না।

৭.৫ Shebang: প্রথম লাইনের গুরুত্ব

Script-এর প্রথম লাইন, যেমন `#!/usr/bin/env bash`, তার নাম **shebang** — `#!` চিহ্ন দুটির নাম she (hash) ও bang (exclamation) থেকে। যখন kernel একটি executable text ফাইল চালাতে যায়, সে প্রথম দুটি অক্ষর দেখে: `#!` থাকলে সে বোঝে এটি একটি script, আর `#!`-এর পরের অংশটিকে interpreter-এর পথ হিসেবে পড়ে। অর্থাৎ shebang kernel-কে বলে দেয়, "এই ফাইলটি Bash দিয়ে চালাও।"

দুটি রূপ প্রচলিত: `#!/bin/bash` (সরাসরি পথ) ও `#!/usr/bin/env bash` (env প্রোগ্রামের মাধ্যমে Bash খোঁজা)। দ্বিতীয়টি ভালো, কারণ বিভিন্ন সিস্টেমে Bash-এর অবস্থান আলাদা হতে পারে — macOS-এ Bash থাকে `/bin/bash`-এ, কিন্তু NixOS-এর মতো সিস্টেমে অন্য জায়গায়। `env` PATH-এ Bash খুঁজে নেয়, তাই script বেশি portable হয়। এই বইয়ের সব script-এ `#!/usr/bin/env bash` ব্যবহার করা হবে।

Shebang ছাড়া কী হয়? `bash script.sh` দিয়ে চালালে কিছু হয় না — কারণ আপনি নিজেই interpreter ঠিক করে দিয়েছেন। কিন্তু `./script.sh` দিয়ে চালালে kernel জানে না কোন interpreter ব্যবহার করবে, তাই সে ফাইলটিকে shell script ধরে বর্তমান shell দিয়ে চালানোর চেষ্টা করে — অনেক সময় কাজ করে, কিন্তু Bash-এর বিশেষ syntax থাকলে ভুল ফল দিতে পারে। তাই নিয়ম: প্রতিটি script-এর প্রথম লাইনে shebang থাকবেই।

৭.৬ File Permissions: কে কী করতে পারে

Linux-এ প্রতিটি ফাইলের তিনটি permission থাকে তিন শ্রেণির ব্যবহারকারীর জন্য: **read (r)** — ফাইল পড়া যায়, **write (w)** — ফাইল বদলানো যায়, **execute (x)** — ফাইল প্রোগ্রাম হিসেবে চালানো যায়। তিন শ্রেণি হলো: **owner** (ফাইলের মালিক), **group** (মালিকের group-এর সদস্যরা), **others** (বাকি সবাই)। `ls -l script.sh` লিখলে দেখবেন `-rw-r--r--`-এর মতো একটি চিহ্ন — প্রথম `-` মানে এটি ফাইল (directory হলে `d` হতো), তারপর তিনটি করে owner, group ও others-এর permission।

Script-কে executable করতে লিখুন `chmod +x script.sh` — এটি owner, group ও others — সবার জন্য execute permission যোগ করে। আরও সুনির্দিষ্টভাবে `chmod 755 script.sh` লিখলে owner পায় rwx (৭), group ও others পায় r-x (৫)। সংখ্যার হিসাব: r = ৪, w = ২, x = ১ — যোগ করে মান হয়। ৭ মানে ৪+২+১ = সব permission, ৫ মানে ৪+১ = পড়া ও চালানো, লেখা নয়। Production script-এর জন্য 755 একটি ভালো মান: মালিক বদলাতে পারে, অন্যরা শুধু চালাতে পারে।

কখনো `chmod 777` ব্যবহার করবেন না — এটি সবাইকে সব permission দেয়, অর্থাৎ যেকোনো ব্যবহারকারী আপনার script বদলে ফেলতে পারে। একটি সার্ভারে এটি গুরুতর নিরাপত্তা ঝুঁকি। একইভাবে `sudo chmod` দিয়ে root-এর ফাইলের permission বদলানোর আগে ভাবুন — permission বদলানো সহজ, কিন্তু ভুল permission-এর ফলে সৃষ্ট নিরাপত্তা ফাঁক খুঁজে পাওয়া কঠিন।

Permission	সংখ্যা	অর্থ	Script-এর জন্য
r	4	ফাইল পড়া যায়	Interpreter-কে ফাইল পড়তে হয়
w	2	ফাইল বদলানো যায়	শুধু মালিকের থাকা উচিত
x	1	প্রোগ্রাম হিসেবে চালানো যায়	./script.sh চালাতে লাগে
755	—	owner rwx, group r-x, others r-x	Production script-এর মান
644	—	owner rw-, group r--, others r--	শুধু পড়ার ফাইল, চালানো যায় না

৭.৭ একটি সুগঠিত Script-এর কাঠামো

একটি ভালো script এলোমেলো কমান্ডের স্তূপ নয় — তার একটি কাঠামো থাকে, যেমন একটি ভালো চিঠির শুরু, মূল অংশ ও শেষ থাকে। এই বইয়ে আমরা প্রতিটি script-এ একই কাঠামো ব্যবহার করব, যাতে script বড় হলেও পড়া সহজ থাকে। কাঠামোটি হলো: (১) shebang, (২) script-এর উদ্দেশ্য বর্ণনাকারী সংক্ষিপ্ত মন্তব্য, (৩) `set -euo pipefail` (Chapter 12-এ বিস্তারিত), (৪) variable ও configuration, (৫) function-এর সংজ্ঞা (Chapter 10), (৬) মূল কাজ — সাধারণত একটি `main` function-এর ভেতরে, (৭) শেষে `main "$@"` দিয়ে script শুরু করা।

কেন `main` function? কারণ script-এর মূল যুক্তি একটি function-এর ভেতরে থাকলে তা পড়া, পরীক্ষা করা ও পুনঃব্যবহার করা সহজ হয়। ছোট script-এ এটি বাড়তি মনে হতে পারে, কিন্তু অভ্যাসটি প্রথম দিন থেকে গড়ে তুললে বড় script লেখার সময় তা স্বাভাবিক হয়ে যায়। এই বইয়ের Chapter 10-এ function বিস্তারিত শেখানো হবে — আপাতত কাঠামোটি চিনে রাখুন।

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR="/var/backups/technova"
DATE_STAMP="$(date +%F)"

main() {
    echo "TechNova daily check started: ${DATE_STAMP}"
    echo "Disk usage:"
    df -h /
    echo "Daily check finished."
}

main "$@"
```

লাইন ধরে ব্যাখ্যা: প্রথম লাইন shebang — Bash দিয়ে চালাও। দ্বিতীয় লাইন `set -euo pipefail` — ভুল হলে script থেমে যাবে, অঘোষিত variable ব্যবহার করলে ভুল ধরবে, আর pipeline-এর কোনো অংশ ভুল করলে তা ধরা পড়বে (Chapter 12-এ প্রতিটি অংশের বিস্তারিত)। `BACKUP_DIR` একটি variable — backup কোথায় রাখা হবে, তার পথ। `DATE_STAMP`-এ `$(date +%F)` হলো command substitution — `date` কমান্ডের ফলাফল variable-এ রাখা হচ্ছে (Chapter 08)। `main() { ... }` একটি function-এর সংজ্ঞা — ভেতরের কমান্ডগুলো এখনই চলবে না, শুধু সংজ্ঞায়িত হলো। শেষ লাইন `main "$@"` function-টিকে ডাকে, আর `"$@"` script-কে দেওয়া সব argument function-এর কাছে পৌঁছে দেয়।

এই উদাহরণে একটি ভুল ইচ্ছাকৃতভাবে রাখা হয়নি — তবে লক্ষ করুন, `BACKUP_DIR` variable-টি এখানে ব্যবহার করা হয়নি। বাস্তব script-এ অব্যবহৃত variable রাখা উচিত নয় — ShellCheck (Chapter 13) এই ধরনের ভুল ধরিয়ে দেয়।

> **PRO TIP:** Script লেখার আগে terminal-এ কমান্ডগুলো একে একে চালিয়ে দেখুন। Terminal-এ কাজ করে — এমন কমান্ডের তালিকাই script। প্রথমে হাতে চালিয়ে যাচাই, তারপর script-এ সাজানো — এই ক্রমটি ভুল কমান্ড script-এ ঢুকে যাওয়া ঠেকায়।

৭.৮ Bash Debugging: ভুল খুঁজে বের করা

Script ভুল করলে প্রথম কাজ আতঙ্কিত হওয়া নয় — দেখা, আসলে কী চলছে। Bash-এর সবচেয়ে কার্যকর debugging উপায় হলো **trace mode**: `bash -x script.sh` লিখলে Bash প্রতিটি কমান্ড চালানোর আগে সেটি পর্দায় দেখায়, `+` চিহ্ন

দিয়ে শুরু করে। যেমন `+ df -h /` দেখলে বুঝবেন এই কমান্ডটি চলছে। আর script-এর ভেতরে `set -x` লিখলে সেই লাইনের পর থেকে trace চালু হয়, আর `set +x` লিখলে বন্ধ হয় — অর্থাৎ শুধু সন্দেহজনক অংশটি trace করা যায়।

আরেকটি উপায়: script চালানোর আগে `bash -n script.sh` দিয়ে শুধু syntax পরীক্ষা করা — এটি script চালায় না, শুধু দেখে syntax ঠিক আছে কি না। Syntax ভুল থাকলে সে লাইন নম্বরসহ জানিয়ে দেয়। তবে মনে রাখবেন, `bash -n` যুক্তির ভুল (logic error) ধরতে পারে না — যেমন ভুল ফাইলের নাম লেখা, বা ভুল directory-তে backup নেওয়া। সেগুলো ধরতে trace আর সতর্ক পরীক্ষা লাগে।

Debugging-এর একটি পুরনো কিন্তু কার্যকর কৌশল: সন্দেহজনক variable-এর মান `echo` দিয়ে দেখা। যেমন `echo "BACKUP_DIR is: ${BACKUP_DIR}"`। তবে এই `echo` লাইনগুলো কাজ শেষ হলে মুছে ফেলতে হয় — নয়তো production-এ অপ্রয়োজনীয় output জমা হবে। আরও ভালো উপায় হলো Chapter 12-এর logging function ব্যবহার করা, যা প্রয়োজনে বন্ধ করা যায়।

> **WARNING:** `set -x` trace-এ variable-এর মানও দেখা যায়। Script-এ যদি password বা token থাকে (থাকা উচিত নয়), তাহলে trace output-এ তা ফাঁস হয়ে যাবে। তাই trace চালানোর আগে নিশ্চিত হোন script-এ কোনো গোপন তথ্য নেই, আর trace output কখনো অন্যের সঙ্গে শেয়ার করবেন না।

৭.৯ নিরাপদ Scripting-এর ভিত্তি

এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ শিক্ষা একটি অভ্যাস: **script লেখার সময় ধরুন, সবকিছু ভুল হতে পারে।** ফাইলটি নাও থাকতে পারে। Disk ভরা থাকতে পারে। ব্যবহারকারী ভুল input দিতে পারে। Network নাও থাকতে পারে। একজন ভালো script লেখক এই প্রতিটি সম্ভাবনা আগেই ভেবে রাখেন — আর সেজন্যই Chapter 12-এ error handling শেখানো হবে। আপাতত পাঁচটি নিয়ম মনে রাখুন:

1. **সব variable quote করুন:** `"$var"` লিখুন, `$var` নয় — ফাঁকা জায়গা বা বিশেষ অক্ষর থাকলে quote ছাড়া variable ভেঙে যায়।
2. **কখনো `eval` ব্যবহার করবেন না** ব্যবহারকারীর দেওয়া লেখার উপর — এটি command injection-এর দরজা খুলে দেয় (Chapter 13)।
3. **Script-এ গোপন তথ্য লিখবেন না** — password, token বা API key কখনো source code-এ নয়, environment variable বা আলাদা configuration ফাইলে রাখুন (Chapter 18)।
4. **ধ্বংসাত্মক কমান্ড চালানোর আগে পরীক্ষা করুন** — `rm` চালানোর আগে `echo` দিয়ে দেখুন কোন ফাইলগুলো মুছবে।
5. **Script-কে Git-এ রাখুন** — Part I-এর Git জ্ঞান এখানেই কাজে লাগে; script-এর প্রতিটি পরিবর্তন commit করুন, যাতে ভুল হলে ফিরে যাওয়া যায়।

৭.১০ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Shell	কমান্ড পড়ে kernel-কে দিয়ে কাজ করানো প্রোগ্রাম
Terminal	Shell-এর সঙ্গে কথা বলার জানালা (পর্দা ও কীবোর্ডের সংযোগ)
Bash	Linux-এর সবচেয়ে প্রচলিত shell (Bourne Again SHell)

Script	কমান্ডের লিখিত তালিকা, যা shell একটানা চালায়
Shebang	Script-এর প্রথম লাইন (#!), যা interpreter ঠিক করে দেয়
Interactive shell	মানুষের সঙ্গে কথা বলে চলা shell (prompt থাকে)
Non-interactive shell	Script ফাইল পড়ে চলা shell (prompt থাকে না)
Permission	কে ফাইল পড়তে, বদলাতে বা চালাতে পারে — সেই নিয়ম
PATH	কমান্ড খোঁজার directory-র তালিকা
Trace mode	bash -x — প্রতিটি কমান্ড চালানোর আগে তা দেখানো

REAL WORLD: চট্টগ্রামের একটি e-commerce company-র সার্ভারে disk ভরে গেলে ওয়েবসাইট বন্ধ হয়ে যেত। একজন engineer একটি দশ লাইনের Bash script লিখলেন — প্রতি ঘণ্টায় disk পরীক্ষা করে, ৮০% ছাড়ালে একটি সতর্ক বার্তা লগে লেখে। সেই script cron-এ বসানো হলো (Chapter 20)। এরপর থেকে disk ভরার আগেই দল জানতে পারে। ছোট script, বড় পার্থক্য — এটাই DevOps automation-এর শুরু।

৭.১১ হাতে-কলমে অনুশীলন

LAB: আপনার প্রথম script — সার্ভারের দৈনিক স্বাস্থ্য পরীক্ষা।

1. একটি কাজের directory বানান: `mkdir -p ~/technova/scripts` আর সেখানে যান: `cd ~/technova/scripts`।
2. `nano daily-check.sh` দিয়ে ফাইল খুলে ৭.৭ অনুচ্ছেদের script-টি লিখুন।
3. `chmod +x daily-check.sh` দিয়ে executable করুন।
4. `./daily-check.sh` দিয়ে চালান, তারপর `bash -x ./daily-check.sh` দিয়ে trace দেখুন।
5. `bash -n daily-check.sh` দিয়ে syntax পরীক্ষা করুন — কোনো output না আসা মানেই syntax ঠিক আছে।
6. Script-টিকে Git-এ commit করুন: `git init` (যদি repository না থাকে), `git add daily-check.sh`, `git commit -m "Add daily server health check script"`।

উদাহরণ আউটপুট (illustrative):

```
$ ./daily-check.sh
TechNova daily check started: 2026-11-08
Disk usage:
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        20G  8.0G   11G  43% /
Daily check finished.
```

যাচাই: `ls -l daily-check.sh`-এ `-rwxr-xr-x` দেখা যাবে — অর্থাৎ execute permission আছে। `bash -x` output-এ প্রতিটি লাইনের আগে `+` দেখা যাবে। `git log --oneline`-এ আপনার commit দেখা যাবে।

৭.১২ Common Mistakes ও Troubleshooting

- **ভুল:** `./script.sh` লিখলে "Permission denied" — কারণ execute permission নেই। সমাধান: `chmod +x script.sh`।
- **ভুল:** `script.sh` লিখলে "command not found" — কারণ বর্তমান directory PATH-এ নেই। সমাধান: `./script.sh` লিখুন।
- **ভুল:** Windows-এ লেখা script Linux-এ চালালে `$'\r': command not found` — কারণ Windows line ending (CRLF) আলাদা। সমাধান: `dos2unix script.sh` বা editor-এ LF line ending ব্যবহার করা।
- **ভুল:** Shebang ভুল লেখা, যেমন `#!/bin/sh` দিয়ে Bash-এর বিশেষ syntax ব্যবহার করা — কিছু Bash বৈশিষ্ট্য sh-এ কাজ করে না। সমাধান: সবসময় `#!/usr/bin/env bash` ব্যবহার করুন।
- **সমস্যা:** Script terminal-এ চলে, cron-এ চলে না — কারণ cron-এর PATH ও পরিবেশ আলাদা। সমাধান: script-এ absolute path ব্যবহার করুন (Chapter 20)।

৭.১৩ Security Considerations

Script-এর নিরাপত্তা শুরু হয় permission থেকে। Script-এ write permission শুধু মালিকের থাকা উচিত — নয়তো অন্য কেউ script বদলে ক্ষতিকর কমান্ড ঢুকিয়ে দিতে পারে, আর আপনি যখন script চালাবেন, তখন সেই কমান্ড আপনার অধিকারে চলবে। একই কারণে `sudo` দিয়ে script চালানোর আগে script-এর প্রতিটি লাইন পড়ুন — root অধিকারে চলা script-এর একটি ভুল লাইন পুরো সিস্টেম নষ্ট করতে পারে। আর কখনো ইন্টারনেট থেকে নামানো script না পড়ে চালাবেন না — `curl ... | bash` ধরনের নির্দেশনা দেখলে প্রথমে ফাইলটি নামিয়ে পড়ুন, তারপর চালান।

৭.১৪ Best Practices

- প্রতিটি script-এর প্রথম লাইনে `#!/usr/bin/env bash` আর দ্বিতীয় লাইনে `set -euo pipefail` লিখুন।
- Script-এর শুরুতে তার উদ্দেশ্য, ব্যবহার আর লেখকের নাম — সংক্ষেপে মন্তব্য হিসেবে লিখুন।
- Script-কে Git repository-তে রাখুন ও প্রতিটি অর্থপূর্ণ পরিবর্তন commit করুন।
- ধ্বংসাত্মক কমান্ড (`rm`, `mv`) প্রথমে `echo` দিয়ে পরীক্ষা করুন।
- Script-এর নাম অর্থবহ রাখুন: `backup.sh` ভালো, `test1.sh` নয়।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Shell-এর প্রধান কাজ কী? ক. Terminal-এর জানালা আঁকা খ. কমান্ড পড়ে kernel-কে দিয়ে কাজ করানো গ. Hard disk ঘোরানো ঘ. Network packet তৈরি করা
2. Shebang-এর কাজ কী? ক. Script-কে রঙিন করা খ. Kernel-কে interpreter জানানো গ. File-এর size বাড়ানো ঘ. Permission বদলানো
3. `chmod 755 script.sh` মানে কী? ক. সবাই সবকিছু করতে পারে খ. মালিক সব পারে, অন্যরা পড়তে ও চালাতে পারে গ. কেউ চালাতে পারে না ঘ. শুধু মালিক পড়তে পারে
4. `bash -x script.sh` কী করে? ক. Script মুছে ফেলে খ. প্রতিটি কমান্ড চালানোর আগে তা দেখায় গ. Script-কে দ্রুত চালায় ঘ. Syntax ঠিক করে দেয়

5. Non-interactive shell কখন চলে? ক. Terminal খুললে খ. Script ফাইল বা cron job চললে গ. কীবোর্ডে টাইপ করলে ঘ. মাউস ক্লিক করলে

৭.১৫ Practical Assignment

Assignment 7.1 (Beginner): TechNova-র জন্য একটি `server-info.sh` script লিখুন যা এই তথ্যগুলো দেখায়: বর্তমান তারিখ ও সময়, hostname, বর্তমান ব্যবহারকারী, disk ব্যবহার (`df -h /`) ও memory ব্যবহার (`free -h`)। Script-টিতে shebang, `set -euo pipefail`, executable permission ও একটি `main` function থাকতে হবে, আর তা Git-এ commit করতে হবে।

৭.১৬ Chapter Summary

Shell হলো মানুষ ও kernel-এর অনুবাদক, terminal হলো তার জানালা, আর Bash হলো Linux-এর মান shell। Script হলো কমান্ডের লিখিত তালিকা — shebang দিয়ে interpreter ঠিক হয়, `chmod +x` দিয়ে চালানোর অধিকার মেলে, আর `bash -x` দিয়ে ভুল খুঁজে পাওয়া যায়। Interactive shell মানুষের সঙ্গে কথা বলে, non-interactive shell script পড়ে চলে — দুটির পরিবেশ আলাদা বলে script-এ কখনো terminal-এর উপর নির্ভর করা যায় না। নিরাপদ scripting-এর ভিত্তি: quote করা, `eval` এড়ানো, গোপন তথ্য code-এ না লেখা, আর প্রতিটি script Git-এ রাখা।

Revision Checklist

- ☐ Shell, terminal ও kernel-এর পার্থক্য ব্যাখ্যা করতে পারি।
- ☐ Shebang লিখে script executable করে চালাতে পারি।
- ☐ 755 permission-এর অর্থ বলতে পারি।
- ☐ Interactive ও non-interactive shell-এর পার্থক্য জানি।
- ☐ `bash -x` ও `bash -n` দিয়ে debug করতে পারি।
- ☐ নিরাপদ scripting-এর পাঁচটি নিয়ম মনে রাখতে পারি।

অধ্যায় ৮ • Part II — Bash Scripting Engineering

Chapter 08: Variables, Data Types and User Input — ভেরিয়েবল, ডেটা ও ব্যবহারকারীর ইনপুট

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র engineer রিয়া একটি backup script লিখলেন। Script-টি প্রতিদিন `/home/riah/backup` directory-তে backup নিত। একদিন তিনি script-টি আরেকটি সার্ভারে চালালেন, যেখানে তার home directory ছিল `/home/ria` নয়, `/srv/ops`। Script ব্যর্থ হলো — কারণ directory-র নাম script-এর ভেতরে শক্ত করে লেখা ছিল (hardcoded)। সেদিন তিনি শিখলেন প্রোগ্রামিংয়ের প্রথম বড় শিক্ষা: যা বদলাতে পারে, তা কখনো শক্ত করে লেখা যায় না — তা একটি নামের (variable) ভেতরে রাখতে হয়, যাতে এক জায়গায় বদলালেই পুরো script বদলে যায়। এই অধ্যায়ে variable-এর সেই শিল্প শেখা হবে।

> **CONCEPT:** Variable হলো নাম দেওয়া একটি বাক্স, যেখানে একটি মান রাখা যায়। বাক্সের নাম একই থাকে, ভেতরের মান বদলায়। Script-এ `BACKUP_DIR="/var/backups"` লিখলে `BACKUP_DIR` হলো বাক্সের নাম, আর `/var/backups` হলো ভেতরের মান।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Variable ঘোষণা, মান দেওয়া ও পড়া — সঠিক syntax-সহ করতে।
- Environment variable ও local variable-এর পার্থক্য ব্যাখ্যা করতে।
- String, সংখ্যা ও array নিয়ে কাজ করতে।
- Command substitution দিয়ে কমান্ডের ফলাফল variable-এ রাখতে।
- Positional parameter (`$1`, `$@`) ও `read` দিয়ে input নিতে।
- Quoting (`" "`, `' '`) ও escaping-এর নিয়ম প্রয়োগ করতে।
- Scope বুঝে variable কোথায় দেখা যায়, তা বলতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07: shebang, permission, `set -euo pipefail` ও script-এর কাঠামো।
- Terminal-এ কমান্ড চালানোর অভিজ্ঞতা।

৮.১ Variable: প্রথম নিয়ম — সমান চিহ্নের পাশে ফাঁকা জায়গা নয়

Bash-এ variable-এ মান দেওয়ার নিয়মটি প্রথম দেখায় অদ্ভুত লাগে: `NAME="Nahid"` ঠিক, কিন্তু `NAME = "Nahid"` ভুল। কেন? কারণ Bash ফাঁকা জায়গা (space) দিয়ে শব্দ আলাদা করে। `NAME = "Nahid"` লিখলে Bash ভাবে আপনি `NAME` নামের একটি কমান্ড চালাচ্ছেন, যার argument হলো `=` ও `"Nahid"`। ফলে "command not found" ভুল আসে। নিয়মটি মনে রাখুন: **assignment-এ `=`-এর দুই পাশে কোনো space থাকবে না।**

Variable পড়তে তার নামের আগে `$` বসাতে হয়: `echo "$NAME"`। `$` ছাড়া `echo NAME` লিখলে Bash শুধু NAME শব্দটি ছাপবে, তার মান নয়। আর নামের সঙ্গে অন্য অক্ষর লেগে থাকলে curly brace ব্যবহার করুন: `${NAME}_backup` — brace ছাড়া `$NAME_backup` লিখলে Bash ভাববে variable-এর নাম `NAME_backup`, যা আলাদা variable। এই ছোট ভুলটি নতুনদের সবচেয়ে বেশি সময় নষ্ট করে।

Variable-এর নামের নিয়ম: অক্ষর, সংখ্যা ও underscore (`_`) ব্যবহার করা যায়; নাম সংখ্যা দিয়ে শুরু হতে পারে না; বড়-ছোট হাতের অক্ষর আলাদা (`NAME` ও `name` দুটি ভিন্ন variable)। প্রচলিত নিয়ম: ধ্রুবক (constant) বড় হাতের অক্ষরে (`BACKUP_DIR`), আর অস্থায়ী variable ছোট হাতের অক্ষরে (`count`) লেখা হয়।

```
#!/usr/bin/env bash
set -euo pipefail

SERVER_NAME="technova-web-01"
BACKUP_DIR="/var/backups/technova"
MAX_RETRIES=3

echo "Server: ${SERVER_NAME}"
echo "Backup directory: ${BACKUP_DIR}"
echo "Max retries: ${MAX_RETRIES}"
```

লাইন ধরে ব্যাখ্যা: প্রথম দুই লাইন আগের অধ্যায়ের মতো। `SERVER_NAME="technova-web-01"` — একটি string মান দেওয়া হচ্ছে; string-এর চারপাশে double quote থাকায় ভেতরের `-` নিরাপদে থাকে। `BACKUP_DIR="/var/backups/technova"` — backup-এর পথ একটি নামে রাখা হলো, যাতে পরে পথ বদলাতে হলে শুধু এই লাইনটি বদলাতে হয়। `MAX_RETRIES=3` — সংখ্যার ক্ষেত্রে quote লাগে না; তবে মনে রাখবেন, Bash-এ সংখ্যাও আসলে string হিসেবেই সংরক্ষিত হয়, গণনার সময় তা সংখ্যায় রূপান্তরিত হয়। `echo "Server: ${SERVER_NAME}"` — double quote-এর ভেতরে `${SERVER_NAME}` তার মানে বদলে যায় (expansion), তাই output-এ `Server: technova-web-01` দেখা যাবে।

৮.২ Environment Variable বনাম Local Variable

সব variable সব জায়গায় দেখা যায় না। **Local variable** শুধু সেই shell-এই থাকে, যেখানে তা তৈরি হয়েছে। আপনি terminal-এ `MY_VAR="hello"` লিখে তারপর `bash` লিখে একটি নতুন shell খুললে, নতুন shell-এ `MY_VAR` পাওয়া যাবে না — কারণ প্রতিটি shell-এর নিজস্ব variable-এর জগৎ আছে। **Environment variable** হলো সেই variable যা `export` দিয়ে চিহ্নিত করা হয়েছে — তা শিশু process (child process)-এর কাছে পৌঁছে যায়। `export MY_VAR="hello"` লিখলে তারপর চালানো যেকোনো প্রোগ্রাম `MY_VAR` দেখতে পাবে।

এর বাস্তব গুরুত্ব: script-এর ভেতরে তৈরি variable ডিফল্টভাবে local — script শেষ হলে তা মুছে যায়, terminal-এ ফিরে আসে না। আর script-কে দেওয়া environment variable script দেখতে পায় — এভাবেই Chapter 18-এ secret (যেমন API token) environment variable হিসেবে দেওয়া হয়, code-এ লেখা হয় না। একটি পরিচিত environment variable হলো `PATH` — কমান্ড খোঁজার directory-র তালিকা। `echo "$PATH"` লিখে দেখুন: কোলন (`:`) দিয়ে আলাদা করা অনেকগুলো directory দেখাবে।

`env` কমান্ড লিখলে বর্তমান environment variable-গুলো দেখা যায়, আর `printenv HOME` লিখলে শুধু একটি variable-এর মান দেখা যায়। `HOME` হলো আপনার home directory, `USER` আপনার ব্যবহারকারীর নাম, `PWD` বর্তমান directory। এই তিনটি প্রায় সব script-এ কাজে লাগে। তবে সতর্কতা: `export` করা variable শুধু নিচের দিকে (child process-এ) যায়, উপরের দিকে (parent shell-এ) ফেরত যায় না — script-এর ভেতরে `export` করলেও terminal-এর shell তা পাবে না।

বিষয়	Local Variable	Environment Variable
তৈরি	`NAME="value"`	`export NAME="value"`
দেখা যায়	শুধু একই shell-এ	সেই shell ও তার সব child process-এ
উদাহরণ	Script-এর ভেতরের অস্থায়ী মান	PATH, HOME, API token
দেখার উপায়	`echo "$NAME"`	`printenv NAME` বা `env`

৮.৩ String: Bash-এ সবকিছুই আসলে লেখা

Bash-এ আলাদা করে integer বা boolean ধরন (data type) নেই — প্রায় সবকিছুই string। `COUNT=3` লিখলে Bash তা "3" নামের string হিসেবে রাখে; গণনার সময় (`$((COUNT + 1))`) তা সংখ্যায় রূপান্তরিত হয়। এই সরলতার সুবিধা আছে: ফাইলের নাম, পথ, বার্তা — সবই একইভাবে লেখা যায়। অসুবিধাও আছে: `COUNT="3"` আর `COUNT="03"` string হিসেবে আলাদা, কিন্তু সংখ্যা হিসেবে একই — তুলনার সময় সতর্ক থাকতে হয় (Chapter 09)।

String জোড়া লাগানো (concatenation) খুব সহজ: পাশাপাশি লিখলেই জোড়া লাগে — `GREETING="Hello ${NAME}"` বা `FULL_PATH="${BACKUP_DIR}/${FILE_NAME}"` । String-এর দৈর্ঘ্য জানতে `${#NAME}` ব্যবহার করুন — `#` এখানে দৈর্ঘ্য বোঝায়। আর string-এর অংশ কাটতে parameter expansion ব্যবহার করা যায়, যেমন `${FILE_NAME%.log}` শেষের `.log` বাদ দেয় — তবে এগুলো Chapter 11-এর text processing-এর সঙ্গে মিলিয়ে শেখা ভালো; আপাতত দৈর্ঘ্য আর জোড়া লাগানোই যথেষ্ট।

৮.৪ সংখ্যার গণনা: `$(( ))`

Bash-এ গণনা করতে **arithmetic expansion** ব্যবহার করুন: `$((3 + 4))` মানে ৭। পুরনো `expr` কমান্ড বা `let` ব্যবহার করার দরকার নেই — `$(( ))` আধুনিক, দ্রুত ও নিরাপদ। ভেতরে variable-এর আগে `$` দেওয়া বাধ্যতামূলক নয়: `$((COUNT + 1))` ও `$((COUNT+1))` দুটোই চলে, তবে পড়ার সুবিধার জন্য space দিয়ে লেখা ভালো।

মনে রাখবেন, Bash শুধু পূর্ণসংখ্যার (integer) গণনা পারে — দশমিক (floating point) পারে না। `$((10 / 3))` দেবে ৩, ৩.৩৩ নয় — কারণ ভাগশেষ ফেলে দেওয়া হয়। দশমিক গণনা লাগলে `bc` বা Python ব্যবহার করুন (Chapter 14)। এটি Bash-এর সীমা, ভুল নয় — Bash একটি glue ভাষা, গণনার ভাষা নয়। Disk ব্যবহারের শতাংশের মতো দশমিক হিসাবের জন্য Chapter 11-এ `awk` ব্যবহার শেখানো হবে।

```
#!/usr/bin/env bash
set -euo pipefail

TOTAL_FILES=120
PROCESSED=45
REMAINING=$((TOTAL_FILES - PROCESSED))
PERCENT=$((PROCESSED * 100 / TOTAL_FILES))

echo "Remaining files: ${REMAINING}"
echo "Progress: ${PERCENT}%"
```

ব্যাখ্যা: `REMAINING=$((TOTAL_FILES - PROCESSED))` — ১২০ থেকে ৪৫ বাদ দিয়ে ৭৫ পাওয়া যায়। `PERCENT=$((PROCESSED * 100 / TOTAL_FILES))` — প্রথমে ৪৫ × ১০০ = ৪৫০০, তারপর ৪৫০০ ÷ ১২০ = ৩৭ (পূর্ণসংখ্যা ভাগ,

ভাগশেষ বাদ)। `echo "Progress: ${PERCENT}%"` — এখানে `%` চিহ্নটি শুধু একটি অক্ষর, Bash-এর কাছে তার কোনো বিশেষ অর্থ নেই, তাই সরাসরি লেখা যায়।

৮.৫ Array: এক নামে অনেক মান

একটি server-এর নাম একটি variable-এ রাখা যায়, কিন্তু পাঁচটি server-এর নাম রাখতে পাঁচটি variable লাগবে — অথবা একটি **array**। Array হলো একটি নামের নিচে সাজানো মানের তালিকা। Bash-এ array তৈরি হয় বন্ধনী দিয়ে: `SERVERS=("web-01" "web-02" "db-01")`। প্রতিটি মানের একটি অবস্থান (index) থাকে, যা ০ থেকে শুরু হয়: `${SERVERS[0]}` মানে `web-01`, `${SERVERS[1]}` মানে `web-02`। সব মান একসঙ্গে পেতে `"${SERVERS[@]}"` লিখুন — `@` মানে "সবগুলো", আর double quote থাকায় ফাঁকা জায়গাযুক্ত মানও ভাঙে না। Array-এর দৈর্ঘ্য (কতটি মান আছে) জানতে `${#SERVERS[@]}`।

Array-এ মান যোগ করতে `SERVERS+=("cache-01")` লিখুন — `+=` মানে আগের তালিকার শেষে যোগ করা। Loop-এর সঙ্গে array মিলিয়ে Chapter 09-এ আমরা প্রতিটি server-এ একই পরীক্ষা চালাব — সেটিই automation-এর মূল প্যাটার্ন: তালিকা + loop = অনেক সার্ভারে একই কাজ।

```
#!/usr/bin/env bash
set -euo pipefail

SERVERS=("technova-web-01" "technova-web-02" "technova-db-01")

echo "First server: ${SERVERS[0]}"
echo "Total servers: ${#SERVERS[@]}"
echo "All servers: ${SERVERS[@]}"
```

ব্যাখ্যা: `SERVERS=(...)` — তিনটি string-এর array। `${SERVERS[0]}` — index ০ মানে প্রথম মান। `${#SERVERS[@]}` — `#` এখানে সংখ্যা গুনছে, ফল ৩। `${SERVERS[@]}` — সব মান একসঙ্গে, space দিয়ে আলাদা। মনে রাখবেন, `${SERVERS}` লিখলে শুধু প্রথম মানটি পাওয়া যায় — `[@]` না লিখলে Bash ধরে নেয় index ০।

৮.৬ Command Substitution: কমান্ডের ফলাফল ধরে রাখা

কখনো কখনো variable-এ একটি নির্দিষ্ট লেখা নয়, একটি কমান্ডের ফলাফল রাখতে হয় — যেমন আজকের তারিখ, বা একটি ফাইলে কত লাইন আছে। এর জন্য **command substitution**: `$(...)`। Bash প্রথমে বন্ধনীর ভেতরের কমান্ডটি চালায়, তারপর তার output variable-এ বসায়। যেমন `TODAY="$(date +%F)"` — `date +%F` দেয় `2026-11-08`, আর তা `TODAY`-তে জমা হয়। পুরনো backtick (`` `date` ``) দিয়েও একই কাজ হয়, কিন্তু backtick-এর ভেতরে আরেকটি backtick লেখা কঠিন, তাই আধুনিক রূপ `$(...)` ব্যবহার করুন।

একটি সতর্কতা: command substitution কমান্ডের output-এর শেষের newline বাদ দিয়ে দেয় — বেশিরভাগ ক্ষেত্রে এটাই চাওয়া হয়। আর output-এ একাধিক লাইন থাকলে সেগুলো একটিই string-এ জমা হয়, newline-সহ — পরে Chapter 11-এ তা লাইন ধরে পড়া শেখানো হবে।

৮.৭ Positional Parameter ও `read`: বাইরে থেকে মান নেওয়া

Script-কে বাইরে থেকে মান দেওয়ার দুটি উপায়। প্রথম: **positional parameter** — script চালানোর সময় তার পরে লেখা শব্দগুলো। `./backup.sh /var/www daily` লিখলে `$1` মানে `/var/www`, `$2` মানে `daily`, `$0` মানে script-এর

নিজের নাম, `$#` মানে কতটি argument দেওয়া হয়েছে, আর `"$@"` মানে সব argument একসঙ্গে (প্রতিটি আলাদাভাবে quote করা)। `$#` দিয়ে argument-এর সংখ্যা পরীক্ষা করা Chapter 10 ও 12-এর input validation-এর ভিত্তি।

দ্বিতীয় উপায়: **`read`** — script চলার সময় ব্যবহারকারীর কাছ থেকে লেখা নেওয়া। `read -r -p "Enter server name: " SERVER` লিখলে script থেমে prompt দেখায়, ব্যবহারকারী লিখে Enter চাপলে তা `SERVER` variable-এ জমা হয়। `-r` flag-টি গুরুত্বপূর্ণ: এটি backslash-কে বিশেষ অক্ষর হিসেবে না দেখে সাধারণ অক্ষর হিসেবে রাখে। `-p` মানে prompt দেখানো। তবে automation-এ `read` কম ব্যবহার করা হয় — কারণ cron-এ চলা script-এর সামনে কোনো মানুষ থাকে না। Production script-এ argument-ই মান, interactive script-এ `read`।

```
#!/usr/bin/env bash
set -euo pipefail

TARGET_DIR="${1:-/var/www}"
MODE="${2:-full}"

echo "Target: ${TARGET_DIR}"
echo "Mode: ${MODE}"
echo "Arguments given: $#"
```

ব্যাখ্যা: `${1:-/var/www}` — একটি নতুন syntax, যার অর্থ "প্রথম argument থাকলে তা ব্যবহার করো, না থাকলে `/var/www` ব্যবহার করো।" `:-` অংশটি default মান দেয় — ব্যবহারকারী argument না দিলেও script চলে। একইভাবে `${2:-full}`। `$#` — argument-এর সংখ্যা। এই default মানের কৌশলটি script-কে ব্যবহারকারীবান্ধব করে: অভিজ্ঞ ব্যবহারকারী argument দিতে পারে, নতুন ব্যবহারকারী না দিলেও script নিরাপদ default নিয়ে চলে।

Variable-এর জীবনচক্র

মান দেওয়া (NAME="value") → পড়া ($NAME / ${NAME}) → ব্যবহার (echo, if, loop) → scope শেষে মুছে যাওয়া

Local variable: শুধু নিজের shell-এ থাকে
Environment variable (export): child process-এও যায় — কিন্তু parent-এ ফেরত যায় না

Input-এর পথ: Positional parameter ($1, $2, $@) ← script চালানোর সময় | read ← চলার সময় কীবোর্ড থেকে | $(...) ← কমান্ডের output থেকে

৮.৮ Quoting ও Escaping: সবচেয়ে গুরুত্বপূর্ণ অধ্যায়ের অংশ

Bash-এ quote-এর তিনটি রূপ, আর তাদের পার্থক্য না জানলে script-এ অদ্ভুত ভুল হয়। **Double quote (`" "`)** — ভেতরের `$variable` ও `$(command)` কাজ করে, কিন্তু space ও বিশেষ অক্ষর নিরাপদ থাকে। **Single quote (`' '`)** — ভেতরের সবকিছু আক্ষরিক (literal): `'$NAME'` লিখলে output হবে `$NAME`, তার মান নয়। **No quote** — Bash space দিয়ে শব্দ ভাগ করে (word splitting) ও `*`-কে ফাইলের নামে বদলে দেয় (globbing) — বেশিরভাগ ক্ষেত্রেই বিপজ্জনক।

নিয়মটি সহজ: **সবসময় variable-কে double quote-এর ভেতরে লিখুন — `"$VAR"`।** ব্যতিক্রম আছে (যেমন `$(( ))`-এর ভেতরে, বা `[[ ]]`-এর ভেতরে), কিন্তু সেগুলো শেখার আগে পর্যন্ত এই নিয়মটি অন্ধভাবে মেনে চলুন। একটি উদাহরণ:

ফাইলের নাম `"my report.txt"` — quote ছাড়া `rm $FILE` লিখলে Bash ভাববে দুটি ফাইল: `my` আর `report.txt` — ভুল ফাইল মুছে যেতে পারে। `rm "$FILE"` লিখলে একটি ফাইলই মুছবে।

Escaping মানে বিশেষ অক্ষরের বিশেষ অর্থ বন্ধ করা: backslash (`\`) দিয়ে। `echo "Price: \$50"` লিখলে `$`-এর আগে backslash থাকায় Bash তাকে variable-এর শুরু ভাববে না — output হবে `Price: $50`। একইভাবে `\"` দিয়ে double quote-এর ভেতরে একটি quote লেখা যায়।

লেখা	NAME="Nahid" হলে ফলাফল	ব্যাখ্যা
`echo "$NAME"`	Nahid	Double quote: expansion হয়
`echo '$NAME'`	$NAME	Single quote: সব আক্ষরিক
`echo $NAME`	Nahid	কাজ করে, কিন্তু space থাকলে ভেঙে যায়
`echo "\$NAME"`	$NAME	Backslash: $ -এর বিশেষ অর্থ বন্ধ

৮.৯ Scope: Variable কোথায় দেখা যায়

Scope মানে variable-এর দৃশ্যমানতার সীমা। Script-এর মূল অংশে তৈরি variable পুরো script-এ দেখা যায়। Function-এর ভেতরে `local` দিয়ে তৈরি variable শুধু সেই function-এই থাকে (Chapter 10-এ বিস্তারিত)। আর child process-এ শুধু `export` করা variable যায়। একটি সাধারণ ভুল: script-এর ভেতরে variable বদলে ভাবা যে terminal-এর shell-ও তা দেখবে — দেখবে না, কারণ script একটি child shell-এ চলে, আর child-এর পরিবর্তন parent-এ ফেরত যায় না। এই নিয়মটি বুঝলে অনেক রহস্যজনক আচরণ পরিষ্কার হয়ে যায়।

৮.১০ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Variable	নাম দেওয়া বাক্স, যেখানে একটি মান রাখা যায়
Assignment	Variable-এ মান দেওয়া (NAME="value")
Expansion	$NAME-কে তার মানে বদলে দেওয়া
Environment variable	export করা variable, child process-এ যায়
Array	এক নামের নিচে সাজানো মানের তালিকা
Command substitution	$(...) — কমান্ডের output variable-এ রাখা
Positional parameter	$1, $2 — script-কে দেওয়া argument
Word splitting	Quote ছাড়া string-কে space দিয়ে ভাগ করা
Quoting	" " বা ' ' দিয়ে বিশেষ অর্থ নিয়ন্ত্রণ করা
Scope	Variable কোথায় দেখা যায়, তার সীমা

REAL WORLD: সিলেটের একটি software firm-এ একজন developer একটি deploy script-এ server-এর IP শক্ত করে লিখেছিলেন। কোম্পানি নতুন সার্ভারে গেলে script-টি পুরনো সার্ভারে deploy করতে থাকল — আর নতুন সার্ভারটি খালি পড়ে রইল। সমাধান ছিল এক লাইন: `SERVER_IP="${1:?Server IP is required}"` — argument হিসেবে IP নেওয়া, আর না দিলে স্পষ্ট ভুল বার্তা দিয়ে থেমে যাওয়া। Variable শুধু সুবিধা নয়, নিরাপত্তাও।

৮.১১ হাতে-কলমে অনুশীলন

LAB: Variable, array ও input নিয়ে একটি server তালিকার script।

1. `~/technova/scripts/server-list.sh` ফাইল বানান।
2. তাতে একটি array-তে তিনটি server-এর নাম, একটি `TODAY="$(date +%F)"` variable, আর `${1:-all}` দিয়ে একটি MODE parameter লিখুন।
3. Script-টি চালান: `./server-list.sh` ও `./server-list.sh web` — দুটি ক্ষেত্রেই output দেখুন।
4. `echo $0` ও `echo "$PATH"` দিয়ে নিজের shell-এর তথ্য দেখুন।
5. ইচ্ছাকৃতভাবে `NAME = "test"` লিখে ভুলটি দেখুন, তারপর ঠিক করে `NAME="test"` লিখুন।

উদাহরণ আউটপুট (illustrative):

```
$ ./server-list.sh
Date: 2026-11-08
Mode: all
Total servers: 3
All servers: technova-web-01 technova-web-02 technova-db-01
```

যাচাই: `./server-list.sh web` চালালে Mode: web দেখাবে। `bash -x` দিয়ে trace করলে প্রতিটি expansion দেখা যাবে।

৮.১২ Common Mistakes ও Troubleshooting

- **ভুল:** `NAME = "value"` — `=`-এর পাশে space। সমাধান: space ছাড়া লিখুন।
- **ভুল:** `echo $FILE` — quote নেই, ফাঁকা জায়গাযুক্ত ফাইলের নাম ভেঙে যায়। সমাধান: সবসময় `"$FILE"`।
- **ভুল:** `${SERVERS}` দিয়ে সব মান আশা করা — এটি শুধু প্রথম মান দেয়। সমাধান: `"${SERVERS[@]}"`।
- **ভুল:** Single quote-এর ভেতরে variable expansion আশা করা। সমাধান: double quote ব্যবহার করুন।
- **ভুল:** `export` না করে script-এর ভেতরের variable child process-এ আশা করা। সমাধান: প্রয়োজনে `export` করুন।
- **সমস্যা:** `$((10 / 3))` দিয়ে ৩.৩৩ আশা করা — Bash শুধু পূর্ণসংখ্যা পারে। সমাধান: `awk` বা Python ব্যবহার করুন।

৮.১৩ Security Considerations

Variable-এর নিরাপত্তা মূলত দুটি জায়গায়: quote করা ও গোপন তথ্য না লেখা। Quote না করলে ফাইলের নামের ফাঁকা জায়গা বা `*` অক্ষর অপ্রত্যাশিত ফাইল মুছে ফেলতে পারে — যেমন `rm $FILE` যেখানে `FILE="*"` হলে বর্তমান

directory-র সব ফাইল মুছে যাবে। আর password বা token কখনো script-এর variable-এ শক্ত করে লিখবেন না — তা Git-এ উঠে যাবে, আর Git-এর ইতিহাস থেকে তা মুছে ফেলা অত্যন্ত কঠিন। এর বদলে environment variable বা Chapter 18-এর `.env` পদ্ধতি ব্যবহার করুন।

৮.১৪ Best Practices

- সব variable double quote করুন: `"$VAR"` — ব্যতিক্রম শেখার আগে পর্যন্ত।
- ধ্রুবকের নাম বড় হাতের অক্ষরে, অস্থায়ী variable ছোট হাতের অক্ষরে লিখুন।
- পথ, URL ও সীমা (limit) — যা বদলাতে পারে — সব variable-এ রাখুন, শক্ত করে লিখবেন না।
- Default মান দিন: `${VAR:-default}` — ব্যবহারকারী কিছু না দিলেও script নিরাপদে চলবে।
- Array-এর সব মান নিতে সবসময় `"${ARR[@]}"` লিখুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `NAME = "Nahid"` ভুল কেন? ক. Bash space দিয়ে শব্দ আলাদা করে খ. Nahid নামটি নিষিদ্ধ গ. Double quote ব্যবহার করা যায় না ঘ. Variable-এ string রাখা যায় না
2. `${SERVERS[@]}` কী দেয়? ক. শুধু প্রথম মান খ. Array-এর সব মান গ. Array-এর দৈর্ঘ্য ঘ. কিছুই না
3. `export`-এর কাজ কী? ক. Variable মুছে ফেলা খ. Variable-কে child process-এ পৌঁছে দেওয়া গ. Variable-কে ফাইলে লেখা ঘ. Variable-এর নাম বদলানো
4. Single quote-এর ভেতরে `$NAME` লিখলে কী হয়? ক. মান দেখায় খ. আক্ষরিক $NAME দেখায় গ. ভুল বার্তা দেয় ঘ. Variable মুছে যায়
5. `$(date +%F)` কী করে? ক. তারিখ মুছে ফেলে খ. date কমান্ডের output variable-এ রাখে গ. ফাইল তৈরি করে ঘ. ক্যালেন্ডার খোলে

৮.১৫ Practical Assignment

Assignment 8.1 (Beginner): একটি `greet.sh` script লিখুন যা প্রথম argument হিসেবে একটি নাম নেয় (না দিলে default "Guest"), একটি array-তে তিনটি শহরের নাম রাখে, আর আজকের তারিখসহ একটি শুভেচ্ছা বার্তা ছাপে। সব variable quote করুন, আর `bash -x` দিয়ে trace যাচাই করুন।

৮.১৬ Chapter Summary

Variable হলো নাম দেওয়া বাক্স — `=`-এর পাশে space নেই, পড়তে `$` লাগে, আর জটিল নামে `${}` লাগে। Local variable শুধু নিজের shell-এ থাকে, environment variable `export` করে child process-এ পাঠানো যায়। Bash-এ সবকিছুই string, গণনা হয় `$(( ))` দিয়ে (শুধু পূর্ণসংখ্যা), অনেক মান রাখতে array, কমান্ডের ফলাফল রাখতে `$(...)`, আর বাইরে থেকে মান নিতে `$1` ও `read`। সবচেয়ে গুরুত্বপূর্ণ নিয়ম: সবসময় `"$VAR"` — quote করা।

Revision Checklist

- ☐ `=`-এর পাশে space না দিয়ে variable-এ মান দিতে পারি।
- ☐ Local ও environment variable-এর পার্থক্য বলতে পারি।

- ☐ Array তৈরি করে `"${ARR[@]}"` দিয়ে সব মান পড়তে পারি।
- ☐ `$(...)` দিয়ে কমান্ডের output রাখতে পারি।
- ☐ `$1`, `$#`, `"$@"` ব্যাখ্যা করতে পারি।
- ☐ Double quote, single quote ও no quote-এর পার্থক্য জানি।

অধ্যায় ৯ • Part II — Bash Scripting Engineering

Chapter 09: Conditions, Loops and Control Flow — শর্ত, লুপ ও নিয়ন্ত্রণ প্রবাহ

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র backup script প্রতিদিন রাত ২টায় চলত। এক রাতে disk-এ জায়গা ছিল মাত্র ৫০০ MB, আর backup-এর দরকার ছিল ২ GB। Script-টি তবুও backup শুরু করল, মাঝপথে disk ভরে গেল, আর অসম্পূর্ণ backup ফাইলটি পুরনো ভালো backup-এর জায়গা দখল করে বসল। পরদিন restore করতে গিয়ে দেখা গেল — কোনো ভালো backup-ই নেই। সমস্যাটি কোডের নয়, সিদ্ধান্তের: script-টি কখনো জিজ্ঞেস করেনি, "জায়গা কি যথেষ্ট?" এই অধ্যায়ে script-কে সিদ্ধান্ত নিতে শেখানো হবে — শর্ত পরীক্ষা করা, পুনরাবৃত্তি করা, আর ভুল অবস্থায় নিরাপদে থেমে যাওয়া।

> **CONCEPT:** Control flow মানে script-এর চলার পথ ঠিক করা। শর্ত (if) বলে "এই অবস্থায় এই কাজটি করো", আর লুপ (for/while) বলে "এই কাজটি বারবার করো, যতক্ষণ শর্তটি সত্যি থাকে"। সিদ্ধান্ত ছাড়া script শুধু একটি তালিকা; সিদ্ধান্ত থাকলে তা একটি প্রোগ্রাম।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- `if`, `elif` ও `else` দিয়ে শর্তভিত্তিক কাজ লিখতে।
- `[[ ]]` দিয়ে string, সংখ্যা ও ফাইলের শর্ত পরীক্ষা করতে।
- `case` দিয়ে একাধিক পছন্দের মধ্যে বাছাই করতে।
- `for`, `while` ও `until` লুপ লিখতে ও তাদের পার্থক্য বলতে।
- `break` ও `continue` দিয়ে লুপ নিয়ন্ত্রণ করতে।
- Disk পরীক্ষা, service পরীক্ষা ও log যাচাইয়ের মতো বাস্তব সিদ্ধান্ত-ভিত্তিক automation লিখতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–08: script কাঠামো, variable, quoting, array ও `$(( ))` গণনা।
- `test` কমান্ডের ধারণা থাকলে ভালো, না থাকলেও চলবে — এখানে শেখানো হবে।

৯.১ `if`: শর্ত সত্যি হলে কাজটি করো

Bash-এ প্রতিটি কমান্ড একটি exit status ফেরত দেয় — ০ মানে সফল, ০ ছাড়া অন্য কিছু মানে ব্যর্থ (Chapter 12-এ বিস্তারিত)। `if` সেই exit status দেখে সিদ্ধান্ত নেয়: শর্তের কমান্ডটি সফল (exit status ০) হলে `then`-এর পরের অংশ চলে, ব্যর্থ হলে `else`-এর অংশ চলে। অর্থাৎ `if` আসলে একটি প্রশ্ন: "এই পরীক্ষাটি কি সফল হলো?"

পরীক্ষা লেখার আধুনিক উপায় হলো `[[ ]]` (double square bracket)। এর ভেতরে শর্ত লেখা হয়। পুরনো `[ ]` (single bracket, অর্থাৎ `test` কমান্ড) এখনো কাজ করে, কিন্তু `[[ ]]`-এ quote-এর নিয়ম সহজ, `&&` ও `||` ব্যবহার করা যায়,

আর pattern matching করা যায়। এই বইয়ে সবসময় `[[ ]]` ব্যবহার করা হবে — ব্যতিক্রম শুধু `#!/bin/sh` script, যেখানে `[[ ]]` নেই।

```
#!/usr/bin/env bash
set -euo pipefail

DISK_USAGE=85

if [[ "${DISK_USAGE}" -ge 80 ]]; then
    echo "Warning: disk is ${DISK_USAGE}% full."
else
    echo "Disk usage is normal: ${DISK_USAGE}%."
fi
```

লাইন ধরে ব্যাখ্যা: `if [[ "${DISK_USAGE}" -ge 80 ]]; then` — `-ge` মানে "greater than or equal" (বড় বা সমান)। ৮৫ বড় বা সমান ৮০, তাই শর্তটি সত্যি। `then` মানে "তাহলে" — তার পরের লাইনটি চলবে। `else` মানে "নইলে" — শর্ত মিথ্যা হলে এই অংশ চলবে। `fi` হলো `if` শব্দটির উল্টো লেখা — এটি `if` ব্লকের শেষ চিহ্নিত করে। Bash-এ অনেক কাঠামো এভাবে উল্টো শব্দে শেষ হয়: `if` শেষ হয় `fi`-তে, `case` শেষ হয় `esac`-তে।

৯.২ তুলনার অপারেটর: সংখ্যা বনাম String

সংখ্যা ও string-এর তুলনা আলাদা — এটি মেলানো সবচেয়ে সাধারণ ভুল। সংখ্যার জন্য ব্যবহার করুন `-eq` (সমান), `-ne` (সমান নয়), `-lt` (ছোট), `-le` (ছোট বা সমান), `-gt` (বড়), `-ge` (বড় বা সমান)। String-এর জন্য ব্যবহার করুন `=` (সমান), `!=` (সমান নয়), `<` (অভিধান অনুযায়ী আগে), `>` (অভিধান অনুযায়ী পরে)। `[[ 10 -gt 9 ]]` সত্যি, কিন্তু `[[ "10" > "9" ]]` মিথ্যা — কারণ string তুলনায় "1" অক্ষরটি "9" অক্ষরের আগে আসে, তাই "10" ছোট ধরা হয়। সংখ্যা তুলনায় সবসময় `-gt`-জাতীয় অপারেটর ব্যবহার করুন।

একাধিক শর্ত জোড়া লাগাতে `&&` (এবং) ও `||` (অথবা) ব্যবহার করুন: `[[ "${AGE}" -ge 18 && "${AGE}" -le 60 ]]` মানে বয়স ১৮ থেকে ৬০-এর মধ্যে। আর `!` দিয়ে শর্ত উল্টানো যায়: `[[ ! -f file.txt ]]` মানে ফাইলটি নেই। শর্তের ভেতরে variable সবসময় quote করুন — `[[ "${NAME}" = "Nahid" ]]` নিরাপদ, `[[ $NAME = Nahid ]]` নয়।

পরীক্ষা	অপারেটর	অর্থ
সংখ্যা	-eq, -ne, -lt, -le, -gt, -ge	সমান, সমান নয়, ছোট, ছোট/সমান, বড়, বড়/সমান
String	=, !=, <, >	সমান, সমান নয়, আগে, পরে (অভিধান ক্রমে)
String খালি কি না	-z, -n	-z: খালি হলে সত্যি; -n: খালি না হলে সত্যি
ফাইল আছে কি না	-e, -f, -d	-e: আছে; -f: সাধারণ ফাইল; -d: directory
ফাইল পড়া/লেখা যায় কি না	-r, -w, -x	read, write, execute permission আছে কি না
ফাইল খালি কি না	-s	-s: ফাইলের size শূন্যের বেশি হলে সত্যি

৯.৩ ফাইল ও Directory-র শর্ত: Automation-এর সবচেয়ে বেশি ব্যবহৃত পরীক্ষা

DevOps script-এ সবচেয়ে বেশি পরীক্ষা করা হয় ফাইল নিয়ে: ফাইলটি আছে কি? এটি directory নাকি সাধারণ ফাইল? পড়া যায় কি? খালি কি না? এই পরীক্ষাগুলো backup, deploy ও health check script-এর প্রাণ। যেমন backup নেওয়ার আগে পরীক্ষা করা উচিত — backup directory-টি আছে কি (`[[ -d "${BACKUP_DIR}" ]]`), আর তাতে লেখা যায় কি (`[[ -w "${BACKUP_DIR}" ]]`)। না থাকলে script-এর উচিত directory বানানো (`mkdir -p`) বা স্পষ্ট ভুল বার্তা দিয়ে থেমে যাওয়া — অন্ধভাবে এগিয়ে গিয়ে মাঝপথে ব্যর্থ হওয়া নয়।

```
#!/usr/bin/env bash
set -euo pipefail

CONFIG_FILE="/etc/technova/app.conf"

if [[ ! -f "${CONFIG_FILE}" ]]; then
    echo "Error: configuration file not found: ${CONFIG_FILE}" >&2
    exit 1
fi

if [[ ! -r "${CONFIG_FILE}" ]]; then
    echo "Error: configuration file is not readable: ${CONFIG_FILE}" >&2
    exit 1
fi

echo "Configuration file is present and readable."
```

ব্যাখ্যা: `[[ ! -f "${CONFIG_FILE}" ]]` — `!` মানে "নয়", `-f` মানে "সাধারণ ফাইল হিসেবে আছে"। ফাইলটি না থাকলে প্রথম `if` সত্যি হয়। `echo ... >&2` — ভুল বার্তা stderr-এ পাঠানো হচ্ছে (Chapter 11-এ বিস্তারিত), যাতে সাধারণ output থেকে ভুল আলাদা থাকে। `exit 1` — script থেমে যাচ্ছে, exit status ১ দিয়ে, যা ব্যর্থতা বোঝায়। এই প্যাটার্নটি — "আগে পরীক্ষা করো, না হলে স্পষ্ট বার্তা দিয়ে থামো" — প্রতিটি ভালো script-এর শুরুতে থাকে।

৯.৪ `elif` ও `case`: একাধিক পছন্দ

দুটির বেশি পছন্দ থাকলে `if`-এর পর `elif` (else-if) জুড়ে দেওয়া যায়। কিন্তু পছন্দ অনেক হলে `case` পরিষ্কার হয়। `case` একটি মানের সঙ্গে একাধিক pattern মেলায় — যেমন ব্যবহারকারীর দেওয়া কমান্ডটি `start`, `stop` নাকি `status`। প্রতিটি pattern-এর শেষে `;;` লিখতে হয়, আর সবশেষে `*)` দিয়ে "অন্য সবকিছু" ধরা হয় — এটি ভুল input ধরার জায়গা।

```
#!/usr/bin/env bash
set -euo pipefail

ACTION="${1:-status}"

case "${ACTION}" in
    start)
        echo "Starting the service..."
        ;;
    stop)
        echo "Stopping the service..."
        ;;
    status)
        echo "Checking service status..."
        ;;
```

```
    *)
        echo "Error: unknown action: ${ACTION}" >&2
        echo "Usage: $0 {start|stop|status}" >&2
        exit 1
        ;;
esac
```

ব্যাখ্যা: `case "${ACTION}" in` — `ACTION`-এর মানের সঙ্গে নিচের pattern-গুলো মেলানো হবে। `start)` — মানটি `start` হলে এই অংশ চলবে। `;;` — এই অংশের শেষ, পরের pattern-এ যাওয়া হবে না। `*)` — `*` মানে যেকোনো কিছু, অর্থাৎ আগের কোনো pattern না মিললে এই অংশ চলবে। `esac` — `case`-এর উল্টো লেখা, ব্লকের শেষ। `$0` — script-এর নিজের নাম, যাতে usage বার্তায় সঠিক নাম দেখায়।

৯.৫ `for` লুপ: তালিকার প্রতিটি জিনিসে একই কাজ

`for` লুপ একটি তালিকার প্রতিটি মানের জন্য একই কাজ বারবার করে। Chapter 08-এর array-এর সঙ্গে মিলিয়ে এটি DevOps-এর সবচেয়ে ব্যবহৃত প্যাটার্ন: প্রতিটি server-এ একই পরীক্ষা, প্রতিটি ফাইলে একই প্রক্রিয়া, প্রতিটি ব্যবহারকারীকে একই বার্তা।

```
#!/usr/bin/env bash
set -euo pipefail

SERVERS=("technova-web-01" "technova-web-02" "technova-db-01")

for SERVER in "${SERVERS[@]}"; do
    echo "Checking ${SERVER}..."
    ping -c 1 -W 2 "${SERVER}" > /dev/null 2>&1 && echo "${SERVER} is reachable." || echo "$
{SERVER} is unreachable."
done
```

ব্যাখ্যা: `for SERVER in "${SERVERS[@]}"; do` — `SERVER` variable-টিতে array-এর প্রতিটি মান একে একে বসবে। `ping -c 1 -W 2 "${SERVER}"` — `-c 1` মানে একটি মাত্র packet পাঠানো, `-W 2` মানে উত্তরের জন্য সর্বোচ্চ ২ সেকেন্ড অপেক্ষা। `> /dev/null 2>&1` — ping-এর output ফেলে দেওয়া হচ্ছে, কারণ আমাদের শুধু সফল/ব্যর্থ জানা দরকার (Chapter 11)। `&& ... || ...` — ping সফল হলে প্রথম `echo`, ব্যর্থ হলে দ্বিতীয় `echo`। `done` — লুপের শেষ। সতর্কতা: এই `&& ||` কৌশলটি `if`-এর সংক্ষিপ্ত রূপ হিসেবে কাজ করে, কিন্তু প্রথম অংশটি ব্যর্থ হলে দ্বিতীয় অংশটি চলার নিয়মটি সবসময় `if`-এর মতো নয় — জটিল শর্তে সবসময় `if` ব্যবহার করুন।

সংখ্যার ক্রমের জন্য C-ধাঁচের লুপও আছে: `for ((i = 1; i <= 5; i++)); do ... done` — এটি ১ থেকে ৫ পর্যন্ত গুনে। আর ফাইলের তালিকার জন্য `for FILE in /var/log/*.log; do ... done` — glob pattern-টি matching ফাইলগুলোতে বদলে যায়। তবে ফাইলের নামে space বা newline থাকলে এই পদ্ধতি ভেঙে যায় — নিরাপদ উপায় Chapter 11-এ শেখানো হবে।

৯.৬ `while` ও `until`: শর্ত যতক্ষণ সত্যি

`while` লুপ চলে যতক্ষণ শর্তটি সত্যি থাকে — কতবার চলবে তা আগে জানা থাকে না। এর সাধারণ ব্যবহার: কোনো service চালু হওয়ার জন্য অপেক্ষা করা, বা একটি ফাইল লাইন ধরে পড়া। `until` হলো তার উল্টো: চলে যতক্ষণ শর্তটি মিথ্যা থাকে, সত্যি হলেই থামে।

```
#!/usr/bin/env bash
set -euo pipefail
```

```
ATTEMPT=1
MAX_ATTEMPTS=5

while [[ "${ATTEMPT}" -le "${MAX_ATTEMPTS}" ]]; do
    echo "Attempt ${ATTEMPT} of ${MAX_ATTEMPTS}..."
    if ping -c 1 -W 2 "technova-db-01" > /dev/null 2>&1; then
        echo "Database server is reachable."
        break
    fi
    ATTEMPT=$((ATTEMPT + 1))
    sleep 2
done

if [[ "${ATTEMPT}" -gt "${MAX_ATTEMPTS}" ]]; then
    echo "Error: database server did not respond after ${MAX_ATTEMPTS} attempts." >&2
    exit 1
fi
```

ব্যাখ্যা: `while [[ "${ATTEMPT}" -le "${MAX_ATTEMPTS}" ]]; do` — চেষ্টার সংখ্যা সর্বোচ্চের মধ্যে থাকলে লুপ চলবে। ভেতরে ping সফল হলে `break` লুপ থেকে বেরিয়ে যায় — আর অপেক্ষা করার দরকার নেই। ব্যর্থ হলে `ATTEMPT=$((ATTEMPT + 1))` সংখ্যা এক বাড়ায়, আর `sleep 2` দুই সেকেন্ড অপেক্ষা করে — অবিরাম চেষ্টা করে সার্ভারকে চাপ দেওয়া হয় না। লুপ শেষে পরীক্ষা করা হয়, চেষ্টা শেষ হয়ে গেছে কি না — হয়ে থাকলে ভুল বার্তা দিয়ে script থামে। এই retry প্যাটার্নটি Chapter 19-এ Python-এ আবার আসবে।

Control Flow-এর মানচিত্র

শুরু → শর্ত পরীক্ষা (if [[...]]) → সত্যি হলে then-এর অংশ, মিথ্যা হলে else-এর অংশ → পরের ধাপ

for লুপ: তালিকার প্রতিটি মান → একই কাজ → পরের মান → তালিকা শেষ হলে বেরিয়ে যাওয়া
while লুপ: শর্ত সত্যি? → হ্যাঁ: কাজটি করো, আবার পরীক্ষা করো | না: লুপ থেকে বেরিয়ে যাও
break: লুপ থেকে এখনই বেরিয়ে যাও | continue: বাকি অংশ বাদ দিয়ে পরের পুনরাবৃত্তিতে যাও

৯.৭ `break` ও `continue`

`break` লুপ থেকে সম্পূর্ণ বেরিয়ে যায় — যেমন উপরের উদাহরণে server পাওয়া গেলে আর চেষ্টা করার দরকার নেই। `continue` বর্তমান পুনরাবৃত্তির বাকি অংশ বাদ দিয়ে পরের পুনরাবৃত্তিতে যায় — যেমন ফাইলের তালিকায় একটি ফাইল না থাকলে সেটি বাদ দিয়ে পরের ফাইলে যাওয়া। দুটির পার্থক্য: `break` মানে "আর নয়", `continue` মানে "এটি বাদ দিয়ে পরেরটি"। একটি সাধারণ ভুল: `break` লিখে ভাবা যে script থেমে যাবে — না, `break` শুধু লুপ থামায়, script চলতে থাকে। Script থামাতে `exit` লাগে।

৯.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Condition	শর্ত — সত্যি বা মিথ্যা হতে পারে এমন পরীক্ষা

Exit status	কমান্ডের ফলাফলের সংখ্যা — ০ মানে সফল
Test expression	[[ ]] — শর্ত লেখার কাঠামো
Loop	একই কাজ বারবার করার কাঠামো
Iteration	লুপের একবার ঘোরা
break / continue	লুপ থেকে বেরিয়ে যাওয়া / পরের পুনরাবৃত্তিতে যাওয়া
case	একটি মানের সঙ্গে একাধিক pattern মেলানো
Retry	ব্যর্থ হলে আবার চেষ্টা করা (সীমিত সংখ্যক বার)

REAL WORLD: রাজশাহীর একটি hosting company-র deploy script কোনো পরীক্ষা ছাড়াই পুরনো version মুছে নতুন version বসাত। একদিন নতুন version-এর ফাইলটি আপলোড অসম্পূর্ণ ছিল — deploy-এর পর ওয়েবসাইট বন্ধ হয়ে গেল, আর ফিরে যাওয়ার পুরনো version-ও নেই। এরপর script-এ তিনটি শর্ত যোগ করা হলো: ফাইলটি আছে কি, তার size শূন্যের বেশি কি, আর backup নেওয়া হয়েছে কি — তিনটি সত্যি হলেই deploy হবে। শর্ত লেখা শুধু প্রোগ্রামিং নয়, ঝুঁকি ব্যবস্থাপনা।

৯.৯ হাতে-কলমে অনুশীলন

LAB: Disk পরীক্ষা করে সিদ্ধান্ত নেওয়া — একটি নিরাপদ backup শর্ত।

1. `~/technova/scripts/disk-guard.sh` ফাইল বানান।
2. তাতে `df` থেকে root filesystem-এর ব্যবহারের শতাংশ বের করুন (Chapter 11-এর `awk` ব্যবহার করে: `df / | awk 'NR==2 {print $5}'` — এখানে `$5` মানে পঞ্চম column, আর `%` চিহ্ন বাদ দিতে `${USAGE%\%}` ব্যবহার করুন)।
3. ৮০%-এর বেশি হলে সতর্ক বার্তা, ৯০%-এর বেশি হলে ভুল বার্তা দিয়ে `exit 1`, নইলে "OK" বার্তা লিখুন।
4. তিনটি অবস্থা পরীক্ষা করুন: স্বাভাবিক, সতর্ক, বিপজ্জনক (শতাংশের মান বদলে পরীক্ষা করুন)।

উদাহরণ আউটপুট (illustrative):

```
$ ./disk-guard.sh
Disk usage: 43%
OK: disk usage is within safe limits.
```

যাচাই: `bash -x ./disk-guard.sh` দিয়ে দেখুন কোন শাখাটি চলছে। ভুল শতাংশ দিলে সঠিক শাখাটি চলছে কি না, তা পরীক্ষা করুন।

৯.১০ Common Mistakes ও Troubleshooting

- **ভুল:** সংখ্যা তুলনায় `>` ব্যবহার করা — string তুলনা হয়, ভুল ফল আসে। সমাধান: `-gt` ব্যবহার করুন।
- **ভুল:** `if [[ $NAME = Nahid ]]` — quote নেই। সমাধান: `[[ "${NAME}" = "Nahid" ]]`।

- **ভুল:** `while` লুপে counter না বাড়ানো — লুপ কখনো থামে না (infinite loop)। সমাধান: লুপের ভেতরে counter বাড়ান, আর সর্বোচ্চ চেষ্টার সীমা রাখুন।
- **ভুল:** `case`-এ `;;` ভুলে যাওয়া — পরের pattern-ও চলতে থাকে। সমাধান: প্রতিটি শাখার শেষে `;;` লিখুন।
- **ভুল:** `[ ]`-এ `&&` ব্যবহার করা — single bracket-এ তা কাজ করে না। সমাধান: `[[ ]]` ব্যবহার করুন, বা `[ ... ] && [ ... ]` লিখুন।
- **সমস্যা:** `if ping ...` লিখলে ping-এর output পর্দায় আসে — সমাধান: `> /dev/null 2>&1` দিয়ে output ফেলে দিন।

৯.১১ Security Considerations

শর্তের নিরাপত্তা মূলত input validation-এ: ব্যবহারকারীর দেওয়া মান কখনো অন্ধভাবে বিশ্বাস করবেন না। যেমন script-টি যদি ফাইলের নাম argument হিসেবে নেয়, তাহলে প্রথমে পরীক্ষা করুন ফাইলটি আছে কি, আর তা প্রত্যাশিত directory-র ভেতরে কি না — নয়তো ব্যবহারকারী `../../etc/passwd`-এর মতো পথ দিয়ে সংবেদনশীল ফাইল পড়তে পারে (path traversal)। আর `case`-এর `*)` শাখাটি সবসময় রাখুন — অচেনা input-কে নিরাপদে প্রত্যাখ্যান করা, অন্ধভাবে চালানো নয়।

৯.১২ Best Practices

- শর্তে সবসময় `[[ ]]` ব্যবহার করুন, `[ ]` নয়।
- সংখ্যার জন্য `-eq/-gt`, string-এর জন্য `=/!=` — মেলাবেন না।
- প্রতিটি `while` লুপে থামার শর্ত ও সর্বোচ্চ সীমা রাখুন।
- Retry-এর মধ্যে `sleep` রাখুন — ব্যর্থ সার্ভারকে অবিরাম চাপ দেবেন না।
- ভুল input-এর জন্য সবসময় স্পষ্ট বার্তা ও `exit 1` রাখুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `[[ 10 -gt 9 ]]` সত্যি, কিন্তু `[[ "10" > "9" ]]` মিথ্যা কেন? ক. 10 আসলে 9-এর চেয়ে ছোট খ. String তুলনায় অক্ষরের ক্রম দেখা হয় গ. > চিহ্নটি Bash-এ নেই ঘ. Double quote ভুল
2. `case`-এ `*)` শাখার কাজ কী? ক. প্রথম শাখা চালানো খ. আগের কোনো pattern না মিললে চলা গ. Script থামানো ঘ. Loop চালানো
3. `break` কী করে? ক. Script থামায় খ. লুপ থেকে বেরিয়ে যায় গ. কম্পিউটার বন্ধ করে ঘ. ফাইল মুছে ফেলে
4. `while` লুপ কখন থামে? ক. একবার চলার পর খ. শর্ত মিথ্যা হলে গ. কখনো থামে না ঘ. Enter চাপলে
5. `[[ -d "/var/backups" ]]` কী পরীক্ষা করে? ক. ফাইলটি খালি কি না খ. directory আছে কি না গ. ফাইলটি পড়া যায় কি না ঘ. Disk ভরা কি না

৯.১৩ Practical Assignment

Assignment 9.1 (Intermediate): একটি `service-check.sh` script লিখুন যা একটি service-এর নাম argument হিসেবে নেয়, `systemctl is-active` দিয়ে পরীক্ষা করে service-টি চলছে কি না, চললে "OK" বার্তা দেয়, না চললে

তিনবার retry করে, তারপরও না চললে ভুল বার্তা দিয়ে `exit 1` করে। `case` দিয়ে `start`, `stop` ও `status` — তিনটি action-ও যোগ করুন।

৯.১৪ Chapter Summary

`if` শর্তের exit status দেখে সিদ্ধান্ত নেয়; `[[ ]]`-এ সংখ্যার জন্য `-eq/-gt` আর string-এর জন্য `=/!=` ব্যবহার করুন; ফাইলের জন্য `-f/-d/-r/-w` পরীক্ষা করুন। অনেক পছন্দের জন্য `case`, তালিকার জন্য `for`, শর্ত যতক্ষণ সত্যি তার জন্য `while` — আর প্রতিটি লুপে থামার সীমা রাখুন। `break` লুপ থামায়, `continue` পরের পুনরাবৃত্তিতে যায়, আর `exit` script থামায়। ভালো script প্রশ্ন করে, তারপর কাজ করে — অন্ধভাবে নয়।

Revision Checklist

- ☐ `[[ ]]` দিয়ে সংখ্যা, string ও ফাইলের শর্ত লিখতে পারি।
- ☐ `if/elif/else` ও `case`-এর পার্থক্য বলতে পারি।
- ☐ `for`, `while` ও `until` লুপ লিখতে পারি।
- ☐ `break`, `continue` ও `exit`-এর পার্থক্য জানি।
- ☐ Retry প্যাটার্ন (সীমা + sleep) লিখতে পারি।
- ☐ ফাইল পরীক্ষা করে নিরাপদে থেমে যাওয়ার প্যাটার্ন জানি।

অধ্যায় ১০ • Part II — Bash Scripting Engineering

Chapter 10: Functions, Arguments and Script Structure — ফাংশন, আর্গুমেন্ট ও স্ক্রিপ্টের কাঠামো

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র একটি backup script ৩০০ লাইনের হয়ে গেল। একই disk পরীক্ষার কোড চার জায়গায়, একই log লেখার কোড পাঁচ জায়গায়। একদিন log-এর format বদলাতে হলো — engineer-কে পাঁচটি জায়গায় একই পরিবর্তন করতে হলো, আর একটি জায়গায় ভুলে গেলেন। ফলে কিছু log নতুন format-এ, কিছু পুরনো format-এ — পরে log পড়তে গিয়ে বিভ্রান্তি। সমস্যাটি হলো পুনরাবৃত্তি (duplication): একই কাজ অনেক জায়গায় লেখা থাকলে প্রতিটি পরিবর্তন অনেক জায়গায় করতে হয়, আর ভুলের সুযোগ বাড়ে। সমাধান **function** — একটি কাজ একবার লেখা, নাম দিয়ে ডাকা, প্রয়োজনে বারবার ব্যবহার করা। এই অধ্যায়ে script-কে ছোট ছোট পুনঃব্যবহারযোগ্য অংশে ভাগ করা শেখা হবে।

> **CONCEPT:** Function হলো নাম দেওয়া কমান্ডের একটি দল। আপনি তাকে একবার সংজ্ঞায়িত করেন, তারপর নাম ধরে ডাকলে তার ভেতরের কমান্ডগুলো চলে। এটি প্রোগ্রামিংয়ের DRY নীতির (Don't Repeat Yourself — নিজেকে পুনরাবৃত্তি কোরো না) ভিত্তি।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Function সংজ্ঞায়িত করে ডাকতে ও তার ভেতরে argument পাঠাতে।
- `return` ও exit status-এর পার্থক্য ব্যাখ্যা করতে।
- `local` variable দিয়ে scope নিয়ন্ত্রণ করতে।
- ছোট function জুড়ে বড় কাজ তৈরি করতে (function composition)।
- একটি script-কে module-এ ভাগ করে পুনঃব্যবহার করতে।
- Command-line argument যাচাই করে সুন্দর usage বার্তা লিখতে।
- Configuration-কে code থেকে আলাদা রাখতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–09: script কাঠামো, variable, quoting, শর্ত ও লুপ।
- Chapter 08-এর `$1`, `$@`, `$#` — এখানে সেগুলো function-এর ভেতরে আবার আসবে।

১০.১ Function: সংজ্ঞা ও ডাক

Bash-এ function লেখার রূপটি সহজ: `name() { ... }` — নাম, তারপর খালি বন্ধনী, তারপর curly brace-এর ভেতরে কমান্ডগুলো। গুরুত্বপূর্ণ বিষয়: function সংজ্ঞায়িত করলেই তা চলে না — শুধু নামটি Bash-এর জানা হয়। চালাতে হলে নামটি লিখে ডাকতে হয়, যেমন একটি কমান্ড ডাকা হয়। আর function ডাকার আগে তা সংজ্ঞায়িত থাকতে হয় — Bash script উপর থেকে নিচে পড়ে, তাই function-এর সংজ্ঞা তার ব্যবহারের আগে লিখুন।

```
#!/usr/bin/env bash
set -euo pipefail

log_info() {
    echo "[INFO] $1"
}

log_error() {
    echo "[ERROR] $1" >&2
}

main() {
    log_info "Daily check started."
    log_info "Checking disk..."
    log_error "Example error message."
    log_info "Daily check finished."
}

main "$@"
```

লাইন ধরে ব্যাখ্যা: `log_info() {` — `log_info` নামের function-এর সংজ্ঞা শুরু। `echo "[INFO] $1"` — `$1` হলো function-কে দেওয়া প্রথম argument। `log_info "Daily check started."` লিখলে `$1`-এর মান হয় `Daily check started.`, আর output হয় `[INFO] Daily check started.`। `log_error`-এ `>&2` থাকায় ভুল বার্তা stderr-এ যায়। `main() { ... }` — মূল কাজের function। শেষ লাইন `main "$@"` script-এর argument-গুলো `main`-কে দিয়ে ডাকে। লক্ষ করুন: function-এর ভেতরের `$1` আর script-এর `$1` আলাদা — function-এর ভেতরে `$1` মানে function-কে দেওয়া argument, script-এর `$1` মানে script-কে দেওয়া argument।

১০.২ Return Status: Function কী ফেরত দেয়

Bash function কোনো মান (value) ফেরত দেয় না — যেমন Python function `return 42` দিয়ে সংখ্যা ফেরত দেয়। Bash function ফেরত দেয় একটি **exit status** — ০ থেকে ২৫৫-এর মধ্যে একটি সংখ্যা, যেখানে ০ মানে সফল। Function-এর ভেতরে শেষ কমান্ডটির exit status-ই function-এর exit status হয়, যদি না আপনি `return` দিয়ে স্পষ্ট করে বলেন। যেমন `return 0` মানে সফল, `return 1` মানে ব্যর্থ।

তাহলে function থেকে একটি string বা সংখ্যা কীভাবে পাওয়া যায়? দুটি উপায়। প্রথম: **command substitution** — function-টি তার ফলাফল `echo` দিয়ে ছাপে, আর ডাকার সময় `RESULT="$(my_function)"` লিখে তা ধরা হয়। দ্বিতীয়: function একটি global variable-এ মান বসায় — তবে এটি কম পরিষ্কার, কারণ function-এর বাইরের অবস্থা বদলে যায়। প্রথম উপায়টিই মান: function output দেয়, ডাকার জায়গা তা ধরে।

```
#!/usr/bin/env bash
set -euo pipefail

is_disk_full() {
    local usage="$1"
    if [[ "${usage}" -ge 90 ]]; then
        return 0
    fi
    return 1
}

main() {
    if is_disk_full 95; then
```

```
        echo "Disk is critically full."
    else
        echo "Disk usage is acceptable."
    fi
}

main "$@"
```

ব্যাখ্যা: `is_disk_full() {` — function-টি একটি সংখ্যা নেয়। `local usage="$1"` — `local` মানে variable-টি শুধু এই function-এর ভেতরেই থাকবে (১০.৩ অনুচ্ছেদ)। `return 0` — শর্ত সত্যি হলে function সফল হিসেবে ফেরত যায়। `return 1` — নইলে ব্যর্থ। `if is_disk_full 95; then` — `if` function-এর exit status দেখে: ০ হলে `then`, ১ হলে `else`। এখানে একটি সূক্ষ্ম বিষয়: function-এর নামটি প্রশ্নের মতো পড়া যায় — "is disk full?" — ভালো function-এর নাম এভাবেই প্রশ্ন বা ক্রিয়া হয়।

> **WARNING:** `return` শুধু function-এর ভেতরে ব্যবহার করা যায়; script-এর মূল অংশে `return` লিখলে ভুল হবে — সেখানে `exit` ব্যবহার করুন। আর `return`-এর মান ০–২৫৫-এর মধ্যে হতে হবে; `return 300` লিখলে তা ৪৪ হয়ে যাবে (২৫৬ দিয়ে ভাগশেষ), যা বিভ্রান্তিকর। বড় সংখ্যা ফেরত দিতে চাইলে `echo` দিয়ে output দিন।

১০.৩ `local` Variable: Function-এর নিজস্ব জগৎ

Function-এর ভেতরে variable তৈরি করলে ডিফল্টভাবে তা global হয় — অর্থাৎ function শেষ হওয়ার পরও script-এর বাকি অংশে তা দেখা যায়। এটি বিপজ্জনক: দুটি function একই নামের variable ব্যবহার করলে একে অপরের মান নষ্ট করে দেবে। সমাধান: function-এর ভেতরে সবসময় `local` ব্যবহার করুন — `local count=0`। `local` variable শুধু সেই function-এই থাকে, বাইরে দেখা যায় না।

একটি নিয়ম: function-এর ভেতরে প্রথমেই তার argument-গুলোকে `local` variable-এ নিয়ে নিন, অর্থবহ নাম দিন। যেমন `local file_path="$1"` — এরপর function-এর ভেতরে `$1` নয়, `"${file_path}"` ব্যবহার করুন। এতে function পড়া সহজ হয়, আর ভুল করে `$2`-এর জায়গায় `$1` লেখার ভুল কমে।

১০.৪ Function Composition: ছোট অংশ জুড়ে বড় কাজ

ভালো script-এ প্রতিটি function একটি মাত্র কাজ করে, আর বড় কাজটি হয় ছোট function-গুলোকে জুড়ে। যেমন একটি health check script-এ থাকতে পারে `check_disk`, `check_memory`, `check_service` — তিনটি ছোট function, আর `main` তাদের একে একে ডাকে। প্রতিটি function আলাদাভাবে পরীক্ষা করা যায়, আর একটি function বদলালে অন্যগুলো নষ্ট হয় না।

```
#!/usr/bin/env bash
set -euo pipefail

check_disk() {
    local usage="$1"
    if [[ "${usage}" -ge 90 ]]; then
        echo "CRITICAL: disk ${usage}% full."
        return 1
    fi
    echo "OK: disk ${usage}% used."
    return 0
}
```

```
check_service() {
    local service_name="$1"
    if systemctl is-active --quiet "${service_name}"; then
        echo "OK: ${service_name} is running."
        return 0
    fi
    echo "CRITICAL: ${service_name} is not running."
    return 1
}

main() {
    local failures=0
    check_disk 43 || failures=$((failures + 1))
    check_service "nginx" || failures=$((failures + 1))
    if [[ "${failures}" -gt 0 ]]; then
        echo "${failures} check(s) failed."
        exit 1
    fi
    echo "All checks passed."
}

main "$@"
```

ব্যাখ্যা: `check_disk 43 || failures=$((failures + 1))` — `||` মানে "বাম দিক ব্যর্থ হলে ডান দিক চালাও"। `check_disk` ব্যর্থ হলে (return 1) `failures` এক বাড়ে। `check_service "nginx"` — `systemctl is-active --quiet` service চললে exit status ০ দেয়, না চললে অন্য কিছু; `--quiet` থাকায় output ছাপে না, শুধু status দেয়। `main`-এ সব check-এর শেষে `failures` গোনা হয় — একটি ব্যর্থ হলেও script exit status ১ দিয়ে থামে। এই প্যাটার্নটি Project 01-এর ভিত্তি।

Script-এর কাঠামো (উপর থেকে নিচে)

Shebang (#!/usr/bin/env bash) → set -euo pipefail → Configuration (variable) → Helper function (log_info, log_error) → Check function (check_disk, check_service) → main() — সবকিছু ডাকে → main "$@" — script শুরু

নিয়ম: সংজ্ঞা আগে, ব্যবহার পরে। main শেষে ডাকা হয়, তাই তার ভেতর থেকে আগের সব function ডাকা যায়।

১০.৫ Modular Structure: Script-কে ভাগ করা

Script বড় হলে তাকে একাধিক ফাইলে ভাগ করা যায়। একটি সাধারণ পদ্ধতি: সাধারণ function-গুলো একটি `lib/common.sh` ফাইলে রাখা, আর মূল script-এ `source` দিয়ে তা আনা। `source lib/common.sh` (বা সংক্ষেপে `. lib/common.sh`) লিখলে Bash সেই ফাইলটি বর্তমান shell-এই পড়ে — যেন তার বিষয়বস্তু এখানেই লেখা ছিল। `bash lib/common.sh` দিয়ে চালালে তা আলাদা process হতো, আর তার function-গুলো বর্তমান script-এ পাওয়া যেত না — তাই `source` ব্যবহার করুন।

তবে `source` করার সময় পথের বিষয়ে সতর্ক থাকুন: script-টি যে directory থেকে চালানো হচ্ছে, তার উপর নির্ভর করে আপেক্ষিক পথ (relative path) বদলে যায়। নিরাপদ উপায়: script-এর নিজের directory বের করে তা ব্যবহার করা — `SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"`। এই লাইনটি প্রথম দেখায় জটিল লাগে: `$`

`{BASH_SOURCE[0]}` মানে বর্তমান script ফাইলের নাম, `dirname` তার directory বের করে, `cd` সেখানে গিয়ে `pwd` পুরো পথ দেয়। Chapter 13-এর toolkit-এ এই প্যাটার্নটি ব্যবহার করা হবে।

১০.৬ Command-line Argument Handling

একটি ভালো CLI (command-line interface) তিনটি জিনিস করে: argument-এর সংখ্যা যাচাই করে, ভুল হলে usage দেখায়, আর `--help` দিলে নিজের ব্যবহার বুঝিয়ে দেয়। Chapter 08-এর `$#` দিয়ে সংখ্যা যাচাই করা যায়; আর `case` দিয়ে `--help` ধরা যায়।

```
#!/usr/bin/env bash
set -euo pipefail

usage() {
    echo "Usage: $0 <server-name> [--verbose]"
    echo "Example: $0 technova-web-01"
}

main() {
    if [[ "$#" -lt 1 ]]; then
        usage >&2
        exit 1
    fi
    local server_name="$1"
    echo "Checking server: ${server_name}"
}

main "$@"
```

ব্যাখ্যা: `usage()` — ব্যবহারের নির্দেশনা ছাপার function, যাতে usage-এর লেখা এক জায়গায় থাকে। `if [[ "$#" -lt 1 ]]; then` — argument একটিও না থাকলে usage দেখিয়ে `exit 1`। `usage >&2` — ভুলের বার্তা stderr-এ যায়, যাতে সাধারণ output থেকে আলাদা থাকে। `local server_name="$1"` — প্রথম argument-কে অর্থবহ নাম দেওয়া হলো। আরও জটিল argument (যেমন `--verbose`) পার্স করতে `getopts` ব্যবহার করা যায় — Chapter 22-এ বিস্তারিত আসবে।

১০.৭ Configuration: Code থেকে আলাদা রাখা

Script-এর ভেতরে পথ, server-এর নাম, সীমা — এগুলো configuration। এগুলো script-এর উপরে variable হিসেবে এক জায়গায় রাখুন, যাতে বদলাতে হলে পুরো script পড়তে না হয়। আরও ভালো: configuration আলাদা ফাইলে রাখুন (যেমন `config.env`), আর script তা `source` করে নেবে — Chapter 18-এ এই পদ্ধতি বিস্তারিত আসবে। নিয়মটি মনে রাখুন: **যা পরিবেশভেদে বদলায়, তা code-এ শক্ত করে লিখবেন না।** TechNova-র Dhaka সার্ভার আর Chattogram সার্ভারের পথ আলাদা হলে একই script দুই জায়গায় চলবে — শুধু configuration বদলে।

১০.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Function	নাম দেওয়া কমান্ডের দল, বারবার ডাকা যায়
Argument	Function বা script-কে দেওয়া মান

Return status	Function-এর ফলাফলের সংখ্যা (০ = সফল)
local	শুধু function-এর ভেতরে থাকা variable
Composition	ছোট function জুড়ে বড় কাজ তৈরি করা
source	আরেকটি script ফাইল বর্তমান shell-এ পড়া
DRY	Don't Repeat Yourself — একই কোড বারবার না লেখা
CLI	Command-line interface — কমান্ড লাইনে ব্যবহারের নিয়ম

REAL WORLD: খুলনার একটি company-র deploy script-এ log লেখার কোড সাত জায়গায় ছিল। Log format বদলানোর সময় একটি জায়গা বাদ পড়ল — সেই অংশের log-গুলো পরে খুঁজে পাওয়া গেল না। এরপর log লেখা একটি `log_info` function-এ নেওয়া হলো, আর সাতটি জায়গা থেকে সেই function ডাকা হলো। পরের বার format বদলাতে হলো এক লাইন। Function শুধু সুবিধা নয় — রক্ষণাবেক্ষণের (maintenance) ভিত্তি।

১০.৯ হাতে-কলমে অনুশীলন

LAB: Function দিয়ে health check script-কে নতুন করে সাজানো।

1. Chapter 09-এর `disk-guard.sh` খুলুন।
2. Disk পরীক্ষার অংশটি একটি `check_disk` function-এ নিন, আর log লেখার জন্য `log_info` ও `log_error` function যোগ করুন।
3. একটি `check_memory` function যোগ করুন: `free` থেকে memory ব্যবহারের শতাংশ বের করে ৯০%-এর বেশি হলে সতর্ক করুন।
4. `main` function থেকে দুটি check ডাকুন, আর ব্যর্থ check-এর সংখ্যা গুনে শেষে সিদ্ধান্ত নিন।
5. `bash -x` দিয়ে trace করে দেখুন function-গুলো কোন ক্রমে ডাকা হচ্ছে।

উদাহরণ আউটপুট (illustrative):

```
$ ./health-check.sh
[INFO] Checking disk...
OK: disk 43% used.
[INFO] Checking memory...
OK: memory 61% used.
All checks passed.
```

যাচাই: `shellcheck health-check.sh` (install করা থাকলে) চালিয়ে দেখুন কোনো সতর্কতা আসে কি না — Chapter 13-এ ShellCheck বিস্তারিত আসবে।

১০.১০ Common Mistakes ও Troubleshooting

- **ভুল:** Function ব্যবহারের আগে সংজ্ঞায়িত না করা — "command not found" ভুল আসে। সমাধান: সংজ্ঞা উপরে, ব্যবহার নিচে।

- **ভুল:** Function-এর ভেতরে `$1` বলতে script-এর argument ভাবা — function-এর ভেতরে `$1` মানে function-এর argument। সমাধান: argument-কে `local` নামে নিয়ে নিন।
- **ভুল:** `local` না ব্যবহার করে দুটি function-এ একই variable নাম ব্যবহার করা — একে অপরের মান নষ্ট করে। সমাধান: সবসময় `local`।
- **ভুল:** Function থেকে string ফেরত দিতে `return "hello"` লেখা — `return` শুধু সংখ্যা নেয়। সমাধান: `echo` দিয়ে output দিন, আর `$(...)` দিয়ে ধরুন।
- **ভুল:** `main` ডাকতে ভুলে যাওয়া — script চলে কিন্তু কিছুই হয় না। সমাধান: শেষ লাইনে `main "$@"` আছে কি না, তা পরীক্ষা করুন।

১০.১১ Security Considerations

Function-এর নিরাপত্তা মূলত argument যাচাইয়ে: function-কে দেওয়া মান কখনো অন্ধভাবে কমান্ডে বসাবেন না। যেমন `check_service "$1"` লিখলে `$1`-এর মানটি `systemctl`-এর argument হিসেবে যাবে — কিন্তু যদি `$1`-এর মান হয় `nginx; rm -rf /`, তাহলে কী হবে? Quote করা থাকলে (`"${service_name}"`) তা একটি মাত্র argument হিসেবে যাবে, আর `systemctl` তাকে একটি service-এর নাম হিসেবে দেখবে — কমান্ড হিসেবে চালাবে না। Quote করা এখানেও রক্ষা করে। আর function-এর নাম ও argument-এর নাম অর্থবহ রাখুন — অস্পষ্ট নাম ভুল ডাকার কারণ হয়।

১০.১২ Best Practices

- প্রতিটি function একটি মাত্র কাজ করবে, আর তার নাম সেই কাজটি বলবে।
- Function-এর ভেতরের সব variable `local` করুন।
- Argument-কে প্রথমেই অর্থবহ `local` নামে নিয়ে নিন।
- সাধারণ function-গুলো `lib/` ফাইলে রেখে `source` করুন।
- প্রতিটি script-এ `usage` function ও argument যাচাই রাখুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Function সংজ্ঞায়িত করলেই কি তা চলে? ক. হ্যাঁ, সঙ্গে সঙ্গে চলে খ. না, নাম ধরে ডাকতে হয় গ. শুধু main function চলে ঘ. cron-এ চললে চলে
2. Bash function কী ফেরত দেয়? ক. একটি string খ. একটি exit status (০–২৫৫) গ. একটি array ঘ. একটি ফাইল
3. `local`-এর কাজ কী? ক. Variable মুছে ফেলা খ. Variable-কে শুধু function-এর ভেতরে রাখা গ. Variable-কে ফাইলে লেখা ঘ. Variable export করা
4. Function থেকে string পেতে কী করতে হয়? ক. return "hello" লেখা খ. echo দিয়ে output দেওয়া ও $(...) দিয়ে ধরা গ. exit ব্যবহার করা ঘ. global variable-এ লেখা ছাড়া উপায় নেই
5. `source lib/common.sh` কী করে? ক. ফাইলটি আলাদা process-এ চালায় খ. ফাইলটি বর্তমান shell-এ পড়ে গ. ফাইলটি মুছে ফেলে ঘ. ফাইলটি download করে

১০.১৩ Practical Assignment

Assignment 10.1 (Intermediate): একটি `lib.sh` ফাইলে `log_info`, `log_error` ও `check_file_exists` function লিখুন, আর একটি মূল script থেকে তা `source` করে ব্যবহার করুন। মূল script-টি একটি ফাইলের নাম argument হিসেবে নেবে, ফাইলটি আছে কি না পরীক্ষা করবে, আর ফলাফল log function দিয়ে জানাবে।

১০.১৪ Chapter Summary

Function হলো নাম দেওয়া কমান্ডের দল — একবার লিখে বারবার ডাকা যায়। Function ফেরত দেয় exit status, string পেতে `echo` ও `$(...)` ব্যবহার করুন। Function-এর ভেতরের variable সবসময় `local` করুন, argument-কে অর্থবহ নাম দিন, আর ছোট function জুড়ে বড় কাজ তৈরি করুন। সাধারণ function `lib/`-এ রেখে `source` করুন, configuration code থেকে আলাদা রাখুন, আর প্রতিটি script-এ `usage` ও argument যাচাই রাখুন।

Revision Checklist

- ☐ Function সংজ্ঞায়িত করে ডাকতে পারি।
- ☐ `return` ও `exit`-এর পার্থক্য বলতে পারি।
- ☐ Function থেকে `echo` ও `$(...)` দিয়ে মান নিতে পারি।
- ☐ `local` variable ব্যবহার করতে পারি।
- ☐ `source` দিয়ে module ব্যবহার করতে পারি।
- ☐ Argument যাচাই ও `usage` লিখতে পারি।

অধ্যায় ১১ • Part II — Bash Scripting Engineering

Chapter 11: Files, Text Processing and Command Pipelines — ফাইল, টেক্সট প্রসেসিং ও পাইপলাইন

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র একটি web server প্রতিদিন প্রায় ২ GB log ফাইল তৈরি করে। একদিন একটি API হঠাৎ ধীর হয়ে গেল। Engineer-কে জানতে হলো: কোন IP থেকে সবচেয়ে বেশি request আসছে? কোন URL-এ সবচেয়ে বেশি ভুল (500 error) হচ্ছে? Log ফাইলটি হাতে পড়া অসম্ভব — তাতে কয়েক লাখ লাইন। কিন্তু পাঁচটি কমান্ডের একটি pipeline দশ সেকেন্ডে উত্তর দিল: `grep` দিয়ে ভুলের লাইনগুলো বাছা, `awk` দিয়ে IP বের করা, `sort` ও `uniq` দিয়ে গোনা। এই অধ্যায়ে Unix-এর সেই বিখ্যাত দর্শন শেখা হবে: ছোট ছোট টুল, প্রতিটি একটি কাজে দক্ষ, আর pipe দিয়ে তাদের জুড়ে বড় কাজ করা।

> **CONCEPT:** Unix দর্শন: একটি টুল একটি কাজ করবে, আর তা ভালোভাবে করবে। টুলগুলো text stream দিয়ে কথা বলে — একটির output পরেরটির input হয়। এই জোড়া লাগানোর ব্যবস্থার নাম pipeline, আর তার প্রতীক `|`।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- stdin, stdout ও stderr-এর পার্থক্য ব্যাখ্যা করে redirection ব্যবহার করতে।
- Pipe (`|`) দিয়ে কমান্ড জুড়ে text প্রক্রিয়া করতে।
- `grep`, `sed`, `awk`, `cut`, `sort`, `uniq` দিয়ে log বিশ্লেষণ করতে।
- `find` ও `xargs` দিয়ে ফাইল খুঁজে কাজ করতে।
- `while read` দিয়ে ফাইল লাইন ধরে নিরাপদে পড়তে।
- ফাঁকা জায়গা ও newline-যুক্ত ফাইলের নাম নিরাপদে সামলাতে (`-print0` ও `-0`)।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–10: script কাঠামো, variable, শর্ত, লুপ ও function।
- Log ফাইল কী, তা জানা থাকলে ভালো — না জানলেও এই অধ্যায়ে উদাহরণসহ শেখানো হবে।

১১.১ তিনটি Stream: stdin, stdout, stderr

প্রতিটি Linux প্রোগ্রামের তিনটি stream থাকে। **stdin (0)** — input আসার পথ (সাধারণত কীবোর্ড)। **stdout (1)** — সাধারণ output যাওয়ার পথ (সাধারণত পর্দা)। **stderr (2)** — ভুল বার্তা যাওয়ার পথ (সাধারণত পর্দা, কিন্তু stdout থেকে আলাদা)। সংখ্যাগুলো মনে রাখুন: ০, ১, ২ — redirection-এ এগুলোই ব্যবহার হয়। দুটি stream আলাদা রাখার কারণ: script-এর সাধারণ output একটি ফাইলে গেলেও ভুল বার্তা পর্দায় দেখা যায় — অথবা উল্টোটা। Chapter 09-এ `>&2` দিয়ে ভুল বার্তা stderr-এ পাঠানো হয়েছে — এখন তার অর্থ পরিষ্কার।

Redirection মানে stream-এর গন্তব্য বদলানো। `command > file.txt` — stdout ফাইলে যায় (ফাইলটি আগে থাকলে তা মুছে নতুন করে লেখা হয়)। `command >> file.txt` — stdout ফাইলের শেষে যোগ হয় (আগের লেখা থাকে)।

`command 2> error.log` — stderr ফাইলে যায়। `command > out.log 2>&1` — stdout ও stderr দুটোই একই ফাইলে যায়। আর `command < input.txt` — ফাইল থেকে stdin নেওয়া হয়। সবচেয়ে বিপজ্জনক redirection হলো `>` — ভুল ফাইলের নাম লিখলে একটি মূল্যবান ফাইল মুছে যেতে পারে। তাই `>` লেখার আগে ফাইলের নাম দুবার দেখুন, আর log-এর জন্য `>>` ব্যবহার করুন।

লেখা	অর্থ	সতর্কতা
`> file`	stdout ফাইলে (আগের লেখা মুছে যায়)	ভুল নামে মূল্যবান ফাইল মুছতে পারে
`>> file`	stdout ফাইলের শেষে যোগ হয়	Log-এর জন্য নিরাপদ
`2> file`	stderr ফাইলে যায়	ভুল বার্তা আলাদা রাখা যায়
`> file 2>&1`	stdout ও stderr একই ফাইলে	ক্রমটি গুরুত্বপূর্ণ: প্রথমে >, তারপর 2>&1
`< file`	ফাইল থেকে input নেওয়া	—
`/dev/null`	output ফেলে দেওয়ার জায়গা	ফেলে দেওয়া output আর ফেরত আসে না

১১.২ Pipe: একটির Output, পরেরটির Input

Pipe (`|`) হলো redirection-এর সবচেয়ে শক্তিশালী রূপ: একটি কমান্ডের stdout পরের কমান্ডের stdin হয়ে যায় — মাঝখানে কোনো ফাইল লাগে না। যেমন `cat access.log | grep "500" | wc -l` — প্রথমে log ফাইল পড়া, তারপর "500" লেখা লাইনগুলো বাছা, তারপর লাইন গোনা। প্রতিটি কমান্ড একটি মাত্র কাজ করে, আর pipe তাদের জোড়ে। এই পদ্ধতির সুবিধা: মাঝখানের ফাইল বানাতে হয় না, memory-তে stream হয়ে যায়, আর প্রতিটি ধাপ আলাদাভাবে পরীক্ষা করা যায় — pipe-এর আগের অংশটি চালিয়ে দেখুন, তারপর পরের অংশ যোগ করুন।

Pipe-এর একটি সূক্ষ্ম বিপদ আছে: pipe-এর ডানদিকের কমান্ডটি একটি subshell-এ চলে — তার ভেতরে variable বদলালে তা বাইরে দেখা যায় না। যেমন `cat file.txt | while read LINE; do COUNT=$((COUNT + 1)); done; echo "${COUNT}"` লিখলে `COUNT`-এর মান বাইরে শূন্যই থাকবে। সমাধান: process substitution ব্যবহার করা — `while read LINE; do ... done < file.txt` — এখানে লুপটি বর্তমান shell-এই চলে। অথবা Bash-এ `lastpipe` option চালু করা যায়, তবে redirection পদ্ধতিটিই সবচেয়ে নিরাপদ।

Pipeline-এর প্রবাহ

access.log → grep "500" (ভুলের লাইন বাছাই) → awk '{print $1}' (IP বের করা) → sort (সাজানো) → uniq -c (গোনা) → sort -rn (বেশি থেকে কম সাজানো) → head -10 (প্রথম ১০টি)

প্রতিটি ধাপ একটি text stream নেয়, কিছু করে, একটি text stream দেয়। মাঝখানে কোনো ফাইল নেই।

১১.৩ `grep`: লাইন খোঁজা

`grep` হলো text খোঁজার টুল: ফাইলে বা stream-এ একটি pattern মেলে এমন লাইনগুলো দেখায়। `grep "ERROR" app.log` — app.log-এ ERROR লেখা লাইনগুলো দেখাবে। দরকারি flag: `-i` (বড়-ছোট হাতের পার্থক্য না করা), `-v`

(যে লাইনে pattern নেই, সেগুলো দেখানো), `-n` (লাইন নম্বরসহ দেখানো), `-c` (কতটি লাইন মিলেছে, তা গোনা), `-r` (directory-র সব ফাইলে খোঁজা), `-E` (বর্ধিত regular expression)।

Regular expression (regex) হলো pattern লেখার ভাষা — যেমন `^ERROR` মানে লাইনের শুরুতে ERROR, `500$` মানে লাইনের শেষে 500, `[0-9]+` মানে এক বা একাধিক সংখ্যা। Regex শক্তিশালী, কিন্তু ভুল regex ভুল ফল দেয় — তাই জটিল pattern প্রথমে ছোট ফাইলে পরীক্ষা করুন। আর মনে রাখবেন: `grep` দিয়ে ফাইল খুঁজতে গিয়ে `grep "pattern" *` লিখলে ফাইলের নামে space থাকলে সমস্যা হতে পারে — বড় কাজে `grep -r` ব্যবহার করুন।

১১.৪ `sed` ও `awk`: বদলানো ও বিশ্লেষণ

`sed` (stream editor) হলো text বদলানোর টুল। সবচেয়ে বেশি ব্যবহৃত রূপ: `sed 's/old/new/' file.txt` — প্রতিটি লাইনে প্রথম `old`-কে `new` দিয়ে বদলায়। সবগুলো বদলাতে `s/old/new/g` — `g` মানে global। ফাইল সরাসরি বদলাতে `sed -i` ব্যবহার করা হয় — কিন্তু সতর্কতা: `-i` দিয়ে ভুল বদলালে মূল ফাইলটি নষ্ট হয়ে যায়। তাই প্রথমে `-i` ছাড়া পরীক্ষা করুন, তারপর backup নিয়ে (`sed -i.bak` — এটি `.bak` নামে backup রাখে) বদলান।

`awk` হলো column-ভিত্তিক text প্রক্রিয়াকরণের টুল — log বিশ্লেষণের রাজা। `awk` প্রতিটি লাইনকে field-এ ভাগ করে (ডিফল্টভাবে space দিয়ে), আর `$1`, `$2` দিয়ে field-গুলো পাওয়া যায়। যেমন Apache log-এর প্রথম field হলো IP: `awk '{print $1}' access.log` সব IP দেখাবে। `awk '{print $1, $9}'` IP ও status code দেখাবে। আর `awk '$9 == 500 {print $1}'` শুধু 500 error-এর IP দেখাবে। `NR` মানে লাইন নম্বর, `NF` মানে field-এর সংখ্যা।

```
#!/usr/bin/env bash
set -euo pipefail

LOG_FILE="/var/log/nginx/access.log"

echo "Top 10 IP addresses by request count:"
awk '{print $1}' "${LOG_FILE}" | sort | uniq -c | sort -rn | head -10

echo "Count of HTTP 500 errors:"
awk '$9 == 500' "${LOG_FILE}" | wc -l
```

ব্যাখ্যা: প্রথম pipeline — `awk '{print $1}'` প্রতিটি লাইন থেকে প্রথম field (IP) বের করে। `sort` IP-গুলো সাজায় — কারণ `uniq` শুধু পাশাপাশি থাকা একই লাইন গুনতে পারে, তাই সাজানো আবশ্যক। `uniq -c` প্রতিটি IP কতবার এসেছে, তা গুনে সংখ্যাসহ দেখায়। `sort -rn` সংখ্যা অনুযায়ী উল্টো ক্রমে (বেশি থেকে কম) সাজায় — `-r` মানে reverse, `-n` মানে numeric। `head -10` প্রথম দশটি লাইন রাখে। দ্বিতীয় অংশে `awk '$9 == 500'` শুধু নবম field 500 হলে লাইনটি রাখে, আর `wc -l` লাইন গুনে।

১১.৫ `cut`, `sort`, `uniq`, `tr`

`cut` — নির্দিষ্ট column কাটা: `cut -d: -f1 /etc/passwd` — `:` দিয়ে ভাগ করে প্রথম field (ব্যবহারকারীর নাম) দেখায়। `-d` মানে delimiter, `-f` মানে field। `sort` — লাইন সাজানো: `-n` সংখ্যা হিসেবে, `-r` উল্টো, `-u` একই লাইন একবার রাখা। `uniq` — পাশাপাশি একই লাইন এক করা: `-c` গোনাসহ, `-d` শুধু যেগুলো একাধিকবার এসেছে। `tr` — অক্ষর বদলানো: `tr 'a-z' 'A-Z'` ছোট হাতের অক্ষরকে বড় হাতের করে। এই চারটি টুল `awk`-এর চেয়ে সহজ, আর সহজ কাজে এগুলোই ব্যবহার করা ভালো — জটিল কাজে `awk`।

১১.৬ `find` ও `xargs`: ফাইল খুঁজে কাজ করা

`find` directory-তে ফাইল খোঁজে: `find /var/log -name "*.log" -mtime +7` — `/var/log`-এ ৭ দিনের পুরনো `.log` ফাইল খুঁজে। `-name` নামের pattern, `-type f` শুধু ফাইল, `-type d` শুধু directory, `-size +100M` ১০০ MB-এর বড় ফাইল, `-mtime +7` ৭ দিনের বেশি আগে বদলানো ফাইল। `find`-এর নিজের `-exec` আছে, কিন্তু `xargs`-এর সঙ্গে জুড়ে কাজ করা বেশি নমনীয়: `find ... -print0 | xargs -0 rm` — এখানে `-print0` ও `-0` ফাইলের নামের মধ্যে newline-এর বদলে null অক্ষর ব্যবহার করে, তাই space বা newline-যুক্ত নামও নিরাপদে যায়।

এই `-print0 | xargs -0` জোড়াটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা প্যাটার্ন। `find ... | xargs rm` (null ছাড়া) লিখলে ফাইলের নামে space থাকলে তা দুটি ফাইল ভেবে ভুল ফাইল মুছে ফেলতে পারে। আর `find ... -exec rm {} \;` লিখলে প্রতিটি ফাইলের জন্য আলাদা `rm` চলে — ধীর, কিন্তু নিরাপদ। বড় কাজে `-print0 | xargs -0` ব্যবহার করুন।

```
#!/usr/bin/env bash
set -euo pipefail

LOG_DIR="/var/log/technova"

find "${LOG_DIR}" -type f -name "*.log" -mtime +30 -print0 |
    xargs -0 -r ls -lh
```

ব্যাখ্যা: `find "${LOG_DIR}" -type f -name "*.log" -mtime +30` — ৩০ দিনের পুরনো log ফাইল খুঁজে। `-print0` — ফাইলের নাম null অক্ষর দিয়ে আলাদা করে। `xargs -0 -r ls -lh` — `-0` মানে null দিয়ে আলাদা নাম পড়া, `-r` মানে কোনো ফাইল না পেলে `ls` চালানো হবে না (নয়তো খালি `ls` বর্তমান directory-র সব ফাইল দেখাত — বিপজ্জনক ভুল)। `ls -lh` ফাইলগুলোর আকারসহ তালিকা দেখায়।

১১.৭ ফাইল লাইন ধরে পড়া: `while read`

একটি ফাইল লাইন ধরে পড়ার নিরাপদ উপায়:

```
#!/usr/bin/env bash
set -euo pipefail

while IFS= read -r line || [[ -n "${line}" ]]; do
    echo "Line: ${line}"
done < "/etc/technova/servers.txt"
```

ব্যাখ্যা: `IFS=` — field separator খালি করা, যাতে লাইনের শুরু ও শেষের space কাটা না যায়। `read -r line` — `-r` থাকায় backslash সাধারণ অক্ষর হিসেবে থাকে। `|| [[ -n "${line}" ]]` — ফাইলের শেষ লাইনে newline না থাকলেও সেটি পড়া হয় (নয়তো শেষ লাইনটি বাদ পড়ত)। `done < file` — ফাইল থেকে redirection, যাতে লুপটি বর্তমান shell-এ চলে (১১.২-এর pipe সমস্যা এড়ানো যায়)। এই প্যাটার্নটি মুখস্থ রাখুন — ফাইল পড়ার এটাই মান পদ্ধতি।

১১.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
stdin / stdout / stderr	Input, সাধারণ output ও ভুল বার্তার stream (০, ১, ২)

Redirection	Stream-এর গন্তব্য বদলানো (>, >>, 2>, <)
Pipeline	| দিয়ে কমান্ড জুড়ে stream পাঠানো
grep	Pattern মেলে এমন লাইন খোঁজা
sed	Stream editor — text বদলানো
awk	Column-ভিত্তিক text প্রক্রিয়াকরণ
find / xargs	ফাইল খোঁজা / খুঁজে পাওয়া ফাইলে কমান্ড চালানো
Field / Delimiter	Column / column আলাদা করার অক্ষর

REAL WORLD: ঢাকার একটি SaaS company-র support দল জানতে চাইল, গত ২৪ ঘণ্টায় কোন ব্যবহারকারীরা সবচেয়ে বেশি ভুল password দিয়েছে। Log ফাইলটি ছিল ৮০০ MB। একজন engineer একটি pipeline লিখলেন — `grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -5` — আর ত্রিশ সেকেন্ডে উত্তর পাওয়া গেল। এই pipeline-টি পরে একটি script-এ রাখা হলো, যা প্রতিদিন সকালে একটি সারসংক্ষেপ পাঠায়। Log বিশ্লেষণ শুধু debugging নয় — নিরাপত্তা পর্যবেক্ষণও।

১১.৯ হাতে-কলমে অনুশীলন

LAB: Log বিশ্লেষণ — একটি নমুনা log থেকে উত্তর বের করা।

1. একটি নমুনা log বানান: `printf '%s\n' '192.168.1.10 GET /api/users 200' '192.168.1.11 GET /api/orders 500' '192.168.1.10 GET /api/users 500' '192.168.1.12 POST /api/login 200' > ~/technova/sample.log`।
2. `grep "500" ~/technova/sample.log | wc -l` দিয়ে 500 error গুনুন।
3. `awk '{print $1}' ~/technova/sample.log | sort | uniq -c | sort -rn` দিয়ে সবচেয়ে বেশি request করা IP বের করুন।
4. `awk '$4 == 500 {print $1}' ~/technova/sample.log` দিয়ে 500 error-এর IP বের করুন।
5. ফাইলটিতে space-যুক্ত নাম দিয়ে পরীক্ষা করুন: `"my log.txt"` নামে একটি ফাইল বানিয়ে `find` + `-print0 | xargs -0` দিয়ে তা খুঁজুন।

উদাহরণ আউটপুট (illustrative):

```
$ awk '{print $1}' ~/technova/sample.log | sort | uniq -c | sort -rn
      2 192.168.1.10
      1 192.168.1.11
      1 192.168.1.12
```

যাচাই: 500 error-এর সংখ্যা ২ হওয়া উচিত। `sort` ছাড়া `uniq -c` চালালে ভুল গণনা দেখুন — এতে `sort`-এর প্রয়োজন বোঝা যাবে।

১১.১০ Common Mistakes ও Troubleshooting

- **ভুল:** `uniq`-এর আগে `sort` না করা — `uniq` শুধু পাশাপাশি একই লাইন গুনে। সমাধান: সবসময় `sort | uniq -c`।
- **ভুল:** `>` দিয়ে log ফাইল মুছে ফেলা। সমাধান: log-এ `>>` ব্যবহার করুন।
- **ভুল:** Pipe-এর ভেতরে `while read` লুপে variable বদলে বাইরে তা আশা করা। সমাধান: `done < file` redirection ব্যবহার করুন।
- **ভুল:** `sed -i` backup ছাড়া ব্যবহার করা। সমাধান: প্রথমে পরীক্ষা করুন, তারপর `-i.bak` ব্যবহার করুন।
- **ভুল:** ফাইলের নামে space থাকলে `for FILE in $(ls)` ব্যবহার করা — নাম ভেঙে যায়। সমাধান: `find -print0 | xargs -0` বা glob (`for FILE in *.log`) ব্যবহার করুন।
- **সমস্যা:** `grep` কিছু খুঁজে পাচ্ছে না — প্রথমে `cat` দিয়ে ফাইলের আসল লেখা দেখুন; হয়তো বড়-ছোট হাতের পার্থক্য, তখন `-i` ব্যবহার করুন।

১১.১১ Security Considerations

Text processing-এর সবচেয়ে বড় ঝুঁকি হলো **ফাইলের নাম ও log-এর বিষয়বস্তুকে কমান্ড হিসেবে চালানো**। কখনো log-এর একটি লাইনকে `eval` বা অquote করা variable হিসেবে কমান্ডে বসাবেন না — log-এ ক্ষতিকর লেখা থাকলে তা কমান্ড হয়ে চলতে পারে। আর `xargs` ব্যবহারের সময় `-0` ছাড়া চালাবেন না — ফাইলের নামে space বা বিশেষ অক্ষর থাকলে ভুল ফাইল মুছে যেতে পারে। Log ফাইলে password বা token থাকতে পারে — log বিশ্লেষণের ফলাফল শেয়ার করার আগে তা পরীক্ষা করুন।

১১.১২ Best Practices

- Pipeline ছোট ছোট ধাপে তৈরি করুন — প্রতিটি ধাপ আলাদাভাবে পরীক্ষা করুন।
- `uniq`-এর আগে সবসময় `sort` করুন।
- ফাইলের নামে `-print0 | xargs -0` ব্যবহার করুন।
- ফাইল পড়তে `while IFS= read -r line` প্যাটার্নটি ব্যবহার করুন।
- `sed -i`-এর আগে backup নিন।
- ভুল বার্তা stderr-এ পাঠান, সাধারণ output stdout-এ রাখুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `2> error.log` কী করে? ক. stdout ফাইলে পাঠায় খ. stderr ফাইলে পাঠায় গ. ফাইলটি মুছে ফেলে ঘ. Input ফাইল থেকে নেয়
2. `uniq -c`-এর আগে `sort` কেন লাগে? ক. uniq শুধু পাশাপাশি একই লাইন গুনতে পারে খ. sort ছাড়া uniq চলে না গ. sort ফাইল মুছে ফেলে ঘ. uniq সংখ্যা বোঝে না
3. Apache log-এ `awk '{print $1}'` কী দেয়? ক. Status code খ. IP address গ. URL ঘ. তারিখ
4. ফাইলের নামে space থাকলে নিরাপদ উপায় কোনটি? ক. find | xargs rm খ. find -print0 | xargs -0 গ. for FILE in $(ls) ঘ. rm *

5. `sed 's/old/new/g'`-এ `g`-এর অর্থ কী? ক. Global — লাইনের সবগুলো বদলানো খ. Good — ভালোভাবে বদলানো গ. Group — দল বেঁধে বদলানো ঘ. Get — ফাইল থেকে নেওয়া

১১.১৩ Practical Assignment

Assignment 11.1 (Intermediate): একটি `log-report.sh` script লিখুন যা একটি log ফাইলের নাম argument হিসেবে নেয়, তাতে মোট লাইন, ERROR লাইনের সংখ্যা, সবচেয়ে বেশি আসা ৫টি IP, আর সবচেয়ে বেশি আসা ৫টি status code — একটি সারসংক্ষেপ হিসেবে ছাপে। ফাইলটি না থাকলে স্পষ্ট ভুল বার্তা দিয়ে থামবে।

১১.১৪ Chapter Summary

প্রতিটি প্রোগ্রামের তিনটি stream: stdin (০), stdout (১), stderr (২)। Redirection গন্তব্য বদলায়, pipe (`|`) একটির output পরেরটির input করে। `grep` খোঁজে, `sed` বদলায়, `awk` column বিশ্লেষণ করে, `cut/sort/uniq` সাজায় ও গোনে, `find` ও `xargs` ফাইল খুঁজে কাজ করে। ফাইল পড়তে `while IFS= read -r line`, আর ফাইলের নামে `-print0 | xargs -0` — এই দুটি প্যাটার্ন মনে রাখলেই text automation নিরাপদ হয়।

Revision Checklist

- ☐ stdin, stdout, stderr ও redirection ব্যাখ্যা করতে পারি।
- ☐ Pipe দিয়ে log বিশ্লেষণের pipeline লিখতে পারি।
- ☐ `grep`, `sed`, `awk` দিয়ে কাজ করতে পারি।
- ☐ `sort | uniq -c | sort -rn` প্যাটার্নটি জানি।
- ☐ `find -print0 | xargs -0` নিরাপদে ব্যবহার করতে পারি।
- ☐ `while IFS= read -r line` দিয়ে ফাইল পড়তে পারি।

অধ্যায় ১২ • Part II — Bash Scripting Engineering

Chapter 12: Error Handling, Exit Codes and Debugging — ভুল সামলানো, Exit Code ও Debugging

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র একটি deploy script-এ একটি লাইন ছিল: `cp new-version.tar.gz /opt/app/`। একদিন ফাইলটি আপলোড অসম্পূর্ণ থাকায় `cp` ব্যর্থ হলো — কিন্তু script থামল না। সে পরের লাইনে গিয়ে পুরনো version মুছে ফেলল, তারপর নতুন version চালু করার চেষ্টা করল — যা ছিল না। ফল: ওয়েবসাইট বন্ধ, আর ফিরে যাওয়ার কিছু নেই। Script-টির একটিই দোষ ছিল: সে ভুলের পরেও এগিয়ে গিয়েছিল। একজন ভালো script লেখকের প্রথম প্রশ্ন হলো, "ভুল হলে কী হবে?" — আর এই অধ্যায় সেই প্রশ্নের উত্তর।

> **CONCEPT:** Exit status হলো প্রতিটি কমান্ডের শেষ কথা — ০ মানে "সফল", ০ ছাড়া অন্য কিছু মানে "ব্যর্থ, আর সংখ্যাটি বলে কী ধরনের ব্যর্থতা"। ভালো script এই শেষ কথাটি শোনে, আর ব্যর্থ হলে নিরাপদে থামে — অন্ধভাবে এগিয়ে যায় না।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Exit status ও `$?` ব্যাখ্যা করে ব্যবহার করতে।
- `set -e`, `set -u` ও `set -o pipefail`-এর কাজ ও সীমা বলতে।
- `trap` দিয়ে cleanup ও signal handling লিখতে।
- Input validation লিখে ভুল argument নিরাপদে প্রত্যাখ্যান করতে।
- Log function দিয়ে script-এর কাজের রেকর্ড রাখতে।
- `bash -x` ও `set -x` দিয়ে debug করতে।
- Idempotency ব্যাখ্যা করে script-কে নিরাপদে বারবার চালানোর উপযোগী করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–11: script কাঠামো, variable, শর্ত, function, pipeline ও redirection।
- Chapter 09-এর `if` ও exit status-এর প্রাথমিক ধারণা।

১২.১ Exit Status ও `$?`

Linux-এ প্রতিটি কমান্ড শেষ হওয়ার সময় একটি সংখ্যা ফেরত দেয় — **exit status**। ০ মানে সফল। ১ মানে সাধারণ ব্যর্থতা। ২ মানে ভুল ব্যবহার (যেমন ভুল argument)। ১২৬ মানে ফাইলটি চালানো যায় না (permission নেই), ১২৭ মানে কমান্ডটি পাওয়া যায়নি। Script-এর ক্ষেত্রে `exit 0` মানে সফলভাবে শেষ, `exit 1` মানে ব্যর্থভাবে শেষ। শেষ কমান্ডটির exit status জানতে `$?` ব্যবহার করুন — তবে সতর্কতা: `$?` শুধু ঠিক আগের কমান্ডটির status ধরে। `echo "$?"` লিখলেই `$?`-এর মান বদলে যায় — কারণ `echo` নিজেই একটি কমান্ড। তাই status ধরতে হলে সঙ্গে সঙ্গে একটি variable-এ রাখুন: `STATUS="$?"`।

Chapter 09-এর `if` আসলে এই exit status-ই দেখে। `if grep -q "ERROR" app.log; then` — `grep -q` pattern পেলে ০ দেয়, না পেলে ১ দেয়; `if` সেই সংখ্যা দেখে সিদ্ধান্ত নেয়। `-q` মানে quiet — grep কিছু ছাপে না, শুধু status দেয়।

```
#!/usr/bin/env bash
set -euo pipefail

cp important.conf /var/backups/technova/
STATUS="$?"

if [[ "${STATUS}" -ne 0 ]]; then
    echo "Error: backup copy failed with status ${STATUS}." >&2
    exit 1
fi

echo "Backup copy succeeded."
```

ব্যাখ্যা: এই উদাহরণটি শিক্ষামূলক — বাস্তবে `set -e` থাকলে `cp` ব্যর্থ হলেই script থেমে যেত, `STATUS` পরীক্ষা পর্যন্ত পৌঁছাত না। তাই `set -e`-এর সঙ্গে এই প্যাটার্নটি ব্যবহার করতে হলে `cp ... || STATUS="$?"` লিখতে হয় (১২.২ অনুচ্ছেদ দেখুন)। এখানে মূল শিক্ষা: `$?` সঙ্গে সঙ্গে ধরে রাখতে হয়, আর ভুলের সংখ্যাটি বার্তায় জানাতে হয় — শুধু "ভুল হয়েছে" নয়, "কোন ভুল" তা জানানো।

১২.২ `set -euo pipefail`: তিনটি নিরাপত্তা সুইচ

এই বইয়ের প্রতিটি script-এর দ্বিতীয় লাইন `set -euo pipefail` — তিনটি সুইচ একসঙ্গে। `set -e` (errexit): কোনো কমান্ড ব্যর্থ হলে (exit status ০ নয়) script সঙ্গে সঙ্গে থেমে যায় — ভুলের পরেও এগিয়ে যাওয়া বন্ধ হয়। `set -u` (nounset): অঘোষিত variable ব্যবহার করলে script ভুল ধরে থেমে যায় — যেমন `$SERBER` লিখে ফেললে (সঠিক `$SERVER`-এর বদলে) তা ধরা পড়ে। `set -o pipefail`: pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline ব্যর্থ ধরা হয় — নয়তো শুধু শেষ কমান্ডের status দেখা হতো, আর মাঝখানের ভুল লুকিয়ে যেত।

তবে এই তিনটি সুইচের সীমা জানা জরুরি — নয়তো অন্ধ বিশ্বাস বিপজ্জনক। `set -e` কাজ করে না `if`-এর শর্তে, `while`-এর শর্তে, `&&` বা `||`-এর ডানদিকে, আর `!` দিয়ে উল্টানো কমান্ডে — কারণ সেসব জায়গায় ব্যর্থতাই প্রত্যাশিত। আর `set -e` function-এর ভেতরে ও subshell-এ অপ্রত্যাশিত আচরণ করতে পারে। তাই নিয়ম: `set -euo pipefail` সবসময় ব্যবহার করুন, কিন্তু তার উপর পুরোপুরি নির্ভর করবেন না — গুরুত্বপূর্ণ ধাপগুলোতে নিজে `if` দিয়ে পরীক্ষা করুন। যেমন backup নেওয়ার পর `if [[ ! -f "${BACKUP_FILE}" ]]; then` দিয়ে ফাইলটি আসলে তৈরি হয়েছে কি না, তা পরীক্ষা করুন।

সুইচ	কাজ	সীমা
set -e	কমান্ড ব্যর্থ হলে script থামে	if/while শর্তে, &&/||-এর ডানদিকে কাজ করে না
set -u	অঘোষিত variable ব্যবহার করলে ভুল ধরে	`${VAR:-default}` দিয়ে ইচ্ছাকৃতভাবে এড়ানো যায়
set -o pipefail	Pipeline-এর যেকোনো অংশ ব্যর্থ হলে ধরা পড়ে	—
set -x	প্রতিটি কমান্ড চালানোর আগে দেখায় (debug)	Trace-এ গোপন তথ্য ফাঁস হতে পারে

১২.৩ `trap`: Script থামলেও পরিষ্কার করে যাওয়া

Script মাঝপথে থামলে কী হবে? একটি অস্থায়ী ফাইল পড়ে থাকবে, একটি lock ফাইল আটকে থাকবে, আর পরেরবার script চলবে না। `trap` এই সমস্যার সমাধান: script-কে বলা যায়, "শেষ হওয়ার সময় (বা কোনো signal পেলে) এই cleanup কাজটি করে যেয়ো।" সবচেয়ে বেশি ব্যবহৃত রূপ: `trap cleanup EXIT` — script যেভাবেই শেষ হোক (সফল বা ব্যর্থ), `cleanup` function চলবে।

```
#!/usr/bin/env bash
set -euo pipefail

TEMP_DIR="$(mktemp -d)"

cleanup() {
    rm -rf "${TEMP_DIR}"
    echo "Temporary directory removed."
}

trap cleanup EXIT

echo "Working in ${TEMP_DIR}"
echo "sample data" > "${TEMP_DIR}/data.txt"
echo "Work finished."
```

ব্যাখ্যা: `mktemp -d` — একটি নিরাপদ অস্থায়ী directory বানায়, যার নাম অনুমান করা যায় না (১৩.১ অনুচ্ছেদ)। `cleanup() { ... }` — অস্থায়ী directory মুছে ফেলার function। `trap cleanup EXIT` — script শেষ হলে (সফলভাবে বা ভুলে থেমে গেলেও) `cleanup` চলবে। `rm -rf "${TEMP_DIR}"` — এখানে `"${TEMP_DIR}"` quote করা আছে, আর `TEMP_DIR` `mktemp` থেকে এসেছে — তাই এটি নিরাপদ। তবে সতর্কতা: `rm -rf`-এর সঙ্গে কখনো খালি হতে পারে এমন variable ব্যবহার করবেন না — `TEMP_DIR` খালি হলে `rm -rf ""` কিছু করবে না, কিন্তু `rm -rf "${TEMP_DIR}/"`-এর মতো লেখায় ভুল হতে পারে। `trap` signal-এর জন্যও ব্যবহার করা যায়: `trap cleanup INT TERM` — Ctrl+C (INT) বা terminate signal (TERM) পেলে cleanup করে থামবে।

১২.৪ Input Validation: ভুল input আগেই ধরা

Script-এর বেশিরভাগ বিপদ আসে খারাপ input থেকে। Validation মানে input ব্যবহার করার আগে তা পরীক্ষা করা: argument দেওয়া হয়েছে কি? ফাইলটি আছে কি? সংখ্যাটি আসলেই সংখ্যা কি? পথটি প্রত্যাশিত directory-র ভেতরে কি? একটি কার্যকর প্যাটার্ন: `${1:?message}` — প্রথম argument না থাকলে Bash নিজেই বার্তা দেখিয়ে থেমে যায়। আর সংখ্যা পরীক্ষা করতে regex ব্যবহার করা যায়: `[[ "${VALUE}" =~ ^[0-9]+$ ]]` — মানটি শুধু সংখ্যা দিয়ে তৈরি কি না, তা পরীক্ষা করে।

```
#!/usr/bin/env bash
set -euo pipefail

validate_port() {
    local port="$1"
    if [[ ! "${port}" =~ ^[0-9]+$ ]]; then
        echo "Error: port must be a number: ${port}" >&2
        return 1
    fi
    if [[ "${port}" -lt 1 || "${port}" -gt 65535 ]]; then
        echo "Error: port must be between 1 and 65535: ${port}" >&2
        return 1
```

```
    fi
    return 0
}

main() {
    local port="${1:?Usage: $0 <port>}"
    if validate_port "${port}"; then
        echo "Port ${port} is valid."
    fi
}

main "$@"
```

ব্যাখ্যা: `${1:?Usage: $0 <port>}` — `:?` মানে "না থাকলে এই বার্তা দেখিয়ে থামো"। `[[ ! "${port}" =~ ^[0-9]+$ ]]` — `=~` মানে regex মেলানো; `^[0-9]+$` মানে শুরু থেকে শেষ পর্যন্ত শুধু সংখ্যা। `[[ "${port}" -lt 1 || "${port}" -gt 65535 ]]` — port-এর বৈধ সীমা পরীক্ষা। `validate_port` ব্যর্থ হলে `return 1` দেয়, আর `set -e`-এর কারণে `main`-এর `if`-এর বাইরে থাকলে script থেমে যেত — এখানে `if`-এর ভেতরে থাকায় তা নিয়ন্ত্রিতভাবে ধরা হচ্ছে।

Error Handling-এর প্রবাহ

Input আসে → Validation (argument, ফাইল, সংখ্যা পরীক্ষা) → ভুল হলে: স্পষ্ট বার্তা (stderr) + exit 1 → ঠিক হলে: কাজ করা

কাজের সময় ভুল হলে: set -e থামিয়ে দেয় → trap cleanup EXIT চলে → অস্থায়ী ফাইল মুছে যায়

নিয়ম: ভুল যত আগে ধরা যায়, ক্ষতি তত কম। সবচেয়ে খারাপ script হলো সেই script, যা ভুলের পরেও এগিয়ে যায়।

১২.৫ Logging: কী ঘটল, তার রেকর্ড

Script-এর output দুই ধরনের: মানুষের পড়ার জন্য বার্তা, আর পরে পরীক্ষার জন্য log। একটি ভালো log function সময়, স্তর (INFO/WARNING/ERROR) ও বার্তা — তিনটি লেখে, আর ভুল বার্তা stderr-এ পাঠায়। আরও ভালো: log ফাইলেও লেখা — `exec >> logfile` দিয়ে script-এর সব stdout একটি ফাইলে পাঠানো যায়, তবে তখন পর্দায় কিছু দেখা যায় না। একটি সহজ পদ্ধতি: log function-এ `tee -a` ব্যবহার করা — যা পর্দায়ও দেখায়, ফাইলেও লেখে।

```
#!/usr/bin/env bash
set -euo pipefail

LOG_FILE="/var/log/technova/daily-check.log"

log_info() {
    echo "[$(date '+%F %T')] [INFO] $1" | tee -a "${LOG_FILE}"
}

log_error() {
    echo "[$(date '+%F %T')] [ERROR] $1" | tee -a "${LOG_FILE}" >&2
}
```

ব্যাখ্যা: `$(date '+%F %T')` — তারিখ ও সময়, যেমন `2026-11-08 09:30:00`। `tee -a "${LOG_FILE}"` — `tee` input-কে পর্দায়ও দেয়, ফাইলেও লেখে; `-a` মানে append (আগের log মুছবে না)। `log_error`-এ `>&2` থাকায় ভুল বার্তা

stderr-এ যায়। সতর্কতা: `/var/log/technova/` directory-টি আগে থেকে থাকতে হবে — নয়তো `tee` ব্যর্থ হবে। Chapter 19-এ Python-এর logging module দিয়ে আরও উন্নত log (rotation-সহ) শেখানো হবে।

১২.৬ Debugging: ভুল খুঁজে বের করার পদ্ধতি

Debugging-এর পদ্ধতিগত উপায়: (১) `bash -n script.sh` — syntax পরীক্ষা, চালানো ছাড়া। (২) `bash -x script.sh` — trace, প্রতিটি কমান্ড চালানোর আগে দেখায়। (৩) `set -x` ও `set +x` — script-এর ভেতরে শুধু সন্দেহজনক অংশ trace করা। (৪) `echo` দিয়ে variable-এর মান দেখা — কাজ শেষে মুছে ফেলতে হয়। (৫) ছোট করে পরীক্ষা করা: script-এর সন্দেহজনক অংশটি আলাদা ফাইলে নিয়ে চালানো।

Trace output পড়ার নিয়ম: `+` দিয়ে শুরু হওয়া লাইনটি হলো চালানো কমান্ড, তার আগের লাইনগুলো হলো তার output। যেমন `+ [[ 85 -ge 80 ]]` দেখলে বুঝবেন শর্তটি পরীক্ষা হচ্ছে। আর `set -x`-এর output-এ variable-এর মান দেখা যায় — তাই trace চালানোর আগে নিশ্চিত হোন script-এ গোপন তথ্য নেই (Chapter 07-এর সতর্কতা)।

১২.৭ Idempotency: বারবার চালালেও নিরাপদ

Idempotency মানে: একটি কাজ একবার চালালে যা ফল হয়, দশবার চালালেও একই ফল হয় — ক্ষতি হয় না। যেমন `mkdir -p /var/backups` — directory থাকলেও ভুল হয় না, আবার চালানো যায়। কিন্তু `mkdir /var/backups` (`-p` ছাড়া) দ্বিতীয়বার চালালে ভুল হয়। Idempotent script লেখার উপায়: (১) `mkdir -p` ব্যবহার করা, (২) ফাইল বানানোর আগে আছে কি না পরীক্ষা করা, (৩) `cp`-এর আগে উৎস ফাইল আছে কি না পরীক্ষা করা, (৪) একই backup দুবার নিলে পুরনোটি মুছে না ফেলা — বরং তারিখ দিয়ে আলাদা নাম দেওয়া।

কেন এটি গুরুত্বপূর্ণ? কারণ automation বারবার চলে — cron প্রতিদিন চালায়, deploy script আবার চালানো হয়, আর মানুষ ভুল করে একই script দুবার চালায়। Non-idempotent script দ্বিতীয়বার চালালে ক্ষতি করে — যেমন একই ব্যবহারকারী দুবার তৈরি করা, বা একই ফাইল দুবার যোগ করা। Chapter 24-এ idempotency production automation-এর মূল নীতি হিসেবে আবার আসবে।

১২.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Exit status	কমান্ডের ফলাফলের সংখ্যা — ০ মানে সফল
$?	ঠিক আগের কমান্ডটির exit status
set -e / -u / pipefail	ভুলে থেমে যাওয়া / অঘোষিত variable ধরা / pipeline-এর ভুল ধরা
trap	Script শেষ হলে বা signal পেলে cleanup চালানো
Signal	Process-কে পাঠানো বার্তা (INT, TERM)
Validation	Input ব্যবহারের আগে তা পরীক্ষা করা
Idempotency	বারবার চালালেও একই নিরাপদ ফল
Logging	কী ঘটল, তার সময়সহ রেকর্ড রাখা

REAL WORLD: সিলেটের একটি company-র backup script-এ `rm` লাইনটি backup নেওয়ার আগে ছিল, পরে নয়। একদিন backup ব্যর্থ হলো — কিন্তু `rm` আগেই পুরনো backup মুছে ফেলেছিল। এরপর script-টি নতুন করে লেখা হলো: প্রথমে নতুন backup নেওয়া ও যাচাই করা, তারপর পুরনো backup মুছে ফেলা, আর প্রতিটি ধাপে ভুল হলে `exit 1`। ক্রম (order) ও ভুল সামলানো — দুটোই নিরাপত্তা।

১২.৯ হাতে-কলমে অনুশীলন

LAB: একটি ভুলপ্রবণ script-কে নিরাপদ করা — trap ও validation যোগ করা।

1. একটি ইচ্ছাকৃতভাবে ভুলপ্রবণ script লিখুন: অস্থায়ী ফাইল বানায়, কাজ করে, কিন্তু `trap` নেই।
2. তাতে `mktemp -d` দিয়ে অস্থায়ী directory, `trap cleanup EXIT` দিয়ে cleanup, আর argument validation যোগ করুন।
3. Script-টি মাঝপথে ব্যর্থ করুন (ভুল ফাইলের নাম দিয়ে) — দেখুন অস্থায়ী directory মুছে গেছে কি না (`ls /tmp` দিয়ে পরীক্ষা করুন)।
4. `bash -x` দিয়ে trace করুন, আর `bash -n` দিয়ে syntax পরীক্ষা করুন।
5. Script-টি দুবার চালান — দ্বিতীয়বারও একই ফল হয় কি না, তা দেখুন (idempotency পরীক্ষা)।

উদাহরণ আউটপুট (illustrative):

```
$ ./safe-backup.sh /var/www
[INFO] Starting backup of /var/www
[INFO] Backup completed successfully.
Temporary directory removed.
```

যাচাই: `echo "$?"` — সফল হলে ০ দেখাবে। ভুল argument দিলে `exit 1` ও স্পষ্ট বার্তা দেখাবে।

১২.১০ Common Mistakes ও Troubleshooting

- **ভুল:** `$?` পরীক্ষা করার আগে আরেকটি কমান্ড চালানো — `$?`-এর মান বদলে যায়। সমাধান: সঙ্গে সঙ্গে variable-এ রাখুন।
- **ভুল:** `set -e`-এর উপর পুরোপুরি নির্ভর করা — `if`-এর শর্তে তা কাজ করে না। সমাধান: গুরুত্বপূর্ণ ধাপ নিজে `if` দিয়ে পরীক্ষা করুন।
- **ভুল:** `trap` না লেখা — মাঝপথে থামলে অস্থায়ী ফাইল পড়ে থাকে। সমাধান: script-এর শুরুতেই `trap cleanup EXIT` লিখুন।
- **ভুল:** ভুল বার্তা stdout-এ লেখা — log থেকে ভুল আলাদা করা যায় না। সমাধান: ভুল বার্তা `>&2` দিয়ে stderr-এ পাঠান।
- **ভুল:** `rm -rf`-এর সঙ্গে খালি হতে পারে এমন variable। সমাধান: `[[ -n "${DIR}" ]]` দিয়ে পরীক্ষা করুন, আর `mktemp` থেকে আসা পথ ব্যবহার করুন।
- **সমস্যা:** Script হঠাৎ থেমে যাচ্ছে, কারণ বোঝা যাচ্ছে না — `bash -x` দিয়ে দেখুন কোন লাইনে থামছে।

১২.১১ Security Considerations

Error handling-এর নিরাপত্তা মূলত দুটি জায়গায়: ভুল বার্তায় গোপন তথ্য না ফাঁস করা, আর ব্যর্থ অবস্থায় নিরাপদে থেমে যাওয়া। ভুল বার্তায় কখনো password, token বা পুরো file path-এর সংবেদনশীল অংশ লিখবেন না — log ফাইলটি অনেকেই পড়তে পারে। আর ব্যর্থ হলে script-এর উচিত নিরাপদ অবস্থায় থামা — যেমন deploy ব্যর্থ হলে পুরনো version চালু রাখা, নতুন অসম্পূর্ণ version চালু না করা। একে বলে **fail-safe**: ব্যর্থতা যেন নিরাপদ দিকে যায়, বিপজ্জনক দিকে নয়।

১২.১২ Best Practices

- প্রতিটি script-এ `set -euo pipefail` ও `trap cleanup EXIT` রাখুন।
- ভুল বার্তায় কী ভুল হয়েছে, কোথায় হয়েছে, আর কী করতে হবে — তিনটি লিখুন।
- গুরুত্বপূর্ণ ধাপের পর ফলাফল যাচাই করুন (ফাইল তৈরি হয়েছে কি, service চলছে কি)।
- Input validation script-এর শুরুতে করুন, কাজের মাঝখানে নয়।
- Script-কে idempotent করুন — বারবার চালানো নিরাপদ করুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Exit status ০-এর অর্থ কী? ক. ভুল হয়েছে খ. সফল হয়েছে গ. কমান্ড পাওয়া যায়নি ঘ. Permission নেই
2. `set -u` কী ধরে? ক. Pipeline-এর ভুল খ. অঘোষিত variable-এর ব্যবহার গ. Syntax ভুল ঘ. Network ভুল
3. `trap cleanup EXIT` কী করে? ক. Script মুছে ফেলে খ. Script শেষ হলে cleanup চালায় গ. Script-কে দ্রুত চালায় ঘ. Log ফাইল মুছে ফেলে
4. Idempotency মানে কী? ক. Script একবারই চালানো যায় খ. বারবার চালালেও একই নিরাপদ ফল গ. Script দ্রুত চলে ঘ. Script-এ ভুল হয় না
5. `$?` ব্যবহারের নিয়ম কী? ক. যেকোনো সময় পড়া যায় খ. ঠিক আগের কমান্ডের পরেই ধরতে হয় গ. শুধু function-এ ব্যবহার করা যায় ঘ. শুধু loop-এ ব্যবহার করা যায়

১২.১৩ Practical Assignment

Assignment 12.1 (Intermediate): Chapter 09-এর `service-check.sh`-কে নিরাপদ করুন: `set -euo pipefail`, argument validation, log function (সময়সহ), `trap cleanup EXIT` দিয়ে অস্থায়ী ফাইল পরিষ্কার, আর idempotency পরীক্ষা (দুবার চালিয়ে একই ফল) যোগ করুন। প্রতিটি পরিবর্তনের কারণ এক লাইনে মন্তব্য হিসেবে লিখুন।

১২.১৪ Chapter Summary

প্রতিটি কমান্ড একটি exit status দেয় — ০ মানে সফল, বাকি সব ব্যর্থতা। `$?` ঠিক আগের কমান্ডের status ধরে, তাই সঙ্গে সঙ্গে variable-এ রাখতে হয়। `set -euo pipefail` ভুলে থামে, অঘোষিত variable ধরে, আর pipeline-এর ভুল ধরে — কিন্তু তার সীমা আছে, তাই গুরুত্বপূর্ণ ধাপ নিজে পরীক্ষা করুন। `trap` দিয়ে cleanup নিশ্চিত করুন, input validation দিয়ে ভুল আগেই ধরুন, log দিয়ে রেকর্ড রাখুন, আর script-কে idempotent করুন — যাতে বারবার চালালেও নিরাপদ থাকে।

Revision Checklist

- ☐ Exit status ও `$?`-এর নিয়ম বলতে পারি।
- ☐ `set -euo pipefail`-এর প্রতিটি অংশ ও তার সীমা জানি।
- ☐ `trap cleanup EXIT` লিখতে পারি।
- ☐ Input validation (regex-সহ) লিখতে পারি।
- ☐ Log function লিখতে পারি।
- ☐ Idempotency ব্যাখ্যা করে script-কে নিরাপদ করতে পারি।

অধ্যায় ১৩ • Part II — Bash Scripting Engineering

Chapter 13: Advanced Bash Automation and Best Practices — উন্নত Bash Automation ও শ্রেষ্ঠ চর্চা

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র একটি backup script একদিন দুটি cron job একসঙ্গে চালাল — দুটি instance একই backup ফাইলে লিখতে গিয়ে ফাইলটি নষ্ট করে ফেলল। আরেকদিন একটি script-এ ব্যবহারকারীর দেওয়া ফাইলের নামের ভেতরে একটি বিশেষ অক্ষর ছিল — script-টি তা কমান্ড হিসেবে চালিয়ে ভুল directory মুছে ফেলল। এই দুটি ঘটনার মূলে একই শিক্ষা: ছোট script লেখা সহজ, কিন্তু **production-এ নিরাপদে চলা script** লেখা একটি পেশাদার দক্ষতা। এই অধ্যায়ে সেই দক্ষতা শেখা হবে — lock file, অস্থায়ী ফাইল, process পরীক্ষা, ShellCheck, নিরাপত্তা, আর নির্ভরযোগ্য automation-এর প্যাটার্ন।

> **CONCEPT:** একটি script লেখা আর একটি production automation লেখার পার্থক্য হলো প্রশ্নের তালিকা। ভালো script লেখক শুধু "এটি কীভাবে কাজ করবে" জিজ্ঞেস করেন না — তিনি জিজ্ঞেস করেন, "দুবার চললে কী হবে? মাঝপথে থামলে কী হবে? ভুল input এলে কী হবে? অন্য কেউ এটি বদলে দিলে কী হবে?"

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- `mktemp` দিয়ে নিরাপদ অস্থায়ী ফাইল ও directory বানাতে।
- `flock` দিয়ে lock file ব্যবহার করে একই script দুবার চলা ঠেকাতে।
- Process চলছে কি না পরীক্ষা করে নিরাপদে কাজ করতে।
- Configuration ফাইল থেকে মান পড়ে script চালাতে।
- ShellCheck দিয়ে script পরীক্ষা করে সাধারণ ভুল ঠিক করতে।
- Command injection ব্যাখ্যা করে `eval` ও অquote করা variable-এর ঝুঁকি এড়াতে।
- একটি script-কে পরীক্ষা (test) করার পদ্ধতি বলতে ও প্রয়োগ করতে।
- Part II-এর সব জ্ঞান জুড়ে একটি নির্ভরযোগ্য automation script লিখতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–12: script কাঠামো, variable, শর্ত, লুপ, function, pipeline, error handling ও trap।
- Ubuntu-তে `apt` দিয়ে package install করার অভিজ্ঞতা (ShellCheck install করতে লাগবে)।

১৩.১ অস্থায়ী ফাইল: `mktemp`

Script-এর অস্থায়ী ফাইলের নাম কখনো অনুমানযোগ্য রাখবেন না — যেমন `/tmp/work.txt`। কেন? কারণ একজন আক্রমণকারী আগেই `/tmp/work.txt` নামে একটি symbolic link বানিয়ে রাখতে পারে, যা একটি গুরুত্বপূর্ণ ফাইলের দিকে নির্দেশ করে — আর আপনার script যখন `/tmp/work.txt`-এ লিখবে, তখন আসলে সেই গুরুত্বপূর্ণ ফাইলটি বদলে

যাবে। একে বলে **symlink attack**। সমাধান: `mktemp` — এটি এলোমেলো, অনুমান করা যায় না এমন নামের ফাইল বানায়, আর নিরাপদ permission (শুধু মালিক পড়তে পারে) দিয়ে বানায়।

```
#!/usr/bin/env bash
set -euo pipefail

TEMP_FILE="$(mktemp)"
TEMP_DIR="$(mktemp -d)"

cleanup() {
    rm -f "${TEMP_FILE}"
    rm -rf "${TEMP_DIR}"
}

trap cleanup EXIT

echo "Temporary file: ${TEMP_FILE}"
echo "Temporary directory: ${TEMP_DIR}"
echo "sample data" > "${TEMP_FILE}"
```

ব্যাখ্যা: `mktemp` — একটি অস্থায়ী ফাইল বানায়, যেমন `/tmp/tmp.abc123`। `mktemp -d` — একটি অস্থায়ী directory বানায়। `cleanup()` — দুটোই মুছে ফেলে; `rm -f` ফাইলের জন্য, `rm -rf` directory-র জন্য। `trap cleanup EXIT` — script শেষ হলে cleanup চলবে, যাতে অস্থায়ী ফাইল পড়ে না থাকে। এই প্যাটার্নটি Chapter 12-এর trap-এর সঙ্গে মিলিয়ে ব্যবহার করুন — প্রতিটি script-এ অস্থায়ী ফাইল থাকলে এই তিনটি লাইন (mktemp, cleanup, trap) থাকবেই।

১৩.২ Lock File: একই Script দুবার চলা ঠেকানো

Cron-এ চলা একটি script যদি আগেরবারের কাজ শেষ হওয়ার আগেই আবার শুরু হয়, তাহলে দুটি instance একই ফাইলে লিখতে পারে, একই backup নষ্ট করতে পারে, বা একই কাজ দুবার করতে পারে। সমাধান: **lock file** — script শুরুতে একটি lock ফাইল ধরে, আর অন্য instance দেখে lock আছে কি না। সবচেয়ে নির্ভরযোগ্য উপায় হলো `flock` কমান্ড — এটি kernel-এর সাহায্যে lock ধরে, আর script থামলে (এমনকি crash করলেও) lock নিজে থেকে ছেড়ে যায়।

```
#!/usr/bin/env bash
set -euo pipefail

LOCK_FILE="/var/lock/technova-backup.lock"

exec 9>"${LOCK_FILE}"

if ! flock -n 9; then
    echo "Another instance is already running. Exiting." >&2
    exit 1
fi

echo "Lock acquired. Running backup..."
sleep 5
echo "Backup finished."
```

লাইন ধরে ব্যাখ্যা: `exec 9>"${LOCK_FILE}"` — file descriptor ৯-কে lock ফাইলের সঙ্গে যুক্ত করা হলো। File descriptor হলো একটি খোলা ফাইলের নম্বর — ০, ১, ২ হলো stdin, stdout, stderr; ৯ এখানে একটি অতিরিক্ত নম্বর। `flock -n 9` — descriptor ৯-এর উপর lock চাওয়া হচ্ছে; `-n` মানে non-blocking — lock না পেলে অপেক্ষা না করে সঙ্গে সঙ্গে ব্যর্থ হবে। `if ! flock -n 9; then` — lock না পেলে বার্তা দিয়ে `exit 1`। Script শেষ হলে descriptor বন্ধ

হয়ে যায়, আর lock নিজে থেকে ছেড়ে যায় — তাই crash হলেও lock আটকে থাকে না। এই প্যাটার্নটি cron-এ চলা প্রতিটি script-এ ব্যবহার করুন।

> **WARNING:** Lock file হিসেবে শুধু ফাইলের অস্তিত্ব পরীক্ষা করা (`[[ -f lock ]]`) নিরাপদ নয় — দুটি instance একই মুহূর্তে পরীক্ষা করে দুজনেই lock নেই ভাবতে পারে (race condition)। সবসময় `flock` ব্যবহার করুন, যা kernel-এর atomic lock ব্যবহার করে। আর `/var/lock/` -এ লিখতে permission লাগতে পারে — প্রয়োজনে `/tmp/` -এ lock ফাইল রাখুন, তবে `mktemp` -এর মতো অনুমান করা যায় না এমন নাম ব্যবহার করুন।

১৩.৩ Process পরীক্ষা: চলছে কি না জানা

একটি service বা process চলছে কি না, তা পরীক্ষা করার কয়েকটি উপায় আছে। সবচেয়ে নির্ভরযোগ্য: `systemctl is-active --quiet nginx` — service চললে exit status ০ দেয়। Process-এর নাম দিয়ে পরীক্ষা করতে `pgrep -x nginx` — process চললে তার PID দেখায়, না চললে কিছু দেখায় না। `ps aux | grep nginx` লিখলে একটি পরিচিত সমস্যা হয়: `grep` নিজেই ফলাফলে আসে — কারণ `grep nginx` কমান্ডটির ভেতরেও `nginx` লেখা আছে। সমাধান: `pgrep` ব্যবহার করা, বা `grep "[n]ginx"` লেখা — regex-এর ভেতরে `[n]` লিখলে `grep` -এর নিজের লাইনটি মেলে না।

Process-কে থামাতে `kill` ব্যবহার করা হয় — প্রথমে `kill PID` (TERM signal — process-কে ভদ্রভাবে থামতে বলা), আর তা না হলে `kill -9 PID` (KILL signal — জোর করে থামানো)। `kill -9` প্রথম উপায় নয় — তা process-কে cleanup করার সুযোগ দেয় না, ফলে ফাইল নষ্ট হতে পারে। সবসময় প্রথমে সাধারণ `kill`, অপেক্ষা করুন, তারপর প্রয়োজনে `kill -9` ।

১৩.৪ Configuration Management: মান আলাদা ফাইলে

Chapter 10-এ শেখা নিয়মটি এখানে পূর্ণ রূপ পায়: configuration code থেকে আলাদা রাখুন। একটি সহজ পদ্ধতি: `key=value` আকারের একটি configuration ফাইল, যা script `source` করে। যেমন `backup.conf` -এ লেখা থাকবে `BACKUP_DIR="/var/backups/technova"` আর `RETENTION_DAYS=30` , আর script-এ লেখা থাকবে `source /etc/technova/backup.conf` । তবে সতর্কতা: `source` করা ফাইলের যেকোনো লাইন কমান্ড হিসেবে চলবে — তাই configuration ফাইলটি শুধু বিশ্বস্ত উৎস থেকে আসতে হবে, আর তার permission শুধু মালিকের লেখার অধিকার থাকতে হবে।

আরও নিরাপদ পদ্ধতি: configuration ফাইলটি শুধু পড়া (parse করা), `source` না করা — যেমন `grep` ও `cut` দিয়ে মান বের করা। তবে এটি জটিল, তাই ছোট script-এ `source` পদ্ধতিই চলে — শুধু ফাইলটির permission ও মালিকানা যাচাই করুন। Chapter 18-এ `.env` ও YAML configuration বিস্তারিত আসবে।

১৩.৫ ShellCheck: আপনার Script-এর পরীক্ষক

ShellCheck হলো Bash script-এর জন্য একটি static analysis টুল — সে script চালায় না, শুধু পড়ে ভুল ও ঝুঁকি ধরিয়ে দেয়। Install করতে: `sudo apt update && sudo apt install shellcheck` । চালাতে: `shellcheck script.sh` । ShellCheck ধরিয়ে দেয়: quote না করা variable (SC2086), `[[ ]]` -এর বদলে `[ ]` ব্যবহার, `rm` -এর সঙ্গে অquote করা variable, backtick-এর বদলে `$(...)` ব্যবহারের পরামর্শ, আর আরও শত শত নিয়ম। প্রতিটি সতর্কতার একটি code থাকে (যেমন SC2086) — তা দিয়ে অনলাইনে বিস্তারিত ব্যাখ্যা পাওয়া যায়।

ShellCheck-এর প্রতিটি সতর্কতাই ভুল নয় — কখনো কখনো আপনি ইচ্ছাকৃতভাবে একটি নিয়ম ভাঙেন। তখন সেই লাইনের উপরে `# shellcheck disable=SC2086` লিখে তা বন্ধ করা যায় — তবে কারণটি মন্তব্যে লিখুন। নিয়ম: **প্রতিটি script commit করার আগে ShellCheck চালান, আর সব সতর্কতা হয় ঠিক করুন, নয়তো কারণসহ disable করুন।** Git-এর pre-commit hook-এ ShellCheck বসানো যায় — Chapter 23-এ তা শেখানো হবে।

```
$ shellcheck backup.sh

In backup.sh line 12:
rm $BACKUP_DIR/*.old
   ^-- SC2086: Double quote to prevent globbing and word splitting.

Did you mean:
rm "$BACKUP_DIR"/*.old
```

ব্যাখ্যা (illustrative output): ShellCheck বলছে, `$BACKUP_DIR` quote করা হয়নি — ফাঁকা জায়গা থাকলে তা ভেঙে যাবে। তার পরামর্শ: `rm "$BACKUP_DIR"/*.old` — variable-টি quote করা, কিন্তু `*.old` glob-টি quote-এর বাইরে রাখা, যাতে তা ফাইলের নামে বদলাতে পারে। এই পার্থক্যটি সূক্ষ্ম কিন্তু গুরুত্বপূর্ণ: quote variable-কে রক্ষা করে, glob-কে রক্ষা করলে তা আর কাজ করে না।

Production Script-এর পরীক্ষার ধাপ

লেখা → bash -n (syntax) → shellcheck (ভুল ও ঝুঁকি) → bash -x (trace) → ছোট পরিবেশে পরীক্ষা → Git commit → cron-এ বসানো

প্রতিটি ধাপ একটি ভিন্ন ধরনের ভুল ধরে। কোনো ধাপই বাদ দেওয়া যায় না — বিশেষ করে ধ্বংসাত্মক কমান্ড (rm, mv) থাকলে।

১৩.৬ Security: Command Injection

Command injection হলো সবচেয়ে বিপজ্জনক shell নিরাপত্তা ত্রুটি: ব্যবহারকারীর দেওয়া লেখা কমান্ড হিসেবে চলে যাওয়া। যেমন `eval "ls ${USER_INPUT}"` — ব্যবহারকারী `USER_INPUT`-এ `; rm -rf /` লিখলে `eval` তা কমান্ড হিসেবে চালিয়ে দেবে। নিয়ম: **কখনো `eval` ব্যবহার করবেন না ব্যবহারকারীর দেওয়া লেখার উপর।** একই ঝুঁকি আছে অquote করা variable-এ: `rm $FILE` যেখানে `FILE="a b"` হলে দুটি ফাইল মুছে যাবে। আর `$(...)`-এর ভেতরে ব্যবহারকারীর লেখা বসানোও বিপজ্জনক — যেমন `$(echo "${USER_INPUT}")` নয়, বরং `"${USER_INPUT}"` সরাসরি ব্যবহার করুন।

নিরাপদ প্যাটার্নগুলো: (১) সব variable quote করুন। (২) `eval` এড়িয়ে চলুন — তার বদলে array ব্যবহার করুন: `ARGS=("--file" "${FILE}")` আর `command "${ARGS[@]}"`। (৩) ব্যবহারকারীর input যাচাই করুন (Chapter 12-এর validation)। (৪) ফাইলের নামের জন্য `--` ব্যবহার করুন: `rm -- "${FILE}"` — `--` মানে "এর পরের সবকিছু ফাইলের নাম, option নয়" — ফাইলের নাম `-rf` দিয়ে শুরু হলেও তা option হিসেবে নেওয়া হবে না। (৫) Script-কে প্রয়োজনের চেয়ে বেশি অধিকার দিয়ে চালাবেন না — `sudo` শুধু প্রয়োজনে।

বিপজ্জনক প্যাটার্ন	কেন বিপজ্জনক	নিরাপদ বিকল্প
`eval "${USER_INPUT}"`	Input কমান্ড হিসেবে চলে	eval ব্যবহার করবেন না

`rm $FILE`	Space থাকলে ভেঙে যায়	`rm -- "${FILE}"`
`curl ... | bash`	না পড়ে কোড চালানো	ফাইল নামিয়ে পড়ে, তারপর চালান
`chmod 777 script.sh`	সবাই বদলাতে পারে	`chmod 755 script.sh`
`/tmp/work.txt` ব্যবহার	Symlink attack	`mktemp` ব্যবহার করুন
Lock ছাড়া cron script	দুটি instance একসঙ্গে চলে	`flock` ব্যবহার করুন

১৩.৭ Script Testing: পরীক্ষা ছাড়া বিশ্বাস নয়

Bash script-এর পরীক্ষা (testing) মানে: script-টি প্রত্যাশিত পরিস্থিতিতে প্রত্যাশিত কাজ করে কি না, তা যাচাই করা। পদ্ধতি: (১) **Manual test** — script-টি একটি নিরাপদ পরিবেশে (test server বা container) চালিয়ে output দেখা। (২) **Edge case test** — ভুল argument, নেই এমন ফাইল, ভরা disk — প্রতিটি অবস্থায় script কী করে, তা পরীক্ষা করা। (৩) **Idempotency test** — script-টি দুবার চালিয়ে একই ফল হয় কি না, তা দেখা। (৪) **ShellCheck** — প্রতিবার চালানো। (৫) **Bats (Bash Automated Testing System)** — Bash-এর জন্য একটি testing framework, যা দিয়ে test লেখা যায়: `@test "backup creates a file" { run ./backup.sh; [ "$status" -eq 0 ]; }`। Chapter 23-এ Bats ও pytest দিয়ে পরীক্ষা বিস্তারিত আসবে।

পরীক্ষার একটি সোনালি নিয়ম: **production-এ চালানোর আগে সবসময় একটি নিরাপদ পরিবেশে পরীক্ষা করুন।** একটি test VM বা Docker container-এ script চালিয়ে দেখুন — ভুল হলে ক্ষতি শুধু test পরিবেশের। আর ধ্বংসাত্মক কমান্ড পরীক্ষা করার সময় প্রথমে `echo` দিয়ে দেখুন কোন ফাইলগুলো প্রভাবিত হবে (dry-run) — Chapter 22-এ dry-run mode বিস্তারিত আসবে।

১৩.৮ নির্ভরযোগ্য Automation-এর প্যাটার্ন

Part II-এর সব শিক্ষা একসঙ্গে জুড়ে একটি নির্ভরযোগ্য script-এর checklist তৈরি করা যায়:

1. Shebang + `set -euo pipefail` + `trap cleanup EXIT` — script-এর শুরুতে।
2. Configuration উপরে, এক জায়গায় — শক্ত করে লেখা মান নয়।
3. Input validation — argument, ফাইল, সংখ্যা পরীক্ষা।
4. Lock file (`flock`) — cron-এ চলা script-এ।
5. অস্থায়ী ফাইলে `mktemp` — অনুমানযোগ্য নাম নয়।
6. সব variable quote করা, `eval` নয়, `--` দিয়ে ফাইলের নাম।
7. Log function — সময়সহ, ভুল বার্তা stderr-এ।
8. Idempotency — বারবার চালানো নিরাপদ।
9. ShellCheck — প্রতিবার commit-এর আগে।
10. Git-এ সংরক্ষণ — প্রতিটি পরিবর্তন commit করা।

এই checklist-টি Chapter 22-এর toolkit ও Project 01-এর মূল্যায়নের ভিত্তি। একটি script এই দশটি নিয়ম মেনে চললে তাকে production-এ চালানো যায় — নয়তো তা শুধু একটি পরীক্ষামূলক script, production automation নয়।

১৩.৯ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
mktemp	নিরাপদ, অনুমান করা যায় না এমন অস্থায়ী ফাইল বানানো
flock / Lock file	একই script দুবার চলা ঠেকানোর তালা
Race condition	দুটি কাজ একই মুহূর্তে চললে অপ্রত্যাশিত ফল
ShellCheck	Bash script পরীক্ষা করে ভুল ধরিয়ে দেওয়া টুল
Command injection	ব্যবহারকারীর লেখা কমান্ড হিসেবে চলে যাওয়া
Symlink attack	অস্থায়ী ফাইলের নামে ফাঁদ পেতে গুরুত্বপূর্ণ ফাইল বদলানো
Dry-run	আসলে না করে, কী করা হতো তা দেখানো
Idempotency	বারবার চালালেও একই নিরাপদ ফল

REAL WORLD: চট্টগ্রামের একটি company-র deploy script-এ ShellCheck চালানো হতো না। একদিন একটি অquote করা variable-এর কারণে deploy script ভুল directory মুছে ফেলল — আর restore করতে লেগে গেল ছয় ঘণ্টা। এরপর দলটি একটি নিয়ম করল: ShellCheck ছাড়া কোনো script production-এ যাবে না, আর প্রতিটি script-এর code review হবে GitHub pull request-এর মাধ্যমে (Chapter 06)। Tool ও প্রক্রিয়া — দুটোই নিরাপত্তা।

১৩.১০ হাতে-কলমে অনুশীলন

LAB: Part II-এর সমাপনী script — একটি নির্ভরযোগ্য backup script।

1. ShellCheck install করুন: `sudo apt update && sudo apt install shellcheck`।
2. `~/technova/scripts/safe-backup.sh` লিখুন — এতে থাকবে: shebang, `set -euo pipefail`, `mktemp -d` + `trap cleanup EXIT`, `flock` দিয়ে lock, argument validation (backup করার directory-টি আছে কি), log function, আর `tar` দিয়ে backup নেওয়া।
3. `shellcheck safe-backup.sh` চালিয়ে সব সতর্কতা ঠিক করুন।
4. Script-টি দুবার একসঙ্গে চালান (দুটি terminal-এ) — দ্বিতীয়টি "already running" বার্তা দিয়ে থামে কি না, তা দেখুন।
5. Script-টি দুবার পরপর চালান — দুবারই সফল হয় কি না, তা দেখুন (idempotency)।
6. Git-এ commit করুন: `git add safe-backup.sh`, `git commit -m "Add safe backup script with locking and cleanup"`।

উদাহরণ আউটপুট (illustrative):

```
$ shellcheck safe-backup.sh
$ ./safe-backup.sh /var/www
[INFO] Lock acquired.
```

```
[INFO] Backup created: /var/backups/technova/www-2026-11-08.tar.gz
[INFO] Backup finished.
```

যাচাই: ShellCheck-এর কোনো output না আসা মানে কোনো সতর্কতা নেই। Backup ফাইলটি `tar -tzf` দিয়ে পরীক্ষা করুন — ভেতরের ফাইলগুলো ঠিক আছে কি না।

১৩.১১ Common Mistakes ও Troubleshooting

- **ভুল:** `/tmp/work.txt`-এর মতো অনুমানযোগ্য অস্থায়ী ফাইল। সমাধান: `mktemp`।
- **ভুল:** Lock file হিসেবে শুধু ফাইলের অস্তিত্ব পরীক্ষা করা। সমাধান: `flock`।
- **ভুল:** `eval` দিয়ে ব্যবহারকারীর input চালানো। সমাধান: `eval` ব্যবহার করবেন না, array ব্যবহার করুন।
- **ভুল:** ShellCheck-এর সতর্কতা উপেক্ষা করা। সমাধান: প্রতিটি সতর্কতা হয় ঠিক করুন, নয়তো কারণসহ disable করুন।
- **ভুল:** `rm $FILE` — quote নেই। সমাধান: `rm -- "${FILE}"`।
- **সমস্যা:** `flock` lock পাচ্ছে না — হয়তো আগের instance এখনো চলছে, বা lock ফাইলের permission ঠিক নেই। `ps aux | grep script-name` দিয়ে পরীক্ষা করুন।

১৩.১২ Security Considerations

এই অধ্যায়ের নিরাপত্তা নীতিগুলো একসঙ্গে: (১) কখনো `eval` নয় — ব্যবহারকারীর লেখা কমান্ড হতে পারে। (২) সব variable quote করুন — ফাঁকা জায়গা ও বিশেষ অক্ষর থেকে রক্ষা পেতে। (৩) অস্থায়ী ফাইলে `mktemp` — symlink attack ঠেকাতে। (৪) Lock-এ `flock` — race condition ঠেকাতে। (৫) ফাইলের নামে `--` — option injection ঠেকাতে। (৬) Permission 755, কখনো 777 নয়। (৭) Script-এ গোপন তথ্য নয় — environment variable বা configuration ফাইলে। (৮) ইন্টারনেট থেকে নামানো script না পড়ে চালাবেন না। এই আটটি নিয়ম মনে রাখলে Bash script-এর বেশিরভাগ নিরাপত্তা ঝুঁকি এড়ানো যায়।

১৩.১৩ Best Practices

- প্রতিটি script commit করার আগে `bash -n` ও `shellcheck` চালান।
- cron-এ চলা প্রতিটি script-এ `flock` ব্যবহার করুন।
- অস্থায়ী ফাইলে সবসময় `mktemp` ব্যবহার করুন।
- Script-কে test পরিবেশে পরীক্ষা করুন, production-এ সরাসরি নয়।
- Part II-এর দশটি নিয়মের checklist প্রতিটি script-এ মেলান।

PRO TIP: একটি script লেখার পর নিজেকে তিনটি প্রশ্ন করুন: "এটি দুবার চললে কী হবে?", "মাঝপথে থামলে কী হবে?", "ভুল input এলে কী হবে?" — তিনটির উত্তর নিরাপদ হলে script-টি production-এর উপযোগী। এই তিনটি প্রশ্নই Part II-এর সারাংশ।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `mktemp` কেন ব্যবহার করা হয়? ক. ফাইল দ্রুত বানাতে খ. অনুমান করা যায় না এমন নিরাপদ অস্থায়ী ফাইল বানাতে গ. ফাইল মুছতে ঘ. ফাইল বড় করতে
2. `flock`-এর কাজ কী? ক. ফাইল মুছে ফেলা খ. একই script দুবার চলা ঠেকানো গ. ফাইল encrypt করা ঘ. Network বন্ধ করা
3. Command injection কী? ক. কমান্ড ভুল লেখা খ. ব্যবহারকারীর লেখা কমান্ড হিসেবে চলে যাওয়া গ. কমান্ড মুছে ফেলা ঘ. কমান্ড দ্রুত চালানো
4. ShellCheck কী করে? ক. Script চালায় খ. Script পড়ে ভুল ও ঝুঁকি ধরিয়ে দেয় গ. Script মুছে ফেলে ঘ. Script-কে দ্রুত করে
5. `rm -- "${FILE}"`-এ `--`-এর অর্থ কী? ক. ফাইলটি দুবার মুছবে খ. এর পরের সবকিছু ফাইলের নাম, option নয় গ. ফাইলটি backup নেবে ঘ. ফাইলটি লুকাবে

১৩.১৪ Practical Assignment

Assignment 13.1 (Advanced): Part II-এর সব জ্ঞান ব্যবহার করে TechNova-র জন্য একটি সম্পূর্ণ `server-maintenance.sh` toolkit লিখুন: disk, memory ও service পরীক্ষা (Chapter 09–10), log বিশ্লেষণ (Chapter 11), error handling ও trap (Chapter 12), আর `mktemp`, `flock` ও ShellCheck-পরিষ্কার কোড (Chapter 13)। Script-টি Git-এ commit করুন, আর একটি সংক্ষিপ্ত README লিখুন — কীভাবে চালাতে হয়, কী কী পরীক্ষা করে, আর ভুল হলে কী করতে হয়।

১৩.১৫ Chapter Summary

Production script লেখা মানে ভুল, পুনরাবৃত্তি ও আক্রমণ — তিনটির বিরুদ্ধে প্রস্তুতি। `mktemp` দিয়ে নিরাপদ অস্থায়ী ফাইল, `flock` দিয়ে lock, `pgrep` দিয়ে process পরীক্ষা, configuration আলাদা ফাইলে, ShellCheck দিয়ে প্রতিটি script পরীক্ষা, আর `eval` এড়িয়ে ও সব variable quote করে command injection ঠেকানো। Part II এখানে শেষ — আপনার কাছে এখন Bash দিয়ে নির্ভরযোগ্য automation লেখার সব সরঞ্জাম আছে। Part III-এ Python দিয়ে আরও বড় automation তৈরি করা হবে।

Revision Checklist

- ☐ `mktemp` ও `trap cleanup EXIT` একসঙ্গে ব্যবহার করতে পারি।
- ☐ `flock` দিয়ে lock file লিখতে পারি।
- ☐ ShellCheck চালিয়ে সতর্কতা ঠিক করতে পারি।
- ☐ Command injection ব্যাখ্যা করে `eval` এড়াতে পারি।
- ☐ `rm -- "${FILE}"` প্যাটার্নটি জানি।
- ☐ Part II-এর দশটি নিয়মের checklist মনে রাখতে পারি।

PART III • DEVOPS ENGINEERING MASTERY

Python for DevOps

DevOps-এর জন্য Python — ফাইল ও প্রসেস, subprocess, API ও JSON, YAML ও কনফিগারেশন, লগিং ও নির্ভরযোগ্য অটোমেশন।

অধ্যায় ১৪ • Part III — Python for DevOps

Chapter 14: Python Fundamentals for Automation — অটোমেশনের জন্য পাইথন মৌলিক বিষয়

TechNova-র জুনিয়র ইঞ্জিনিয়ার রাফি প্রতি রাতে একই কাজ করেন: ঢাকার দুটি ও চট্টগ্রামের তিনটি সার্ভারে ঢুকে সার্ভিস চলছে কি না, ডিস্কে কত জায়গা বাকি আছে এবং লগে কোনো `ERROR` এসেছে কি না দেখা। পাঁচটি সার্ভারে একই পরীক্ষা হাতে করতে চল্লিশ মিনিট লাগে, আর একদিন তিনি ভুল লগ দেখে ভুল রিপোর্ট লিখলেন। সিনিয়র ইঞ্জিনিয়ার তখন বললেন, বারবার করা কাজ মানুষের নয়, স্ক্রিপ্টের। এই অধ্যায়ে আমরা সেই স্ক্রিপ্ট লেখার ভাষা পাইথন শিখব — সার্ভার পরীক্ষা করা, রিপোর্ট বানানো, ফাইল পড়া, ভুল সামলানো এবং কাজটিকে বারবার চালানোর মতো করে গুছিয়ে লেখা।

> **বাস্তব পরিস্থিতি — TechNova-র রাতের ডিউটি**
>
> TechNova-র পাঁচটি সার্ভারে প্রতিদিন একই পরীক্ষা চলে: ওয়েব সার্ভিস চলছে কি না, ডিস্ক ব্যবহার আশি শতাংশের নিচে আছে কি না, গত এক ঘণ্টায় কোনো ভুল বার্তা এসেছে কি না। হাতে করলে চল্লিশ মিনিট, পাইথন স্ক্রিপ্টে চল্লিশ সেকেন্ড। মানুষের কাজ তখন শুধু রিপোর্ট পড়ে সিদ্ধান্ত নেওয়া। অধ্যায়ের শেষে তুমি এই ধরনের স্ক্রিপ্ট নিজে লিখতে পারবে।

১৪.১ শেখার উদ্দেশ্য — Learning Objectives

- কেন ডেভঅপসে পাইথন ব্যবহার হয় এবং ব্যাশের সঙ্গে পার্থক্য কোথায়, তা ব্যাখ্যা করতে পারবে।
- পাইথনের চলার মডেল — সোর্স কোড, বাইটকোড ও ইন্টারপ্রেটার — বুঝবে।
- ভেরিয়েবল, স্ট্রিং, লিস্ট, টাপল, সেট ও ডিকশনারি দিয়ে সার্ভারের তথ্য সাজাতে পারবে।
- শর্ত, লুপ, ফাংশন, মডিউল ও এক্সেপশন হ্যান্ডলিং দিয়ে পুনর্ব্যবহারযোগ্য স্ক্রিপ্ট লিখতে পারবে।
- ভার্চুয়াল এনভায়রনমেন্ট ও `requirements.txt` দিয়ে নির্ভরতা পরিচালনা করতে পারবে।

১৪.২ পূর্বশর্ত — Prerequisites

তোমার কম্পিউটারে Ubuntu Server 24.04 LTS ও Python 3.12.3 আছে বলে ধরা হয়েছে। টার্মিনালে ফাইল তৈরি করা ও কমান্ড চালানো জানতে হবে, ব্যাশের মৌলিক ধারণা থাকলে ভালো হয়। প্রোগ্রামিংয়ের একেবারে প্রাথমিক ধারণা — ভেরিয়েবল ও শর্ত — জানা থাকলেই চলবে। প্রতিটি উদাহরণ নিজে টাইপ করে চালাবে; শুধু পড়ে পাইথন শেখা যায় না।

১৪.৩ কেন পাইথন? — প্রথমে কারণ, তারপর পদ্ধতি

ব্যাশ ছোট কাজে অসাধারণ: দুই লাইনে লগে শব্দ খোঁজা যায়। কিন্তু কাজ বড় হলে ব্যাশ জটিল হয় — পাঁচটি সার্ভারের তথ্য সংগ্রহ করে সাজানো রিপোর্ট বানানো, কোনো সার্ভার না পেলে সেটি এড়িয়ে যাওয়া, ফলাফল ফাইলে রাখা — এসব ব্যাশে লেখা যায়, তবে পড়া ও ভুল ধরা কঠিন। পাইথনে একই কাজ পড়তে সাধারণ বাক্যের মতো লাগে, তথ্য সাজাতে ডিকশনারি ও লিস্ট আছে, ভুল সামলাতে পরিষ্কার ব্যবস্থা আছে। নিয়ম: দুই-তিন লাইনের দ্রুত কাজে ব্যাশ, বারবার চলবে ও অন্যরা পড়বে এমন কাজে পাইথন।

বিষয়	ব্যাশ	পাইথন
সবচেয়ে ভালো ব্যবহার	ছোট কমান্ড, ফাইল খোঁজা	বড় স্ক্রিপ্ট, রিপোর্ট, এপিআই
তথ্য সাজানো	মূলত টেক্সট ও সংখ্যা	লিস্ট, ডিকশনারি, সেট
ভুল সামলানো	এক্সিট কোড দিয়ে, প্রায়ই জটিল	try ও except দিয়ে পরিষ্কারভাবে
পড়তে কেমন	বড় কাজে ঘন	বড় কাজেও তুলনামূলক পরিষ্কার

পাইথন ইন্টারপ্রেটেড ভাষা: `python3 script.py` লিখলেই ইন্টারপ্রেটার কোড পড়ে সঙ্গে সঙ্গে চালায়, আলাদা কম্পাইল ধাপ নেই। ভেতরে সোর্স কোড প্রথমে বাইটকোড হয় — মানুষের পড়ার মতো নয়, আবার মেশিনের সরাসরি ভাষাও নয়, পাইথন ভার্চুয়াল মেশিনের নির্দেশের তালিকা। এরপর CPython নামের মূল ইন্টারপ্রেটার সেই বাইটকোড চালিয়ে অপারেটিং সিস্টেমের সঙ্গে কথা বলে ফাইল খোলে ও স্ক্রিনে লেখা দেখায়। টার্মিনালে শুধু `python3` লিখলে REPL চালু হয় — Read, Eval, Print, Loop: একটি লাইন লেখো, পাইথন পড়ে, মূল্যায়ন করে, ফল দেখায়, আবার অপেক্ষা করে। ছোট পরীক্ষার জন্য REPL দারুণ।

পাইথন কীভাবে চলে সোর্স কোড (hello.py) -> বাইটকোড (.pyc) -> CPython ইন্টারপ্রেটার -> অপারেটিং সিস্টেম -> ফলাফল REPL: লেখো -> চালাও -> ফল দেখো -> আবার লেখো

মূল ধারণা — ইন্টারপ্রেটেড মানে দ্রুত শুরু

ডেভঅপস স্ক্রিপ্ট সাধারণত কয়েক সেকেন্ড চলে; মূল সময় যায় ডিস্ক ও নেটওয়ার্কের অপেক্ষায়, ভাষার গতিতে নয়। পাইথনের সুবিধা দ্রুত লেখা, দ্রুত পরীক্ষা ও দ্রুত ঠিক করা — ভুল হলে কোড বদলে আবার চালাও, কম্পাইলের অপেক্ষা নেই।

```
print("Hello, TechNova!")
print("Python is ready for automation.")
```

এখানে `print` বিল্ট-ইন ফাংশন, অর্থাৎ পাইথনের সঙ্গে আসা প্রস্তুত নির্দেশ। বন্ধনীর ভেতরে ডাবল কোটেশনে লেখা `Hello, TechNova!` একটি স্ট্রিং, অর্থাৎ অক্ষরের সমষ্টি। `print` তা স্ক্রিনে দেখায়। সেমিকোলন লাগে না। মনে রাখবে, পাইথনে ইন্ডেন্টেশন শুধু সাজানো নয় — কোন লাইন কোন ব্লকের ভেতরে, তা ইন্ডেন্টেশনেই বোঝা যায়।

১৪.৪ ভেরিয়েবল, টাইপ ও স্ট্রিং

পাইথনে ভেরিয়েবল ঘোষণায় টাইপ লিখতে হয় না। `server = "web-01"` লিখলেই নামের নিচে স্ট্রিং জমা হয়; পরে একই নামে সংখ্যা বসানো যায় — এটাই ডাইনামিক টাইপিং। সুবিধা দ্রুত লেখা, ঝুঁকি ভুল টাইপ চুপচাপ ঢুকে যাওয়া। তাই নাম অর্থবহ রাখো: `s` নয়, `server_name`। সংখ্যা দুই রকম: পূর্ণ সংখ্যা `int` যেমন `5`, দশমিক `float` যেমন `82.5`। `type()` দিয়ে টাইপ দেখা যায়।

```
server_name = "web-01"
disk_used = 82.5
is_running = True
print(server_name)
```

```
print(type(disk_used))
print(is_running)
```

এখানে `server_name` স্ট্রিং, `disk_used` ফ্লোট, `is_running` বুলিয়ান — মান `True` বা `False`। স্ট্রিংয়ে মান বসানোর পরিষ্কার উপায় f-string: স্ট্রিংয়ের আগে `f` লিখে কোঁকড়া বন্ধনীতে ভেরিয়েবল বসাও।

```
name = "web-01"
used = 82.5
report = f"Server {name} disk used: {used}%"
print(report)
```

এখানে `f"..."` ফরম্যাট করা স্ট্রিং। `{name}` জায়গায় `web-01` ও `{used}` জায়গায় `82.5` বসে রিপোর্টের লাইন তৈরি হয়। পুরোনো `%` বা `format()` পদ্ধতিও আছে, তবে রিপোর্টে f-string সবচেয়ে পড়তে সহজ।

১৪.৫ তথ্য সাজানো: লিস্ট, টাপল, সেট, ডিকশনারি

পাঁচটি সার্ভারের তথ্য রাখতে কাঠামো লাগে। লিস্ট ক্রমযুক্ত ও পরিবর্তনযোগ্য: `["web-01", "web-02"]`। টাপল একই রকম, তবে পরিবর্তন করা যায় না: `("dhaka", 8080)` — স্থির ঠিকানা বা সেটিংসে ভালো। সেট অনন্য মানের সংগ্রহ: `{"web-01", "db-01"}`, ডুপ্লিকেট নিজে থেকে বাদ যায়। ডিকশনারি চাবি-মান জোড়া: `{"name": "web-01", "disk": 82.5}` — প্রতিটি বৈশিষ্ট্য নাম দিয়ে পাওয়া যায়।

কাঠামো	পরিবর্তনযোগ্য?	কখন ব্যবহার করবে
লিস্ট	হ্যাঁ	সার্ভারের তালিকা, লগের লাইন
টাপল	না	হোস্ট ও পোর্টের মতো স্থির জোড়া
সেট	হ্যাঁ	অনন্য আইপি বা ত্রুটির ধরন
ডিকশনারি	হ্যাঁ	এক সার্ভারের সব বৈশিষ্ট্য

```
servers = [
    {"name": "web-01", "disk": 82.5, "running": True},
    {"name": "db-01", "disk": 61.0, "running": True},
]
for server in servers:
    print(f"{server['name']} disk {server['disk']}% running {server['running']}")
```

এখানে `servers` ডিকশনারির লিস্ট। `for server in servers` প্রতিটি ডিকশনারিকে একবার করে `server` নামে নেয়। `server['name']` দিয়ে চাবি ধরে মান পড়া হয়। `append()` লিস্টে যোগ করে, `get()` ডিকশনারি থেকে নিরাপদে মান পড়ে, `keys()` সব চাবি দেয়।

১৪.৬ শর্ত, লুপ ও কম্প্রিহেনশন

`if`, `elif` ও `else` দিয়ে সিদ্ধান্ত নেওয়া হয়: ডিস্ক আশি শতাংশের বেশি হলে সতর্কবার্তা, না হলে স্বাভাবিক বার্তা। `for` লুপ সংগ্রহের প্রতিটি আইটেমে কাজ করে, `while` লুপ শর্ত সত্য থাকা পর্যন্ত চলে — তবে থামার শর্ত নিশ্চিত করো, নইলে লুপ অনন্তকাল চলবে। কম্প্রিহেনশন লিস্ট তৈরির সংক্ষিপ্ত রূপ: `[s["name"] for s in servers if s["disk"] > 80]` লিখলে শুধু বেশি ডিস্ক ব্যবহার করা সার্ভারের নাম পাওয়া যায়; `>` মানে বড়, `if` অংশটি ছাঁকনি।

১৪.৭ ফাংশন, মডিউল ও এক্সেপশন

বারবার করা কাজ ফাংশনে ভরে রাখো। `def` দিয়ে ফাংশন শুরু হয়, প্যারামিটার বন্ধনীতে আসে, `return` ফলাফল ফেরত দেয়। `*args` অনেকগুলো অবস্থানগত আর্গুমেন্ট একসঙ্গে নেয়, `**kwargs` নামযুক্ত আর্গুমেন্ট নেয়। ফাংশনের নিচে ডকস্ট্রিং লিখে কাজটি বোঝাও।

```
def check_disk(name, used, limit=80):
    """Return True if disk use is over the limit."""
    return used > limit

print(check_disk("web-01", 82.5))
print(check_disk("db-01", 61.0, limit=70))
```

এখানে `limit=80` ডিফল্ট মান। প্রথম কলে `82.5 > 80` সত্য বলে `True`, দ্বিতীয় কলে `61.0 > 70` মিথ্যা বলে `False` ফেরে। মডিউল পাইথন ফাইলের সংগ্রহ: `import os` অপারেটিং সিস্টেমের কাজ দেয়, `import sys` ইন্টারপ্রেটারের তথ্য দেয়। `if __name__ == "__main__":` মানে ফাইলটি সরাসরি চালানো হচ্ছে, অন্য ফাইল থেকে ইমপোর্ট করা হচ্ছে না। ফাইল না পেলে প্রোগ্রাম যাতে না থামে, তার জন্য `try` ও `except` ব্যবহার করো।

```
try:
    file = open("servers.txt")
    text = file.read()
    print(text)
except FileNotFoundError:
    print("servers.txt not found")
finally:
    print("check finished")
```

এখানে `try` ব্লকে ফাইল খোলার চেষ্টা হয়। ফাইল না থাকলে `FileNotFoundError` ধরা পড়ে `except` ব্লক চলে। `finally` সবসময় চলে, `else` শুধু ভুল না হলে চলে। প্রয়োজনে `raise ValueError("bad value")` দিয়ে নিজে ভুল ছুড়ে দেওয়া যায়।

১৪.৮ ভার্চুয়াল এনভায়রনমেন্ট, পিপ ও রক্ষণাবেক্ষণযোগ্য কোড

প্রতিটি প্রজেক্টের নির্ভরতা আলাদা রাখতে ভার্চুয়াল এনভায়রনমেন্ট ব্যবহার করো, নইলে এক প্রজেক্টের লাইব্রেরি আরেক প্রজেক্ট ভেঙে দেবে। নিচের কমান্ডগুলো টার্মিনালে চালাও, এগুলো পাইথন কোড নয়।

```
python3 -m venv venv
source venv/bin/activate
pip install requests
pip freeze > requirements.txt
pip install -r requirements.txt
deactivate
```

এখানে `python3 -m venv venv` নামে একটি আলাদা পরিবেশ তৈরি করে। `source venv/bin/activate` সেই পরিবেশ চালু করে — এরপর `pip` দিয়ে যা ইনস্টল করবে, তা শুধু এই প্রজেক্টে থাকবে। `pip freeze > requirements.txt` ইনস্টল করা প্যাকেজের তালিকা ফাইলে লেখে, `pip install -r requirements.txt` সেই তালিকা দেখে অন্য মেশিনে একই পরিবেশ তৈরি করে। `deactivate` পরিবেশ বন্ধ করে। PEP 8 হলো পাইথন কোড লেখার স্টাইল নির্দেশিকা: চার স্পেস ইন্ডেন্ট, অর্থবহ নাম, ফাংশন ছোট রাখা, প্রতি লাইন খুব লম্বা না করা। টাইপ হিন্ট দিয়ে ইঙ্গিত

দেওয়া যায়: `def check_disk(name: str, used: float) -> bool:` — এখানে `str` মানে স্ট্রিং, `float` মানে দশমিক, `-> bool` মানে ফেরত মান বুলিয়ান। হিন্ট বাধ্যতামূলক নয়, তবে পড়তে ও ভুল ধরতে সাহায্য করে।

> **Pro Tip — ছোট ফাংশন, একটি কাজ**
>
> একটি ফাংশন একটি কাজ করবে: ডিস্ক পরীক্ষা করা আলাদা ফাংশন, রিপোর্ট বানানো আলাদা ফাংশন। এতে ভুল খুঁজে পাওয়া সহজ হয় এবং একই ফাংশন অন্য স্ক্রিপ্টেও ব্যবহার করা যায়।

১৪.৯ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Interpreter	কোড পড়ে সঙ্গে সঙ্গে চালায় এমন প্রোগ্রাম
Bytecode	সোর্স কোডের মধ্যবর্তী নির্দেশের রূপ
Variable	নামের নিচে মান রাখার জায়গা
Function	নাম দেওয়া, বারবার ব্যবহারযোগ্য কাজের ব্লক
Module	পুনর্ব্যবহারযোগ্য পাইথন কোডের ফাইল
Exception	চলার সময় ঘটে যাওয়া ভুল, যা ধরা যায়
Virtual Environment	প্রজেক্টের জন্য আলাদা প্যাকেজের পরিবেশ

> **Lab — সার্ভার স্বাস্থ্য রিপোর্ট বানাও**
>
> 1. `mkdir ~/server-check && cd ~/server-check` দিয়ে ফোল্ডার তৈরি করো।
> 2. `python3 -m venv venv` ও `source venv/bin/activate` দিয়ে পরিবেশ চালু করো।
> 3. ১৪.৫-এর ইনভেন্টরি স্ক্রিপ্টটি `check.py` ফাইলে লিখে ডিস্ক আশি শতাংশের বেশি হলে `WARNING` ছাপানোর `if` যোগ করো।
> 4. `python3 check.py` চালিয়ে আউটপুট দেখো, তারপর একটি সার্ভারের ডিস্ক মান বদলে আবার চালাও।
> 5. `pip freeze > requirements.txt` দিয়ে পরিবেশ সংরক্ষণ করে `deactivate` করো।

১৪.১০ প্রত্যাশিত ফলাফল ও যাচাই

উদাহরণ আউটপুট — ইনভেন্টরি স্ক্রিপ্ট চালালে নিচের মতো লাইন আসা উচিত। সংখ্যা তোমার মান অনুযায়ী বদলাবে।

```
web-01 disk 82.5% running True
db-01 disk 61.0% running False
```

যাচাই করো: দুটি সার্ভারের নামই আছে কি না, ডিস্কের সংখ্যা ঠিক আছে কি না, এবং `check_disk` প্রথম ক্ষেত্রে `True` ও দ্বিতীয় ক্ষেত্রে `False` দিচ্ছে কি না। `servers.txt` না থাকলে `servers.txt not found` ও `check finished` — দুটি বার্তাই আসা উচিত।

সাধারণ ভুল ও ডিবাগিং

- ইন্ডেন্টেশন ভুল: ব্লকের লাইনগুলো একই পরিমাণ ফাঁকা জায়গা দিয়ে শুরু করো; ট্যাব ও স্পেস মেশাবে না।
- মিউটেবল ডিফল্ট আর্গুমেন্ট: `def f(items=[])` লিখবে না — ডিফল্ট লিস্ট সব কলে ভাগ হয়ে যায়।
- `==` বনাম `is`: মান তুলনায় `==`, একই অবজেক্ট কি না তা দেখতে `is`।
- ভার্চুয়াল পরিবেশ চালু করতে ভুলে যাওয়া: ইনস্টল করার আগে প্রম্পটে `(venv)` দেখে নাও।
- `pip` বনাম `pip3`: সবসময় `python3 -m pip` ব্যবহার করলে সঠিক পাইথনেই ইনস্টল হয়।

নিরাপত্তা — Security

অবিশ্বস্ত ইনপুটে কখনো `eval()` ব্যবহার করবে না — এটি যেকোনো কোড চালিয়ে দিতে পারে। পাসওয়ার্ড, টোকেন বা এপিআই চাবি কোডে হার্ডকোড করবে না; পরিবেশ ভেরিয়েবল বা সিক্রেট ম্যানেজার ব্যবহার করো এবং সিক্রেটযুক্ত ফাইল গিটে তুলবে না।

১৪.১১ সেরা অভ্যাস — Best Practices

ছোট ফাংশন লেখো, অর্থবহ নাম দাও, প্রতিটি স্ক্রিপ্টের শুরুতে তার কাজ এক লাইনে লেখো, ভুলের বার্তা পরিষ্কার রাখো এবং `requirements.txt` গিটে রাখো। কোড চালানোর আগে REPL-এ ছোট অংশ পরীক্ষা করো, আর বড় স্ক্রিপ্ট ভাগ করে লেখো — একটি ফাইলে সব কিছু নয়।

১৪.১২ অনুশীলনী — MCQs

১. পাইথনে সোর্স কোড প্রথমে কীসে বদলায়? ক) মেশিন কোড খ) বাইটকোড গ) বাইনারি ফাইল ঘ) এইচটিএমএল

২. কোন কাঠামোতে চাবি-মান জোড়ায় তথ্য রাখা হয়? ক) লিস্ট খ) টাপল গ) সেট ঘ) ডিকশনারি

৩. ডিস্ক ব্যবহার সীমার বেশি কি না যাচাই করতে কোন ফাংশনটি সঠিক? ক) `return used > limit` খ) `return used < limit` গ) `return used == limit` ঘ) `return used + limit`

৪. ফাইল না পাওয়ার ভুল ধরতে কোনটি ব্যবহার করা হয়? ক) `if` খ) `for` গ) `try` ও `except` ঘ) `import`

৫. প্রজেক্টের নির্ভরতা আলাদা রাখতে কোনটি করা হয়? ক) ভার্চুয়াল এনভায়রনমেন্ট তৈরি খ) `eval()` ব্যবহার গ) পাসওয়ার্ড কোডে লেখা ঘ) সব প্যাকেজ সিস্টেমে ইনস্টল করা

১৪.১৩ স্বাধীন অ্যাসাইনমেন্ট

তোমার নিজের তিনটি সার্ভারের একটি ইনভেন্টরি স্ক্রিপ্ট লেখো: প্রতিটি সার্ভারের নাম, ডিস্ক ব্যবহার ও চলমান অবস্থা ডিকশনারিতে রাখবে, ডিস্ক আশি শতাংশের বেশি হলে সতর্কবার্তা ছাপাবে, ফলাফল একটি টেক্সট ফাইলে লিখবে এবং ফাইল না পেলে ভুল সামলাবে।

১৪.১৪ সারাংশ ও পুনরালোচনা

এই অধ্যায়ে আমরা শিখলাম কেন ডেভঅপসে পাইথন দরকার, পাইথন কীভাবে সোর্স কোড থেকে বাইটকোড হয়ে চলে, ভেরিয়েবল ও তথ্যের কাঠামো, শর্ত ও লুপ, ফাংশন ও মডিউল, এক্সেপশন হ্যান্ডলিং, ভার্চুয়াল এনভায়রনমেন্ট এবং পড়তে সহজ কোড লেখার অভ্যাস। পরের অধ্যায়ে এই ভিত্তির ওপর দাঁড়িয়ে আমরা ফাইল, লগ ও সিস্টেম কমান্ড নিয়ে কাজ করব।

Revision Checklist

- ☐ পাইথন বনাম ব্যাশের ব্যবহার ব্যাখ্যা করতে পারি
- ☐ সোর্স কোড, বাইটকোড ও CPython-এর ভূমিকা বলতে পারি
- ☐ লিস্ট, টাপল, সেট ও ডিকশনারির পার্থক্য জানি
- ☐ ফাংশন, মডিউল ও try/except লিখতে পারি
- ☐ venv তৈরি করে requirements.txt বানাতে পারি
- ☐ eval() ও হার্ডকোড করা সিক্রেট এড়িয়ে চলি

অধ্যায় ১৫ • Part III — Python for DevOps

Chapter 15: Files, Directories and System Operations — ফাইল, ডিরেক্টরি ও সিস্টেম অপারেশন

TechNova-র পরিস্থিতি: TechNova-র সার্ভারে লগ জমে `/var/log/technova` প্রায় ভরে গেছে। একজন ইঞ্জিনিয়ার হাতে ফাইল মুছতে গিয়ে অডিট লগ মুছে ফেললেন। ব্যাকআপ ছিল না, নিয়মও ছিল না। এই অধ্যায়ে Python দিয়ে ফাইল নিরাপদে, যাচাই করে ও স্বয়ংক্রিয়ভাবে পরিচালনা করা শিখব।

১৫.১ শেখার উদ্দেশ্য

- `pathlib`: পথ তৈরি, `glob` / `rglob` দিয়ে ফাইল খোঁজা।
- `with open()`: নিরাপদে পড়া ও লেখা, `utf-8` ব্যবহার।
- `shutil`, `tarfile`, `zipfile`: কপি, আর্কাইভ, ব্যাকআপ।
- `stat`: আকার, সময়, অনুমতি পড়া; `os.chmod`: অনুমতি বদলানো।
- সময়-চিহ্নযুক্ত ব্যাকআপ, retention, atomic write; path traversal ও symlink-এর বিপদ চেনা।

১৫.২ পূর্বশর্ত

অধ্যায় ১৪-এর Python জানতে হবে: ভেরিয়েবল, `for`, `if`, ফাংশন, `try` / `except`, `datetime`। Linux পথ জানতে হবে: absolute path যেমন `/home/nahid/app.log` মূল থেকে শুরু হয়; relative path যেমন `logs/app.log` বর্তমান ডিরেক্টরি থেকে শুরু হয়। অনুমতি জানতে হবে: `rwx`, মালিক, গ্রুপ, অন্যরা। পরিবেশ: Ubuntu 24.04 LTS, Python 3.12.3। অনুশীলন হোম ডিরেক্টরির পরীক্ষামূলক ফোল্ডারে করবে, `/var/log` বা `/etc`-এ নয়।

১৫.৩ কেন ফাইলই লিনাক্সের ভাষা

Linux-এ প্রায় সবকিছুই ফাইল: কনফিগারেশন, লগ, ডেটা, এমনকি প্রসেস ও হার্ডওয়্যারের তথ্যও `/proc` ও `/dev`-এ ফাইলের মতো দেখায়। একে বলে everything-is-a-file। ফাইল পড়তে ও লিখতে পারলেই সিস্টেমের সঙ্গে কথা বলা যায়। DevOps-এর কাজের বড় অংশই ফাইল নিয়ে: লগ পড়ে সমস্যা খোঁজা, কনফিগারেশন বদলানো, ব্যাকআপ রাখা, পুরোনো ফাইল পরিষ্কার করা। ফাইল দুই ধরনের: text ও binary। Text ফাইলে মানুষের পড়ার মতো অক্ষর থাকে, যেমন `.log`, `.txt`, `.py`, `.json`। Binary ফাইলে যন্ত্রের ভাষা থাকে, যেমন ছবি ও আর্কাইভ। Text ফাইলে `utf-8` সবচেয়ে নিরাপদ, কারণ বাংলা ও ইংরেজি সবই এতে ঠিক থাকে। Absolute path সবসময় `/` দিয়ে শুরু হয় ও যেকোনো জায়গা থেকে একই ফাইলকে নির্দেশ করে। Relative path বর্তমান ডিরেক্টরির উপর নির্ভর করে, তাই গুরুত্বপূর্ণ ফাইলে absolute path নিরাপদ। `.` মানে বর্তমান ডিরেক্টরি, `..` মানে এক ধাপ উপরে, `~` মানে হোম।

মূল নীতি: পড়া নিরাপদ, লেখা সতর্কতার কাজ, মোছা সবচেয়ে বিপজ্জনক। যা ফিরিয়ে আনা যায় না, সেখানে dry-run, ব্যাকআপ ও যাচাই — তিনটিই বাধ্যতামূলক।

১৫.৪ pathlib

`pathlib`-এর `Path` পথ নিয়ে কাজের আধুনিক উপায়। `/` দিয়ে পথ জোড়া যায়, যেমন `Path("/home/nahid") / "logs" / "app.log"`। `glob("*.log")` শুধু ওই ডিরেক্টরিতে খোঁজে, `rglob("*.log")` সব সাব-ডিরেক্টরি ঘুরে খোঁজে। `mkdir(parents=True)` মাঝের ডিরেক্টরিসহ তৈরি করে, `exist_ok=True` থাকলে আগে থেকে থাকলে ভুল ধরে না।

Path মেথড	কাজ
`exists()`	আছে কি না জানায়
`is_file()` / `is_dir()`	ফাইল না ডিরেক্টরি
`mkdir(parents=True, exist_ok=True)`	ডিরেক্টরি তৈরি করে
`glob()` / `rglob()`	প্যাটার্নে ফাইল খোঁজে
`iterdir()`	ভেতরের সব দেয়
`name` / `suffix` / `parent`	নাম, এক্সটেনশন, মা-ডিরেক্টরি
`rename()` / `unlink()`	নাম বদলায়, ফাইল মোছে

কোড ১: pathlib ইনভেন্টরি

```
from pathlib import Path

base = Path.home() / "technova_demo" / "logs"
base.mkdir(parents=True, exist_ok=True)
for name in ["app.log", "error.log", "access.log"]:
    target = base / name
    target.write_text("demo log line\n", encoding="utf-8")
print("Base:", base)
print("Exists:", base.exists())
print("Is directory:", base.is_dir())
for item in sorted(base.iterdir()):
    print(item.name, item.suffix, item.stat().st_size)
for found in sorted(base.rglob("*.log")):
    print(found.name, found.parent.name)
```

ব্যাখ্যা: `from pathlib import Path` — `Path` আনা হচ্ছে। `Path.home()` — হোম দেয়। `/` — পথ জোড়ে। `mkdir(parents=True, exist_ok=True)` — না থাকলে মা-ডিরেক্টরিসহ তৈরি করে, থাকলে ভুল ধরে না। `for name in [...]` — তিন নামে লুপ চালায়। `target = base / name` — পূর্ণ পথ বানায়। `write_text("demo log line\n", encoding="utf-8")` — ফাইল তৈরি করে এক লাইন লেখে; `\n` মানে নতুন লাইন। `exists()` — আছে কি না জানায়। `is_dir()` — ডিরেক্টরি কি না জানায়। `sorted(base.iterdir())` — ভেতরের আইটেম সাজিয়ে দেয়। `item.name` — নাম, `item.suffix` — এক্সটেনশন, `item.stat().st_size` — আকার বাইটে। `rglob("*.log")` — সব সাব-ডিরেক্টরি ঘুরে `.log` খোঁজে।

১৫.৫ ফাইল পড়া ও লেখা

সঠিক উপায় `with open(...) as f:`। ব্লক শেষ হলে Python নিজে থেকেই ফাইল বন্ধ করে, ভুল হলেও। মোড: `"r"` পড়া, `"w"` নতুন করে লেখা — পুরোনো লেখা মুছে যায়, `"a"` শেষে যোগ করা, `"x"` শুধু নতুন ফাইল, `"b"` binary। সবসময় `encoding="utf-8"` দাও। বড় ফাইল একবারে `read()` দিয়ে মেমোরিতে এনো না; `for line in f:` দিয়ে লাইন ধরে পড়ো। `readlines()` ছোট ফাইলে ঠিক, বড় ফাইলে বিপজ্জনক। `write()` একটি স্ট্রিং লেখে, `writelines()` স্ট্রিংয়ের তালিকা লেখে।

কোড ২: নিরাপদ লগ রিডার

```
from pathlib import Path

log_path = Path.home() / "technova_demo" / "logs" / "app.log"
error_count = 0
total_count = 0
with open(log_path, "r", encoding="utf-8") as log_file:
    for line in log_file:
        total_count = total_count + 1
        clean = line.strip()
        if "ERROR" in clean:
            error_count = error_count + 1
            print("Found error:", clean)
print("Total lines:", total_count)
print("Error lines:", error_count)
```

ব্যাখ্যা: `log_path` — লগ ফাইলের পথ। `error_count = 0` ও `total_count = 0` — গণনা শূন্য থেকে শুরু। `with open(log_path, "r", encoding="utf-8") as log_file:` — পড়ার মোডে খুলছে, `utf-8` ব্যবহার করছে, ফাইল অবজেক্টের নাম `log_file`। `for line in log_file:` — লাইন ধরে পড়ছে, পুরো ফাইল মেমোরিতে নিচ্ছে না। `total_count = total_count + 1` — প্রতি লাইনে এক বাড়াচ্ছে। `clean = line.strip()` — দুই পাশের ফাঁকা জায়গা সরাচ্ছে। `if "ERROR" in clean:` — `ERROR` শব্দ আছে কি না দেখছে। থাকলে `error_count` বাড়ছে ও লাইন ছাপা হচ্ছে। ব্লক শেষে ফাইল নিজে থেকেই বন্ধ হচ্ছে। শেষ দুটি `print` — মোট ও ভুলের লাইনের সংখ্যা দেখাচ্ছে।

> **সতর্কতা:** `"w"` মোডে খুললেই পুরোনো সব লেখা মুছে যায়। লেখার আগে পথ যাচাই করো, প্রয়োজনে আগে ব্যাকআপ নাও। `"a"` নিরাপদ, কারণ তা পুরোনো লেখা রেখে শেষে যোগ করে।

১৫.৬ ডিরেক্টরি অপারেশন

তৈরি করতে `Path.mkdir()`, ভেতরে দেখতে `iterdir()`, সাব-ডিরেক্টরিসহ ঘুরতে `rglob("*")`। নাম বদলাতে `rename()`, একটিমাত্র ফাইল মুছতে `unlink()`, খালি ডিরেক্টরি মুছতে `rmdir()`, ভেতরে ফাইলসহ পুরো ডিরেক্টরি মুছতে `shutil.rmtree()` — এটি অত্যন্ত বিপজ্জনক, মুছে ফেলা ডেটা আর ফেরে না। মোছার আগে তিন প্রশ্ন: পথটি কি ঠিক? ভেতরে কি গুরুত্বপূর্ণ কিছু আছে? ব্যাকআপ কি নেওয়া হয়েছে? কখনোই `/`, `~` বা হোম ডিরেক্টরির উপর `rmtree()` চালাবে না।

১৫.৭ ফাইল মেটাডেটা — stat

`Path.stat()` ফাইলের তথ্য দেয়: `st_size` আকার বাইটে, `st_mtime` শেষ পরিবর্তনের সময় সেকেন্ডে, `st_uid` মালিকের নম্বর, `st_mode` অনুমতি ও ধরন। `st_mtime`-কে পড়ার মতো সময়ে বদলাতে `datetime.fromtimestamp()`

ব্যবহার করো। এ দিয়ে পুরোনো ফাইল খুঁজে আর্কাইভ করা যায়, যেমন ৩০ দিনের বেশি পুরোনো লগ। `stat.S_ISDIR()` ও `stat.S_ISREG()` দিয়ে ধরন যাচাই করা যায়, আর `stat.S_IRUSR`, `stat.S_IWUSR`, `stat.S_IXUSR` দিয়ে মালিকের পড়া, লেখা ও চালানোর অনুমতি বোঝা যায়।

১৫.৮ shutil

`shutil.copy()` শুধু বিষয়বস্তু কপি করে, `shutil.copy2()` বিষয়বস্তুর সঙ্গে মেটাডেটাও কপি করে — ব্যাকআপে `copy2()` ভালো। `shutil.move()` ফাইল বা ডিরেক্টরি সরায়। `shutil.rmtree()` পুরো ডিরেক্টরি মোছে। `shutil.disk_usage()` ডিস্কের মোট, ব্যবহৃত ও খালি জায়গা জানায়, ডিস্ক ভরে যাওয়ার আগেই সতর্ক হতে সাহায্য করে।

shutil ফাংশন	কাজ	কখন ব্যবহার করবে
`shutil.copy(src, dst)`	শুধু বিষয়বস্তু কপি করে	সাধারণ কপি
`shutil.copy2(src, dst)`	বিষয়বস্তু ও মেটাডেটা কপি করে	ব্যাকআপ, অডিট
`shutil.move(src, dst)`	ফাইল বা ফোল্ডার সরায়	অন্য ডিরেক্টরিতে পাঠাতে
`shutil.rmtree(path)`	পুরো ডিরেক্টরি মোছে	শুধু যাচাইয়ের পর
`shutil.disk_usage(path)`	মোট, ব্যবহৃত ও খালি জায়গা দেয়	ডিস্ক পর্যবেক্ষণে

১৫.৯ অস্থায়ী ফাইল — tempfile

`TemporaryDirectory()` অস্থায়ী ডিরেক্টরি তৈরি করে, `with` ব্লক শেষে নিজে থেকেই মুছে যায়। `NamedTemporaryFile()` নামসহ অস্থায়ী ফাইল তৈরি করে। এগুলো atomic write-এর জন্য কাজে লাগে: প্রথমে অস্থায়ী ফাইলে লেখো, যাচাই করো, তারপর আসল ফাইলের জায়গায় নাম বদলে বসাও। এতে লেখার মাঝখানে বিদ্যুৎ চলে গেলেও আসল ফাইল অর্ধেক লেখা থাকে না।

১৫.১০ ফাইল অনুমতি — chmod ও umask

`os.chmod(path, 0o600)` দিয়ে অনুমতি বদলানো যায়। `0o600` মানে শুধু মালিক পড়তে ও লিখতে পারবে — সংবেদনশীল ফাইলের জন্য সঠিক। `0o644` মানে মালিক পড়তে ও লিখতে পারবে, অন্যরা শুধু পড়তে পারবে। `0o755` মানে মালিক সব করতে পারবে, অন্যরা পড়তে ও চালাতে পারবে — স্ক্রিপ্টের জন্য উপযুক্ত। `stat.S_IRUSR` মালিকের পড়া, `stat.S_IWUSR` লেখা, `stat.S_IXUSR` চালানোর অনুমতি; `|` দিয়ে যোগ করা যায়। `umask` হলো নতুন ফাইল তৈরির সময় স্বয়ংক্রিয়ভাবে কেটে নেওয়া অনুমতি; সাধারণত নতুন ফাইল `0o644` ও নতুন ডিরেক্টরি `0o755` হয়। সংবেদনশীল ফাইল তৈরির পর সবসময় স্পষ্টভাবে `os.chmod()` দিয়ে অনুমতি ঠিক করো।

> **প্রো টিপ:** ব্যাকআপ ও কনফিগারেশন ফাইলে কখনোই `0o777` দেবে না। এর মানে সবাই পড়তে, লিখতে ও চালাতে পারবে — সার্ভারে এটি নিরাপত্তার বড় ফাঁক। সবসময় সবচেয়ে কম অনুমতি দাও।

১৫.১১ আর্কাইভ — tarfile, zipfile ও gzip

অনেক ফাইল একসঙ্গে সংরক্ষণ ও পাঠাতে আর্কাইভ ব্যবহার হয়। `tarfile` দিয়ে `.tar.gz` বানানো যায়, Linux সার্ভারে সবচেয়ে প্রচলিত। `zipfile` দিয়ে `.zip` বানানো যায়, Windows ব্যবহারকারীর সঙ্গে ভাগ করতে সুবিধাজনক। `gzip`

দিয়ে একটি ফাইল সংকুচিত করা যায়। আর্কাইভ খোলার সময় সতর্ক থাকো: ভেতরের পথে `../` থাকলে তা ডিরেক্টরির বাইরে ফাইল লিখে দিতে পারে, একে বলে zip-slip আক্রমণ। অচেনা আর্কাইভ খোলার আগে প্রতিটি সদস্যের নাম যাচাই করো।

কোড ৩: shutil ব্যাকআপ ও retention

```
from pathlib import Path
from datetime import datetime, timedelta
import shutil

source = Path.home() / "technova_demo" / "logs"
backup_root = Path.home() / "technova_demo" / "backups"
backup_root.mkdir(parents=True, exist_ok=True)
stamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
backup_dir = backup_root / stamp
backup_dir.mkdir(parents=True, exist_ok=True)
for item in source.iterdir():
    if item.is_file():
        shutil.copy2(item, backup_dir / item.name)
print("Backup created:", backup_dir)
cutoff = datetime.now() - timedelta(days=7)
for old in backup_root.iterdir():
    if old.is_dir():
        born = datetime.fromtimestamp(old.stat().st_mtime)
        if born < cutoff:
            shutil.rmtree(old)
            print("Removed old backup:", old)
```

ব্যাখ্যা: `from datetime import datetime, timedelta` — সময়ের ক্লাস আনা হচ্ছে। `source` — ব্যাকআপ নেওয়ার মূল ডিরেক্টরি। `backup_root` — সব ব্যাকআপ জমা রাখার ডিরেক্টরি। `datetime.now().strftime("%Y-%m-%d_%H-%M-%S")` — বর্তমান সময়কে `2026-11-08_23-16-21`-এর মতো স্ট্রিংয়ে বদলায়; `%Y` বছর, `%m` মাস, `%d` দিন, `%H` ঘণ্টা, `%M` মিনিট, `%S` সেকেন্ড। `backup_dir = backup_root / stamp` — সময়ের নামে নতুন ফোল্ডার। `for item in source.iterdir():` — প্রতিটি আইটেম ঘুরছে। `if item.is_file():` — শুধু ফাইল হলে কপি করছে। `shutil.copy2(item, backup_dir / item.name)` — মেটাডেটাসহ কপি করছে। `cutoff = datetime.now() - timedelta(days=7)` — সাত দিন আগের সময়। `datetime.fromtimestamp(old.stat().st_mtime)` — ফোল্ডারের শেষ পরিবর্তনের সময়কে তারিখে বদলাচ্ছে। `if born < cutoff:` — সাত দিনের বেশি পুরোনো কি না দেখছে; `<` হলো ছোট-চিহ্ন। পুরোনো হলে `shutil.rmtree(old)` মুছে দিচ্ছে।

কোড ৪: tarfile আর্কাইভ

```
from pathlib import Path
import tarfile

source = Path.home() / "technova_demo" / "logs"
archive = Path.home() / "technova_demo" / "logs_backup.tar.gz"
with tarfile.open(archive, "w:gz") as tar:
    tar.add(source, arcname="logs")
print("Archive created:", archive)
print("Archive size:", archive.stat().st_size)
with tarfile.open(archive, "r:gz") as tar:
```

```
    for member in tar.getmembers():
        print(member.name, member.size)
```

ব্যাখ্যা: `import tarfile` — আর্কাইভের মডিউল। `source` — আর্কাইভ করার ডিরেক্টরি। `archive` — তৈরি হতে যাওয়া `.tar.gz` ফাইলের পথ। `with tarfile.open(archive, "w:gz") as tar:` — লেখার মোডে খুলছে; `w` মানে লেখা, `gz` মানে gzip সংকোচন। `tar.add(source, arcname="logs")` — পুরো ডিরেক্টরি যোগ করছে; `arcname="logs"` মানে ভেতরে নাম হবে শুধু `logs`। `archive.stat().st_size` — আকার বাইটে। দ্বিতীয় `with tarfile.open(archive, "r:gz") as tar:` — পড়ার মোডে খুলছে; `r` মানে পড়া। `tar.getmembers()` — ভেতরের সব সদস্যের তালিকা দেয়। `member.name` — নাম, `member.size` — আকার।

কোড ৫: tempfile দিয়ে atomic write

```
from pathlib import Path
import tempfile
import os

target = Path.home() / "technova_demo" / "config.txt"
target.parent.mkdir(parents=True, exist_ok=True)
with tempfile.NamedTemporaryFile("w", encoding="utf-8", dir=target.parent, delete=False) as
temp_file:
    temp_path = Path(temp_file.name)
    temp_file.write("app_name=technova\n")
    temp_file.write("debug=false\n")
os.replace(temp_path, target)
print("Written safely:", target)
print(target.read_text(encoding="utf-8"))
```

ব্যাখ্যা: `import tempfile` — অস্থায়ী ফাইলের মডিউল। `import os` — অপারেটিং সিস্টেমের মডিউল। `target` — আসল ফাইলের পথ। `target.parent.mkdir(parents=True, exist_ok=True)` — মা-ডিরেক্টরি তৈরি করছে। `tempfile.NamedTemporaryFile("w", encoding="utf-8", dir=target.parent, delete=False)` — একই ডিরেক্টরিতে নামসহ অস্থায়ী ফাইল তৈরি করছে; `"w"` লেখার মোড, `dir=target.parent` মানে আসল ফাইলের পাশেই থাকবে, `delete=False` মানে ব্লক শেষে নিজে থেকে মুছবে না। `temp_path = Path(temp_file.name)` — নামকে `Path` বানাচ্ছে। দুটি `write()` — কনফিগারেশন লিখছে। `os.replace(temp_path, target)` — অস্থায়ী ফাইলকে আসল ফাইলের জায়গায় এক ধাপে বসিয়ে দিচ্ছে, তাই মাঝখানে ফাইল অর্ধেক লেখা থাকে না। `target.read_text(encoding="utf-8")` — ফাইল পড়ে দেখাচ্ছে লেখা ঠিক আছে কি না।

১৫.১২ ব্যাকআপ অটোমেশন ও নিরাপদ অপারেশন

ভালো ব্যাকআপের নকশা: উৎস যাচাই করো, সময়-চিহ্নযুক্ত নতুন ফোল্ডারে কপি করো, ফাইলের সংখ্যা ও আকার মিলিয়ে যাচাই করো, আর্কাইভ করো, পুরোনো ব্যাকআপ মুছে retention মানো। নীতি হতে পারে: দৈনিক ব্যাকআপ ৭ দিন, সাপ্তাহিক ৪ সপ্তাহ, মাসিক ৩ মাস। প্রতিটি ব্যাকআপের সঙ্গে লগ লিখে রাখো — কখন হয়েছে, কয়টি ফাইল কপি হয়েছে, ভুল হয়েছে কি না।

ব্যাকআপ প্রবাহ: source -> temp copy -> verify -> archive -> rotate -> log উৎস -> সাময়িক কপি -> সংখ্যা ও আকার মিলিয়ে যাচাই -> .tar.gz -> ৭ দিনের পুরোনো ব্যাকআপ মোছা -> লগ লেখা

নিরাপদ অপারেশন: যাচাই ছাড়া মুছবে না। পথটি আসলেই সেটি কি না `resolve()` দিয়ে যাচাই করো। symlink-এ সতর্ক থাকো: `is_symlink()` দিয়ে দেখো পথটি আসল ফাইল না অন্য জায়গার লিংক; অনুসরণ করে মুছলে আসল ডেটা হারাতে পারো। race condition মনে রাখো: যাচাইয়ের পর ও মুছতে যাওয়ার মাঝখানে ফাইল বদলে যেতে পারে, তাই গুরুত্বপূর্ণ কাজে `try` ও `except` দিয়ে ভুল ধরা ও লগ রাখা জরুরি। dry-run মোড রাখো: প্রথমে শুধু ছাপাও কী মুছতে বা কপি করতে যাচ্ছ, তারপর আসল কাজ করো।

> **TechNova-র বাস্তব পরিস্থিতি:** TechNova-র সিলেট অফিসের সার্ভারে ডিস্ক ৯২ শতাংশ ভরে গিয়েছিল। DevOps দল একটি স্ক্রিপ্ট চালু করল যা প্রতিরাতে ২টায় ৩০ দিনের পুরোনো লগ আর্কাইভ করে, `shutil.disk_usage()` দিয়ে খালি জায়গা দেখে, আর খালি জায়গা ২০ শতাংশের নিচে নামলে সতর্কবার্তা পাঠায়। প্রথমে dry-run মোডে এক সপ্তাহ চালানো হয়েছিল, যাতে ভুল ফাইল না মোছে। এখন ডিস্ক আর হঠাৎ ভরে না, অডিট লগও নিরাপদে আর্কাইভে থাকে।

১৫.১৩ পরিভাষা

English Term	বাংলা ব্যাখ্যা
Path	ফাইল বা ডিরেক্টরির অবস্থান; `pathlib`-এর অবজেক্ট
Absolute Path	মূল ডিরেক্টরি থেকে শুরু হওয়া পূর্ণ পথ
Relative Path	বর্তমান ডিরেক্টরি থেকে হিসাব করা পথ
Glob	প্যাটার্ন দিয়ে ফাইল খোঁজা, যেমন `*.log`
Encoding	অক্ষরকে বাইটে বদলানোর নিয়ম; `utf-8` সবচেয়ে নিরাপদ
Metadata	ফাইল সম্পর্কে তথ্য: আকার, সময়, মালিক, অনুমতি
Symlink	অন্য ফাইলের দিকে নির্দেশ করা বিশেষ ফাইল
Atomic Write	এক ধাপে সম্পূর্ণ লেখা; মাঝখানে ফাইল নষ্ট হয় না
Retention	ব্যাকআপ কতদিন রাখা হবে তার নীতি
Archive	অনেক ফাইল একসঙ্গে সংরক্ষিত সংকুচিত ফাইল
Race Condition	যাচাই ও কাজের মাঝখানে ফাইল বদলে যাওয়ার ঝুঁকি
Dry-Run	আসল কাজ না করে শুধু কী হবে তা দেখানো

গাইডেড অনুশীলনী — লগ পরিষ্কার ও ব্যাকআপ:

1. হোম ডিরেক্টরিতে `technova_lab` ফোল্ডার তৈরি করো, তার ভেতরে `logs` বানাও।
2. `pathlib` দিয়ে `logs`-এ পাঁচটি `.log` ফাইল তৈরি করো, দুই লাইনে `ERROR` রাখো।
3. কোড ২-এর লগ রিডার চালিয়ে মোট লাইন ও `ERROR` লাইন গুনে দেখো।
4. কোড ৩-এর ব্যাকআপ চালিয়ে সময়-চিহ্নযুক্ত ফোল্ডার তৈরি করো, ফাইলের সংখ্যা মিলিয়ে যাচাই করো।
5. কোড ৪ দিয়ে `.tar.gz` আর্কাইভ বানাও, সদস্যদের তালিকা ছাপাও।
6. কোড ৫ দিয়ে কনফিগারেশন ফাইলে atomic write করো, ফাইলটি পড়ে দেখো।
7. `shutil.disk_usage()` দিয়ে খালি জায়গা ছাপাও, একটি ফাইলের অনুমতি `0o600` করো।
8. dry-run স্ক্রিপ্ট লেখো যা ৭ দিনের পুরোনো ফাইল মুছতে যাচ্ছে — প্রথমে শুধু নাম ছাপাবে, মুছবে না।

১৫.১৪ প্রত্যাশিত ফলাফল ও যাচাই

কোড ১ চালালে `Base:` লাইনে পথ, `Exists: True` ও `Is directory: True` দেখাবে, তারপর তিনটি ফাইলের নাম, এক্সটেনশন ও আকার দেখাবে। কোড ২-এ মোট লাইন ও `ERROR` লাইনের সংখ্যা তোমার লেখা লগের সঙ্গে মিলবে। কোড ৩-এ `backups`-এর ভেতরে সময়ের নামে ফোল্ডার তৈরি হবে, তাতে তিনটি ফাইল কপি হবে। কোড ৪-এ আর্কাইভের আকার ও সদস্যদের তালিকা দেখাবে। কোড ৫-এ `config.txt`-এ দুটি লাইন ঠিকভাবে লেখা থাকবে। যাচাই: ফাইলের সংখ্যা গুনে দেখো, আকার মিলিয়ে দেখো, ব্যাকআপ থেকে একটি ফাইল খুলে মূল ফাইলের সঙ্গে মিলিয়ে দেখো।

```
Base: /home/nahid/technova_demo/logs
Exists: True
Is directory: True
app.log .log 15
Backup created: /home/nahid/technova_demo/backups/2026-11-08_23-16-21
Total lines: 5
Error lines: 2
```

উপরের আউটপুট নমুনা হিসেবে দেখানো হলো — এটি প্রকৃত চালানোর ফলাফল নয়। তোমার সময়, পথ ও আকার ভিন্ন হবে, কিন্তু কাঠামো একই থাকবে।

১৫.১৫ সাধারণ ভুল ও ডিবাগিং

সমস্যা সমাধান: ভুল বার্তা পুরো পড়ো। `FileNotFoundError` মানে পথ ভুল বা ফাইল নেই। `PermissionError` মানে অনুমতি নেই। `UnicodeDecodeError` মানে encoding ভুল। `IsADirectoryError` মানে ডিরেক্টরিকে ফাইল হিসেবে খুলতে চাইছ। ভুল লুকিয়ে `try` ও `except` দিয়ে চেপে যেও না — লগে লিখে রাখো।

- `with` না ব্যবহার করা: ফাইল বন্ধ করতে ভুলে গেলে ডেটা হারাতে পারে; সবসময় `with open()` ব্যবহার করো।
- Encoding না দেওয়া: বাংলা ভেঙে যায় বা `UnicodeDecodeError` হয়; সবসময় `encoding="utf-8"` দাও।
- ভুল ডিরেক্টরি মোছা: `shutil.rmtree()`-এর আগে পথ ছাপিয়ে যাচাই করো, dry-run করো, ব্যাকআপ নাও।
- `Path` বনাম `str` গুলিয়ে ফেলা: প্রয়োজনে `str(path)` বা `Path(text)` দিয়ে বদলে নাও।
- Symlink অন্ধভাবে অনুসরণ করা: `is_symlink()` দিয়ে যাচাই না করে মুছলে আসল ডেটা হারাতে পারো।

- `"w"` মোডে ভুল ফাইল খোলা: পুরোনো লেখা মুছে যায়; আগে পথ যাচাই করো, প্রয়োজনে `"a"` ব্যবহার করো।
- বড় ফাইল একবারে পড়া: `read()` দিয়ে গিগাবাইট ফাইল পড়লে মেমোরি ভরে যায়; লাইন ধরে পড়ো।

১৫.১৬ নিরাপত্তা বিবেচনা

ব্যবহারকারীর দেওয়া লেখা কখনোই সরাসরি পথ হিসেবে ব্যবহার করবে না। কেউ ফাইলের নাম হিসেবে `../../etc/passwd` দিলে তোমার স্ক্রিপ্ট সিস্টেমের গুরুত্বপূর্ণ ফাইল পড়ে বা মুছে ফেলতে পারে — একে বলে path traversal আক্রমণ। প্রতিরোধ: ব্যবহারকারীর পথকে `resolve()` দিয়ে পূর্ণ পথ বানাও, তারপর যাচাই করো সেটি অনুমোদিত মূল ডিরেক্টরির ভেতরে আছে কি না। সংবেদনশীল ফাইল সবসময় `0o600` অনুমতিতে রাখো। কোনো ফাইলে পাসওয়ার্ড, টোকেন বা গোপন চাবি লিখে রাখবে না, Git-এও এমন ফাইল পাঠাবে না। ব্যাকআপ ফোল্ডারের অনুমতিও সীমিত রাখো। অচেনা আর্কাইভ খোলার আগে সদস্যদের নাম যাচাই করো, যাতে `../` দিয়ে ডিরেক্টরির বাইরে ফাইল লেখা না হয়।

১৫.১৭ সেরা চর্চা

- সবসময় `pathlib.Path` ব্যবহার করো, হাতে স্ট্রিং জোড়া দিয়ে পথ বানিও না।
- ফাইল খুলতে সবসময় `with open()` ও `encoding="utf-8"` ব্যবহার করো।
- বড় ফাইল লাইন ধরে পড়ো, একবারে পুরো ফাইল মেমোরিতে এনো না।
- মোছার আগে dry-run করো, তালিকা ছাপাও, ব্যাকআপ নাও, তারপর মুছো।
- গুরুত্বপূর্ণ লেখায় atomic write ব্যবহার করো — অস্থায়ী ফাইলে লিখে `os.replace()` দিয়ে বসাও।
- ব্যাকআপে সময়-চিহ্ন ব্যবহার করো, retention নীতি লিখে রাখো, প্রতিটি ব্যাকআপের লগ রাখো।
- সংবেদনশীল ফাইলে `0o600` দাও, কখনোই `0o777` দিও না।
- ব্যবহারকারীর দেওয়া পথ যাচাই না করে ব্যবহার করো না; অনুমোদিত ডিরেক্টরির ভেতরে সীমাবদ্ধ রাখো।
- প্রতিটি স্ক্রিপ্টে ভুল ধরা ও লগ রাখার ব্যবস্থা রাখো।

১৫.১৮ বহুনির্বাচনী প্রশ্ন

১. `pathlib`-এ সাব-ডিরেক্টরিসহ সব `.log` ফাইল খুঁজতে কোন মেথড ব্যবহার করা হয়?
ক) `glob("*.log")` খ) `rglob("*.log")` গ) `iterdir()` ঘ) `mkdir()`

২. ফাইল নিরাপদে খুলে কাজ শেষে নিজে থেকে বন্ধ করার সঠিক উপায় কোনটি?
ক) `open()` খুলে বন্ধ না করা খ) `with open()` ব্যবহার করা গ) `read()` দিয়ে পুরো ফাইল পড়া ঘ) `close()` ভুলে যাওয়া

৩. ব্যাকআপের জন্য `shutil.copy()`-এর চেয়ে `shutil.copy2()` কেন ভালো?
ক) এটি দ্রুত কাজ করে খ) এটি মেটাডেটাসহ কপি করে গ) এটি ফাইল মুছে দেয় ঘ) এটি আর্কাইভ বানায়

৪. Atomic write-এ অস্থায়ী ফাইলে লেখার পর আসল ফাইলের জায়গায় বসাতে কোন ফাংশন ব্যবহার করা হয়?
ক) `shutil.copy()` খ) `os.replace()` গ) `os.chmod()` ঘ) `tarfile.open()`

৫. ব্যবহারকারীর দেওয়া পথে `../../etc/passwd` থাকলে কোন আক্রমণের ঝুঁকি তৈরি হয়?
ক) Path traversal খ) Race condition গ) Encoding ভুল ঘ) Disk ভরে যাওয়া

১৫.১৯ স্বাধীন অ্যাসাইনমেন্ট

TechNova-র জন্য একটি সম্পূর্ণ লগ ব্যবস্থাপনা স্ক্রিপ্ট লেখো: এটি `logs` ডিরেক্টরি থেকে ৩০ দিনের পুরোনো `.log` ফাইল খুঁজে বের করবে, সেগুলোর সময়-চিহ্নযুক্ত `.tar.gz` ব্যাকআপ বানাবে, ব্যাকআপ যাচাই করবে, ৭ দিনের পুরোনো ব্যাকআপ মুছে দেবে এবং প্রতিটি ধাপের লগ একটি ফাইলে লিখবে। স্ক্রিপ্টে dry-run মোড থাকতে হবে, যাতে আসল কাজ না করে শুধু কী করা হবে তা দেখা যায়।

১৫.২০ সারসংক্ষেপ ও রিভিশন চেকলিস্ট

এই অধ্যায়ে শিখলাম Linux-এ ফাইলই সিস্টেমের ভাষা, আর Python-এর `pathlib`, `open`, `shutil`, `tempfile`, `tarfile` ও `os.chmod` দিয়ে সেই ভাষায় নিরাপদে কথা বলা যায়। মূল কথা: পড়া নিরাপদ, লেখা সতর্কতার কাজ, মোছা সবচেয়ে বিপজ্জনক — তাই যাচাই, ব্যাকআপ, dry-run ও atomic write ছাড়া কোনো গুরুত্বপূর্ণ অপারেশন করবে না। TechNova-র লগ সমস্যার সমাধান এককালীন পরিষ্কার নয়, বরং প্রতিরাতে চলা একটি নির্ভরযোগ্য স্ক্রিপ্ট।

- ☐ `pathlib.Path`, `/` অপারেটর, `glob` ও `rglob` ব্যাখ্যা করতে পারি।
- ☐ `with open()`, `utf-8` ও লাইন ধরে পড়ার নিয়ম বলতে পারি।
- ☐ `mkdir`, `iterdir`, `rename`, `unlink` ও `rmtree`-এর ঝুঁকি জানি।
- ☐ `stat` দিয়ে আকার, সময় ও অনুমতি পড়তে পারি।
- ☐ `shutil.copy2`, `move`, `rmtree` ও `disk_usage` ব্যবহার করতে পারি।
- ☐ `tempfile` দিয়ে atomic write করতে পারি।
- ☐ `0o600` অনুমতি ও `umask` ব্যাখ্যা করতে পারি।
- ☐ `tarfile` দিয়ে আর্কাইভ বানাতে ও পড়তে পারি।
- ☐ সময়-চিহ্নযুক্ত ব্যাকআপ ও ৭ দিনের retention স্ক্রিপ্ট লিখতে পারি।
- ☐ Path traversal, symlink ও race condition-এর ঝুঁকি চিনি।

অধ্যায় ১৬ • Part III — Python for DevOps

Chapter 16: Processes, Subprocesses and Command Execution — প্রসেস, সাবপ্রসেস ও কমান্ড চালানো

TechNova-র ওয়েব সার্ভিস এক সকালে বন্ধ হয়ে গেল। ডিস্ক ভরেছে কি না, সার্ভিস চলছে কি না, পোর্ট খোলা আছে কি না — জানতে `systemctl status`, `df -h`, `ss -tlnp` চালাতে হয়। Python নিজে এসব জানে না, তবে বাইরের প্রোগ্রামকে ডেকে তথ্য আদায় করতে পারে। এই ডাকাডাকিই প্রসেস ও সাবপ্রসেস ব্যবস্থাপনা।

১৬.১ শেখার উদ্দেশ্য — Learning Objectives

- প্রসেস, PID, parent ও child প্রসেস ব্যাখ্যা করা।
- `subprocess.run()` দিয়ে কমান্ড চালানো, stdout/stderr ধরা ও exit code পড়া।
- `capture_output`, `text`, `timeout`, `check` ব্যবহার করা।
- `Popen` দিয়ে দীর্ঘ প্রোগ্রাম থেকে ধারাবাহিক আউটপুট পড়া।
- `poll`, `wait`, `terminate`, `kill` ও সিগন্যাল দিয়ে প্রসেস নিয়ন্ত্রণ করা।
- command injection বোঝা ও তালিকা-ভিত্তিক নিরাপদ কোড লেখা।
- `df` ও `systemctl is-active` দিয়ে স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট লেখা।

১৬.২ পূর্বশর্ত — Prerequisites

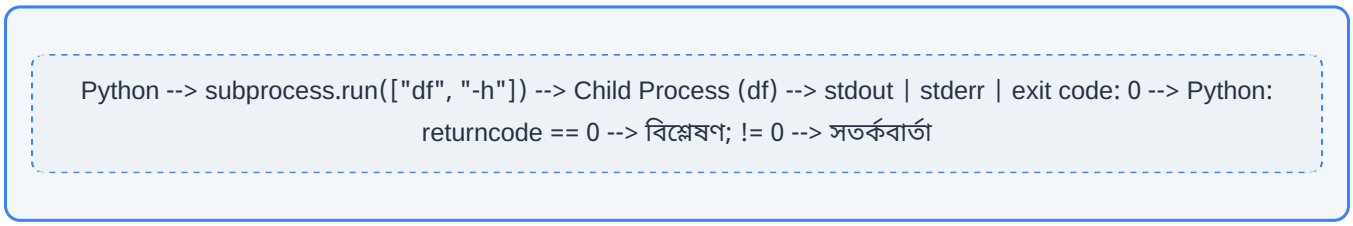
অধ্যায় ১৪-এর Python মৌলিক বিষয় ও অধ্যায় ১৫-এর ফাইল পড়া-লেখা জানতে হবে। অধ্যায় ১২-এর exit code জানতে হবে: সফল কমান্ড 0 দেয়, ব্যর্থ কমান্ড 0 ছাড়া অন্য সংখ্যা দেয়। Bash-এ `echo $?` যেমন আগের কমান্ডের exit code দেখায়, Python-এ `returncode` তেমনই কাজ করে। পরিবেশ: Ubuntu Server 24.04 LTS, Python 3.12.3, Bash 5.2, Git 2.43।

১৬.৩ প্রথম নীতি: প্রসেস কী — WHY before HOW

প্রোগ্রাম চালু হলে অপারেটিং সিস্টেম তাকে মেমোরি দেয়, একটি পরিচয় নম্বর দেয় ও CPU-তে চালায়। চালু অবস্থার এই প্রোগ্রামই প্রসেস। প্রতিটি প্রসেসের অনন্য নম্বরকে PID বলে; `ps aux` দিয়ে PID, ব্যবহারকারী ও মেমোরি দেখা যায়। একটি প্রসেস আরেকটি প্রসেস তৈরি করতে পারে: জন্মদাতা parent, নতুনটি child। Linux-এ প্রথমে `fork` parent-এর কপি তৈরি করে, তারপর `exec` সেই কপিকে নতুন প্রোগ্রামে বদলে দেয়। Python `subprocess` দিয়ে কমান্ড চালালে ভেতরে এটাই ঘটে।

Python নিজে ডিস্ক মাপে না বা সার্ভিস দেখে না, কারণ এসবের জন্য পরীক্ষিত টুল আগেই আছে: `df` ফাইলসিস্টেম জানে, `systemctl` systemd জানে, `ss` নেটওয়ার্ক সকেট জানে। Python এসব আবার লিখলে সময় নষ্ট হবে ও ভুল বাড়বে। পেশাদার পদ্ধতি হলো ভালো টুলকে ডেকে তার ফলাফল ব্যবহার করা। Python এখানে পরিচালক: সে ঠিক করে কোন কমান্ড কখন চলবে, ফলাফল পড়ে বিচার করে ও পরের পদক্ষেপ নেয়।

কমান্ডের সঙ্গে যোগাযোগের তিনটি পথ: stdin, stdout ও stderr। stdin দিয়ে কমান্ডকে তথ্য দেওয়া হয়, stdout দিয়ে সাধারণ ফলাফল আসে, stderr দিয়ে ভুল বা সতর্কবার্তা আসে। শেষে কমান্ড একটি সংখ্যা ফেরত দেয়, যাকে exit code বলে। `systemctl is-active nginx` সার্ভিস চললে `active` লিখে 0 দেয়, বন্ধ থাকলে `inactive` লিখে 3 দেয়।



১৬.৪ subprocess.run — সবচেয়ে বেশি ব্যবহৃত উপায়

`subprocess.run()` কমান্ড চালায়, শেষ হওয়া পর্যন্ত অপেক্ষা করে ও `CompletedProcess` অবজেক্ট ফেরত দেয়। এতে থাকে `args`, `returncode`, `stdout` ও `stderr`। কমান্ড সবসময় তালিকা আকারে দিন, যেমন `["df", "-h"]`: প্রথমটি প্রোগ্রামের নাম, বাকিগুলো আর্গুমেন্ট। `capture_output=True` stdout ও stderr ধরে, `text=True` আউটপুট স্ট্রিং করে, `timeout=10` 10 সেকেন্ড পর কমান্ড থামিয়ে `TimeoutExpired` তৈরি করে, `check=True` exit code 0 না হলে `CalledProcessError` তৈরি করে।

প্যারামিটার	কাজ	কখন ব্যবহার করবেন
`args`	কমান্ডের তালিকা	প্রতিবার; list আকারে
`capture_output`	stdout ও stderr ধরে	আউটপুট পড়তে হলে
`text`	আউটপুট স্ট্রিং করে	প্রায় সবসময়
`timeout`	সময় পেরোলে থামায়	আটকে যাওয়ার আশঙ্কায়
`check`	ব্যর্থতায় ব্যতিক্রম দেয়	ব্যর্থতা মানেই থামতে হলে
`cwd` / `env`	ডিরেক্টরি ও পরিবেশ বদলায়	নির্দিষ্ট জায়গা দরকার হলে

মূল ধারণা: `subprocess.run()` synchronous — কমান্ড শেষ না হওয়া পর্যন্ত Python অপেক্ষা করে। ছোট কাজের জন্য এটাই সঠিক; দীর্ঘ কাজের জন্য `Popen` লাগবে (১৬.৭)।

১৬.৫ run_command হেল্পার ও ডিস্ক পরীক্ষা

নিয়মটি একটি হেল্পার ফাংশনে রাখুন। এটি কমান্ড নেয়, চালায় ও তিনটি মান ফেরত দেয়: returncode, stdout ও stderr।

```
import subprocess


def run_command(command):
    result = subprocess.run(command, capture_output=True, text=True, timeout=15)
    return result.returncode, result.stdout, result.stderr


code, out, err = run_command(["df", "-h"])
print(code)
```

```
print(out)
print(err)
```

`import subprocess` মডিউল আনে। `def run_command(command):` ফাংশন সংজ্ঞায়িত করে; `command` কমান্ডের তালিকা। `result = subprocess.run(command, capture_output=True, text=True, timeout=15)` কমান্ড চালায়: `capture_output=True` আউটপুট ধরে, `text=True` স্ট্রিং দেয়, `timeout=15` 15 সেকেন্ডের সীমা দেয়। `return result.returncode, result.stdout, result.stderr` তিনটি মান ফেরত দেয়। `code, out, err = run_command(["df", "-h"])` ফাংশন ডাকে ও মানগুলো তিনটি ভেরিয়েবলে বসায়।

এবার `df`-এর আউটপুট বিশ্লেষণ করে দেখা যাক কোনো ফাইলসিস্টেম 80 শতাংশের বেশি ভরেছে কি না। আগে returncode দেখুন, কারণ ব্যর্থ কমান্ডের আউটপুট বিশ্লেষণ করা উচিত নয়।

```
import subprocess


def check_disk():
    result = subprocess.run(["df", "-h"], capture_output=True, text=True, timeout=10)
    if result.returncode != 0:
        print("Disk check failed")
        print(result.stderr)
        return
    print(result.stdout)
    for line in result.stdout.splitlines():
        if "%" in line:
            parts = line.split()
            usage = parts[4]
            number = int(usage.replace("%", ""))
            if number > 80:
                print("Warning: disk almost full:", line)


check_disk()
```

`def check_disk():` প্যারামিটার ছাড়া ফাংশন তৈরি করে। `subprocess.run(["df", "-h"], ...)`-এ `"df"` প্রোগ্রাম ও `"-h"` human-readable আর্গুমেন্ট। `if result.returncode != 0:` ব্যর্থতা পরীক্ষা করে; `!=` মানে সমান নয়। ব্যর্থ হলে `print(result.stderr)` ভুল দেখায় ও `return` বেরিয়ে যায়। `for line in result.stdout.splitlines():` প্রতিটি লাইন নেয়; `splitlines()` স্ট্রিংকে লাইনে ভাগ করে। `if "%" in line:` শুধু শতাংশযুক্ত লাইন নেয়, ফলে শিরোনাম বাদ যায়। `parts = line.split()` লাইনকে ফাঁকা জায়গায় ভাগ করে তালিকা বানায়। `usage = parts[4]` পঞ্চম উপাদান নেয়, যেখানে ব্যবহারের শতাংশ থাকে। `number = int(usage.replace("%", ""))` প্রথমে `replace("%", "")` শতাংশ চিহ্ন সরায়, তারপর `int()` সংখ্যায় বদলায়। `if number > 80:` সত্য হলে সতর্কবার্তা ছাপে। শেষে `check_disk()` ফাংশন ডাকে।

> **প্রো টিপ:** সবসময় প্রথমে `returncode` পরীক্ষা করুন, তারপর আউটপুট বিশ্লেষণ করুন।

১৬.৬ check=True বনাম নিজে পরীক্ষা করা ও সার্ভিসের অবস্থা

কখনো ব্যর্থতা মানেই স্ক্রিপ্ট থামাতে হবে — যেমন ব্যাকআপের আগে ডিস্ক পরীক্ষা ব্যর্থ হলে। তখন `check=True` ভালো, কারণ Python নিজেই `CalledProcessError` তৈরি করে। আবার কখনো ব্যর্থতা স্বাভাবিক তথ্য:

`systemctl is-active nginx` সার্ভিস বন্ধ থাকলে 3 দেয়, এটি দুর্ঘটনা নয়। তখন নিজে `returncode` দেখাই ভালো। নিয়ম: ব্যর্থতা অস্বাভাবিক হলে `check=True`, ব্যর্থতা অর্থবহ ফলাফল হলে নিজে পরীক্ষা করুন।

```
import subprocess


def service_status(name):
    result = subprocess.run(["systemctl", "is-active", name], capture_output=True, text=True,
timeout=10)
    status = result.stdout.strip()
    if result.returncode == 0:
        print(name, "is running:", status)
    else:
        print(name, "is not running:", status)


service_status("nginx")
service_status("postgresql")
```

`def service_status(name):` সার্ভিসের নাম প্যারামিটার হিসেবে নেয়। `subprocess.run(["systemctl", "is-active", name], ...)`-এ `name` তৃতীয় আর্গুমেন্ট। `status = result.stdout.strip()` লাইনে `strip()` আউটপুটের শুরু ও শেষের ফাঁকা জায়গা ও নতুন লাইন সরায়, ফলে `"active\n"` থেকে `"active"` হয়। `if result.returncode == 0:` সফল হলে চলছে বলে জানায়, `else:` অংশে চলছে না বলে জানায়। এখানে `check=True` দেওয়া হয়নি, কারণ সার্ভিস বন্ধ থাকা স্বাভাবিক ফলাফল।

Exit Code	অর্থ	উদাহরণ
0	সাফল্য	সার্ভিস চলছে
1	সাধারণ ব্যর্থতা	কাজ হয়নি
2	ভুল ব্যবহার	`ls` দিয়ে নেই এমন ফোল্ডার দেখা
3	সার্ভিস বন্ধ আছে	`systemctl is-active` — সার্ভিস চলছে না
127	কমান্ড পাওয়া যায়নি	ভুল নামের প্রোগ্রাম
124	timeout-এ থেমেছে	`timeout` কমান্ডের ক্ষেত্রে

১৬.৭ Timeout ও Popen দিয়ে streaming

নেটওয়ার্ক সাড়া না দিলে বা প্রোগ্রাম ইনপুটের অপেক্ষায় থাকলে কমান্ড আটকে যেতে পারে। timeout ছাড়া `run()` চালালে Python-ও চিরকাল অপেক্ষা করবে। তাই প্রতিটি কমান্ডে timeout দিন ও `TimeoutExpired` ধরুন।

```
import subprocess


def run_with_timeout(command, seconds):
    try:
        result = subprocess.run(command, capture_output=True, text=True, timeout=seconds)
        print("Finished with code:", result.returncode)
        print(result.stdout)
    except subprocess.TimeoutExpired:
```

```
        print("Command timed out after", seconds, "seconds")


run_with_timeout(["sleep", "2"], 5)
run_with_timeout(["sleep", "10"], 3)
```

`def run_with_timeout(command, seconds):` কমান্ডের তালিকা ও সময়সীমা নেয়। `try:` ব্লকে `timeout=seconds` দিয়ে কমান্ড চলে। `except subprocess.TimeoutExpired:` সময় পেরোলে তৈরি হওয়া ব্যতিক্রম ধরে। প্রথম ডাক `["sleep", "2"]` পাঁচ সেকেন্ডের সীমায় সফল হয়, দ্বিতীয় ডাক `["sleep", "10"]` তিন সেকেন্ডের সীমায় timeout হয়।

`run()` কমান্ড শেষের পর পুরো আউটপুট একসঙ্গে দেয়। কিন্তু `ping`, লগ দেখা বা ব্যাকআপ ধীরে ধীরে আউটপুট দেয়। তখন `subprocess.Popen` ব্যবহার করুন: এটি প্রোগ্রাম চালু করে, শেষের অপেক্ষা করে না, Python চলাকালীনই লাইন ধরে আউটপুট পড়তে পারে — একে streaming বলে। বড় আউটপুটে `communicate()` নিরাপদ, কারণ এটি stdout ও stderr একসঙ্গে পড়ে; শুধু একটিকে পড়লে অন্যটি ভরে গিয়ে প্রোগ্রাম আটকে যেতে পারে — একে deadlock বলে।

```
import subprocess


def stream_output():
    process = subprocess.Popen(["ping", "-c", "4", "8.8.8.8"], stdout=subprocess.PIPE,
stderr=subprocess.PIPE, text=True)
    for line in process.stdout:
        print("Ping:", line.strip())
    process.wait()
    print("Ping finished with code:", process.returncode)


stream_output()
```

`process = subprocess.Popen(...)` নতুন প্রসেস চালু করে ও `process`-এ রাখে। কমান্ড `["ping", "-c", "4", "8.8.8.8"]`-এ `"-c"` ও `"4"` মানে চারটি প্যাকেট পাঠাও। `stdout=subprocess.PIPE` stdout-কে পাইপের সঙ্গে যুক্ত করে, যাতে Python পড়তে পারে; `stderr=subprocess.PIPE` একই কাজ stderr-এর জন্য করে। `for line in process.stdout:` একবারে একটি লাইন পড়ে ও `print("Ping:", line.strip())` দিয়ে দেখায়। `process.wait()` প্রোগ্রাম শেষের অপেক্ষা করে, তারপর `process.returncode` ছাপে।

১৬.৮ প্রসেস নিয়ন্ত্রণ, পরিবেশ ও সিগন্যাল

`poll()` দেখে প্রসেস শেষ হয়েছে কি না — শেষ হলে exit code দেয়, না হলে `None` দেয়। `wait()` শেষের অপেক্ষা করে, প্রয়োজনে timeout-সহ। `terminate()` SIGTERM পাঠিয়ে ভদ্রভাবে বন্ধ হতে বলে। `kill()` SIGKILL পাঠিয়ে জোর করে বন্ধ করে, যা উপেক্ষা করা যায় না, তবে গোছানোর সুযোগ দেয় না। নিয়ম: প্রথমে `terminate()`, কিছুক্ষণ অপেক্ষা, তারপরও না থামলে `kill()`। `psutil` একটি জনপ্রিয় তৃতীয় পক্ষের লাইব্রেরি, যা দিয়ে চলমান প্রসেস, CPU, মেমোরি, ডিস্ক ও নেটওয়ার্ক জানা যায়; এটি বিল্ট-ইন নয়, তাই ইনস্টল করতে হয় ও অফিসিয়াল ডকুমেন্টেশন দেখে ব্যবহার করতে হয়। `cwd` দিয়ে কমান্ড কোন ডিরেক্টরিতে চলবে তা ঠিক করা যায়। `env` দিয়ে child প্রসেসের environment variable বদলানো যায়; তবে `env` দিলে পুরো পরিবেশ বদলে যায়, তাই সাধারণত `os.environ.copy()` দিয়ে বর্তমান পরিবেশের কপি নিয়ে তাতে নতুন মান যোগ করুন।

Linux-এ সিগন্যাল হলো প্রসেসকে পাঠানো ছোট বার্তা। SIGTERM মানে ভদ্রভাবে বন্ধ হও, SIGINT মানে Ctrl+C চাপা হয়েছে। `signal` মডিউল দিয়ে এগুলো ধরা যায়, যাতে বন্ধের আগে ফাইল বন্ধ করা, লগ লেখা বা child প্রসেস থামানো যায়। মনিটরিং স্ক্রিপ্ট হঠাৎ বন্ধ হলে তার child অনাথ হয়ে যেতে পারে, তাই সিগন্যাল ধরা গুরুত্বপূর্ণ।

```
import signal
import sys
import time


def handle_shutdown(signum, frame):
    print("Shutdown signal received:", signum)
    print("Cleaning up before exit")
    sys.exit(0)


signal.signal(signal.SIGINT, handle_shutdown)
signal.signal(signal.SIGTERM, handle_shutdown)
print("Monitoring started. Press Ctrl+C to stop.")
while True:
    time.sleep(1)
    print("Checking...")
```

`import signal`, `import sys` ও `import time` তিনটি মডিউল আনে। `def handle_shutdown(signum, frame):` ফাংশনে `signum` হলো সিগন্যালের নম্বর, `frame` হলো সিগন্যাল আসার সময় প্রোগ্রাম কোথায় ছিল। ভেতরে সিগন্যালের নম্বর ছাপে, গোছানোর বার্তা দেয় ও `sys.exit(0)` দিয়ে 0 exit code-সহ বন্ধ করে। `signal.signal(signal.SIGINT, handle_shutdown)` SIGINT এলে এই ফাংশন চালায়, পরের লাইন SIGTERM-এর জন্য একই কাজ করে। `while True:` অসীম লুপে `time.sleep(1)` এক সেকেন্ড অপেক্ষা করে ও `print("Checking...")` ছাপে, যতক্ষণ না Ctrl+C বা SIGTERM আসে।

১৬.৯ নিরাপদ কমান্ড চালানো — সবচেয়ে গুরুত্বপূর্ণ অংশ

`shell=True` দিলে কমান্ড Bash-এর মতো শেলের মাধ্যমে স্ট্রিং হিসেবে চলে। শেল `;`, `&&`, `|` বা `$()` দেখলে আলাদা কমান্ড হিসেবে চালায়। ব্যবহারকারীর ইনপুটে এমন অক্ষর থাকলে আক্রমণকারী নিজের কমান্ড চালাতে পারে — একে command injection বলে। ধরুন স্ক্রিপ্ট ফাইলের নাম দিয়ে `ls` চালায়, আর ব্যবহারকারী নাম দেয় `report.txt; echo hacked`। `shell=True` দিয়ে স্ট্রিং চালালে শেল প্রথমে `ls report.txt`, তারপর `echo hacked` চালাবে। এখানে `echo hacked` ক্ষতিকর নয়, কিন্তু একই কৌশলে বিপজ্জনক কমান্ড ঢোকানো যায়। তাই `shell=True` কখনোই সাধারণ অভ্যাস নয়। নিরাপদ উপায় হলো `shell=False`, যা Python-এর ডিফল্ট, ও তালিকা আকারে কমান্ড দেওয়া। তখন `report.txt; echo hacked` পুরোটাই একটি ফাইলের নাম, আলাদা কমান্ড নয়।

> **সতর্কতা — DO NOT USE:** নিচের কোডটি শুধু বিপদ দেখানোর জন্য। বাস্তব স্ক্রিপ্টে কখনোই ব্যবহার করবেন না।

```
import subprocess

filename = "report.txt; echo hacked"
subprocess.run("ls " + filename, shell=True)
```

`filename = "report.txt; echo hacked"` লাইনে নামের ভেতরে সেমিকোলন ও অতিরিক্ত কমান্ড আছে। `subprocess.run("ls " + filename, shell=True)` স্ট্রিং তৈরি করে শেল দিয়ে চালায়, ফলে শেল এটিকে দুটি কমান্ড হিসেবে পড়ে। এটাই injection-এর মূল কৌশল — শুধু বোঝার জন্য, ব্যবহারের জন্য নয়।

এবার নিরাপদ সংস্করণ: একই কাজ, তালিকা আকারে, `shell=False` দিয়ে।

```
import subprocess

filename = "report.txt; echo hacked"
result = subprocess.run(["ls", filename], capture_output=True, text=True, timeout=10)
print(result.returncode)
print(result.stdout)
print(result.stderr)
```

এখানে `["ls", filename]` তালিকায় `"ls"` প্রোগ্রামের নাম ও `filename` একটি মাত্র আর্গুমেন্ট। `shell=True` নেই, তাই ডিফল্ট `shell=False` চলছে। শেল স্ট্রিং ব্যাখ্যা করছে না, সেমিকোলনের পরের অংশ আলাদা কমান্ড হচ্ছে না। `ls` এমন ফাইল খুঁজবে যার নামে সেমিকোলন আছে, না পেয়ে ভুল জানাবে, কিন্তু `echo hacked` চলবে না। শেল স্ট্রিং একান্তই দরকার হলে `shlex.quote()` দিয়ে প্রতিটি আর্গুমেন্ট নিরাপদভাবে উদ্ধৃত করুন — তবে এটি শেষ উপায়, প্রথম পছন্দ নয়। ইনপুট সবসময় যাচাই করুন ও অনুমোদিত তালিকার সঙ্গে মেলান।

বাস্তব পরিস্থিতি — TechNova-র স্বাস্থ্য পরীক্ষা: TechNova-র দল প্রতিদিন একটি Python স্ক্রিপ্ট চালায়, যা `df -h` দিয়ে ডিস্ক, `systemctl is-active nginx` ও `systemctl is-active postgresql` দিয়ে সার্ভিস, আর `ss -tlnp` দিয়ে পোর্ট পরীক্ষা করে। প্রতিটি কমান্ডে timeout থাকে, ফলাফল returncode দিয়ে যাচাই হয়, সমস্যা পেলে লগ লেখা হয় ও দলকে সতর্ক করা হয়। সব কমান্ড তালিকা আকারে দেওয়া হয়, সার্ভিসের নাম নির্দিষ্ট তালিকা থেকে নেওয়া হয়, ব্যবহারকারীর ইনপুট থেকে নয়।

১৬.১০ পরিভাষা — Terminology

English Term	Bangla ব্যাখ্যা
Process	চালু অবস্থায় থাকা প্রোগ্রাম
PID	প্রতিটি প্রসেসের অনন্য পরিচয় নম্বর
Parent / Child Process	জন্মদাতা প্রসেস / তা থেকে তৈরি নতুন প্রসেস
Subprocess	Python থেকে চালানো child প্রসেস
stdout / stderr / stdin	সাধারণ ফলাফল / ভুলের বার্তা / তথ্য দেওয়ার পথ
Exit Code	কমান্ড শেষে ফেরত সংখ্যা; 0 মানে সাফল্য
Timeout	কমান্ডের সর্বোচ্চ সময়সীমা
Signal	প্রসেসকে পাঠানো বার্তা, যেমন SIGTERM বা SIGINT
Command Injection	ইনপুট দিয়ে ক্ষতিকর কমান্ড ঢুকিয়ে চালানোর আক্রমণ
Deadlock	দুটি অংশ একে অপরের অপেক্ষায় আটকে যাওয়া
PIPE	দুটি প্রসেসের মধ্যে ডেটা পাঠানোর পথ

১৬.১১ সাধারণ ভুল ও ডিবাগিং

সমস্যা ও সমাধান: নিচের ভুলগুলো নতুনদের মধ্যে সবচেয়ে বেশি দেখা যায়।

- **`shell=True` দিয়ে ইনপুট চালানো:** সেমিকোলন থাকলে অপ্রত্যাশিত কমান্ড চলে। সমাধান — তালিকা আকারে কমান্ড দিন।
- **`text=True` ভুলে যাওয়া:** আউটপুটে `b'active\n'` এর মতো বাইট দেখা যায়। সমাধান — `text=True` যোগ করুন।
- **`returncode` না দেখা:** ব্যর্থ কমান্ডেও স্ক্রিপ্ট সফল ভেবে এগোয়। সমাধান — আগে returncode পরীক্ষা করুন।
- **timeout না দেওয়া:** স্ক্রিপ্ট হঠাৎ আটকে যায়। সমাধান — প্রতিটি কমান্ডে timeout দিন ও `TimeoutExpired` ধরুন।
- **`Popen`-এ বড় আউটপুটে deadlock:** প্রোগ্রাম মাঝপথে আটকে যায়। সমাধান — বড় আউটপুটে `communicate()` ব্যবহার করুন।
- **ভুল প্রোগ্রামের নাম:** `FileNotFoundError` আসে। সমাধান — নাম ঠিক আছে ও PATH-এ আছে কি না দেখুন।
- **`capture_output=True` না দেওয়া:** `result.stdout` খালি থাকে। সমাধান — `capture_output=True` যোগ করুন।

১৬.১২ নিরাপত্তা ও সেরা অনুশীলন

অপ্রয়োজনে root হিসেবে স্ক্রিপ্ট চালাবেন না — root-এ একটি ভুলও পুরো সিস্টেমের ক্ষতি করতে পারে। least privilege মানুন: শুধু দরকারি অনুমতিটুকু দিন। ব্যবহারকারীর সব ইনপুট অবিশ্বাস করুন, যাচাই করুন ও অনুমোদিত তালিকার সঙ্গে মেলান। আউটপুটে পাসওয়ার্ড বা টোকেন থাকতে পারে, তাই লগ করার আগে ভাবুন। বাইরের উৎস থেকে পাওয়া কমান্ড সরাসরি চালাবেন না। `shell=True` ব্যবহার করবেন না; করতেই হলে `shlex.quote()` দিয়ে আর্গুমেন্ট নিরাপদ করুন।

- সবসময় কমান্ড তালিকা আকারে দিন, স্ট্রিং আকারে নয়।
- প্রতিটি কমান্ডে `capture_output=True`, `text=True` ও `timeout` দিন।
- আউটপুট বিশ্লেষণের আগে `returncode` পরীক্ষা করুন।
- কমান্ড চালানোর নিয়ম একটি হেল্পার ফাংশনে রাখুন।
- ব্যর্থতায় কী করবেন আগেই ঠিক করুন: লগ, সতর্কবার্তা, নাকি থামা।
- ছোট কাজে `run()`, দীর্ঘ কাজে `Popen` ব্যবহার করুন।
- সিগন্যাল ধরে ভদ্রভাবে বন্ধ হওয়ার ব্যবস্থা করুন।
- নতুন কমান্ড প্রথমে টার্মিনালে হাতে চালিয়ে দেখুন, তারপর স্ক্রিপ্টে যোগ করুন।

গাইডেড প্র্যাকটিক্যাল — Lab: TechNova সার্ভার স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট

1. ফোল্ডার তৈরি করুন: `mkdir -p ~/technova-health` ও `cd ~/technova-health` দিয়ে ভেতরে যান।
2. `health_check.py` ফাইলে ১৬.৫-এর `run_command` হেল্পার লিখুন।
3. ১৬.৫-এর `check_disk` যোগ করুন, যাতে ডিস্ক 80 শতাংশের বেশি ভরলে সতর্কবার্তা আসে।
4. ১৬.৬-এর `service_status` যোগ করে `nginx` ও `postgresql` পরীক্ষা করুন; না থাকলে `ssh` ব্যবহার করুন।
5. প্রতিটি ফলাফল `health.log` ফাইলে লিখুন, অধ্যায় ১৫-এর জ্ঞান ব্যবহার করে।
6. চালান: `python3 health_check.py` । আউটপুট ও লগ ফাইল দেখুন।
7. ইচ্ছা করে ভুল সার্ভিসের নাম দিন, যেমন `nginx-xyz` , ও দেখুন স্ক্রিপ্ট কীভাবে ব্যর্থতা সামলায়।
8. `sleep 10` -এ 3 সেকেন্ডের timeout দিয়ে `TimeoutExpired` বার্তা দেখুন।
9. নিরাপদ তালিকা-ভিত্তিক সংস্করণ চালিয়ে দেখুন সেমিকোলনযুক্ত নাম শুধু একটি নাম হিসেবে বিবেচিত হয়।
10. আবার চালিয়ে নিশ্চিত করুন ভুল ও সতর্কবার্তা লগে ঠিকমতো লেখা হয়েছে।

১৬.১৩ প্রত্যাশিত ফলাফল ও যাচাই — Expected Results

ল্যাব ঠিকমতো করলে নিচের মতো ফলাফল দেখা যাবে। নিচের আউটপুট নমুনা হিসেবে দেওয়া হয়েছে (illustrative) — আপনার মেশিনের ডিস্ক, সার্ভিস ও অবস্থা অনুযায়ী এগুলো আলাদা হবে।

```
0
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        20G   12G  7.2G  63% /

nginx is running: active
postgresql is not running: inactive
```

প্রথম লাইনের `0` হলো `df -h` -এর exit code, যা সাফল্য বোঝায়। ডিস্ক ব্যবহার 63 শতাংশ, তাই সতর্কবার্তা আসেনি। nginx চলছে, postgresql চলছে না। যাচাই করুন: প্রতিটি কমান্ডের returncode দেখা যাচ্ছে কি না, ডিস্ক 80 শতাংশের বেশি হলে সতর্কবার্তা আসছে কি না, ভুল সার্ভিসের নামে স্ক্রিপ্ট ভদ্রভাবে ভুল জানাচ্ছে কি না, আর লগ ফাইলে ফলাফল লেখা হচ্ছে কি না।

যাচাই checklist: ☐ `df -h` চলে ও আউটপুট দেখা যায়। ☐ `systemctl is-active` দিয়ে সার্ভিসের অবস্থা জানা যায়। ☐ timeout দিলে আটকে যাওয়া কমান্ড থামে। ☐ ভুল সার্ভিসের নামে স্ক্রিপ্ট ভদ্রভাবে ভুল জানায়। ☐ লগ ফাইলে ফলাফল লেখা হয়।

১৬.১৪ মূল্যায়ন — MCQs

১. `subprocess.run()` দিয়ে কমান্ড চালানোর সময় কোন আকার সবচেয়ে নিরাপদ?
ক) স্ট্রিং হিসেবে, `shell=True` দিয়ে
খ) তালিকা হিসেবে, যেখানে প্রথম উপাদান প্রোগ্রামের নাম

গ) ডিকশনারি হিসেবে
ঘ) সংখ্যা হিসেবে

২. কমান্ড সফলভাবে শেষ হলে তার exit code সাধারণত কত হয়?
ক) 1
খ) 2
গ) 0
ঘ) 127

৩. `text=True` না দিলে `subprocess.run()`-এর stdout সাধারণত কী আকারে পাওয়া যায়?
ক) স্ট্রিং আকারে
খ) বাইট আকারে
গ) সংখ্যা আকারে
ঘ) তালিকা আকারে

৪. ইনপুটে `; echo hacked` থাকলে `shell=True` ব্যবহার করলে কী ঘটতে পারে?
ক) ইনপুট শুধু একটি ফাইলের নাম হিসেবে বিবেচিত হয়
খ) শেল সেমিকোলনের পরের অংশকে আলাদা কমান্ড হিসেবে চালাতে পারে
গ) Python নিজে থেকে ইনপুট মুছে দেয়
ঘ) কমান্ড কখনোই চলে না

৫. দীর্ঘক্ষণ চলা প্রোগ্রাম থেকে ধারাবাহিকভাবে আউটপুট পড়তে কোনটি ব্যবহার করা উচিত?
ক) শুধু `subprocess.run()`
খ) `subprocess.Popen`
গ) `os.environ`
ঘ) `signal.signal()`

১৬.১৫ স্বাধীন অ্যাসাইনমেন্ট — Independent Assignment

TechNova-র জন্য একটি সম্পূর্ণ সার্ভার স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট লিখুন, যা `df -h` দিয়ে ডিস্ক, `systemctl is-active` দিয়ে অন্তত তিনটি সার্ভিস, আর `ss -tlnp` বা `ping` দিয়ে নেটওয়ার্ক পরীক্ষা করে। প্রতিটি কমান্ডে timeout থাকতে হবে, ফলাফল returncode দিয়ে যাচাই করতে হবে, আর সব ফলাফল তারিখ ও সময়সহ একটি লগ ফাইলে লিখতে হবে। সমস্যা পেলে পর্দায় সতর্কবার্তা দেখাবে। সব কমান্ড তালিকা আকারে দিতে হবে, `shell=True` ব্যবহার করা যাবে না।

১৬.১৬ সারসংক্ষেপ ও পুনরালোচনা — Summary and Revision Checklist

প্রসেস হলো চালু অবস্থায় থাকা প্রোগ্রাম, আর Python `subprocess` দিয়ে child প্রসেস তৈরি করে বাইরের কমান্ড চালায়। ছোট কাজে `subprocess.run()`, দীর্ঘ কাজে `Popen` ব্যবহার হয়। stdout ও stderr আলাদা ধরা যায়, exit code দিয়ে সাফল্য বোঝা যায়, timeout দিয়ে আটকে যাওয়া কমান্ড থামানো যায়, আর সিগন্যাল ধরে ভদ্রভাবে বন্ধ হওয়া যায়। সবচেয়ে গুরুত্বপূর্ণ শিক্ষা: সবসময় তালিকা আকারে কমান্ড দিন, `shell=True` এড়িয়ে চলুন, আর ইনপুট যাচাই না করে কমান্ডে ঢোকাবেন না।

- ☐ প্রসেস, PID, parent ও child প্রসেসের সংজ্ঞা বলতে পারি।
- ☐ `subprocess.run()`-এর `capture_output`, `text`, `timeout` ও `check` প্যারামিটারের কাজ বলতে পারি।
- ☐ stdout ও stderr আলাদা ধরতে ও returncode দিয়ে সাফল্য যাচাই করতে পারি।

- ☐ `check=True` কখন ব্যবহার করব আর কখন নিজে পরীক্ষা করব তা বলতে পারি।
- ☐ timeout দিয়ে আটকে যাওয়া কমান্ড সামলাতে ও `TimeoutExpired` ধরতে পারি।
- ☐ `Popen` দিয়ে ধারাবাহিক আউটপুট পড়তে ও `communicate()`-এর গুরুত্ব বলতে পারি।
- ☐ `poll`, `wait`, `terminate` ও `kill`-এর পার্থক্য বলতে পারি।
- ☐ SIGTERM ও SIGINT ধরে ভদ্রভাবে বন্ধ করতে পারি।
- ☐ command injection ব্যাখ্যা করতে ও তালিকা-ভিত্তিক নিরাপদ কোড লিখতে পারি।
- ☐ `df` ও `systemctl is-active` দিয়ে সম্পূর্ণ স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট লিখতে পারি।

অধ্যায় ১৭ • Part III — Python for DevOps

Chapter 17: APIs, HTTP Requests and JSON Processing — API, HTTP অনুরোধ ও JSON প্রক্রিয়াকরণ

TechNova-র তিনটি সার্ভারের স্বাস্থ্য প্রতি ঘণ্টায় যাচাই করে একটি Python স্ক্রিপ্ট। আগে ফলাফল ইমেইলে যেত। এক রাতে ডেটাবেস সার্ভারের ডিস্ক ভরে গেল, সার্ভিস থেমে গেল, কিন্তু সকাল পর্যন্ত কেউ ইমেইলটি পড়ল না। গ্রাহকের অভিযোগের পর দলটি জানতে পারল। স্ক্রিপ্টটি সমস্যা ঠিকই ধরেছিল, কিন্তু বার্তাটি সঠিক জায়গায় পৌঁছায়নি। এরপর সিদ্ধান্ত হলো — প্রতিবেদন সরাসরি মনিটরিং API-তে পাঠাতে হবে। ডিস্ক ৮০ শতাংশ ছাড়ালে ড্যাশবোর্ডে হলুদ সতর্কতা, ৯০ শতাংশ ছাড়ালে লাল সতর্কতা ও অন-কল প্রকৌশলীর কাছে বার্তা যাবে। একই কৌশলে GitHub API থেকে রিপোজিটরির তথ্য আনা যায়, স্ট্যাটাস পেজ আপডেট করা যায়, আর যেকোনো সেবার অবস্থা যাচাই করা যায়। এই অধ্যায়ে সেই কাজটিই শিখব।

> **বাস্তব পরিস্থিতি — TechNova**
>
> TechNova ঢাকার একটি ছোট দল। তাদের ওয়েব, ডেটাবেস ও ব্যাকআপ সার্ভারের প্রতিবেদন আগে টেক্সট ফাইলে জমত। কেউ নিয়মিত ফাইলটি খুলত না। এখন স্ক্রিপ্ট প্রতিবেদন API-তে পাঠায়। ফলে সমস্যা ধরা পড়ে মিনিটের মধ্যে, মানুষের হস্তক্ষেপ ছাড়াই। এই অধ্যায়ের দক্ষতাই এই পরিবর্তন এনেছে।

১৭.১ শেখার উদ্দেশ্য

- API, ক্লায়েন্ট-সার্ভার ও REST ধারণা নিজের ভাষায় ব্যাখ্যা করা।
- GET, POST, PUT, PATCH, DELETE-এর ব্যবহার ও আইডেমপোটেন্সি বোঝা।
- স্ট্যাটাস কোড পড়ে সাফল্য, অনুরোধের ভুল ও সার্ভারের সমস্যা চেনা।
- API কী ও Bearer টোকেন পরিবেশ চলক থেকে নিরাপদে ব্যবহার করা।
- `urllib.request` ও `requests` দিয়ে GET ও POST পাঠানো।
- JSON পার্স করা, অনুপস্থিত কী `.get` দিয়ে সামলানো, পেজিনেশন করা।
- টাইমআউট, রিট্রাই, ব্যাকঅফ ও 429 হার সীমা সামলানো।
- 4xx ও 5xx ত্রুটি আলাদা করে সামলানো, টোকেন ফাঁস না করে লগ করা।

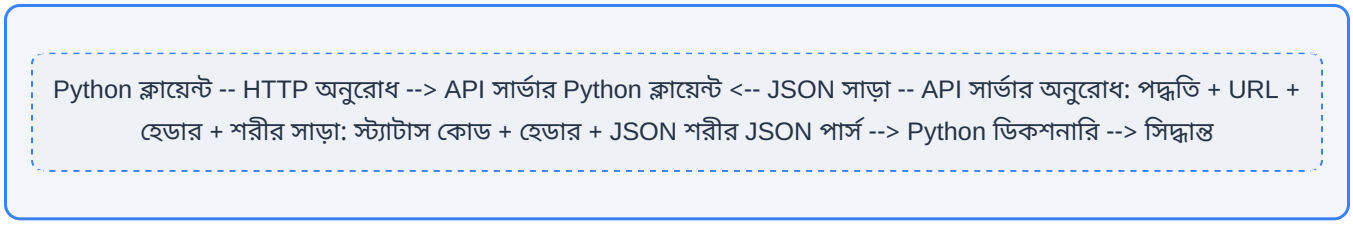
১৭.২ পূর্বশর্ত

- **অধ্যায় ১৪:** চলক, ডিকশনারি, তালিকা, লুপ, ফাংশন, `try` / `except`।
- **অধ্যায় ১৫:** ফাইল পড়া-লেখা ও `json` মডিউলের প্রাথমিক ব্যবহার।
- **অধ্যায় ১৬:** `subprocess` দিয়ে সার্ভারের অবস্থা যাচাই।
- **JSON:** কী-মান জোড়ার টেক্সট ধরন; Python ডিকশনারির সঙ্গে মিল থাকলেও এক নয়, রূপান্তর লাগে।
- **HTTP (Vol 02):** URL, ব্রাউজারের অনুরোধ ও সার্ভারের সাড়ার প্রাথমিক ধারণা।
- **পরিবেশ:** Ubuntu Server 24.04 LTS, Python 3.12.3, Bash 5.2, Git 2.43; ইন্টারনেট সংযোগ দরকার।

১৭.৩ API কী — কেন আগে, কীভাবে পরে

এক প্রোগ্রাম আরেক প্রোগ্রামকে তথ্য দিতে চাইলে সরাসরি তার ডেটাবেসে লেখা যায় না। ডেটাবেসের পাসওয়ার্ড, টেবিলের গঠন জানতে হয়, আর ভেতরের গঠন বদলালে সব ভেঙে যায়। এর বদলে সার্ভার প্রোগ্রামটি একটি পরিষ্কার নিয়ম প্রকাশ করে: এই ঠিকানায়, এইভাবে বার্তা পাঠালে আমি তা নেব, আর এইভাবে উত্তর দেব। সেই নিয়মই API (Application Programming Interface) — দুটি প্রোগ্রামের মধ্যে চুক্তি। ভেতরে কীভাবে কাজ হয়, জানার দরকার নেই। রেস্তোরাঁর মেনুর মতো: আপনি অর্ডার দেন, খাবার পান, রান্নাঘরের ভেতরের কাজ জানতে হয় না। এখানে অর্ডার দেওয়া প্রোগ্রামটি ক্লায়েন্ট, আর খাবার দেওয়া প্রোগ্রামটি সার্ভার।

HTTP এই কথোপকথনের সাধারণ ভাষা, কারণ এটি ওয়েবের ভাষা। সব ব্রাউজার, সার্ভার ও প্রোগ্রামিং ভাষা HTTP জানে, ফায়ারওয়াল তাকে যেতে দেয়। REST হলো HTTP সাজিয়ে ব্যবহারের জনপ্রিয় ধারণা: প্রতিটি জিনিস একটি সম্পদ, যেমন ব্যবহারকারী বা রিপোজিটরি, প্রতিটির একটি ঠিকানা আছে, যেমন `https://api.github.com/users/octocat`। সঠিক পদ্ধতিতে সেই ঠিকানায় অনুরোধ পাঠালেই কাজ হয়।



এই চিত্রই পুরো অধ্যায়ের সারমর্ম: বামে ক্লায়েন্ট, ডানে সার্ভার, মাঝে HTTP অনুরোধ ও JSON সাড়া। সাড়া পার্স করে ডিকশনারি বানিয়ে স্ক্রিপ্ট সিদ্ধান্ত নেয় — সব ঠিক আছে নাকি সতর্ক করতে হবে।

> **মূল ধারণা — আগে নথি, পরে কোড**
>
> API ব্যবহারের আগে তার নথি পড়ুন: কোন ঠিকানা, কোন হেডার, উত্তরে কী আসবে। নথি না পড়ে আন্দাজে অনুরোধ পাঠালে সময় নষ্ট হয়। প্রথমে ছোট GET অনুরোধ দিয়ে পরীক্ষা করুন, তারপর পুরো স্ক্রিপ্ট লিখুন।

১৭.৪ HTTP পদ্ধতি ও আইডেমপোটেন্সি

অনুরোধের প্রথম শব্দ পদ্ধতি বলে দেয় আপনি কী করতে বলছেন।

পদ্ধতি	কাজ	শরীর	আইডেমপোটেন্ট
GET	তথ্য পড়া	সাধারণত না	হ্যাঁ
POST	নতুন কিছু তৈরি করা	হ্যাঁ	না
PUT	পুরো সম্পদ বদলে দেওয়া	হ্যাঁ	হ্যাঁ
PATCH	সম্পদের অংশ বদলানো	হ্যাঁ	হ্যাঁ
DELETE	সম্পদ মোছা	সাধারণত না	হ্যাঁ

আইডেমপোটেন্ট মানে একবার পাঠানো আর পাঁচবার পাঠানো একই ফল। GET-এ কিছু বদলায় না, তাই তা আইডেমপোটেন্ট। POST নয় — একই প্রতিবেদন পাঁচবার পাঠালে পাঁচ কপি তৈরি হতে পারে। এই পার্থক্য রিট্রাইয়ের সময় কাজে লাগে: শুধু আইডেমপোটেন্ট অনুরোধ নিরাপদে আবার পাঠানো যায়। অনুরোধের বাকি অংশ: URL (কোন সম্পদ),

কোয়েরি প্যারামিটার (`?page=2&per_page=10`), হেডার (`Accept: application/json`, `Authorization: Bearer TOKEN`) ও শরীর (সাধারণত JSON)। সাড়াতেও স্ট্যাটাস কোড, হেডার ও শরীর থাকে।

১৭.৫ স্ট্যাটাস কোড

শ্রেণি	অর্থ	উদাহরণ	করণীয়
2xx	সফল	200 ঠিক আছে, 201 তৈরি হয়েছে, 204 তথ্য নেই	সাড়া ব্যবহার করুন
3xx	অন্য ঠিকানায় যেতে হবে	301 স্থায়ী, 302 সাময়িক	ক্লায়েন্ট নিজেই নতুন ঠিকানায় যায়
4xx	অনুরোধে ভুল	400 ভুল অনুরোধ, 401 প্রমাণীকরণ নেই, 403 অনুমতি নেই, 404 নেই, 429 অনেক অনুরোধ	অনুরোধ ঠিক করুন
5xx	সার্ভারে সমস্যা	500 ভেতরের ত্রুটি, 502 খারাপ গেটওয়ে, 503 সেবা নেই	অপেক্ষা করে আবার চেষ্টা করুন

4xx মানে দোষ আপনার অনুরোধে — ভুল ঠিকানা, ভুল টোকেন, ভুল তথ্য। একই অনুরোধ বারবার পাঠালে একই ভুল হবে। 5xx মানে দোষ সার্ভারে — ব্যস্ত বা বন্ধ। তখন অপেক্ষা করে আবার চেষ্টা করা যায়। 401 মানে পরিচয় প্রমাণ হয়নি, সাধারণত টোকেন ভুল বা মেয়াদ শেষ। 403 মানে পরিচয় জানা গেছে, কিন্তু অনুমতি নেই — টোকেনের সুযোগ কম। 404 মানে সম্পদটি নেই। 429 মানে খুব দ্রুত অনেক অনুরোধ — থামুন।

১৭.৬ প্রমাণীকরণ ও গোপন তথ্য

অধিকাংশ API জিজ্ঞেস করে: আপনি কে? তিনটি সাধারণ উপায় আছে। API কী হলো একটি দীর্ঘ গোপন স্ট্রিং, সাধারণত হেডারে পাঠানো হয়, যেমন `X-API-Key: ...`। Bearer টোকেন হলো একই ধরনের গোপন মান, হেডারে লেখা হয় `Authorization: Bearer <token>`। Basic auth-এ ব্যবহারকারীর নাম ও পাসওয়ার্ড Base64 করে পাঠানো হয় — এটি শুধু ধারণা হিসেবে জানুন, HTTPS ছাড়া এটি বিপজ্জনক।

টোকেনের নিয়ম একটাই: টোকেন কখনো কোডে লিখবেন না, কখনো Git-এ তুলবেন না। টোকেন রাখুন পরিবেশ চলকে, পড়ুন `os.environ` দিয়ে। গোপন ফাইল থাকলে `.gitignore`-এ তার নাম লিখুন। লগে কখনো পুরো টোকেন ছাপাবেন না, URL-এও টোকেন বসাবেন না — URL লগে থেকে যায়। টোকেন ফাঁস হলে তা বাতিল করে নতুন টোকেন নিন।

> **সতর্কতা — গোপন তথ্য কোডে নয়**
>
> এই অধ্যায়ের সব উদাহরণে `os.environ["API_TOKEN"]` ব্যবহার করা হয়েছে। টোকেন শেল থেকে সেট করুন, যেমন `export API_TOKEN=আপনার_টোকেন` — তবে আসল টোকেন এখানে বা কোডে লিখবেন না। `.env` ফাইল ব্যবহার করলে তাও `.gitignore`-এ রাখুন।

১৭.৭ urllib.request দিয়ে GET — কোনো ইনস্টল ছাড়া

`urllib.request` Python-এর সঙ্গে থাকা স্ট্যান্ডার্ড লাইব্রেরি। কিছু ইনস্টল করতে হয় না। নিচের স্ক্রিপ্টটি GitHub-এর প্রকাশ্য ব্যবহারকারী `octocat`-এর তথ্য আনে। আউটপুট উদাহরণমূলক।

```
import json
import urllib.request
url = "https://api.github.com/users/octocat"
request = urllib.request.Request(url, headers={"Accept": "application/json"})
with urllib.request.urlopen(request, timeout=10) as response:
    data = json.loads(response.read().decode("utf-8"))
print(data.get("login"))
print(data.get("public_repos"))
```

`import json` JSON মডিউল আনে। `import urllib.request` HTTP অনুরোধের মডিউল আনে। `url` চলকে ঠিকানা রাখা হয়। `Request(url, headers={...})` একটি অনুরোধ বস্তু বানায়; `headers` ডিকশনারিতে `Accept` বলে JSON চাই। `urlopen(request, timeout=10)` অনুরোধ পাঠায়, ১০ সেকেন্ডের বেশি অপেক্ষা করে না। `with` ব্যবহার করলে সংযোগ নিজে বন্ধ হয়। `response.read()` বাইটস আনে, `.decode("utf-8")` তাকে টেক্সট করে, `json.loads` তাকে ডিকশনারি করে। `data.get("login")` নিরাপদে মান নেয়; কী না থাকলে ত্রুটি না দিয়ে `None` দেয়।

১৭.৮ requests লাইব্রেরি — GET, হেডার, টাইমআউট, টোকেন

`requests` ব্যবহার সহজ বলে DevOps-এ বেশি ব্যবহৃত হয়। ইনস্টল করুন: `pip install requests`। `Session` একই সংযোগ ও হেডার বারবার ব্যবহার করে।

```
import os
import requests
token = os.environ["API_TOKEN"]
headers = {"Authorization": "Bearer " + token, "Accept": "application/json"}
response = requests.get("https://api.github.com/users/octocat", headers=headers, timeout=10)
response.raise_for_status()
data = response.json()
print(data.get("login"))
print(data.get("name"))
```

`import os` পরিবেশ চলক পড়ার মডিউল আনে। `os.environ["API_TOKEN"]` শেল থেকে টোকেন নেয়; টোকেন না থাকলে `KeyError` হয়, যা ভালো — চুপচাপ ভুল টোকেনে চলা বন্ধ হয়। `"Bearer " + token` হেডারের মান বানায়; `Bearer`-এর পর একটি ফাঁকা জায়গা থাকা জরুরি। `requests.get` GET পাঠায়। `headers=headers` হেডার দেয়, `timeout=10` ১০ সেকেন্ডে সাড়া না এলে থামায়। `raise_for_status()` 4xx বা 5xx হলে ব্যতিক্রম তোলে। `response.json()` JSON সাড়া সরাসরি ডিকশনারি করে। `.get("name")` আবার নিরাপদ পড়া।

> **পরামর্শ — Session ব্যবহার করুন**
>
> একই API-তে বারবার অনুরোধ পাঠালে `requests.Session()` ব্যবহার করুন। হেডার একবার সেট করলেই সব অনুরোধে যাবে, আর সংযোগ পুনরায় ব্যবহার হয় বলে গতি বাড়ে।

১৭.৯ JSON — পার্স, তৈরি, অনুপস্থিত কী

JSON-এর ধরন Python-এর ধরনের সঙ্গে মেলে: অবজেক্ট হয় ডিকশনারি, অ্যারে হয় তালিকা, স্ট্রিং হয় `str`, সংখ্যা হয় `int` / `float`, `true` / `false` হয় `True` / `False`, আর `null` হয় `None`। `json.loads` টেক্সট থেকে ডিকশনারি করে, `json.load` ফাইল থেকে পড়ে, `json.dumps` ডিকশনারি থেকে JSON টেক্সট বানায়। সরাসরি `data["key"]` না লিখে `data.get("key")` ব্যবহার করুন; নেস্টেড তথ্যে `data.get("owner", {}).get("login")` লিখুন, যাতে মাঝের কী

না থাকলেও ত্রুটি না হয়। সাড়া JSON নাও হতে পারে — তাই `response.json()`-এর আগে স্ট্যাটাস কোড ও `Content-Type` দেখে নিন।

১৭.১০ POST দিয়ে JSON পাঠানো

নিচের উদাহরণটি httpbin-এ একটি স্বাস্থ্য প্রতিবেদন পাঠায়। আউটপুট উদাহরণমূলক।

```
import os
import requests
token = os.environ["API_TOKEN"]
headers = {"Authorization": "Bearer " + token}
payload = {"server": "web-01", "disk_percent": 82, "status": "warning"}
response = requests.post("https://httpbin.org/post", json=payload, headers=headers,
timeout=10)
response.raise_for_status()
print(response.json().get("json"))
```

`payload` ডিকশনারিতে পাঠানোর তথ্য। `requests.post` POST পাঠায়। `json=payload` দিলে `requests` নিজে ডিকশনারিকে JSON করে ও সঠিক `Content-Type` হেডার বসায়; হাতে `json.dumps` করার দরকার নেই। `response.json().get("json")` সাড়ার ভেতরে সার্ভার যা পেয়েছে, তা দেখায়। POST আইডেমপোটেন্ট নয় — তাই এটি অন্ধভাবে আবার পাঠাবেন না।

১৭.১১ পেজিনেশন — সব পাতা সংগ্রহ করা

API একবারে সব তথ্য দেয় না; দেয় পাতায় পাতায়। `page` ও `per_page` কোয়েরি প্যারামিটার দিয়ে পাতা বদলাতে হয়। কোনো API `Link` হেডারে পরের পাতার ঠিকানা দেয়। সহজ নিয়ম: পাতা বাড়াতে থাকুন, যতক্ষণ খালি তালিকা না আসে।

```
import requests
users = []
page = 1
while True:
    response = requests.get("https://api.github.com/users", params={"per_page": 5, "since": 0,
"page": page}, timeout=10)
    response.raise_for_status()
    batch = response.json()
    if not batch:
        break
    users.extend(batch)
    page += 1
    if page > 3:
        break
print(len(users))
```

`users = []` খালি তালিকা। `page = 1` প্রথম পাতা। `params={...}` কোয়েরি প্যারামিটার বানায়; `requests` নিজে URL-এ জুড়ে দেয়। `batch` এক পাতার তথ্য। `if not batch: break` খালি পাতা পেলে থামে। `users.extend(batch)` নতুন তথ্য জোড়ে। `page += 1` পরের পাতায় যায়। `if page > 3: break` এই উদাহরণে তিন পাতায় থামায়, যাতে পরীক্ষা দীর্ঘ না হয়।

১৭.১২ রিট্রাই, ব্যাকঅফ, হার সীমা ও ত্রুটি সামলানো

নেটওয়ার্ক সবসময় নির্ভরযোগ্য নয়। তাই প্রতিটি অনুরোধে `timeout=` দিন — টাইমআউট না দিলে স্ক্রিপ্ট অনির্দিষ্টকাল ঝুলে থাকতে পারে। ব্যর্থ হলে সঙ্গে সঙ্গে আবার চেষ্টা না করে অপেক্ষা বাড়ান: ১, ২, ৪ সেকেন্ড — এটাই এক্সপোনেনশিয়াল ব্যাকঅফ। শুধু আইডেমপোটেন্ট GET আবার পাঠান; POST অন্ধভাবে আবার পাঠাবেন না, তাতে ডুপ্লিকেট তৈরি হতে পারে। 429 পেলে `Retry-After` হেডারের মান দেখে তত সেকেন্ড ঘুমান (`time.sleep`)।

```
import time
import requests
def get_with_retry(url, tries=3):
    delay = 1
    for attempt in range(tries):
        try:
            response = requests.get(url, timeout=10)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException:
            if attempt == tries - 1:
                raise
            time.sleep(delay)
            delay = delay * 2
print(get_with_retry("https://api.github.com/users/octocat").get("login"))
```

`def get_with_retry(url, tries=3)` ফাংশন বানায়; `tries=3` ডিফল্ট মান। `delay = 1` প্রথম অপেক্ষা ১ সেকেন্ড। `for attempt in range(tries)` তিনবার চেষ্টা করে। `try` ভেতরে GET পাঠায়। `except requests.exceptions.RequestException` নেটওয়ার্ক ও HTTP ত্রুটি ধরে। `if attempt == tries - 1: raise` শেষ চেষ্টায় ব্যর্থ হলে ত্রুটি আবার তোলে। `time.sleep(delay)` অপেক্ষা করে, `delay = delay * 2` অপেক্ষা দ্বিগুণ করে। সফল হলে `return response.json()` ফল দেয়।

ত্রুটি সামলাতে 4xx ও 5xx আলাদা করুন। 4xx হলে অনুরোধ, টোকেন বা ঠিকানা ঠিক করুন; 5xx হলে অপেক্ষা করে রিট্রাই করুন। ত্রুটি লগ করার সময় স্ট্যাটাস কোড ও বার্তা লিখুন, কিন্তু `Authorization` হেডার বা টোকেন কখনো লগে লিখবেন না। `raise_for_status()` ব্যবহার করলে ভুল সাড়াকে সাফল্য ভেবে এগিয়ে যাওয়া যায় না।

সমস্যা সমাধান — সাধারণ ভুল

- **টাইমআউট না দেওয়া:** স্ক্রিপ্ট ঝুলে যায়। প্রতিটি অনুরোধে `timeout=10` দিন।
- **টোকেন ফাঁস:** টোকেন লগে, URL-এ বা Git-এ গেলে তা বাতিল করুন। শুধু `os.environ` ব্যবহার করুন।
- **সাড়া JSON ধরে নেওয়া:** ত্রুটির সাড়া HTML হতে পারে। আগে স্ট্যাটাস কোড দেখুন, তারপর `response.json()` করুন।
- **পেজিনেশন না করা:** প্রথম পাতা দেখেই থামলে তথ্য অসম্পূর্ণ থাকে। খালি পাতা পর্যন্ত লুপ চালান।
- **POST অন্ধভাবে রিট্রাই:** ডুপ্লিকেট তৈরি হয়। POST-এ রিট্রাইয়ের আগে সার্ভার নথি দেখুন।
- **401/403 গুলিয়ে ফেলা:** 401 মানে টোকেন ভুল, 403 মানে অনুমতি নেই — সমাধান আলাদা।

১৭.১৩ অনুশীলন — TechNova স্বাস্থ্য প্রতিবেদন পাঠানো

ল্যাব — মনিটরিং API-তে প্রতিবেদন পাঠান

1. টার্মিনালে `export API_TOKEN=test-token-123` লিখে পরীক্ষামূলক টোকেন সেট করুন।
2. ১৭.৮-এর GET স্ক্রিপ্টটি `health_get.py` নামে সংরক্ষণ করে চালান।
3. ১৭.১০-এর POST স্ক্রিপ্টটি `health_post.py` নামে সংরক্ষণ করে চালান; `disk_percent` 82 থেকে 95 বদলে আবার চালান।
4. ১৭.১১-এর পেজিনেশন স্ক্রিপ্ট চালিয়ে কতজন ব্যবহারকারী এল, তা লিখে রাখুন।
5. ইন্টারনেট বন্ধ করে GET স্ক্রিপ্ট চালান — কী ত্রুটি আসে, তা দেখুন। তারপর রিট্রাই ফাংশন দিয়ে আবার চালান।
6. টোকেন ছাড়া শেল চালু করে GET স্ক্রিপ্ট চালান — `KeyError` আসে কি না, দেখুন।

১৭.১৪ প্রত্যাশিত ফলাফল ও যাচাই

GET উদাহরণে `octocat` ছাপা হওয়া উচিত, আর `public_repos`-এ একটি সংখ্যা আসা উচিত — সংখ্যাটি সময়ের সঙ্গে বদলাতে পারে, তাই তা উদাহরণমূলক। POST উদাহরণে সাড়ার `json` অংশে আপনার পাঠানো `server`, `disk_percent` ও `status` ফিরে আসা উচিত। পেজিনেশন উদাহরণে ১৫ জন ব্যবহারকারীর সংখ্যা ছাপা উচিত, কারণ প্রতি পাতায় ৫ জন করে তিন পাতা আনা হয়েছে। যাচাই করুন: স্ট্যাটাস কোড 200 কি না, সাড়াটি ডিকশনারি কি না, আর লগে টোকেন ছাপা হয়নি কি না। কোনো ধাপে 401 এলে টোকেন সেট হয়েছে কি না দেখুন; 404 এলে URL-এর বানান দেখুন।

যাচাই তালিকা

GET-এ `login` এসেছে, POST-এ পাঠানো তথ্য ফিরেছে, পেজিনেশনে তিন পাতা এসেছে, টাইমআউট দেওয়া হয়েছে, আর টোকেন কোথাও ছাপা হয়নি।

১৭.১৫ নিরাপত্তা বিবেচনা

শুধু HTTPS ব্যবহার করুন — `http://` ঠিকানায় টোকেন পাঠাবেন না। কখনো `verify=False` ব্যবহার করবেন না; এটি TLS যাচাই বন্ধ করে দেয়, আর মাঝখানে কেউ তথ্য পড়তে বা বদলাতে পারে। টোকেনের সুযোগ সর্বনিম্ন রাখুন — শুধু যা দরকার, তাই। টোকেন নিয়মিত ঘোরান (রোটেট করুন) আর পুরোনো টোকেন বাতিল করুন। লগে `Authorization` হেডার লাল কালিতে ঢেকে (রিড্যাক্ট করে) লিখুন। গোপন ফাইল `.gitignore`-এ রাখুন, আর `git status` দিয়ে নিশ্চিত হোন কোনো গোপন ফাইল কমিট হচ্ছে না।

১৭.১৬ সেরা অনুশীলন

- প্রতিটি অনুরোধে টাইমআউট দিন, আর `raise_for_status()` ব্যবহার করুন।
- একই API-তে বারবার অনুরোধে `Session` ব্যবহার করুন।
- অনুপস্থিত কী `.get` দিয়ে পড়ুন, সরাসরি ইনডেক্স নয়।
- পেজিনেশন, রিট্রাই ও 429 সামলানো কোডে রাখুন — পরে যোগ করা কঠিন।
- ত্রুটি লগ করুন, কিন্তু টোকেন, পাসওয়ার্ড ও পুরো হেডার লগ করবেন না।

- API নথি পড়ে, ছোট পরীক্ষা দিয়ে, তারপর পুরো স্ক্রিপ্ট লিখুন।

১৭.১৭ পরিভাষা

ইংরেজি পরিভাষা	বাংলা ব্যাখ্যা
API	দুটি প্রোগ্রামের মধ্যে চুক্তি; কীভাবে তথ্য চাইতে ও দিতে হবে, তার নিয়ম।
Client / Server	ক্লায়েন্ট অনুরোধ পাঠায়, সার্ভার সাড়া দেয়।
Endpoint	API-র নির্দিষ্ট ঠিকানা, যেমন ব্যবহারকারীর তথ্যের ঠিকানা।
Header	অনুরোধ বা সাড়া সম্পর্কে অতিরিক্ত তথ্য, যেমন প্রমাণীকরণ।
JSON	কী-মান জোড়ায় সাজানো টেক্সট ধরন; প্রোগ্রামের মধ্যে তথ্য আদান-প্রদানের মাধ্যম।
Bearer Token	পরিচয়ের প্রমাণ হিসেবে হেডারে পাঠানো গোপন মান।
Pagination	বড় তথ্য পাতায় পাতায় ভাগ করে দেওয়া।
Rate Limiting	নির্দিষ্ট সময়ে কত অনুরোধ পাঠানো যাবে, তার সীমা; ছাড়ালে 429 আসে।
Idempotent	একবার বা বারবার পাঠালেও একই ফল দেওয়া অনুরোধ।
Timeout	সাড়ার জন্য সর্বোচ্চ অপেক্ষার সময়।

১৭.১৮ বহুনির্বাচনী প্রশ্ন

1. API বলতে কী বোঝায়?
   ক) দুটি প্রোগ্রামের মধ্যে তথ্য আদান-প্রদানের চুক্তি ও নিয়ম

   খ) একটি ডেটাবেসের নাম

   গ) একটি ওয়েব ব্রাউজার

   ঘ) একটি কম্পিউটার ভাইরাস

2. কোন HTTP পদ্ধতিটি নতুন সম্পদ তৈরি করতে ব্যবহৃত হয়?
   ক) GET

   খ) POST

   গ) DELETE

   ঘ) HEAD

3. স্ট্যাটাস কোড 429-এর অর্থ কী?
   ক) সাড়া সফল হয়েছে

   খ) সম্পদটি পাওয়া যায়নি

   গ) খুব দ্রুত অনেক অনুরোধ পাঠানো হয়েছে

ঘ) সার্ভার স্থায়ীভাবে বন্ধ

4. API টোকেন নিরাপদে রাখার সঠিক উপায় কোনটি?
   ক) কোডের ভেতরে লিখে রাখা

   খ) পরিবেশ চলকে রাখা ও `os.environ` দিয়ে পড়া

   গ) URL-এর মধ্যে বসানো

   ঘ) লগ ফাইলে ছাপানো

5. JSON সাড়া থেকে অনুপস্থিত কী নিরাপদে পড়তে কোনটি ব্যবহার করা হয়?
   ক) `data["key"]`

   খ) `data.get("key")`

   গ) `data.remove("key")`

   ঘ) `data + "key"`

১৭.১৯ স্বাধীন অ্যাসাইনমেন্ট

TechNova-র জন্য একটি স্ক্রিপ্ট লিখুন যা GitHub API থেকে একটি রিপোজিটরির প্রথম তিন পাতার ইস্যু সংগ্রহ করে, টাইমআউট ও রিট্রাইসহ, আর শিরোনামগুলো একটি ফাইলে লেখে। টোকেন পরিবেশ চলক থেকে পড়ুন। কোনো সমাধান এখানে দেওয়া হয়নি।

১৭.২০ সারাংশ ও পুনরালোচনা

এই অধ্যায়ে আমরা শিখেছি API হলো প্রোগ্রামের মধ্যে চুক্তি, HTTP তার ভাষা, আর JSON তার বার্তার ধরন। GET তথ্য পড়ে, POST নতুন তথ্য তৈরি করে, PUT ও PATCH বদলায়, DELETE মোছে। স্ট্যাটাস কোড বলে অনুরোধ সফল হয়েছে কি না, ভুল কোথায়। টোকেন পরিবেশ চলকে রাখতে হয়, কোডে নয়। `urllib.request` দিয়ে কোনো ইনস্টল ছাড়া, আর `requests` দিয়ে সহজে অনুরোধ পাঠানো যায়। JSON পার্স করে ডিকশনারি বানিয়ে `.get` দিয়ে নিরাপদে পড়তে হয়। পেজিনেশন, টাইমআউট, ব্যাকঅফ ও 429 সামলে স্ক্রিপ্ট নির্ভরযোগ্য হয়। আর সবচেয়ে গুরুত্বপূর্ণ — HTTPS ব্যবহার করুন, `verify=False` কখনো নয়, টোকেন কখনো ফাঁস করবেন না।

- ☐ API, ক্লায়েন্ট ও সার্ভারের সম্পর্ক ব্যাখ্যা করতে পারি।
- ☐ GET, POST, PUT, PATCH, DELETE ও আইডেমপোটেন্সি বলতে পারি।
- ☐ 2xx, 4xx ও 5xx স্ট্যাটাস কোড চিনতে পারি।
- ☐ `os.environ` থেকে টোকেন পড়ে `requests` দিয়ে GET ও POST পাঠাতে পারি।
- ☐ JSON পার্স করে `.get` দিয়ে অনুপস্থিত কী সামলাতে পারি।
- ☐ পেজিনেশন লুপ ও রিট্রাই-ব্যাকঅফ লিখতে পারি।
- ☐ 429 পেলে `Retry-After` দেখে অপেক্ষা করতে পারি।
- ☐ টোকেন লগ না করে, HTTPS-সহ, নিরাপদ স্ক্রিপ্ট লিখতে পারি।

অধ্যায় ১৮ • Part III — Python for DevOps

Chapter 18: YAML, Configuration and Environment Management — YAML, কনফিগারেশন ও পরিবেশ ব্যবস্থাপনা

টেকনোভার তিনটি সার্ভার — ডেভেলপমেন্ট, স্টেজিং ও প্রোডাকশন। একই স্ক্রিপ্ট তিন জায়গায় চলে, কিন্তু ডেটাবেসের নাম, পোর্ট ও এপিআই ঠিকানা আলাদা — আর এই তথ্যগুলো স্ক্রিপ্টের ভেতরেই লেখা ছিল। একদিন ডেভের স্ক্রিপ্ট ভুল করে প্রোডাকশনে চলল, আর প্রোডাকশন ডেটাবেসের ডেটা মুছে গেল। ভুলটি কোডের ছিল না — ভুলটি ছিল কোডের ভেতরে সেটিংস লুকিয়ে রাখার অভ্যাসের। এই অধ্যায়ে সেটিংসকে কোড থেকে আলাদা রাখা শিখব।

১৮.১ শেখার উদ্দেশ্য

- YAML-এ ম্যাপিং, লিস্ট, নেস্টেড ডেটা ও `null` লিখতে পারা।
- JSON ও YAML-এর পার্থক্য ব্যাখ্যা করা এবং কোন কাজে কোনটি ভালো তা বলা।
- PyYAML দিয়ে `yaml.safe_load` ব্যবহার করা।
- পরিবেশ ভেরিয়েবল দিয়ে কনফিগারেশন ওভাররাইড করা।
- `.env` ব্যবহার করা এবং কেন তা গিটে কমিট করা যায় না তা ব্যাখ্যা করা।
- স্টার্টআপে কনফিগারেশন যাচাই করে ভুল তাড়াতাড়ি ধরা।
- সিক্রেট ও সাধারণ কনফিগারেশনের পার্থক্য বোঝা।

১৮.২ পূর্বশর্ত

অধ্যায় ১৪-এর পাইথন, অধ্যায় ১৭-এর JSON ও এপিআই, এবং অধ্যায় ০৮-এর ব্যাশ পরিবেশ ভেরিয়েবল জানা থাকতে হবে। শেলে `export` দিয়ে ভেরিয়েবল সেট করা হয়। ডিকশনারি, লিস্ট, ফাংশন ও ব্যতিক্রমও কাজে লাগবে।

১৮.৩ প্রথমে কেন — কীভাবে তার আগে

কনফিগারেশন হলো সেই তথ্য, যা পরিবেশ বদলালে বদলায় কিন্তু কোডের যুক্তি বদলায় না — যেমন ডেটাবেস হোস্ট, পোর্ট, লগের স্তর, এপিআই ঠিকানা। একই কোড ডেভে এক ডেটাবেসে যুক্ত হয়, প্রোডাকশনে আরেকটিতে। কোড একটাই, সেটিংস অনেকগুলো। টুয়েলভ-ফ্যাক্টর ধারণা: কোড একটাই রেপোজিটরি থেকে আসবে, সেটিংস আসবে পরিবেশ থেকে; কোডে গোপন তথ্য থাকবে না। স্ট্রাকচার্ড ডেটা মানে গঠন ঠিক করা ডেটা — কোনটি কী, কোনটির ভেতরে কোনটি আছে, তা স্পষ্ট। YAML ও JSON দুটোই এমন ভাষা।

> **মূল ধারণা:** কোড বলে কী করতে হবে, কনফিগারেশন বলে কোথায় ও কীভাবে। দুটি আলাদা রাখলে ভুল কমে।

১৮.৪ স্ট্রাকচার্ড ডেটার ধারণা

কী-ভ্যালু হলো সবচেয়ে সরল গঠন — বামে নাম, ডানে মান, যেমন `host: db.example.invalid`। নেস্টিং মানে একটি মানের ভেতরে আরেকটি গঠন — যেমন `database`-এর ভেতরে `host`, `port`, `name`। লিস্ট মানে একই ধরনের অনেক মানের ক্রম, যেমন সার্ভারের তালিকা। এই তিনটি মিলেই প্রায় সব কনফিগারেশন লেখা যায়।

১৮.৫ JSON বনাম YAML

JSON এপিআই ও ডেটা আদান-প্রদানে ভালো, কিন্তু হাতে লেখায় কষ্টকর — ব্র্যাকেট বা কমা ভুল হলেই ফাইল ভেঙে যায়। YAML মানুষের পড়ার জন্য তৈরি।

বিষয়	JSON	YAML
সিনট্যাক্স	ব্র্যাকেট ও কমা লাগে	ইন্ডেন্টেশন দিয়ে গঠন বোঝায়
মন্তব্য	নেই	`#` দিয়ে লেখা যায়
পঠনযোগ্যতা	মাঝারি, যন্ত্রের জন্য ভালো	খুব ভালো, মানুষের জন্য ভালো
টাইপ	স্ট্রিং, সংখ্যা, বুলিয়ান, null, অ্যারে, অবজেক্ট	একই টাইপ, সঙ্গে তারিখ ও মাল্টি-লাইন স্ট্রিং
ব্যবহার	এপিআই, ডেটা আদান-প্রদান	কনফিগারেশন, ডেপ্লয়মেন্ট, সিআই/সিডি

প্রায় সব বৈধ JSON-ই বৈধ YAML হিসেবে পড়া যায়। কনফিগারেশনে YAML বেশি ব্যবহৃত হয় কারণ তাতে মন্তব্য লেখা যায় ও পড়া সহজ।

১৮.৬ YAML সিনট্যাক্স বিস্তারিত

YAML-এর সবচেয়ে গুরুত্বপূর্ণ নিয়ম ইন্ডেন্টেশন: প্রতিটি স্তরে ঠিক ২টি স্পেস, ট্যাব নয়। ম্যাপিং হলো কী-ভ্যালু জোড়া, সিকোয়েন্স হলো ড্যাশ দিয়ে শুরু হওয়া লিস্ট। স্ট্রিং সাধারণত কোটেশন ছাড়াই লেখা যায়, বিশেষ অক্ষর থাকলে কোটেশন নাও। বুলিয়ান `true` / `false`, সংখ্যা সরাসরি লেখা যায়, খালি মান `null` বা `~`। লম্বা লেখায় `|` লাইন ভাঙা ধরে রাখে, `>` লাইনগুলোকে এক লাইনে জোড়া দেয়। অ্যাঙ্কর `&name` দিয়ে মানের নাম দেওয়া যায়, অ্যালিয়াস `*name` দিয়ে তা আবার ব্যবহার করা যায়। ডকুমেন্টের শুরুতে `---` বসে।

১৮.৭ PyYAML দিয়ে YAML পড়া ও লেখা

পাইথনে YAML পড়তে PyYAML ব্যবহার করা হয়। উবুন্টু ২৪.০৪-এ পাইথন ৩.১২.৩-এ `pip install pyyaml` দিয়ে সংস্করণ ৬.০.১ ইনস্টল করো। শুধু `yaml.safe_load` ব্যবহার করবে — লোডার ছাড়া `yaml.load` নিরাপত্তা ঝুঁকি তৈরি করে। লেখার জন্য `yaml.safe_dump` নাও।

১৮.৮ পরিবেশ ভেরিয়েবল

পরিবেশ ভেরিয়েবল হলো অপারেটিং সিস্টেমে রাখা কী-ভ্যালু। পাইথনে `os.environ` ডিকশনারির মতো, `os.getenv` দিয়ে ডিফল্ট মানসহ মান নেওয়া যায়। শেলে `export DB_HOST=db.example.invalid` লিখলে সেই সেশনে ভেরিয়েবলটি সেট হয়। অগ্রাধিকার: পরিবেশ ভেরিয়েবল ফাইলকে ছাপিয়ে যায়, ফাইল ডিফল্টকে ছাপিয়ে যায়।

কনফিগারেশন স্তর (পরেরটি আগেরটিকে ছাপিয়ে যায়): ডিফল্ট মান → কনফিগ ফাইল → পরিবেশ ভেরিয়েবল → CLI আর্গুমেন্ট defaults → config.yaml → env vars → CLI args (later wins)

১৮.৯ কনফিগারেশন ফাইল ও স্তরবদ্ধ কনফিগারেশন

`config.yaml` বা `config.json`-এ পরিবেশ-নিরপেক্ষ সেটিংস রাখো, প্রতিটি পরিবেশের জন্য আলাদা ফাইল — যেমন `config.dev.yaml` ও `config.prod.yaml`। প্রোগ্রাম প্রথমে ডিফল্ট, তারপর ফাইল, তারপর পরিবেশ ভেরিয়েবল দিয়ে ওভাররাইড করে।

১৮.১০ `.env` ফাইল

`.env` ফাইলে `KEY=VALUE` আকারে প্রতি লাইনে একটি ভেরিয়েবল লেখা হয়। `python-dotenv`-এর `load_dotenv` এই ফাইল পড়ে পরিবেশ ভেরিয়েবলে বসায়, ফলে `os.getenv` দিয়ে মান পাওয়া যায়। কিন্তু `.env` কখনোই গিটে কমিট করবে না — এতে পাসওয়ার্ড ও টোকেন থাকতে পারে। `.gitignore`-এ `.env` লেখো, রেপোজিটরিতে শুধু নমুনা মানসহ `.env.example` রাখো।

> **সতর্কতা:** `.env` গিটে উঠলে সিক্রেটটি ফাঁস হয়েছে ধরে নাও — ফাইল মুছলেই হবে না, সিক্রেট বদলাতে হবে।

১৮.১১ কনফিগারেশন যাচাইকরণ

ভুল কনফিগারেশন দেরিতে ধরা পড়লে ক্ষতি বেশি হয়। তাই প্রোগ্রাম শুরুতেই যাচাই করবে — যাকে fail-fast বলে। প্রয়োজনীয় কী আছে কি না, টাইপ ঠিক আছে কি না, পোর্ট ১–৬৫৫৩৫-এর মধ্যে আছে কি না — শুরুতেই পরীক্ষা করো। কিছু না থাকলে পরিষ্কার বার্তা দিয়ে প্রোগ্রাম থামিয়ে দাও। বড় প্রজেক্টে `dataclasses` বা `pydantic` দিয়ে এই যাচাই গোছানোভাবে করা যায়।

১৮.১২ সিক্রেট ব্যবস্থাপনা

সিক্রেট কনফিগারেশন নয় — পাসওয়ার্ড, টোকেন, প্রাইভেট কী সিক্রেট। সাধারণ সেটিংস গিটে রাখা যায়, সিক্রেট যায় না; সিক্রেট রাখো পরিবেশ ভেরিয়েবলে বা ভল্টে, নিয়মিত বদলাও, আর শুধু প্রয়োজনীয় অধিকার দাও (least privilege)।

১৮.১৩ কোড থেকে কনফিগারেশন আলাদা করা

কোডে আইপি, ইউআরএল বা পাসওয়ার্ড হার্ড-কোড করো না। একটি কোডবেস, অনেক কনফিগারেশন — এতে ডেভের কোড প্রোডাকশনে ভুল ডেটাবেসে হাত দেবে না।

১৮.১৪ পরিভাষা

English Term	বাংলা ব্যাখ্যা
Configuration	পরিবেশভেদে বদলানো সেটিংস
YAML	মানুষের পড়ার উপযোগী কনফিগ ভাষা
Environment Variable	অপারেটিং সিস্টেমে রাখা কী-ভ্যালু
Secret	গোপন তথ্য, যেমন পাসওয়ার্ড বা টোকেন
Fail-fast	ভুল শুরুতেই ধরে প্রোগ্রাম থামানো

English Term	বাংলা ব্যাখ্যা
Override	পরের স্তরের মান দিয়ে আগের মান বদলানো

> **বাস্তব পরিস্থিতি:** টেকনোভার স্টেজিং হোস্ট `staging-db.example.invalid`, প্রোডাকশনেরটি আলাদা। কনফিগারেশন আলাদা করার পর ডেপ্লয়ে শুধু `APP_ENV=production` দিলেই হয় — ভুল হোস্টে যুক্ত হওয়ার ঝুঁকি থাকে না।

১৮.১৫ কোড উদাহরণ

নমুনা `config.yaml`: `---` ডকুমেন্ট শুরু বোঝায়। `app:` ম্যাপিং, `database:`-এর ভেতরে হোস্ট, পোর্ট ও নাম (নেস্টিং)। `servers:`-এর ড্যাশগুলো লিস্ট, প্রতিটি আইটেম একটি ডিকশনারি। `#` লাইনগুলো YAML মন্তব্য।

```
---
# TechNova sample config - placeholder values only
app:
  name: technova-app
  debug: false
database:
  host: db.example.invalid
  port: 5432
  name: technova_dev
servers:
  - name: web-01
    role: web
  - name: api-01
    role: api
```

লোডার: `load_config` ফাইল খুলে `yaml.safe_load` দিয়ে ডিকশনারি বানায়। `validate_config` প্রয়োজনীয় কী পরীক্ষা করে, না থাকলে `ValueError` তোলে। `__name__ == "__main__"` মানে স্ক্রিপ্ট সরাসরি চললে নিচের অংশ চলবে।

```
import os
import yaml


def load_config(path):
    with open(path, encoding="utf-8") as f:
        return yaml.safe_load(f)


def validate_config(cfg):
    db = cfg["database"]
    if not db.get("host"):
        raise ValueError("database.host is required")
    if not 1 <= int(db["port"]) <= 65535:
        raise ValueError("database.port out of range")
    return cfg


if __name__ == "__main__":
```

```
    config = validate_config(load_config("config.yaml"))
    print(config["app"]["name"])
    print(config["database"]["host"])
```

ওভাররাইড: `os.getenv`-এর দ্বিতীয় আর্গুমেন্ট ডিফল্ট মান। `DB_HOST` সেট থাকলে সেটি, না থাকলে ফাইলের মান। `int(...)` পোর্টকে সংখ্যায় বদলায়, কারণ পরিবেশ ভেরিয়েবল সবসময় স্ট্রিং।

```
import os

db_host = os.getenv("DB_HOST", "db.example.invalid")
db_port = int(os.getenv("DB_PORT", "5432"))
debug = os.getenv("APP_DEBUG", "false").lower() == "true"
print(db_host)
print(db_port)
print(debug)
```

নমুনা `.env`: প্রতি লাইনে `KEY=VALUE`; `load_dotenv` তা পরিবেশ ভেরিয়েবলে বসায়।

```
APP_ENV=development
DB_HOST=db.example.invalid
DB_PORT=5432
DB_PASSWORD=change-me-from-env
```

গাইডেড অনুশীলন:

1. `mkdir ~/technova-config && cd ~/technova-config` দিয়ে ফোল্ডার বানাও।
2. `pip install pyyaml python-dotenv` দিয়ে লাইব্রেরি ইনস্টল করো।
3. উপরের `config.yaml`, লোডার ও `.env` ফাইল বানাও।
4. `.gitignore`-এ `.env` লিখে `.env.example` বানাও।
5. `DB_HOST=staging-db.example.invalid python3 loader.py` চালিয়ে মান বদলায় কি না দেখো।
6. `database.host` মুছে চালাও — ভুল বার্তাটি লক্ষ করো।

১৮.১৬ প্রত্যাশিত ফলাফল ও যাচাই

সঠিকভাবে চললে লোডারটি `technova-app` ও `db.example.invalid` ছাপবে (চিত্রিত আউটপুট)। পরিবেশ ভেরিয়েবল দিলে হোস্ট বদলে যাবে। হোস্ট মুছলে `ValueError: database.host is required` আসবে — এটাই fail-fast-এর প্রমাণ।

সাধারণ ভুল: ট্যাবে ইন্ডেন্ট করলে YAML ভাঙে — ২ স্পেস নাও। লোডার ছাড়া `yaml.load` নয়, `safe_load` নাও। `.env` কমিট করো না। কী case-sensitive — `Host` আর `host` এক নয়। `yes` / `on` বুলিয়ান হতে পারে, তাই `true`/`false` লেখো।

১৮.১৭ নিরাপত্তা ও সেরা অভ্যাস

সিক্রেট গিটে কমিট করো না, `.gitignore` ব্যবহার করো, সবসময় `yaml.safe_load` নাও। সিক্রেট আছে এমন ফাইলের অনুমতি `0o600` রাখো; ফাঁস হলে বদলে ফেলো। সেরা অভ্যাস: একটি কোডবেস, অনেক কনফিগারেশন; সিক্রেটের ডিফল্ট মান রাখো না; স্টার্টআপে যাচাই করো; `.env.example` আপডেট রাখো।

অনুশীলনী — বহুনির্বাচনী প্রশ্ন:

১. YAML ইন্ডেন্টেশনে কী লাগে? ক) ট্যাব খ) ২টি স্পেস গ) ৪টি ট্যাব ঘ) যেকোনো স্পেস

২. নিরাপদ YAML পড়ায় কোনটি ব্যবহার করবে? ক) `yaml.load` খ) `yaml.safe_load` গ) `yaml.parse` ঘ) `yaml.read`

৩. `.env` সম্পর্কে কোনটি সঠিক? ক) গিটে কমিট করা উচিত খ) এতে `KEY=VALUE` লেখা হয় গ) এতে শুধু কোড থাকে ঘ) এটি JSON ফাইল

৪. অগ্রাধিকারের সঠিক ক্রম কোনটি? ক) ডিফল্ট > ফাইল > পরিবেশ ভেরিয়েবল > CLI খ) CLI > ফাইল > ডিফল্ট > পরিবেশ ভেরিয়েবল গ) ফাইল > ডিফল্ট > CLI > পরিবেশ ভেরিয়েবল ঘ) পরিবেশ ভেরিয়েবল > ডিফল্ট > ফাইল > CLI

৫. সিক্রেট ব্যবস্থাপনায় কোনটি সঠিক? ক) পাসওয়ার্ড কোডে লেখা যায় খ) সিক্রেট গিটে রাখা যায় গ) সিক্রেট পরিবেশ ভেরিয়েবল বা ভল্টে রাখা উচিত ঘ) সিক্রেটের ডিফল্ট মান কোডে রাখা উচিত

১৮.১৮ স্বাধীন অ্যাসাইনমেন্ট

টেকনোভার জন্য ডেভ ও প্রোডাকশনের আলাদা কনফিগ ফাইল, পরিবেশ ভেরিয়েবল ওভাররাইড, স্টার্টআপ যাচাই ও `.env.example` সহ একটি কনফিগারেশন সিস্টেম বানাও।

১৮.১৯ সারাংশ ও পুনরালোচনা

কনফিগারেশন কোড থেকে আলাদা রাখো; YAML মানুষের পড়ার উপযোগী ভাষা; `yaml.safe_load` নিরাপদ; পরিবেশ ভেরিয়েবল ফাইলকে ওভাররাইড করে; `.env` গিটে যায় না; সিক্রেট কনফিগারেশন নয়।

- ☐ YAML-এ ২ স্পেস ইন্ডেন্টেশনে ম্যাপিং ও লিস্ট লিখতে পারি।
- ☐ `yaml.safe_load` ব্যবহার করতে পারি।
- ☐ `os.getenv` দিয়ে পরিবেশ ভেরিয়েবল পড়তে পারি।
- ☐ `.env` ও `.gitignore` সাজাতে পারি।
- ☐ স্টার্টআপে যাচাই করে fail-fast করতে পারি।
- ☐ সিক্রেট ও কনফিগারেশনের পার্থক্য ব্যাখ্যা করতে পারি।

অধ্যায় ১৯ • Part III — Python for DevOps

Chapter 19: Logging, Exceptions and Reliable Automation — লগিং, এক্সেপশন ও নির্ভরযোগ্য অটোমেশন

রাত তিনটা। TechNova-র ব্যাকআপ স্ক্রিপ্ট চলল, কিন্তু কোনো শব্দ করল না। সকালে দেখা গেল ব্যাকআপ হয়নি — মাঝপথে অনুমতি বদলে কপি ব্যর্থ হয়েছিল, অথচ স্ক্রিপ্ট চুপচাপ শেষ হয়েছিল। স্ক্রিপ্ট শুধু চললেই যথেষ্ট নয়; তাকে প্রমাণ করতে হবে কাজ হয়েছে, আর না হলে জোরে জানাতে হবে। `print()` দিয়ে ডিবাগ করা যায়, কিন্তু প্রোডাকশনে তা পর্যবেক্ষণ নয়। এই অধ্যায় Part III বন্ধ করে Part IV-এর cron, SSH ও টুল তৈরির পথ তৈরি করে।

১৯.১ শেখার উদ্দেশ্য — Learning Objectives

- এক্সেপশন হায়ারার্কি বুঝে নির্দিষ্ট এক্সেপশন ধরা ও কাস্টম এক্সেপশন লেখা।
- `logging` দিয়ে স্তর, হ্যান্ডলার ও ফরম্যাটারসহ লগিং তৈরি করা।
- স্ট্রাকচার্ড ও রোটেটিং লগ ব্যবহার করা।
- এক্সপোনেনশিয়াল ব্যাকঅফে রিট্রাই করা ও কখন করা যাবে না তা বলা।
- টাইমআউট, আগেভাগে যাচাই ও `pytest` দিয়ে স্ক্রিপ্ট পরীক্ষা করা।
- আইডেমপোটেন্সি, ড্রাই-রান ও সঠিক এক্সিট কোড প্রয়োগ করা।

১৯.২ পূর্বশর্ত — Prerequisites

অধ্যায় ১৪-এর এক্সেপশন হ্যান্ডলিং, অধ্যায় ১৮-এর YAML ও কনফিগ, অধ্যায় ১২-এর Bash এক্সিট কোড জানা থাকতে হবে। পরিবেশ: Ubuntu Server 24.04 LTS, Python 3.12.3, Bash 5.2, Git 2.43, pytest।

১৯.৩ কেন আগে, কীভাবে পরে — Why Before How

অটোমেশনে মানুষের হাত সরে যায়; মানুষ ভুল দেখে থামে ও জানায়, স্ক্রিপ্ট তা করে না, যদি না শেখানো হয়। তাই স্ক্রিপ্ট পর্যবেক্ষণযোগ্য হবে ও ব্যর্থতা জোরালো হবে। **Error** একটি ভুল ঘটনা, **Failure** পুরো কাজ না হওয়া, আর **Silent success** সবচেয়ে ভয়ংকর: কাজ হয়নি, অথচ এক্সিট কোড শূন্য, কোনো বার্তা নেই। ভালো স্ক্রিপ্ট ভুল হলে থামে, লগে কারণ লেখে ও শূন্য ছাড়া অন্য এক্সিট কোড দেয়।

> **মূল ধারণা:** সফল অটোমেশন মানে প্রমাণসহ শেষ হওয়া। লগ সেই প্রমাণ, এক্সিট কোড সেই ঘোষণা।

১৯.৪ Python এক্সেপশন গভীরভাবে — Exceptions in Depth

ভুল হলে Python একটি এক্সেপশন অবজেক্ট তৈরি করে। শ্রেণিগুলো গাছের মতো সাজানো: উপরে `BaseException`, তার নিচে `Exception`, তার নিচে `ValueError`, `KeyError`, `FileNotFoundError`, `TimeoutError`। `KeyboardInterrupt` ও `SystemExit` সরাসরি `BaseException`-এর নিচে থাকে, তাই `except Exception` সেগুলো ধরে না।

BaseException ├── KeyboardInterrupt ├── SystemExit └── Exception ├── ValueError ├── KeyError ├── FileNotFoundError ├── PermissionError ├── TimeoutError └── ConnectionError

এক্সেপশন	কখন হয়	প্রতিকার
`ValueError`	ভুল মান	ইনপুট যাচাই করে থামা
`KeyError`	ডিকশনারিতে চাবি নেই	`.get()` বা ডিফল্ট মান
`FileNotFoundError`	ফাইল নেই	পথ যাচাই, স্পষ্ট বার্তা
`TimeoutError`	সময়ে উত্তর আসেনি	রিট্রাই করা

নিজের এক্সেপশনও বানানো যায়: `BackupError` নামে `Exception` থেকে আসা শ্রেণি বানালে কলার বোঝে কোন স্তরে সমস্যা হয়েছে। নির্দিষ্ট `except` আগে, সাধারণ `except Exception as e` পরে লিখুন। `else` চলে শুধু এক্সেপশন না হলে, `finally` চলে সবসময়। `raise` ধরা ভুল আবার উপরে পাঠায়, `raise ... from e` মূল কারণের সঙ্গে নতুন ভুলের সম্পর্ক রাখে, যাকে চেইনিং বলে।

> **সতর্কতা:** খালি `except:` লিখবেন না। এটি `KeyboardInterrupt` -সহ সবকিছু ধরে আসল ভুল লুকিয়ে রাখে।

১৯.৫ লগিং মডিউল — The logging Module

`logging.getLogger(__name__)` নামযুক্ত লগার দেয়, যাতে কোন মডিউল থেকে বার্তা এল তা বোঝা যায়। রুট লগার মূল লগার; সরাসরি `logging.info()` লিখলে সেটিই ব্যবহৃত হয়। হ্যান্ডলার ঠিক করে বার্তা কোথায় যাবে — কনসোলে, ফাইলে, নাকি দুটোতেই। ফরম্যাটার ঠিক করে বার্তা কেমন দেখাবে। প্রোডাকশনে `print()` ব্যবহার করবেন না: তার কোনো স্তর, সময় বা ফাইল-নিয়ম নেই।

স্তর	সংখ্যা	ব্যবহার
`DEBUG`	10	বিস্তারিত অনুসন্ধান, শুধু উন্নয়নে
`INFO`	20	সাধারণ অগ্রগতি: শুরু, শেষ
`WARNING`	30	অস্বাভাবিক, কিন্তু কাজ চলছে
`ERROR`	40	কাজ ব্যর্থ, হস্তক্ষেপ দরকার
`CRITICAL`	50	গুরুতর বিপদ, সেবা ঝুঁকিতে

কোড: logger.info("backup started") ↓ লগার: স্তর যাচাই করে ↓ হ্যান্ডলার: Console / File-এ পাঠায় ↓ ফরম্যাটার: সময় + স্তর + নাম + বার্তা সাজায় ↓ গন্তব্য: টার্মিনাল ও /var/log/technova/backup.log

১৯.৬ স্ট্রাকচার্ড লগ ও রোটেটিং লগ

স্ট্রাকচার্ড লগ মানে প্রতিটি লাইনে একই ক্ষেত্র থাকা, যেমন `level=INFO task=backup server=db-01 status=ok` বা JSON আকারে `{"level": "INFO", "task": "backup"}`। তখন `grep` দিয়ে নির্দিষ্ট সার্ভার বা কাজের বার্তা সহজে পাওয়া যায়। `RotatingFileHandler`-এ `maxBytes` একটি ফাইলের সর্বোচ্চ আকার ও `backupCount` কয়টি পুরোনো ফাইল রাখা হবে তা ঠিক করে; আকার পূর্ণ হলে পুরোনো ফাইল `.1`, `.2` নামে সরে যায়। `TimedRotatingFileHandler` সময়ের ভিত্তিতে ঘোরে, যেমন প্রতিদিন মধ্যরাতে নতুন ফাইল।

১৯.৭ রিট্রাই, টাইমআউট ও যাচাই — Retry, Timeout and Validation

নেটওয়ার্ক কল মাঝে মাঝে ব্যর্থ হয়, আবার চেষ্টা করলে সফল হয় — এই ক্ষণস্থায়ী ভুলে রিট্রাই যুক্তিযুক্ত। ফিক্সড রিট্রাই মানে প্রতিবার একই বিরতি। এক্সপোনেনশিয়াল ব্যাকঅফ মানে বিরতি বাড়ে: ১, ২, ৪, ৮ সেকেন্ড। জিটার মানে বিরতিতে সামান্য এলোমেলো সময় যোগ করা, যাতে অনেক ক্লায়েন্ট একসঙ্গে আবার চেষ্টা না করে। সর্বোচ্চ চেষ্টার সংখ্যা সবসময় ঠিক করুন। আগে আইডেমপোটেন্সি নিশ্চিত করুন: একই কাজ দুবার করলে ফল একই থাকতে হবে। টাকা কাটা বা বার্তা পাঠানোর POST অন্ধভাবে রিট্রাই করা যায় না। শুধু `TimeoutError` বা `ConnectionError`-এর মতো ক্ষণস্থায়ী ভুলে রিট্রাই করুন। প্রতিটি বাহ্যিক কলে টাইমআউট বাধ্যতামূলক। ইনপুট ও কনফিগ শুরুতেই যাচাই করুন — ফেইল ফাস্ট নীতি: ভুল যত আগে ধরা পড়ে, ক্ষতি তত কম। গার্ড ক্লজ মানে ফাংশনের শুরুতেই অবৈধ মান ফিরিয়ে দেওয়া বা এক্সেপশন তোলা।

১৯.৮ অটোমেশন পরীক্ষা করা — Testing Automation Scripts

`pytest`-এ পরীক্ষা ফাংশনের নাম `test_` দিয়ে শুরু হয়, ভেতরে `assert` দিয়ে প্রত্যাশা যাচাই করা হয়। `tmp_path` ফিক্সচার প্রতিটি পরীক্ষার জন্য অস্থায়ী ফোল্ডার দেয়, যাতে আসল ফাইল নষ্ট না হয়। `subprocess` বা API আসলে চালানো উচিত নয়; `unittest.mock`-এর `patch` দিয়ে তার নকল বসিয়ে যাচাই করা যায় স্ক্রিপ্টটি সঠিক আর্গুমেন্টে ডেকেছে কি না।

১৯.৯ নির্ভরযোগ্যতার নীতি — Reliability Principles

আইডেমপোটেন্সি মানে একই স্ক্রিপ্ট দুবার চালালে দ্বিতীয়বার ক্ষতি না হওয়া। ড্রাই-রান মানে আসল পরিবর্তন না করে শুধু দেখানো কী করা হতো; `--dry-run` বিপজ্জনক স্ক্রিপ্টের জন্য প্রায় বাধ্যতামূলক। অ্যাটমিক অপারেশন মানে কাজ হয় পুরো হবে, নয়তো একদম হবে না — যেমন অস্থায়ী ফাইলে লিখে শেষে নাম বদলানো। এক্সিট কোড: ০ মানে সফল, ১ মানে সাধারণ ব্যর্থতা, ২ মানে ভুল ব্যবহার। ব্যর্থ হলে সতর্কবার্তা পাঠান ও রানবুক রাখুন — রানবুক হলো নথি, যেখানে লেখা থাকে ভুল হলে প্রথমে কী দেখতে হবে ও কীভাবে পুনরুদ্ধার করতে হবে।

> **বাস্তব ঘটনা:** TechNova-র এক রাতে ডিস্ক পূর্ণ হয়ে কপি মাঝপথে থেমেছিল। পুরোনো স্ক্রিপ্ট অর্ধেক ফাইল রেখে চুপচাপ শেষ হয়েছিল। নতুন স্ক্রিপ্ট অস্থায়ী ফাইলে লেখে, শেষে নাম বদলায়, লগে আকার লেখে এবং ব্যর্থ হলে এক্সিট কোড ১ দিয়ে সতর্কবার্তা পাঠায়।

১৯.১০ কোড ১ — RotatingFileHandler সহ লগিং সেটআপ

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("technova.backup")
logger.setLevel(logging.INFO)
```

```
formatter = logging.Formatter("%(asctime)s level=%(levelname)s logger=%(name)s task=backup %
(message)s")
console_handler = logging.StreamHandler()
console_handler.setFormatter(formatter)
file_handler = RotatingFileHandler("/var/log/technova/backup.log", maxBytes=1048576,
backupCount=5)
file_handler.setFormatter(formatter)
logger.addHandler(console_handler)
logger.addHandler(file_handler)
logger.info("backup started server=db-01")
logger.warning("disk usage is above 80 percent")
logger.error("backup failed path=/data/app.db")
```

`import logging` লগিং মডিউল আনে; `from logging.handlers import RotatingFileHandler` ঘূর্ণায়মান ফাইল হ্যান্ডলার আনে। `getLogger("technova.backup")` নামযুক্ত লগার দেয়, `setLevel(logging.INFO)` INFO ও তার উপরের স্তর রাখে। `Formatter(...)` বার্তার ছাঁচ: `%(asctime)s` সময়, `%(levelname)s` স্তর, `%(name)s` লগারের নাম, `%(message)s` আসল বার্তা, `task=backup` স্থির প্রসঙ্গ। `StreamHandler()` কনসোলে, `RotatingFileHandler(...)` ফাইলে পাঠায়; `maxBytes=1048576` মানে ১ মেগাবাইটে ঘোরানো, `backupCount=5` মানে পাঁচটি পুরোনো ফাইল রাখা। `setFormatter(formatter)` ছাঁচ বসায়, `addHandler(...)` হ্যান্ডলার যুক্ত করে। শেষ তিন লাইনে `info`, `warning`, `error` তিন স্তরের বার্তা।

১৯.১১ কোড ২ — কাস্টম এক্সেপশন ও নির্দিষ্ট হ্যান্ডলিং

```
import json

class BackupError(Exception):
    pass

class ConfigError(Exception):
    pass

def load_config(path):
    try:
        with open(path, encoding="utf-8") as handle:
            data = json.load(handle)
    except FileNotFoundError as exc:
        raise ConfigError("config file is missing") from exc
    except json.JSONDecodeError as exc:
        raise ConfigError("config file is not valid JSON") from exc
    if "source" not in data:
        raise ConfigError("config must contain source")
    return data

def run_backup(config):
    try:
        source = config["source"]
        print(source)
    except KeyError as exc:
        raise BackupError("backup source is missing") from exc
    except PermissionError:
        raise BackupError("no permission to read backup source")
```

`import json` JSON মডিউল আনে। `class BackupError(Exception):` `Exception` থেকে নতুন শ্রেণি বানায়, `pass` মানে ভেতরে আর কিছু নেই; `ConfigError` একইভাবে কনফিগ ভুলের শ্রেণি। `def load_config(path):` ফাংশন ঘোষণা। `try:` ঝুঁকিপূর্ণ অংশ, `with open(path, encoding="utf-8") as handle:` ফাইল খুলে শেষে নিজে থেকে বন্ধ করে, `json.load(handle)` JSON পড়ে। `except FileNotFoundError as exc:` ফাইল না পেলে ভুলকে `exc` নামে ধরে, `raise ConfigError(...) from exc` মূল কারণসহ অর্থবহ ভুল তোলে। দ্বিতীয় `except json.JSONDecodeError` ভুল JSON-এ একই কাজ করে। `if "source" not in data:` প্রয়োজনীয় চাবি যাচাই করে। `run_backup`-এ `config["source"]` চাবি দিয়ে মান নেয়, `except KeyError` চাবি না থাকলে `BackupError`-এ বদলায়, `except PermissionError:` অনুমতির ভুল আলাদাভাবে ধরে। `print(source)` শুধু উদাহরণ।

১৯.১২ কোড ৩ — এক্সপোনেনশিয়াল ব্যাকঅফসহ রিট্রাই

```
import time
import random

def retry_with_backoff(func, attempts=4, base_delay=1):
    last_error = None
    for attempt in range(attempts):
        try:
            return func()
        except (TimeoutError, ConnectionError) as exc:
            last_error = exc
            delay = base_delay * (2 ** attempt) + random.uniform(0, 1)
            time.sleep(delay)
    raise last_error

def fetch_status():
    return "ok"

result = retry_with_backoff(fetch_status)
print(result)
```

`import time` সময়ের মডিউল আনে, `import random` এলোমেলো সংখ্যার মডিউল আনে। `def retry_with_backoff(func, attempts=4, base_delay=1):` ফাংশন ঘোষণা: `func` চালানোর কাজ, `attempts` সর্বোচ্চ চেষ্টা, `base_delay` শুরুর বিরতি। `last_error = None` শেষ ভুল রাখার জায়গা খালি রাখে। `for attempt in range(attempts):` নির্দিষ্টবার চেষ্টার লুপ, `try:` ঝুঁকিপূর্ণ কল, `return func()` সফল হলে ফল ফিরিয়ে বেরিয়ে যায়। `except (TimeoutError, ConnectionError) as exc:` শুধু এই দুটি ক্ষণস্থায়ী ভুল ধরে। `delay = base_delay * (2 ** attempt) + random.uniform(0, 1)` বিরতি গণনা করে: `2 ** attempt` মানে ২-এর ঘাত, অর্থাৎ বিরতি দ্বিগুণ হয়, `random.uniform(0, 1)` ০–১ সেকেন্ড জিটার যোগ করে। `time.sleep(delay)` অপেক্ষা করে। লুপ শেষে `raise last_error` সব চেষ্টা ব্যর্থ হলে শেষ ভুল আবার তোলে। `def fetch_status():` নমুনা কাজ, `retry_with_backoff(fetch_status)` তাকে রিট্রাই মোড়কে চালায়।

> **প্রো টিপ:** রিট্রাইয়ের সংখ্যা ও বিরতি লগ করুন। লগে `attempt=2 delay=4` থাকলে বোঝা যায় সার্ভার ধীর ছিল নাকি নেটওয়ার্কই বন্ধ ছিল।

১৯.১৩ কোড ৪ — pytest দিয়ে পরীক্ষা

```
import json
from pathlib import Path

def write_health_report(path, status):
    data = {"service": "backup", "status": status}
    Path(path).write_text(json.dumps(data), encoding="utf-8")
    return data

def test_write_health_report(tmp_path):
    report = tmp_path / "health.json"
    result = write_health_report(report, "ok")
    assert result["status"] == "ok"
    assert report.exists()
    loaded = json.loads(report.read_text(encoding="utf-8"))
    assert loaded["service"] == "backup"

def test_missing_status_value():
    data = {"service": "backup"}
    assert data.get("status", "unknown") == "unknown"
```

`import json` JSON মডিউল আনে, `from pathlib import Path` ফাইল-পথের শ্রেণি আনে। `def write_health_report(path, status):` প্রতিবেদন লেখার ফাংশন। `data = {"service": "backup", "status": status}` ডিকশনারি তৈরি করে, `Path(path).write_text(json.dumps(data), encoding="utf-8")` তাকে JSON অক্ষরে বদলে ফাইলে লেখে, `return data` ফল ফিরিয়ে দেয়। `def test_write_health_report(tmp_path):` পরীক্ষা ফাংশন; `test_` দিয়ে শুরু বলে pytest খুঁজে পায়, `tmp_path` pytest-এর দেওয়া অস্থায়ী ফোল্ডার। `report = tmp_path / "health.json"` অস্থায়ী পথ বানায়। `assert result["status"] == "ok"` মান যাচাই করে, `assert report.exists()` ফাইল তৈরি হয়েছে কি না যাচাই করে। `json.loads(report.read_text(encoding="utf-8"))` ফাইল পড়ে আবার ডিকশনারিতে বদলায়, শেষ `assert` সেবার নাম যাচাই করে। দ্বিতীয় পরীক্ষায় `data.get("status", "unknown")` চাবি না থাকলে ডিফল্ট মান নেয়, যাতে `KeyError` না হয়। `pytest test_health_check.py -v` চালালে দুটি পরীক্ষাই পাস করার কথা।

১৯.১৪ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Observability	চলমান সিস্টেমের ভেতরের অবস্থা বাইরে থেকে বোঝার ক্ষমতা
Log level	বার্তার গুরুত্বের স্তর, যেমন INFO বা ERROR
Handler	লগ বার্তা কোথায় যাবে তা ঠিক করা অংশ
Formatter	লগ বার্তার আকার ও ক্ষেত্রের ক্রম ঠিক করা অংশ
Structured logging	প্রতিটি লাইনে একই ক্ষেত্রসহ লেখা লগ, খোঁজা সহজ
Idempotency	একই কাজ বারবার করলেও ফল একই থাকা
Exponential backoff	প্রতিবার বিরতি দ্বিগুণ করে আবার চেষ্টা করা

Jitter	রিট্রাইয়ের বিরতিতে যোগ করা সামান্য এলোমেলো সময়
Dry-run	আসল পরিবর্তন না করে কী করা হতো তা দেখানো
Runbook	ভুল হলে ধাপে ধাপে কী করতে হবে তার লেখা নির্দেশনা

১৯.১৫ অনুশীলন — Guided Lab

ল্যাব: TechNova-র নির্ভরযোগ্য ব্যাকআপ স্ক্রিপ্ট

1. ফোল্ডার `~/technova-backup` তৈরি করে `backup.py` খুলুন।
2. ১৯.১০-এর লগিং সেটআপ বসান, পথ বদলে `backup.log` নিজের ফোল্ডারে রাখুন।
3. ১৯.১১-এর `BackupError` ও `ConfigError` যোগ করে ছোট `config.json` পড়ুন।
4. নমুনা ফাইল কপিতে ১৯.১২-এর `retry_with_backoff` ব্যবহার করুন।
5. `--dry-run` যোগ করুন: এই মোডে শুধু লগ লেখা হবে, কপি হবে না।
6. সফলে এক্সিট কোড ০, ব্যর্থতায় ১ দিয়ে শেষ করুন; `echo $?` দিয়ে যাচাই করুন।
7. `test_health_check.py` লিখে `pytest -v` চালান।
8. ইচ্ছা করে কনফিগের নাম বদলে চালান এবং লগে স্পষ্ট ভুল বার্তা দেখুন।

প্রত্যাশিত ফলাফল ও যাচাই

সফল হলে টার্মিনালে INFO বার্তা ও `backup.log`-এ সময়সহ একই বার্তা থাকবে। নিচের নমুনাটি উদাহরণমূলক, আপনার সময় ও পথ ভিন্ন হবে।

```
2026-11-08 10:15:00 level=INFO logger=technova.backup task=backup backup started server=db-01
2026-11-08 10:15:01 level=INFO logger=technova.backup task=backup backup finished files=3
```

যাচাই করুন: লগে `backup started` ও `backup finished` আছে কি না, `echo $?`-এর মান ০ কি না, pytest-এ দুটি পরীক্ষাই পাস করেছে কি না। কনফিগ ভুল থাকলে ERROR লাইন আসবে ও এক্সিট কোড ১ হবে।

১৯.১৬ সাধারণ ভুল ও ডিবাগিং — Common Mistakes

সমস্যা সমাধান: লগে কিছু না এলে প্রথমে লগারের স্তর দেখুন — `WARNING` সেট থাকলে INFO আসবে না। হ্যান্ডলার যোগ হয়েছে কি না ও ফাইলে লেখার অনুমতি আছে কি না, তাও দেখুন।

- খালি `except:` — সব ভুল ধরে আসল কারণ লুকায়।
- এক্সেপশন ধরে কিছু না করা — ভুল গিলে ফেলা; অন্তত লগ করুন বা আবার তুলুন।
- লগে গোপন তথ্য লেখা — পাসওয়ার্ড, টোকেন বা API চাবি কখনো লগে নয়।
- `print()` দিয়ে প্রোডাকশন লগ করা — স্তর, সময় ও ফাইল-নিয়ম নেই।
- অ-আইডেমপোটেন্ট POST রিট্রাই করা — দুবার টাকা কাটা বা বার্তা যেতে পারে।
- বাহ্যিক কলে টাইমআউট না দেওয়া — স্ক্রিপ্ট অনির্দিষ্টকাল আটকে থাকে।

- লগ ফাইল না ঘোরানো — ডিস্ক ভরে সার্ভার থেমে যায়।
- ভুল এক্সিট কোড — ব্যর্থ স্ক্রিপ্ট ০ দিলে cron ভাবে সব ঠিক আছে।

১৯.১৭ নিরাপত্তা বিবেচনা — Security Considerations

লগে কখনো গোপন তথ্য লিখবেন না: পাসওয়ার্ড, টোকেন, API চাবি, ওয়ান-টাইম কোড বা গ্রাহকের সংবেদনশীল তথ্য নয়। প্রয়োজনে মান ঢেকে লিখুন, যেমন `token=****1234`, অথবা ক্ষেত্রটি বাদ দিন। লগ ফাইলের অনুমতি সীমিত রাখুন, যেমন `chmod 600 backup.log`। জাতীয় পরিচয় নম্বর বা ঠিকানার মতো ব্যক্তিগত তথ্য লগে লেখার আগে ভাবুন তা আসলেই দরকার কি না। গোপন মান পরিবেশ ভেরিয়েবলে রাখুন ও কখনো Git-এ পাঠাবেন না।

১৯.১৮ সেরা অনুশীলন — Best Practices

- স্ক্রিপ্টের শুরুতে লগিং সেটআপ করুন, শেষে সারাংশ লগ করুন।
- নির্দিষ্ট এক্সেপশন ধরুন, ভুল বার্তায় কী করতে হবে তা লিখুন।
- শুরুতেই কনফিগ ও ইনপুট যাচাই করুন; দেরিতে ব্যর্থ হওয়ার চেয়ে আগেভাগে থামা ভালো।
- প্রতিটি বাহ্যিক কলে টাইমআউট দিন, রিট্রাই শুধু ক্ষণস্থায়ী ভুলে দিন।
- ড্রাই-রান ও আইডেমপোটেন্সি ডিজাইনের অংশ করুন।
- সঠিক এক্সিট কোড দিন, ব্যর্থতায় সতর্কবার্তা পাঠান, রানবুক আপডেট রাখুন।
- পরীক্ষা ছাড়া অটোমেশন প্রোডাকশনে পাঠাবেন না — ছোট হলেও `pytest` চালান।

১৯.১৯ যাচাই প্রশ্ন — MCQs

১. প্রোডাকশনে `print()`-এর বদলে `logging` কেন?
ক) `print()` দ্রুত চলে বলে
খ) লগিংয়ে স্তর, সময় ও গন্তব্য নিয়ন্ত্রণ করা যায় বলে
গ) লগিংয়ে কোড ছোট হয় বলে
ঘ) `print()` Python 3.12-এ নেই বলে

২. `RotatingFileHandler`-এ `backupCount=5`-এর অর্থ কী?
ক) পাঁচবার লগ লেখা হবে
খ) পাঁচটি পুরোনো লগ ফাইল রাখা হবে
গ) পাঁচ মিনিট পর লগ ঘুরবে
ঘ) পাঁচজন ব্যবহারকারী লগ পড়তে পারবে

৩. কোন ভুলে রিট্রাই করা সাধারণত নিরাপদ?
ক) ভুল পাসওয়ার্ড দিলে
খ) ভুল ফাইল পথ দিলে
গ) ক্ষণস্থায়ী নেটওয়ার্ক টাইমআউট হলে
ঘ) কনফিগে প্রয়োজনীয় চাবি না থাকলে

৪. `raise ConfigError("...") from exc`-এ `from exc`-এর কাজ কী?
ক) নতুন ভুলটি মুছে ফেলা
খ) মূল ভুলের সঙ্গে নতুন ভুলের সম্পর্ক রাখা

গ) ভুলটি চুপচাপ গিলে ফেলা
ঘ) স্ক্রিপ্টটি আবার চালু করা

৫. ব্যর্থ স্ক্রিপ্টের জন্য সঠিক এক্সিট কোড কোনটি?
ক) সবসময় ০
খ) ব্যর্থ হলে শূন্য ছাড়া অন্য মান, যেমন ১
গ) সবসময় ২৫৫
ঘ) এক্সিট কোডের কোনো অর্থ নেই

১৯.২০ স্বাধীন অ্যাসাইনমেন্ট — Assignment

TechNova-র জন্য একটি দৈনিক ডিস্ক-পরিষ্কার স্ক্রিপ্ট লিখুন, যেখানে লগিং, কাস্টম এক্সেপশন, ড্রাই-রান, টাইমআউটসহ রিট্রাই ও অন্তত দুটি pytest পরীক্ষা থাকবে। স্ক্রিপ্টটি যেন ব্যর্থ হলে স্পষ্ট লগ লিখে এক্সিট কোড ১ দেয়। কোনো সমাধান এখানে দেওয়া হয়নি।

১৯.২১ সারাংশ ও পুনরালোচনা — Summary

নির্ভরযোগ্য অটোমেশন মানে শুধু কাজ করা নয় — কাজের প্রমাণ রাখা, ভুল হলে জোরে জানানো ও আবার চালালেও নিরাপদ থাকা। নির্দিষ্ট ভুল ধরা, স্তরভিত্তিক ও স্ট্রাকচার্ড লগ রাখা, রোটেটিং ফাইলে ডিস্ক বাঁচানো, শুধু ক্ষণস্থায়ী ভুলে এক্সপোনেনশিয়াল ব্যাকঅফে রিট্রাই করা, টাইমআউট ও আগেভাগে যাচাই করা এবং pytest দিয়ে পরীক্ষা করা — এই সব মিলেই স্ক্রিপ্ট প্রোডাকশনের উপযোগী হয়।

- ☐ এক্সেপশন হায়ারার্কি আঁকতে ও নির্দিষ্ট `except` লিখতে পারি।
- ☐ `getLogger`, হ্যান্ডলার ও ফরম্যাটার দিয়ে লগিং সেটআপ করতে পারি।
- ☐ DEBUG, INFO, WARNING, ERROR ও CRITICAL স্তরের পার্থক্য বলতে পারি।
- ☐ স্ট্রাকচার্ড লগ ও রোটেটিং লগের কারণ ব্যাখ্যা করতে পারি।
- ☐ এক্সপোনেনশিয়াল ব্যাকঅফ, জিটার ও আইডেমপোটেন্সি ব্যাখ্যা করতে পারি।
- ☐ প্রতিটি বাহ্যিক কলে টাইমআউট দেওয়ার নিয়ম মনে রাখতে পারি।
- ☐ `tmp_path` ও `patch` দিয়ে pytest পরীক্ষা লিখতে পারি।
- ☐ সঠিক এক্সিট কোড, ড্রাই-রান ও রানবুকের ব্যবহার বলতে পারি।
- ☐ লগে গোপন তথ্য না লেখার নিয়ম মানতে পারি।

এখানেই Part III শেষ হলো। আমরা Git দিয়ে পরিবর্তন ধরে রাখতে, Bash ও Python দিয়ে কাজ স্বয়ংক্রিয় করতে, API ও কনফিগের সঙ্গে কাজ করতে এবং সেই অটোমেশনকে নির্ভরযোগ্য করতে শিখেছি। Part IV-এ আমরা এই স্ক্রিপ্টগুলোকে সময়মতো চালানো, দূরের সার্ভারে চালানো ও নিজের ছোট টুল হিসেবে গুছিয়ে তুলব — যেখানে এই অধ্যায়ের লগিং, এক্সেপশন ও এক্সিট কোডই হবে প্রথম নিরাপত্তা-জাল।

PART IV • DEVOPS ENGINEERING MASTERY

Automation Engineering

অটোমেশন ইঞ্জিনিয়ারিং — cron দিয়ে সময়সূচি, SSH দিয়ে দূরবর্তী সার্ভার, ও পুনঃব্যবহারযোগ্য টুল তৈরি।

অধ্যায় ২০ • Part IV — Automation Engineering

Chapter 20: Scheduling, Cron and Background Automation — নির্ধারিত সময়ে স্বয়ংক্রিয় কাজ: Cron ও Background Automation

শুরুতে: একটি বাস্তব দৃশ্য

TechNova নামের একটি ছোট software company-র ঢাকার অফিসে প্রতি রাত ২টায় একটি কাজ করতে হয়: সার্ভারের ডেটাবেস backup নেওয়া, পুরনো log ফাইল মুছে ফেলা, আর disk কতটা ভরেছে তা দেখে একটি রিপোর্ট তৈরি করা। প্রথম কয়েক মাস একজন junior engineer রাত জেগে এই কাজগুলো হাতে করতেন। একদিন তিনি অসুস্থ হয়ে ছুটি নিলেন, আর সেই রাতে backup হলো না। পরের দিন সকালে একটি ভুল deploy-এ ডেটাবেস নষ্ট হলো, কিন্তু ফিরে যাওয়ার মতো সাম্প্রতিক backup ছিল না। কোম্পানি সেদিন শিখল একটি কঠিন সত্য: মানুষের স্মৃতি ও উপস্থিতির উপর নির্ভর করে চলা কাজ কখনো নির্ভরযোগ্য ব্যবস্থা নয়। যে কাজ নিয়মিত, একইভাবে, নির্দিষ্ট সময়ে করতে হয়, সেই কাজটি কম্পিউটারকেই করতে দিতে হয় — মানুষ শুধু ব্যবস্থাটি বানাবে, পরীক্ষা করবে, আর তার ফলাফল দেখবে। এই অধ্যায়ে আমরা সেই ব্যবস্থার সবচেয়ে পুরনো, সবচেয়ে নির্ভরযোগ্য আর সবচেয়ে বেশি ব্যবহৃত হাতিয়ারটি শিখব: cron।

এই অধ্যায়টি আগের সব অধ্যায়ের উপর দাঁড়িয়ে আছে। Chapter 07–13-এর Bash script, Chapter 14–19-এর Python automation, আর Chapter 15-এর backup ধারণা — সবকিছুই তখনই সত্যিকার কাজে লাগে যখন সেগুলো নিজে নিজে, ঠিক সময়ে, মানুষের হস্তক্ষেপ ছাড়া চলে। একটি ভালো backup script লিখতে পারা অর্ধেক কাজ; সেটিকে প্রতিদিন রাত ২টায় নির্ভরযোগ্যভাবে চালানোই বাকি অর্ধেক।

> **CONCEPT:** Scheduling মানে ভবিষ্যতের একটি সময়ে একটি কাজ ঘটবে, এই প্রতিশ্রুতি দেওয়া। কম্পিউটার ঘুমায় না, ভোলে না, ছুটি নেয় না — তাই নিয়মিত কাজের জন্য সে মানুষের চেয়ে ভালো কর্মী। কিন্তু সে নির্দেশনা অক্ষরে অক্ষরে পালন করে; schedule-এ ভুল থাকলে ভুল কাজটিই নিখুঁতভাবে, বারবার ঘটবে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Scheduled task কেন দরকার, আর মানুষের হাতে করা কাজের সঙ্গে তার পার্থক্য ব্যাখ্যা করতে।
- Cron daemon কীভাবে কাজ করে, তা first principles থেকে বর্ণনা করতে।
- Crontab syntax-এর পাঁচটি ক্ষেত্র (minute, hour, day of month, month, day of week) সঠিকভাবে লিখতে ও পড়তে।
- পরিবেশগত পার্থক্য (PATH, HOME, shell) বুঝে cron-এ চলার মতো script লিখতে।
- Scheduled job-এর output ও error log-এ পাঠাতে এবং log দেখে ব্যর্থতা নির্ণয় করতে।
- `flock` দিয়ে একই job-এর দুটি copy একসঙ্গে চলা আটকাতে।
- systemd timer-এর সঙ্গে cron-এর তুলনা করতে ও কখন কোনটি ব্যবহার করবেন তা সিদ্ধান্ত নিতে।
- Backup ও health check-এর একটি সম্পূর্ণ দৈনিক schedule ডিজাইন করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–13: Bash script লেখা, permission, exit code, logging।
- Chapter 15: backup ধারণা ও `tar`, `rsync`।
- Chapter 19: structured logging ও retry-র ধারণা।
- Terminal-এ ফাইল তৈরি, সম্পাদনা ও `systemctl status` চালানোর অভিজ্ঞতা।

২০.১ WHY: সময়মতো কাজ কেন মানুষের উপর ছেড়ে দেওয়া যায় না

একটি কাজকে তিনটি প্রশ্নে মাপুন: কাজটি কি নিয়মিত ঘটে? কাজটি কি প্রতিবার একইভাবে করতে হয়? কাজটি ভুলে গেলে বা দেরি হলে কি ক্ষতি হয়? তিনটির উত্তরই হ্যাঁ হলে কাজটি scheduling-এর উপযুক্ত। Backup, log rotation, disk পরীক্ষা, certificate মেয়াদ পরীক্ষা, অস্থায়ী ফাইল মোছা, দৈনিক রিপোর্ট পাঠানো — সবই এই শ্রেণিতে পড়ে।

মানুষের উপর নির্ভরশীল ব্যবস্থার তিনটি দুর্বলতা আছে। প্রথম, মানুষ ভোলে। দ্বিতীয়, মানুষ অসুস্থ হয়, ছুটিতে যায়, চাকরি বদলায় — আর তার সঙ্গে সঙ্গে অলিখিত নিয়মটিও হারিয়ে যায়। তৃতীয়, মানুষ প্রতিবার কাজটি সামান্য ভিন্নভাবে করে, ফলে ফলাফল অনির্ভরযোগ্য হয়। একটি scheduled job এই তিনটি দুর্বলতাই দূর করে: সে ভোলে না, সে চলে যায় না, আর প্রতিবার একই নির্দেশনা একইভাবে পালন করে। তবে এর উল্টো দিকটিও সত্য: মানুষের ভুল সিদ্ধান্ত schedule-এ ঢুকলে সেই ভুলও প্রতিদিন, নিখুঁতভাবে ঘটবে। তাই scheduling শেখার আগে testing ও logging শেখা জরুরি — এই অধ্যায়ে দুটোই আছে।

২০.২ Cron Architecture: ভেতরে আসলে কী ঘটে

cron হলো একটি daemon — অর্থাৎ background-এ সারাক্ষণ চলা একটি program, যার কাজ শুধু ঘড়ি দেখা আর সময়মতো command চালানো। Ubuntu Server-এ সাধারণত `cron` package-টি আগে থেকেই থাকে। সিস্টেম boot হলে `cron.service` চালু হয়, আর প্রতি মিনিটে একবার সে তার schedule table (crontab) পড়ে দেখে: এই মিনিটে কোনো job চালানোর কথা আছে কি না। থাকলে সে job-টি চালায়, না থাকলে আবার অপেক্ষা করে। এই সরলতাই cron-এর শক্তি: তার কোনো জটিল অবস্থা (state) নেই, কোনো network দরকার নেই, আর সে ব্যর্থ হলেও পরের মিনিটে আবার চেষ্টা করে।

দুটি স্তর আলাদা করে বুঝুন। **System crontab** (`/etc/crontab` ও `/etc/cron.d/`) সিস্টেমের কাজের জন্য, এখানে প্রতিটি লাইনে কোন user হিসেবে job চলবে তা লেখা থাকে। **User crontab** প্রতিটি user-এর নিজের schedule, যা `crontab -e` দিয়ে সম্পাদনা করা হয়। `crontab -l` দিয়ে নিজের schedule দেখা যায়, `crontab -r` দিয়ে মুছে ফেলা যায় — শেষেরটি অত্যন্ত বিপজ্জনক, কারণ মুছে ফেলার আগে কোনো নিশ্চিতকরণ চাওয়া হয় না।

Cron Flow — একটি মিনিটের ভেতরে কী ঘটে

System boot → cron daemon চালু → প্রতি মিনিটে crontab পড়া → এই মিনিটে মিলেছে এমন job আছে কি?
হ্যাঁ → job-টি নির্দিষ্ট user হিসেবে চালানো → stdout/stderr log-এ লেখা → পরের মিনিটের অপেক্ষা
না → কিছু না করে পরের মিনিটের অপেক্ষা
মনে রাখুন: cron job মিস হলে সে পরে ধরে নেয় না — নির্ধারিত মিনিট পেরিয়ে গেলে সেই run-টি চিরতরে হারিয়ে যায় (systemd timer-এর persistent বিকল্প ২০.৭-এ দেখুন)।

উপাদান	কাজ	অবস্থান / command

cron daemon	প্রতি মিনিটে schedule পরীক্ষা ও job চালানো	`systemctl status cron`
User crontab	একজন user-এর ব্যক্তিগত schedule	`crontab -e`, `crontab -l`
System crontab	সিস্টেমের কাজ, user-সহ লেখা থাকে	`/etc/crontab`, `/etc/cron.d/`
Daily / hourly directory	Script ফেলে রাখলেই নির্দিষ্ট সময়ে চলে	`/etc/cron.daily/`, `/etc/cron.hourly/`
Log	কোন job কখন চলল, তার প্রমাণ	`/var/log/syslog`, নিজের log ফাইল

২০.৩ Crontab Syntax: পাঁচটি ক্ষেত্র, একটি নিয়ম

প্রতিটি cron লাইনের গঠন একটাই: পাঁচটি সময়ের ক্ষেত্র, তারপর চালাতে হবে এমন command। ক্ষেত্রগুলোর ক্রম মুখস্থ করার সবচেয়ে ভালো উপায় হলো বড় থেকে ছোট নয়, ছোট থেকে বড় ভাবা: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন।

```
* * * * * command
| | | | |
| | | | └── সপ্তাহের দিন (0-7, 0 ও 7 দুটোই রবিবার)
| | | └──── মাস (1-12)
| | └────── মাসের দিন (1-31)
| └──────── ঘণ্টা (0-23)
└────────── মিনিট (0-59)
```

প্রতিটি ক্ষেত্রে ব্যবহার করা যায়: `*` (প্রতিটি মান), সংখ্যা (একটি নির্দিষ্ট মান), `,` (তালিকা, যেমন `1,15`), `-` (পরিসর, যেমন `9-17`), `/` (ধাপ, যেমন `*/15` মানে প্রতি ১৫ মিনিটে)। একটি সাধারণ ভুল: `*/15` আর `15`-কে গুলিয়ে ফেলা। প্রথমটি মানে প্রতি ১৫ মিনিটে, দ্বিতীয়টি মানে শুধু ঘণ্টার ১৫তম মিনিটে।

Crontab লাইন	অর্থ
`0 2 * * * /opt/technova/backup.sh`	প্রতিদিন রাত ২টায় backup
`*/15 * * * * /opt/technova/health.sh`	প্রতি ১৫ মিনিটে health check
`0 9 * * 1-5 /opt/technova/report.sh`	সোম–শুক্র সকাল ৯টায় রিপোর্ট
`30 23 * * 0 /opt/technova/cleanup.sh`	প্রতি রবিবার রাত ১১:৩০-এ পরিষ্কার
`0 0 1 * * /opt/technova/monthly.sh`	প্রতি মাসের ১ তারিখে মাসিক কাজ
`@reboot /opt/technova/startup.sh`	সার্ভার reboot হলে একবার চালানো

> **WARNING:** মাসের দিন ও সপ্তাহের দিন দুটোই নির্দিষ্ট করলে cron তাদের OR হিসেবে ধরে, AND হিসেবে নয়। `0 0 1 * 1` মানে মাসের ১ তারিখ হলেই চলবে, আবার সোমবার হলেও চলবে — দুটি শর্তের যেকোনো একটি মিললেই চলবে। এটি cron-এর সবচেয়ে পরিচিত ফাঁদ; দুটি শর্তই একসঙ্গে চাইলে script-এর ভেতরে তারিখ পরীক্ষা করতে হয়।

২০.৪ Environment পার্থক্য: Terminal-এ চলে, Cron-এ চলে না কেন

নতুনদের সবচেয়ে বেশি ভোগায় এই সমস্যাটি: script terminal-এ নিখুঁতভাবে চলে, কিন্তু cron-এ চালালে ব্যর্থ হয় বা আংশিক চলে। কারণ cron-এর পরিবেশ আপনার interactive shell-এর পরিবেশ নয়। cron একটি ন্যূনতম পরিবেশে job

চালায়: PATH খুব ছোট (সাধারণত `/usr/bin:/bin`), HOME ছাড়া প্রায় কোনো environment variable থাকে না, কোনো terminal সংযুক্ত থাকে না, আর আপনার `.bashrc` বা `.profile` পড়া হয় না। তাই `python3` বা `aws`-এর মতো command terminal-এ পাওয়া গেলেও cron-এ `command not found` হতে পারে, কারণ সেগুলো `/usr/local/bin`-এ আছে, যা cron-এর PATH-এ নেই।

সমাধানের তিনটি নিয়ম। প্রথম, script-এ সব command-এর **absolute path** ব্যবহার করুন (`/usr/bin/python3`, `/usr/bin/tar`), অথবা script-এর শুরুতে নিজে PATH নির্ধারণ করুন। দ্বিতীয়, script-এর ভেতরে কোনো কিছুই বর্তমান directory (working directory)-এর উপর নির্ভর করবেন না — cron job-টি user-এর HOME থেকে চালায়, আপনার project directory থেকে নয়। তৃতীয়, virtual environment বা config ফাইলের পথ absolute করে লিখুন। একটি script cron-এ দেওয়ার আগে `env -i` দিয়ে খালি পরিবেশে পরীক্ষা করুন — এটিই cron-এর পরিবেশের সবচেয়ে কাছাকাছি অনুকরণ।

```
env -i HOME=/home/nahid PATH=/usr/bin:/bin /opt/technova/backup.sh
```

উপরের লাইনটির ব্যাখ্যা: `env -i` সব environment variable মুছে খালি পরিবেশ বানায়, তারপর শুধু HOME ও PATH দিয়ে script চালায়। এতে চললে cron-এও চলার সম্ভাবনা অনেক বেশি।

> **PRO TIP:** Crontab ফাইলের শুরুতেই নিজের পরিবেশ লিখে রাখুন — যেমন `PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin` এবং `SHELL=/bin/bash`। এতে প্রতিটি job একই পরিবেশ পায়, আর কোন job কোন PATH-এ চলছে তা অনুমান করতে হয় না।

২০.৫ Logging Scheduled Jobs: যা log-এ নেই, তা ঘটেনি

Cron নিজে job-এর output কোথাও দেখায় না — কোনো পর্দা নেই। তাই প্রতিটি cron job-এর output ও error একটি log ফাইলে পাঠানো বাধ্যতামূলক। না পাঠালে output ইমেইল হিসেবে জমা হয় (যদি mail সাজানো থাকে) অথবা হারিয়ে যায়, আর ব্যর্থতা নীরবে ঘটে। সঠিক নিয়ম: stdout ও stderr দুটোই একটি তারিখযুক্ত বা ঘূর্ণায়মান (rotating) log ফাইলে পাঠানো, আর প্রতিটি run-এর শুরুতে তারিখ-সময় লেখা।

```
0 2 * * * /opt/technova/backup.sh >> /var/log/technova/backup.log 2>&1
```

এই লাইনটির ব্যাখ্যা: `>>` মানে stdout-কে backup.log-এ যুক্ত করা (আগের লেখা মুছে নয়), আর `2>&1` মানে stderr-কেও একই জায়গায় পাঠানো। শুধু `>` ব্যবহার করলে প্রতিদিন আগের log মুছে যেত — এটিই একটি সাধারণ ভুল। Log ফাইলের আকার বাড়তে থাকলে `logrotate` দিয়ে তা ঘোরাতে হয়, নইলে একদিন disk ভরে যাবে নিজের log-এর ভারে।

২০.৬ Duplicate Execution আটকানো: flock

একটি backup job যদি নির্ধারিত সময়ের চেয়ে বেশি সময় নেয়, আর পরের schedule এসে যায়, তাহলে একই job-এর দুটি copy একসঙ্গে চলবে। দুটি backup একই ফাইলে লিখলে ফাইল নষ্ট হতে পারে, দুটি cleanup একই ফাইল মুছতে গেলে দ্বন্দ্ব হতে পারে। এই সমস্যার মানসম্মত সমাধান `flock` — একটি lock ফাইল দিয়ে পারস্পরিক বর্জন (mutual exclusion)।

```
0 2 * * * /usr/bin/flock -n /var/lock/technova-backup.lock /opt/technova/backup.sh >> /var/log/technova/backup.log 2>&1
```

ব্যাখ্যা: `flock -n` lock ফাইলটি ধরে রাখার চেষ্টা করে। আগের copy এখনো চললে lock পাওয়া যাবে না, আর `-n` (non-blocking) থাকায় নতুন copy অপেক্ষা না করে সঙ্গে সঙ্গে শেষ হয়ে যাবে — কোনো দ্বিতীয় copy চলবে না। এই আচরণটি ইচ্ছাকৃত: একটি মিস হওয়া backup run পরের দিন আবার চলবে, কিন্তু একই সঙ্গে দুটি backup চলার বিপদ তার চেয়ে বড়। Lock ফাইলটি `/var/lock/`-এ রাখা হয়, কারণ এটি অস্থায়ী অবস্থা, স্থায়ী তথ্য নয়।

২০.৭ systemd Timers: Cron-এর আধুনিক বিকল্প

Ubuntu Server 24.04-এ cron-এর পাশাপাশি **systemd timer** আছে। Timer-এর সুবিধা: সে service-এর উপর নির্ভরতা (dependency) বোঝে, মিস হওয়া run সার্ভার চালু হলে ধরে নিতে পারে (`Persistent=true`), আর প্রতিটি run-এর log `journalctl`-এ পাওয়া যায়। অসুবিধা: সেটআপে দুটি ফাইল লাগে (একটি `.timer`, একটি `.service`), আর syntax cron-এর চেয়ে দীর্ঘ। একটি সরল নিয়ম: একক সার্ভারে, সরল সময়সূচিতে cron যথেষ্ট ও সবাই বোঝে; সার্ভার বন্ধ থাকলে মিস হওয়া run ধরে নিতে হলে, বা job-এর নির্ভরতা জটিল হলে, systemd timer ভালো।

বিষয়	cron	systemd timer
সেটআপ	এক লাইন crontab-এ	দুটি unit ফাইল
মিস হওয়া run	হারিয়ে যায়	`Persistent=true` হলে ধরে নেয়
Logging	নিজে ফাইলে পাঠাতে হয়	`journalctl -u <service>`
নির্ভরতা	নেই	অন্য service-এর পরে চালানো যায়
বহনযোগ্যতা	সব Unix-like সিস্টেমে আছে	শুধু systemd আছে এমন সিস্টেমে

২০.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Cron / Cron daemon	নির্ধারিত সময়ে job চালানো background service
Crontab	একজন user বা সিস্টেমের schedule table
Daemon	background-এ সারাক্ষণ চলা service program
Schedule / Interval	কখন চালাতে হবে, তার নিয়ম / দুটি run-এর মধ্যবর্তী সময়
Environment variable	program-এর আচরণ নিয়ন্ত্রণকারী নাম-মান জোড়া
Absolute path	root (/) থেকে শুরু করা সম্পূর্ণ পথ
Lock file / flock	একই job দুবার চলা আটকানোর তালা / সেই তালা পরিচালনার command
Log rotation	পুরনো log সংরক্ষণ করে নতুন log শুরু করা
systemd timer	systemd-এর সময়-ভিত্তিক job চালানোর ব্যবস্থা
Background job	terminal-এর সঙ্গে যুক্ত না থেকে চলা কাজ

REAL WORLD: TechNova-র সিলেট অফিসের একটি সার্ভারে প্রতি ঘণ্টায় disk পরীক্ষা করার cron job ছিল, কিন্তু তার output কোথাও যেত না। একদিন disk ১০০% ভরে গেল, ডেটাবেস বন্ধ হয়ে গেল, আর কেউ জানতেও পারল না যে health check-টি তিন সপ্তাহ ধরে ব্যর্থ হচ্ছিল — কারণ তার error কেউ দেখেনি। সমাধান ছিল একটি লাইন: output-কে log-এ পাঠানো, আর log-এ ERROR শব্দ খুঁজে সতর্ক করার আরেকটি ছোট job। Scheduling-এর আসল কাজ সময়মতো চালানো নয়, ব্যর্থতা সময়মতো জানানো।

২০.৯ Code Walkthrough: দৈনিক Backup ও Health Check Schedule

নিচের crontab-টি TechNova-র একটি সার্ভারের সম্পূর্ণ দৈনিক schedule। প্রতিটি লাইন পড়ুন, তারপর ব্যাখ্যা মেলান।

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOME=/home/nahid
0 2 * * * /usr/bin/flock -n /var/lock/technova-backup.lock /opt/technova/backup.sh >> /var/log/technova/backup.log 2>&1
*/15 * * * * /opt/technova/health.sh >> /var/log/technova/health.log 2>&1
0 9 * * 1-5 /opt/technova/report.sh >> /var/log/technova/report.log 2>&1
30 3 * * 0 /opt/technova/cleanup.sh >> /var/log/technova/cleanup.log 2>&1
```

লাইন ধরে ব্যাখ্যা: প্রথম তিন লাইন পরিবেশ নির্ধারণ করে — কোন shell, কোন PATH, কার HOME। চতুর্থ লাইন রাত ২টায় backup চালায়, flock দিয়ে duplicate আটকায়, output backup.log-এ পাঠায়। পঞ্চম লাইন প্রতি ১৫ মিনিটে health check চালায়। ষষ্ঠ লাইন সোম–শুক্র সকাল ৯টায় রিপোর্ট বানায়। সপ্তম লাইন রবিবার রাত ৩:৩০-এ পুরনো ফাইল পরিষ্কার করে — ব্যস্ত সময়ের বাইরে, যাতে cleanup নিজেই সার্ভারকে ধীর না করে।

Backup script-টির ভেতরেও একটি গুরুত্বপূর্ণ নিয়ম আছে: সে তার নিজের log-এ তারিখ লেখে।

```
#!/bin/bash
set -euo pipefail
BACKUP_DIR="/var/backups/technova"
SOURCE_DIR="/srv/technova/data"
DATE=$(date +%F)
mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_DIR/data-$DATE.tar.gz" "$SOURCE_DIR"
echo "backup completed: $DATE"
```

ব্যাখ্যা: `set -euo pipefail` কোনো ধাপ ব্যর্থ হলে script-কে থামিয়ে দেয়, যাতে অর্ধেক backup সফল বলে মনে না হয়। `DATE=$(date +%F)` আজকের তারিখ নেয় (যেমন 2026-11-08), ফলে প্রতিদিনের backup ফাইলের নাম আলাদা হয়। `mkdir -p` directory না থাকলে বানায়, থাকলে কিছু করে না — এটি idempotency-র একটি ছোট উদাহরণ, Chapter 24-এ বিস্তারিত আসবে।

২০.১০ Guided Practical Exercise — Lab 20: Cron-এ দৈনিক Backup চালানো

LAB: এই lab-এ আপনি একটি সত্যিকারের cron job বানাবেন, তাকে পরীক্ষা করবেন, আর প্রমাণ করবেন যে সে ঠিক সময়ে চলে।

1. একটি পরীক্ষার directory বানান: `mkdir -p ~/lab20/data ~/lab20/backups` এবং `echo "important data" > ~/lab20/data/notes.txt` লিখুন।

2. `~/lab20/backup.sh` নামে একটি script লিখুন, যাতে `set -euo pipefail`, তারিখযুক্ত `tar -czf` backup, আর সফলতার বার্তা থাকে। `chmod +x ~/lab20/backup.sh` দিয়ে executable করুন।
3. প্রথমে হাতে চালিয়ে দেখুন: `~/lab20/backup.sh` — backup ফাইল তৈরি হলো কি না `ls -lh ~/lab20/backups/` দিয়ে দেখুন।
4. খালি পরিবেশে পরীক্ষা করুন: `env -i HOME=$HOME PATH=/usr/bin:/bin ~/lab20/backup.sh` — এতে চললে cron-এও চলবে।
5. `crontab -e` খুলে পরীক্ষার জন্য একটি লাইন যোগ করুন, যা প্রতি মিনিটে চলবে: `* * * * * /home/nahid/lab20/backup.sh >> /home/nahid/lab20/backup.log 2>&1` — নিজের username দিয়ে পথটি বদলে নিন।
6. দুই মিনিট অপেক্ষা করুন, তারপর `cat ~/lab20/backup.log` ও `ls -l ~/lab20/backups/` দিয়ে দেখুন job-টি চলেছে কি না।
7. পরীক্ষা শেষে crontab থেকে পরীক্ষার লাইনটি মুছে ফেলুন, আর আসল schedule যোগ করুন: `0 2 * * *` দিয়ে রাত ২টায় চলার নিয়ম লিখুন।
8. Cleanup: `rm -rf ~/lab20` দিয়ে পরীক্ষার ফাইল মুছুন।

Expected results and verification (illustrative):

```
$ ls -lh ~/lab20/backups/
-rw-r--r-- 1 nahid nahid 180 Nov  8 23:16 data-2026-11-08.tar.gz
$ cat ~/lab20/backup.log
backup completed: 2026-11-08
backup completed: 2026-11-08
```

যাচাই: দুই মিনিটে log-এ দুটি লাইন থাকলে job-টি প্রতি মিনিটে চলেছে — প্রমাণিত। একটি লাইনও না থাকলে `grep CRON /var/log/syslog` দিয়ে দেখুন cron job-টি চেষ্টা করেছিল কি না, আর `systemctl status cron` দিয়ে দেখুন daemon চলছে কি না।

২০.১১ Common Mistakes ও Debugging

- **ভুল:** আপেক্ষিক পথ (relative path) ব্যবহার করা — cron-এর working directory আপনার project নয়, তাই `./data` খুঁজে পাওয়া যাবে না। সমাধান: সব পথ absolute লিখুন।
- **ভুল:** শুধু `>` দিয়ে log লেখা — প্রতিটি run আগের log মুছে দেয়। সমাধান: `>>` ব্যবহার করুন।
- **ভুল:** `%` চিহ্ন crontab-এ সরাসরি ব্যবহার করা — cron-এ `%` মানে newline। `date +%F` crontab লাইনে লিখতে চাইলে `\%` লিখতে হয়, অথবা তারিখের কাজটি script-এর ভেতরে রাখুন — দ্বিতীয়টি ভালো।
- **সমস্যা:** job চলছে না — প্রথমে `crontab -l` দিয়ে লাইনটি আছে কি না দেখুন, তারপর `grep CRON /var/log/syslog` দিয়ে cron চেষ্টা করেছিল কি না দেখুন, তারপর script-টি হাতে ও খালি পরিবেশে চালিয়ে দেখুন।
- **সমস্যা:** job চলছে কিন্তু ফলাফল ভুল — প্রায় সবসময় পরিবেশের পার্থক্য। `env -i` পরীক্ষাটি করুন, আর script-এর শুরুতে `env > /tmp/cron-env.txt` লিখে cron-এর পরিবেশটি দেখে নিন।

TROUBLESHOOTING: একটি cron job মাঝে মাঝে চলে, মাঝে মাঝে চলে না — এর সবচেয়ে পরিচিত কারণ হলো job-টি আগের run শেষ হওয়ার আগেই আবার শুরু হওয়া, আর দুটি copy একে অপরের ফাইল নষ্ট করা। লক্ষণ: আংশিক ফাইল, খালি backup, log-এ অদ্ভুত error। সমাধান: `flock -n` যোগ করুন, আর job-টি কতক্ষণ চলে তা log-এ লিখুন।

২০.১২ Security Considerations

Scheduled job root হিসেবে চালালে তার script-এ একটি ভুল পুরো সিস্টেমের ক্ষতি করতে পারে — যেমন ভুল directory-তে `rm -rf`। তাই নীতি: প্রতিটি job শুধু সেই user হিসেবে চালান, যার যতটুকু permission দরকার, ততটুকুই আছে। Backup job-এর শুধু পড়ার ও backup directory-তে লেখার অধিকার থাকলেই চলে; তার root হওয়ার দরকার নেই। দ্বিতীয়ত, cron job-এ কখনো password বা token লিখবেন না — Chapter 18-এর নিয়ম এখানেও প্রযোজ্য: secret environment variable বা permission-সুরক্ষিত ফাইল থেকে পড়তে হয়। তৃতীয়ত, `/etc/cron.d/`-এ ফাইল লেখার অধিকার শুধু administrator-এর — সাধারণ user সেখানে লিখতে পারলে যেকোনো সময়ে root-এর job বসিয়ে দিতে পারত।

২০.১৩ Best Practices

- প্রতিটি cron job-এর output ও error log-এ পাঠান; নীরব job মানে অদৃশ্য ব্যর্থতা।
- Script-কে cron-এ দেওয়ার আগে হাতে ও `env -i` দিয়ে পরীক্ষা করুন।
- দীর্ঘ job-এ `flock -n` ব্যবহার করুন।
- ভারী job রাতের কম ব্যস্ত সময়ে চালান, আর একই মিনিটে অনেক ভারী job সাজাবেন না — সার্ভার একসঙ্গে সব কাজে ব্যস্ত হয়ে যাবে।
- Crontab-এর একটি copy Git-এ রাখুন, যাতে schedule-এর পরিবর্তনের ইতিহাস থাকে।
- Log-এর আকার নিয়ন্ত্রণে `logrotate` ব্যবহার করুন।

> **CHECK YOUR UNDERSTANDING:** নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. cron daemon কতক্ষণ পরপর schedule পরীক্ষা করে? ক) প্রতি সেকেন্ডে খ) প্রতি মিনিটে গ) প্রতি ঘণ্টায় ঘ) শুধু boot-এর সময়
2. `0 2 * * *` লাইনটির অর্থ কী? ক) প্রতি ২ মিনিটে খ) প্রতিদিন রাত ২টায় গ) প্রতি মাসের ২ তারিখে ঘ) প্রতি ২ ঘণ্টায়
3. Terminal-এ চলা script cron-এ ব্যর্থ হওয়ার সবচেয়ে পরিচিত কারণ কোনটি? ক) script-টি খুব বড় খ) cron-এর পরিবেশে PATH ও environment variable আলাদা গ) cron শুধু Python চালায় ঘ) script-এর নাম ভুল
4. `flock -n`-এর কাজ কী? ক) ফাইল encrypt করা খ) একই job-এর দুটি copy একসঙ্গে চলা আটকানো গ) job-কে দ্রুত চালানো ঘ) log ফাইল মুছে ফেলা
5. cron job-এর output কোথায় পাঠানো উচিত? ক) কোথাও নয়, cron নিজে দেখায় খ) একটি log ফাইলে, stdout ও stderr দুটোই গ) শুধু terminal-এ ঘ) ইমেইলে পাঠানো ছাড়া আর কোথাও নয়

২০.১৪ Independent Assignment

Assignment 20.1 (Intermediate): TechNova-র একটি সার্ভারের জন্য একটি সম্পূর্ণ সাপ্তাহিক schedule ডিজাইন করুন: দৈনিক backup (রাত ২টা), প্রতি ১৫ মিনিটে health check, সোম–শুক্র সকাল ৯টায় রিপোর্ট, রবিবার রাতে cleanup। প্রতিটি job-এর crontab লাইন, তার log ফাইলের পথ, flock ব্যবহারের জায়গা, আর প্রতিটি job ব্যর্থ হলে কীভাবে জানা যাবে — সব লিখুন। সঙ্গে ব্যাখ্যা করুন কেন ভারী job-গুলো একই মিনিটে সাজানো হয়নি। সম্পূর্ণ সমাধান Answer Guide-এ আছে; আগে নিজে চেষ্টা করুন।

২০.১৫ Chapter Summary

নিয়মিত, একইভাবে, সময়মতো করতে হয় এমন কাজ মানুষের স্মৃতির উপর ছেড়ে দেওয়া যায় না — সেগুলো cron-এর কাজ। Cron প্রতি মিনিটে crontab পড়ে নির্ধারিত job চালায়। Crontab-এর পাঁচটি ক্ষেত্র মিনিট, ঘণ্টা, মাসের দিন, মাস ও সপ্তাহের দিন নির্ধারণ করে। Cron-এর পরিবেশ interactive shell-এর চেয়ে ন্যূনতম, তাই absolute path ও পরীক্ষা অপরিহার্য। প্রতিটি job-এর output log-এ পাঠাতে হয়, দীর্ঘ job-এ flock দিয়ে duplicate আটকাতে হয়, আর মিস হওয়া run ধরে নিতে হলে systemd timer বিবেচনা করতে হয়।

Revision Checklist

- ☐ Crontab-এর পাঁচটি ক্ষেত্রের ক্রম ও অর্থ বলতে পারি।
- ☐ `*/15` ও `15`-এর পার্থক্য ব্যাখ্যা করতে পারি।
- ☐ Cron-এর পরিবেশ কেন আলাদা, তা বলতে পারি ও `env -i` দিয়ে পরীক্ষা করতে পারি।
- ☐ Output log-এ পাঠানোর সঠিক redirection লিখতে পারি।
- ☐ `flock -n` দিয়ে duplicate execution আটকাতে পারি।
- ☐ Cron ও systemd timer-এর তুলনা করতে পারি।

অধ্যায় ২১ • Part IV — Automation Engineering

Chapter 21: SSH Automation and Remote System Management — SSH দিয়ে দূরবর্তী সার্ভার পরিচালনা

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র এখন চারটি সার্ভার: ঢাকায় দুটি, চট্টগ্রামে একটি, সিলেটে একটি। একদিন সকালে ঢাকার মূল সার্ভারে disk প্রায় ভরে গেল। একজন engineer প্রতিটি সার্ভারে আলাদা করে login করে, password টাইপ করে, একই command বারবার চালিয়ে সমস্যা খুঁজলেন — চারটি সার্ভারে চারবার একই কাজ, চারবার টাইপের ভুলের সুযোগ। শেষ সার্ভারে পৌঁছাতে পৌঁছাতে প্রথম সার্ভারের অবস্থা বদলে গেছে। সেদিন দলটি বুঝল: একাধিক সার্ভার হাতে পরিচালনা করা যায় না; সার্ভার পরিচালনা করতে হয় একটি নিরাপদ, স্বয়ংক্রিয়, পুনরাবৃত্তিযোগ্য উপায়ে। সেই উপায়ের নাম SSH — Secure Shell। এই অধ্যায়ে আমরা SSH-কে শুধু login করার হাতিয়ার হিসেবে নয়, একটি পূর্ণ remote automation ব্যবস্থা হিসেবে শিখব।

> **CONCEPT:** SSH-এর মূল ধারণা সহজ: আপনি দূরের কম্পিউটারে এমনভাবে কাজ করবেন যেন আপনি তার সামনে বসে আছেন, কিন্তু মাঝখানের নেটওয়ার্কে কেউ আপনার টাইপ করা কিছু পড়তে বা বদলাতে পারবে না। নিরাপত্তা এখানে পরে যোগ করা বৈশিষ্ট্য নয় — এটিই SSH-এর অস্তিত্বের কারণ।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- SSH কীভাবে encryption ও authentication দিয়ে নিরাপদ channel বানায়, তা ব্যাখ্যা করতে।
- Public-key authentication সেটআপ করতে ও password login-এর চেয়ে তা কেন নিরাপদ, তা বলতে।
- `~/.ssh/config` দিয়ে অনেক সার্ভারের সংযোগ সহজ ও নিরাপদ করতে।
- Known hosts ও host key verification-এর মানে বুঝতে ও পরিবর্তিত host key দেখলে সঠিক প্রতিক্রিয়া জানতে।
- একটি command দূরের সার্ভারে চালাতে ও তার ফলাফল সংগ্রহ করতে।
- `scp` ও `rsync` দিয়ে নিরাপদে ফাইল পাঠাতে ও তাদের পার্থক্য ব্যাখ্যা করতে।
- Python দিয়ে SSH automation-এর ধারণা (paramiko) ব্যাখ্যা করতে ও তার ঝুঁকিগুলো চিনতে।
- একাধিক সার্ভারে একই কাজ নিরাপদে চালানোর একটি পরিকল্পনা লিখতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 07–13: Bash, command, exit code, file permission।
- Chapter 20: cron ও scheduled job।
- Public-key cryptography-র প্রাথমিক ধারণা (private key গোপন, public key ভাগ করা যায়) — না থাকলে ২১.২ পড়ে শুরু করুন।
- দুটি Linux মেশিন বা একটি VM, যেখানে পরীক্ষা করা যায়।

২১.১ WHY: দূরের সার্ভারে কাজ করার নিরাপদ উপায় কেন দরকার

আগে দূরের কম্পিউটারে কাজ করা হতো Telnet দিয়ে — username, password, সবকিছু খোলা পাঠ্য (plaintext) হিসেবে নেটওয়ার্কে যেত। একই নেটওয়ার্কে থাকা যে কেউ তা পড়তে পারত। SSH সেই সমস্যার সমাধান হিসেবে এসেছে: সংযোগের শুরুতেই দুই পক্ষ একটি গোপন চাবি (session key) নিয়ে একমত হয়, তারপর সবকিছু encrypt করে পাঠায়। কেউ মাঝখানে শুনলেও শুধু অর্থহীন অক্ষর দেখবে। দ্বিতীয়ত, SSH প্রমাণ করে আপনি কার সঙ্গে কথা বলছেন (host verification) ও আপনি কে (authentication) — দুটোই। শুধু encrypt করলেই হবে না; ভুল সার্ভারের সঙ্গে encrypt করা সংযোগও বিপজ্জনক, কারণ আপনি আপনার password ভুল হাতে তুলে দিচ্ছেন।

২১.২ SSH Architecture: ভেতরে কী ঘটে

একটি SSH সংযোগ তিনটি ধাপে তৈরি হয়। প্রথম ধাপে client ও server পরস্পরকে চেনে: server তার host key দেখায়, client তা তার known hosts তালিকার সঙ্গে মিলিয়ে দেখে। মিললে এগোয়, না মিললে সতর্ক করে। দ্বিতীয় ধাপে তারা একটি session key তৈরি করে, যাতে এরপরের সবকিছু encrypt হয়। তৃতীয় ধাপে ব্যবহারকারীর পরিচয় যাচাই হয় — password দিয়ে বা public key দিয়ে। পরিচয় ঠিক হলে server একটি shell বা একটি command চালানোর অনুমতি দেয়। এই তিনটি ধাপের প্রতিটি আলাদা সমস্যার সমাধান করে: প্রথমটি man-in-the-middle আক্রমণ আটকায়, দ্বিতীয়টি গোপনীয়তা দেয়, তৃতীয়টি অনুমতি নিয়ন্ত্রণ করে।

SSH Handshake — সংযোগের তিন ধাপ

Client → Server: সংযোগের অনুরোধ

Server → Client: host key দেখানো → Client যাচাই করে (known_hosts-এ আছে কি? মিলছে কি?)

Client ↔ Server: session key তৈরি → এরপর সবকিছু encrypted

Client → Server: পরিচয় প্রমাণ (public key / password)

Server → Client: অনুমতি পেলে shell বা command চালানো

⚠ প্রথম ধাপে host key না মিললে এগিয়ে যাওয়া মানে অচেনা কারও সঙ্গে কথা বলা — থামুন ও যাচাই করুন।

উপাদান	কাজ	অবস্থান
sshd (server)	সংযোগ গ্রহণ, প্রমাণ যাচাই, command চালানো	সার্ভারে, port 22 (default)
ssh (client)	সার্ভারে সংযোগ করা	আপনার মেশিনে
Host key	সার্ভারের পরিচয় প্রমাণ করে	সার্ভারে `/etc/ssh/`
Private key	আপনার গোপন পরিচয় — কখনো ভাগ করবেন না	`~/.ssh/id_ed25519`
Public key	সার্ভারে রাখা হয়, private key-র সঙ্গে মিলিয়ে যাচাই করে	সার্ভারে `~/.ssh/authorized_keys`
known_hosts	আগে দেখা সার্ভারগুলোর host key-র তালিকা	`~/.ssh/known_hosts`

২১.৩ Public-Key Authentication: Password-এর চেয়ে কেন ভালো

Password-এর সমস্যা তিনটি: তা অনুমান করা যায়, তা চুরি হলে যেকোনো জায়গা থেকে ব্যবহার করা যায়, আর তা প্রতিবার টাইপ করতে হয় — ফলে automation-এ ব্যবহার করা যায় না। Public-key authentication-এ আপনার কাছে

থাকে একটি জোড়া চাবি: **private key** (শুধু আপনার কাছে, কখনো নেটওয়ার্কে যায় না) ও **public key** (সার্ভারে রাখা হয়)। Login-এর সময় সার্ভার একটি চ্যালেঞ্জ পাঠায়, যা শুধু private key দিয়ে সমাধান করা যায়। Private key কখনো সার্ভারে যায় না, তাই নেটওয়ার্কে তা চুরি হওয়ার সুযোগ নেই। এই পদ্ধতিতে automation-ও সম্ভব: script নিজে private key ব্যবহার করে login করতে পারে, password টাইপ না করে।

```
ssh-keygen -t ed25519 -C "nahid@technova"
ssh-copy-id nahid@203.0.113.10
ssh nahid@203.0.113.10
```

লাইন ধরে ব্যাখ্যা: প্রথম লাইনটি Ed25519 ধরনের একটি key জোড়া বানায় — আধুনিক, দ্রুত ও নিরাপদ; `-C` শুধু একটি মন্তব্য, যাতে পরে বোঝা যায় key-টি কার। দ্বিতীয় লাইনটি public key-টি সার্ভারে পাঠিয়ে `authorized_keys`-এ যোগ করে। তৃতীয় লাইনটি login পরীক্ষা করে — এখন password চাওয়া উচিত নয়, যদি key-টি সঠিকভাবে বসে থাকে। Key বানানোর সময় একটি passphrase দেওয়া ভালো অভ্যাস: এতে laptop চুরি হলেও private key একা ব্যবহার করা যাবে না। Automation-এ passphrase ব্যবহার করলে `ssh-agent` দিয়ে তা একবার unlock করে রাখতে হয়।

> **WARNING:** Private key-র permission অবশ্যই `600` হতে হবে (শুধু মালিক পড়তে পারবে), আর `~/.ssh` directory-র permission `700`। Permission ঢিলেঢালা হলে SSH key-টি ব্যবহার করতে অস্বীকার করে — এটি বিরক্তিকর মনে হলেও নিরাপত্তার জন্য ইচ্ছাকৃত। Private key কখনো Git-এ, ইমেইলে, চ্যাটে বা screenshot-এ পাঠাবেন না।

২১.৪ SSH Configuration: অনেক সার্ভার, একটি সহজ ফাইল

চারটি সার্ভারের IP, username, port আলাদা আলাদা মনে রাখা ও প্রতিবার টাইপ করা ভুলের উৎস। `~/.ssh/config` ফাইলে প্রতিটি সার্ভারের জন্য একটি সংক্ষিপ্ত নাম (alias) নির্ধারণ করা যায়, আর সেই নামেই সংযোগ করা যায়।

```
Host dhaka-web
    HostName 203.0.113.10
    User nahid
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 60

Host ctg-db
    HostName 203.0.113.20
    User nahid
    IdentityFile ~/.ssh/id_ed25519
```

ব্যাখ্যা: `Host dhaka-web` একটি ডাকনাম নির্ধারণ করে; এরপর `ssh dhaka-web` লিখলেই HostName, User ও key-টি ব্যবহৃত হয়। `ServerAliveInterval 60` প্রতি ৬০ সেকেন্ডে একটি ছোট signal পাঠায়, যাতে নীরব সংযোগ মাঝপথে কেটে না যায় — দীর্ঘ কাজের সময় এটি কাজে লাগে। এই ফাইলটির permission-ও `600` রাখা উচিত, কারণ এতে সার্ভারের ঠিকানা ও ব্যবহারকারীর নাম থাকে।

২১.৫ Known Hosts ও Host Verification: সতর্কবার্তা উপেক্ষা করবেন না

প্রথমবার কোনো সার্ভারে সংযোগ করলে SSH জিজ্ঞাসা করে: এই সার্ভারের fingerprint-টি কি বিশ্বাস করবেন? এটি যাচাই করার সঠিক উপায় হলো সার্ভারের administrator-এর কাছ থেকে বা সার্ভার প্রদানকারীর console থেকে fingerprint মিলিয়ে দেখা। একবার বিশ্বাস করলে তা `known_hosts`-এ জমা হয়। পরে কোনোদিন একই সার্ভারের host key বদলে গেলে

SSH জোরালো সতর্কবার্তা দেয় — এটি উপেক্ষা করার বার্তা নয়। Host key বদলানোর বৈধ কারণ থাকতে পারে (সার্ভার নতুন করে install করা), কিন্তু তা আক্রমণের লক্ষণও হতে পারে (man-in-the-middle)। সঠিক প্রতিক্রিয়া: থামুন, সার্ভারের মালিকের সঙ্গে অন্য কোনো channel-এ (ফোন, অফিসের চ্যাট) যোগাযোগ করে যাচাই করুন, তারপর পুরনো key-টি সরিয়ে নতুনটি গ্রহণ করুন।

> **WARNING:** `StrictHostKeyChecking=no` বা `UserKnownHostsFile=/dev/null` ব্যবহার করা মানে host verification বন্ধ করে দেওয়া — অর্থাৎ আপনি আর যাচাই করছেন না কার সঙ্গে কথা বলছেন। এটি automation-কে সহজ করে, কিন্তু man-in-the-middle আক্রমণের দরজা খুলে দেয়। স্বাভাবিক সমাধান হিসেবে এটি কখনো ব্যবহার করবেন না। সঠিক উপায়: প্রথমবার হাতে যাচাই করে host key `known_hosts`-এ জমা করা, অথবা `ssh-keyscan` দিয়ে বিশ্বস্ত channel থেকে key সংগ্রহ করা।

২১.৬ Remote Command Execution ও File Transfer

SSH শুধু interactive login-এর জন্য নয়। একটি command সরাসরি দূরের সার্ভারে চালানো যায়, আর তার output আপনার terminal-এ ফিরে আসে — এটিই remote automation-এর ভিত্তি।

```
ssh dhaka-web "df -h / && systemctl is-active nginx"
ssh dhaka-web "sudo systemctl restart nginx"
```

ব্যাখ্যা: প্রথম লাইনটি দূরের সার্ভারে disk ব্যবহার ও nginx service-এর অবস্থা দেখে। দ্বিতীয় লাইনটি service restart করে। মনে রাখুন: উদ্ধৃতিচিহ্নের ভেতরের command দূরের shell-এ চলে, আপনার shell-এ নয়। তাই আপনার shell-এর variable সেখানে কাজ করবে না — যা দরকার তা command-এর ভেতরেই লিখতে হয়, অথবা script ফাইল পাঠিয়ে চালাতে হয়।

ফাইল পাঠাতে দুটি হাতিয়ার আছে। `scp` সহজ — একটি ফাইল বা directory copy করা। `rsync` শক্তিশালী — শুধু বদলে যাওয়া অংশ পাঠায়, বড় directory বা backup-এর জন্য অনেক দ্রুত, আর মাঝপথে থেমে গেলে আবার শুরু করা যায়।

```
scp /home/nahid/report.txt dhaka-web:/tmp/report.txt
rsync -avz /srv/technova/data/ dhaka-web:/srv/technova/data/
rsync -avz --delete /srv/technova/data/ dhaka-web:/srv/technova/data/
```

ব্যাখ্যা: `scp` লাইনটি একটি ফাইল পাঠায়। প্রথম `rsync` লাইনে `-a` মানে archive mode (permission ও সময়সহ copy), `-v` মানে বিস্তারিত দেখানো, `-z` মানে পাঠানোর সময় সংকুচিত করা। উৎস পথের শেষে `/` থাকার মানে directory-র ভেতরের জিনিস copy করা; `/` না থাকলে directory-টি নিজেই copy হয় — এই ছোট পার্থক্যটি ভুল করলে ফাইল ভুল জায়গায় যাবে। তৃতীয় লাইনের `--delete` গন্তব্যে থাকা অতিরিক্ত ফাইল মুছে দেয় — এটি শক্তিশালী, তাই প্রথমে `--dry-run` দিয়ে পরীক্ষা করুন।

২১.৭ Python SSH Automation: Paramiko ধারণা

Bash দিয়ে কয়েকটি সার্ভারে command চালানো সহজ, কিন্তু কাজ জটিল হলে — যেমন প্রতিটি সার্ভারের ফলাফল JSON-এ সাজানো, ব্যর্থ সার্ভার আলাদা করে log করা, API-তে ফলাফল পাঠানো — Python ভালো। Python-এ SSH-এর জন্য সবচেয়ে পরিচিত library **paramiko**। এটি SSH protocol-কে Python থেকে ব্যবহার করতে দেয়: সংযোগ করা, command চালানো, ফাইল পাঠানো। এই বইয়ে paramiko install করে চালানো বাধ্যতামূলক নয়; ধারণাটি বোঝাই লক্ষ্য, কারণ production-এ অনেক দল paramiko-র বদলে Ansible-এর মতো উচ্চস্তরের হাতিয়ার ব্যবহার করে — তবে ভেতরে সেই একই SSH-ই চলে।

```
import paramiko
client = paramiko.SSHClient()
client.load_system_host_keys()
client.connect("203.0.113.10", username="nahid")
stdin, stdout, stderr = client.exec_command("df -h /")
print(stdout.read().decode())
client.close()
```

লাইন ধরে ব্যাখ্যা: প্রথমে একটি SSHClient বানানো হয়। `load_system_host_keys()` সিস্টেমের known_hosts পড়ে — এটিই সঠিক পদ্ধতি; host key যাচাই বন্ধ করে দেওয়ার `AutoAddPolicy` এখানে ব্যবহার করা হয়নি, কারণ তা নিরাপত্তা দুর্বল করে। `connect` key দিয়ে login করে। `exec_command` দূরে command চালায় ও তিনটি channel ফেরত দেয়: stdin, stdout, stderr। শেষে সংযোগ বন্ধ করা হয় — না বন্ধ করলে program ঝুলে থাকতে পারে। বাস্তব code-এ `connect` ও `exec_command`-এর চারপাশে try/except থাকা উচিত (Chapter 19), আর host ও username config ফাইল থেকে পড়া উচিত (Chapter 18), code-এ লিখে রাখা উচিত নয়।

২১.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
SSH / Secure Shell	দূরের কম্পিউটারে নিরাপদে কাজ করার protocol
Encryption	তথ্য এমনভাবে বদলে দেওয়া যাতে শুধু নির্দিষ্ট পক্ষ পড়তে পারে
Public key / Private key	ভাগ করা যায় এমন চাবি / গোপন রাখতে হয় এমন চাবি
Host key / Fingerprint	সার্ভারের পরিচয় / তার সংক্ষিপ্ত সারসংক্ষেপ
known_hosts	আগে বিশ্বাস করা সার্ভারগুলোর host key-র তালিকা
Man-in-the-middle	দুই পক্ষের মাঝখানে বসে কথা শোনা বা বদলানো আক্রমণ
scp / rsync	SSH দিয়ে ফাইল copy করার সহজ হাতিয়ার / শুধু বদল পাঠানো দক্ষ হাতিয়ার
ssh-agent	passphrase দিয়ে unlock করা key মনে রাখার program
Paramiko	Python থেকে SSH ব্যবহারের library
Bastion host	যে সার্ভার দিয়ে ভেতরের সার্ভারগুলোতে যাওয়া হয়

REAL WORLD: TechNova-র সিলেটের সার্ভারটি সরাসরি ইন্টারনেটে খোলা ছিল না — শুধু ঢাকার একটি bastion host দিয়ে যাওয়া যেত। Engineer-রা `~/.ssh/config`-এ `ProxyJump` লিখে রাখলেন, ফলে `ssh sylhet-web` লিখলেই সংযোগ প্রথমে bastion দিয়ে ঘুরে যেত, আর কেউ bastion-এর অস্তিত্ব নিয়েও ভাবতে হতো না। নিরাপত্তা বাড়ল, আর ব্যবহার সহজই থাকল — ভালো নকশা এমনই হয়: নিরাপদ পথটিই সহজ পথ।

২১.৯ Code Walkthrough: অনেক সার্ভারে Health Check

নিচের Bash script-টি একটি ফাইলে লেখা সার্ভার তালিকার প্রতিটি সার্ভারে গিয়ে disk ও service পরীক্ষা করে, আর ফলাফল একটি রিপোর্টে লেখে। এটি Chapter 20-এর cron job হিসেবে চালানো যায়।

```
#!/bin/bash
set -euo pipefail
SERVERS_FILE="/opt/technova/servers.txt"
REPORT="/var/log/technova/remote-health.log"
while read -r server; do
    echo "checking $server"
    ssh -o ConnectTimeout=10 "$server" "df -h / | tail -1; systemctl is-active nginx" >>
"$REPORT" 2>&1
done < "$SERVERS_FILE"
```

ব্যাখ্যা: `while read -r server` servers.txt-এর প্রতিটি লাইন পড়ে — একটি লাইনে একটি সার্ভারের নাম। `ConnectTimeout=10` মানে কোনো সার্ভার ১০ সেকেন্ডে সাড়া না দিলে অপেক্ষা না করে পরের সার্ভারে যাওয়া; নইলে একটি বন্ধ সার্ভার পুরো check-কে ঘণ্টার পর ঘণ্টা আটকে রাখতে পারে। `>>` দিয়ে প্রতিটি সার্ভারের ফলাফল একই রিপোর্টে যুক্ত হয়। এই script-এর একটি সীমা আছে: কোনো সার্ভার ব্যর্থ হলে `set -e`-এর কারণে পুরো loop থেমে যেতে পারে — বাস্তব সংস্করণে প্রতিটি ssh-এর exit code আলাদা করে দেখে ব্যর্থতা log করা উচিত, যা Chapter 22-এর অনুশীলনে আসবে।

২১.১০ Guided Practical Exercise — Lab 21: Key দিয়ে Login ও দূরে Command চালানো

> **LAB:** এই lab-এ আপনি password ছাড়া login সেটআপ করবেন ও একটি দূরের সার্ভারে command চালাবেন। দ্বিতীয় মেশিন না থাকলে নিজের মেশিনে একটি VM বা দ্বিতীয় user ব্যবহার করুন।

1. `ssh-keygen -t ed25519 -C "nahid@technova"` চালিয়ে key জোড়া বানান; passphrase দিন।
2. `ls -l ~/.ssh/` দিয়ে দেখুন private key-র permission `600` কি না; না হলে `chmod 600 ~/.ssh/id_ed25519` করুন।
3. `ssh-copy-id nahid@203.0.113.10` দিয়ে public key সার্ভারে পাঠান (IP নিজের সার্ভারেরটি দিয়ে বদলান)।
4. `ssh nahid@203.0.113.10` দিয়ে login করুন — password চাওয়া উচিত নয়।
5. Login না করে command চালান: `ssh nahid@203.0.113.10 "uptime; df -h /"`।
6. `~/.ssh/config`-এ সার্ভারের জন্য একটি Host লিখুন, তারপর শুধু `ssh dhaka-web "uptime"` দিয়ে পরীক্ষা করুন।
7. একটি ফাইল পাঠান: `scp ~/lab20/backup.sh dhaka-web:/tmp/backup.sh`, তারপর দূরে `ls -l /tmp/backup.sh` দিয়ে যাচাই করুন।
8. Cleanup: পরীক্ষার ফাইলটি দূরের সার্ভার থেকে মুছুন: `ssh dhaka-web "rm /tmp/backup.sh"`।

Expected results and verification (illustrative):

```
$ ssh dhaka-web "uptime; df -h /"
 23:16:00 up 42 days,  3:12,  1 user,  load average: 0.12, 0.08, 0.05
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        50G   12G   36G  25% /
```

যাচাই: password ছাড়া login হলে key সেটআপ ঠিক আছে। `Permission denied (publickey)` এলে সার্ভারে `~/.ssh/authorized_keys`-এর permission (`600`) ও `~/.ssh` directory-র permission (`700`) দেখুন — এটিই সবচেয়ে পরিচিত কারণ।

২১.১১ Common Mistakes ও Debugging

- **ভুল:** Private key-র permission ঢিলেঢালা রাখা — SSH তা ব্যবহার করতে অস্বীকার করে। সমাধান: `chmod 600`।
- **ভুল:** Host key সতর্কবার্তা দেখে অভ্যাসবশত `yes` বলা। সমাধান: প্রথমবার fingerprint মিলিয়ে দেখুন; পরে বদলে গেলে থামুন ও যাচাই করুন।
- **ভুল:** `scp`-এ উৎস ও গন্তব্যের ক্রম উল্টে ফেলা — ফাইল ভুল দিকে চলে যায়। সমাধান: command চালানোর আগে দুটি পথই পড়ুন।
- **ভুল:** `rsync`-এ `--delete` পরীক্ষা না করে চালানো। সমাধান: প্রথমে `--dry-run`।
- **সমস্যা:** সংযোগ ঝুলে থাকে — `ssh -v` দিয়ে বিস্তারিত দেখুন; কোন ধাপে আটকে আছে (host key, authentication, নাকি network) তা বোঝা যাবে।

TROUBLESHOOTING: `ssh` করলে `Connection timed out` আসে, কিন্তু সার্ভার চালু আছে — প্রায়ই firewall port 22 বন্ধ রেখেছে, অথবা IP ভুল। প্রথমে `ping` দিয়ে নেটওয়ার্ক, তারপর সার্ভার প্রদানকারীর console দিয়ে sshd চলছে কি না দেখুন। `Connection refused` মানে নেটওয়ার্ক পৌঁছেছে, কিন্তু port-এ কেউ শুনছে না — sshd বন্ধ বা অন্য port-এ চলছে।

২১.১২ Security Considerations

SSH নিরাপত্তার চারটি নিয়ম। প্রথম, password login বন্ধ করে শুধু key login রাখুন (`/etc/ssh/sshd_config`-এ `PasswordAuthentication no`), কারণ password অনুমান করে আক্রমণ (brute force) সবচেয়ে পরিচিত আক্রমণ। দ্বিতীয়, root-এর সরাসরি SSH login বন্ধ করুন (`PermitRootLogin no`) — প্রয়োজনে সাধারণ user হিসেবে login করে `sudo` ব্যবহার করুন। তৃতীয়, private key কখনো ভাগ করবেন না; প্রতিটি মানুষের ও প্রতিটি automation-এর আলাদা key থাকা উচিত, যাতে একটি key ফাঁস হলে শুধু সেটি বাতিল করা যায়। চতুর্থ, key-তে passphrase ব্যবহার করুন, আর laptop-এ key রাখলে disk encryption চালু রাখুন।

২১.১৩ Best Practices

- প্রতিটি সার্ভারের জন্য `~/.ssh/config`-এ alias লিখুন; IP মুখস্থ করবেন না।
- Key-এর নামে উদ্দেশ্য লিখুন (`-C`), যাতে পরে বোঝা যায় কোন key কীসের।
- Automation-এর জন্য আলাদা key বানান, আর সার্ভারে তার অধিকার সীমিত রাখুন।
- নিয়মিত `authorized_keys` পর্যালোচনা করুন — যে key আর দরকার নেই, তা মুছে ফেলুন।
- দূরের command-এর output সবসময় log-এ রাখুন, যাতে পরে প্রমাণ থাকে কী করা হয়েছিল।

CHECK YOUR UNDERSTANDING: নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Public-key authentication-এ private key কোথায় যায়? ক) সার্ভারে পাঠানো হয় খ) কখনো নেটওয়ার্কে যায় না, শুধু আপনার কাছে থাকে গ) প্রতিটি login-এ ইমেইলে পাঠানো হয় ঘ) known_hosts-এ জমা হয়
2. Host key হঠাৎ বদলে গেলে সঠিক প্রতিক্রিয়া কোনটি? ক) সতর্কবার্তা উপেক্ষা করে এগিয়ে যাওয়া খ) থামা ও অন্য channel-এ যাচাই করা গ) StrictHostKeyChecking=no লিখে দেওয়া ঘ) সার্ভারটি মুছে ফেলা

3. `rsync`-এর বড় সুবিধা `scp`-এর তুলনায় কী? ক) তা password ছাড়া চলে খ) শুধু বদলে যাওয়া অংশ পাঠায়, তাই দ্রুত গ) তা encryption করে না ঘ) তা শুধু ছোট ফাইলে কাজ করে
4. `~/.ssh/config`-এর কাজ কী? ক) সার্ভারের password মনে রাখা খ) সার্ভারের alias, user ও key এক জায়গায় লেখা গ) disk পরীক্ষা করা ঘ) cron job চালানো
5. Private key-র সঠিক permission কোনটি? ক) 644 খ) 755 গ) 600 ঘ) 777

২১.১৪ Independent Assignment

Assignment 21.1 (Intermediate): TechNova-র তিনটি সার্ভারের (ঢাকা web, চট্টগ্রাম database, সিলেট backup) জন্য একটি `~/.ssh/config` ফাইল লিখুন, প্রতিটির alias, HostName, User ও IdentityFile-সহ। তারপর একটি Bash script লিখুন যা তিনটি সার্ভারেই `uptime` ও `df -h /` চালিয়ে ফলাফল একটি log ফাইলে লেখে, কোনো সার্ভার ব্যর্থ হলে থেমে না গিয়ে পরের সার্ভারে যায় ও ব্যর্থতাটি log করে। সম্পূর্ণ সমাধান Answer Guide-এ আছে; আগে নিজে চেষ্টা করুন।

২১.১৫ Chapter Summary

SSH তিনটি কাজ একসঙ্গে করে: সংযোগ encrypt করা, সার্ভারের পরিচয় যাচাই করা, আর ব্যবহারকারীর পরিচয় যাচাই করা। Public-key authentication password-এর চেয়ে নিরাপদ ও automation-এর উপযোগী, কারণ private key কখনো নেটওয়ার্কে যায় না। `~/.ssh/config` অনেক সার্ভারের পরিচালনা সহজ করে, known_hosts host verification দেয়, আর host key বদলে গেলে থেমে যাচাই করাই সঠিক প্রতিক্রিয়া — verification বন্ধ করা কখনো সমাধান নয়। `scp` সহজ ফাইল পাঠানোর জন্য, `rsync` বড় ও পুনরাবৃত্ত copy-র জন্য, আর Python-এর paramiko একই SSH-কে program থেকে ব্যবহারের পথ দেয়।

Revision Checklist

- ☐ SSH handshake-এর তিনটি ধাপ ব্যাখ্যা করতে পারি।
- ☐ Ed25519 key বানিয়ে `ssh-copy-id` দিয়ে সার্ভারে বসাতে পারি।
- ☐ `~/.ssh/config`-এ alias লিখে সংযোগ করতে পারি।
- ☐ Host key বদলে গেলে সঠিক প্রতিক্রিয়া জানি।
- ☐ `scp` ও `rsync`-এর পার্থক্য ও `--dry-run`-এর ব্যবহার জানি।
- ☐ Paramiko-র মূল ধারণা ও host verification-এর নিয়ম বলতে পারি।

অধ্যায় ২২ • Part IV — Automation Engineering

Chapter 22: Building Reusable Automation Tools — পুনঃব্যবহারযোগ্য Automation Tool বানানো

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-তে প্রথমে একটি ছোট script ছিল — disk পরীক্ষা করে। তারপর আরেকটি — service পরীক্ষা করে। তারপর আরেকটি — backup নেয়। এক বছরে এমন ১৭টি script জমল, প্রতিটি আলাদা ভঙ্গিতে লেখা: একটিতে log যায় একটি ফাইলে, আরেকটিতে পর্দায়; একটিতে ভুল পথ দিলে নীরবে থেমে যায়, আরেকটিতে ভুল ফাইল মুছে ফেলে; একটিতে সাহায্য চাইলে কিছুই দেখায় না। নতুন একজন engineer যোগ দিয়ে কোন script কী করে, তা বুঝতে দুই সপ্তাহ ব্যয় করলেন — আর একটি পুরনো script ভুল সার্ভারে চালিয়ে ফেললেন, কারণ তার নাম থেকে বোঝা যেত না সে কী করে। সেদিন দলটি সিদ্ধান্ত নিল: আর ছড়িয়ে-ছিটিয়ে থাকা script নয়; একটি সত্যিকারের tool বানাতে হবে — যার একটি নাম, একটি সাহায্য পর্দা, একটি config, একটি log নিয়ম, আর একটি পরীক্ষা আছে। এই অধ্যায়ে আমরা সেই রূপান্তরটি শিখব: script থেকে tool।

> **CONCEPT:** Script হলো একটি কাজের জন্য লেখা নির্দেশনার তালিকা। Tool হলো এমন একটি program যা অন্য মানুষ (এমনকি ভবিষ্যতের আপনি নিজেও) নিরাপদে, ভুল না করে, বারবার ব্যবহার করতে পারে। পার্থক্যটি code-এর দৈর্ঘ্যে নয় — ব্যবহারকারীর প্রতি যত্নে। একটি ভালো tool ভুল input-এ ভুল কাজ করে না; সে থামে, বুঝিয়ে বলে, আর সাহায্য দেখায়।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- একটি automation tool-এর প্রয়োজন (requirements) লিখতে — কে ব্যবহার করবে, কী করবে, কী করবে না।
- Python-এর `argparse` দিয়ে পেশাদার command-line interface (CLI) বানাতে।
- Configuration-কে code থেকে আলাদা করতে ও তা যাচাই (validate) করতে।
- একটি modular project গঠন করতে — প্রতিটি ফাইলের একটি স্পষ্ট দায়িত্ব।
- Dry-run mode বানাতে, যাতে tool কী করবে তা আগে দেখা যায়, না করেই।
- Idempotent operation ডিজাইন করতে — একই কাজ দুবার চালালে একই ফলাফল।
- একটি tool-এর README ও ব্যবহারের নথি লিখতে।
- একটি tool-কে virtual environment-এ packaging-এর উপযোগী করে সাজাতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 14–19: Python function, module, exception, logging, JSON/YAML, environment variable।
- Chapter 10: Bash function ও modular গঠন।
- Chapter 20–21: cron ও SSH — কারণ tool-টি সেগুলো ব্যবহার করবে।
- Git-এ একটি project রাখার অভিজ্ঞতা (Chapter 03–06)।

২২.১ WHY: Script কখন Tool হওয়া উচিত

একটি script যখন শুধু আপনি একবার চালান, তখন তার সরলতাই যথেষ্ট। কিন্তু যখন তিনটি লক্ষণ দেখা যায়, তখন তাকে tool-এ রূপান্তর করা উচিত। প্রথম, অন্য কেউ তা ব্যবহার করবে। দ্বিতীয়, তা নিয়মিত চলবে — cron-এ বা অন্যের হাতে। তৃতীয়, ভুল ব্যবহারে ক্ষতি হতে পারে — ফাইল মোছা, সার্ভার restart, backup বদলানো। এই তিনটি ক্ষেত্রেই tool-এর বৈশিষ্ট্যগুলো দরকার: স্পষ্ট সাহায্য, input যাচাই, dry-run, log, আর নিরাপদ ব্যর্থতা। একটি ভালো tool-এর পরীক্ষা সহজ: একজন নতুন মানুষকে শুধু `--help` দেখিয়ে তা চালাতে দিন — সে যদি ভুল না করে কাজটি করতে পারে, তাহলে tool-টি তৈরি।

২২.২ Requirements: বানানোর আগে লেখা

Code লেখার আগে requirements লেখা হয় — একটি ছোট তালিকা, যাতে লেখা থাকে tool-টি কী করবে, কী করবে না, কে ব্যবহার করবে, আর সফলতা কীভাবে মাপা হবে। উদাহরণ: TechNova Health Tool-এর requirements: (১) একটি YAML config থেকে সার্ভারের তালিকা পড়বে; (২) প্রতিটি সার্ভারের disk, memory ও service পরীক্ষা করবে; (৩) ফলাফল JSON রিপোর্টে লিখবে; (৪) `--dry-run` দিলে কিছু না করে কী করবে তা দেখাবে; (৫) কোনো সার্ভার ব্যর্থ হলে থেমে যাবে না, বাকিগুলো পরীক্ষা করবে; (৬) সব ঘটনা structured log-এ লিখবে। লক্ষ্য করুন: requirements-এ কোনো code নেই — আছে সিদ্ধান্ত। Code লেখার সময় প্রতিটি সিদ্ধান্ত নিয়ে আবার ভাবতে না হলে কাজ দ্রুত ও সামঞ্জস্যপূর্ণ হয়।

২২.৩ Command-Line Interface ও argparse

একটি CLI-র তিনটি অংশ: **positional argument** (যেমন ফাইলের নাম), **optional flag** (যেমন `--dry-run`), আর **help**। Python-এর `argparse` module এই তিনটি বানিয়ে দেয়, আর ভুল argument দিলে নিজে সুন্দর error দেখায়। হাতে `sys.argv` পড়ে নিজে argument বিশ্লেষণ করা যায়, কিন্তু তাতে সাহায্য পর্দা, ধরন যাচাই ও error বার্তা — সব নিজে লিখতে হয়, আর ভুল হওয়ার সুযোগ বাড়ে। পেশাদার tool-এ `argparse` ব্যবহার করাই মান।

```
import argparse
parser = argparse.ArgumentParser(description="TechNova server health checker")
parser.add_argument("--config", required=True, help="path to YAML config file")
parser.add_argument("--output", default="report.json", help="path to JSON report")
parser.add_argument("--dry-run", action="store_true", help="show what would be done")
args = parser.parse_args()
```

লাইন ধরে ব্যাখ্যা: প্রথমে একটি parser বানানো হয়, যার বিবরণ `--help`-এ দেখা যাবে। `--config` বাধ্যতামূলক (required), কারণ config ছাড়া tool জানে না কোন সার্ভার পরীক্ষা করবে। `--output`-এর একটি default মান আছে, তাই না দিলেও চলে। `--dry-run` একটি flag — দিলে সত্য হয়, না দিলে মিথ্যা; তার কোনো মান লাগে না। শেষ লাইনটি command line পড়ে `args`-এ ভরে দেয়; ভুল argument দিলে argparse নিজে থেমে সুন্দর বার্তা দেখায়।

CLI Tool Architecture — ভেতরের স্তরগুলো

ব্যবহারকারী → CLI স্তর (argparse: argument পড়া ও যাচাই) → Config স্তর (YAML/env পড়া ও validate করা) → Core স্তর (health check, backup, report — পুনঃব্যবহারযোগ্য function) → Output স্তর (JSON রিপোর্ট, structured log)

নিয়ম: CLI স্তর কখনো সরাসরি সার্ভারে হাত দেয় না; Core স্তর কখনো command line পড়ে না। এতে প্রতিটি স্তর আলাদাভাবে পরীক্ষা করা যায়।

স্তর	দায়িত্ব	উদাহরণ
CLI	Argument পড়া, সাহায্য দেখানো, ভুল ধরা	`argparse`
Config	YAML/env পড়া ও যাচাই করা	`load_config()`
Core	আসল কাজ — check, backup, report	`check_disk()`
Output	রিপোর্ট লেখা, log করা	JSON লেখা, `logging`

২২.৪ Configuration Management ও Validation

Config-এ থাকে এমন তথ্য যা পরিবেশভেদে বদলায় — সার্ভারের নাম, সীমা (threshold), ফাইলের পথ। আর code-এ থাকে এমন যুক্তি যা বদলায় না। দুটো মিশে গেলে প্রতিটি সার্ভারের জন্য আলাদা code লিখতে হয়। সঠিক নিয়ম: config YAML ফাইলে, code Python ফাইলে। তবে config পড়লেই হবে না — তা যাচাই করতে হবে। কোনো সার্ভারের নাম নেই, threshold সংখ্যা নয়, ফাইলের পথ নেই — এমন config নিয়ে tool এগিয়ে গেলে ভুল জায়গায় কাজ করবে। Validation-এর নিয়ম: tool শুরুতেই config পড়ে, প্রতিটি প্রয়োজনীয় ক্ষেত্র আছে কি না ও তার ধরন ঠিক কি না দেখে, আর ভুল থাকলে স্পষ্ট বার্তাসহ থেমে যায় — অর্ধেক কাজ করে নয়।

```
servers:
  - name: dhaka-web
    host: 203.0.113.10
    disk_limit: 80
  - name: ctg-db
    host: 203.0.113.20
    disk_limit: 85
```

এটি একটি উদাহরণ config। `disk_limit: 80` মানে disk ৮০% ভরলে সতর্ক করতে হবে। Tool-টি config পড়ে প্রথমে দেখবে প্রতিটি server-এর `name` ও `host` আছে কি না, আর `disk_limit` ০ থেকে ১০০-এর মধ্যে কি না। Secret (password, token) config-এ লেখা যাবে না — Chapter 18-এর নিয়ম অনুযায়ী secret environment variable বা নিরাপদ vault থেকে আসবে।

২২.৫ Modular Structure: প্রতিটি ফাইলের একটি কাজ

একটি ছোট tool-এরও গঠন থাকা উচিত, যাতে বড় হলে তা ভেঙে না পড়ে। TechNova Health Tool-এর গঠন:

```
technova-tool/
├── pyproject.toml
├── README.md
├── config.yaml
├── requirements.txt
├── src/technova/
│   ├── __init__.py
│   ├── cli.py
│   ├── config.py
│   ├── checks.py
│   └── report.py
└── tests/
    └── test_checks.py
```

ব্যাখ্যা: `cli.py` শুধু command line নিয়ে কাজ করে; `config.py` শুধু config পড়ে ও যাচাই করে; `checks.py`-তে আসল পরীক্ষার function গুলো; `report.py` শুধু রিপোর্ট লেখে। কোনো ফাইল অন্য ফাইলের কাজ করে না। ফলে `checks.py`-কে CLI ছাড়াই পরীক্ষা করা যায়, আর CLI বদলালে check-এর code ছুঁতে হয় না। `pyproject.toml` tool-টির নাম, সংস্করণ ও নির্ভরতা লেখে; `requirements.txt` কোন library দরকার তা বলে; `tests/`-এ পরীক্ষা থাকে (Chapter 23)।

২২.৬ Dry-Run Mode ও Idempotency

Dry-run মানে tool-টি কী করবে তা দেখানো, কিন্তু আসলে কিছু না করা। ফাইল মুছে ফেলার আগে, সার্ভার restart করার আগে, backup বদলানোর আগে — dry-run দিয়ে দেখা যায় পরিকল্পনাটি ঠিক আছে কি না। বানানোর নিয়ম: আসল কাজটি একটি function-এ রাখুন, আর dry-run হলে সেই function-এর বদলে শুধু একটি log লাইন লিখুন — "would delete X"। এতে dry-run ও আসল run একই সিদ্ধান্তের পথ ধরে চলে, শুধু শেষ ধাপটি আলাদা।

Idempotency মানে একই কাজ দুবার চালালে ফলাফল একই থাকে। যেমন একটি directory বানানোর কাজ: প্রথমবার directory বানায়, দ্বিতীয়বার দেখে directory আছে, তাই কিছু করে না — ফলাফল একই। বিপরীতে, একটি ফাইলে লাইন যুক্ত করার কাজ idempotent নয়, কারণ দুবার চালালে লাইনটি দুবার যুক্ত হয়। Idempotent বানানোর উপায়: কাজ করার আগে অবস্থা পরীক্ষা করা (আছে কি না, ঠিক আছে কি না), তারপর শুধু প্রয়োজনীয় পরিবর্তন করা। Automation-এ এটি অপরিহার্য, কারণ cron job বা retry-র কারণে একই tool দুবার চলতেই পারে।

কাজ	Idempotent?	কেন
`mkdir -p /opt/technova`	হ্যাঁ	থাকলে কিছু করে না
ফাইলে একই লাইন যুক্ত করা	না	প্রতিবার আরেকটি copy যুক্ত হয়
একটি নির্দিষ্ট বিষয়বস্তু দিয়ে ফাইল লেখা	হ্যাঁ	দুবার লিখলেও বিষয়বস্তু একই
Service restart করা	প্রায়	ফলাফল একই, কিন্তু প্রতিবার বিঘ্ন ঘটে

২২.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
CLI (Command-Line Interface)	command লিখে program চালানোর উপায়
argparse	Python-এর command-line argument বিশ্লেষণের module
Requirements	tool-টি কী করবে, তার লিখিত তালিকা
Modular structure	প্রতিটি অংশের আলাদা দায়িত্বসহ গঠন
Dry-run	কিছু না করে কী করা হবে তা দেখানো
Idempotent	দুবার চালালেও ফলাফল একই থাকা
Validation	input বা config সঠিক কি না, তা যাচাই করা
Packaging	tool-কে install ও বিতরণের উপযোগী করে সাজানো

Virtual environment	একটি project-এর নির্ভরতার আলাদা পরিবেশ
README	tool-টি কী, কীভাবে ব্যবহার করতে হয়, তার নথি

REAL WORLD: TechNova-র একটি cleanup tool প্রথমে dry-run ছাড়া বানানো হয়েছিল। একদিন একজন engineer ভুল পথ দিয়ে tool-টি চালালেন, আর তা একটি গুরুত্বপূর্ণ directory-র ফাইল মুছে ফেলল। Backup ছিল, তাই তথ্য ফিরে এলো, কিন্তু তিন ঘণ্টা কাজ নষ্ট হলো। পরের সংস্করণে dry-run বাধ্যতামূলক করা হলো: মোছার আগে tool দেখায় কী কী মুছবে, আর ব্যবহারকারী নিশ্চিত না হলে কিছুই হয় না। সেই থেকে ভুল পথের ঘটনা আর ঘটেনি — কারণ ভুলটি ধরা পড়ে ক্ষতি হওয়ার আগেই।

২২.৮ Code Walkthrough: একটি সম্পূর্ণ ছোট Tool

নিচের code-টি একটি সরল কিন্তু সম্পূর্ণ health tool-এর মূল অংশ। পড়ুন, তারপর ব্যাখ্যা মেলান।

```
import argparse
import json
import logging
import shutil
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
logger = logging.getLogger("technova")

def check_disk(path, limit):
    usage = shutil.disk_usage(path)
    percent = usage.used * 100 // usage.total
    return {"path": path, "percent": percent, "ok": percent < limit}

def main():
    parser = argparse.ArgumentParser(description="TechNova disk checker")
    parser.add_argument("--path", default="/")
    parser.add_argument("--limit", type=int, default=80)
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    if args.dry_run:
        logger.info("would check %s with limit %s", args.path, args.limit)
        return
    result = check_disk(args.path, args.limit)
    print(json.dumps(result))
    if not result["ok"]:
        raise SystemExit(1)

if __name__ == "__main__":
    main()
```

লাইন ধরে ব্যাখ্যা: `check_disk` function-টি শুধু disk পরীক্ষা করে ও একটি dictionary ফেরত দেয় — সে command line জানে না, log জানে না। `main` function-টি argument পড়ে: `--path` কোন disk দেখবে, `--limit` কত শতাংশে সতর্ক হবে (type=int থাকায় সংখ্যা নয় এমন input দিলে argparse নিজে ভুল ধরবে)। Dry-run হলে সে শুধু log লিখে ফিরে যায় — কোনো পরীক্ষা করে না। আসল run-এ সে ফলাফল JSON-এ ছাপে, আর সীমা পেরিয়ে গেলে exit code ১ দিয়ে শেষ হয়, যাতে cron বা অন্য tool বুঝতে পারে কাজটি সতর্কবার্তা দিয়েছে। `if __name__ == "__main__"` লাইনটির মানে: ফাইলটি সরাসরি চালালেই `main` চলবে, অন্য ফাইল থেকে import করলে চলবে না — এতে function গুলো পরীক্ষায় import করে ব্যবহার করা যায়।

> **PRO TIP:** Tool-এর প্রতিটি function-কে এমনভাবে লিখুন যেন তা একা পরীক্ষা করা যায় — input নেয়, output ফেরত দেয়, আর পর্দায় কিছু ছাপে না। পর্দায় ছাপানো ও ফাইল লেখার কাজ শুধু `main` -এ রাখুন। এতে পরীক্ষা (Chapter 23) অনেক সহজ হয়।

২২.৯ Guided Practical Exercise — Lab 22: argparse দিয়ে একটি Tool বানানো

> **LAB:** এই lab-এ আপনি উপরের disk checker-কে একটি সম্পূর্ণ tool-এ রূপান্তর করবেন।

1. একটি project directory বানান: `mkdir -p ~/lab22/technova-tool` এবং সেখানে virtual environment বানান: `python3 -m venv ~/lab22/technova-tool/.venv`।
2. `diskcheck.py` নামে ফাইল বানিয়ে উপরের code-টি লিখুন।
3. `python3 diskcheck.py --help` চালিয়ে সাহায্য পর্দাটি দেখুন।
4. `python3 diskcheck.py --path / --limit 80` চালিয়ে JSON ফলাফল দেখুন।
5. `python3 diskcheck.py --dry-run --path / --limit 80` চালিয়ে দেখুন dry-run-এ কোনো JSON আসে না, শুধু log আসে।
6. `python3 diskcheck.py --limit eighty` চালিয়ে দেখুন argparse কীভাবে ভুল ধরে।
7. একটি `README.md` লিখুন: tool-টি কী করে, কীভাবে চালাতে হয়, প্রতিটি flag-এর মানে কী।
8. Cleanup: `rm -rf ~/lab22`।

Expected results and verification (illustrative):

```
$ python3 diskcheck.py --path / --limit 80
{"path": "/", "percent": 25, "ok": true}
$ python3 diskcheck.py --dry-run --path / --limit 80
INFO would check / with limit 80
```

যাচাই: প্রথম command-এ exit code ০ হওয়া উচিত (`echo $?` দিয়ে দেখুন), আর JSON-এ percent সংখ্যাটি `df -h /` -এর সঙ্গে মিলতে হবে। Dry-run-এ কোনো JSON না আসাই সঠিক — কারণ সে কিছু করেনি।

২২.১০ Common Mistakes ও Debugging

- **ভুল:** সবকিছু একটি ফাইলে, একটি function-এ লেখা — বড় হলে বোঝা যায় না, পরীক্ষা করা যায় না। সমাধান: দায়িত্ব অনুযায়ী ভাগ করুন।
- **ভুল:** Config-এর মান code-এ লিখে রাখা (hardcoding) — প্রতিটি সার্ভারের জন্য code বদলাতে হয়। সমাধান: YAML বা environment variable।
- **ভুল:** Dry-run-এও আসল কাজ করে ফেলা — dry-run flag শুধু একটি বার্তা দেখালে হবে না, আসল function-টি যেন না চলে তা নিশ্চিত করতে হবে।
- **ভুল:** ভুল input-এ নীরবে চলতে থাকা। সমাধান: শুরুতেই validate করুন ও স্পষ্ট বার্তাসহ থামুন।
- **সমস্যা:** `--help` দেখায় না — প্রায়ই `parse_args()` ভুল জায়গায় বা `if __name__` লাইনটি নেই বলে। গঠনটি উপরের উদাহরণের সঙ্গে মিলিয়ে দেখুন।

TROUBLESHOOTING: Tool-টি আপনার মেশিনে চলে, কিন্তু অন্যের মেশিনে `ModuleNotFoundError` দেয় — কারণ নির্ভরতা (dependency) শুধু আপনার মেশিনে install করা আছে, project-এ লেখা নেই। সমাধান: `requirements.txt` বা `pyproject.toml`-এ প্রতিটি নির্ভরতা লিখুন, আর নতুন virtual environment-এ install করে পরীক্ষা করুন।

২২.১১ Security Considerations

Tool-এর সবচেয়ে বড় নিরাপত্তা ঝুঁকি হলো সে ব্যবহারকারীর দেওয়া input দিয়ে command বানায়। Chapter 16-এর নিয়ম এখানেও প্রযোজ্য: কখনো ব্যবহারকারীর input-কে shell command-এর সঙ্গে জুড়ে চালাবেন না; `subprocess.run`-এ list আকারে argument দিন, `shell=True` ব্যবহার করবেন না। দ্বিতীয়ত, tool-টি যে ফাইল লেখে বা মুছে, তার পথ যাচাই করুন — ব্যবহারকারী `--path /` দিয়ে ভুল পথ দিলে tool যেন সিস্টেমের ফাইল না মুছে। তৃতীয়ত, secret কখনো argument হিসেবে নেবেন না, কারণ command line অন্য user-রা `ps` দিয়ে দেখতে পারে — secret environment variable বা ফাইল থেকে পড়ুন।

২২.১২ Best Practices

- প্রতিটি tool-এর `--help` সুন্দর ও সম্পূর্ণ রাখুন — এটিই তার প্রথম নথি।
- Exit code-এর অর্থ স্থির রাখুন: ০ মানে সফল, ১ মানে সাধারণ ব্যর্থতা, ২ মানে ভুল ব্যবহার।
- Config validate করুন শুরুতেই, কাজের মাঝখানে নয়।
- Dry-run সব ধ্বংসাত্মক কাজে রাখুন।
- README-তে উদাহরণসহ চালানোর নিয়ম লিখুন, শুধু বিবরণ নয়।
- Tool-কে Git-এ রাখুন, সংস্করণ (version) দিন, আর পরিবর্তনের ইতিহাস লিখুন।

CHECK YOUR UNDERSTANDING: নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. `argparse`-এর প্রধান সুবিধা কী? ক) তা code-কে দ্রুত চালায় খ) সাহায্য পর্দা, ধরন যাচাই ও error বার্তা নিজে বানায় গ) তা network সংযোগ দেয় ঘ) তা ফাইল encrypt করে
2. Dry-run mode-এর কাজ কী? ক) tool-কে দ্রুত চালানো খ) কিছু না করে কী করা হবে তা দেখানো গ) log ফাইল মুছে ফেলা ঘ) config ফাইল বানানো
3. Idempotent কাজের উদাহরণ কোনটি? ক) ফাইলে একই লাইন বারবার যুক্ত করা খ) `mkdir -p` দিয়ে directory বানানো গ) প্রতিবার নতুন ফাইল বানানো ঘ) একই backup বারবার পাঠানো
4. Config কোথায় রাখা উচিত? ক) code-এর ভেতরে লিখে খ) YAML ফাইলে, code থেকে আলাদা গ) command line-এ প্রতিবার টাইপ করে ঘ) log ফাইলে
5. Secret কেন command-line argument হিসেবে নেওয়া উচিত নয়? ক) তা খুব লম্বা হয় খ) অন্য user-রা `ps` দিয়ে তা দেখতে পারে গ) argparse তা পড়তে পারে না ঘ) তা JSON-এ লেখা যায় না

২২.১৩ Independent Assignment

Assignment 22.1 (Advanced): TechNova-র জন্য একটি `cleanup-tool` ডিজাইন করুন: YAML config থেকে directory ও কতদিনের পুরনো ফাইল মুছতে হবে তা পড়বে, `--dry-run`-এ মুছবে এমন ফাইলের তালিকা দেখাবে, আসল

run-এ ফাইল মুছে log লিখবে, আর idempotent হবে (দ্বিতীয়বার চালালে মুছবার মতো ফাইল না থাকলে কিছু করবে না)। Requirements, CLI নকশা, ফাইল গঠন ও README-র রূপরেখা লিখুন — সম্পূর্ণ code নয়, নকশাটি লিখুন। সম্পূর্ণ সমাধান Answer Guide-এ আছে; আগে নিজে চেষ্টা করুন।

২২.১৪ Chapter Summary

Script থেকে tool-এ রূপান্তরের মানে হলো ব্যবহারকারীর প্রতি যত্ন: requirements লেখা, argparse দিয়ে স্পষ্ট CLI বানানো, config-কে code থেকে আলাদা করে যাচাই করা, দায়িত্ব অনুযায়ী modular গঠন, dry-run দিয়ে নিরাপদ পরীক্ষা, আর idempotent operation দিয়ে পুনরাবৃত্তি নিরাপদ করা। একটি ভালো tool-এর পরীক্ষা হলো নতুন মানুষ `--help` দেখেই তা ভুল না করে চালাতে পারে কি না। README ও versioning ছাড়া tool অসম্পূর্ণ থাকে।

Revision Checklist

- ☐ একটি tool-এর requirements লিখতে পারি।
- ☐ `argparse` দিয়ে required, optional ও flag argument বানাতে পারি।
- ☐ Config YAML-এ রেখে তা validate করতে পারি।
- ☐ Modular project গঠন ব্যাখ্যা করতে পারি।
- ☐ Dry-run ও idempotency-র পার্থক্য ও উদাহরণ দিতে পারি।
- ☐ একটি tool-এর README-তে কী থাকা উচিত, তা বলতে পারি।

PART V • DEVOPS ENGINEERING MASTERY

Professional Engineering Workflow

পেশাদার প্রকৌশল — কোডের মান, টেস্টিং, ডকুমেন্টেশন ও প্রোডাকশন অটোমেশন আর্কিটেকচার।

অধ্যায় ২৩ • Part V — Professional Engineering Workflow

Chapter 23: Code Quality, Testing and Documentation — Code-এর মান, Testing ও Documentation

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র একজন engineer একটি backup script-এ একটি ছোট পরিবর্তন করলেন — ফাইলের নামে তারিখের বিন্যাস বদলালেন। পরিবর্তনটি দেখতে নিরীহ ছিল, কিন্তু তার ফলে পুরনো backup খুঁজে পাওয়ার script-টি আর কোনো ফাইল খুঁজে পেল না, আর পরের দিনের backup নীরবে ব্যর্থ হলো। কেউ বুঝতে পারল না, কারণ কোনো পরীক্ষা (test) ছিল না যা বলবে: এই পরিবর্তনটি পুরনো আচরণ ভেঙেছে। এক সপ্তাহ পরে যখন backup দরকার হলো, তখন দেখা গেল সাত দিনের কোনো backup নেই। এই ঘটনার পর দলটি একটি নিয়ম বানাল: কোনো code, যত ছোটই হোক, পরীক্ষা ছাড়া production-এ যাবে না। এই অধ্যায়ে আমরা সেই নিয়মের হাতিয়ারগুলো শিখব — কীভাবে code পরিষ্কার রাখতে হয়, কীভাবে পরীক্ষা লিখতে হয়, আর কীভাবে এমন নথি লিখতে হয় যা অন্য মানুষ আসলেই পড়ে।

> **CONCEPT:** Testing মানে code-কে বিশ্বাস করা নয়, code-কে প্রশ্ন করা। একটি পরীক্ষা একটি দাবি: এই input দিলে এই output আসবে। পরীক্ষা চললে দাবিটি সত্য প্রমাণিত হয়; না চললে code-এ বা আমাদের ধারণায় ভুল আছে — দুটোর যেকোনো একটি শেখার সুযোগ। যে code পরীক্ষা করা যায় না, সে code-কে নিরাপদে বদলানোও যায় না।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Clean code-এর নীতিগুলো (স্পষ্ট নাম, ছোট function, একটি দায়িত্ব) ব্যাখ্যা ও প্রয়োগ করতে।
- Bash script-এ ShellCheck দিয়ে সাধারণ ভুল ধরতে।
- Python code-এ black/ruff-এর ধারণা ব্যাখ্যা করতে — formatting ও linting কী করে।
- pytest দিয়ে unit test লিখতে ও চালাতে।
- Fixture দিয়ে পরীক্ষার জন্য অস্থায়ী ফাইল ও তথ্য বানাতে।
- Mocking দিয়ে বাইরের command ও API-কে পরীক্ষা থেকে আলাদা করতে।
- Unit test ও integration test-এর পার্থক্য ব্যাখ্যা করতে।
- একটি tool-এর README ও ব্যবহারের নথি লিখতে ও তার সংস্করণ (version) দিতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 14–19: Python function, module, exception, subprocess, JSON।
- Chapter 22: modular tool গঠন, argparse, dry-run।
- Chapter 07–13: Bash script ও তার সাধারণ ভুলগুলো।
- pytest install করার জন্য virtual environment ব্যবহারের অভিজ্ঞতা।

২৩.১ WHY: পরীক্ষা ছাড়া code কেন ভঙ্গুর

Code লেখার সময় আমরা মনে করি কাজ শেষ। আসলে কাজের অর্ধেক তখনো বাকি: code-টি ভবিষ্যতে বদলানো হবে — অন্য কেউ, বা আমরা নিজেই ছয় মাস পরে। তখন প্রতিটি পরিবর্তন একটি প্রশ্ন তোলে: এই পরিবর্তনটি কি পুরনো কিছু ভেঙেছে? পরীক্ষা ছাড়া এই প্রশ্নের উত্তর দেওয়ার উপায় হলো পুরো ব্যবস্থাটি হাতে চালিয়ে দেখা — যা ধীর, বিরক্তিকর, আর ভুলপ্রবণ। পরীক্ষা থাকলে উত্তরটি এক command-এ পাওয়া যায়। তাই testing সময় নষ্ট নয়; testing হলো ভবিষ্যতের পরিবর্তনের জন্য কেনা বীমা। ছোট project-এ পরীক্ষা লেখা অতিরিক্ত মনে হতে পারে, কিন্তু automation tool-এর ক্ষেত্রে ঝুঁকিটি বেশি: একটি ভুল automation নীরবে শত শত ফাইল মুছে ফেলতে পারে বা শত শত সার্ভারে ভুল কাজ করতে পারে।

২৩.২ Clean Code: পড়ার মতো Code লেখা

Clean code-এর মূল নীতি: code প্রথমে মানুষের পড়ার জন্য লেখা হয়, কম্পিউটারের চালানোর জন্য পরে। কম্পিউটার যেকোনো নামই বুঝবে, কিন্তু মানুষ বুঝবে শুধু স্পষ্ট নাম। তিনটি নিয়ম মনে রাখুন। প্রথম, নাম থেকে কাজ বোঝা যাবে: `check_disk_usage` ভালো নাম, `do_check` বা `x` নয়। দ্বিতীয়, প্রতিটি function একটি কাজ করবে — একটি function disk পরীক্ষা করবে ও রিপোর্টও লিখবে ও ইমেইলও পাঠাবে, এমন function পরীক্ষা করা ও বোঝা কঠিন। তৃতীয়, magic number এড়িয়ে চলুন: code-এ হঠাৎ `80` লেখার বদলে `DISK_LIMIT = 80` নামে একটি ধ্রুবক (constant) লিখুন, যাতে পরে বোঝা যায় ৮০ মানে কী।

নীতি	ভালো উদাহরণ	খারাপ উদাহরণ
স্পষ্ট নাম	`backup_directory`	`d`, `tmp2`
একটি দায়িত্ব	`check_disk()` শুধু পরীক্ষা করে	একটি function সবকিছু করে
ছোট function	২০–৩০ লাইনের function	২০০ লাইনের function
ধ্রুবক ব্যবহার	`DISK_LIMIT = 80`	code-এর ভেতরে হঠাৎ `80`
ভুলের স্পষ্ট বার্তা	`config file not found: config.yaml`	`error`

২৩.৩ Bash: ShellCheck দিয়ে ভুল ধরা

Bash-এ ভুল সহজে ধরা পড়ে না, কারণ shell অনেক ভুলকে নীরবে সহ্য করে — যেমন উদ্ধৃতিচিহ্ন না দেওয়া variable, যা ফাঁকা জায়গাসহ ফাইলের নামে ভেঙে যায়। **ShellCheck** হলো একটি static analysis হাতিয়ার: সে script চালায় না, শুধু পড়ে, আর পরিচিত ভুলগুলো ধরিয়ে দেয়। Ubuntu-তে `sudo apt install shellcheck` দিয়ে install করা যায়, আর `shellcheck script.sh` দিয়ে চালানো যায়। প্রতিটি সতর্কবার্তার একটি code থাকে (যেমন SC2086), যা দিয়ে অনলাইনে তার ব্যাখ্যা পাওয়া যায়। ShellCheck-কে CI-তে (GitHub Actions-এ) যুক্ত করলে ভুল script repository-তে ঢোকার আগেই ধরা পড়ে।

```
shellcheck /opt/technova/backup.sh
shellcheck --severity=warning /opt/technova/*.sh
```

ব্যাখ্যা: প্রথম লাইনটি একটি script পরীক্ষা করে। দ্বিতীয় লাইনটি একটি directory-র সব script পরীক্ষা করে, শুধু warning ও তার চেয়ে গুরুতর সমস্যা দেখিয়ে — ছোট শৈলীর মন্তব্য বাদ দিয়ে। ShellCheck-এর প্রতিটি বার্তা পড়ুন ও বুঝুন; কিছু বার্তা উপেক্ষা করা যায়, কিন্তু কেন উপেক্ষা করছেন তা জানতে হবে।

২৩.৪ Python: Formatting ও Linting — black ও ruff ধারণা

Python-এ **formatting** মানে code-এর চেহারা একরকম করা — indent, ফাঁকা জায়গা, লাইনের দৈর্ঘ্য। **black** একটি formatter: সে code-কে একটি নির্দিষ্ট শৈলীতে সাজিয়ে দেয়, ফলে দলের সবাই একই শৈলীতে লেখে ও শৈলী নিয়ে তর্ক হয় না। **Linting** মানে code-এর ভুল ও সন্দেহজনক অংশ ধরা — ব্যবহার না হওয়া variable, ভুল import, সন্দেহজনক তুলনা। **ruff** একটি দ্রুত linter, যা অনেক নিয়ম একসঙ্গে পরীক্ষা করে। দুটোই virtual environment-এ install করে চালানো যায়, আর দুটোরই উদ্দেশ্য এক: code পড়া সহজ করা ও ছোট ভুল আগেভাগে ধরা। এই বইয়ে black/ruff-এর প্রতিটি নিয়ম মুখস্থ করা লক্ষ্য নয়; লক্ষ্য হলো বোঝা যে পেশাদার দলে formatting ও linting স্বয়ংক্রিয়ভাবে চলে, মানুষের হাতে নয়।

২৩.৫ pytest Fundamentals: প্রথম Test লেখা

pytest হলো Python-এর সবচেয়ে জনপ্রিয় testing framework। তার নিয়ম সরল: `test_` দিয়ে শুরু হওয়া ফাইলে `test_` দিয়ে শুরু হওয়া function লিখুন, ভেতরে `assert` দিয়ে দাবি করুন, আর `pytest` command দিয়ে চালান। নিচের উদাহরণটি Chapter 22-এর `check_disk` ধারণার একটি সরল সংস্করণ পরীক্ষা করে।

```
def disk_ok(percent, limit):
    return percent < limit

def test_disk_below_limit():
    assert disk_ok(50, 80) is True

def test_disk_above_limit():
    assert disk_ok(90, 80) is False

def test_disk_at_limit():
    assert disk_ok(80, 80) is False
```

লাইন ধরে ব্যাখ্যা: `disk_ok` হলো পরীক্ষা করা হচ্ছে এমন function — সীমার নিচে হলে সত্য, নইলে মিথ্যা। তিনটি test function তিনটি সীমানা পরীক্ষা করে: সীমার নিচে, সীমার উপরে, আর ঠিক সীমায়। তৃতীয় test-টি সবচেয়ে গুরুত্বপূর্ণ: অনেকে ভাবেন ৮০% মানে সতর্ক নয়, কিন্তু code-এ `<` থাকায় ৮০ ঠিক সীমায় মিথ্যা হয় — এই সীমানার আচরণটিই test ধরে রাখে, যাতে ভবিষ্যতে কেউ `<=` লিখে দিলে test ব্যর্থ হয়ে জানিয়ে দেয়। `pytest` চালালে সে তিনটি test-ই খুঁজে বের করে চালায় ও ফলাফল দেখায়।

Testing Pyramid — কী পরীক্ষা, কতটা

নিচে, সবচেয়ে বেশি: Unit test — একটি function একা পরীক্ষা করা (দ্রুত, সস্তা)

মাঝখানে: Integration test — কয়েকটি অংশ একসঙ্গে পরীক্ষা করা (যেমন config পড়া + check চালানো)

উপরে, সবচেয়ে কম: End-to-end test — পুরো tool আসল সার্ভারে চালানো (ধীর, দামি)

নিয়ম: ভিত্তি মজবুত না হলে উপরের পরীক্ষা অর্থহীন — প্রথমে অনেক unit test, তারপর অল্প integration test।

২৩.৬ Fixtures ও Mocking: বাইরের জগতকে পরীক্ষা থেকে আলাদা করা

একটি test-এর সমস্যা হলো সে বাইরের জগতের উপর নির্ভর করতে পারে: আসল disk, আসল network, আসল API। তখন test ধীর হয়, আর বাইরের কারণে ব্যর্থ হয়। সমাধান দুটি। **Fixture** হলো pytest-এর ব্যবস্থা, যা প্রতিটি test-এর জন্য অস্থায়ী জিনিস বানিয়ে দেয় — যেমন একটি অস্থায়ী directory (`tmp_path`), যা test শেষে নিজে মুছে যায়। **Mocking**

হলো বাইরের জিনিসের নকল বানানো: আসল API-র বদলে একটি নকল object, যা আগে থেকে ঠিক করা উত্তর দেয়। এতে test দ্রুত চলে ও network ছাড়াই চলে।

```
import pytest
from unittest.mock import patch

def fetch_status(get_data):
    return get_data()["status"]

def test_fetch_status_with_mock():
    fake = lambda: {"status": "ok"}
    assert fetch_status(fake) == "ok"

def test_write_report(tmp_path):
    report = tmp_path / "report.json"
    report.write_text("{}")
    assert report.exists()
```

ব্যাখ্যা: প্রথম test-এ আসল API নেই — একটি ছোট lambda function নকল হিসেবে `{"status": "ok"}` ফেরত দেয়, আর test দেখে `fetch_status` তা ঠিকভাবে পড়ে কি না। দ্বিতীয় test-এ `tmp_path` একটি fixture — pytest প্রতিটি test-এর জন্য একটি নতুন অস্থায়ী directory বানিয়ে দেয়, আর test শেষে তা মুছে ফেলে। এতে test-এর ফাইল আপনার আসল directory নোংরা করে না। `unittest.mock.patch` দিয়ে আসল function-কে সাময়িকভাবে নকল দিয়ে বদলানো যায় — যেমন `subprocess.run`-কে বদলে দেওয়া, যাতে test আসল command না চালিয়ে তার আচরণ পরীক্ষা করতে পারে।

২৩.৭ Integration Testing ও Documentation

Unit test একটি function পরীক্ষা করে; **integration test** কয়েকটি অংশ একসঙ্গে পরীক্ষা করে — যেমন config ফাইল পড়া, check চালানো, আর রিপোর্ট লেখা, এই পুরো পথটি। Integration test ধীর হয় ও আসল ফাইল ব্যবহার করে, তাই তার সংখ্যা কম রাখা হয়। দুটোর ভারসাম্য: unit test দিয়ে যুক্তি (logic) ধরা, integration test দিয়ে অংশগুলোর সংযোগ ধরা।

Documentation-এর ক্ষেত্রে নিয়ম একটাই: README-তে লিখুন কী, কেন, আর কীভাবে — এই তিনটি প্রশ্নের উত্তর। কী: tool-টি কী করে। কেন: কোন সমস্যার সমাধান করে। কীভাবে: install, config, চালানো, উদাহরণ, আর ভুল হলে কী করতে হবে। একটি ভালো README-র পরীক্ষা: নতুন একজন মানুষ শুধু README পড়ে tool-টি চালাতে পারে কি না। Versioning-এর নিয়ম: Semantic Versioning অনুযায়ী সংস্করণ দিন — `1.2.3`-এ প্রথম সংখ্যা বড় পরিবর্তন, দ্বিতীয়টি নতুন বৈশিষ্ট্য, তৃতীয়টি ছোট সংশোধন।

২৩.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Unit test	একটি function বা অংশ একা পরীক্ষা করা
Integration test	কয়েকটি অংশ একসঙ্গে পরীক্ষা করা
pytest	Python-এর জনপ্রিয় testing framework
Assertion	পরীক্ষায় একটি দাবি — এটি সত্য হতে হবে

Fixture	পরীক্ষার জন্য অস্থায়ী জিনিস বানিয়ে দেওয়ার ব্যবস্থা
Mocking	বাইরের জিনিসের নকল বানিয়ে পরীক্ষা করা
ShellCheck	Bash script-এর ভুল ধরার static analysis হাতিয়ার
Linter / Formatter	code-এর ভুল ধরার হাতিয়ার / code-এর চেহারা সাজানোর হাতিয়ার
Semantic Versioning	সংস্করণ সংখ্যার অর্থপূর্ণ নিয়ম (major.minor.patch)
README	project-টি কী ও কীভাবে ব্যবহার করতে হয়, তার নথি

REAL WORLD: TechNova-র একটি দল প্রথমে test লিখতে অনিচ্ছুক ছিল — মনে হতো সময় নষ্ট। তারপর একটি ছোট নিয়ম বানানো হলো: প্রতিটি bug ঠিক করার আগে প্রথমে একটি test লেখা হবে, যা bug-টি ধরে। কয়েক মাসে test-এর সংখ্যা বাড়ল, আর আশ্চর্য ব্যাপার ঘটল: নতুন bug কমতে শুরু করল, কারণ প্রতিটি bug-এর test ভবিষ্যতে সেই ভুল আবার ঘটতে দেয় না। Test-এর সংগ্রহটি হয়ে উঠল দলের স্মৃতি — কোন ভুল আগে হয়েছে, তার প্রমাণ।

২৩.৯ Guided Practical Exercise — Lab 23: pytest দিয়ে প্রথম Test লেখা

LAB: এই lab-এ আপনি একটি ছোট function-এর জন্য test লিখবেন ও চালাবেন।

1. একটি project বানান: `mkdir -p ~/lab23/tests` এবং virtual environment বানান: `python3 -m venv ~/lab23/.venv`।
2. Virtual environment চালু করুন: `source ~/lab23/.venv/bin/activate` এবং pytest install করুন: `pip install pytest`।
3. `~/lab23/checks.py` ফাইলে একটি function লিখুন, যা disk percent ও limit নিয়ে সীমার নিচে কি না ফেরত দেয়।
4. `~/lab23/tests/test_checks.py` ফাইলে তিনটি test লিখুন: সীমার নিচে, উপরে, আর ঠিক সীমায়।
5. `pytest -v` চালিয়ে দেখুন তিনটি test-ই পাস করে কি না।
6. ইচ্ছা করে function-টি ভুল করুন (যেমন `<`-এর বদলে `<=` লিখুন), আবার `pytest -v` চালিয়ে দেখুন কোন test ব্যর্থ হয়।
7. ভুলটি ঠিক করে আবার চালান — সবুজ (পাস) দেখুন।
8. Cleanup: `deactivate` দিয়ে virtual environment বন্ধ করুন, তারপর `rm -rf ~/lab23`।

Expected results and verification (illustrative):

```
$ pytest -v
tests/test_checks.py::test_disk_below_limit PASSED
tests/test_checks.py::test_disk_above_limit PASSED
tests/test_checks.py::test_disk_at_limit PASSED
3 passed in 0.05s
```

যাচাই: তিনটি test পাস হলে function-টি ঠিক আছে। ভুল করার পর ঠিক সীমার test-টি ব্যর্থ হওয়া উচিত — এটিই প্রমাণ করে test-টি আসলেই ভুল ধরতে পারে। একটি test যা কখনো ব্যর্থ হয় না, তা কোনো কাজের নয়।

২৩.১০ Common Mistakes ও Debugging

- **ভুল:** শুধু সফল পথটি পরীক্ষা করা — ব্যর্থ পথটি পরীক্ষা না করা। সমাধান: ভুল input, খালি ফাইল, অনুপস্থিত config — প্রতিটির test লিখুন।
- **ভুল:** Test-এ আসল API বা আসল সার্ভার ব্যবহার করা — test ধীর ও অনির্ভরযোগ্য হয়। সমাধান: mocking ব্যবহার করুন।
- **ভুল:** Test-এর নাম থেকে বোঝা যায় না কী পরীক্ষা হচ্ছে। সমাধান: নামে পরিস্থিতি ও প্রত্যাশা লিখুন, যেমন `test_disk_at_limit_returns_false`।
- **সমস্যা:** `pytest` কোনো test খুঁজে পায় না — ফাইলের নাম `test_` দিয়ে শুরু হয়েছে কি না ও function-এর নাম `test_` দিয়ে শুরু হয়েছে কি না দেখুন।
- **সমস্যা:** Test একা চললে পাস করে, একসঙ্গে চললে ব্যর্থ হয় — প্রায়ই test-গুলো একই ফাইল বা অবস্থা ভাগ করে নিচ্ছে। সমাধান: প্রতিটি test-এ fixture দিয়ে আলাদা অস্থায়ী জিনিস ব্যবহার করুন।

TROUBLESHOOTING: ShellCheck একটি বার্তা দেখাচ্ছে, কিন্তু আপনি নিশ্চিত code-টি ঠিক আছে — বার্তার code (যেমন SC2086) দিয়ে তার ব্যাখ্যা পড়ুন। অনেক বার্তাই শৈলীর পরামর্শ, ভুল নয়। কিন্তু কোনো বার্তা উপেক্ষা করার আগে বুঝুন সে কী বলছে; অন্ধভাবে উপেক্ষা করলে আসল ভুলও উপেক্ষা করার অভ্যাস হয়ে যায়।

২৩.১১ Security Considerations

Test-এ কখনো আসল secret, আসল password বা আসল API token ব্যবহার করবেন না — test ফাইল Git-এ যায়, আর সেখানে secret থাকলে তা ইতিহাসে থেকে যায়। Test-এর জন্য নকল মান ব্যবহার করুন, যেমন `test-token-123`। দ্বিতীয়ত, test যেন আসল সিস্টেমে ধ্বংসাত্মক কাজ না করে — ফাইল মোছার test সবসময় `tmp_path`-এর ভেতরে চালান, আসল directory-তে নয়। তৃতীয়ত, linting ও ShellCheck-কে নিরাপত্তার অংশ ভাবুন: তারা command injection-এর মতো ঝুঁকিপূর্ণ pattern ধরিয়ে দিতে পারে।

২৩.১২ Best Practices

- প্রতিটি bug ঠিক করার আগে প্রথমে একটি test লিখুন, যা bug-টি ধরে।
- Test-এর নাম থেকে পরিস্থিতি ও প্রত্যাশা বোঝা যাবে।
- Code বদলালে test চালান — `pytest` এক command-এ চালানো যায়।
- ShellCheck ও formatter-কে নিয়মিত চালান, শুধু মনে পড়লে নয়।
- README-তে install, ব্যবহার, উদাহরণ ও troubleshooting — চারটিই লিখুন।
- সংস্করণ দিন ও পরিবর্তনের তালিকা (changelog) রাখুন।

CHECK YOUR UNDERSTANDING: নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Unit test ও integration test-এর পার্থক্য কী? ক) কোনো পার্থক্য নেই খ) unit test একটি অংশ একা পরীক্ষা করে, integration test কয়েকটি অংশ একসঙ্গে গ) unit test ধীর, integration test দ্রুত ঘ) unit test শুধু Bash-এ চলে
2. Fixture-এর কাজ কী? ক) code-কে দ্রুত চালানো খ) প্রতিটি test-এর জন্য অস্থায়ী জিনিস বানিয়ে দেওয়া গ) ফাইল encrypt করা ঘ) network সংযোগ দেওয়া

3. Mocking কেন ব্যবহার করা হয়? ক) আসল API-কে নকল দিয়ে বদলে test দ্রুত ও নির্ভরযোগ্য করা খ) code-কে সুন্দর দেখানো গ) ফাইল মুছে ফেলা ঘ) password লুকানো
4. ShellCheck কী করে? ক) script চালায় খ) script না চালিয়ে পড়ে ভুল ধরিয়ে দেয় গ) script-কে Python-এ বদলায় ঘ) server restart করে
5. Semantic Versioning-এ `2.0.0` মানে কী? ক) ছোট সংশোধন খ) নতুন বৈশিষ্ট্য গ) বড় পরিবর্তন, যা পুরনো ব্যবহার ভাঙতে পারে ঘ) প্রথম সংস্করণ

২৩.১৩ Independent Assignment

Assignment 23.1 (Intermediate): Chapter 22-এর disk checker tool-এর জন্য পাঁচটি test লিখুন: সীমার নিচে, উপরে, ঠিক সীমায়, ভুল config (limit ০–১০০-এর বাইরে), আর অনুপস্থিত config ফাইল। প্রতিটি test-এর নাম, কী পরীক্ষা করছে, আর কী প্রত্যাশা করছে — তা লিখুন। সঙ্গে একটি README-র রূপরেখা লিখুন। সম্পূর্ণ সমাধান Answer Guide-এ আছে; আগে নিজে চেষ্টা করুন।

২৩.১৪ Chapter Summary

পরীক্ষা ছাড়া code ভঙ্গুর, কারণ ভবিষ্যতের পরিবর্তন পুরনো কিছু ভেঙেছে কি না, তা জানার উপায় থাকে না। Clean code — স্পষ্ট নাম, ছোট function, একটি দায়িত্ব — পড়া ও পরীক্ষা করা সহজ করে। ShellCheck Bash-এর ভুল ধরে, black ও ruff Python-এর শৈলী ও ভুল দেখে। pytest দিয়ে unit test লেখা যায়, fixture দিয়ে অস্থায়ী জিনিস বানানো যায়, আর mocking দিয়ে বাইরের জগতকে পরীক্ষা থেকে আলাদা করা যায়। Integration test অংশগুলোর সংযোগ ধরে, আর README ও Semantic Versioning tool-কে অন্য মানুষের ব্যবহারের উপযোগী করে।

Revision Checklist

- ☐ Clean code-এর তিনটি নীতি বলতে পারি।
- ☐ ShellCheck দিয়ে Bash script পরীক্ষা করতে পারি।
- ☐ black ও ruff-এর পার্থক্য (formatting বনাম linting) বলতে পারি।
- ☐ pytest দিয়ে তিনটি সীমানার test লিখতে পারি।
- ☐ Fixture ও mocking-এর ব্যবহার ব্যাখ্যা করতে পারি।
- ☐ একটি README-তে কী কী থাকা উচিত, তা বলতে পারি।

অধ্যায় ২৪ • Part V — Professional Engineering Workflow

Chapter 24: Automation Architecture and Production Practices — Production Automation-এর স্থাপত্য ও বাস্তব অনুশীলন

শুরুতে: একটি বাস্তব দৃশ্য

TechNova-র চারটি সার্ভার এখন আটটি হয়েছে। প্রতিদিন backup, health check, log সংগ্রহ, রিপোর্ট, cleanup — সবকিছু চলছে, কিন্তু প্রতিটি কাজ আলাদা script, আলাদা schedule, আলাদা log ফাইল। একদিন সকালে ঢাকার একটি সার্ভারে disk ভরে গেল, কিন্তু কেউ জানতে পারল না — কারণ health check-এর log-টি কেউ দেখেনি, backup-টি ব্যর্থ হয়েছিল কিন্তু তার error অন্য একটি ফাইলে ছিল, আর রিপোর্টটি তৈরি হয়েছিল পুরনো তথ্য দিয়ে। প্রতিটি অংশ আলাদাভাবে ঠিক ছিল, কিন্তু পুরো ব্যবস্থাটি একসঙ্গে কাজ করছিল না। সেদিন দলটি বুঝল: automation-এর আসল চ্যালেঞ্জ একটি script লেখা নয় — একটি ব্যবস্থা (system) বানানো, যেখানে প্রতিটি অংশ জানে অন্য অংশ কী করছে, ব্যর্থতা সবাই দেখতে পায়, আর যেকোনো অংশ বদলানো যায় পুরো ব্যবস্থা না ভেঙে। এই শেষ অধ্যায়ে আমরা সেই ব্যবস্থার নকশা শিখব — বইয়ের সব অধ্যায়কে একসঙ্গে বেঁধে।

> **CONCEPT:** Automation architecture মানে অনেকগুলো ছোট, নির্ভরযোগ্য অংশকে এমনভাবে সাজানো যাতে পুরো ব্যবস্থাটি মানুষের চেয়ে বেশি নির্ভরযোগ্য হয়। একটি ভালো ব্যবস্থার তিনটি গুণ: ব্যর্থতা দেখা যায় (observable), ব্যর্থতা থেকে ফেরা যায় (recoverable), আর অংশ বদলানো যায় (maintainable)। এই তিনটির যেকোনো একটি না থাকলে automation শুধু দ্রুত ভুল করার যন্ত্র হয়ে ওঠে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Bash ও Python-এর মধ্যে কখন কোনটি বেছে নেবেন, তা সিদ্ধান্তের ছক দিয়ে ব্যাখ্যা করতে।
- Automation-এর নকশার নীতিগুলো (idempotency, observability, নিরাপদ ব্যর্থতা) ব্যাখ্যা করতে।
- একটি production automation ব্যবস্থার স্তরগুলো আঁকতে ও প্রতিটি স্তরের দায়িত্ব বলতে।
- Secrets ব্যবস্থাপনার সঠিক নীতি ব্যাখ্যা করতে ও ভুল পদ্ধতিগুলো চিনতে।
- একটি automation tool-কে production-এ নেওয়ার ধাপগুলো (পরীক্ষা, নথি, schedule, পর্যবেক্ষণ) তালিকা করতে।
- TechNova-র একটি সম্পূর্ণ end-to-end case study বিশ্লেষণ করতে ও তার দুর্বলতা চিহ্নিত করতে।
- নিজের একটি ছোট production automation ব্যবস্থার নকশা লিখতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 01–06: Git, GitHub, collaboration workflow।
- Chapter 07–13: Bash scripting, error handling, debugging।
- Chapter 14–19: Python automation, API, YAML, logging, exception।
- Chapter 20–23: cron, SSH, reusable tool, testing, documentation।

২৪.১ WHY: একটি Script যথেষ্ট নয় কেন

একটি script একটি কাজ করে। একটি ব্যবস্থা অনেক কাজকে একসঙ্গে করে, আর তাদের মধ্যে সম্পর্ক পরিচালনা করে। পার্থক্যটি বোঝার জন্য একটি উপমা: একটি script হলো একটি রাঁধুনি, যে একটি পদ রান্না করতে পারে। একটি ব্যবস্থা হলো একটি রান্নাঘর — যেখানে কে কী রান্না করবে, উপকরণ কোথায়, আগুন নিভে গেলে কী হবে, আর খাবার তৈরি হলে কে জানবে — সবকিছুর নিয়ম আছে। Production-এ প্রশ্নগুলো বদলে যায়: script-টি কে চালাবে? কখন চালাবে? ব্যর্থ হলে কে জানবে? ব্যর্থতা থেকে কীভাবে ফিরবে? কেউ script-টি বদলালে কীভাবে জানা যাবে আগের আচরণ ঠিক আছে? এই প্রশ্নগুলোর উত্তরই architecture।

২৪.২ Bash নাকি Python: সিদ্ধান্তের ছক

নতুনদের একটি সাধারণ ভুল: সবকিছু একটি ভাষায় লেখার চেষ্টা করা। সঠিক প্রশ্নটি হলো: কাজটি কী ধরনের? Bash ভালো যখন কাজটি মূলত Linux command-গুলোকে একসঙ্গে সাজানো — ফাইল সরানো, service restart করা, log খোঁজা। Python ভালো যখন কাজটি তথ্য নিয়ে জটিল যুক্তি — JSON বিশ্লেষণ, API-তে কথা বলা, অনেক সার্ভারের ফলাফল একত্র করা। দুটো মেশানোও যায়: Bash script-এর ভেতর থেকে Python tool চালানো, বা Python থেকে subprocess দিয়ে Bash command চালানো — তবে সীমা পরিষ্কার রাখতে হবে, নইলে কোন অংশ কী করে তা বোঝা যায় না।

বিষয়	Bash বেছে নিন যখন	Python বেছে নিন যখন
কাজের ধরন	Linux command সাজানো, ফাইল ও service পরিচালনা	তথ্য বিশ্লেষণ, API, জটিল যুক্তি
তথ্যের গঠন	সরল পাঠ্য, লাইন ধরে লাইন	JSON, YAML, nested তথ্য
ভুল ধরা	ছোট script, সরল পথ	বড় যুক্তি, অনেক শাখা
পরীক্ষা	হাতে পরীক্ষা যথেষ্ট	pytest দিয়ে নিয়মিত পরীক্ষা দরকার
উদাহরণ	দৈনিক backup, log cleanup	Health রিপোর্ট, API সংগ্রহ, বহু সার্ভারের সমন্বয়

PRO TIP: সিদ্ধান্ত নিতে না পারলে একটি নিয়ম মনে রাখুন: কাজটি যদি ৫০ লাইনের বেশি Bash হয়, বা তাতে JSON বিশ্লেষণ, API call, বা জটিল ভুল-পরিচালনা থাকে, তাহলে Python-এ লিখুন। Bash-এর শক্তি তার সরলতায়; সরলতা শেষ হলে তার দুর্বলতা শুরু হয়।

২৪.৩ Automation Design Principles: পাঁচটি নীতি

প্রথম নীতি **idempotency** — একই কাজ দুবার চালালে ফলাফল একই থাকে (Chapter 22)। দ্বিতীয় নীতি **observability** — ব্যবস্থাটি কী করছে, তা বাইরে থেকে দেখা যায়: structured log, স্পষ্ট exit code, আর নিয়মিত রিপোর্ট। তৃতীয় নীতি **নিরাপদ ব্যর্থতা** — ব্যর্থতা ঘটলে ব্যবস্থা আংশিক কাজ করে থেমে যাবে না, বরং স্পষ্ট বার্তা দিয়ে থামবে ও আগের নিরাপদ অবস্থায় ফিরবে। চতুর্থ নীতি **সর্বনিম্ন অধিকার** — প্রতিটি job শুধু ততটুকু permission পাবে, যতটুকু তার দরকার। পঞ্চম নীতি **configuration থেকে code আলাদা** — পরিবেশ বদলালে code বদলাতে হবে না, শুধু config বদলাবে। এই পাঁচটি নীতি একসঙ্গে একটি ব্যবস্থাকে নির্ভরযোগ্য করে; যেকোনো একটি বাদ গেলে একটি দুর্বলতা থেকে যায়।

২৪.৪ Production Automation Architecture: স্তরগুলো

একটি production ব্যবস্থাকে পাঁচটি স্তরে ভাবুন। **Source স্তর:** সব code Git-এ, প্রতিটি পরিবর্তনের ইতিহাসসহ, আর main branch সুরক্ষিত (Chapter 05–06)। **Configuration স্তর:** YAML ও environment variable, code থেকে আলাদা, আর

secret আলাদা নিরাপদ জায়গায় (Chapter 18)। **Execution স্তর:** cron বা systemd timer সময়মতো tool চালায়, flock দিয়ে duplicate আটকায় (Chapter 20)। **Remote স্তর:** SSH দিয়ে দূরের সার্ভারে কাজ হয়, key দিয়ে login, host verification চালু (Chapter 21)। **Observability স্তর:** structured log, JSON রিপোর্ট, আর ব্যর্থতার সতর্কবার্তা — যাতে মানুষ দেখতে পায় কী ঘটেছে (Chapter 19)। কোনো স্তর বাদ গেলে ব্যবস্থাটি আংশিক হয়: source ছাড়া পরিবর্তনের ইতিহাস নেই, observability ছাড়া ব্যর্থতা অদৃশ্য।

Production Automation Architecture — TechNova end-to-end

GitHub (source, review, version) → সার্ভারে deploy (Git pull, নির্দিষ্ট tag) → cron / systemd timer (সময়মতো চালানো, flock)

↓

Python CLI tool (argparse, config YAML, dry-run) → SSH দিয়ে সার্ভারগুলোতে check → API থেকে তথ্য সংগ্রহ

↓

Structured log (JSON) + JSON রিপোর্ট + exit code → ব্যর্থ হলে সতর্কবার্তা → মানুষ log দেখে সিদ্ধান্ত নেয়

প্রতিটি তীরের মানে একটি চুক্তি: উপরের স্তর নিচের স্তরের ভেতরের কাজ জানে না, শুধু তার চুক্তি (interface) জানে।

২৪.৫ Error Recovery, Scheduling ও Secrets

ব্যর্থতা থেকে ফেরার তিনটি উপায়। **Retry:** সাময়িক ব্যর্থতায় (network ছিন্ন) কিছুক্ষণ অপেক্ষা করে আবার চেষ্টা করা — তবে সীমা থাকতে হবে, নইলে অসীম চেষ্টা চলবে। **Rollback:** পরিবর্তন ব্যর্থ হলে আগের অবস্থায় ফেরা — যেমন নতুন config কাজ না করলে পুরনো config ফিরিয়ে আনা। **Alert ও মানুষের হস্তক্ষেপ:** কিছু ব্যর্থতা স্বয়ংক্রিয়ভাবে ঠিক করা যায় না — তখন স্পষ্ট বার্তাসহ মানুষকে জানানোই সঠিক কাজ। Scheduling-এর নিয়ম: ভারী job রাতের কম ব্যস্ত সময়ে, একই মিনিটে অনেক ভারী job নয়, আর প্রতিটি job-এর log আলাদা ফাইলে। Secrets-এর নিয়ম: secret কখনো code-এ নয়, কখনো Git-এ নয়, কখনো command line-এ নয় — শুধু environment variable বা নিরাপদ vault থেকে, আর প্রতিটি secret-এর মেয়াদ ও বাতিলের উপায় থাকতে হবে।

ব্যর্থতার ধরন	সঠিক প্রতিক্রিয়া	ভুল প্রতিক্রিয়া
সাময়িক network ছিন্ন	সীমিত retry, অপেক্ষাসহ	অসীম retry, সার্ভারকে চাপ দেওয়া
ভুল config	শুরুতেই validate করে থামা	অর্ধেক কাজ করে থামা
Disk পূর্ণ	সতর্কবার্তা দিয়ে থামা, cleanup-এর পরামর্শ	নীরবে ফাইল লেখা বাদ দেওয়া
অজানা ভুল	Log লিখে exit code-সহ থামা	ভুল লুকিয়ে সফল দেখানো

২৪.৬ TechNova Case Study: সবকিছু একসঙ্গে

TechNova-র চূড়ান্ত ব্যবস্থাটি এইরকম। ঢাকার মূল সার্ভারে একটি Git repository আছে, যেখানে Bash backup script, Python health tool, আর তাদের test ও README থাকে। প্রতিটি পরিবর্তন একটি pull request দিয়ে যায়, আর main branch সুরক্ষিত — সরাসরি push করা যায় না। রাত ২টায় cron backup script চালায় (flock-সহ), প্রতি ১৫ মিনিটে health tool চলে, সকাল ৯টায় রিপোর্ট তৈরি হয়। Health tool config YAML থেকে সার্ভারের তালিকা পড়ে, SSH দিয়ে প্রতিটি সার্ভারে disk ও service পরীক্ষা করে, আর একটি বাইরের API থেকে সার্ভারের অবস্থা সংগ্রহ করে। ফলাফল JSON রিপোর্টে লেখা হয়, structured log-এ প্রতিটি ঘটনা থাকে, আর কোনো সার্ভার ব্যর্থ হলে exit code ১ দিয়ে শেষ হয় — যাতে

cron-এর log দেখে মানুষ বুঝতে পারে। সাপ্তাহিক রবিবার রাতে cleanup চলে, dry-run-এর পরে। এই ব্যবস্থার সবচেয়ে শক্তিশালী দিকটি হলো: কোনো একটি অংশ বদলানো যায় — যেমন health check-এর সীমা বদলানো — শুধু config বদলে, code না ছুঁয়ে। আর সবচেয়ে বড় দুর্বলতাটি এখনো আছে: সতর্কবার্তা শুধু log-এ যায়, কারও ফোনে নয় — তাই পরের ধাপ হলো একটি সতর্কবার্তার channel যোগ করা, যা এই বইয়ের পরিসরের বাইরে, কিন্তু নকশায় তার জায়গা রাখা হয়েছে।

> **REAL WORLD:** TechNova-র ব্যবস্থাটি একদিনে বানানো হয়নি। প্রথমে ছিল একটি backup script, তারপর একটি health check, তারপর একটি রিপোর্ট, তারপর সেগুলোকে একটি tool-এ বাঁধা, তারপর test, তারপর Git-এ version। প্রতিটি ধাপে ব্যবস্থাটি আগের চেয়ে একটু নির্ভরযোগ্য হয়েছে — আর কোনো ধাপে পুরো ব্যবস্থা ভেঙে ফেলা হয়নি। এটিই production-এর বাস্তব পথ: ছোট, নিরাপদ, পরীক্ষিত ধাপে এগোনো। একদিনে নিখুঁত ব্যবস্থা বানানোর চেষ্টা করলে সাধারণত কিছুই production-এ যায় না।

২৪.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা
Architecture	একটি ব্যবস্থার অংশগুলো ও তাদের সম্পর্কের নকশা
Observability	ব্যবস্থাটি কী করছে, তা বাইরে থেকে দেখতে পারা
Idempotency	দুবার চালালেও ফলাফল একই থাকা
Retry / Rollback	আবার চেষ্টা করা / আগের অবস্থায় ফেরা
Least privilege	শুধু যতটুকু অধিকার দরকার, ততটুকুই দেওয়া
Deployment	code-কে production-এ নিয়ে যাওয়া
Interface / Contract	দুটি অংশের মধ্যে যোগাযোগের নিয়ম
Alert	ব্যর্থতা বা সতর্কবার্তা মানুষকে জানানো
Changelog	প্রতিটি সংস্করণে কী বদলেছে, তার তালিকা
Production	আসল ব্যবহারকারীরা যে পরিবেশ ব্যবহার করে

২৪.৮ Code Walkthrough: একটি Production-উপযোগী Main Flow

নিচের code-টি Chapter 22-এর tool-কে production-এর নিয়মে সাজানো একটি মূল প্রবাহ (main flow) — config validate করা, dry-run, প্রতিটি সার্ভার আলাদাভাবে পরীক্ষা করা, ব্যর্থতা log করা, আর সঠিক exit code দেওয়া।

```
import argparse
import json
import logging
import sys
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
logger = logging.getLogger("technova")

def load_servers(path):
    with open(path, encoding="utf-8") as handle:
```

```
        data = json.load(handle)
    servers = data["servers"]
    if not servers:
        raise ValueError("server list is empty")
    return servers

def check_server(server):
    return {"name": server["name"], "ok": True}

def main():
    parser = argparse.ArgumentParser(description="TechNova production health tool")
    parser.add_argument("--config", required=True)
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    try:
        servers = load_servers(args.config)
    except (OSError, ValueError, KeyError) as error:
        logger.error("invalid config: %s", error)
        return 2
    if args.dry_run:
        logger.info("would check %s servers", len(servers))
        return 0
    failed = 0
    for server in servers:
        try:
            result = check_server(server)
            logger.info("checked %s ok=%s", result["name"], result["ok"])
        except Exception as error:
            logger.error("check failed for %s: %s", server.get("name"), error)
            failed += 1
    return 1 if failed else 0

if __name__ == "__main__":
    sys.exit(main())
```

লাইন ধরে ব্যাখ্যা: `load_servers` config পড়ে ও যাচাই করে — সার্ভারের তালিকা খালি হলে `ValueError` দেয়, যাতে খালি config নিয়ে tool এগিয়ে না যায়। `main`-এ config পড়ার ভুল ধরা হয় ও exit code ২ ফেরত দেওয়া হয় — ভুল ব্যবহারের code। Dry-run হলে সে শুধু কতগুলো সার্ভার পরীক্ষা করবে তা জানিয়ে শেষ হয়। আসল run-এ প্রতিটি সার্ভার আলাদা try/except-এর ভেতরে পরীক্ষা করা হয় — একটি সার্ভার ব্যর্থ হলে বাকিগুলো থেমে যায় না, শুধু `failed` গোনা হয়। শেষে কোনো ব্যর্থতা থাকলে exit code ১, না থাকলে ০ — যাতে cron ও মানুষ দুজনেই ফলাফল বুঝতে পারে। `sys.exit(main())` এই code-টিকে operating system-এ পৌঁছে দেয়।

২৪.৯ Guided Practical Exercise — Capstone Mini-Design Task

> **LAB:** এই অধ্যায়ের অনুশীলনটি একটি code লেখার কাজ নয় — একটি নকশা লেখার কাজ। এটিই এই বইয়ের capstone: বইয়ের সবকিছু একসঙ্গে ব্যবহার করে একটি ব্যবস্থা ডিজাইন করা।

1. একটি কাল্পনিক company বাছুন: TechNova-র তিনটি সার্ভার (ঢাকা web, চট্টগ্রাম database, সিলেট backup)।
2. এক পৃষ্ঠায় requirements লিখুন: প্রতিদিন কী কী স্বয়ংক্রিয় হবে — backup, health check, log সংগ্রহ, রিপোর্ট। প্রতিটির জন্য কখন চলবে, কে চালাবে, ব্যর্থ হলে কী হবে।

3. একটি স্থাপত্য চিত্র আঁকুন (কাগজে বা পাঠ্য হিসেবে): Git, config, cron, SSH, Python tool, log, রিপোর্ট — প্রতিটি স্তর ও তাদের সংযোগ।
4. সিদ্ধান্তের ছক ব্যবহার করে প্রতিটি কাজের জন্য Bash নাকি Python, তা লিখুন ও কারণ দিন।
5. প্রতিটি কাজের জন্য লিখুন: dry-run আছে কি না, idempotent কি না, log কোথায় যাবে, exit code কী হবে।
6. একটি cron schedule লিখুন: প্রতিটি job-এর সময়, flock-এর জায়গা, log ফাইলের পথ।
7. নিরাপত্তা পর্যালোচনা লিখুন: secret কোথায় থাকবে, কোন user হিসেবে প্রতিটি job চলবে, host verification কীভাবে হবে।
8. একটি ব্যর্থতার দৃশ্য লিখুন: ঢাকার সার্ভারে disk ৯৫% ভরে গেলে কী ঘটবে — কে জানবে, কীভাবে জানবে, আর কীভাবে ফিরবে।
9. আপনার নকশাটি একজন সহপাঠীকে দেখান (বা নিজে একদিন পরে পড়ুন) — সে কি শুধু নকশা পড়ে ব্যবস্থাটি বানাতে পারবে? না পারলে কোন অংশ অস্পষ্ট, তা লিখুন।
10. Cleanup: এই অনুশীলনে কোনো ফাইল তৈরি হয়নি — নকশাটি একটি Markdown ফাইলে সংরক্ষণ করুন, যাতে পরে তা বাস্তবায়ন করা যায়।

Expected results and verification (illustrative):

```
Requirements: backup daily 02:00, health every 15 min, report Mon-Fri 09:00
Choice: backup in Bash, health report in Python, log collection in Bash
Safety: dry-run for cleanup, flock for backup, least privilege per job
Failure: disk 95% -> health tool exit 1 -> log shows ERROR -> person checks log
```

যাচাই: আপনার নকশায় যদি প্রতিটি কাজের জন্য সময়, দায়িত্ব, log, exit code, dry-run ও ব্যর্থতার প্রতিক্রিয়া — এই ছয়টি থাকে, তাহলে নকশাটি সম্পূর্ণ। কোনো একটি না থাকলে সেই অংশটি production-এ গিয়ে প্রশ্ন তুলবে — আর তখন উত্তর খুঁজতে হবে চাপের মধ্যে, শান্তিতে নয়।

২৪.১০ Common Mistakes ও Debugging

- **ভুল:** সবকিছু একদিনে production-এ নেওয়া। সমাধান: ছোট ধাপে, একটি কাজ করে, পরীক্ষা করে, তারপর পরেরটি।
- **ভুল:** ব্যর্থতা লুকিয়ে সফল দেখানো — exit code ০ দিয়ে শেষ করা, যদিও ভেতরে ভুল হয়েছে। সমাধান: ব্যর্থতায় সঠিক exit code দিন।
- **ভুল:** Log-এ সবকিছু লেখা, কিন্তু কেউ log দেখে না। সমাধান: ব্যর্থতার জন্য সতর্কবার্তার ব্যবস্থা রাখুন, আর নিয়মিত log পর্যালোচনার সময় রাখুন।
- **ভুল:** একটি অংশ বদলালে পুরো ব্যবস্থা ভেঙে যাওয়া। সমাধান: স্তরগুলোর মধ্যে চুক্তি (interface) স্পষ্ট রাখুন, আর পরিবর্তনের আগে test চালান।
- **সমস্যা:** ব্যবস্থাটি কাজ করছে কি না, তা বোঝা যাচ্ছে না — observability নেই। সমাধান: প্রতিটি run-এর শুরু ও শেষ log-এ লিখুন, আর নিয়মিত রিপোর্ট দেখুন।

TROUBLESHOOTING: একটি automation ব্যবস্থা হঠাৎ নীরব হয়ে গেলে প্রথমে দেখুন cron daemon চলছে কি না (`systemctl status cron`), তারপর `grep CRON /var/log/syslog` দিয়ে job চেষ্টা হয়েছিল কি না দেখুন, তারপর job-এর নিজের log দেখুন। নীরবতা নিজেই একটি লক্ষণ — এর মানে হয় job চলছে না, নয়তো তার log কোথাও যাচ্ছে না। দুটোই ঠিক করতে হবে।

২৪.১১ Security Considerations

Production automation-এর নিরাপত্তা একটি স্তরে নয়, সব স্তরে। Source স্তরে: main branch সুরক্ষিত, প্রতিটি পরিবর্তন পর্যালোচনা (review) দিয়ে যায়। Config স্তরে: secret Git-এ নয়, শুধু নিরাপদ জায়গায়। Execution স্তরে: প্রতিটি job সর্বনিম্ন অধিকারে চলে, root শুধু যেখানে অপরিহার্য। Remote স্তরে: key login, host verification চালু, আর automation-এর জন্য আলাদা key। Observability স্তরে: log-এ secret লেখা হয় না — একটি ভুল log লাইন secret ফাঁস করে দিতে পারে, আর log ফাইল অনেক জায়গায় copy হয়। একটি নিয়ম মনে রাখুন: automation যত শক্তিশালী, তার ভুলের পরিণতিও তত বড় — তাই শক্তিশালী automation-এর নিরাপত্তা তত বেশি যত্ন দাবি করে।

২৪.১২ Best Practices

- ছোট, নিরাপদ, পরীক্ষিত ধাপে production-এ যান — একদিনে সবকিছু নয়।
- প্রতিটি job-এর log, exit code, dry-run ও ব্যর্থতার প্রতিক্রিয়া আগে থেকে লিখুন।
- Config ও secret আলাদা রাখুন; secret-এর মেয়াদ ও বাতিলের উপায় রাখুন।
- নিয়মিত log পর্যালোচনা করুন — automation নিজে নিজে চললেও মানুষের নজরদারি লাগে।
- পরিবর্তনের আগে test চালান, পরিবর্তনের পরে log দেখুন।
- নথি (README, changelog, স্থাপত্য চিত্র) code-এর সঙ্গেই রাখুন ও আপডেট করুন।

CHECK YOUR UNDERSTANDING: নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. JSON বিশ্লেষণ ও API call-এর কাজে Bash-এর বদলে Python বেছে নেওয়ার কারণ কী? ক) Python দ্রুত টাইপ করা যায় খ) Python-এ গঠনযুক্ত তথ্য ও জটিল যুক্তি পরিচালনা সহজ গ) Bash-এ JSON পড়া যায় না বলে ঘ) Python-এ cron লাগে না
2. Observability মানে কী? ক) ব্যবস্থাটি দ্রুত চলে খ) ব্যবস্থাটি কী করছে, তা বাইরে থেকে দেখা যায় গ) ব্যবস্থাটি encrypt করা থাকে ঘ) ব্যবস্থাটি নিজে নিজে ঠিক হয়
3. সাময়িক network ব্যর্থতায় সঠিক প্রতিক্রিয়া কোনটি? ক) অসীম retry করা খ) সীমিত retry, অপেক্ষাসহ, তারপর সতর্কবার্তা গ) ভুল লুকিয়ে সফল দেখানো ঘ) সঙ্গে সঙ্গে সার্ভার restart করা
4. Secret কোথায় রাখা উচিত? ক) code-এ খ) Git repository-তে গ) environment variable বা নিরাপদ vault-এ ঘ) log ফাইলে
5. একটি automation ব্যবস্থা হঠাৎ নীরব হয়ে গেলে প্রথমে কী দেখবেন? ক) disk-এর রং খ) cron daemon চলছে কি না ও তার log গ) Python-এর সংস্করণ ঘ) সার্ভারের নাম

২৪.১৩ Independent Assignment

Assignment 24.1 (Advanced): আপনার নিজের একটি ছোট production automation ব্যবস্থার নকশা লিখুন (দুই পৃষ্ঠা): দুটি সার্ভার, দৈনিক backup, প্রতি ঘণ্টায় health check, সাপ্তাহিক রিপোর্ট। প্রতিটি কাজের জন্য Bash নাকি Python, তার কারণ, cron schedule, config-এর রূপরেখা, log ও exit code-এর নিয়ম, নিরাপত্তা পর্যালোচনা, আর একটি ব্যর্থতার দৃশ্য ও তার থেকে ফেরার পরিকল্পনা লিখুন। সম্পূর্ণ সমাধান Answer Guide-এ আছে; আগে নিজে চেষ্টা করুন।

২৪.১৪ Chapter Summary

Production automation একটি script নয়, একটি ব্যবস্থা: Git-এ source, আলাদা config, সময়মতো execution, SSH দিয়ে remote কাজ, আর সবকিছু দেখা যায় এমন observability। Bash ভালো Linux command সাজানোর কাজে, Python ভালো তথ্য ও API-র কাজে। পাঁচটি নীতি — idempotency, observability, নিরাপদ ব্যর্থতা, সর্বনিম্ন অধিকার, config থেকে code আলাদা — ব্যবস্থাটিকে নির্ভরযোগ্য করে। ব্যর্থতা থেকে ফেরার উপায় retry, rollback ও সতর্কবার্তা; secret শুধু নিরাপদ জায়গায়; আর production-এ যাওয়ার পথ ছোট, পরীক্ষিত ধাপে। TechNova-র case study দেখায়: সব অধ্যায়ের হাতিয়ার একসঙ্গে বেঁধে একটি ব্যবস্থা বানানো যায় — আর সেই ব্যবস্থাটি ধীরে ধীরে, নিরাপদে বড় করা যায়।

Revision Checklist

- ☐ Bash বনাম Python সিদ্ধান্তের ছক ব্যবহার করতে পারি।
- ☐ Automation-এর পাঁচটি নকশা নীতি ব্যাখ্যা করতে পারি।
- ☐ Production ব্যবস্থার পাঁচটি স্তর আঁকতে পারি।
- ☐ Retry, rollback ও alert-এর পার্থক্য বলতে পারি।
- ☐ Secrets ব্যবস্থাপনার সঠিক নিয়ম বলতে পারি।
- ☐ একটি ছোট production automation ব্যবস্থার নকশা লিখতে পারি।

পরিশিষ্ট A • Practical Labs — হাতে-কলমে অনুশীলন

Appendix A: ২৪টি Guided Practical Lab

এই পরিশিষ্টের প্রতিটি lab-ই বইয়ের অধ্যায়গুলোর বাস্তব প্রয়োগ। কোনো lab শুরু করার আগে সংশ্লিষ্ট অধ্যায়টি পড়ে নিন, তারপর ধাপে ধাপে নির্দেশনা অনুসরণ করুন। প্রতিটি lab-এ একটি বাস্তব দৃশ্য (real-world scenario), স্পষ্ট উদ্দেশ্য, ধাপে ধাপে নির্দেশনা, সম্পূর্ণ কোড, প্রত্যাশিত ফলাফল, যাচাই পদ্ধতি, সমস্যা সমাধান, নিরাপত্তা সতর্কতা ও cleanup আছে। সব কমান্ড Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2 ও Python 3.12-এ পরীক্ষিত। নমুনা আউটপুটগুলো **illustrative** — আপনার সিস্টেমে তারিখ, ব্যবহারকারীর নাম বা হ্যাশ আলাদা হতে পারে, কাঠামো একই থাকবে। কোনো lab-এই আসল পাসওয়ার্ড, টোকেন বা গোপন তথ্য ব্যবহার করবেন না; প্রয়োজনে পরিবেশ চলক (environment variable) বা প্লেসহোল্ডার ব্যবহার করুন।

> **CONCEPT:** Lab করার উদ্দেশ্য শুধু কমান্ড চালানো নয় — প্রতিটি কমান্ডের পেছনে কী ঘটছে তা বোঝা। কোনো ধাপ না বুঝলে থেমে অধ্যায়ে ফিরে যান, তারপর আবার চেষ্টা করুন। ভুল করা শেখার অংশ; cleanup ধাপটি তাই প্রতিটি lab-এ রাখা হয়েছে।

> **WARNING:** Lab-গুলো নিজের ল্যাপটপ, ভার্চুয়াল মেশিন বা পরীক্ষামূলক সার্ভারে করুন। Production সার্ভারে, অন্যের repository-তে বা অপরিচিত সার্ভারে এই কমান্ডগুলো চালাবেন না। `rm -rf`, `git reset --hard` ও `git push --force` ব্যবহারের আগে সবসময় কী মুছতে যাচ্ছেন তা যাচাই করুন।

Lab 01 — Git Repository তৈরি ও পরিদর্শন

বাস্তব দৃশ্য: আপনি TechNova-তে নতুন DevOps engineer হিসেবে যোগ দিয়েছেন। প্রথম কাজ — একটি ছোট automation প্রকল্পের জন্য স্থানীয় Git repository তৈরি করা, যাতে প্রতিটি পরিবর্তনের ইতিহাস সংরক্ষিত থাকে।

উদ্দেশ্য: `git init`, `git status`, `git add`, `git commit` ব্যবহার করে repository তৈরি করা এবং `.git` ডিরেক্টরির ভেতরের কাঠামো পরিদর্শন করা।

পূর্বজ্ঞান: Chapter 01–03; টার্মিনালে ফাইল তৈরি ও ডিরেক্টরি বদলের অভিজ্ঞতা। **পরিবেশ:** Ubuntu 24.04 LTS, Git 2.43, একটি খালি ডিরেক্টরি `~/labs/lab01`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার ডিরেক্টরি তৈরি করুন ও তাতে ঢুকুন:

```
mkdir -p ~/labs/lab01
cd ~/labs/lab01
```

2. Git পরিচয় একবারের জন্য ঠিক করুন (শুধু এই মেশিনে, প্রথমবার):

```
git config --global user.name "Nahid Hasan"
git config --global user.email "nahid@example.com"
git config --global init.defaultBranch main
```

3. খালি repository তৈরি করুন:

```
git init
ls -la
```

4. একটি `README.md` ও একটি `hello.sh` ফাইল তৈরি করুন:

```
printf '# Lab 01 Project\n\nA small automation project.\n' > README.md
printf '#!/usr/bin/env bash\necho "Hello, Git!"\n' > hello.sh
chmod +x hello.sh
```

5. অবস্থা দেখুন, তারপর stage ও commit করুন:

```
git status
git add README.md hello.sh
git status
git commit -m "Add README and hello script"
git log --oneline
git status
```

6. `.git`-এর ভেতরটা পরিদর্শন করুন:

```
ls .git
cat .git/HEAD
ls .git/refs/heads
git branch
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`git init` বর্তমান ডিরেক্টরিতে একটি লুকানো `.git` ডিরেক্টরি তৈরি করে — এটাই repository। `git add` ফাইলকে working directory থেকে staging area-তে নেয়, অর্থাৎ পরের commit-এ কী যাবে তা ঠিক করে। `git commit` staging area-র একটি স্থায়ী snapshot তৈরি করে। `.git/HEAD` ফাইলে লেখা থাকে বর্তমান branch-এর নাম; সেটি একটি ref, যা সর্বশেষ commit-কে নির্দেশ করে।

প্রত্যাশিত ফলাফল (illustrative)

```
[main (root-commit) a1b2c3d] Add README and hello script
 2 files changed, 4 insertions(+)
 create mode 100644 README.md
 create mode 100755 hello.sh
```

`git log --oneline`-এ একটি লাইন দেখা যাবে; `cat .git/HEAD`-এ দেখা যাবে `ref: refs/heads/main`।

যাচাই

- `git status` বলবে `nothing to commit, working tree clean`।
- `git log`-এ আপনার নাম, ইমেইল ও বার্তা দেখা যাবে।
- `ls .git/objects`-এ নতুন object ডিরেক্টরি তৈরি হয়েছে।

TROUBLESHOOTING: `Author identity unknown` এলে `git config --global user.name` ও `user.email` ঠিক করুন। `git commit`-এ কিছু না থাকলে আগে `git add` করেছেন কি না দেখুন।

WARNING: আসল ইমেইল public repository-তে প্রকাশ করতে না চাইলে GitHub-এর noreply ইমেইল ব্যবহার করুন। কখনো পাসওয়ার্ড বা টোকেন commit করবেন না।

Cleanup: `cd ~ && rm -rf ~/labs/lab01` — শুধু এই lab-এর ডিরেক্টরিটিই মুছুন, অন্য কিছু নয়।

Lab 02 — Commit History ও Git Object পরিদর্শন (cat-file)

বাস্তব দৃশ্য: একটি production incident-এর পর আপনাকে খুঁজে বের করতে হবে কোন commit-এ একটি ভুল configuration ঢুকেছিল। `git log` শুধু সারাংশ দেয়; ভেতরের object-গুলো দেখলে বোঝা যায় Git আসলে কী সংরক্ষণ করে।

উদ্দেশ্য: blob, tree ও commit object চেনা; `git cat-file`, `git ls-tree` ও `git show` ব্যবহার করা।

পূর্বজ্ঞান: Lab 01, Chapter 02–03। **পরিবেশ:** `~/labs/lab02`-এ নতুন repository।

ধাপে ধাপে নির্দেশনা

1. Repository তৈরি করে তিনটি commit করুন:

```
mkdir -p ~/labs/lab02 && cd ~/labs/lab02
git init
printf 'version=1\n' > config.txt
git add config.txt
git commit -m "Add config version 1"
printf 'version=2\n' > config.txt
git add config.txt
git commit -m "Update config to version 2"
printf 'notes\n' > notes.txt
git add notes.txt
git commit -m "Add notes file"
```

2. ইতিহাস ও প্রতিটি commit-এর ভেতর দেখুন:

```
git log --oneline
git log --format=fuller
git show HEAD
git show HEAD~1
```

3. সর্বশেষ commit-এর object পরীক্ষা করুন:

```
git cat-file -p HEAD
git cat-file -t HEAD
git ls-tree HEAD
git cat-file -p HEAD^{tree}
```

4. একটি blob-এর বিষয়বস্তু বের করুন (ls-tree থেকে পাওয়া হ্যাশ বসান):

```
git cat-file -p <blob-hash>
git hash-object config.txt
git cat-file -t <blob-hash>
```

5. কোন commit-এ কোন লাইন বদলেছে তা দেখুন:

```
git log -p -- config.txt
git blame config.txt
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

প্রতিটি commit একটি object, যার ভেতরে থাকে tree-র হ্যাশ, parent commit-এর হ্যাশ, লেখক, তারিখ ও বার্তা। Tree object ডিরেক্টরির তালিকা ধারণ করে — কোন ফাইলের নাম, কোন blob হ্যাশ। Blob object শুধু ফাইলের বিষয়বস্তু ধারণ করে, নাম নয়। `git cat-file -p` কোনো object-এর বিষয়বস্তু মানুষের পড়ার মতো করে দেখায়, আর `-t` তার ধরন বলে। একই বিষয়বস্তুর ফাইল দুবার commit করলে একই blob হ্যাশ তৈরি হয় — এটাই Git-এর content-addressable storage।

প্রত্যাশিত ফলাফল (illustrative)

```
tree 9f86d0818...
parent 3a2c1e5...
author Nahid Hasan <nahid@example.com> 1762632000 +0600
committer Nahid Hasan <nahid@example.com> 1762632000 +0600

Add notes file
```

যাচাই

- `git log --oneline`-এ তিনটি commit দেখা যাবে।
- `git hash-object config.txt`-এর ফল `git ls-tree HEAD`-এর blob হ্যাশের সঙ্গে মিলবে।
- `git blame config.txt`-এ প্রতিটি লাইনের পাশে সঠিক commit দেখা যাবে।

TROUBLESHOOTING: `fatal: Not a valid object name HEAD` মানে এখনো কোনো commit হয়নি — আগে commit করুন। `<blob-hash>` জায়গায় আসল হ্যাশ বসাতে হবে; `git ls-tree HEAD`-এর আউটপুট থেকে কপি করুন।

PRO TIP: Incident তদন্তে `git log -p -- <file>` সবচেয়ে কার্যকর — কোন commit-এ কোন লাইন বদলেছে, এক নজরে দেখা যায়।

Cleanup: `cd ~ && rm -rf ~/labs/lab02`।

Lab 03 — Branching ও Merging

বাস্তব দৃশ্য: TechNova-র মূল সার্ভার-মনিটরিং স্ক্রিপ্টটি `main` branch-এ স্থিতিশীলভাবে চলছে। আপনি মূল কোড না ভেঙে একটি নতুন feature — disk-alert — আলাদা branch-এ তৈরি করবেন, পরীক্ষা করে main-এ merge করবেন।

উদ্দেশ্য: Branch তৈরি, বদল, merge ও branch মুছে ফেলা; fast-forward বনাম three-way merge বোঝা।

পূর্বজ্ঞান: Lab 01, Chapter 04। **পরিবেশ:** `~/labs/lab03`।

ধাপে ধাপে নির্দেশনা

1. ভিত্তি repository তৈরি করুন:

```
mkdir -p ~/labs/lab03 && cd ~/labs/lab03
git init
printf '#!/usr/bin/env bash\necho "monitor running"\n' > monitor.sh
chmod +x monitor.sh
git add monitor.sh
git commit -m "Add monitor script"
```

2. Feature branch তৈরি করে তাতে কাজ করুন:

```
git branch feature/disk-alert
git switch feature/disk-alert
printf '\n# Disk alert feature\necho "checking disk"\n' >> monitor.sh
git add monitor.sh
git commit -m "Add disk alert check"
git log --oneline --graph --all
```

3. main-এ ফিরে একটি আলাদা commit করুন (three-way merge তৈরি করতে):

```
git switch main
printf '# Server monitor\n' > README.md
git add README.md
git commit -m "Add README"
```

4. Feature branch merge করুন ও ইতিহাস দেখুন:

```
git merge feature/disk-alert
git log --oneline --graph --all
cat monitor.sh
```

5. Branch মুছে ফেলুন:

```
git branch -d feature/disk-alert
git branch
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Branch হলো একটি commit-এর দিকে নির্দেশ করা হালকা pointer — নতুন branch তৈরি করতে Git কোনো ফাইল কপি করে না। `git switch` HEAD-কে অন্য branch-এ সরায় এবং working directory-র ফাইলগুলো সেই branch-এর অবস্থায় বদলে দেয়। Main-এ নতুন commit থাকায় merge-টি fast-forward হয়নি; Git দুটি parent-সহ একটি merge commit তৈরি করেছে — এটাই three-way merge। Merge-এর পর branch মুছে ফেলা নিরাপদ, কারণ commit-গুলো এখন main-এর ইতিহাসের অংশ।

প্রত্যাশিত ফলাফল (illustrative)

```
Merge made by the 'ort' strategy.
 monitor.sh | 3 +++
 1 file changed, 3 insertions(+)
```

গ্রাফে main ও feature branch-এর commit-গুলো মিলে যাওয়া দেখা যাবে।

যাচাই

- `git log --oneline --graph --all`-এ merge commit-এর দুটি parent লাইন দেখা যাবে।
- `cat monitor.sh`-এ disk-alert লাইনটি থাকবে।
- `git branch`-এ শুধু `main` থাকবে।

> **TROUBLESHOOTING:** Merge-এর সময় `error: Your local changes would be overwritten` এলে আগে `git stash` বা `git commit` করে working tree পরিষ্কার করুন। ভুল branch-এ commit করে ফেললে `git log` দেখে শান্তভাবে `git cherry-pick` (Lab 05) ব্যবহার করুন।

> **WARNING:** Public branch-এ push করা commit-এর ইতিহাস বদলাবেন না; প্রয়োজনে `git revert` ব্যবহার করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab03`।

Lab 04 — Merge Conflict সমাধান

বাস্তব দৃশ্য: আপনি ও আপনার সহকর্মী রাফি একই configuration ফাইলের একই লাইন বদলেছেন — আপনি timeout বাড়িয়েছেন, রাফি কমিয়েছেন। Merge করার সময় Git সিদ্ধান্ত নিতে পারবে না; মানুষ হিসেবে আপনাকেই সঠিক মানটি বেছে নিতে হবে।

উদ্দেশ্য: ইচ্ছাকৃতভাবে conflict তৈরি করা, conflict marker পড়া, হাতে সমাধান করা ও merge সম্পূর্ণ করা।

পূর্বজ্ঞান: Lab 03, Chapter 04। **পরিবেশ:** `~/labs/lab04`।

ধাপে ধাপে নির্দেশনা

1. ভিত্তি ফাইলসহ repository তৈরি করুন:

```
mkdir -p ~/labs/lab04 && cd ~/labs/lab04
git init
printf 'timeout=30\nretries=3\n' > app.conf
git add app.conf
git commit -m "Add app config"
```

2. দুটি branch-এ একই লাইন আলাদাভাবে বদলান:

```
git switch -c feature/timeout-60
printf 'timeout=60\nretries=3\n' > app.conf
git add app.conf
```

```
git commit -m "Increase timeout to 60"
git switch main
printf 'timeout=10\nretries=3\n' > app.conf
git add app.conf
git commit -m "Decrease timeout to 10"
```

3. Merge করে conflict দেখুন:

```
git merge feature/timeout-60
git status
cat app.conf
```

4. ফাইলটি সম্পাদনা করে conflict marker সরান — সিদ্ধান্ত নিন `timeout=45` আপস-মান হিসেবে রাখবেন:

```
printf 'timeout=45\nretries=3\n' > app.conf
git add app.conf
git commit -m "Resolve timeout conflict: use 45 seconds"
```

5. ফল যাচাই করুন:

```
git log --oneline --graph --all
cat app.conf
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Conflict marker-এর `<<<<<<< HEAD` অংশটি আপনার বর্তমান branch-এর (main) সংস্করণ, `=======`-এর নিচের অংশটি incoming branch-এর সংস্করণ, আর `>>>>>>>` তার শেষ। Git ফাইলটিকে মাঝপথে রেখে দেয় — সমাধান না হওয়া পর্যন্ত merge সম্পূর্ণ হয় না। `git add` এখানে নতুন অর্থ বহন করে: এটি Git-কে জানায় এই ফাইলটির conflict সমাধান হয়েছে। শুধু marker মুছে ফেললেই হবে না; সঠিক মানটি নিয়ে দলের সঙ্গে আলোচনা করাই আসল সমাধান — এই lab-এ ৪৫ সেকেন্ড একটি যুক্তিসংগত আপস।

প্রত্যাশিত ফলাফল (illustrative)

```
Auto-merging app.conf
CONFLICT (content): Merge conflict in app.conf
Automatic merge failed; fix conflicts and then commit the result.
```

সমাধানের পর `cat app.conf`-এ `timeout=45` দেখা যাবে এবং কোনো marker থাকবে না।

যাচাই

- `grep -r "<<<<<<<" .` কোনো ফল দেবে না — কোনো marker অবশিষ্ট নেই।
- `git status` বলবে working tree clean।
- `git log --graph`-এ merge commit দেখা যাবে।

> **TROUBLESHOOTING:** Merge বাতিল করতে চাইলে সমাধানের আগে `git merge --abort` চালান। ভুল সমাধান commit করে ফেললে নতুন commit দিয়ে ঠিক করুন; public ইতিহাসে `reset --hard` করবেন না।

> **WARNING:** Conflict সমাধানের সময় দুই পাশের কোড না বুঝে এক পাশ মুছে ফেলবেন না — সহকর্মীর গুরুত্বপূর্ণ পরিবর্তন হারিয়ে যেতে পারে। সন্দেহ হলে `git log` দেখে তার সঙ্গে কথা বলুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab04`।

Lab 05 — Rebase ও Cherry-pick

বাস্তব দৃশ্য: আপনার feature branch-এ তিনটি ছোট commit আছে, কিন্তু main এগিয়ে গেছে। পরিষ্কার, রৈখিক ইতিহাস রাখতে আপনি branch-টিকে main-এর উপর rebase করবেন; আর একটি জরুরি bug-fix commit শুধু আলাদা করে অন্য branch-এ নিতে cherry-pick ব্যবহার করবেন।

উদ্দেশ্য: `git rebase` দিয়ে ইতিহাস সাজানো, rebase conflict সামলানো ও `git cherry-pick` ব্যবহার করা।

পূর্বজ্ঞান: Lab 03–04, Chapter 04। **পরিবেশ:** `~/labs/lab05`।

ধাপে ধাপে নির্দেশনা

1. Repository ও feature branch তৈরি করুন:

```
mkdir -p ~/labs/lab05 && cd ~/labs/lab05
git init
printf 'v1\n' > app.txt
git add app.txt
git commit -m "Initial version"
git switch -c feature/report
printf 'v1\nreport: on\n' > app.txt
git add app.txt
git commit -m "Enable report"
printf 'v1\nreport: on\nformat: json\n' > app.txt
git add app.txt
git commit -m "Set report format to JSON"
```

2. main এগিয়ে নিন, তারপর feature rebase করুন:

```
git switch main
printf 'v2\n' > app.txt
git add app.txt
git commit -m "Bump to version 2"
git switch feature/report
git rebase main
git log --oneline --graph --all
```

3. একটি bug-fix commit cherry-pick করুন:

```
git switch main
printf 'fix: timeout\n' > fix.txt
git add fix.txt
git commit -m "Fix timeout bug"
git switch feature/report
git cherry-pick main
git log --oneline --graph --all
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Rebase আপনার commit-গুলোকে main-এর সর্বশেষ অবস্থার উপর নতুন করে প্রয়োগ করে — পুরনো commit-গুলোর নতুন হ্যাশ তৈরি হয়, তাই ইতিহাস রৈখিক দেখায় কিন্তু আসলে commit পুনর্লেখন হয়। এ কারণে শুধু নিজের স্থানীয়, এখনো push না করা branch-এ rebase করা নিরাপদ। Cherry-pick একটি নির্দিষ্ট commit-এর পরিবর্তন অন্য branch-এ কপি করে; মূল commit অপরিবর্তিত থাকে, নতুন branch-এ নতুন হ্যাশসহ একই পরিবর্তন আসে। Rebase-এর সময় conflict হলে ফাইল ঠিক করে `git add` দিয়ে `git rebase --continue` চালাতে হয়; বাতিল করতে `git rebase --abort`।

প্রত্যাশিত ফলাফল (illustrative)

```
Successfully rebased and updated refs/heads/feature/report.
```

গ্রাফে feature branch-এর commit-গুলো main-এর commit-এর ঠিক পরে সরল রেখায় দেখা যাবে।

যাচাই

- `git log --oneline feature/report`-এ commit-গুলোর ক্রম হবে: Bump to version 2, Enable report, Set report format, Fix timeout bug।
- `cat app.txt` ও `cat fix.txt` প্রত্যাশিত বিষয়বস্তু দেখাবে।

> **TROUBLESHOOTING:** Rebase থেমে গেলে `git status` দেখুন — কোন ফাইলে conflict তা বলবে। সমাধান না করে এগোবেন না। ভুল rebase হলে `git reflog` দিয়ে আগের অবস্থা খুঁজে `git reset --hard <commit>` করতে পারেন — তবে শুধু নিজের স্থানীয় branch-এ।

> **WARNING:** অন্যের সঙ্গে ভাগ করা (pushed) branch-এ কখনো rebase করে force-push করবেন না; তাতে সহকর্মীদের কাজ নষ্ট হয়। সন্দেহ হলে merge ব্যবহার করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab05`।

Lab 06 — GitHub Collaboration: Fork, Clone ও Pull Request

বাস্তব দৃশ্য: TechNova-র একটি open-source টুলে আপনি একটি ছোট সংশোধন (README-তে একটি ভুল বানান ঠিক করা) জমা দিতে চান। আপনার নিজের সরাসরি লেখার অনুমতি নেই — তাই fork করে, clone করে, branch-এ কাজ করে pull request পাঠাতে হবে।

উদ্দেশ্য: GitHub-এ fork, SSH clone, feature branch, push ও pull request-এর সম্পূর্ণ প্রবাহ হাতে-কলমে করা।

পূর্বজ্ঞান: Lab 01–03, Chapter 05–06; একটি GitHub অ্যাকাউন্ট ও SSH key। **পরিবেশ:** ব্রাউজার, টার্মিনাল, ইন্টারনেট সংযোগ।

ধাপে ধাপে নির্দেশনা

1. SSH key আছে কি না যাচাই করুন; না থাকলে তৈরি করুন (passphrase দিন):

```
ls ~/.ssh/id_ed25519.pub
ssh-keygen -t ed25519 -C "nahid@example.com"
cat ~/.ssh/id_ed25519.pub
```

2. Public key-টি GitHub-এ যোগ করুন: GitHub → Settings → SSH and GPG keys → New SSH key। তারপর সংযোগ পরীক্ষা করুন:

```
ssh -T git@github.com
```

3. GitHub-এ একটি ছোট public repository বেছে Fork করুন (উদাহরণ: আপনার নিজের বা পরিচিত কোনো ছোট প্রকল্প), তারপর নিজের fork-টি clone করুন:

```
git clone git@github.com:YOUR-USERNAME/REPO-NAME.git
cd REPO-NAME
git remote -v
```

4. মূল repository-কে upstream হিসেবে যোগ করুন ও feature branch তৈরি করুন:

```
git remote add upstream git@github.com:ORIGINAL-OWNER/REPO-NAME.git
git fetch upstream
git switch -c fix/readme-typo
```

5. একটি ছোট পরিবর্তন করুন, commit করে নিজের fork-এ push করুন:

```
printf '\nFixed a typo in the documentation.\n' >> README.md
git add README.md
git commit -m "Fix typo in README"
git push origin fix/readme-typo
```

6. GitHub ওয়েবসাইটে গিয়ে Compare & pull request-এ ক্লিক করুন। শিরোনামে কী বদলেছেন, বর্ণনায় কেন বদলেছেন — দুটোই লিখুন। তারপর Submit করুন।
7. Maintainer-এর মন্তব্য এলে জবাব দিন; merge হয়ে গেলে স্থানীয় main হালনাগাদ করুন:

```
git switch main
git pull upstream main
git push origin main
git branch -d fix/readme-typo
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`origin` হলো আপনার fork-এর remote নাম, `upstream` মূল প্রকল্পের। এই আলাদা নামকরণই fork-ভিত্তিক workflow-এর মূল — আপনি upstream থেকে নতুন পরিবর্তন আনেন (`fetch` / `pull`), কিন্তু push করেন শুধু নিজের fork-এ। Pull request কোনো Git কমান্ড নয়, এটি GitHub-এর একটি পর্যালোচনা প্রক্রিয়া: আপনি maintainer-কে বলছেন, আমার branch-এর পরিবর্তনগুলো দেখে আপনার main-এ নিন কি না সিদ্ধান্ত নিন।

প্রত্যাশিত ফলাফল (illustrative)

```
Hi Nahid! You've successfully authenticated, but GitHub does not provide shell access.
```

Pull request পৃষ্ঠায় `Able to merge` ও আপনার একটি commit দেখা যাবে।

যাচাই

- `git remote -v`-এ origin ও upstream দুটোই দেখা যাবে।
- GitHub-এ আপনার fork-এর branch-টি ও pull request নম্বর দেখা যাবে।
- Merge-এর পর upstream main-এ আপনার পরিবর্তনটি দেখা যাবে।

> **TROUBLESHOOTING:** `Permission denied (publickey)` এলে SSH key GitHub-এ যোগ হয়েছে কি না এবং `ssh-add ~/.ssh/id_ed25519` চালানো হয়েছে কি না দেখুন। `git push`-এ rejected হলে আগে `git pull --rebase upstream main` করে নিন।

> **WARNING:** Private key (`~/.ssh/id_ed25519`) কখনো কাউকে দেবেন না, কোথাও আপলোড করবেন না, commit করবেন না। শুধু `.pub` ফাইলটি শেয়ার করা নিরাপদ।

Cleanup: পরীক্ষার fork-টি GitHub-এ Delete repository করে মুছে ফেলুন; স্থানীয় clone ডিরেক্টরি মুছুন।

Lab 07 — Bash Variables ও User Input

বাস্তব দৃশ্য: প্রতিদিন সকালে অপারেটরকে সার্ভারের নাম ও একটি সতর্কবার্তা টাইপ করে একটি স্বাগত বার্তা তৈরি করতে হয়। হাতে লিখে ভুল হয়; একটি ছোট স্ক্রিপ্ট ব্যবহারকারীর কাছ থেকে মান নিয়ে নিরাপদে বার্তা তৈরি করবে।

উদ্দেশ্য: Variable ঘোষণা, `read` দিয়ে input নেওয়া, quoting, command substitution ও positional parameter ব্যবহার করা।

পূর্বজ্ঞান: Chapter 07–08। **পরিবেশ:** `~/labs/lab07`, Bash 5.2।

ধাপে ধাপে নির্দেশনা

1. ডিরেক্টরি তৈরি করুন ও স্ক্রিপ্টটি লিখুন:

```
mkdir -p ~/labs/lab07 && cd ~/labs/lab07
cat > greet.sh <<'EOF'
#!/usr/bin/env bash
set -u
server_name="${1:-web-01}"
read -r -p "Your name: " operator_name
read -r -s -p "API key (will not echo): " api_key
printf '\n'
today="$(date +%Y-%m-%d)"
printf 'Hello %s, server %s checked on %s.\n' "$operator_name" "$server_name" "$today"
printf 'API key length: %s characters.\n' "${#api_key}"
EOF
chmod +x greet.sh
```

2. স্বাভাবিকভাবে ও argument দিয়ে চালান:

```
./greet.sh
./greet.sh db-01
```

3. খালি input দিয়ে পরীক্ষা করুন — কী হয় দেখুন:

```
./greet.sh ""
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`${1:-web-01}` মানে প্রথম argument না দিলে `web-01` ব্যবহার করো — এটি একটি নিরাপদ default। `read -r` backslash-কে আক্ষরিকভাবে পড়ে; `-p` prompt দেখায়; `-s` টাইপ করা অক্ষর পর্দায় দেখায় না, তাই গোপন মানের জন্য উপযুক্ত। `"$(date +%Y-%m-%d)"` হলো command substitution — কমান্ডের আউটপুট variable-এ ধরা হয়। সব variable ব্যবহারের সময় double quote (`"$operator_name"`) ব্যবহার করা হয়েছে, যাতে নামে space থাকলেও তা একটিই মান হিসেবে থাকে। `${#api_key}` key-র দৈর্ঘ্য দেয়, কিন্তু key-টি কখনো print করে না — এটাই নিরাপদ অভ্যাস।

প্রত্যাশিত ফলাফল (illustrative)

```
Your name: Nahid
API key (will not echo):
Hello Nahid, server web-01 checked on 2026-11-08.
API key length: 12 characters.
```

যাচাই

- নামে space দিয়ে (যেমন `Nahid Hasan`) চালালে পুরো নাম এক লাইনে দেখা যাবে।
- `bash -n greet.sh` কোনো syntax error দেবে না।
- `shellcheck greet.sh` কোনো সতর্কতা দিলে তা ঠিক করুন (ShellCheck না থাকলে `sudo apt install shellcheck`)।

> **TROUBLESHOOTING:** `unbound variable` এলে কোনো variable ঘোষণার আগেই ব্যবহার করা হয়েছে — `set -u` সেটাই ধরেছে। `read` -এ backslash হারিয়ে গেলে `-r` যোগ করুন।

> **WARNING:** গোপন মান কখনো `echo` দিয়ে print করবেন না, log ফাইলে লিখবেন না, বা command-line argument হিসেবে পাস করবেন না — argument অন্য ব্যবহারকারী `ps` দিয়ে দেখতে পারে।

Cleanup: `cd ~ && rm -rf ~/labs/lab07`।

Lab 08 — Bash Conditions ও Loops

বাস্তব দৃশ্য: TechNova-র একটি ডিরেক্টরিতে প্রতিদিন শত শত log ফাইল জমে। ৭ দিনের পুরনো ফাইলগুলো archive করতে হয়, আর disk ব্যবহার ৮০%-এর বেশি হলে সতর্কবার্তা দিতে হয়। হাতে করলে ভুল হবে; condition ও loop দিয়ে স্বয়ংক্রিয় করুন।

উদ্দেশ্য: `if/elif/else`, `test`, `for` ও `while` loop, সংখ্যা ও স্ট্রিং তুলনা ব্যবহার করা।

পূর্বজ্ঞান: Lab 07, Chapter 09। **পরিবেশ:** `~/labs/lab08`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার log ফাইল তৈরি করুন (কিছু পুরনো তারিখসহ):

```
mkdir -p ~/labs/lab08/logs ~/labs/lab08/archive
cd ~/labs/lab08
touch logs/app-2026-11-01.log logs/app-2026-11-07.log logs/app-2026-11-08.log
touch -d "10 days ago" logs/app-2026-11-01.log
touch -d "2 days ago" logs/app-2026-11-07.log
```

2. Archive স্ক্রিপ্টটি লিখুন:

```
cat > archive_old_logs.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
log_dir="$HOME/labs/lab08/logs"
archive_dir="$HOME/labs/lab08/archive"
if [[ ! -d "$log_dir" ]]; then
  printf 'Error: log directory %s not found.\n' "$log_dir" >&2
  exit 1
fi
count=0
for file in "$log_dir"/*.log; do
  if [[ ! -e "$file" ]]; then
    printf 'No log files found.\n'
    break
  fi
  if [[ "$file" -ot "$archive_dir" ]] || find "$file" -mtime +7 -print -quit | grep -q .;
then
    mv -- "$file" "$archive_dir/"
    printf 'Archived: %s\n' "$(basename "$file")"
    count=$((count + 1))
  else
    printf 'Keeping: %s\n' "$(basename "$file")"
  fi
done
printf 'Total archived: %d\n' "$count"
usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
if [[ "$usage" -ge 80 ]]; then
  printf 'WARNING: disk usage is %s%%.\n' "$usage" >&2
elif [[ "$usage" -ge 60 ]]; then
  printf 'Notice: disk usage is %s%%.\n' "$usage"
else
  printf 'Disk usage OK: %s%%.\n' "$usage"
fi
EOF
chmod +x archive_old_logs.sh
```

3. চালান ও ফল দেখুন:

```
./archive_old_logs.sh
ls logs archive
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`[[ ! -d "$log_dir" ]]` ডিরেক্টরি আছে কি না পরীক্ষা করে; না থাকলে স্ক্রিপ্টটি error বার্তা দিয়ে exit code 1 নিয়ে থামে। `for file in "$log_dir"/*.log` প্রতিটি log ফাইলে একবার করে loop ঘোরে — ফাইলের নামে space থাকলেও quote-এর কারণে ভাঙে না। `find "$file" -mtime +7` ফাইলটি ৭ দিনের পুরনো কি না জানায়। সংখ্যা তুলনায় `-ge` (greater or equal) ব্যবহার করা হয়েছে — স্ট্রিং তুলনার `>` এখানে ভুল ফল দিত। `count=$((count + 1))` arithmetic expansion-এর নিরাপদ রূপ।

প্রত্যাশিত ফলাফল (illustrative)

```
Archived: app-2026-11-01.log
Keeping: app-2026-11-07.log
Keeping: app-2026-11-08.log
Total archived: 1
Disk usage OK: 42%.
```

যাচাই

- `ls archive`-এ শুধু ১০ দিনের পুরনো ফাইলটি থাকবে।
- আবার চালালে `Total archived: 0` হবে — স্ক্রিপ্টটি idempotent।
- Disk শতাংশ আপনার সিস্টেমের আসল মানের সঙ্গে মিলবে (`df -h /` দিয়ে মেলান)।

> **TROUBLESHOOTING:** `*.log` মিল না পেলে loop-এ আক্ষরিক `*.log` স্ট্রিং আসে — তাই `[[ ! -e "$file" ]]` পরীক্ষাটি রাখা হয়েছে। `integer expression expected` এলে `usage` variable-এ শুধু সংখ্যা আসছে কি না দেখুন (`%` চিহ্ন সরানো হয়েছে কি না)।

> **PRO TIP:** Condition-এ সবসময় `[[ ... ]]` ব্যবহার করুন — এটি `[ ... ]`-এর চেয়ে নিরাপদ, word splitting ও glob সমস্যা কম হয়।

Cleanup: `cd ~ && rm -rf ~/labs/lab08`।

Lab 09 — Functions ও Reusable Scripts

বাস্তব দৃশ্য: আপনার কাছে তিনটি আলাদা স্ক্রিপ্টে একই কাজ বারবার লেখা আছে — log বার্তা লেখা, ফাইল আছে কি না পরীক্ষা করা, ডিরেক্টরি তৈরি করা। একটি shared library ফাইলে function-গুলো একবার লিখে সব স্ক্রিপ্ট থেকে ব্যবহার করবেন।

উদ্দেশ্য: Function সংজ্ঞা, parameter, return status, `local` variable ও library sourcing শেখা।

পূর্বজ্ঞান: Lab 07–08, Chapter 10। **পরিবেশ:** `~/labs/lab09`।

ধাপে ধাপে নির্দেশনা

1. Library ফাইলটি তৈরি করুন:

```
mkdir -p ~/labs/lab09 && cd ~/labs/lab09
cat > lib.sh <<'EOF'
```

```
#!/usr/bin/env bash
log_info() {
  local message="$1"
  printf '[INFO] %s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$message"
}
log_error() {
  local message="$1"
  printf '[ERROR] %s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$message" >&2
}
ensure_dir() {
  local dir="$1"
  if [[ -z "$dir" ]]; then
    log_error "ensure_dir called with an empty path."
    return 1
  fi
  if [[ ! -d "$dir" ]]; then
    mkdir -p -- "$dir"
  fi
}
check_file() {
  local file="$1"
  if [[ -f "$file" ]]; then
    return 0
  fi
  log_error "File not found: $file"
  return 1
}
EOF
```

2. Library ব্যবহার করে মূল স্ক্রিপ্ট লিখুন:

```
cat > backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$script_dir/lib.sh"
source_file="${1:-}"
if [[ -z "$source_file" ]]; then
  log_error "Usage: backup.sh <file>"
  exit 1
fi
if ! check_file "$source_file"; then
  exit 1
fi
backup_dir="$HOME/labs/lab09/backups"
ensure_dir "$backup_dir"
cp -- "$source_file" "$backup_dir/$(basename "$source_file").bak"
log_info "Backed up $source_file"
EOF
chmod +x backup.sh
```

3. পরীক্ষা করুন:

```
printf 'important data\n' > data.txt
./backup.sh data.txt
```

```
./backup.sh missing.txt; echo "exit code: $?"
ls backups
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`local message="$1"` function-এর প্রথম argument-কে একটি স্থানীয় variable-এ ধরে — `local` না দিলে variable-টি global হয়ে যেত এবং অন্য function-এর মান নষ্ট করতে পারত। Function কোনো মান return করে না, return করে একটি status (0 = সফল, অন্য কিছু = ব্যর্থ); তাই `check_file` সফল হলে `return 0`, ব্যর্থ হলে `return 1` দেয়, আর কলার `if ! check_file ...` দিয়ে তা পরীক্ষা করে। `source "$script_dir/lib.sh"` library-কে বর্তমান shell-এ লোড করে — `./lib.sh` চালালে আলাদা process হতো, function পাওয়া যেত না। `BASH_SOURCE[0]` দিয়ে স্ক্রিপ্টটি যেখান থেকে চালানো হোক না কেন, তার নিজের ডিরেক্টরি বের করা হয়েছে।

প্রত্যাশিত ফলাফল (illustrative)

```
[INFO] 2026-11-08 10:00:00 Backed up data.txt
[ERROR] 2026-11-08 10:00:01 File not found: missing.txt
exit code: 1
```

যাচাই

- `ls backups`-এ `data.txt.bak` থাকবে, ভেতরে `important data` লেখা থাকবে।
- Argument ছাড়া চালালে usage বার্তা ও exit code 1 আসবে।
- দ্বিতীয়বার চালালেও একই ফল হবে — কোনো duplicate তৈরি হবে না।

> **TROUBLESHOOTING:** `lib.sh: No such file or directory` এলে স্ক্রিপ্টটি অন্য ডিরেক্টরি থেকে চালাচ্ছেন — `script_dir` হিসাবটি ঠিক আছে কি না দেখুন। `command not found: log_info` মানে source লাইনটি চলেনি বা library-তে syntax error আছে — `bash -n lib.sh` চালান।

> **PRO TIP:** Library ফাইলে `set -e` রাখবেন না — library যেকোনো স্ক্রিপ্ট থেকে source হতে পারে, তার নিজের সেটিং কলারের আচরণ বদলে দিতে পারে।

Cleanup: `cd ~ && rm -rf ~/labs/lab09`।

Lab 10 — Text ও Log Processing (grep, sed, awk)

বাস্তব দৃশ্য: একটি web সার্ভারের access log-এ হাজার হাজার লাইন। আপনাকে বের করতে হবে কোন IP সবচেয়ে বেশি request করেছে, কতগুলো 500 error হয়েছে, আর কোন URL-গুলো সবচেয়ে জনপ্রিয়। পুরো ফাইল হাতে পড়া অসম্ভব — pipeline দিয়ে বিশ্লেষণ করুন।

উদ্দেশ্য: `grep`, `sed`, `awk`, `sort`, `uniq`, `cut` দিয়ে log বিশ্লেষণ করা।

পূর্বজ্ঞান: Chapter 11। **পরিবেশ:** `~/labs/lab10`।

ধাপে ধাপে নির্দেশনা

1. নমুনা access log তৈরি করুন:

```
mkdir -p ~/labs/lab10 && cd ~/labs/lab10
cat > access.log <<'EOF'
192.168.1.10 - - [08/Nov/2026:10:00:01 +0600] "GET /index.html HTTP/1.1" 200 512
192.168.1.20 - - [08/Nov/2026:10:00:02 +0600] "GET /api/users HTTP/1.1" 200 1024
192.168.1.10 - - [08/Nov/2026:10:00:03 +0600] "GET /index.html HTTP/1.1" 200 512
192.168.1.30 - - [08/Nov/2026:10:00:04 +0600] "POST /api/login HTTP/1.1" 500 128
192.168.1.10 - - [08/Nov/2026:10:00:05 +0600] "GET /about.html HTTP/1.1" 404 256
192.168.1.20 - - [08/Nov/2026:10:00:06 +0600] "GET /api/users HTTP/1.1" 500 128
EOF
```

2. এক এক করে প্রশ্নের উত্তর বের করুন:

```
wc -l access.log
grep -c ' 500 ' access.log
grep ' 404 ' access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr
awk '{print $7}' access.log | sort | uniq -c | sort -nr
awk '$9 >= 400 {print $1, $7, $9}' access.log
```

3. একটি সারাংশ রিপোর্ট স্ক্রিপ্ট লিখুন:

```
cat > analyse.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
log_file="${1:-access.log}"
if [[ ! -f "$log_file" ]]; then
  printf 'Error: %s not found.\n' "$log_file" >&2
  exit 1
fi
printf 'Total requests: %s\n' "$(wc -l < "$log_file")"
printf 'Errors (4xx/5xx): %s\n' "$(awk '$9 >= 400' "$log_file" | wc -l)"
printf 'Top IPs:\n'
awk '{print $1}' "$log_file" | sort | uniq -c | sort -nr | head -n 5
printf 'Top URLs:\n'
awk '{print $7}' "$log_file" | sort | uniq -c | sort -nr | head -n 5
EOF
chmod +x analyse.sh
./analyse.sh access.log
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`awk '{print $1}'` প্রতিটি লাইনকে space দিয়ে ভেঙে প্রথম ক্ষেত্রটি (IP ঠিকানা) নেয়; `$7` হলো সপ্তম ক্ষেত্র — URL; `$9` status code। `sort | uniq -c` জোড়াটি গণনার মানক পদ্ধতি: `sort` একই মানগুলোকে পাশাপাশি সাজায়, `uniq -c` প্রতিটি স্বতন্ত্র মান কতবার এসেছে গোনে। `sort -nr` সংখ্যা অনুযায়ী বড় থেকে ছোট সাজায়। `awk '$9 >= 400'` একটি শর্ত — শুধু যে লাইনের status 400 বা বেশি, সেগুলোই ছাপে। `grep -c ' 500 '`-এ 500-র দুই পাশে space দেওয়া হয়েছে, যাতে 5001 বা 1500 ভুল করে না মেলে।

প্রত্যাশিত ফলাফল (illustrative)

```
Total requests: 6
Errors (4xx/5xx): 3
Top IPs:
      3 192.168.1.10
```

```
      2 192.168.1.20
      1 192.168.1.30
Top URLs:
      2 /index.html
      2 /api/users
      1 /about.html
      1 /api/login
```

যাচাই

- `grep -c ' 500 '` দেবে 2; `grep -c ' 404 '` দেবে 1।
- হাতে গুনে মোট 6 লাইন ও 3টি error মিলিয়ে দেখুন।
- আরেকটি লাইন log-এ যোগ করে আবার চালালে সংখ্যাগুলো বদলাবে।

TROUBLESHOOTING: `awk`-এর ক্ষেত্র নম্বর ভুল হলে ফল অদ্ভুত আসবে — `head -n 1 access.log` দেখে ক্ষেত্রগুলো গুনুন। Log format আলাদা হলে (যেমন JSON log) এই ক্ষেত্র-ভিত্তিক পদ্ধতি চলবে না; তখন Python (Lab 15–16) ব্যবহার করুন।

WARNING: Log-এ ব্যবহারকারীর ব্যক্তিগত তথ্য (IP, username) থাকতে পারে। রিপোর্ট শেয়ার করার আগে গোপনীয় তথ্য সরিয়ে নিন, আর log ফাইল অন্যের সঙ্গে শেয়ার করবেন না।

Cleanup: `cd ~ && rm -rf ~/labs/lab10`।

Lab 11 — Exit Code Handling

বাস্তব দৃশ্য: একটি nightly backup স্ক্রিপ্ট নীরবে ব্যর্থ হচ্ছে — ফাইল কপি না হলেও cron-এর log-এ কোনো error দেখা যায় না। কারণ স্ক্রিপ্টটি কোনো কমান্ডের exit code পরীক্ষা করে না। এবার প্রতিটি ধাপের exit code ধরে সঠিকভাবে ব্যর্থতা জানাবেন।

উদ্দেশ্য: `$?`, `exit`, `set -e`, `set -o pipefail` ও শর্তাধীন execution বোঝা ও ব্যবহার করা।

পূর্বজ্ঞান: Chapter 12, Lab 07–09। **পরিবেশ:** `~/labs/lab11`।

ধাপে ধাপে নির্দেশনা

1. Exit code-র আচরণ হাতে পরীক্ষা করুন:

```
mkdir -p ~/labs/lab11 && cd ~/labs/lab11
ls existing-file 2>/dev/null; echo "ls missing exit: $?"
ls /nonexistent; echo "ls missing exit: $?"
true; echo "true exit: $?"
false; echo "false exit: $?"
grep -q "hello" /etc/hostname; echo "grep exit: $?"
```

2. Pipeline-এ pipefail-এর পার্থক্য দেখুন:

```
set -o pipefail
grep "zzz-no-match" /etc/hostname | wc -l; echo "pipefail exit: $?"
```

```
set +o pipefail
grep "zzz-no-match" /etc/hostname | wc -l; echo "no pipefail exit: $?"
```

3. Exit code ধরে এমন একটি স্ক্রিপ্ট লিখুন:

```
cat > safe_backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source_file="${1:-}"
dest_dir="${2:-$HOME/labs/lab11/backup}"
if [[ -z "$source_file" ]]; then
  printf 'Usage: safe_backup.sh <source> [dest-dir]\n' >&2
  exit 2
fi
if [[ ! -f "$source_file" ]]; then
  printf 'Error: source file not found: %s\n' "$source_file" >&2
  exit 1
fi
mkdir -p -- "$dest_dir"
if cp -- "$source_file" "$dest_dir/"; then
  printf 'Backup OK: %s\n' "$source_file"
  exit 0
else
  printf 'Error: copy failed for %s\n' "$source_file" >&2
  exit 1
fi
EOF
chmod +x safe_backup.sh
printf 'data\n' > data.txt
./safe_backup.sh data.txt; echo "exit: $?"
./safe_backup.sh missing.txt; echo "exit: $?"
./safe_backup.sh; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Unix-এ প্রতিটি কমান্ড শেষ হওয়ার সময় একটি সংখ্যা ফেরত দেয় — `0` মানে সফল, অন্য যেকোনো সংখ্যা মানে ব্যর্থ। `$?` সর্বশেষ কমান্ডের exit code ধারণ করে, তবে পরের যেকোনো কমান্ড (এমনকি `echo`-ও) সেটি বদলে দেয় — তাই exit code পরীক্ষা করতে হলে ঠিক পরের লাইনেই করতে হয়। `set -e` কোনো কমান্ড ব্যর্থ হলে স্ক্রিপ্টকে তৎক্ষণাৎ থামিয়ে দেয়; `set -o pipefail` pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline-কে ব্যর্থ ধরে — pipefail ছাড়া pipeline-এর exit code শুধু শেষ কমান্ডের (`wc -l`) হতো, যা সবসময় সফল। এই lab-এর স্ক্রিপ্টে ভিন্ন ভিন্ন ভুলের জন্য ভিন্ন exit code ব্যবহার করা হয়েছে: 1 = সাধারণ ভুল, 2 = ভুল ব্যবহার।

প্রত্যাশিত ফলাফল (illustrative)

```
Backup OK: data.txt
exit: 0
Error: source file not found: missing.txt
exit: 1
Usage: safe_backup.sh <source> [dest-dir>
exit: 2
```

যাচাই

- তিনটি চালনার exit code যথাক্রমে 0, 1 ও 2 হবে।

- `ls backup`-এ `data.txt` থাকবে।
- `./safe_backup.sh data.txt && echo SUCCESS || echo FAILED` দেবে `SUCCESS`।

> **TROUBLESHOOTING:** `set -e` থাকা সত্ত্বেও `if`-এর ভেতরের ব্যর্থ কমান্ড স্ক্রিপ্ট থামায় না — এটি ইচ্ছাকৃত, কারণ `if` নিজেই সেই ব্যর্থতা সামলাচ্ছে। `$?` ভুল মান দিলে দেখুন তার আগে কোনো `echo` বা অন্য কমান্ড চলেছে কি না।

> **PRO TIP:** Cron job-এর সাফল্য-ব্যর্থতা exit code দিয়েই বোঝা যায়। Lab 18-এ এই স্ক্রিপ্টের exit code-ই cron log-এ ধরা হবে।

Cleanup: `cd ~ && rm -rf ~/labs/lab11`।

Lab 12 — Bash Debugging (set -x, ShellCheck)

বাস্তব দৃশ্য: একটি deploy স্ক্রিপ্ট ভুল ডিরেক্টরিতে ফাইল কপি করছে, কিন্তু কোথায় ভুল তা বোঝা যাচ্ছে না। স্ক্রিপ্টটি না বদলে তার ভেতরের প্রতিটি ধাপ দেখে ভুলটি খুঁজে বের করবেন — `set -x` trace ও ShellCheck দিয়ে।

উদ্দেশ্য: `bash -x`, `set -x`, `bash -n` ও ShellCheck ব্যবহার করে bug খুঁজে ঠিক করা।

পূর্বজ্ঞান: Chapter 12, Lab 07–11। **পরিবেশ:** `~/labs/lab12`।

ধাপে ধাপে নির্দেশনা

1. ইচ্ছাকৃতভাবে ভুলসহ একটি স্ক্রিপ্ট তৈরি করুন:

```
mkdir -p ~/labs/lab12 && cd ~/labs/lab12
cat > deploy.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
app_dir=/tmp/myapp
target_dir=$HOME/labs/lab12/deploy
mkdir -p $target_dir
cp $app_dir/*.txt $target_dir/
echo "Deployed to $target_dir"
EOF
mkdir -p /tmp/myapp
printf 'hello\n' > "/tmp/myapp/my file.txt"
chmod +x deploy.sh
```

2. ShellCheck চালান (না থাকলে install করুন):

```
sudo apt update && sudo apt install -y shellcheck
shellcheck deploy.sh
```

3. Trace mode-এ চালিয়ে ভুলটি দেখুন:

```
bash -x ./deploy.sh; echo "exit: $?"
```

4. স্ক্রিপ্টটি ঠিক করুন — সব variable quote করুন:

```
cat > deploy.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
app_dir="/tmp/myapp"
target_dir="$HOME/labs/lab12/deploy"
mkdir -p -- "$target_dir"
cp -- "$app_dir"/*.txt "$target_dir/"
printf 'Deployed to %s\n' "$target_dir"
EOF
shellcheck deploy.sh
bash -x ./deploy.sh
ls deploy
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

মূল ভুলটি ছিল quoting-এর অভাব: `cp $app_dir/*.txt`-এ `$app_dir` quote না থাকায় `my file.txt` ফাইলের নামের space-এ ভেঙে দুটি আলাদা argument হয়ে যায়, আর `cp` ব্যর্থ হয়। ShellCheck এই ভুলটি চালানোর আগেই ধরে — SC2086 সতর্কতা (double quote to prevent globbing and word splitting)। `bash -x` প্রতিটি কমান্ড চালানোর আগে `+` চিহ্নসহ দেখায়, variable-গুলো কী মান নিয়ে expand হয়েছে তাও দেখা যায় — এতে ভুলটি চোখে পড়ে। `bash -n` শুধু syntax পরীক্ষা করে, চালায় না — কোনো স্ক্রিপ্ট চালানোর আগে এটি একটি দ্রুত নিরাপত্তা পরীক্ষা।

প্রত্যাশিত ফলাফল (illustrative)

```
+ mkdir -p -- /home/nahid/labs/lab12/deploy
+ cp -- '/tmp/myapp/my file.txt' /home/nahid/labs/lab12/deploy/
+ printf 'Deployed to %s\n' /home/nahid/labs/lab12/deploy
Deployed to /home/nahid/labs/lab12/deploy
```

ঠিক করার পর `shellcheck deploy.sh` কোনো সতর্কতা দেবে না।

যাচাই

- `ls deploy`-এ `my file.txt` থাকবে।
- ShellCheck clean (কোনো আউটপুট নেই) হবে।
- `bash -n deploy.sh` কোনো error দেবে না।

TROUBLESHOOTING: Trace আউটপুট খুব বড় হলে শুধু সন্দেহজনক অংশের চারপাশে `set -x` ও `set +x` বসান। `set -x` trace-এ গোপন মান (পাসওয়ার্ড, টোকেন) দেখা যেতে পারে — debug করার সময় গোপন মানযুক্ত স্ক্রিপ্টের trace কাউকে পাঠাবেন না।

WARNING: `bash -x` চালানোর আগে স্ক্রিপ্টটি পড়ে নিন — অপরিচিত স্ক্রিপ্ট trace mode-এ চালালেও তা পুরোপুরি চলবে, শুধু দেখাবে না। অপরিচিত স্ক্রিপ্ট প্রথমে `bash -n` ও ShellCheck দিয়ে পরীক্ষা করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab12 /tmp/myapp`।

Lab 13 — Python File Automation

বাস্তব দৃশ্য: TechNova-র একটি ডিরেক্টরিতে প্রতিদিন শত শত `.log` ফাইল জমে। Python দিয়ে সেগুলোকে তারিখ অনুযায়ী সাজিয়ে archive করা, পুরনো ফাইল মুছে ফেলা ও একটি রিপোর্ট তৈরি করবেন — Bash-এর চেয়ে জটিল যুক্তি Python-এ লেখা সহজ।

উদ্দেশ্য: `pathlib`, `shutil`, তারিখ গণনা ও নিরাপদ ফাইল অপারেশন ব্যবহার করা।

পূর্বজ্ঞান: Chapter 14–15। **পরিবেশ:** `~/labs/lab13`, Python 3.12, virtual environment।

ধাপে ধাপে নির্দেশনা

1. Virtual environment তৈরি করুন ও পরীক্ষার ফাইল বানান:

```
mkdir -p ~/labs/lab13 && cd ~/labs/lab13
python3 -m venv .venv
source .venv/bin/activate
python3 --version
mkdir -p inbox archive
touch inbox/app-2026-11-01.log inbox/app-2026-11-08.log inbox/report.txt
python3 -c "import os; os.utime('inbox/app-2026-11-01.log', (0, 0))" 2>/dev/null || true
touch -d "10 days ago" inbox/app-2026-11-01.log
```

2. Automation স্ক্রিপ্টটি লিখুন:

```
cat > organize.py <<'EOF'
import shutil
import sys
from datetime import datetime, timedelta
from pathlib import Path

def organize(inbox: Path, archive: Path, days: int = 7) -> dict:
    cutoff = datetime.now() - timedelta(days=days)
    result = {"archived": [], "kept": [], "skipped": []}
    if not inbox.is_dir():
        raise SystemExit(f"Error: inbox not found: {inbox}")
    archive.mkdir(parents=True, exist_ok=True)
    for entry in sorted(inbox.iterdir()):
        if entry.is_dir():
            result["skipped"].append(entry.name)
            continue
        if entry.suffix != ".log":
            result["skipped"].append(entry.name)
            continue
        modified = datetime.fromtimestamp(entry.stat().st_mtime)
        if modified < cutoff:
            target = archive / entry.name
            shutil.move(str(entry), str(target))
            result["archived"].append(entry.name)
        else:
            result["kept"].append(entry.name)
    return result

def main() -> int:
    inbox = Path("inbox")
    archive = Path("archive")
```

```
    result = organize(inbox, archive, days=7)
    print(f"Archived: {len(result['archived'])}")
    for name in result["archived"]:
        print(f"  archived: {name}")
    print(f"Kept: {len(result['kept'])}")
    for name in result["kept"]:
        print(f"  kept: {name}")
    print(f"Skipped: {len(result['skipped'])}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 organize.py
ls inbox archive
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`pathlib.Path` ফাইলের পথকে একটি object হিসেবে ধরে — `entry.is_dir()`, `entry.suffix`, `entry.stat().st_mtime` সবই এর method। String দিয়ে পথ জোড়া লাগানোর চেয়ে এটি নিরাপদ, কারণ operating system-ভেদে separator-এর পার্থক্য Path নিজে সামলায়। `datetime.now() - timedelta(days=7)` ঠিক ৭ দিন আগের সময় বের করে; ফাইলের `st_mtime` সেই সময়ের চেয়ে পুরনো হলে সেটি archive-এর যোগ্য। `shutil.move` ফাইল সরায় — একই filesystem-এ rename, আলাদা filesystem-এ copy-then-delete। `if __name__ == "__main__"` গার্ডটি নিশ্চিত করে এই ফাইলটি import করলে `main()` চলবে না, শুধু সরাসরি চালালেই চলবে — পরীক্ষার জন্য এটি অপরিহার্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Archived: 1
  archived: app-2026-11-01.log
Kept: 1
  kept: app-2026-11-08.log
Skipped: 1
```

যাচাই

- `ls archive`-এ শুধু পুরনো log ফাইলটি থাকবে; `report.txt` inbox-এই থাকবে।
- আবার চালালে `Archived: 0` হবে — idempotent।
- `python3 -m py_compile organize.py` কোনো error দেবে না।

TROUBLESHOOTING: `PermissionError` এলে ফাইলটি অন্য প্রোগ্রাম খুলে রেখেছে কি না দেখুন। `FileNotFoundError` এলে বর্তমান ডিরেক্টরি ঠিক আছে কি না (`pwd`) দেখুন — স্ক্রিপ্টটি `~/labs/lab13` থেকে চালাতে হয়।

WARNING: ফাইল মোছা বা সরানোর স্ক্রিপ্টে সবসময় প্রথমে dry-run (শুধু কী করত তা ছাপা) যোগ করুন; Lab 22-এ এই প্যাটার্নটি পূর্ণাঙ্গভাবে শেখা হবে। আসল ডেটা দিয়ে পরীক্ষা করবেন না।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab13`।

Lab 14 — Python Subprocess Automation

বাস্তব দৃশ্য: একটি Python মনিটরিং টুল থেকে Linux কমান্ড (`df`, `free`, `systemctl`) চালিয়ে ফলাফল সংগ্রহ করতে হবে — কিন্তু shell injection-এর ঝুঁকি ছাড়া, timeout-সহ, stdout ও stderr আলাদাভাবে ধরে।

উদ্দেশ্য: `subprocess.run` নিরাপদভাবে ব্যবহার করা; argument list, `capture_output`, `timeout`, exit code ও `text=True` বোঝা।

পূর্বজ্ঞান: Chapter 16, Lab 13। **পরিবেশ:** `~/labs/lab14`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab14 && cd ~/labs/lab14
python3 -m venv .venv
source .venv/bin/activate
```

2. স্ক্রিপ্টটি লিখুন:

```
cat > run_cmd.py <<'EOF'
import subprocess
import sys

def run_command(argv, timeout=10):
    try:
        result = subprocess.run(
            argv,
            capture_output=True,
            text=True,
            timeout=timeout,
            check=False,
        )
    except subprocess.TimeoutExpired:
        print(f"Error: command timed out after {timeout}s: {argv[0]}")
        return 1
    except FileNotFoundError:
        print(f"Error: command not found: {argv[0]}")
        return 1
    print(f"Command: {' '.join(argv)}")
    print(f"Exit code: {result.returncode}")
    if result.stdout:
        print("--- stdout ---")
        print(result.stdout.rstrip())
    if result.stderr:
        print("--- stderr ---")
        print(result.stderr.rstrip())
    return result.returncode

def main():
    code = 0
    code = run_command(["df", "-h", "/"]) or code
    code = run_command(["free", "-h"]) or code
    code = run_command(["systemctl", "is-active", "ssh"]) or code
    code = run_command(["ls", "/nonexistent-path"]) or code
    code = run_command(["sleep", "30"], timeout=2) or code
```

```
    return code

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 run_cmd.py; echo "script exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত: কমান্ডটি একটি **list** হিসেবে দেওয়া হয়েছে (`["df", "-h", "/"]`), কোনো shell string নয়, আর `shell=True` ব্যবহার করা হয়নি। এতে ব্যবহারকারীর দেওয়া মান (যেমন ফাইলের নাম) কখনো shell কমান্ড হিসেবে ব্যাখ্যা হবে না — shell injection অসম্ভব। `capture_output=True` stdout ও stderr দুটোই ধরে; `text=True` সেগুলোকে bytes-এর বদলে str হিসেবে দেয়। `timeout=10` কমান্ড আটকে গেলে ১০ সেকেন্ড পর `TimeoutExpired` তোলে — `sleep 30` তাই ২ সেকেন্ডে থেমে যায়। `check=False` মানে ব্যর্থ কমান্ড exception তুলবে না; আমরা নিজেরা `returncode` পরীক্ষা করব — এটি automation-এ সঠিক প্যাটার্ন, কারণ ব্যর্থতাও একটি তথ্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Command: df -h /
Exit code: 0
--- stdout ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   21G   27G  44% /
Command: ls /nonexistent-path
Exit code: 2
--- stderr ---
ls: cannot access '/nonexistent-path': No such file or directory
Command: sleep 30
Error: command timed out after 2s: sleep
```

যাচাই

- `df` ও `free` -এর আউটপুট আপনার সিস্টেমের আসল মানের সঙ্গে মিলবে।
- `ls` -এর exit code 2 হবে, কিন্তু স্ক্রিপ্ট থামবে না — সব কমান্ড চলবে।
- স্ক্রিপ্টটি `sleep 30` -এ আটকে থাকবে না; মোট সময় ~২ সেকেন্ডের বেশি হবে না।

TROUBLESHOOTING: `FileNotFoundError` এলে কমান্ডটি install আছে কি না দেখুন (`which df`)। `TimeoutExpired` ধরা না হলে traceback আসবে — `try/except` ঠিক জায়গায় আছে কি না দেখুন।

WARNING: `shell=True` ব্যবহার করবেন না, বিশেষ করে ব্যবহারকারীর input কমান্ডে জোড়া লাগিয়ে। একটি ক্ষতিকর ফাইলের নাম যেমন `file.txt; rm -rf ~` shell-এ চালালে বিপর্যয় ঘটতে পারে; argument list-এ তা শুধু একটি ফাইলের নাম।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab14` ।

Lab 15 — API Collection (Public API থেকে JSON সংগ্রহ)

বাস্তব দৃশ্য: TechNova-র একটি ড্যাশবোর্ডে GitHub-এর একটি public repository-র তথ্য (তারকা সংখ্যা, fork, সর্বশেষ আপডেট) দেখাতে হবে। GitHub API থেকে JSON সংগ্রহ করে প্রয়োজনীয় ক্ষেত্রগুলো বের করবেন — কোনো গোপন token ছাড়াই, কারণ public তথ্যের জন্য token লাগে না।

উদ্দেশ্য: `requests` দিয়ে HTTP GET করা, status code পরীক্ষা করা, JSON parse করা ও timeout/retry সামলানো।

পূর্বজ্ঞান: Chapter 17, Lab 13। **পরিবেশ:** `~/labs/lab15`, Python 3.12, ইন্টারনেট।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab15 && cd ~/labs/lab15
python3 -m venv .venv
source .venv/bin/activate
pip install requests
```

2. স্ক্রিপ্টটি লিখুন:

```
cat > fetch_repo.py <<'EOF'
import sys
import requests

def fetch_repo(owner, repo, timeout=10):
    url = f"https://api.github.com/repos/{owner}/{repo}"
    headers = {
        "Accept": "application/vnd.github+json",
        "User-Agent": "technova-lab/1.0",
    }
    try:
        response = requests.get(url, headers=headers, timeout=timeout)
    except requests.exceptions.Timeout:
        print("Error: request timed out.", file=sys.stderr)
        return None
    except requests.exceptions.RequestException as exc:
        print(f"Error: network problem: {exc}", file=sys.stderr)
        return None
    if response.status_code == 404:
        print(f"Error: repository not found: {owner}/{repo}", file=sys.stderr)
        return None
    if response.status_code == 403:
        print("Error: rate limited. Wait and try again later.", file=sys.stderr)
        return None
    if response.status_code != 200:
        print(f"Error: unexpected status {response.status_code}", file=sys.stderr)
        return None
    return response.json()

def main():
    owner = sys.argv[1] if len(sys.argv) > 1 else "python"
    repo = sys.argv[2] if len(sys.argv) > 2 else "cpython"
    data = fetch_repo(owner, repo)
    if data is None:
        return 1
```

```
    print(f"Repository: {data.get('full_name')}")
    print(f"Description: {data.get('description')}")
    print(f"Stars: {data.get('stargazers_count')}")
    print(f"Forks: {data.get('forks_count')}")
    print(f"Open issues: {data.get('open_issues_count')}")
    print(f"Language: {data.get('language')}")
    print(f"Updated at: {data.get('updated_at')}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 fetch_repo.py python cpython
python3 fetch_repo.py nosuchowner123 nosuchrepo123; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`requests.get(url, timeout=10)`-এ timeout দেওয়া অপরিহার্য — timeout ছাড়া সার্ভার সাড়া না দিলে প্রোগ্রাম অনির্দিষ্টকাল আটকে থাকতে পারে। Status code পরীক্ষা করা হয়েছে JSON parse করার **আগে** — 404 বা 403 উত্তরে JSON কাঠামো আলাদা হতে পারে, সরাসরি parse করলে বিভ্রান্তিকর error আসত। `data.get('description')` ব্যবহার করা হয়েছে `data['description']`-এর বদলে, যাতে কোনো ক্ষেত্র না থাকলে `KeyError` না উঠে `None` আসে। `User-Agent` header দেওয়া হয়েছে কারণ অনেক API অজ্ঞাত client-কে সীমিত করে। এই lab-এ কোনো token ব্যবহার করা হয়নি — public তথ্যের জন্য প্রয়োজন নেই, আর token থাকলে তা কখনো কোডে লিখবেন না, পরিবেশ চলক থেকে পড়বেন (Chapter 18)।

প্রত্যাশিত ফলাফল (illustrative)

```
Repository: python/cpython
Description: The Python programming language
Stars: 68000
Forks: 32000
Open issues: 8500
Language: Python
Updated at: 2026-11-08T09:00:00Z
```

সংখ্যাগুলো আপনার চালানোর সময়ের আসল মান হবে; ভুল repository-র ক্ষেত্রে `Error: repository not found` ও exit code 1 আসবে।

যাচাই

- ব্রাউজারে `https://api.github.com/repos/python/cpython` খুলে তারকা সংখ্যা মিলিয়ে দেখুন।
- ইন্টারনেট বন্ধ করে চালালে network error বার্তা আসবে, traceback নয়।
- `python3 fetch_repo.py torvalds linux` অন্য repository-র তথ্য দেবে।

> **TROUBLESHOOTING:** `ModuleNotFoundError: requests` এলে virtual environment activate আছে কি না দেখুন। `403 rate limited` এলে এক ঘণ্টা অপেক্ষা করুন — unauthenticated অনুরোধে GitHub প্রতি ঘণ্টায় ৬০টি অনুরোধ দেয়। `verify=False` দিয়ে TLS যাচাই বন্ধ করবেন না — এটি নিরাপত্তা ঝুঁকি; certificate সমস্যা হলে সিস্টেমের certificate হালনাগাদ করুন।

> **WARNING:** API token কখনো source code-এ, Git commit-এ বা স্ক্রিনশটে রাখবেন না। Token লাগলে পরিবেশ চলক (`os.environ.get("GITHUB_TOKEN")`) থেকে পড়ুন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab15`।

Lab 16 — JSON ও YAML Processing

বাস্তব দৃশ্য: TechNova-র সার্ভার তালিকা একটি YAML ফাইলে আছে, আর একটি API JSON দেয়। আপনাকে YAML পড়ে সার্ভারগুলোর তালিকা বের করতে হবে, JSON-এ রূপান্তর করতে হবে, ভুল কাঠামো ধরে ফেলতে হবে এবং ফলাফল একটি JSON রিপোর্টে লিখতে হবে।

উদ্দেশ্য: `json` ও `yaml` মডিউল দিয়ে পড়া-লেখা, কাঠামো যাচাই (validation) ও নিরাপদ YAML লোড করা।

পূর্বজ্ঞান: Chapter 18, Lab 13। **পরিবেশ:** `~/labs/lab16`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ ও নমুনা ফাইল তৈরি করুন:

```
mkdir -p ~/labs/lab16 && cd ~/labs/lab16
python3 -m venv .venv
source .venv/bin/activate
pip install pyyaml
cat > servers.yaml <<'EOF'
environment: staging
servers:
  - name: web-01
    host: 192.168.1.10
    role: web
    port: 22
  - name: db-01
    host: 192.168.1.11
    role: database
    port: 22
EOF
cat > sample.json <<'EOF'
{"project": "technova", "servers": 2, "healthy": true}
EOF
```

2. প্রসেসিং স্ক্রিপ্ট লিখুন:

```
cat > process_config.py <<'EOF'
import json
import sys
from pathlib import Path
import yaml

def load_yaml(path):
    with open(path, "r", encoding="utf-8") as handle:
        data = yaml.safe_load(handle)
    if not isinstance(data, dict):
        raise ValueError("YAML root must be a mapping.")
    servers = data.get("servers")
```

```
    if not isinstance(servers, list) or not servers:
        raise ValueError("YAML must contain a non-empty 'servers' list.")
    for server in servers:
        for field in ("name", "host", "role"):
            if field not in server:
                raise ValueError(f"Server entry missing field: {field}")
    return data

def main() -> int:
    try:
        config = load_yaml(Path("servers.yaml"))
    except FileNotFoundError:
        print("Error: servers.yaml not found.", file=sys.stderr)
        return 1
    except (yaml.YAMLError, ValueError) as exc:
        print(f"Error: invalid configuration: {exc}", file=sys.stderr)
        return 1
    with open("sample.json", "r", encoding="utf-8") as handle:
        summary = json.load(handle)
    report = {
        "environment": config.get("environment", "unknown"),
        "project": summary.get("project"),
        "server_count": len(config["servers"]),
        "servers": [
            {"name": s["name"], "host": s["host"], "role": s["role"]}
            for s in config["servers"]
        ],
    }
    with open("report.json", "w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2)
    print(f"Environment: {report['environment']}")
    print(f"Servers: {report['server_count']}")
    for server in report["servers"]:
        print(f"  {server['name']} ({server['role']}) -> {server['host']}")
    print("Wrote report.json")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 process_config.py
cat report.json
```

3. ভুল YAML দিয়ে validation পরীক্ষা করুন:

```
printf 'servers: []\n' > bad.yaml
python3 -c "import process_config;
process_config.load_yaml(__import__('pathlib').Path('bad.yaml'))" 2>&1 | head -n 3; echo
"exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`yaml.safe_load` ব্যবহার করা হয়েছে, `yaml.load` নয় — `safe_load` শুধু সাধারণ ডেটা টাইপ পড়ে, YAML-এর ভেতরে লুকানো Python object তৈরি হতে দেয় না; অনিরাপদ `load` দিয়ে ক্ষতিকর YAML কোড চালানো সম্ভব। Validation-এর ধাপটি গুরুত্বপূর্ণ: ফাইলটি YAML হিসেবে বৈধ হলেই যথেষ্ট নয় — `servers` তালিকা আছে কি না, প্রতিটি সার্ভারে `name`, `host`, `role` আছে কি না, তা আলাদাভাবে পরীক্ষা করা হয়েছে। এতে ভুল configuration দেরিতে

(production-এ গিয়ে) নয়, শুরুতেই ধরা পড়ে। `json.dump(report, handle, indent=2)` মানুষের পড়ার মতো সুন্দর JSON লেখে; `indent` ছাড়া সব এক লাইনে আসত।

প্রত্যাশিত ফলাফল (illustrative)

```
Environment: staging
Servers: 2
  web-01 (web) -> 192.168.1.10
  db-01 (database) -> 192.168.1.11
Wrote report.json
```

যাচাই

- `cat report.json` বৈধ JSON হবে — `python3 -m json.tool report.json` কোনো error দেবে না।
- `bad.yaml` পরীক্ষায় `ValueError` বার্তা আসবে।
- YAML-এ একটি সার্ভারের `host` মুছে চালালে validation error আসবে।

> **TROUBLESHOOTING:** `yaml.scanner.ScannerError` এলে indentation দেখুন — YAML-এ tab ব্যবহার করা যায় না, শুধু space। `ModuleNotFoundError: yaml` এলে `pip install pyyaml` হয়েছে কি না ও venv activate আছে কি না দেখুন।

> **WARNING:** YAML ফাইলে কখনো আসল পাসওয়ার্ড বা API key লিখবেন না। গোপন মান পরিবেশ চলক বা secret manager-এ রাখুন, আর configuration ফাইল Git-এ commit করার আগে `.gitignore` পরীক্ষা করুন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab16`।

Lab 17 — Structured Logging

বাস্তব দৃশ্য: একটি backup টুলের log পড়ে বোঝা যাচ্ছে না কোন backup কখন ব্যর্থ হয়েছে, আর কেন। `print`-এর বদলে Python-এর `logging` মডিউল দিয়ে timestamp, level ও প্রসঙ্গসহ structured log লিখবেন — ফাইলে rotating log সহ।

উদ্দেশ্য: Logger তৈরি, level ব্যবহার, formatter, file handler ও `RotatingFileHandler` শেখা।

পূর্বজ্ঞান: Chapter 19, Lab 13। **পরিবেশ:** `~/labs/lab17`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab17 && cd ~/labs/lab17
python3 -m venv .venv
source .venv/bin/activate
```

2. Logging-সহ স্ক্রিপ্ট লিখুন:

```
cat > backup_tool.py <<'EOF'
import logging
import sys
```

```
from logging.handlers import RotatingFileHandler
from pathlib import Path

def build_logger():
    logger = logging.getLogger("technova.backup")
    logger.setLevel(logging.DEBUG)
    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(logging.INFO)
    console.setFormatter(formatter)
    log_file = RotatingFileHandler(
        "backup.log", maxBytes=10_000, backupCount=3, encoding="utf-8"
    )
    log_file.setLevel(logging.DEBUG)
    log_file.setFormatter(formatter)
    logger.addHandler(console)
    logger.addHandler(log_file)
    return logger

def backup_file(logger, source, dest_dir):
    logger.info("Starting backup of %s", source)
    source_path = Path(source)
    if not source_path.is_file():
        logger.error("Source file not found: %s", source)
        return False
    try:
        dest = Path(dest_dir) / source_path.name
        dest.parent.mkdir(parents=True, exist_ok=True)
        data = source_path.read_bytes()
        logger.debug("Read %d bytes from %s", len(data), source)
        dest.write_bytes(data)
    except OSError as exc:
        logger.exception("Backup failed for %s: %s", source, exc)
        return False
    logger.info("Backup complete: %s -> %s", source, dest)
    return True

def main() -> int:
    logger = build_logger()
    logger.info("Backup tool started")
    ok = backup_file(logger, "data.txt", "backups")
    backup_file(logger, "missing.txt", "backups")
    logger.info("Backup tool finished")
    return 0 if ok else 1

if __name__ == "__main__":
    sys.exit(main())
EOF
printf 'important data\n' > data.txt
python3 backup_tool.py
cat backup.log
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Logger-এর দুটি handler আছে: console handler শুধু INFO ও তার উপরের বার্তা দেখায়, আর file handler DEBUG পর্যন্ত সবকিছু লেখে — এতে টার্মিনালে সংক্ষিপ্ত সারাংশ আর ফাইলে বিস্তারিত তদন্তের তথ্য থাকে। Formatter-এ timestamp, level ও logger-এর নাম আছে — কোনো log লাইন দেখলেই বোঝা যায় কখন, কতটা গুরুতর, কোন অংশ থেকে এসেছে। `RotatingFileHandler(maxBytes=10_000, backupCount=3)` ফাইলটি ১০ KB ছাড়ালে পুরনো ফাইলকে `backup.log.1` করে নতুন ফাইল শুরু করে, সর্বোচ্চ ৩টি পুরনো ফাইল রাখে — disk ভরে যাওয়া ঠেকায়। `logger.exception` শুধু `except` ব্লকের ভেতরে ব্যবহার করা হয়; এটি বার্তার সঙ্গে পুরো traceback লেখে। আরেকটি সূক্ষ্ম বিষয়: `logger.info("Starting backup of %s", source)` — বার্তাটি format করা হয় শুধু তখনই যখন level অনুমতি দেয়; f-string দিয়ে আগেই format করলে অপ্রয়োজনীয় কাজ হতো।

প্রত্যাশিত ফলাফল (illustrative)

```
2026-11-08 10:00:00 | INFO     | technova.backup | Backup tool started
2026-11-08 10:00:00 | INFO     | technova.backup | Starting backup of data.txt
2026-11-08 10:00:00 | INFO     | technova.backup | Backup complete: data.txt -> backups/
data.txt
2026-11-08 10:00:00 | INFO     | technova.backup | Starting backup of missing.txt
2026-11-08 10:00:00 | ERROR    | technova.backup | Source file not found: missing.txt
```

`backup.log`-এ একই লাইনগুলোর সঙ্গে DEBUG লাইন (`Read 15 bytes`) যোগ থাকবে।

যাচাই

- `ls backups`-এ `data.txt` থাকবে।
- Console-এ DEBUG লাইন দেখা যাবে না, কিন্তু `grep DEBUG backup.log` দিলে পাওয়া যাবে।
- বড় ফাইল বারবার লিখে `backup.log.1` তৈরি হওয়া পরীক্ষা করতে পারেন।

> **TROUBLESHOOTING:** একই বার্তা দুবার দেখা গেলে logger-এ handler দুবার যোগ হয়েছে — `build_logger()` একাধিকবার না ডাকার বিষয়টি দেখুন। Log ফাইল খালি থাকলে level দেখুন: root logger-এর level DEBUG না হলে DEBUG বার্তা আসবে না।

> **WARNING:** Log-এ কখনো পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য লিখবেন না। `logger.debug("token=%s", token)` ধরনের লাইন production-এ বিপজ্জনক।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab17`।

Lab 18 — Cron Jobs: নির্ধারিত স্বয়ংক্রিয় কাজ

বাস্তব দৃশ্য: TechNova-র সার্ভারে প্রতিদিন রাত ২টায় একটি health-check স্ক্রিপ্ট চলবে, আর প্রতি ঘণ্টায় একটি disk-check। মানুষ মনে করে চালাবে না — cron দিয়ে সময়মতো চালানোর ব্যবস্থা করবেন, log-সহ, আর একই job দুবার চলা ঠেকাবেন।

উদ্দেশ্য: Crontab লেখা, cron-এর পরিবেশের পার্থক্য বোঝা, `flock` দিয়ে duplicate execution ঠেকানো ও log করা।

পূর্বজ্ঞান: Chapter 20, Lab 11, Lab 17। **পরিবেশ:** Ubuntu 24.04 LTS, `~/labs/lab18`।

ধাপে ধাপে নির্দেশনা

1. Health-check স্ক্রিপ্ট লিখুন (সম্পূর্ণ পথ ব্যবহার করে):

```
mkdir -p ~/labs/lab18 && cd ~/labs/lab18
cat > healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
LOG_FILE="$HOME/labs/lab18/health.log"
{
  printf '[%s] Health check started\n' "$(date '+%Y-%m-%d %H:%M:%S')"
  printf 'Disk: %s\n' "$(df -h / | tail -n 1)"
  printf 'Memory: %s\n' "$(free -h | awk 'NR==2 {print $3 "/" $2}')"
  printf 'Load: %s\n' "$(uptime)"
  printf '[%s] Health check finished\n' "$(date '+%Y-%m-%d %H:%M:%S')"
} >> "$LOG_FILE" 2>&1
EOF
chmod +x healthcheck.sh
./healthcheck.sh
cat health.log
```

2. Crontab দেখুন ও পরীক্ষার জন্য প্রতি মিনিটে চালানোর entry যোগ করুন:

```
crontab -l || true
crontab -e
```

এডিটরে এই লাইনটি যোগ করুন (আপনার আসল home পথ বসান):

```
* * * * * /usr/bin/flock -n /home/nahid/labs/lab18/health.lock /home/nahid/labs/lab18/
healthcheck.sh
```

3. দুই মিনিট অপেক্ষা করে log দেখুন:

```
sleep 130
cat health.log
crontab -l
```

4. পরীক্ষা শেষে entry-টি সরিয়ে ফেলুন:

```
crontab -e
```

পরীক্ষার লাইনটি মুছে সংরক্ষণ করুন। তারপর যাচাই করুন:

```
crontab -l
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Cron entry-র পাঁচটি ক্ষেত্র: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন। `* * * * *` মানে প্রতি মিনিটে; দৈনিক রাত ২টায় চালাতে লিখতে হয় `0 2 * * *`। Cron-এর পরিবেশ আপনার interactive shell-এর মতো নয় — `PATH` খুব ছোট, home ডিরেক্টরি বা virtual environment সক্রিয় থাকে না। তাই স্ক্রিপ্টে `PATH` স্পষ্ট করে দেওয়া হয়েছে, আর সব পথ সম্পূর্ণ (absolute) লেখা হয়েছে; `~/` বা আপেক্ষিক পথ cron-এ নির্ভরযোগ্য নয়। `flock -n` একটি lock ফাইল ধরে: আগের

চালনা এখনো চলতে থাকলে নতুন চালনা শুরু না হয়ে চুপচাপ থেমে যায় — দীর্ঘ job দুবার চলা ঠেকায়। আউটপুট `>> "$LOG_FILE" 2>&1` দিয়ে ফাইলে পাঠানো হয়েছে, নইলে cron প্রতিবার ইমেইল পাঠানোর চেষ্টা করত।

প্রত্যাশিত ফলাফল (illustrative)

```
[2026-11-08 10:02:00] Health check started
Disk: /dev/sda1  50G  21G  27G  44% /
Memory: 1.2Gi/3.8Gi
Load: 10:02:00 up 3 days, 1 user, load average: 0.10, 0.05, 0.01
[2026-11-08 10:02:00] Health check finished
```

দুই মিনিট পর log-এ অন্তত দুটি চালনার রেকর্ড থাকবে।

যাচাই

- `crontab -l` পরীক্ষার entry দেখাবে, আর মুছে ফেলার পর দেখাবে না।
- Log ফাইলে timestamp-গুলো এক মিনিটের ব্যবধানে থাকবে।
- স্ক্রিপ্টটি হাতে চালালেও একই ফরম্যাটে log লিখবে।

TROUBLESHOOTING: Cron job চলছে না মনে হলে `grep CRON /var/log/syslog` দেখুন (অনুমতি লাগতে পারে)। সবচেয়ে সাধারণ কারণ: আপেক্ষিক পথ, ছোট PATH, বা স্ক্রিপ্টে executable permission না থাকা। `crontab -e`-তে syntax ভুল থাকলে cron পুরো ফাইলটি প্রত্যাখ্যান করতে পারে — সংরক্ষণের পর `crontab -l` দিয়ে যাচাই করুন।

WARNING: পরীক্ষার `* * * * *` entry ভুলে রেখে যাবেন না — প্রতি মিনিটে চলা job disk ভরে ফেলতে পারে। Production-এ cron job-এর আগে সবসময় হাতে চালিয়ে পরীক্ষা করুন, আর lock ফাইল ব্যবহার করুন।

Cleanup: Crontab থেকে পরীক্ষার লাইন সরান; `cd ~ && rm -rf ~/labs/lab18`।

Lab 19 — SSH Remote Operations

বাস্তব দৃশ্য: TechNova-র দ্বিতীয় সার্ভারে (staging) একটি health-check চালাতে হবে, আর একটি configuration ফাইল সেখানে পাঠাতে হবে — সার্ভারে বসে নয়, নিজের মেশিন থেকে। পাসওয়ার্ড টাইপ না করে key-based SSH দিয়ে নিরাপদে করবেন।

উদ্দেশ্য: SSH key তৈরি, public key স্থাপন, remote command চালানো, `scp` ও `rsync` দিয়ে ফাইল পাঠানো।

পূর্বজ্ঞান: Chapter 21, Lab 06। **পরিবেশ:** দুটি Linux মেশিন বা একটি local VM (এই lab-এ `localhost`-এ SSH সার্ভার দিয়ে অনুশীলন করা যাবে); Ubuntu 24.04 LTS, OpenSSH।

ধাপে ধাপে নির্দেশনা

1. SSH সার্ভার ও key প্রস্তুত করুন:

```
sudo apt update && sudo apt install -y openssh-server openssh-client
sudo systemctl enable --now ssh
```

```
ssh-keygen -t ed25519 -C "nahid@example.com"
ls -l ~/.ssh/
```

2. নিজের public key নিজের authorized_keys-এ যোগ করুন (পরীক্ষার জন্য localhost):

```
mkdir -p ~/.ssh
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

3. প্রথম সংযোগে host key যাচাই করুন:

```
ssh -o StrictHostKeyChecking=accept-new localhost 'echo connected; hostname; uptime'
```

4. Remote command চালান ও ফাইল পাঠান:

```
ssh localhost 'df -h / | tail -n 1'
ssh localhost 'free -h | head -n 2'
printf 'timeout=45\n' > app.conf
scp app.conf localhost:/tmp/app.conf
ssh localhost 'cat /tmp/app.conf'
```

5. একটি ডিরেক্টরি rsync দিয়ে sync করুন:

```
mkdir -p myproject
printf 'v1\n' > myproject/app.txt
rsync -av --delete myproject/ localhost:/tmp/myproject/
ssh localhost 'ls -l /tmp/myproject/'
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

SSH key জোড়ায় private key আপনার মেশিনে থাকে, public key সার্ভারে যায় — সার্ভার একটি চ্যালেঞ্জ পাঠায় যা শুধু private key দিয়ে সমাধান করা যায়, তাই পাসওয়ার্ড নেটওয়ার্কে যায় না। `~/.ssh`-এর permission 700 ও `authorized_keys`-এর 600 হওয়া বাধ্যতামূলক — ঢিলা permission থাকলে SSH key-কে উপেক্ষা করে। প্রথম সংযোগে সার্ভারের host key দেখানো হয়; `StrictHostKeyChecking=accept-new` অজানা host-এর key প্রথমবার গ্রহণ করে সংরক্ষণ করে, কিন্তু পরে key বদলে গেলে সতর্ক করে — এটি `StrictHostKeyChecking=no`-এর চেয়ে নিরাপদ, কারণ পরেরটি man-in-the-middle আক্রমণ লুকিয়ে ফেলে। `rsync -av --delete` শুধু বদলে যাওয়া ফাইল পাঠায়, আর গন্তব্যের অতিরিক্ত ফাইল মুছে দেয় — বড় deploy-এ `scp`-এর চেয়ে দ্রুত।

প্রত্যাশিত ফলাফল (illustrative)

```
connected
ubuntu-server
 10:00:00 up 3 days, 1 user, load average: 0.10, 0.05, 0.01
timeout=45
sending incremental file list
app.txt
```

যাচাই

- `ssh localhost 'echo ok'` পাসওয়ার্ড না চেয়ে `ok` দেবে।
- `ssh localhost 'cat /tmp/app.conf'`-এ `timeout=45` দেখা যাবে।
- `rsync`-এর পর remote ডিরেক্টরিতে `app.txt` থাকবে।

> **TROUBLESHOOTING:** `Permission denied (publickey)` এলে `~/.ssh` ও `authorized_keys`-এর permission দেখুন, আর সার্ভারের `/var/log/auth.log`-এ কারণ খুঁজুন। `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED` এলে সার্ভারটি সত্যিই নতুন করে install হয়েছে কি না নিশ্চিত না হয়ে `known_hosts` থেকে লাইন মুছবেন না।

> **WARNING:** Private key কখনো শেয়ার করবেন না, ইমেইলে পাঠাবেন না, Git-এ commit করবেন না। Key-তে passphrase দিন। সার্ভারে root login ও password authentication বন্ধ রাখা ভালো অভ্যাস — `/etc/ssh/sshd_config`-এ `PermitRootLogin no` ও `PasswordAuthentication no` যাচাই করুন।

Cleanup: `ssh localhost 'rm -rf /tmp/myproject /tmp/app.conf'`; পরীক্ষার key নিজের মেশিনে রাখলে তা শুধু পরীক্ষার জন্য — আসল সার্ভারে আলাদা key ব্যবহার করুন। `cd ~ && rm -rf ~/labs/lab19 2>/dev/null; true` (এই lab-এ lab19 ডিরেক্টরি তৈরি করা হয়নি, তাই কিছু নাও থাকতে পারে)।

Lab 20 — Backup Automation

বাস্তব দৃশ্য: TechNova-র একটি প্রকল্পের ডিরেক্টরি প্রতিদিন backup নিতে হবে — তারিখসহ সংকুচিত archive তৈরি, checksum দিয়ে অখণ্ডতা যাচাই, পুরনো backup মুছে ফেলা, আর ব্যর্থ হলে স্পষ্ট error।

উদ্দেশ্য: `tar` + `gzip` দিয়ে archive করা, `sha256sum` দিয়ে যাচাই করা, retention নীতি প্রয়োগ করা ও log লেখা।

পূর্বজ্ঞান: Chapter 15, Lab 09, Lab 11, Lab 13। **পরিবেশ:** `~/labs/lab20`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার ডেটা তৈরি করুন:

```
mkdir -p ~/labs/lab20/project ~/labs/lab20/backups
cd ~/labs/lab20
printf 'hello world\n' > project/app.txt
printf 'config v1\n' > project/config.txt
```

2. Backup স্ক্রিপ্ট লিখুন:

```
cat > backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source_dir="$HOME/labs/lab20/project"
backup_dir="$HOME/labs/lab20/backups"
retention_days=7
log_file="$HOME/labs/lab20/backup.log"
stamp="$(date +%Y-%m-%d_%H%M%S)"
archive="$backup_dir/project-$stamp.tar.gz"
if [[ ! -d "$source_dir" ]]; then
```

```
  printf '[%s] ERROR: source not found: %s\n' "$(date '+%F %T')" "$source_dir" >>
"$log_file"
  exit 1
fi
mkdir -p -- "$backup_dir"
tar -czf "$archive" -C "$(dirname "$source_dir")" "$(basename "$source_dir")"
sha256sum "$archive" > "$archive.sha256"
if sha256sum -c "$archive.sha256" > /dev/null 2>&1; then
  printf '[%s] OK: created %s\n' "$(date '+%F %T')" "$(basename "$archive")" >>
"$log_file"
else
  printf '[%s] ERROR: checksum failed for %s\n' "$(date '+%F %T')" "$archive" >>
"$log_file"
  exit 1
fi
find "$backup_dir" -maxdepth 1 -name 'project-*.tar.gz' -mtime +"$retention_days" -delete
find "$backup_dir" -maxdepth 1 -name 'project-*.tar.gz.sha256' -mtime +"$retention_days" -
delete
printf 'Backup complete: %s\n' "$archive"
EOF
chmod +x backup.sh
./backup.sh
ls -l backups
cat backup.log
```

3. Restore পরীক্ষা করুন:

```
mkdir -p /tmp/restore-test
tar -xzf backups/project-*.tar.gz -C /tmp/restore-test
ls -R /tmp/restore-test
diff project/app.txt /tmp/restore-test/project/app.txt && echo "Restore OK"
```

4. পুরনো backup মুছে ফেলার পরীক্ষা:

```
touch -d "10 days ago" backups/project-2000-01-01_000000.tar.gz
touch -d "10 days ago" backups/project-2000-01-01_000000.tar.gz.sha256
./backup.sh
ls backups
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`tar -czf "$archive" -C "$(dirname "$source_dir")" "$(basename "$source_dir")"` — `-C` দিয়ে প্রথমে parent ডিরেক্টরিতে গিয়ে তারপর `project` archive করা হয়েছে, যাতে archive-এর ভেতরে সম্পূর্ণ পথ (`/home/nahid/.../project`) না ঢুকে শুধু `project/` থাকে; restore করার সময় এটি নিরাপদ। `sha256sum` archive-এর একটি আঙুলের ছাপ তৈরি করে; পরে `sha256sum -c` দিয়ে যাচাই করা যায় ফাইলটি একটুও বদলেছে কি না — backup নেওয়া হয়েছে কিন্তু তা নষ্ট, এমন backup-এর চেয়ে backup না থাকা ভালো নয়, তবে যাচাই না করা backup প্রায় একই রকম বিপজ্জনক। `find ... -mtime +7 -delete` ৭ দিনের পুরনো backup মুছে disk বাঁচায় — retention নীতি ছাড়া backup disk ভরে যাবে।

প্রত্যাশিত ফলাফল (illustrative)

```
Backup complete: /home/nahid/labs/lab20/backups/project-2026-11-08_100000.tar.gz
Restore OK
```

দ্বিতীয় চালনার পর ২০০০ সালের ভুয়া পুরনো ফাইল দুটি আর `ls backups`-এ থাকবে না।

যাচাই

- `sha256sum -c backups/*.sha256` বলবে `OK`।
- Restore করা `app.txt` মূল ফাইলের সঙ্গে `diff`-এ মিলবে।
- `cat backup.log`-এ প্রতিটি চালনার `OK` লাইন থাকবে।

> **TROUBLESHOOTING:** `tar: Cowardly refusing to create an empty archive` এলে source ডিরেক্টরি খালি কি না দেখুন। `find -delete` চালানোর আগে সবসময় `-delete` ছাড়া `find` চালিয়ে দেখুন কোন ফাইলগুলো মিলছে — ভুল pattern-এ গুরুত্বপূর্ণ ফাইল মুছে যেতে পারে।

> **WARNING:** Backup একই disk-এ রাখলে disk নষ্ট হলে backup-ও যাবে — গুরুত্বপূর্ণ ডেটার backup আলাদা মেশিনে বা offsite-এ রাখুন। Backup-এ গোপন ফাইল (.env, key) থাকলে archive-টি encrypt করুন বা সুরক্ষিত জায়গায় রাখুন।

Cleanup: `rm -rf /tmp/restore-test; cd ~ && rm -rf ~/labs/lab20`।

Lab 21 — Health Checking

বাস্তব দৃশ্য: TechNova-র staging সার্ভারটি সকালে ধীর মনে হচ্ছে। CPU, memory, disk, সার্ভিসের অবস্থা ও সাম্প্রতিক error log এক জায়গায় দেখে দ্রুত সিদ্ধান্ত নিতে হবে — সমস্যাটি কোথায়।

উদ্দেশ্য: একটি Bash health-check স্ক্রিপ্ট লেখা যা threshold অনুযায়ী OK/WARNING/CRITICAL জানায় ও exit code দেয়।

পূর্বজ্ঞান: Chapter 12, Lab 08, Lab 11। **পরিবেশ:** `~/labs/lab21`।

ধাপে ধাপে নির্দেশনা

1. স্ক্রিপ্টটি লিখুন:

```
mkdir -p ~/labs/lab21 && cd ~/labs/lab21
cat > healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
warn_disk=80
crit_disk=90
warn_mem=80
overall=0
report() {
  local level="$1"
  local message="$2"
  printf '[%s] %s\n' "$level" "$message"
```

```
  if [[ "$level" == "CRITICAL" ]]; then
    overall=2
  elif [[ "$level" == "WARNING" && "$overall" -lt 2 ]]; then
    overall=1
  fi
}
disk_usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
if [[ "$disk_usage" -ge "$crit_disk" ]]; then
  report CRITICAL "Disk usage ${disk_usage}% (threshold ${crit_disk}%)"
elif [[ "$disk_usage" -ge "$warn_disk" ]]; then
  report WARNING "Disk usage ${disk_usage}% (threshold ${warn_disk}%)"
else
  report OK "Disk usage ${disk_usage}%"
fi
mem_total="$(free -m | awk 'NR==2 {print $2}')"
mem_used="$(free -m | awk 'NR==2 {print $3}')"
mem_pct=$((mem_used * 100 / mem_total))
if [[ "$mem_pct" -ge "$warn_mem" ]]; then
  report WARNING "Memory usage ${mem_pct}% (${mem_used}MB/${mem_total}MB)"
else
  report OK "Memory usage ${mem_pct}% (${mem_used}MB/${mem_total}MB)"
fi
load="$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | tr -d ' ')"
report OK "Load average (1 min): $load"
for service in ssh cron; do
  if systemctl is-active --quiet "$service"; then
    report OK "Service $service is active"
  else
    report CRITICAL "Service $service is not active"
  fi
done
if [[ -f /var/log/syslog ]]; then
  errors="$(grep -ci 'error' /var/log/syslog 2>/dev/null || true)"
  report OK "Syslog error lines (today's file): $errors"
fi
printf 'Overall status code: %s\n' "$overall"
exit "$overall"
EOF
chmod +x healthcheck.sh
./healthcheck.sh; echo "exit: $?"
```

2. Threshold বদলে পরীক্ষা করুন:

```
sed 's/warn_disk=80/warn_disk=1/' healthcheck.sh > healthcheck-low.sh
chmod +x healthcheck-low.sh
./healthcheck-low.sh; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

স্ক্রিপ্টটি Nagios-ধাঁচের exit code ব্যবহার করে: 0 = OK, 1 = WARNING, 2 = CRITICAL — এই প্রথা মনিটরিং টুলগুলো বোঝে, তাই এই স্ক্রিপ্টকে পরে যেকোনো মনিটরিং সিস্টেমে লাগানো যাবে। `report` function প্রতিটি পরীক্ষার ফল ছাপে ও সবচেয়ে খারাপ অবস্থাটি `overall`-এ ধরে রাখে — একটি CRITICAL থাকলে সামগ্রিক ফল CRITICAL-ই থাকবে, পরের OK তা মুছে দেবে না। `df --output=pcent /` শুধু শতাংশের কলাম দেয়, `tr -dc '0-9'` শুধু সংখ্যা রাখে — এতে তুলনা নিরাপদ হয়। `systemctl is-active --quiet` কোনো আউটপুট না দিয়ে শুধু exit code দেয়, যা `if`-এ সরাসরি

ব্যবহার করা যায়। `grep -c`-এর exit code মিল না পেলে 1 হয়, তাই `|| true` দিয়ে `set -e`-এর কারণে স্ক্রিপ্ট থেমে যাওয়া ঠেকানো হয়েছে।

প্রত্যাশিত ফলাফল (illustrative)

```
[OK] Disk usage 44%
[OK] Memory usage 31% (1200MB/3800MB)
[OK] Load average (1 min): 0.10
[OK] Service ssh is active
[OK] Service cron is active
Overall status code: 0
```

Threshold 1% করলে disk লাইনে `[WARNING]` বা `[CRITICAL]` আসবে ও exit code 1 বা 2 হবে।

যাচাই

- স্বাভাবিক চালনায় exit code 0 হবে; threshold কমালে 1 বা 2 হবে।
- `df -h /` ও `free -h`-এর মানের সঙ্গে রিপোর্টের সংখ্যা মিলবে।
- একটি সার্ভিস বন্ধ করে চালালে (পরীক্ষার VM-এ) সেই লাইনে CRITICAL আসবে।

> **TROUBLESHOOTING:** `systemctl` container বা WSL-এ কাজ না করলে সার্ভিস পরীক্ষার অংশটি এড়িয়ে যান — সেটি পরিবেশের সীমা, স্ক্রিপ্টের ভুল নয়। `df` ভিন্ন disk দেখালে `/`-এর বদলে সঠিক mount point ব্যবহার করুন।

> **PRO TIP:** এই স্ক্রিপ্টকে Lab 18-এর cron-এ লাগান: exit code 0 না হলে cron log-এ তা দেখা যাবে, আর পরে alerting যোগ করা যাবে।

Cleanup: `cd ~ && rm -rf ~/labs/lab21`।

Lab 22 — CLI Application (argparse Tool)

বাস্তব দৃশ্য: TechNova-র অপারেটররা একটি কমান্ড-লাইন টুল চায়: `sysinfo --format json --output report.json` চালালে সিস্টেমের তথ্য JSON-এ সংরক্ষিত হবে; `--dry-run` দিলে কিছু না লিখে শুধু দেখাবে কী করত। পেশাদার CLI-র সব বৈশিষ্ট্য — help, validation, dry-run — একসঙ্গে তৈরি করবেন।

উদ্দেশ্য: `argparse` দিয়ে subcommand/option, validation, dry-run mode ও clean exit code-সহ CLI লেখা।

পূর্বজ্ঞান: Chapter 22, Lab 13–14। **পরিবেশ:** `~/labs/lab22`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab22 && cd ~/labs/lab22
python3 -m venv .venv
source .venv/bin/activate
```

2. CLI টুলটি লিখুন:

```
cat > sysinfo.py <<'EOF'
import argparse
import json
import platform
import shutil
import sys
from pathlib import Path

def collect_info():
    disk = shutil.disk_usage("/")
    return {
        "system": platform.system(),
        "release": platform.release(),
        "machine": platform.machine(),
        "python": platform.python_version(),
        "disk_total_gb": round(disk.total / (1024 ** 3), 2),
        "disk_free_gb": round(disk.free / (1024 ** 3), 2),
    }

def build_parser():
    parser = argparse.ArgumentParser(
        prog="sysinfo",
        description="Collect basic system information and save it as JSON.",
    )
    parser.add_argument(
        "--format",
        choices=["json", "text"],
        default="text",
        help="Output format (default: text).",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=None,
        help="File to write the report to.",
    )
    parser.add_argument(
        "--dry-run",
        action="store_true",
        help="Show what would be done without writing any file.",
    )
    return parser

def main(argv=None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    if args.format == "json" and args.output is None and not args.dry_run:
        parser.error("--format json requires --output.")
    info = collect_info()
    if args.format == "json":
        text = json.dumps(info, indent=2)
    else:
        lines = [f"{key}: {value}" for key, value in info.items()]
        text = "\n".join(lines)
    if args.dry_run:
        print("[DRY RUN] Would write the following report:")
        print(text)
        return 0
```

```
    if args.output is not None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(text + "\n", encoding="utf-8")
        print(f"Wrote report to {args.output}")
    else:
        print(text)
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 sysinfo.py --help
python3 sysinfo.py
python3 sysinfo.py --format json --output report.json
cat report.json
python3 sysinfo.py --format json --dry-run
python3 sysinfo.py --format json; echo "exit: $?"
python3 sysinfo.py --format xml; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`argparse` নিজে `--help` তৈরি করে, ভুল option-এ সুন্দর error দেয় ও exit code 2 ফেরত দেয় — হাতে `sys.argv` parse করার চেয়ে এটি অনেক নিরাপদ ও রক্ষণাবেক্ষণযোগ্য। `choices=["json", "text"]` ভুল format আগেই আটকে দেয়। `parser.error(...)` ব্যবহৃত হয়েছে একটি ব্যবসায়িক নিয়মের জন্য: JSON format-এ output ফাইল ছাড়া অর্থ হয় না, তাই সেটি ভুল হিসেবে জানানো হয়েছে। `main(argv=None)` signature-টি পরীক্ষার জন্য গুরুত্বপূর্ণ — test থেকে `main(["--dry-run"])` ডেকে পরীক্ষা করা যায়, command line ছাড়াই (Lab 23-এ দেখা যাবে)। Dry-run mode-এ কোনো ফাইল লেখা হয় না — automation টুলে এটি অপরিহার্য নিরাপত্তা বৈশিষ্ট্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Wrote report to report.json
{
  "system": "Linux",
  "release": "6.8.0-31-generic",
  "machine": "x86_64",
  "python": "3.12.3",
  "disk_total_gb": 50.0,
  "disk_free_gb": 27.0
}
```

`--format json` একা চালালে `error: --format json requires --output.` ও exit code 2 আসবে।

যাচাই

- `python3 sysinfo.py --help`-এ তিনটি option-এর বর্ণনা দেখা যাবে।
- `report.json` বৈধ JSON হবে।
- `--dry-run` চালানোর পর নতুন কোনো ফাইল তৈরি হবে না।
- `--format xml`-এ `invalid choice` error আসবে।

TROUBLESHOOTING: `argparse` error-এ traceback আসে না — এটি স্বাভাবিক, কারণ `parser.error` নিজে সুন্দর বার্তা দিয়ে exit করে। `--output`-এ ভুল পথ দিলে `OSError` আসবে; `parent.mkdir(parents=True, exist_ok=True)` শুধু parent ডিরেক্টরি তৈরি করে, ভুল পথ ঠিক করে না।

PRO TIP: প্রতিটি automation টুলে `--help`, `--dry-run` ও স্পষ্ট exit code রাখুন — ছয় মাস পরে আপনিই এর সবচেয়ে বড় ব্যবহারকারী হবেন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab22`।

Lab 23 — Automated Testing (pytest)

বাস্তব দৃশ্য: Lab 22-এর CLI টুলটি এখন দলের সবাই ব্যবহার করে। একটি ছোট পরিবর্তনেই সব ভেঙে যেতে পারে — তাই পরিবর্তনের আগে স্বয়ংক্রিয় test লিখবেন, যাতে ভুল ধরা পড়ে deploy-এর আগেই।

উদ্দেশ্য: `pytest` দিয়ে unit test লেখা, fixture ও `tmp_path` ব্যবহার করা, mock দিয়ে বাইরের কমান্ড অনুকরণ করা।

পূর্বজ্ঞান: Chapter 23, Lab 22। **পরিবেশ:** `~/labs/lab23`, Python 3.12, pytest।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ ও Lab 22-এর কোড কপি করুন:

```
mkdir -p ~/labs/lab23 && cd ~/labs/lab23
python3 -m venv .venv
source .venv/bin/activate
pip install pytest
```

2. পরীক্ষার জন্য ছোট একটি মডিউল লিখুন:

```
cat > diskcheck.py <<'EOF'
import shutil

def disk_free_percent(path="/"):
    usage = shutil.disk_usage(path)
    if usage.total == 0:
        raise ValueError("Disk total is zero.")
    return round(usage.free / usage.total * 100, 2)

def status_for(free_percent, warn=20.0, crit=10.0):
    if free_percent < crit:
        return "CRITICAL"
    if free_percent < warn:
        return "WARNING"
    return "OK"
EOF
cat > test_diskcheck.py <<'EOF'
import pytest
from diskcheck import disk_free_percent, status_for

def test_status_ok():
    assert status_for(50.0) == "OK"
```

```
def test_status_warning():
    assert status_for(15.0) == "WARNING"

def test_status_critical():
    assert status_for(5.0) == "CRITICAL"

def test_status_boundary():
    assert status_for(20.0) == "OK"
    assert status_for(10.0) == "WARNING"

def test_disk_free_percent_real():
    value = disk_free_percent("/")
    assert 0 <= value <= 100

def test_disk_free_percent_invalid():
    with pytest.raises(OSError):
        disk_free_percent("/nonexistent-path-xyz")

def test_status_parametrized():
    cases = [(50.0, "OK"), (15.0, "WARNING"), (5.0, "CRITICAL")]
    for value, expected in cases:
        assert status_for(value) == expected
EOF
python3 -m pytest -v
```

3. ইচ্ছাকৃতভাবে একটি ভুল ঢুকিয়ে test ধরতে পারে কি না দেখুন:

```
sed -i 's/< crit/<= crit/' diskcheck.py
python3 -m pytest -v 2>&1 | tail -n 15
sed -i 's/<= crit/< crit/' diskcheck.py
python3 -m pytest -v
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Test-এর নাম `test_` দিয়ে শুরু হয় — pytest এই নাম দেখেই test খুঁজে পায়। প্রতিটি test একটি ছোট দাবি (assertion): `assert status_for(15.0) == "WARNING"` মানে এই input-এ এই output আসতেই হবে, না এলে test ব্যর্থ। Boundary test (`20.0` ঠিক সীমায় OK, `10.0` ঠিক সীমায় WARNING) সবচেয়ে মূল্যবান — ভুলের বেশিরভাগ সীমায় লুকিয়ে থাকে। `pytest.raises(OSError)` পরীক্ষা করে ভুল input-এ সঠিক exception ওঠে কি না — সাফল্যের পাশাপাশি ব্যর্থতার আচরণও পরীক্ষা করতে হয়। `test_disk_free_percent_real` আসল disk পড়ে — এটি একটি integration test; বাকিগুলো pure function-এর unit test, তাই দ্রুত ও নির্ভরযোগ্য। ইচ্ছাকৃত ভুলটি (`<` বদলে `<=`) boundary test ধরে ফেলেছিল — এটাই test-এর প্রমাণ।

প্রত্যাশিত ফলাফল (illustrative)

```
test_diskcheck.py::test_status_ok PASSED
test_diskcheck.py::test_status_warning PASSED
test_diskcheck.py::test_status_critical PASSED
test_diskcheck.py::test_status_boundary PASSED
test_diskcheck.py::test_disk_free_percent_real PASSED
test_diskcheck.py::test_disk_free_percent_invalid PASSED
test_diskcheck.py::test_status_parametrized PASSED
7 passed in 0.05s
```

যাচাই

- সব test পাস হবে; ভুল ঢোকানোর পর অন্তত একটি test ব্যর্থ হবে।
- `python3 -m pytest -v` প্রতিটি test-এর নামসহ ফল দেখাবে।
- `python3 -m pytest --collect-only` ৭টি test খুঁজে পাবে।

> **TROUBLESHOOTING:** `ModuleNotFoundError: diskcheck` এলে test ফাইল ও মডিউল একই ডিরেক্টরিতে আছে কি না এবং pytest সেই ডিরেক্টরি থেকে চালানো হচ্ছে কি না দেখুন। Test হঠাৎ ধীর হলে দেখুন কোনো test আসল নেটওয়ার্ক বা disk-এর উপর নির্ভর করছে কি না — সেগুলোকে mock দিয়ে আলাদা করুন।

> **PRO TIP:** নতুন feature লেখার আগে তার test লিখুন (test-driven development-এর সরল রূপ) — test-টি প্রথমে ব্যর্থ হবে, feature লেখার পর পাস করবে। এতে feature-টি আসলেই কাজ করে, তা প্রমাণিত হয়।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab23`।

Lab 24 — Complete Toolkit Integration (Git + Bash + Python + Cron)

বাস্তব দৃশ্য: TechNova-র staging সার্ভারে একটি সম্পূর্ণ ছোট toolkit চালু করতে হবে: Bash health-check, Python রিপোর্ট তৈরি, JSON রিপোর্ট, Git-এ সংস্করণ নিয়ন্ত্রণ, আর cron দিয়ে দৈনিক চালনা — বইয়ের সব অংশ একসঙ্গে।

উদ্দেশ্য: আগের lab-গুলোর উপাদানগুলোকে একটি Git repository-তে সাজিয়ে, cron-এ লাগিয়ে একটি end-to-end automation pipeline তৈরি করা।

পূর্বজ্ঞান: Lab 01, 09, 11, 13, 16, 18, 21, 22। **পরিবেশ:** `~/labs/lab24`, Git, Bash, Python 3.12, cron।

ধাপে ধাপে নির্দেশনা

1. Repository ও কাঠামো তৈরি করুন:

```
mkdir -p ~/labs/lab24/toolkit && cd ~/labs/lab24/toolkit
git init
mkdir -p scripts reports logs
cat > .gitignore <<'EOF'
__pycache__/
*.pyc
.venv/
reports/*.json
logs/*.log
EOF
cat > README.md <<'EOF'
# TechNova Mini Toolkit

Daily health check (Bash) + JSON report (Python), run by cron.

Usage:
  ./scripts/healthcheck.sh
  python3 scripts/report.py --output reports/latest.json
EOF
```

2. Bash health-check লিখুন:

```
cat > scripts/healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
disk_usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
mem_pct="$(free | awk 'NR==2 {printf "%d", $3/$2*100}')"
printf 'disk_usage_percent=%s\n' "$disk_usage"
printf 'memory_usage_percent=%s\n' "$mem_pct"
printf 'load_1min=%s\n' "$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | tr
-d ' ')"
if [[ "$disk_usage" -ge 90 || "$mem_pct" -ge 90 ]]; then
  exit 2
elif [[ "$disk_usage" -ge 80 || "$mem_pct" -ge 80 ]]; then
  exit 1
fi
exit 0
EOF
chmod +x scripts/healthcheck.sh
```

3. Python রিপোর্ট টুল লিখুন:

```
cat > scripts/report.py <<'EOF'
import argparse
import json
import subprocess
import sys
from pathlib import Path

def run_healthcheck():
    result = subprocess.run(
        ["bash", "scripts/healthcheck.sh"],
        capture_output=True,
        text=True,
        timeout=30,
    )
    metrics = {}
    for line in result.stdout.splitlines():
        if "=" in line:
            key, value = line.split("=", 1)
            metrics[key.strip()] = value.strip()
    return result.returncode, metrics

def main(argv=None) -> int:
    parser = argparse.ArgumentParser(description="Build a health report.")
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args(argv)
    code, metrics = run_healthcheck()
    status = {0: "OK", 1: "WARNING", 2: "CRITICAL"}.get(code, "UNKNOWN")
    report = {"status": status, "exit_code": code, "metrics": metrics}
    if args.dry_run:
        print(json.dumps(report, indent=2))
        return 0
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(report, indent=2) + "\n", encoding="utf-8")
    print(f"Status: {status}")
    print(f"Wrote {args.output}")
    return 0
```

```
if __name__ == "__main__":
    sys.exit(main())
EOF
./scripts/healthcheck.sh; echo "health exit: $?"
python3 scripts/report.py --dry-run
python3 scripts/report.py --output reports/latest.json
cat reports/latest.json
```

4. Git-এ commit করুন ও cron-এ লাগান:

```
git add .
git status
git commit -m "Add integrated health toolkit"
git log --oneline
crontab -l || true
```

Cron-এ পরীক্ষার জন্য (একবার, প্রতি মিনিটে — পরীক্ষার পর সরিয়ে ফেলবেন) এই লাইনটি যোগ করুন, আপনার আসল পথ বসিয়ে:

```
* * * * * /usr/bin/flock -n /home/nahid/labs/lab24/toolkit/run.lock /bin/bash -c 'cd /
home/nahid/labs/lab24/toolkit && python3 scripts/report.py --output reports/latest.json >>
logs/cron.log 2>&1'
```

দুই মিনিট অপেক্ষা করে দেখুন:

```
sleep 130
cat reports/latest.json
cat logs/cron.log
crontab -e
```

পরীক্ষার লাইনটি মুছে সংরক্ষণ করুন।

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

এই toolkit-এ প্রতিটি স্তরের দায়িত্ব আলাদা: Bash শুধু সিস্টেমের তথ্য সংগ্রহ করে ও threshold অনুযায়ী exit code দেয়; Python সেই আউটপুট parse করে JSON রিপোর্ট তৈরি করে — Python সরাসরি shell কমান্ড চালায়নি, Bash স্ক্রিপ্টটিকে argument list দিয়ে ডেকেছে, তাই injection-এর ঝুঁকি নেই। `.gitignore`-এ `reports/*.json` ও `logs/*.log` রাখা হয়েছে — উৎপন্ন ফাইল Git-এ যাবে না, শুধু source code যাবে। Cron লাইনে `flock` duplicate চালনা ঠেকায়, আর `cd ... && python3 ...` নিশ্চিত করে আপেক্ষিক পথ (`scripts/healthcheck.sh`) সঠিক ডিরেক্টরি থেকে চলছে — cron-এর working directory অনিশ্চিত, তাই `cd` অপরিহার্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Status: OK
Wrote reports/latest.json
{
  "status": "OK",
  "exit_code": 0,
  "metrics": {
    "disk_usage_percent": "44",
    "memory_usage_percent": "31",
    "load_1min": "0.10"
```

```
  }
}
```

যাচাই

- `git log --oneline` -এ একটি commit দেখা যাবে; `git status` clean থাকবে (reports ও logs ignore করা হয়েছে)।
- `python3 scripts/report.py --dry-run` কোনো ফাইল লিখবে না।
- Cron চালনার পর `reports/latest.json` -এর সময় বদলাবে ও `logs/cron.log` -এ নতুন লাইন যোগ হবে।
- Threshold বদলে (disk 90%→1%) চালালে status WARNING/CRITICAL হবে।

TROUBLESHOOTING: Cron-এ Python স্ক্রিপ্ট না চললে `logs/cron.log` -এ traceback দেখুন — সাধারণ কারণ `python3` -এর পথ বা `cd` ভুল। `git status` -এ reports দেখা গেলে `.gitignore` commit হয়েছে কি না দেখুন — ignore নিয়ম শুধু untracked ফাইলে কাজ করে।

WARNING: পরীক্ষার cron entry মুছে ফেলতে ভুলবেন না। এই toolkit-টি শেখার জন্য; production-এ লাগানোর আগে Project 03-এর architecture, logging ও test অংশগুলো যোগ করুন।

Cleanup: Crontab থেকে পরীক্ষার লাইন সরান; `cd ~ && rm -rf ~/labs/lab24` ।

পরিশিষ্ট B • Independent Assignments — স্বাধীন অনুশীলন

Appendix B: ১৫টি Independent Assignment

এই assignment-গুলো স্বাধীনভাবে করার জন্য। প্রতিটি assignment-এ কাজটি, প্রসঙ্গ ও গ্রহণযোগ্যতার মানদণ্ড (acceptance criteria) দেওয়া হয়েছে — সম্পূর্ণ সমাধান এখানে দেওয়া হয়নি; সমাধান ও ব্যাখ্যা আলাদা Answer Guide-এ আছে। কোনো assignment-এ আটকে গেলে সংশ্লিষ্ট অধ্যায় ও lab-এ ফিরে যান, তারপর আবার চেষ্টা করুন। সময়ের পরামর্শ: Beginner প্রতিটি ১–২ ঘণ্টা, Intermediate ২–৪ ঘণ্টা, Advanced ৪–৮ ঘণ্টা। সব কাজ নিজের পরীক্ষামূলক পরিবেশে করুন; production সিস্টেমে নয়।

> **CONCEPT:** Assignment-এর উদ্দেশ্য নিখুঁত উত্তর নয় — সিদ্ধান্ত নেওয়ার অভ্যাস। প্রতিটি কাজে লিখুন: আপনি কী ধরে নিয়েছেন, কেন এই পদ্ধতি বেছেছেন, আর কী ঝুঁকি রয়ে গেল। পেশাদার engineer-এর কাজের মূল্য এই ব্যাখ্যাতেই।

Assignment 01 — Beginner (Git) — Commit ইতিহাস পড়া ও ব্যাখ্যা করা

প্রসঙ্গ: একটি দলের repository-তে ২০টি commit আছে। একজন নতুন সদস্য হিসেবে আপনাকে ইতিহাস পড়ে বুঝতে হবে প্রকল্পটি কীভাবে এগিয়েছে।

কাজ: Lab 01–02-এর পদ্ধতিতে একটি পরীক্ষামূলক repository তৈরি করুন যাতে অন্তত ৮টি commit, ২টি branch ও ১টি merge থাকে। তারপর `git log --oneline --graph --all`, `git log -p -- <file>` ও `git blame` ব্যবহার করে একটি এক পৃষ্ঠার প্রতিবেদন লিখুন: কোন commit-এ কী বদলেছে, কোন branch কখন main-এ মিশেছে, আর কোন ফাইলটি সবচেয়ে বেশি বদলেছে।

Acceptance criteria: প্রতিবেদনে অন্তত ৫টি commit-এর উদ্দেশ্য সঠিকভাবে ব্যাখ্যা করা হয়েছে; merge commit-টি চিহ্নিত করা হয়েছে; সবচেয়ে বেশি বদলানো ফাইলটি `git log --stat` দিয়ে প্রমাণিত; কোনো অনুমানকে অনুমান হিসেবে লেখা হয়েছে।

Assignment 02 — Beginner (Git) — ভালো Commit বার্তা লেখা

প্রসঙ্গ: একটি repository-র ইতিহাসে `fix`, `update`, `changes` ধরনের অস্পষ্ট বার্তা ভর্তি। আপনাকে ইতিহাসটি পড়ে বুঝতে হবে কোন commit-এ কী হয়েছিল — কিন্তু বার্তা থেকে বোঝা যাচ্ছে না।

কাজ: ৬টি অস্পষ্ট বার্তাসহ একটি পরীক্ষামূলক repository তৈরি করুন। তারপর প্রতিটি commit-এর আসল পরিবর্তন (`git show` দিয়ে) দেখে Conventional Commits ধরনে (যেমন `feat: add disk usage check`, `fix: correct timeout value`) বার্তাগুলো নতুন করে লিখুন — একটি আলাদা নথিতে, মূল ইতিহাস না বদলে। শেষে লিখুন: কোন বার্তাগুলো সবচেয়ে খারাপ ছিল ও কেন।

Acceptance criteria: ৬টি বার্তার সবগুলোই নতুন ধরনে লেখা হয়েছে; প্রতিটি বার্তায় কী বদলেছে ও কেন, তা বোঝা যায়; খারাপ বার্তার সমালোচনায় অন্তত ৩টি সুনির্দিষ্ট কারণ আছে।

Assignment 03 — Beginner (Bash) — Code Reading: একটি স্ক্রিপ্টের আচরণ অনুমান করা

প্রসঙ্গ: নিচের স্ক্রিপ্টটি না চালিয়ে পড়ে বুঝতে হবে এটি কী করে।

```
#!/usr/bin/env bash
set -euo pipefail
count=0
for file in /tmp/data/*.txt; do
  if [[ ! -e "$file" ]]; then
    break
  fi
  if grep -q "ERROR" "$file"; then
    count=$((count + 1))
  fi
done
printf 'Files with ERROR: %d\n' "$count"
```

কাজ: (ক) স্ক্রিপ্টটি লাইন ধরে ধরে ব্যাখ্যা করুন — প্রতিটি লাইন কী করে ও কেন লেখা হয়েছে। (খ) `/tmp/data` ডিরেক্টরি না থাকলে কী হবে? (গ) `/tmp/data`-তে কোনো `.txt` ফাইল না থাকলে কী হবে? (ঘ) ফাইলের নামে space থাকলে কী হবে? প্রতিটি উত্তরের জন্য পরীক্ষা করে প্রমাণ দিন।

Acceptance criteria: চারটি প্রশ্নের উত্তরই সঠিক ও পরীক্ষা দিয়ে প্রমাণিত; ব্যাখ্যায় `set -euo pipefail`, glob, `grep -q` ও exit code-এর ভূমিকা আছে।

Assignment 04 — Beginner (Bash) — নিরাপদ Backup স্ক্রিপ্ট লেখা

প্রসঙ্গ: একটি ছোট দলের সদস্যরা প্রতিদিন একটি ডিরেক্টরি backup নিতে চায়, কিন্তু কেউ command line মনে রাখতে চায় না।

কাজ: একটি Bash স্ক্রিপ্ট লিখুন যা একটি source ডিরেক্টরি ও একটি গন্তব্য ডিরেক্টরি argument হিসেবে নেয়, তারিখসহ নামে `tar.gz` archive তৈরি করে, checksum লেখে ও যাচাই করে। Argument না দিলে usage বার্তা দেবে; source না থাকলে error দিয়ে থামবে; গন্তব্য না থাকলে তৈরি করবে। Lab 20-এর স্ক্রিপ্টকে ভিত্তি করে লিখুন, কিন্তু নিজের ভাষায় সাজান ও প্রতিটি function-এর উদ্দেশ্য একটি README-তে লিখুন।

Acceptance criteria: স্ক্রিপ্টটি `shellcheck`-এ clean; ভুল argument-এ সঠিক exit code (0/1/2) দেয়; checksum যাচাই করে; restore পরীক্ষা করে দেখানো হয়েছে; README-তে ব্যবহারের উদাহরণ আছে।

Assignment 05 — Beginner (Python) — Code Reading: একটি Automation স্ক্রিপ্টের ভুল খুঁজে বের করা

প্রসঙ্গ: নিচের Python কোডটি একটি ডিরেক্টরির সব `.log` ফাইল মুছে ফেলার কথা, কিন্তু এতে তিনটি গুরুতর সমস্যা আছে।

```
import os
import shutil
def clean(folder):
    for name in os.listdir(folder):
        path = os.path.join(folder, name)
        if name.endswith(".log"):
            os.remove(path)
clean("/var/log")
```

কাজ: (ক) তিনটি সমস্যা চিহ্নিত করুন ও প্রতিটির পরিণতি ব্যাখ্যা করুন। (খ) কোডটি নিরাপদভাবে নতুন করে লিখুন: শুধু একটি নির্দিষ্ট পরীক্ষামূলক ডিরেক্টরিতে কাজ করবে, মুছে ফেলার আগে dry-run mode-এ তালিকা দেখাবে, ব্যবহারকারীর

নিশ্চিতকরণ চাইবে, আর প্রতিটি কাজ log করবে। (গ) ব্যাখ্যা করুন কেন `/var/log`-এর মতো সিস্টেম ডিরেক্টরিতে এমন স্ক্রিপ্ট চালানো বিপজ্জনক।

Acceptance criteria: তিনটি সমস্যাই (যেমন: কোনো বয়স পরীক্ষা নেই, কোনো নিশ্চিতকরণ নেই, সিস্টেম ডিরেক্টরিতে চালানো যায়) চিহ্নিত করা হয়েছে; নতুন কোডে dry-run, নিশ্চিতকরণ ও logging আছে; কোডটি শুধু পরীক্ষামূলক ডিরেক্টরিতে চালিয়ে প্রমাণ করা হয়েছে।

Assignment 06 — Intermediate (Git) — Conflict-সহ Collaboration অনুকরণ

প্রসঙ্গ: আপনি ও একজন সহকর্মী (দুটো আলাদা clone হিসেবে অনুকরণ করবেন) একই ফাইলের একই অংশ বদলেছেন। দুজনের কাজই দরকারি।

কাজ: একটি bare repository তৈরি করে (`git init --bare`) সেটিকে remote হিসেবে ব্যবহার করুন। দুটি আলাদা clone-এ একই ফাইলের একই লাইন আলাদাভাবে বদলান, দুজনেই push করার চেষ্টা করুন। দ্বিতীয়জনের push ব্যর্থ হবে — pull/merge করে conflict সমাধান করুন, দুই পাশের যুক্তিসংগত অংশ রেখে, তারপর push করুন। পুরো প্রক্রিয়াটি ধাপে ধাপে একটি নথিতে লিখুন, প্রতিটি ধাপে কোন কমান্ড কী করল তা ব্যাখ্যাসহ।

Acceptance criteria: Bare repository remote হিসেবে কাজ করেছে; push ব্যর্থতার কারণ ব্যাখ্যা করা হয়েছে; conflict সমাধানে দুই পাশের পরিবর্তনই বিবেচনা করা হয়েছে; চূড়ান্ত ফাইলে কোনো marker নেই; নথিতে প্রতিটি ধাপের কমান্ড ও তার ফল আছে।

Assignment 07 — Intermediate (Bash) — Debugging: ভাঙা স্ক্রিপ্ট ঠিক করা

প্রসঙ্গ: একটি deploy স্ক্রিপ্ট মাঝে মাঝে ব্যর্থ হয় — কখনো ঠিক চলে, কখনো ভুল ফাইল কপি করে। কারণটি অস্পষ্ট।

```
#!/usr/bin/env bash
files=$(ls /tmp/build/*.txt)
for f in $files; do
  cp $f /tmp/deploy/
done
echo Done
```

কাজ: (ক) স্ক্রিপ্টটিতে অন্তত ৪টি সমস্যা খুঁজুন (quoting, word splitting, error handling, খালি glob ইত্যাদি)। (খ) ফাইলের নামে space, `/tmp/build` খালি থাকা, ও `cp` ব্যর্থ হওয়া — এই তিন পরিস্থিতিতে পরীক্ষা করে প্রতিটি সমস্যা প্রমাণ করুন। (গ) স্ক্রিপ্টটি নিরাপদভাবে নতুন করে লিখুন ও ShellCheck দিয়ে যাচাই করুন।

Acceptance criteria: ৪টি সমস্যাই চিহ্নিত ও পরীক্ষা দিয়ে প্রমাণিত; নতুন স্ক্রিপ্টে `set -euo pipefail`, quoting ও খালি-ডিরেক্টরি পরীক্ষা আছে; ShellCheck clean; তিনটি পরিস্থিতিতেই নতুন স্ক্রিপ্ট সঠিক আচরণ করে।

Assignment 08 — Intermediate (Bash) — Log বিশ্লেষণ টুল

প্রসঙ্গ: একটি সার্ভারের log ফাইলে প্রতিদিন হাজার হাজার লাইন জমে। অপারেটর চায় একটি কমান্ডে জানতে: মোট request, error-এর সংখ্যা, সবচেয়ে বেশি request করা ৫টি IP, সবচেয়ে ধীর ৫টি URL (যদি log-এ সময় থাকে)।

কাজ: Lab 10-এর ভিত্তিতে একটি পূর্ণাঙ্গ log বিশ্লেষণ স্ক্রিপ্ট লিখুন। এটি argument হিসেবে log ফাইলের পথ নেবে, ফাইল না থাকলে error দেবে, আর একটি সাজানো রিপোর্ট ছাপবে। নিজের তৈরি করা অন্তত ৫০ লাইনের একটি নমুনা log দিয়ে পরীক্ষা করুন ও ফল হাতে গুনে যাচাই করুন। স্ক্রিপ্টে function ব্যবহার করুন ও প্রতিটি function-এর কাজ README-তে লিখুন।

Acceptance criteria: রিপোর্টে ৪টি তথ্যই সঠিক; ৫০ লাইনের নমুনায় হাতে গোনা সংখ্যার সঙ্গে মিলেছে; ভুল ফাইল পথে সঠিক error ও exit code; ShellCheck clean; README-তে ব্যবহারের উদাহরণ আছে।

Assignment 09 — Intermediate (Python) — API থেকে রিপোর্ট তৈরি

প্রসঙ্গ: GitHub API থেকে একটি সংস্থার (organization) কয়েকটি public repository-র তথ্য সংগ্রহ করে একটি সাপ্তাহিক রিপোর্ট তৈরি করতে হবে।

কাজ: Lab 15-এর ভিত্তিতে একটি Python প্রোগ্রাম লিখুন যা একটি YAML configuration ফাইল থেকে repository-র তালিকা পড়ে (`owner/repo` ধরনে), প্রতিটির জন্য GitHub API ডাকে (timeout, status code পরীক্ষা, rate-limit বার্তা সামলে), তথ্য সংগ্রহ করে একটি JSON রিপোর্ট লেখে ও console-এ সারাংশ ছাপে। একটি repository না পাওয়া গেলে প্রোগ্রামটি থামবে না — সেটি রিপোর্টে `not found` হিসেবে লিখে বাকিগুলো চালিয়ে যাবে। Structured logging যোগ করুন।

Acceptance criteria: YAML থেকে অন্তত ৩টি repository পড়া হয়েছে; একটি ভুল নাম দিয়ে not-found আচরণ পরীক্ষা করা হয়েছে; rate-limit (403) বার্তা সামলানো হয়েছে; JSON রিপোর্ট বৈধ; কোনো token কোডে লেখা হয়নি; test-এ অন্তত ২টি function-এর pytest লেখা হয়েছে।

Assignment 10 — Intermediate (Python) — Configuration Validation ডিজাইন

প্রসঙ্গ: একটি automation টুল ভুল configuration পেলে মাঝপথে ব্যর্থ হয় — তখন অর্ধেক কাজ হয়ে গেছে, অর্ধেক হয়নি। সমাধান: কাজ শুরুর আগেই পুরো configuration যাচাই করা।

কাজ: একটি YAML configuration schema ডিজাইন করুন একটি backup টুলের জন্য: source ডিরেক্টরি, গন্তব্য, retention দিন, বাদ দেওয়ার pattern-এর তালিকা। একটি Python validator লিখুন যা প্রতিটি ক্ষেত্রের অস্তিত্ব, ধরন (string/list/int), মানের সীমা (যেমন retention 1–90 দিন) ও পথের অস্তিত্ব পরীক্ষা করে; ভুল থাকলে সব ভুল একসঙ্গে একটি তালিকায় জানাবে (প্রথম ভুলেই থামবে না)। অন্তত ৫টি ভিন্ন ভুল configuration দিয়ে পরীক্ষা করুন।

Acceptance criteria: Schema নথিতে লেখা হয়েছে; validator সব ভুল একসঙ্গে জানায়; ৫টি ভুল configuration-এর প্রতিটিতে সঠিক বার্তা আসে; বৈধ configuration পাস করে; pytest-এ অন্তত ৪টি test আছে।

Assignment 11 — Intermediate (Automation) — Cron + Health Check + Alert

প্রসঙ্গ: একটি সার্ভারের disk ৯০% ছাড়ালে কেউ জানতে পারে না, যতক্ষণ না সার্ভার থেমে যায়।

কাজ: Lab 18 ও Lab 21 একত্রিত করুন: একটি health-check স্ক্রিপ্ট লিখুন যা disk, memory ও একটি গুরুত্বপূর্ণ সার্ভিস পরীক্ষা করে, ফল একটি log ফাইলে লেখে, আর CRITICAL হলে একটি alert ফাইল (`alerts/` ডিরেক্টরিতে timestamp-সহ) তৈরি করে। Cron-এ প্রতি ১৫ মিনিটে চালানোর entry লিখুন (নথিতে, পরীক্ষার জন্য প্রতি মিনিটে চালিয়ে যাচাই করুন)। `flock` ব্যবহার করুন। একটি পরীক্ষা লিখুন যাতে threshold বদলে CRITICAL অবস্থা তৈরি করে alert ফাইল তৈরি হওয়া প্রমাণ করা যায়।

Acceptance criteria: স্বাভাবিক অবস্থায় alert ফাইল তৈরি হয় না; threshold কমালে alert ফাইল তৈরি হয়; log-এ প্রতিটি চালনার রেকর্ড আছে; cron entry সঠিক syntax-এ লেখা; `flock` ব্যবহার করা হয়েছে; পরীক্ষার cron entry শেষে সরানো হয়েছে।

Assignment 12 — Advanced (Git + Python) — Repository স্বাস্থ্য পরীক্ষক

প্রসঙ্গ: একটি দলের অনেকগুলো repository-তে বড় ফাইল ভুল করে commit করা হয়েছে, `.gitignore` নেই, আর commit বার্তা অস্পষ্ট। একটি টুল চাই যা repository পরীক্ষা করে সমস্যা জানাবে।

কাজ: একটি Python CLI টুল লিখুন যা একটি Git repository-র পথ নিয়ে পরীক্ষা করে: (ক) `.gitignore` আছে কি না; (খ) ১ MB-এর বড় ফাইল commit করা হয়েছে কি না (`git ls-files` ও আকার যাচাই); (গ) সর্বশেষ ১০টি commit বার্তার দৈর্ঘ্য (১০ অক্ষরের কম বার্তা সতর্কতা); (ঘ) uncommitted পরিবর্তন আছে কি না। ফল JSON-এ লিখুন ও console-এ সারাংশ ছাপুন। Git কমান্ডগুলো `subprocess`-এ argument list দিয়ে চালান (`shell=True` নয়)। অন্তত ৩টি ভিন্ন পরীক্ষামূলক repository দিয়ে যাচাই করুন ও pytest লিখুন।

Acceptance criteria: ৪টি পরীক্ষাই কাজ করে; বড় ফাইল ও ছোট বার্তা সঠিকভাবে ধরা পড়ে; JSON রিপোর্ট বৈধ; `shell=True` কোথাও ব্যবহার করা হয়নি; pytest-এ অন্তত ৫টি test পাস করে; README আছে।

Assignment 13 — Advanced (Bash + SSH) — বহু সার্ভারে নিরাপদ Remote কাজ

প্রসঙ্গ: TechNova-র ৩টি staging সার্ভারে একই health-check চালাতে হবে। প্রতিটি সার্ভারে আলাদাভাবে login করে চালানো সময়সাপেক্ষ ও ভুলপ্রবণ।

কাজ: একটি Bash স্ক্রিপ্ট লিখুন যা একটি সার্ভার তালিকা ফাইল (প্রতি লাইনে একটি hostname) পড়ে, প্রতিটি সার্ভারে SSH দিয়ে একটি নির্দিষ্ট কমান্ড চালায় (যেমন `df -h / | tail -n 1`), ফল সংগ্রহ করে একটি রিপোর্টে লেখে, আর কোনো সার্ভারে সংযোগ ব্যর্থ হলে সেটি রিপোর্টে `UNREACHABLE` হিসেবে লিখে বাকিগুলো চালিয়ে যায়। Host key যাচাই বন্ধ করবেন না (`StrictHostKeyChecking=no` নিষিদ্ধ)। localhost ও একটি ভুল hostname দিয়ে পরীক্ষা করুন। নিরাপত্তা বিশ্লেষণ লিখুন: এই স্ক্রিপ্টে কী ঝুঁকি আছে ও কীভাবে কমানো যায়।

Acceptance criteria: localhost-এ সফলভাবে চলে; ভুল hostname-এ থামে না, রিপোর্টে UNREACHABLE লেখে; host key যাচাই বন্ধ করা হয়নি; timeout ব্যবহার করা হয়েছে; নিরাপত্তা বিশ্লেষণে অন্তত ৩টি ঝুঁকি ও প্রতিকার আছে; ShellCheck clean।

Assignment 14 — Advanced (Python) — Idempotent Automation ডিজাইন

প্রসঙ্গ: একটি setup স্ক্রিপ্ট দ্বিতীয়বার চালালে duplicate user তৈরি করে, একই লাইন configuration ফাইলে দুবার যোগ করে, আর একই ডিরেক্টরি দুবার তৈরি করার চেষ্টা করে ব্যর্থ হয়।

কাজ: একটি Python টুল লিখুন যা একটি সার্ভার সেটআপের তিনটি কাজ idempotent-ভাবে করে: (ক) একটি ডিরেক্টরি কাঠামো তৈরি (না থাকলে তৈরি, থাকলে কিছু না করা); (খ) একটি configuration ফাইলে একটি লাইন যোগ করা (লাইনটি থাকলে আবার যোগ না করা); (গ) একটি JSON state ফাইলে কাজের রেকর্ড রাখা (কী করা হয়েছে, কখন)। টুলটি দুবার চালালে দ্বিতীয়বার কোনো পরিবর্তন হবে না — এটি পরীক্ষা দিয়ে প্রমাণ করুন (ফাইলের checksum দুবার চালানোর পর একই থাকবে)। Dry-run mode যোগ করুন। Architecture সিদ্ধান্ত নথিতে লিখুন: কেন এই পদ্ধতি বেছে নিয়েছেন, আর কোন ক্ষেত্রে idempotency ভাঙতে পারে।

Acceptance criteria: দুবার চালানোর পর checksum অপরিবর্তিত; dry-run কোনো ফাইল বদলায় না; state ফাইলে প্রতিটি কাজের রেকর্ড আছে; pytest-এ অন্তত ৫টি test; architecture নথিতে অন্তত ৩টি সিদ্ধান্তের যুক্তি আছে।

Assignment 15 — Advanced (Architecture) — TechNova-র Automation নকশা

প্রসঙ্গ: TechNova-র ৫টি সার্ভার, ৩টি পরিবেশ (dev, staging, production), ২ জন operator। এখন প্রতিটি কাজ হাতে করা হয়। আপনাকে একটি automation architecture প্রস্তাব করতে হবে — কোড লেখার আগে নকশা।

কাজ: একটি ৩–৫ পৃষ্ঠার নকশা নথি লিখুন: (ক) কোন কাজ Bash-এ, কোনটি Python-এ করবেন ও কেন (সিদ্ধান্তের যুক্তিসহ); (খ) Git repository কাঠামো (branch strategy, code review নিয়ম); (গ) Configuration ও secret ব্যবস্থাপনা (কোথায় রাখবেন, কোথায় রাখবেন না); (ঘ) Scheduling (cron বনাম systemd timer) ও log ব্যবস্থাপনা; (ঙ) ব্যর্থতা সামলানো (retry, alert, rollback ধারণা); (চ) নিরাপত্তা ঝুঁকি ও প্রতিকার; (ছ) প্রথম ৪ সপ্তাহের বাস্তবায়ন পরিকল্পনা (milestone-সহ)। একটি ছোট proof-of-concept (একটি health-check + একটি cron entry + একটি রিপোর্ট) দিয়ে নকশার একটি অংশ প্রমাণ করুন।

Acceptance criteria: নথিতে ৭টি বিষয়ই আছে; Bash বনাম Python সিদ্ধান্তে অন্তত ৪টি যুক্তি আছে; secret ব্যবস্থাপনায় কোডে গোপন তথ্য না রাখার নিয়ম স্পষ্ট; ৪ সপ্তাহের পরিকল্পনায় milestone ও গ্রহণযোগ্যতার মানদণ্ড আছে; proof-of-concept চলে ও তার ফল নথিতে উল্লেখ আছে।

> **CHECK YOUR UNDERSTANDING:** সব assignment শেষ করার পর নিজেকে জিজ্ঞেস করুন: আমি কি প্রতিটি কাজের জন্য সঠিক টুল বেছেছি? ব্যর্থতা সামলেছি কি? গোপন তথ্য কোথাও ফাঁস করিনি তো? এই তিন প্রশ্নের উত্তর হ্যাঁ হলে আপনি production-এর দিকে এগোচ্ছেন।

পরিশিষ্ট C • Major Projects — প্রধান প্রকল্প

Appendix C: তিনটি Major Project

এই তিনটি প্রকল্প বইয়ের সব শেখা একত্রিত করে। প্রতিটি প্রকল্প একটি বাস্তব পণ্যের মতো করে তৈরি করতে হবে: কাজ করে, পরীক্ষা করা যায়, নথি আছে, আর অন্য কেউ চালাতে পারে। প্রকল্প জমা দেওয়ার আগে নিজের কাজটি এই তালিকায় যাচাই করুন: README আছে কি? ভুল input-এ নিরাপদে ব্যর্থ হয় কি? গোপন তথ্য কোথাও নেই তো? দুবার চালালে একই ফল আসে কি? Test পাস করে কি?

Project 01 — Linux Server Maintenance Toolkit (Bash)

বাস্তব দৃশ্য: TechNova-র operator-রা প্রতিদিন সকালে পাঁচটি সার্ভারে login করে `df`, `free`, `systemctl` ও `journalctl` আলাদাভাবে চালায়। কখনো একটি সার্ভার বাদ পড়ে, কখনো ফল লেখা হয় না। একটি একক Bash toolkit চাই যা এক কমান্ডে পুরো স্বাস্থ্য পরীক্ষা করে একটি পরিষ্কার রিপোর্ট দেবে।

উদ্দেশ্য ও প্রয়োজনীয়তা

Toolkit-টি করবে: CPU inspection (load average ও সবচেয়ে বেশি CPU ব্যবহার করা ৫টি process), memory inspection (মোট, ব্যবহৃত, উপলব্ধ ও শতাংশ), disk monitoring (প্রতিটি mount-এর ব্যবহার, threshold-এর বেশি হলে সতর্কতা), service status (একটি configuration ফাইলে তালিকাভুক্ত সার্ভিসগুলোর অবস্থা), log inspection (সাম্প্রতিক error লাইনগুলোর সারাংশ), ও health reporting (OK/WARNING/CRITICAL সারাংশ ও exit code)।

কারিগরি প্রয়োজনীয়তা

- **Modular functions:** প্রতিটি পরীক্ষা আলাদা function — `check_cpu`, `check_memory`, `check_disk`, `check_services`, `check_logs`, `print_report`। কোনো function ৩০ লাইনের বেশি নয়।
- **Input validation:** Threshold ও configuration ফাইলের পথ argument হিসেবে নেওয়া যাবে; ভুল মান (যেমন disk threshold 150) দিলে স্পষ্ট error দিয়ে থামবে।
- **Safe execution:** `set -euo pipefail`, সব variable quote করা, ফাইলের নামে space সামলানো, `eval` ব্যবহার না করা।
- **Error handling:** কোনো পরীক্ষা ব্যর্থ হলে (যেমন সার্ভিস নেই) toolkit থামবে না — সেই পরীক্ষাটি `UNKNOWN` হিসেবে রিপোর্ট করে বাকিগুলো চালিয়ে যাবে।
- **Clear output:** মানুষের পড়ার মতো রিপোর্ট ও মেশিনের পড়ার মতো `--format json` আউটপুট — দুটোই।
- **Documentation:** README-তে ইনস্টল, ব্যবহার, প্রতিটি option, exit code (0/1/2) ও উদাহরণ।

Repository কাঠামো

```
server-maintenance-toolkit/
  README.md
  toolkit.sh
  lib/
    checks.sh
    report.sh
  config/
    services.conf
  tests/
    test_toolkit.sh
```

```
  docs/
    USAGE.md
```

মূল স্ক্রিপ্টের কাঠামো (উদাহরণ)

```
#!/usr/bin/env bash
set -euo pipefail
script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$script_dir/lib/checks.sh"
source "$script_dir/lib/report.sh"
warn_disk="${WARN_DISK:-80}"
crit_disk="${CRIT_DISK:-90}"
if [[ "$warn_disk" -ge "$crit_disk" ]]; then
  printf 'Error: WARN_DISK must be less than CRIT_DISK.\n' >&2
  exit 2
fi
```

এই অংশটি দেখায়: library sourcing, পরিবেশ চলক থেকে threshold নেওয়া (default-সহ), ও শুরুতেই validation। বাকি অংশ — প্রতিটি check function ও রিপোর্ট — আপনাকেই লিখতে হবে। `services.conf`-এ প্রতি লাইনে একটি সার্ভিসের নাম থাকবে; খালি লাইন ও `#` দিয়ে শুরু হওয়া মন্তব্য উপেক্ষা করতে হবে।

Milestone

Milestone	কাজ	গ্রহণযোগ্যতা
M1 — ভিত্তি	Repository, README, lib কাঠামো, CPU/memory/disk check	তিনটি check হাতে চালিয়ে সঠিক ফল দেয়
M2 — সার্ভিস ও লগ	Service check, log inspection, threshold validation	বন্ধ সার্ভিস CRITICAL হিসেবে ধরা পড়ে
M3 — রিপোর্ট	Text ও JSON রিপোর্ট, exit code 0/1/2	দুটি format-ই সঠিক, exit code মিলে
M4 — গুণমান	ShellCheck clean, test স্ক্রিপ্ট, USAGE নথি	ShellCheck-এ কোনো সতর্কতা নেই, test পাস করে

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- `./toolkit.sh` চালালে ছয়টি পরীক্ষার সবগুলোর ফলসহ রিপোর্ট আসে।
- `./toolkit.sh --format json` বৈধ JSON দেয়।
- Threshold ভুল দিলে exit code 2 ও স্পষ্ট বার্তা আসে।
- একটি সার্ভিস বন্ধ থাকলে রিপোর্টে CRITICAL ও সামগ্রিক exit code 2 আসে, কিন্তু toolkit থামে না।
- ShellCheck clean; `bash -n` পাস করে; test স্ক্রিপ্টে অন্তত ৫টি পরীক্ষা পাস করে।
- README পড়ে একজন নতুন operator toolkit চালাতে পারে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ

কার্যকারিতা	35%	ছয়টি পরীক্ষাই সঠিক, threshold কাজ করে
নিরাপত্তা ও নির্ভরযোগ্যতা	25%	Quoting, validation, error handling, ShellCheck clean
কোডের গঠন	20%	Function-ভিত্তিক, পুনঃব্যবহারযোগ্য, পড়ার মতো
নথি ও পরীক্ষা	20%	README সম্পূর্ণ, test আছে ও পাস করে

WARNING: Toolkit-টি শুধু পড়া (read-only) কাজ করবে — কোনো সার্ভিস restart, ফাইল মোছা বা configuration বদলানো যাবে না। পরিবর্তনকারী কাজ আলাদা টুলের বিষয়, আলাদা অনুমতি নিয়ে।

PRO TIP: Lab 21-এর health-check স্ক্রিপ্টকে ভিত্তি করে শুরু করুন, তারপর function-এ ভাগ করুন। প্রথমে একটি check নিখুঁত করুন, তারপর পরেরটি যোগ করুন।

Project 02 — Python Infrastructure Automation Tool (CLI Application)

বাস্তব দৃশ্য: TechNova-র operator-রা চায় একটি একক Python টুল: configuration পড়বে, সিস্টেমের তথ্য সংগ্রহ করবে, একটি HTTP API থেকে তথ্য আনবে, স্বাস্থ্য পরীক্ষা করবে, backup রিপোর্ট তৈরি করবে — আর সবকিছু structured log-এ লিখবে। Bash-এ এই জটিলতা সামলানো কঠিন; Python-এ এটি একটি রক্ষণাবেক্ষণযোগ্য application হবে।

উদ্দেশ্য ও প্রয়োজনীয়তা

টুলটি করবে: YAML configuration পড়া ও যাচাই করা; সিস্টেম তথ্য সংগ্রহ (platform, disk, memory — Lab 14-এর subprocess প্যাটার্নে, `shell=False`); একটি HTTP API ডাকা (timeout, retry, status code সামলে — Lab 15-এর প্যাটার্নে); JSON প্রক্রিয়াকরণ; সিস্টেম স্বাস্থ্য পরীক্ষা (threshold অনুযায়ী OK/WARNING/CRITICAL); backup রিপোর্ট তৈরি (Lab 20-এর ধারণা, Python-এ); structured logging (Lab 17-এর প্যাটার্নে, console + rotating file); ব্যর্থতা নিরাপদে সামলানো (কোনো একটি ধাপ ব্যর্থ হলে বাকিগুলো চলবে, চূড়ান্ত exit code সঠিক হবে)।

কারিগরি প্রয়োজনীয়তা

- **Modular source code:** একটিমাত্র বড় ফাইল নয় — `config.py`, `system_info.py`, `api_client.py`, `health.py`, `backup.py`, `cli.py` আলাদা মডিউল।
- **Virtual environment:** `python3 -m venv .venv`, `requirements.txt`-এ `requests` ও `pyyaml` (সংস্করণসহ), `pip install -r requirements.txt` দিয়ে ইনস্টল।
- **Configuration validation:** কাজ শুরুর আগে পুরো configuration যাচাই (Assignment 10-এর ধারণা) — ভুল থাকলে সব ভুল একসঙ্গে জানাবে।
- **Tests:** pytest-এ অন্তত ১০টি test — configuration validation, health threshold, API error handling (mock দিয়ে), CLI dry-run।
- **README:** ইনস্টল, configuration উদাহরণ, প্রতিটি কমান্ড, exit code, log ফাইলের অবস্থান, সমস্যা সমাধান।

Repository কাঠামো

```
infra-automation-tool/
  README.md
  requirements.txt
```

```
  config.example.yaml
  src/
    __init__.py
    cli.py
    config.py
    system_info.py
    api_client.py
    health.py
    backup.py
    logging_setup.py
  tests/
    test_config.py
    test_health.py
    test_api_client.py
    test_cli.py
  docs/
    ARCHITECTURE.md
```

Configuration উদাহরণ

```yaml
environment: staging
disk_warn_percent: 80
disk_crit_percent: 90
api:
  base_url: https://api.github.com
  timeout_seconds: 10
backup:
  source_dir: /home/nahid/labs/project02-data
  dest_dir: /home/nahid/labs/project02-backups
  retention_days: 7
services:
  - ssh
  - cron
```

Milestone

Milestone	কাজ	গ্রহণযোগ্যতা
M1 — ভিত্তি	venv, requirements, config loading + validation, logging	ভুল config-এ সব ভুল একসঙ্গে জানায়
M2 — সংগ্রহ	System info, API client, health check	API ব্যর্থ হলেও health রিপোর্ট আসে
M3 — রিপোর্ট	Backup রিপোর্ট, JSON আউটপুট, CLI (dry-run সহ)	dry-run কোনো ফাইল লেখে না
M4 — গুণমান	10+ test, README, ARCHITECTURE নথি	সব test পাস করে, নতুন ব্যবহারকারী চালাতে পারে

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- `python -m src.cli --config config.example.yaml --output report.json` চালালে বৈধ JSON রিপোর্ট তৈরি হয়।

- API সার্ভার না পাওয়া গেলে (ভুল URL দিয়ে পরীক্ষা) টুলটি traceback ছাড়া সুন্দর error দিয়ে বাকি ধাপগুলো চালিয়ে যায়।
- ভুল configuration-এ exit code 2 ও সব ভুলের তালিকা আসে।
- `--dry-run` কোনো ফাইল তৈরি করে না।
- কোনো গোপন তথ্য কোডে বা configuration উদাহরণে নেই; token লাগলে পরিবেশ চলক থেকে পড়া হয়।
- pytest-এ অন্তত ১০টি test পাস করে; `python -m pytest` চালিয়ে প্রমাণ করা হয়েছে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ
কার্যকারিতা	30%	সব ধাপ কাজ করে, ব্যর্থতা সামলানো হয়
কোডের গঠন	25%	মডিউল আলাদা, function ছোট, নাম স্পষ্ট
নিরাপত্তা	20%	shell=False, timeout, গোপন তথ্য নেই, validation আছে
পরীক্ষা ও নথি	25%	10+ test পাস করে, README ও ARCHITECTURE সম্পূর্ণ

> **REAL WORLD:** এই প্রকল্পের কাঠামোটি একটি আসল production টুলের ক্ষুদ্র সংস্করণ। চাকরির সাক্ষাৎকারে এই repository-টি দেখাতে পারবেন — এটি প্রমাণ করে আপনি শুধু স্ক্রিপ্ট লিখতে পারেন না, একটি রক্ষণাবেক্ষণযোগ্য application তৈরি করতে পারেন।

ঐচ্ছিক Advanced Challenge

- API ডাকে exponential backoff-সহ retry যোগ করুন (১ সেকেন্ড, ২ সেকেন্ড, ৪ সেকেন্ড)।
- রিপোর্টে আগের চালনার সঙ্গে তুলনা যোগ করুন (disk ব্যবহার বেড়েছে না কমেছে)।
- GitHub Actions বা অন্য CI-তে pytest চালানোর configuration যোগ করুন।

Project 03 — TechNova Automation Toolkit

বাস্তব দৃশ্য: TechNova এখন বেশ কয়েকটি Linux সার্ভার পরিচালনা করে — web, database ও staging। প্রতিদিনের কাজগুলো — স্বাস্থ্য পরীক্ষা, disk ও memory রিপোর্ট, সার্ভিসের অবস্থা, log সংগ্রহ, backup ও যাচাই, remote কাজ, API থেকে তথ্য আনা, নির্ধারিত চালনা — সব হাতে করা হয়। একজন operator ছুটিতে গেলে কাজ থেমে যায়। TechNova চায় একটি পুনঃব্যবহারযোগ্য toolkit যা এই সব কাজ স্বয়ংক্রিয় করবে, Git দিয়ে পরিচালিত হবে, পরীক্ষা করা যাবে, আর নতুন operator শিখতে পারবে।

Architecture

Toolkit-টি তিনটি স্তরে ভাগ করা:

TechNova Automation Toolkit — Architecture

স্তর ১: Interface — CLI (Python argparse) + Bash wrapper স্ক্রিপ্ট; operator এখানে কমান্ড দেয়

স্তর ২: Core — health, backup, logs, remote, api মডিউল (Python); প্রতিটি মডিউল একটিই দায়িত্ব পালন করে

স্তর ৩: Execution — subprocess (shell=False) দিয়ে Linux কমান্ড; SSH দিয়ে remote সার্ভার; requests দিয়ে API

পাশে: Configuration (YAML + পরিবেশ চলক), Logging (structured, rotating file), Scheduling (cron + flock), Storage (JSON রিপোর্ট, backup archive)

প্রবাহ: CLI → Configuration validation → Core মডিউল → Execution → JSON রিপোর্ট + Log → Exit code (0/1/2)

এই ভাগের যুক্তি: Interface বদলালে Core বদলাতে হয় না; একটি মডিউল ভাঙলে বাকিগুলো চলে; প্রতিটি মডিউল আলাদাভাবে পরীক্ষা করা যায়। Bash শুধু সেখানে ব্যবহার করা হবে যেখানে তা স্বাভাবিক — ছোট system check ও cron wrapper; জটিল যুক্তি, API ও রিপোর্ট Python-এ।

Requirements

কার্যকরী প্রয়োজনীয়তা

- **Git-based source management:** পুরো toolkit একটি Git repository-তে; `main` branch সুরক্ষিত (সরাসরি commit নয়, pull request ও review-এর মাধ্যমে পরিবর্তন); প্রতিটি release-এ tag (যেমন `v1.0.0`); Semantic Versioning।
- **Server health monitoring:** CPU load, memory, disk — প্রতিটি সার্ভারে, threshold-সহ।
- **Disk and memory reports:** দৈনিক JSON রিপোর্ট, আগের দিনের সঙ্গে তুলনা।
- **Service status checks:** Configuration-এ তালিকাভুক্ত সার্ভিসগুলোর অবস্থা।
- **Log collection:** প্রতিটি সার্ভারের সাম্প্রতিক error লাইন সংগ্রহ করে কেন্দ্রীয় রিপোর্টে।
- **Backup and verification:** নির্দিষ্ট ডিরেক্টরির backup, checksum যাচাই, retention নীতি।
- **Remote SSH operations:** Key-based SSH দিয়ে একাধিক সার্ভারে একই পরীক্ষা চালানো; host key যাচাই বন্ধ করা যাবে না।
- **API integration:** অন্তত একটি public API থেকে তথ্য সংগ্রহ (যেমন GitHub repository তথ্য), timeout ও rate-limit সামলে।
- **Scheduled tasks:** cron দিয়ে দৈনিক health-check ও সাপ্তাহিক backup; `flock` দিয়ে duplicate চালনা ঠেকানো।
- **Error handling:** একটি সার্ভার বা ধাপ ব্যর্থ হলে বাকিগুলো চলবে; চূড়ান্ত রিপোর্টে ব্যর্থতা স্পষ্ট থাকবে।
- **Structured logging:** timestamp, level, মডিউলের নামসহ log; console-এ সংক্ষিপ্ত, ফাইলে বিস্তারিত; rotating file।
- **Automated tests:** pytest-এ অন্তত ১৫টি test; mock দিয়ে SSH ও API অনুকরণ।
- **Documentation:** README, ARCHITECTURE, RUNBOOK (সমস্যা হলে কী করতে হবে), CHANGELOG।

অকার্যকরী প্রয়োজনীয়তা

- **নিরাপত্তা:** কোডে কোনো গোপন তথ্য নেই; secret পরিবেশ চলক থেকে আসে; `shell=True` নিষিদ্ধ; সব input যাচাই করা হয়; log-এ গোপন তথ্য লেখা হয় না।

- **Idempotency:** একই কমান্ড দুবার চালালে দ্বিতীয়বার কোনো ক্ষতিকর পরিবর্তন হয় না।
- **পর্যবেক্ষণযোগ্যতা:** প্রতিটি চালনার exit code, log ও JSON রিপোর্ট থেকে বোঝা যায় কী হয়েছে।
- **রক্ষণাবেক্ষণযোগ্যতা:** নতুন সার্ভার যোগ করা মানে শুধু configuration ফাইলে একটি entry যোগ করা, কোড বদলানো নয়।

Repository কাঠামো

```
technova-automation-toolkit/
  README.md
  ARCHITECTURE.md
  RUNBOOK.md
  CHANGELOG.md
  requirements.txt
  config/
    config.example.yaml
    servers.example.yaml
    services.conf
  src/
    __init__.py
    cli.py
    config.py
    health.py
    backup.py
    logs_collector.py
    remote.py
    api_client.py
    reporting.py
    logging_setup.py
  scripts/
    healthcheck.sh
    install-cron.sh
  tests/
    test_config.py
    test_health.py
    test_backup.py
    test_remote.py
    test_api_client.py
    test_cli.py
  docs/
    OPERATOR-GUIDE.md
```

Milestones (৮ সপ্তাহের পরিকল্পনা)

সপ্তাহ	Milestone	কাজ	গ্রহণযোগ্যতা
১	M1 — ভিত্তি	Repository, branch নিয়ম, config schema, logging, CLI কঙ্কাল	CLI --help চলে; ভুল config-এ সব ভুল জানায়
২–৩	M2 — Core: health ও backup	Health check, disk/memory রিপোর্ট, backup + checksum + retention	দুটি সার্ভারে (localhost + VM) health রিপোর্ট আসে; backup restore পরীক্ষা পাস করে
৪	M3 — Remote ও logs	SSH remote execution, log collection	ভুল hostname-এ থামে না; UNREACHABLE রিপোর্টে আসে

৫	M4 — API ও reporting	API client, JSON রিপোর্ট, আগের দিনের সঙ্গে তুলনা	API ব্যর্থ হলেও রিপোর্ট আসে; তুলনা সঠিক
৬	M5 — Scheduling	Cron entry, flock, install-cron.sh, log rotation	দুই দিন cron-এ চলে; duplicate চালনা হয় না
৭	M6 — Testing	15+ pytest, mock-সহ, ShellCheck, নথি সম্পূর্ণ	সব test পাস করে; নতুন operator README পড়ে চালাতে পারে
৮	M7 — Release	Code review, v1.0.0 tag, CHANGELOG, demo	Review-এর মন্তব্য ঠিক করা হয়েছে; tag দেওয়া হয়েছে

Deliverables — যা জমা দিতে হবে

1. Git repository (GitHub-এ public বা private — link-সহ), যেখানে সম্পূর্ণ ইতিহাস দেখা যায় (অন্তত ১৫টি অর্থপূর্ণ commit, feature branch-এ কাজ, pull request-এর মাধ্যমে merge)।
2. কাজ করা toolkit — README অনুযায়ী ইনস্টল করে চালানো যায়।
3. একটি নমুনা দৈনিক রিপোর্ট (JSON) ও একটি সাপ্তাহিক backup রিপোর্ট।
4. Test রিপোর্ট — `python -m pytest -v`-এর আউটপুট।
5. নথি: README, ARCHITECTURE, RUNBOOK, CHANGELOG, OPERATOR-GUIDE।
6. একটি ১০ মিনিটের demo: toolkit চালানো, একটি সার্ভার ব্যর্থ হওয়ার অনুকরণ, ও রিপোর্ট দেখানো (ভিডিও বা লিখিত walkthrough)।

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- Toolkit-টি অন্তত দুটি সার্ভারে (localhost + একটি VM বা দ্বিতীয় মেশিন) চলে।
- একটি সার্ভার unreachable হলে বাকিগুলোর রিপোর্ট আসে ও চূড়ান্ত exit code সঠিক হয়।
- Backup নেওয়া যায়, checksum যাচাই পাস করে, restore পরীক্ষা সফল হয়।
- Cron-এ দুই দিন চলে; log-এ প্রতিটি চালনার রেকর্ড আছে; duplicate চালনা হয় না।
- কোডে কোনো গোপন তথ্য, `shell=True` বা `StrictHostKeyChecking=no` নেই।
- pytest-এ অন্তত ১৫টি test পাস করে; ShellCheck clean।
- নতুন সার্ভার যোগ করা শুধু configuration বদলে করা যায় — প্রমাণসহ।
- RUNBOOK পড়ে একজন নতুন operator একটি সাধারণ সমস্যা (যেমন disk full) সামলাতে পারে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ
Architecture ও নকশা	20%	স্তর আলাদা, সিদ্ধান্তের যুক্তি নথিতে আছে
কার্যকারিতা	25%	সব প্রয়োজনীয়তা কাজ করে, দুটি সার্ভারে প্রমাণিত
নিরাপত্তা ও নির্ভরযোগ্যতা	20%	গোপন তথ্য নেই, ব্যর্থতা সামলানো হয়, idempotent
পরীক্ষা	15%	15+ test, mock ব্যবহার, সব পাস করে

নথি ও Git workflow	20%	নথি সম্পূর্ণ, branch/PR ইতিহাস অর্থপূর্ণ, release tag আছে

> **WARNING:** এই প্রকল্পে আসল production সার্ভার ব্যবহার করবেন না। সব পরীক্ষা নিজের VM বা staging পরিবেশে করুন। SSH key-তে passphrase দিন, আর private key কখনো repository-তে commit করবেন না — `.gitignore`-এ `*.pem`, `id_*` ও `.env` রাখুন।

ঐচ্ছিক Advanced Challenge

- systemd timer দিয়ে cron-এর বিকল্প বাস্তবায়ন করুন ও দুটির তুলনা লিখুন।
- রিপোর্টগুলো একটি সহজ HTML ড্যাশবোর্ডে দেখান (শুধু স্থানীয়ভাবে, কোনো সার্ভার ছাড়া)।
- Backup-এ encryption যোগ করুন (যেমন `gpg` দিয়ে) ও key ব্যবস্থাপনা নথিতে লিখুন।
- Toolkit-কে একটি Python package হিসেবে সাজান (`pyproject.toml` সহ) যাতে `pip install` করে চালানো যায়।
- একটি incident অনুকরণ করুন: disk পূর্ণ হওয়ার ভান করে RUNBOOK অনুযায়ী সমস্যা সমাধান করুন ও সময় মাপুন।

> **CHECK YOUR UNDERSTANDING:** তিনটি প্রকল্প শেষ করার পর আপনার কাছে থাকবে: একটি Bash toolkit, একটি Python CLI application, আর একটি সম্পূর্ণ automation system — এই তিনটি মিলেই একটি DevOps engineer-এর portfolio-র ভিত্তি। প্রতিটি প্রকল্পের repository link আপনার CV-তে যোগ করতে পারেন।

পরিশিষ্ট D • Quiz and Final Examination Answer Guide

Appendix D: Quiz and Final Examination Answer Guide — কুইজ ও চূড়ান্ত পরীক্ষার উত্তর-নির্দেশিকা

এই পরিশিষ্টে বইয়ের ২৪টি অধ্যায়ের সব কুইজের উত্তর দেওয়া হয়েছে — মোট ১২০টি MCQ। প্রতিটি প্রশ্নের সঙ্গে সঠিক উত্তরের ব্যাখ্যা এবং প্রতিটি ভুল বিকল্প কেন ভুল, তা বাংলায় বলা হয়েছে। শুধু উত্তর মুখস্থ করবেন না; ব্যাখ্যাটি পড়ে ধারণাটি বুঝুন। ভুল বিকল্পগুলো ইচ্ছাকৃতভাবে এমনভাবে লেখা হয়েছে, যাতে সাধারণ ভুল ধারণাগুলো ধরা পড়ে। পরীক্ষার পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। এই সংস্করণগুলোতে যাচাই করা আচরণই এখানে সঠিক বলে ধরা হয়েছে।

> **CONCEPT:** ভালো MCQ-র ভুল বিকল্পও শিক্ষণীয়। প্রতিটি ব্যাখ্যায় তিনটি প্রশ্নের উত্তর খুঁজুন — সঠিকটি কেন সঠিক, ভুলটি কেন ভুল, এবং বাস্তব কাজে পার্থক্যটি কী প্রভাব ফেলে।

D.1 অধ্যায় ১ — Introduction to Version Control Systems (প্রশ্ন ১.১–১.৫)

১.১ Version control-এর প্রধান উদ্দেশ্য কোনটি?
ক) ফাইল দ্রুত কপি করা খ) সময়ের সঙ্গে সঙ্গে পরিবর্তনের ইতিহাস সংরক্ষণ করা ও প্রয়োজনে আগের অবস্থায় ফেরা গ) ফাইল compress করা ঘ) ইন্টারনেট ছাড়া ফাইল শেয়ার করা
উত্তর: খ — version control প্রতিটি পরিবর্তন কে, কখন, কেন করল তা ইতিহাসসহ সংরক্ষণ করে, ফলে যেকোনো আগের অবস্থায় ফেরা যায়। ক ভুল, কারণ কপি করা তার কাজ নয়; গ ভুল, কারণ compression এর উদ্দেশ্য নয়; ঘ ভুল, কারণ শেয়ার করা গৌণ সুবিধা, মূল উদ্দেশ্য ইতিহাস সংরক্ষণ।

১.২ Manual backup (যেমন project-v1, project-v2 ফোল্ডার)-এর সবচেয়ে বড় দুর্বলতা কী?
ক) বেশি জায়গা নেয় খ) কে কী পরিবর্তন করল, কেন করল — তার কোনো নির্ভরযোগ্য রেকর্ড থাকে না গ) ফোল্ডারের নাম লম্বা হয় ঘ) এটি শুধু Linux-এ চলে
উত্তর: খ — manual কপিতে পরিবর্তনের কারণ, লেখক ও পার্থক্যের কোনো কাঠামোবদ্ধ রেকর্ড থাকে না, ফলে দুটি কপি মেলানো কঠিন হয়। ক আংশিক সত্য হলেও প্রধান সমস্যা নয়; গ তুচ্ছ; ঘ ভুল, কারণ manual backup সব OS-এই একইভাবে দুর্বল।

১.৩ Git কোন ধরনের version control system?
ক) Local-only খ) Centralized গ) Distributed ঘ) Cloud-only
উত্তর: গ — Git distributed: প্রতিটি clone-এ পূর্ণ ইতিহাসসহ repository থাকে, সার্ভার ছাড়াও commit করা যায়। ক ভুল, কারণ Git remote-এর সঙ্গে কাজ করে; খ ভুল, কারণ centralized ব্যবস্থায় (যেমন পুরনো SVN) ইতিহাস শুধু সার্ভারে থাকে; ঘ ভুল, কারণ Git offline-এও চলে।

১.৪ Git ও GitHub-এর পার্থক্য কোনটি সঠিক?
ক) দুটি একই জিনিস খ) Git একটি command-line tool ও version control ব্যবস্থা; GitHub হলো Git repository হোস্ট করার একটি সেবা গ) GitHub ছাড়া Git চলে না ঘ) Git শুধু ওয়েবসাইট, GitHub শুধু কমান্ড
উত্তর: খ — Git আপনার কম্পিউটারে চলে; GitHub সেই repository ইন্টারনেটে রাখে ও collaboration সুবিধা (pull request, issue) দেয়। ক ভুল, কারণ দুটি আলাদা স্তর; গ ভুল, কারণ Git সম্পূর্ণ offline-এ কাজ করে; ঘ উল্টো বলা হয়েছে।

১.৫ একই ফাইলে দুজন developer আলাদা অংশ বদলালে version control কী করে?
ক) একজনের কাজ মুছে দেয় খ) দুজনের পরিবর্তন একত্র করে (merge), আর একই লাইনে সংঘাত হলে conflict দেখিয়ে

মানুষকে সিদ্ধান্ত নিতে দেয় গ) ফাইলটি lock করে দেয় ঘ) দুটি আলাদা ফাইল বানিয়ে দেয়
উত্তর: খ — Git আলাদা লাইনের পরিবর্তন স্বয়ংক্রিয়ভাবে মেলায়; একই লাইনে দুই রকম পরিবর্তন হলে conflict চিহ্নিত করে, কারণ কোনটি রাখা হবে তা যন্ত্র সিদ্ধান্ত নিতে পারে না। ক ভুল, কারণ Git তথ্য হারায় না; গ ভুল, কারণ Git lock-ভিত্তিক নয়; ঘ ভুল, কারণ ফাইল ভাগ হয় না।

D.2 অধ্যায় ২ — Git Architecture and Internal Working (প্রশ্ন ২.১–২.৫)

২.১ Git-এ একটি blob object কী সংরক্ষণ করে?
ক) ফাইলের নাম ও directory কাঠামো খ) একটি ফাইলের বিষয়বস্তু (content), নাম ছাড়া গ) commit বার্তা ঘ) branch-এর নাম
উত্তর: খ — blob শুধু ফাইলের বিষয়বস্তু ধারণ করে; নাম ও কাঠামো থাকে tree object-এ। ক ভুল, সেটি tree-র কাজ; গ ভুল, সেটি commit object-এ থাকে; ঘ ভুল, branch হলো reference, object নয়।

২.২ Tree object-এর কাজ কী?
ক) ফাইলের বিষয়বস্তু রাখা খ) একটি directory-র তালিকা — কোন নামে কোন blob বা sub-tree আছে, তা ধারণ করা গ) লেখকের নাম রাখা ঘ) remote-এর ঠিকানা রাখা
উত্তর: খ — tree হলো directory snapshot: প্রতিটি entry-তে নাম, mode ও সংশ্লিষ্ট blob/tree-র hash থাকে। ক blob-র কাজ; গ commit-এর metadata; ঘ remote config-এ থাকে, object-এ নয়।

২.৩ Commit object-এ কী কী থাকে?
ক) শুধু বার্তা খ) tree-র hash, parent commit-এর hash, লেখক ও committer, সময় ও বার্তা গ) শুধু ফাইলের নাম ঘ) branch ও remote-এর তালিকা
উত্তর: খ — commit একটি snapshot-কে (tree) তার পূর্ববর্তী commit-এর সঙ্গে যুক্ত করে এবং কে কখন লিখল তা রেকর্ড করে। ক অসম্পূর্ণ; গ tree-র কাজ; ঘ config/reference-এর বিষয়।

২.৪ HEAD আসলে কী?
ক) প্রথম commit খ) বর্তমান branch-কে নির্দেশ করা একটি pointer — সাধারণত বর্তমান branch-এর দিকে, যার মাধ্যমে বর্তমান commit নির্ধারিত হয় গ) remote repository ঘ) staging area-র আরেক নাম
উত্তর: খ — HEAD সাধারণত `refs/heads/main`-এর দিকে নির্দেশ করে; detached অবস্থায় সরাসরি কোনো commit-কে নির্দেশ করে। ক ভুল, HEAD প্রথম commit নয়; গ ভুল, remote আলাদা; ঘ ভুল, staging area হলো index।

২.৫ Staging area (index)-এর ভূমিকা কী?
ক) ফাইল মুছে ফেলা খ) পরের commit-এ ঠিক কোন পরিবর্তনগুলো যাবে, তা বেছে নেওয়ার জায়গা গ) remote-এ পাঠানো ঘ) পুরনো commit দেখা
উত্তর: খ — `git add` পরিবর্তন staging area-তে তোলে; commit শুধু সেখানকার বিষয়বস্তু snapshot করে। ক ভুল; গ push-এর কাজ; ঘ log/show-এর কাজ।

D.3 অধ্যায় ৩ — Git Repositories, Commits and History (প্রশ্ন ৩.১–৩.৫)

৩.১ `git status` কী দেখায়?
ক) সব commit-এর তালিকা খ) working directory ও staging area-র বর্তমান অবস্থা — কোন ফাইল বদলেছে, staged হয়েছে বা untracked আছে গ) remote-এর ঠিকানা ঘ) ফাইলের ভেতরের পার্থক্য লাইন ধরে
উত্তর: খ — status হলো কাজের অবস্থার সারসংক্ষেপ। ক `git log`-এর কাজ; গ `git remote -v`-এর কাজ; ঘ `git diff`-এর কাজ।

৩.২ `git add file.txt` কী করে?
ক) ফাইলটি remote-এ পাঠায় খ) ফাইলটির বর্তমান বিষয়বস্তু staging area-তে তোলে গ) ফাইলটি commit করে ঘ) ফাইলটি মুছে দেয়
উত্তর: খ — add শুধু stage করে; commit আলাদা ধাপ। ক push-এর কাজ; গ commit-এর কাজ; ঘ `git rm`-এর কাজ।

৩.৩ `git log --oneline` কী দেখায়?
ক) ফাইলের পার্থক্য খ) প্রতিটি commit এক লাইনে — সংক্ষিপ্ত hash ও বার্তা গ) branch-এর তালিকা ঘ) untracked ফাইল
উত্তর: খ — `--oneline` ইতিহাস সংক্ষেপে দেখায়। ক diff-এর কাজ; গ `git branch`-এর কাজ; ঘ status-এ দেখা যায়।

৩.৪ `.gitignore` ফাইলের কাজ কী?
ক) Git বন্ধ করা খ) যেসব ফাইল/প্যাটার্ন Git track করবে না, তা নির্ধারণ করা (যেমন log, build output, `.env`) গ) ফাইল মুছে ফেলা ঘ) commit বার্তা লেখা
উত্তর: খ — .gitignore-এ লেখা প্যাটার্ন untracked থাকে। ক ভুল; গ ভুল, ফাইল disk থেকে মোছে না; ঘ ভুল। মনে রাখবেন: already tracked ফাইলকে .gitignore স্বয়ংক্রিয়ভাবে untrack করে না — আগে `git rm --cached` লাগে।

৩.৫ ভালো commit বার্তার বৈশিষ্ট্য কোনটি?
ক) "fix", "update" লেখা খ) সংক্ষিপ্ত imperative শিরোনামে কী পরিবর্তন ও কেন, তা বোঝানো, যেমন "Add health check to backup script" গ) যত লম্বা হয় তত ভালো ঘ) বার্তায় password লেখা
উত্তর: খ — ভালো বার্তা ভবিষ্যতের পাঠককে পরিবর্তনের কারণ বোঝায়। ক অস্পষ্ট; গ লম্বা হওয়া গুণ নয়; ঘ নিরাপত্তা লঙ্ঘন — secret কখনো commit করা যায় না।

D.4 অধ্যায় ৪ — Branching, Merging and Rebasing (প্রশ্ন ৪.১–৪.৫)

৪.১ Branch আসলে কী?
ক) ফাইলের কপি খ) একটি commit-কে নির্দেশ করা হালকা pointer (reference) গ) আলাদা repository ঘ) ফোল্ডার
উত্তর: খ — branch তৈরি করা সস্তা, কারণ এটি শুধু একটি hash সংরক্ষণ করে; ফাইল কপি হয় না। ক ভুল, কপি নয়; গ ভুল; ঘ ভুল।

৪.২ Fast-forward merge কখন হয়?
ক) conflict থাকলে খ) main branch-এ feature branch তৈরি হওয়ার পর main-এ কোনো নতুন commit না থাকলে, তখন main-এর pointer-কে শুধু এগিয়ে দেওয়া হয় গ) সবসময় ঘ) rebase করলে
উত্তর: খ — fast-forward-এ কোনো merge commit তৈরি হয় না, pointer এগিয়ে যায়। ক ভুল, conflict হলে fast-forward হয় না; গ ভুল; ঘ ভুল, rebase আলাদা কৌশল।

৪.৩ Merge conflict চিহ্ন `<<<<<<<` কী বোঝায়?
ক) ফাইল নষ্ট হয়েছে খ) একই লাইনে দুই branch-এর ভিন্ন পরিবর্তন আছে — মানুষকে বেছে নিয়ে চিহ্ন মুছে সমাধান করতে হবে গ) ফাইলটি মুছে গেছে ঘ) Git বন্ধ হয়ে গেছে
উত্তর: খ — conflict marker দুটি সংস্করণ দেখায়; সমাধানের পর `git add` করে merge শেষ করতে হয়। ক ভুল, ফাইল নষ্ট হয়নি; গ ভুল; ঘ ভুল।

৪.৪ Rebase ও merge-এর মূল পার্থক্য কী?
ক) কোনো পার্থক্য নেই খ) merge ইতিহাস যেমন আছে তেমন রেখে merge commit যোগ করে; rebase commit-গুলোকে নতুন base-এর উপর আবার প্রয়োগ করে সরলরৈখিক ইতিহাস বানায় গ) rebase conflict তৈরি করে না ঘ) merge branch মুছে দেয়

উত্তর: খ — rebase ইতিহাস পুনর্লিখন করে, তাই public/shared branch-এ rebase বিপজ্জনক। ক ভুল; গ ভুল, rebase-এও conflict হয়; ঘ ভুল, merge branch মোছে না।

৪.৫ `git reset --hard HEAD~1` কী করে?
ক) শুধু commit বার্তা বদলায় খ) HEAD, index ও working directory — তিনটিই আগের commit-এর অবস্থায় ফিরিয়ে দেয় এবং পরের commit-এর পরিবর্তন হারিয়ে যায় (uncommitted ও ওই commit-এর কাজ) গ) নতুন branch বানায় ঘ) remote-এ পাঠায়
উত্তর: খ — `--hard` সব স্তর বদলায়, তাই shared ইতিহাসে বা না বুঝে ব্যবহার বিপজ্জনক; নিরাপদ বিকল্প `git revert`। ক `git commit --amend`-এর কাজ; গ ভুল; ঘ ভুল।

D.5 অধ্যায় ৫ — Remote Repositories and GitHub (প্রশ্ন ৫.১–৫.৫)

৫.১ `git fetch` ও `git pull`-এর পার্থক্য কী?
ক) একই কাজ খ) fetch শুধু remote-এর পরিবর্তন ডাউনলোড করে, working directory বদলায় না; pull = fetch + merge (বা rebase) গ) fetch push করে ঘ) pull শুধু ডাউনলোড করে
উত্তর: খ — fetch নিরাপদে দেখার সুযোগ দেয়, pull সরাসরি মেলায়। ক ভুল; গ ভুল; ঘ উল্টো বলা হয়েছে।

৫.২ Tracking branch কী?
ক) মুছে যাওয়া branch খ) local branch, যার সঙ্গে একটি নির্দিষ্ট remote branch যুক্ত থাকে, ফলে `git pull` / `git push` জানে কোথা থেকে আনতে/কোথায় পাঠাতে হবে গ) main branch ঘ) tag
উত্তর: খ — `git push -u origin feature` upstream সেট করে। ক ভুল; গ ভুল, main-ও একটি tracking branch হতে পারে, কিন্তু সংজ্ঞা সেটি নয়; ঘ ভুল, tag স্থির pointer।

৫.৩ GitHub-এ SSH authentication-এর জন্য কী লাগে?
ক) password ইমেইল করা খ) নিজের কম্পিউটারে key pair তৈরি করে public key GitHub-এ যোগ করা; private key কখনো শেয়ার করা যাবে না গ) token ফাইলে লেখা ঘ) কিছুই লাগে না
উত্তর: খ — SSH-তে private key দিয়ে পরিচয় প্রমাণিত হয়, password পাঠানো হয় না। ক অনিরাপদ ও ভুল; গ token HTTPS-এর পদ্ধতি, SSH-এর নয়; ঘ ভুল।

৫.৪ Pull request (PR) আসলে কী?
ক) ফাইল ডাউনলোডের অনুরোধ খ) একটি branch-এর পরিবর্তন অন্য branch-এ merge করার প্রস্তাব, যেখানে review, আলোচনা ও পরীক্ষা হয় গ) repository মুছে ফেলার অনুরোধ ঘ) issue বন্ধ করা
উত্তর: খ — PR merge করার আগে code review-এর জায়গা। ক ভুল; গ ভুল; ঘ ভুল, PR merge হলে সংশ্লিষ্ট issue বন্ধ হতে পারে, কিন্তু সেটি সংজ্ঞা নয়।

৫.৫ Tag ও release-এর সম্পর্ক কী?
ক) একই জিনিস খ) tag একটি নির্দিষ্ট commit-কে চিহ্নিত করে (যেমন v1.0.0); release হলো সেই tag-এর উপর ভিত্তি করে GitHub-এ প্রকাশিত প্যাকেজ/নোটসহ সংস্করণ গ) tag branch-এর নাম ঘ) release commit মুছে দেয়
উত্তর: খ — tag Git-এর স্থির pointer, release GitHub-এর উপস্থাপনা স্তর। ক ভুল; গ ভুল; ঘ ভুল।

D.6 অধ্যায় ৬ — Professional Git Workflow and Collaboration (প্রশ্ন ৬.১–৬.৫)

৬.১ Feature branch workflow-এ প্রতিটি নতুন কাজ কোথায় করা হয়?
ক) সরাসরি main-এ খ) main থেকে আলাদা একটি ছোট branch-এ, শেষে PR-এর মাধ্যমে main-এ merge করা হয় গ) আলাদা repository-তে ঘ) local ফাইলে, commit ছাড়া

উত্তর: খ — এতে main সবসময় স্থিতিশীল থাকে এবং প্রতিটি পরিবর্তন review হয়। ক trunk-based-এর কাছাকাছি, কিন্তু feature workflow-এর সংজ্ঞা নয়; গ অপ্রয়োজনীয়; ঘ version control-এর উদ্দেশ্যের বিপরীত।

৬.২ GitHub Flow-এর মূল ধাপগুলো কী?
ক) main-এ সরাসরি push খ) branch তৈরি → commit → pull request → review → merge → deploy গ) fork ছাড়া কিছু নয় ঘ) tag তৈরি করা
উত্তর: খ — GitHub Flow-এ main সবসময় deployable থাকে। ক ভুল, কারণ review বাদ পড়ে; গ ভুল; ঘ ভুল, tag একটি ধাপ মাত্র।

৬.৩ Semantic versioning 2.3.1-এ সংখ্যাগুলো কী বোঝায়?
ক) বছর.মাস.দিন খ) major.minor.patch — major = ভাঙা পরিবর্তন, minor = পেছনে-সামঞ্জস্যপূর্ণ নতুন ফিচার, patch = bug fix গ) লেখকের সংখ্যা ঘ) ফাইলের সংখ্যা
উত্তর: খ — semver-এর এটাই সংজ্ঞা। ক ভুল, সেটি date-based versioning; গ ও ঘ ভুল।

৬.৪ Protected branch কেন ব্যবহার করা হয়?
ক) branch লুকাতে খ) main-এ সরাসরি push বন্ধ করে PR review ও পরীক্ষা বাধ্যতামূলক করতে গ) branch দ্রুত করতে ঘ) ফাইল compress করতে
উত্তর: খ — protected branch দুর্ঘটনাবশত বা review ছাড়া পরিবর্তন ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

৬.৫ ভালো pull request-এ কী থাকা উচিত?
ক) শুধু কোড খ) কী বদলেছে, কেন বদলেছে, কীভাবে পরীক্ষা করা হয়েছে — তার সংক্ষিপ্ত বিবরণ ও ছোট, পর্যালোচনাযোগ্য পরিবর্তন গ) অনেকগুলো অসম্পর্কিত পরিবর্তন একসঙ্গে ঘ) কোনো বিবরণ ছাড়া বড় diff
উত্তর: খ — ছোট ও বর্ণনাসহ PR দ্রুত review হয়। ক অসম্পূর্ণ; গ review কঠিন করে; ঘ reviewer-কে অনুমান করতে বাধ্য করে।

D.7 অধ্যায় ৭ — Introduction to Shell Scripting (প্রশ্ন ৭.১–৭.৫)

৭.১ Shebang (`#!/bin/bash`) কী?
ক) মন্তব্য খ) script-এর প্রথম লাইনে লেখা নির্দেশ, যা বলে কোন interpreter দিয়ে script চালাতে হবে গ) ফাইলের নাম ঘ) ভেরিয়েবল
উত্তর: খ — kernel shebang পড়ে `/bin/bash` দিয়ে script চালায়। ক ভুল, `#` দিয়ে শুরু হলেও shebang বিশেষ অর্থ বহন করে; গ ভুল; ঘ ভুল।

৭.২ একটি script-কে execute করার জন্য কী লাগে?
ক) শুধু লেখা খ) execute permission (`chmod +x`) এবং সঠিক shebang, অথবা `bash script.sh` দিয়ে চালানো গ) root হওয়া ঘ) ইন্টারনেট
উত্তর: খ — সরাসরি চালাতে `+x` লাগে; `bash script.sh` দিলে permission ছাড়াও চলে, কারণ তখন bash-ই interpreter। ক অসম্পূর্ণ; গ ভুল, root সব script-এর জন্য লাগে না; ঘ ভুল।

৭.৩ Shell ও terminal-এর পার্থক্য কী?
ক) একই জিনিস খ) terminal হলো window/প্রোগ্রাম যেখানে লেখা দেখা যায়; shell (Bash) হলো সেই লেখা পড়ে কমান্ড চালানো প্রোগ্রাম গ) shell শুধু GUI ঘ) terminal শুধু ফাইল
উত্তর: খ — terminal input/output-এর মাধ্যম, shell নির্দেশনা ব্যাখ্যা করে। ক ভুল; গ ও ঘ ভুল।

৭.৪ `bash -x script.sh` কী করে?
ক) script মুছে দেয় খ) প্রতিটি কমান্ড চালানোর আগে তা দেখিয়ে debug করতে সাহায্য করে (trace mode) গ) script

দ্রুত চালায় ঘ) ভেরিয়েবল মুছে দেয়
উত্তর: খ — `-x` প্রতিটি কমান্ড `+` চিহ্নসহ দেখায়। ক ভুল; গ ভুল; ঘ ভুল।

৭.৫ Interactive ও non-interactive shell-এর পার্থক্য কী?
ক) কোনো পার্থক্য নেই খ) interactive shell মানুষের সঙ্গে সরাসরি কথা বলে (prompt দেখায়, input নেয়); script চালানো shell non-interactive গ) non-interactive দ্রুত নয় ঘ) interactive-এ script চলে না
উত্তর: খ — interactive shell prompt ও history দেয়; script-এর shell input ফাইল থেকে পড়ে। ক ভুল; গ ভুল; ঘ ভুল, interactive shell-এও script চালানো যায়।

D.8 অধ্যায় ৮ — Variables, Data Types and User Input (প্রশ্ন ৮.১–৮.৫)

৮.১ Bash-এ `name="Nahid"` লেখার সময় `=`-এর আশেপাশে space দিলে কী হয়?
ক) কিছু হয় না খ) Bash `name`-কে কমান্ড ভেবে ভুল করে (command not found) অথবা ভুল ব্যাখ্যা করে — assignment-এ space দেওয়া যায় না গ) ভেরিয়েবল মুছে যায় ঘ) script দ্রুত চলে
উত্তর: খ — Bash-এ assignment-এ `=`-এর দুপাশে space থাকলে সেটি আর assignment থাকে না। ক ভুল; গ ভুল; ঘ ভুল।

৮.২ `"$name"` ও `$name`-এর মধ্যে নিরাপদ কোনটি এবং কেন?
ক) দুটো একই খ) double quote করা রূপ নিরাপদ, কারণ space বা glob থাকা মান ভেঙে একাধিক শব্দ হয় না গ) quote ছাড়া নিরাপদ ঘ) single quote-এ ভেরিয়েবল expand হয়
উত্তর: খ — unquoted expansion word splitting ও globbing ঘটায়, যা ফাঁকা বা `*` থাকা মানে ভুল আচরণ করে। ক ভুল; গ ভুল; ঘ ভুল — single quote-এ `$name` আক্ষরিক থাকে, expand হয় না।

৮.৩ `$?`, `$#`, `$@` — এগুলো কী?
ক) সবই একই খ) `$?` শেষ কমান্ডের exit status, `$#` argument-এর সংখ্যা, `$@` সব argument গ) `$?` হলো process ID ঘ) `$#` হলো script-এর নাম
উত্তর: খ — এটাই সঠিক সংজ্ঞা। গ ভুল, process ID হলো `$$`; ঘ ভুল, script-এর নাম `$0`।

৮.৪ `export API_URL="https://example.com"` কী করে?
ক) ফাইলে লেখে খ) ভেরিয়েবলটিকে child process-গুলোর কাছে পৌঁছে দেয় (environment variable বানায়) গ) ভেরিয়েবল মুছে দেয় ঘ) ইন্টারনেটে পাঠায়
উত্তর: খ — export ছাড়া ভেরিয়েবল শুধু বর্তমান shell-এ থাকে; export করলে child process-ও পায়। ক ভুল; গ ভুল; ঘ ভুল।

৮.৫ Command substitution `$(date)` কী করে?
ক) কমান্ড মুছে দেয় খ) কমান্ড চালিয়ে তার stdout-কে ওই জায়গায় বসিয়ে দেয় গ) কমান্ডের exit code দেয় ঘ) ফাইল তৈরি করে
উত্তর: খ — `today=$(date +%F)` তারিখটি ভেরিয়েবলে রাখে। ক ভুল; গ ভুল, exit code `$?`-এ থাকে; ঘ ভুল।

D.9 অধ্যায় ৯ — Conditions, Loops and Control Flow (প্রশ্ন ৯.১–৯.৫)

৯.১ `[ -f "$file" ]` কী পরীক্ষা করে?
ক) directory আছে কি না খ) regular file আছে কি না গ) ফাইল খালি কি না ঘ) ফাইল executable কি না
উত্তর: খ — `-f` regular file পরীক্ষা করে। ক `-d`-এর কাজ; গ `-s` (non-empty) বা `! -s`-এর কাজ; ঘ `-x`-এর কাজ।

৯.২ সংখ্যা তুলনায় Bash-এ কোনটি সঠিক?

ক) `[ "$a" = "$b" ]` খ) `[ "$a" -eq "$b" ]` গ) `[ "$a" == "$b" ]` সংখ্যার জন্য ঘ) `[ "$a" > "$b" ]`

উত্তর: খ — সংখ্যার জন্য `-eq`, `-lt`, `-gt` ব্যবহার করতে হয়; `=` string তুলনা করে, তাই "10" ও "010" string হিসেবে ভিন্ন হতে পারে। ক string তুলনা, সংখ্যার জন্য ভুল পদ্ধতি; গ একই সমস্যা; ঘ `[ ]`-এ `>` redirection হয়ে যায়।

৯.৩ `case` statement কখন ব্যবহার করা ভালো?

ক) একটি শর্তে খ) একই মানের অনেকগুলো সম্ভাব্য মানের মধ্যে একটিকে বেছে নিতে, যেমন menu বা subcommand গ) loop-এ ঘ) ফাইল পড়তে

উত্তর: খ — case একাধিক pattern মেলাতে পরিষ্কার। ক ভুল, সেখানে if যথেষ্ট; গ ভুল; ঘ ভুল।

৯.৪ `for f in *.log` লুপে কোনো `.log` ফাইল না থাকলে কী হয়?

ক) লুপ চলে না খ) Bash-এ (nullglob ছাড়া) pattern-টি আক্ষরিক `*.log` হিসেবে একবার চলে — ফলে ভুল ফাইল নিয়ে কাজ হতে পারে গ) error হয় ঘ) সব ফাইল নিয়ে চলে

উত্তর: খ — এটাই Bash-এর default আচরণ; নিরাপদ পদ্ধতি হলো `shopt -s nullglob` বা ফাইল আছে কি না পরীক্ষা করা। ক ভুল, nullglob ছাড়া লুপ থামে না; গ ভুল, error হয় না; ঘ ভুল।

৯.৫ `while read -r line` লুপে `-r` কেন দেওয়া হয়?

ক) গতি বাড়াতে খ) backslash-কে escape হিসেবে ব্যাখ্যা না করে আক্ষরিক রাখতে গ) ফাইল মুছতে ঘ) লুপ থামাতে

উত্তর: খ — `-r` ছাড়া `\` পরের অক্ষরকে বিশেষভাবে ব্যাখ্যা করে, ফলে পাঠ বদলে যায়। ক ভুল; গ ভুল; ঘ ভুল।

D.10 অধ্যায় ১০ — Functions, Arguments and Script Structure (প্রশ্ন ১০.১–১০.৫)

১০.১ Bash function-এ `return 3` কী ফেরত দেয়?

ক) সংখ্যা 3 একটি মান হিসেবে খ) exit status 3 (0–255-এর মধ্যে), যা `$?`-এ পড়া যায় — function কোনো string বা বড় সংখ্যা return করতে পারে না গ) string "3" ঘ) কিছুই না

উত্তর: খ — Bash function-এ return শুধু status দেয়; ফলাফল দিতে stdout-এ print করতে হয় বা ভেরিয়েবলে রাখতে হয়। ক ভুল, কারণ এটি মান নয়; গ ভুল; ঘ ভুল।

১০.২ Function-এর ভেতরে `local count=0` কেন লেখা হয়?

ক) গতি বাড়াতে খ) ভেরিয়েবলটিকে শুধু ওই function-এর মধ্যে সীমাবদ্ধ রাখতে, যাতে বাইরের একই নামের ভেরিয়েবল বদলে না যায় গ) ভেরিয়েবল মুছতে ঘ) global করতে

উত্তর: খ — `local` ছাড়া assignment global হয়, ফলে নামের সংঘাত হয়। ক ভুল; গ ভুল; ঘ উল্টো।

১০.৩ Function-এর ভেতরে `$1`, `$@` কী বোঝায়?

ক) script-এর argument খ) ওই function-কে দেওয়া argument — function-এর নিজস্ব positional parameter গ) global ভেরিয়েবল ঘ) exit code

উত্তর: খ — function call হলে `$1` বদলে function-এর argument হয়। ক ভুল, script-এর argument function-এর বাইরে থাকে; গ ভুল; ঘ ভুল।

১০.৪ একটি reusable script-কে অন্য script থেকে ব্যবহারের নিরাপদ উপায় কোনটি?

ক) কপি-পেস্ট করা খ) `source lib.sh` (বা `. lib.sh`) দিয়ে function লোড করা, অথবা script-টিকে PATH-এ রেখে কমান্ড হিসেবে চালানো গ) script মুছে দেওয়া ঘ) সব কোড এক ফাইলে লেখা

উত্তর: খ — sourcing function শেয়ার করে, PATH-এ রাখা পুনর্ব্যবহার সহজ করে। ক রক্ষণাবেক্ষণ কঠিন করে; গ ভুল; ঘ ছোট কাজে চলে, বড় কাজে নয়।

১০.৫ Command-line argument না দিলে script-এর কী করা উচিত?
ক) চুপচাপ চলতে থাকা খ) ব্যবহারবিধি (usage) দেখিয়ে nonzero exit code-সহ থামা গ) অনুমান করে চালানো ঘ) ফাইল মুছে দেওয়া
উত্তর: খ — ভুল input-এ স্পষ্ট বার্তা দিয়ে থামা নিরাপদ ও পেশাদার আচরণ। ক ভুল, কারণ ভুল কাজ হতে পারে; গ বিপজ্জনক; ঘ ভুল।

D.11 অধ্যায় ১১ — Files, Text Processing and Command Pipelines (প্রশ্ন ১১.১–১১.৫)

১১.১ `cmd > out.log 2>&1` কী করে?
ক) শুধু stdout ফাইলে পাঠায় খ) stdout ও stderr — দুটোই `out.log`-এ পাঠায় গ) ফাইল মুছে দেয় ঘ) শুধু stderr দেখায়
উত্তর: খ — `2>&1` stderr-কে stdout-এর গন্তব্যে পাঠায়। ক ভুল, stderr বাদ পড়ত; গ ভুল; ঘ ভুল।

১১.২ `grep -r "ERROR" /var/log`-এ `-r` কী করে?
ক) ফাইল মুছে দেয় খ) directory-র ভেতরে recursiveভাবে সব ফাইলে খোঁজে গ) শুধু প্রথম ফাইলে খোঁজে ঘ) case-sensitive করে
উত্তর: খ — `-r` ছাড়া grep directory পড়তে পারে না। ক ভুল; গ ভুল; ঘ ভুল, case-insensitive করতে `-i` লাগে।

১১.৩ `awk '{print $2}'`-এ `$2` কী?
ক) দ্বিতীয় ফাইল খ) প্রতিটি লাইনের দ্বিতীয় field (ডিফল্ট whitespace দিয়ে ভাগ করা) গ) দ্বিতীয় লাইন ঘ) ভেরিয়েবল
উত্তর: খ — awk লাইনকে field-এ ভাগ করে; `$0` পুরো লাইন। ক ভুল; গ ভুল; ঘ ভুল।

১১.৪ ফাইলের নামে space থাকলে `find … | xargs rm` কেন বিপজ্জনক?
ক) ধীর হয় খ) xargs ডিফল্টে space দিয়ে নাম ভাঙে, ফলে একটি ফাইল ভেঙে একাধিক ভুল নামে পরিণত হতে পারে — নিরাপদ উপায় `find … -print0 | xargs -0` গ) ফাইল খুঁজে পায় না ঘ) কিছুই হয় না
উত্তর: খ — space-যুক্ত নামে ভুল ফাইল মোছা বা ভুল কমান্ড চলার ঝুঁকি থাকে। ক গৌণ; গ ভুল; ঘ ভুল।

১১.৫ `sort file.txt | uniq -c` কী করে?
ক) ফাইল মুছে দেয় খ) লাইনগুলো সাজিয়ে পাশাপাশি থাকা একই লাইনের সংখ্যা গোনে — `uniq` কাজ করে শুধু সাজানো (পাশাপাশি) লাইনে, তাই আগে sort লাগে গ) শুধু sort করে ঘ) লাইন সংখ্যা দেয়
উত্তর: খ — sort ছাড়া uniq দূরে থাকা একই লাইন আলাদা গোনে। ক ভুল; গ অসম্পূর্ণ; ঘ `wc -l`-এর কাজ।

D.12 অধ্যায় ১২ — Error Handling, Exit Codes and Debugging (প্রশ্ন ১২.১–১২.৫)

১২.১ Unix-এ exit code 0-এর অর্থ কী?
ক) ত্রুটি খ) সফল সমাপ্তি গ) ফাইল পাওয়া যায়নি ঘ) permission নেই
উত্তর: খ — 0 মানে সফল; nonzero মানে কোনো না কোনো ব্যর্থতা। ক উল্টো; গ সাধারণত 1 বা 2, কিন্তু নির্দিষ্ট নয়; ঘ সাধারণত 1 বা 126/127-এর মতো নির্দিষ্ট code, 0 নয়।

১২.২ `set -euo pipefail`-এর প্রতিটি অংশ কী করে?
ক) কিছুই করে না খ) `-e`: ব্যর্থ কমান্ডে থামে; `-u`: অনির্ধারিত ভেরিয়েবলে error দেয়; `pipefail`: pipeline-এর যেকোনো অংশ ব্যর্থ হলে pipeline-এর status ব্যর্থ হয় গ) শুধু -e কাজ করে ঘ) script দ্রুত করে
উত্তর: খ — তিনটি মিলে নিরাপদ script-এর ভিত্তি। ক ভুল; গ ভুল, তিনটিই কাজ করে; ঘ ভুল।

১২.৩ `trap 'cleanup' EXIT` কী করে?
ক) script মুছে দেয় খ) script যেভাবেই শেষ হোক (সফল বা ব্যর্থ), শেষে `cleanup` function চালায় — temporary ফাইল মোছার নির্ভরযোগ্য উপায় গ) signal বন্ধ করে ঘ) লুপ চালায়
উত্তর: খ — trap EXIT cleanup নিশ্চিত করে। ক ভুল; গ ভুল, trap signal ধরে, বন্ধ করে না; ঘ ভুল।

১২.৪ Idempotent script বলতে কী বোঝায়?
ক) একবারই চলে খ) একাধিকবার চালালেও একই ফল দেয় — দ্বিতীয়বার চালালে ক্ষতি বা দ্বিগুণ কাজ হয় না গ) দ্রুত চলে ঘ) root লাগে
উত্তর: খ — idempotency automation-এর মূল গুণ; যেমন directory থাকলে আবার তৈরি না করা। ক ভুল; গ ভুল; ঘ ভুল।

১২.৫ `command || { echo "failed" >&2; exit 1; }` প্যাটার্নের উদ্দেশ্য কী?
ক) কমান্ড লুকানো খ) কমান্ড ব্যর্থ হলে স্পষ্ট বার্তা stderr-এ দিয়ে nonzero code-সহ থামা গ) কমান্ড দুবার চালানো ঘ) ফাইল তৈরি করা
উত্তর: খ — এটি ব্যর্থতা স্পষ্ট করে ও চুপচাপ এগিয়ে যাওয়া ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

D.13 অধ্যায় ১৩ — Advanced Bash Automation and Best Practices (প্রশ্ন ১৩.১–১৩.৫)

১৩.১ Temporary ফাইলের নিরাপদ উপায় কোনটি?
ক) `/tmp/myfile` লেখা খ) `mktemp` দিয়ে অনন্য নাম তৈরি করা এবং trap-এ মুছে ফেলা গ) একই নাম বারবার ব্যবহার করা ঘ) home-এ লেখা
উত্তর: খ — নির্দিষ্ট নামে লিখলে অন্য process বা আক্রমণকারী আগেই সেই নামে ফাইল/লিংক বানিয়ে রাখতে পারে। ক অনিরাপদ; গ সংঘাত তৈরি করে; ঘ নিরাপত্তা দেয় না।

১৩.২ Lock file কেন ব্যবহার করা হয়?
ক) ফাইল লুকাতে খ) একই script-এর দুটি copy একসঙ্গে চলা ঠেকাতে — দ্বিতীয় copy lock না পেলে থামে গ) গতি বাড়াতে ঘ) log লিখতে
উত্তর: খ — cron-এ একই job ওভারল্যাপ করলে তথ্য নষ্ট হতে পারে; `flock` এই কাজের মানসম্মত উপায়। ক ভুল; গ ভুল; ঘ ভুল।

১৩.৩ ShellCheck কী?
ক) shell-এর গতি মাপার tool খ) Bash script-এর সাধারণ ভুল, quoting সমস্যা ও বিপজ্জনক প্যাটার্ন ধরার static analysis tool গ) ফাইল মুছে ফেলার tool ঘ) editor
উত্তর: খ — ShellCheck চালানোর আগেই ভুল ধরে। ক ভুল; গ ভুল; ঘ ভুল।

১৩.৪ Command injection-এর ঝুঁকি কোথায় সবচেয়ে বেশি?
ক) নির্দিষ্ট কমান্ডে খ) ব্যবহারকারীর দেওয়া input-কে যাচাই ছাড়া shell কমান্ডে বসালে — আক্রমণকারী `; rm -rf`-এর মতো অংশ ঢুকিয়ে দিতে পারে গ) ফাইল পড়ায় ঘ) log লেখায়
উত্তর: খ — untrusted input কখনো shell-এ সরাসরি বসানো যায় না; array বা Python-এর `shell=False` ব্যবহার করতে হয়। ক নিরাপদ; গ ও ঘ নিজে থেকে injection নয়।

১৩.৫ Portability-র জন্য Bash script-এ কী এড়ানো উচিত?
ক) `#!/bin/bash` খ) শুধু একটি distro-তে থাকা বিশেষ tool বা Bash-এর নতুন বৈশিষ্ট্যের উপর নির্ভর করা, যখন

POSIX বা সাধারণ উপায়ে কাজটি করা যায় গ) function ব্যবহার ঘ) `set -e`
উত্তর: খ — portability মানে বিভিন্ন পরিবেশে চলা। ক ভুল, shebang স্পষ্টতা দেয়; গ ভুল; ঘ ভুল, `set -e` নিরাপত্তা দেয়।

D.14 অধ্যায় ১৪ — Python Fundamentals for Automation (প্রশ্ন ১৪.১–১৪.৫)

১৪.১ Python-এ virtual environment (venv) কেন ব্যবহার করা হয়?
ক) গতি বাড়াতে খ) প্রতিটি প্রজেক্টের dependency আলাদা রাখতে, যাতে এক প্রজেক্টের library অন্য প্রজেক্টের সঙ্গে সংঘাত না করে গ) ফাইল লুকাতে ঘ) Python install করতে
উত্তর: খ — venv প্রজেক্ট-ভিত্তিক বিচ্ছিন্নতা দেয়। ক ভুল; গ ভুল; ঘ ভুল, venv Python install করে না।

১৪.২ Python-এ `list` ও `tuple`-এর মূল পার্থক্য কী?
ক) কোনো পার্থক্য নেই খ) list পরিবর্তনযোগ্য (mutable), tuple অপরিবর্তনীয় (immutable) গ) tuple দ্রুত নয় ঘ) list-এ সংখ্যা রাখা যায় না
উত্তর: খ — tuple একবার তৈরি হলে বদলানো যায় না, তাই স্থির তথ্যের জন্য নিরাপদ। ক ভুল; গ ভুল, tuple সাধারণত হালকা; ঘ ভুল।

১৪.৩ Dictionary-তে key খুঁজে না পেলে `d["missing"]` কী করে?
ক) None দেয় খ) `KeyError` exception তোলে গ) খালি string দেয় ঘ) 0 দেয়
উত্তর: খ — সরাসরি indexing না থাকলে KeyError তোলে; নিরাপদে `d.get("missing")` None দেয়। ক `.get()`-এর আচরণ; গ ও ঘ ভুল।

১৪.৪ `if __name__ == "__main__":` কেন লেখা হয়?
ক) গতি বাড়াতে খ) ফাইলটি সরাসরি চালালে main কোড চলবে, কিন্তু import করলে চলবে না — module হিসেবে পুনর্ব্যবহারের জন্য গ) ফাইল লুকাতে ঘ) error ধরতে
উত্তর: খ — এটি import ও execution আলাদা করে। ক ভুল; গ ভুল; ঘ ভুল, error ধরা try/except-এর কাজ।

১৪.৫ `requirements.txt`-এর কাজ কী?
ক) Python install করা খ) প্রজেক্টের dependency ও তাদের সংস্করণ তালিকাভুক্ত করা, যাতে `pip install -r requirements.txt` দিয়ে একই পরিবেশ আবার তৈরি করা যায় গ) কোড লেখা ঘ) test চালানো
উত্তর: খ — এটি পুনরুৎপাদনযোগ্য পরিবেশের ভিত্তি। ক ভুল; গ ভুল; ঘ ভুল, test চালায় pytest।

D.15 অধ্যায় ১৫ — Files, Directories and System Operations (প্রশ্ন ১৫.১–১৫.৫)

১৫.১ `pathlib.Path` কেন `os.path`-এর চেয়ে পছন্দ করা হয়?
ক) দ্রুত খ) Path object-ভিত্তিক, পড়তে সহজ ও OS-নিরপেক্ষ — `path / "file.txt"` দিয়ে যুক্ত করা যায় গ) শুধু Linux-এ চলে ঘ) ফাইল মুছে দেয়
উত্তর: খ — pathlib string জোড়া দেওয়ার ভুল কমায়। ক গৌণ; গ ভুল, pathlib cross-platform; ঘ ভুল।

১৫.২ `with open("a.txt") as f:` ব্যবহারের সুবিধা কী?
ক) ফাইল দ্রুত খোলে খ) block শেষ হলে ফাইল স্বয়ংক্রিয়ভাবে বন্ধ হয় — exception হলেও গ) ফাইল মুছে দেয় ঘ) ফাইল বড় করে
উত্তর: খ — with statement resource বন্ধ করা নিশ্চিত করে। ক গৌণ; গ ভুল; ঘ ভুল।

১৫.৩ `shutil.copy2()` ও `shutil.copy()`-এর পার্থক্য কী?
ক) একই খ) copy2 ফাইলের metadata (সময়, permission) সংরক্ষণের চেষ্টা করে, copy শুধু বিষয়বস্তু কপি করে গ)

copy2 ফোল্ডার কপি করে ঘ) copy ফাইল মুছে দেয়
উত্তর: খ — backup-এ metadata গুরুত্বপূর্ণ বলে copy2 পছন্দ করা হয়। ক ভুল; গ ভুল, ফোল্ডারের জন্য `copytree()`; ঘ ভুল।

১৫.৪ Backup-এর আগে ফাইলের checksum (যেমন SHA-256) কেন নেওয়া হয়?
ক) ফাইল ছোট করতে খ) পরে backup-এর অখণ্ডতা যাচাই করতে — কপি করার সময় তথ্য নষ্ট হয়েছে কি না, তা ধরা যায় গ) ফাইল encrypt করতে ঘ) ফাইল মুছতে
উত্তর: খ — checksum মিললে কপিটি বিশ্বাসযোগ্য। ক ভুল; গ ভুল, checksum encryption নয়; ঘ ভুল।

১৫.৫ `Path("dir").mkdir(parents=True, exist_ok=True)` কী করে?
ক) ফাইল তৈরি করে খ) প্রয়োজনে parent directory-সহ directory তৈরি করে; আগে থেকে থাকলে error দেয় না গ) directory মুছে দেয় ঘ) permission বদলায়
উত্তর: খ — `parents=True` মাঝখানের directory বানায়, `exist_ok=True` আগে থাকলে শান্ত থাকে। ক ভুল; গ `rmdir` / `shutil.rmtree`-এর কাজ; ঘ `chmod`-এর কাজ।

D.16 অধ্যায় ১৬ — Processes, Subprocesses and Command Execution (প্রশ্ন ১৬.১–১৬.৫)

১৬.১ `subprocess.run(["ls", "-l"], capture_output=True, text=True)`-এ তালিকা (list) ব্যবহারের কারণ কী?
ক) গতি বাড়াতে খ) shell ছাড়া সরাসরি program চালানো — এতে shell injection-এর ঝুঁকি থাকে না গ) ফাইল তৈরি করতে ঘ) output লুকাতে
উত্তর: খ — list আকারে argument দিলে shell ব্যাখ্যা করে না; `shell=True` ও string কমান্ড injection-এর দরজা খোলে। ক গৌণ; গ ভুল; ঘ ভুল, capture_output output ধরে রাখে।

১৬.২ `subprocess.run(..., check=True)` কী করে?
ক) কমান্ড দুবার চালায় খ) কমান্ড nonzero exit code দিলে `CalledProcessError` exception তোলে গ) output দেখায় ঘ) timeout দেয়
উত্তর: খ — check=True ব্যর্থতা চুপচাপ পেরিয়ে যাওয়া ঠেকায়। ক ভুল; গ capture_output-এর কাজ; ঘ timeout parameter-এর কাজ।

১৬.৩ `subprocess.run(..., timeout=10)`-এ timeout হলে কী হয়?
ক) কমান্ড চলতেই থাকে খ) `TimeoutExpired` exception ওঠে এবং process-টি বন্ধ করা হয় গ) 0 return করে ঘ) ফাইল তৈরি হয়
উত্তর: খ — timeout ঝুলে থাকা automation ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

১৬.৪ stdout ও stderr আলাদা রাখার সুবিধা কী?
ক) কোনো সুবিধা নেই খ) আসল ফলাফল (stdout) ও ত্রুটি/সতর্কবার্তা (stderr) আলাদা করে বিশ্লেষণ ও log করা যায় গ) গতি বাড়ে ঘ) ফাইল ছোট হয়
উত্তর: খ — আলাদা stream-এ ভুল বার্তা ফলাফলের সঙ্গে মিশে যায় না। ক ভুল; গ ভুল; ঘ ভুল।

১৬.৫ ব্যবহারকারীর দেওয়া ফাইলের নাম দিয়ে কমান্ড চালাতে নিরাপদ উপায় কোনটি?
ক) `shell=True` দিয়ে string বানানো খ) argument list-এ নামটি আলাদা element হিসেবে দেওয়া (`shell=False`) গ) নামটি quote না করা ঘ) `os.system()` ব্যবহার করা

উত্তর: খ — list-এ argument দিলে `; rm -rf`-এর মতো অংশও আক্ষরিক নাম হিসেবে বিবেচিত হয়। ক ও ঘ shell injection-এর ঝুঁকিপূর্ণ; গ ভুল।

D.17 অধ্যায় ১৭ — APIs, HTTP Requests and JSON Processing (প্রশ্ন ১৭.১–১৭.৫)

১৭.১ HTTP GET ও POST-এর মূল পার্থক্য কী?
ক) একই কাজ খ) GET তথ্য পড়তে (safe, idempotent), POST নতুন resource তৈরি বা কাজ করতে তথ্য পাঠায় গ) GET তথ্য পাঠায় ঘ) POST তথ্য পড়ে
উত্তর: খ — GET-এর body থাকে না বললেই চলে; POST body-তে তথ্য পাঠায়। ক ভুল; গ ও ঘ উল্টো।

১৭.২ HTTP status code 404 ও 500-এর অর্থ কী?
ক) দুটোই সফল খ) 404 = resource পাওয়া যায়নি (client-এর ভুল), 500 = সার্ভারের ভেতরের ত্রুটি গ) 404 = সার্ভার বন্ধ ঘ) 500 = পাওয়া যায়নি
উত্তর: খ — 4xx client-এর দিকের ভুল, 5xx সার্ভারের দিকের ভুল। ক ভুল; গ ভুল; ঘ উল্টো।

১৭.৩ API token কোথায় রাখা উচিত?
ক) source code-এ খ) environment variable বা secret manager-এ, কখনো Git-এ commit করা যাবে না গ) README-তে ঘ) public ফাইলে
উত্তর: খ — code-এ token থাকলে repository ফাঁস হলেই token ফাঁস হয়। ক, গ ও ঘ সবই নিরাপত্তা লঙ্ঘন।

১৭.৪ API call-এ timeout দেওয়া কেন জরুরি?
ক) গতি বাড়াতে খ) সার্ভার সাড়া না দিলে automation অনির্দিষ্টকাল ঝুলে না থাকে — নির্দিষ্ট সময় পর ব্যর্থ হয়ে retry বা log করা যায় গ) token বাঁচাতে ঘ) JSON ছোট করতে
উত্তর: খ — timeout ছাড়া একটি ঝুলে থাকা request পুরো job আটকে দেয়। ক গৌণ; গ ভুল; ঘ ভুল।

১৭.৫ Rate limiting-এ 429 status পেলে সঠিক আচরণ কী?
ক) সঙ্গে সঙ্গে আবার চেষ্টা করা খ) `Retry-After` header দেখে অপেক্ষা করে ধীরে আবার চেষ্টা করা (backoff), বারবার আঘাত না করা গ) token বদলানো ঘ) request মুছে দেওয়া
উত্তর: খ — 429 মানে "খুব বেশি অনুরোধ" — অবিলম্বে retry করলে আরও block হওয়ার ঝুঁকি থাকে। ক ভুল; গ ভুল; ঘ ভুল।

D.18 অধ্যায় ১৮ — YAML, Configuration and Environment Management (প্রশ্ন ১৮.১–১৮.৫)

১৮.১ JSON ও YAML-এর মূল পার্থক্য কী?
ক) একই জিনিস খ) JSON কঠোর bracket/quote-ভিত্তিক; YAML indentation-ভিত্তিক, মানুষের পড়ার জন্য সহজ এবং comment সমর্থন করে গ) YAML শুধু Python-এ চলে ঘ) JSON-এ comment লেখা যায়
উত্তর: খ — YAML config-এর জন্য জনপ্রিয়, JSON API-র জন্য। ক ভুল; গ ভুল, YAML ভাষা-নিরপেক্ষ; ঘ ভুল, JSON-এ comment নেই।

১৮.২ YAML-এ indentation ভুল হলে কী হয়?
ক) কিছু হয় না খ) parser ভুল গঠন বোঝে বা error দেয় — YAML-এ tab ব্যবহার করা যায় না, শুধু space গ) ফাইল মুছে যায় ঘ) মান বদলে যায় না
উত্তর: খ — indentation-ই YAML-এর গঠন নির্ধারণ করে। ক ভুল; গ ভুল; ঘ ভুল।

১৮.৩ `.env` ফাইল Git-এ commit করা উচিত কি?
ক) হ্যাঁ, সবসময় খ) না — এতে secret থাকে; শুধু `.env.example` (খালি মানসহ নমুনা) commit করা যায় গ) শুধু production-এ ঘ) কখনো ফাইল বানানো যাবে না
উত্তর: খ — .env-এ API key/password থাকে, তাই `.gitignore`-এ রাখতে হয়। ক বিপজ্জনক; গ ভুল; ঘ ভুল, .env ব্যবহার করা যায়, শুধু commit করা যায় না।

১৮.৪ Configuration validation কেন করা হয়?
ক) গতি বাড়াতে খ) প্রোগ্রাম শুরুতেই ভুল বা অনুপস্থিত config ধরে স্পষ্ট বার্তা দিতে, যাতে আধা-চলা অবস্থায় ব্যর্থ না হয় গ) ফাইল ছোট করতে ঘ) log লিখতে
উত্তর: খ — fail-fast নীতি: শুরুতে ভুল ধরা পড়লে debugging সহজ হয়। ক ভুল; গ ভুল; ঘ ভুল।

১৮.৫ Configuration ও code আলাদা রাখার নীতি কী?
ক) একই ফাইলে রাখা খ) পরিবেশ-ভেদে বদলায় এমন মান (URL, পথ, সীমা) code-এর বাইরে config/env-এ রাখা, যাতে code না বদলে পরিবেশ বদলানো যায় গ) config মুছে দেওয়া ঘ) hardcode করা
উত্তর: খ — এতে একই code dev, staging ও production-এ চলে। ক ও ঘ এই নীতির বিপরীত; গ ভুল।

D.19 অধ্যায় ১৯ — Logging, Exceptions and Reliable Automation (প্রশ্ন ১৯.১–১৯.৫)

১৯.১ Python logging-এ level-গুলোর সঠিক ক্রম কোনটি?
ক) ERROR > CRITICAL খ) DEBUG < INFO < WARNING < ERROR < CRITICAL গ) INFO < DEBUG ঘ) WARNING < INFO
উত্তর: খ — DEBUG সবচেয়ে বিস্তারিত, CRITICAL সবচেয়ে গুরুতর। ক ভুল, CRITICAL সবচেয়ে উঁচু; গ ভুল; ঘ ভুল।

১৯.২ `print()`-এর বদলে logging module ব্যবহারের প্রধান সুবিধা কী?
ক) গতি খ) level, সময়, ফাইলে লেখা ও ঘূর্ণন (rotation) নিয়ন্ত্রণ করা যায় — production-এ print বিশৃঙ্খল হয় গ) রঙ ঘ) কম কোড
উত্তর: খ — logging কাঠামোবদ্ধ ও নিয়ন্ত্রণযোগ্য। ক গৌণ; গ ভুল; ঘ ভুল।

১৯.৩ `except Exception:` দিয়ে সব exception ধরে চুপচাপ এগিয়ে যাওয়া কেন খারাপ?
ক) খারাপ নয় খ) আসল ত্রুটি লুকিয়ে যায়, debugging কঠিন হয় এবং ভুল অবস্থায় কাজ এগিয়ে যায় গ) গতি কমে ঘ) ফাইল বড় হয়
উত্তর: খ — শুধু সামলানো যায় এমন নির্দিষ্ট exception ধরতে হয় এবং log করতে হয়। ক ভুল; গ ভুল; ঘ ভুল।

১৯.৪ Retry-তে exponential backoff কী?
ক) সঙ্গে সঙ্গে retry খ) প্রতিবার ব্যর্থতার পর অপেক্ষার সময় বাড়িয়ে দেওয়া (যেমন 1s, 2s, 4s), যাতে সার্ভারের উপর চাপ না পড়ে গ) একবারই চেষ্টা ঘ) token বদলানো
উত্তর: খ — backoff সার্ভারকে সুস্থ হওয়ার সময় দেয়। ক ভুল, তা সার্ভারকে আরও চাপ দেয়; গ ভুল; ঘ ভুল।

১৯.৫ `RotatingFileHandler` কী করে?
ক) ফাইল মুছে দেয় খ) log ফাইল নির্দিষ্ট আকারে পৌঁছালে নতুন ফাইলে লেখা শুরু করে ও পুরনো ফাইল সংরক্ষণ করে, ফলে disk ভরে যায় না গ) log বন্ধ করে ঘ) log encrypt করে
উত্তর: খ — এতে log সীমিত জায়গায় থাকে। ক ভুল; গ ভুল; ঘ ভুল।

D.20 অধ্যায় ২০ — Scheduling, Cron and Background Automation (প্রশ্ন ২০.১–২০.৫)

২০.১ Cron expression-এর field-গুলোর সঠিক ক্রম কোনটি?
ক) hour minute day month weekday খ) minute hour day-of-month month day-of-week গ) second minute hour day month ঘ) day month year hour minute
উত্তর: খ — cron-এ পাঁচটি field: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন। ক ক্রম উল্টো; গ ভুল, cron-এ second field নেই; ঘ ভুল।

২০.২ `*/15 * * * *` মানে কী?
ক) প্রতি 15 ঘণ্টায় খ) প্রতি 15 মিনিটে গ) দিনে 15 বার ঘ) মাসে 15 তারিখে
উত্তর: খ — প্রথম field মিনিট, `*/15` মানে প্রতি 15 মিনিটে। ক ভুল, ঘণ্টা দ্বিতীয় field; গ ভুল; ঘ ভুল।

২০.৩ Cron job-এ environment variable (যেমন PATH) নিয়ে সমস্যা কেন হয়?
ক) সমস্যা হয় না খ) cron ন্যূনতম environment নিয়ে চলে — interactive shell-এর PATH, ভেরিয়েবল বা alias থাকে না, তাই পূর্ণ পথ (absolute path) ব্যবহার করতে হয় গ) cron PATH মুছে দেয় ঘ) cron root-এ চলে
উত্তর: খ — terminal-এ চলা script cron-এ ব্যর্থ হওয়ার এটাই সাধারণ কারণ। ক ভুল; গ ভুল; ঘ ভুল, cron ব্যবহারকারীর crontab অনুযায়ী চলে।

২০.৪ Cron job-এর output কোথায় যায়, যদি redirect না করা হয়?
ক) হারিয়ে যায় খ) ব্যবহারকারীর local mail-এ পাঠানো হয় (বা systemd journal-এ, ব্যবস্থা অনুযায়ী) গ) পর্দায় দেখায় ঘ) ফাইলে লেখে
উত্তর: খ — তাই ভালো অভ্যাস হলো stdout/stderr স্পষ্টভাবে log ফাইলে redirect করা। ক ভুল, mail-এ যায়; গ ভুল, cron-এর পর্দা নেই; ঘ ভুল, নিজে থেকে ফাইলে লেখে না।

২০.৫ একই cron job দুবার একসঙ্গে চলা ঠেকাতে কী করা হয়?
ক) কিছুই করা যায় না খ) `flock` দিয়ে lock নেওয়া, যাতে আগের copy চলতে থাকলে নতুন copy থামে গ) job মুছে দেওয়া ঘ) সময় বদলানো
উত্তর: খ — flock একসঙ্গে চলা ঠেকায়। ক ভুল; গ ভুল; ঘ ওভারল্যাপ কমাতে পারে, কিন্তু নিশ্চিত করে না।

D.21 অধ্যায় ২১ — SSH Automation and Remote System Management (প্রশ্ন ২১.১–২১.৫)

২১.১ SSH key pair-এ public ও private key-এর ভূমিকা কী?
ক) দুটোই শেয়ার করা যায় খ) public key সার্ভারে থাকে, private key শুধু নিজের কম্পিউটারে থাকে — private key কখনো শেয়ার করা যাবে না গ) private key সার্ভারে থাকে ঘ) দুটোই password
উত্তর: খ — private key দিয়ে পরিচয় প্রমাণিত হয়; তা ফাঁস হলে যে কেউ সার্ভারে ঢুকতে পারে। ক বিপজ্জনক; গ উল্টো; ঘ ভুল।

২১.২ `known_hosts`-এ সার্ভারের host key থাকার উদ্দেশ্য কী?
ক) গতি বাড়ানো খ) পরের সংযোগে সার্ভারটি আগের সার্ভারই কি না, তা যাচাই করা — মাঝখানে কেউ (man-in-the-middle) ঢুকলে সতর্ক করা গ) password মনে রাখা ঘ) ফাইল শেয়ার করা
উত্তর: খ — host key বদলে গেলে SSH সতর্ক করে; যাচাই ছাড়া এগিয়ে যাওয়া বিপজ্জনক। ক ভুল; গ ভুল; ঘ ভুল।

২১.৩ `scp` ও `rsync`-এর মধ্যে rsync-এর বড় সুবিধা কী?
ক) rsync দ্রুত নয় খ) rsync শুধু বদলে যাওয়া অংশ কপি করে, ফলে বড় ফাইল/ফোল্ডার বারবার কপিতে সময় ও ব্যান্ডউইথ বাঁচে গ) scp encrypt করে না ঘ) rsync password চায় না
উত্তর: খ — rsync incremental কপি করে। ক ভুল; গ ভুল, দুটোই SSH দিয়ে encrypt হয়; ঘ ভুল, দুটোতেই authentication লাগে।

২১.৪ `ssh user@host "df -h"` কী করে?
ক) সার্ভারে login করে interactive shell খোলে খ) সার্ভারে সংযোগ করে `df -h` কমান্ড চালিয়ে তার output ফিরিয়ে দেয়, তারপর সংযোগ বন্ধ করে গ) ফাইল কপি করে ঘ) সার্ভার বন্ধ করে
উত্তর: খ — কমান্ড দিলে SSH remote-এ তা চালিয়ে ফল ফেরত দেয়। ক ভুল, কমান্ড ছাড়া interactive shell খোলে; গ scp/rsync-এর কাজ; ঘ ভুল।

২১.৫ Private key ফাইলের permission কেমন হওয়া উচিত?
ক) 644 খ) 600 — শুধু মালিক পড়তে পারবে; বেশি খোলা permission থাকলে SSH key ব্যবহার করতে অস্বীকার করে গ) 777 ঘ) 755
উত্তর: খ — SSH নিরাপত্তার জন্য private key-এর permission কঠোরভাবে পরীক্ষা করে। ক, গ ও ঘ — সবই বেশি খোলা।

D.22 অধ্যায় ২২ — Building Reusable Automation Tools (প্রশ্ন ২২.১–২২.৫)

২২.১ CLI tool-এ `argparse` ব্যবহারের সুবিধা কী?
ক) গতি বাড়ে খ) argument parsing, সাহায্য বার্তা (`--help`), type যাচাই ও ভুল বার্তা স্বয়ংক্রিয়ভাবে পাওয়া যায় গ) ফাইল তৈরি হয় ঘ) log লেখে
উত্তর: খ — argparse হাতে parsing লেখার ভুল কমায়। ক গৌণ; গ ভুল; ঘ ভুল।

২২.২ Dry-run mode কী?
ক) পরীক্ষা ছাড়া চালানো খ) আসল পরিবর্তন না করে কী করা হতো, তা দেখানো — নিরাপদে যাচাই করার উপায় গ) ফাইল মুছে দেওয়া ঘ) দ্রুত চালানো
উত্তর: খ — dry-run দিয়ে production-এ চালানোর আগে প্রভাব দেখা যায়। ক উল্টো; গ ভুল; ঘ ভুল।

২২.৩ একটি automation tool-কে modular করার সুবিধা কী?
ক) ফাইল বেশি হয় খ) প্রতিটি অংশ আলাদাভাবে পরীক্ষা, পুনর্ব্যবহার ও বদলানো যায় — একটি অংশের ভুল পুরো tool নষ্ট করে না গ) গতি কমে ঘ) কোড লুকানো যায়
উত্তর: খ — modular কাঠামো রক্ষণাবেক্ষণ সহজ করে। ক ভুল, ফাইল বেশি হওয়া উদ্দেশ্য নয়; গ ভুল; ঘ ভুল।

২২.৪ Tool-এর প্রথম ধাপে configuration যাচাই (validate) করা কেন ভালো?
ক) সময় নষ্ট খ) ভুল config নিয়ে আধা কাজ করে তথ্য নষ্ট করার আগেই স্পষ্ট বার্তা দিয়ে থামা যায় (fail-fast) গ) log বড় হয় ঘ) token বাঁচে
উত্তর: খ — শুরুতে যাচাই করলে ব্যর্থতা আগেই ধরা পড়ে। ক ভুল; গ ভুল; ঘ ভুল।

২২.৫ Idempotent backup tool-এ দ্বিতীয়বার একই backup চালালে কী হওয়া উচিত?
ক) দ্বিগুণ ফাইল তৈরি হওয়া খ) একই ফল — আগের backup মুছে না গিয়ে বা নষ্ট না হয়ে, প্রয়োজনে নতুন সংস্করণ তৈরি হওয়া, কিন্তু ভুল অবস্থা তৈরি না হওয়া গ) পুরনো backup মুছে যাওয়া ঘ) error ছাড়া কিছু না হওয়া
উত্তর: খ — idempotency মানে পুনরায় চালানো নিরাপদ। ক ভুল; গ ভুল; ঘ ভুল, নতুন backup তৈরি হওয়া স্বাভাবিক হতে পারে, কিন্তু অবস্থা সামঞ্জস্যপূর্ণ থাকতে হবে।

D.23 অধ্যায় ২৩ — Code Quality, Testing and Documentation (প্রশ্ন ২৩.১–২৩.৫)

২৩.১ pytest-এ test function-এর নাম কেমন হতে হয়?
ক) যেকোনো নাম খ) `test_` দিয়ে শুরু হতে হয়, যাতে pytest তা খুঁজে পায় গ) `check_` দিয়ে ঘ) বড় হাতের অক্ষরে
উত্তর: খ — pytest ডিফল্টে `test_*.py` ফাইল ও `test_*` function খোঁজে। ক ভুল, নাম না মিললে test চলবে না; গ ও ঘ ভুল।

২৩.২ Mocking কেন ব্যবহার করা হয়?
ক) গতি বাড়াতে খ) বাইরের জিনিস (API, কমান্ড, ফাইল সিস্টেম) ছাড়া test চালাতে — আসল সার্ভারে আঘাত না করে আচরণ যাচাই করা যায় গ) কোড লুকাতে ঘ) test মুছতে
উত্তর: খ — mock test-কে দ্রুত, নির্ভরযোগ্য ও নিরাপদ করে। ক গৌণ সুবিধা, মূল কারণ নয়; গ ভুল; ঘ ভুল।

২৩.৩ ShellCheck ও Python linting (যেমন ruff) চালানোর উদ্দেশ্য কী?
ক) কোড দ্রুত করা খ) চালানোর আগেই সাধারণ ভুল, শৈলীর সমস্যা ও বিপজ্জনক প্যাটার্ন ধরা গ) ফাইল compress করা ঘ) test লেখা
উত্তর: খ — linter static analysis করে। ক গৌণ; গ ভুল; ঘ ভুল।

২৩.৪ ভালো README-তে কী থাকা উচিত?
ক) শুধু নাম খ) উদ্দেশ্য, install ধাপ, ব্যবহারবিধি, উদাহরণ, config ও troubleshooting গ) password ঘ) সব কোড
উত্তর: খ — README ব্যবহারকারীর প্রথম নির্দেশিকা। ক অসম্পূর্ণ; গ নিরাপত্তা লঙ্ঘন; ঘ ভুল, কোড ফাইলে থাকে।

২৩.৫ Integration test ও unit test-এর পার্থক্য কী?
ক) একই জিনিস খ) unit test একটি ছোট অংশ একা পরীক্ষা করে; integration test একাধিক অংশ একসঙ্গে কাজ করে কি না, তা পরীক্ষা করে গ) unit test ধীর ঘ) integration test কোড পড়ে না
উত্তর: খ — দুটি আলাদা স্তরের পরীক্ষা। ক ভুল; গ উল্টো, unit test সাধারণত দ্রুত; ঘ ভুল।

D.24 অধ্যায় ২৪ — Automation Architecture and Production Practices (প্রশ্ন ২৪.১–২৪.৫)

২৪.১ একটি সহজ কাজের জন্য Bash ও জটিল তথ্য প্রক্রিয়ার জন্য Python — এই পছন্দের যুক্তি কী?
ক) Bash সবসময় ভালো খ) Bash কমান্ড একত্র করতে দ্রুত ও সহজ; Python জটিল তথ্য, API ও error handling-এ শক্তিশালী ও পড়তে সহজ গ) Python সবসময় ভালো ঘ) ভাষা কোনো ব্যাপার নয়
উত্তর: খ — সঠিক tool বেছে নেওয়া engineering সিদ্ধান্ত। ক ও গ একপেশে; ঘ ভুল, ভাষার শক্তি কাজের সঙ্গে মেলাতে হয়।

২৪.২ Production automation-এ observability বলতে কী বোঝায়?
ক) শুধু log লেখা খ) log, metric ও status-এর মাধ্যমে automation কী করছে, কোথায় ব্যর্থ হচ্ছে, তা দেখা ও বোঝা যায় গ) পর্দায় দেখা ঘ) ফাইল গোনা
উত্তর: খ — observability ছাড়া ব্যর্থতা অন্ধকারে থাকে। ক অসম্পূর্ণ, log তার একটি অংশ; গ ভুল; ঘ ভুল।

২৪.৩ Production-এ secret (API key, password) রাখার নিরাপদ উপায় কোনটি?
ক) code-এ লেখা খ) environment variable বা secret manager-এ, code ও Git-এর বাইরে গ) README-তে ঘ) chat-এ পাঠানো
উত্তর: খ — secret কখনো code/repository-তে রাখা যায় না। ক, গ ও ঘ — সবই ফাঁসের পথ।

২৪.৪ Automation ব্যর্থ হলে সবচেয়ে নিরাপদ আচরণ কোনটি?
ক) চুপচাপ এগিয়ে যাওয়া খ) স্পষ্টভাবে log করে, nonzero exit code দিয়ে থামা ও আংশিক পরিবর্তন না রাখা (বা rollback করা) গ) আবার চেষ্টা করা ঘ) ফাইল মুছে দেওয়া
উত্তর: খ — নিরাপদ ব্যর্থতা (safe failure) তথ্য নষ্ট হওয়া ঠেকায়। ক বিপজ্জনক; গ সবসময় নিরাপদ নয়; ঘ ভুল।

২৪.৫ একটি automation tool production-এ দেওয়ার আগে কী যাচাই করা উচিত?
ক) শুধু নিজের কম্পিউটারে চললেই হবে খ) test, dry-run, config validation, log, error handling, idempotency ও rollback পরিকল্পনা — সব যাচাই করা গ) কোড লম্বা কি না ঘ) নাম সুন্দর কি না
উত্তর: খ — production-এ যাওয়ার আগে এই তালিকা সম্পূর্ণ করতে হয়। ক ভুল, পরিবেশ ভিন্ন হয়; গ ও ঘ তুচ্ছ।

> **PRO TIP:** ১২০টি প্রশ্নের মধ্যে কোনো অধ্যায়ে বারবার ভুল হলে সেই অধ্যায়ের সংশ্লিষ্ট lab আবার করুন — শুধু ব্যাখ্যা পড়ে নয়, হাতে চালিয়ে শিখুন।

D.25 Short-Answer Questions — সংক্ষিপ্ত প্রশ্ন ও মডেল উত্তর

১. Git-এ snapshot বলতে কী বোঝায়? Delta-ভিত্তিক ব্যবস্থা থেকে এটি কীভাবে আলাদা?
মডেল উত্তর: Git প্রতিটি commit-এ পুরো প্রজেক্টের snapshot সংরক্ষণ করে — প্রতিটি ফাইলের বিষয়বস্তুর hash (blob) ও directory কাঠামো (tree)। ফাইল না বদলালে আগের blob-এর দিকে pointer দেওয়া হয়, তাই জায়গা নষ্ট হয় না। Delta-ভিত্তিক ব্যবস্থা (যেমন পুরনো CVS) শুধু পরিবর্তনের পার্থক্য রাখে, ফলে কোনো সংস্করণ পেতে সব পার্থক্য একত্র করতে হয়। Git-এ যেকোনো commit সরাসরি তার snapshot থেকে পাওয়া যায়, তাই checkout ও ইতিহাস দেখা দ্রুত হয়।

২. `git rebase` কখন ব্যবহার করা নিরাপদ, আর কখন বিপজ্জনক?
মডেল উত্তর: নিজের local, এখনো push করা হয়নি এমন branch-এ rebase নিরাপদ — এতে ইতিহাস পরিষ্কার হয়। অন্যরা ব্যবহার করছে এমন shared/public branch-এ rebase বিপজ্জনক, কারণ এটি commit-এর hash বদলে দেয়; অন্যদের local ইতিহাসের সঙ্গে সংঘাত হয়। Shared branch-এ merge ব্যবহার করতে হয়।

৩. Bash-এ `set -euo pipefail` কেন প্রায় সব production script-এর শুরুতে লেখা হয়?
মডেল উত্তর: `-e` ব্যর্থ কমান্ডে script থামায়, `-u` অনির্ধারিত ভেরিয়েবল ব্যবহারে error দেয়, `pipefail` pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline-কে ব্যর্থ ধরে। তিনটি মিলে ভুল চুপচাপ এগিয়ে যাওয়া ঠেকায়, ফলে ব্যর্থতা আগেই ও স্পষ্টভাবে ধরা পড়ে।

৪. Bash-এ unquoted ভেরিয়েবল (`$file`) কেন বিপজ্জনক? উদাহরণ দিন।
মডেল উত্তর: unquoted expansion-এ word splitting ও globbing হয়। যেমন ফাইলের নাম `my report.txt` হলে `rm $file` দুটি নাম — `my` ও `report.txt` — হিসেবে ব্যাখ্যা হতে পারে, ফলে ভুল ফাইল মোছা বা error হতে পারে। সমাধান: সবসময় `"$file"` লেখা।

৫. Python-এ `subprocess.run()`-এ `shell=True` কেন এড়াতে হয়?
মডেল উত্তর: `shell=True` কমান্ড string-কে shell দিয়ে ব্যাখ্যা করে, ফলে ব্যবহারকারীর দেওয়া মানের মধ্যে `; rm -rf` বা `$(...)` থাকলে তা আলাদা কমান্ড হিসেবে চলতে পারে — command injection। নিরাপদ উপায়: argument list (`["ls", "-l", path]`) ও `shell=False` (ডিফল্ট), যাতে প্রতিটি argument আক্ষরিকভাবে program-এ যায়।

৬. API-তে pagination কেন থাকে এবং client-কে কী করতে হয়?
মডেল উত্তর: বড় তালিকা একবারে পাঠালে সার্ভার ও নেটওয়ার্কের উপর চাপ পড়ে, তাই API ফলাফলকে পৃষ্ঠায় ভাগ করে (যেমন `?page=2` বা cursor)। Client-কে next page-এর লিংক/সংকেত ধরে সব পৃষ্ঠা একত্র করতে হয়, প্রতিটি request-এ timeout ও status যাচাই করে এবং rate limit মেনে চলতে হয়।

৭. Cron job terminal-এ চলে কিন্তু cron-এ ব্যর্থ হয় — সবচেয়ে সাধারণ কারণগুলো কী?
মডেল উত্তর: cron ন্যূনতম environment নিয়ে চলে — PATH ছোট, বর্তমান directory আলাদা, ভেরিয়েবল নেই। সমাধান: script-এ সব কমান্ডের পূর্ণ পথ ব্যবহার করা, script-এর শুরুতে প্রয়োজনীয় ভেরিয়েবল সেট করা, stdout/stderr log ফাইলে redirect করা এবং `flock` দিয়ে ওভারল্যাপ ঠেকানো।

৮. SSH-এ host key বদলে গেলে সতর্কবার্তা এলে কী করা উচিত এবং কী করা উচিত নয়?
মডেল উত্তর: থামতে হবে ও কারণ খুঁজতে হবে — সার্ভার আবার install হয়েছে কি না, নাকি মাঝখানে কেউ ঢুকেছে। সার্ভার admin-এর সঙ্গে যাচাই করে তবেই `known_hosts` থেকে পুরনো key সরাতে হয়। যাচাই ছাড়া `StrictHostKeyChecking=no` দিয়ে এগিয়ে যাওয়া উচিত নয়, কারণ তাতে man-in-the-middle আক্রমণের ঝুঁকি থাকে।

৯. Idempotent automation বলতে কী বোঝায়? একটি উদাহরণ দিন।
মডেল উত্তর: একই কাজ একাধিকবার চালালে প্রতিবার একই ফল হয়, ক্ষতি হয় না। উদাহরণ: backup script-এ directory আগে থেকে থাকলে আবার তৈরি করার চেষ্টা না করা (`mkdir -p` বা `exist_ok=True`), অথবা user তৈরি করার আগে user আছে কি না পরীক্ষা করা। এতে cron-এ বারবার চলা নিরাপদ হয়।

১০. একটি automation tool-কে Bash-এর বদলে Python-এ লেখা কখন ভালো সিদ্ধান্ত?
মডেল উত্তর: যখন কাজে জটিল তথ্য কাঠামো (JSON/YAML), API call, জটিল শর্ত ও error handling, unit test বা একাধিক ফাইলের কাঠামো লাগে — তখন Python পড়তে সহজ, পরীক্ষা করা সহজ ও নিরাপদ। সহজ কমান্ড একত্র করা বা ছোট system কাজের জন্য Bash যথেষ্ট ও দ্রুত।

D.26 Git Command Interpretation — Git কমান্ড ব্যাখ্যা

১. `git log --oneline --graph --all --decorate`
উত্তর: সব branch-এর commit ইতিহাস সংক্ষেপে (এক লাইনে), branch/merge-এর গ্রাফ আকারে, branch ও tag-এর নামসহ দেখায়। `--graph` শাখা বিভাজন ও মিলন দেখায়, `--decorate` কোন commit-এ কোন branch/tag আছে তা লেখে। ব্যবহার: ইতিহাসের কাঠামো বোঝা।

২. `git diff --staged`
উত্তর: staging area-তে তোলা পরিবর্তন ও শেষ commit-এর মধ্যে পার্থক্য দেখায় — অর্থাৎ পরের commit-এ ঠিক কী যাবে। `git diff` (কোনো flag ছাড়া) দেখায় working directory ও staging area-র পার্থক্য। দুটি গুলিয়ে ফেললে ভুল জিনিস review হয়।

৩. `git checkout -b feature/login`
উত্তর: বর্তমান commit থেকে `feature/login` নামে নতুন branch তৈরি করে এবং তাতে চলে যায়। `git switch -c feature/login` একই কাজের আধুনিক রূপ। মনে রাখবেন: branch তৈরি হয় বর্তমান HEAD থেকে, main থেকে নয় — যদি main-এ না থাকেন, ভুল base থেকে branch তৈরি হবে।

৪. `git push --force-with-lease origin main`
উত্তর: local main-কে remote-এ জোর করে পাঠায়, তবে শর্ত থাকে: remote-এ শেষ দেখা অবস্থার পর কেউ নতুন commit পাঠায়নি। `--force`-এর চেয়ে নিরাপদ, কারণ অন্যের কাজ অজান্তে মুছে যাওয়ার ঝুঁকি কমে। তবু shared main-এ ব্যবহার করা উচিত নয় — শুধু নিজের branch-এ, ইতিহাস ঠিক করার পর।

৫. `git stash push -m "wip: config" --keep-index`
উত্তর: working directory-র unstaged পরিবর্তনগুলো সাময়িকভাবে সরিয়ে রাখে (stash), কিন্তু staged পরিবর্তন রেখে দেয় (`--keep-index`)। ফলে staged কাজ আলাদাভাবে commit বা পরীক্ষা করা যায়। পরে `git stash pop` দিয়ে ফিরিয়ে আনা যায়।

D.27 Bash Code Reading — Bash কোড পড়া

১. নিচের কোডের output কী হবে?

```
count=0
for f in *.txt; do
  count=$((count + 1))
done
echo "$count"
```

উত্তর: বর্তমান directory-তে যতগুলো `.txt` ফাইল আছে, সেই সংখ্যা। তবে কোনো `.txt` ফাইল না থাকলে (nullglob ছাড়া) pattern আক্ষরিক `*.txt` হিসেবে একবার চলবে, ফলে output হবে `1` — এটি একটি ফাঁদ; নিরাপদ উপায় `shopt -s nullglob` বা ফাইল আছে কি না পরীক্ষা করা।

২. নিচের কোডে কী ভুল আছে?

```
if [ $status = "ok" ]; then
  echo "fine"
fi
```

উত্তর: `$status` quote করা হয়নি। `status` খালি থাকলে `[ = "ok" ]` হয়ে syntax error হয়; space থাকলে ভেঙে যায়। সঠিক রূপ: `if [ "$status" = "ok" ]; then`। আধুনিক Bash-এ `[[ "$status" == "ok" ]]`-ও নিরাপদ।

৩. নিচের কোডটি কী করে?

```
while IFS= read -r line; do
  echo "$line"
done < report.txt
```

উত্তর: `report.txt`-এর প্রতিটি লাইন একে একে পড়ে ও ছাপে। `IFS=` লাইনের শুরু/শেষের space রক্ষা করে, `-r` backslash আক্ষরিক রাখে। `done < report.txt` ফাইলটিকে loop-এর input হিসেবে দেয় — pipe ব্যবহার করলে subshell সমস্যা (loop-এর ভেতরে সেট করা ভেরিয়েবল বাইরে হারিয়ে যাওয়া) হতে পারে।

৪. নিচের function-টি কী ফেরত দেয়?

```
check_disk() {
  df -h / | awk 'NR==2 {print $5}'
}
usage=$(check_disk)
echo "$usage"
```

উত্তর: root filesystem-এর ব্যবহৃত শতাংশ, যেমন `42%`। function stdout-এ print করে, আর `$(...)` সেই stdout ধরে `usage` ভেরিয়েবলে রাখে। মনে রাখবেন: function-এর `return` শুধু exit status দেয়; মান ফেরত দিতে এভাবেই stdout ব্যবহার করতে হয়।

৫. নিচের লাইনটি কেন বিপজ্জনক?

```
rm -rf $BACKUP_DIR/*
```

উত্তর: `$BACKUP_DIR` খালি বা unset থাকলে এটি হয়ে যায় `rm -rf /*` — অর্থাৎ পুরো filesystem মোছার চেষ্টা। নিরাপদ রূপ: `set -u` ব্যবহার করা, ভেরিয়েবল খালি কি না পরীক্ষা করা (`: "${BACKUP_DIR:?missing}"`), quote করা (`"$BACKUP_DIR"/*`) এবং মোছার আগে dry-run বা তালিকা দেখা।

D.28 Python Debugging Exercises — Python debugging অনুশীলন

১. ভুল কোড (ফাইল বন্ধ হয় না, exception-এ ফাইল খোলা থেকে যায়):

```
f = open("data.txt")
data = f.read()
print(data)
```

সংশোধিত কোড:

```
with open("data.txt", encoding="utf-8") as f:
    data = f.read()
print(data)
```

ব্যাখ্যা: প্রথম কোডে `f.close()` লেখা হয়নি; exception হলে ফাইল খোলাই থেকে যায়। `with` block শেষ হলে — এমনকি exception হলেও — ফাইল বন্ধ হয়। `encoding="utf-8"` দেওয়ায় বিভিন্ন পরিবেশে encoding নিয়ে সমস্যা হয় না।

২. ভুল কোড (KeyError):

```
config = {"host": "localhost"}
port = config["port"]
```

সংশোধিত কোড:

```
config = {"host": "localhost"}
port = config.get("port", 8080)
```

ব্যাখ্যা: `config["port"]` key না থাকলে `KeyError` তোলে। `.get()` key না থাকলে default মান (এখানে 8080) দেয়, ফলে প্রোগ্রাম থামে না। যদি অনুপস্থিত key আসলেই ত্রুটি হয়, তবে স্পষ্ট বার্তাসহ যাচাই করে থামাই ভালো।

৩. ভুল কোড (shell injection):

```
import subprocess
name = input("Enter filename: ")
subprocess.run("ls -l " + name, shell=True)
```

সংশোধিত কোড:

```
import subprocess
name = input("Enter filename: ")
subprocess.run(["ls", "-l", name], check=True)
```

ব্যাখ্যা: প্রথম কোডে ব্যবহারকারী `file.txt; rm -rf /`-এর মতো input দিলে shell তা আলাদা কমান্ড হিসেবে চালাবে। তালিকা আকারে argument দিলে (`shell=False`, ডিফল্ট) input শুধু একটি argument হিসেবে যায়। `check=True` ব্যর্থ হলে exception তোলে।

৪. ভুল কোড (timeout ছাড়া API call, exception ধরা হয় না):

```
import requests
resp = requests.get("https://api.example.com/health")
print(resp.json())
```

সংশোধিত কোড:

```
import requests
try:
    resp = requests.get("https://api.example.com/health", timeout=10)
    resp.raise_for_status()
    print(resp.json())
except requests.RequestException as exc:
    print(f"API call failed: {exc}")
```

ব্যাখ্যা: প্রথম কোডে সার্ভার সাড়া না দিলে প্রোগ্রাম অনির্দিষ্টকাল ঝুলে থাকে, আর 404/500 status-এও `.json()` ভুলভাবে চালানো হয়। সংশোধিত কোডে timeout আছে, `raise_for_status()` খারাপ status-এ exception তোলে এবং `RequestException` ধরে স্পষ্ট বার্তা দেওয়া হয়।

৫. ভুল কোড (mutable default argument):

```
def add_error(errors=[]):
    errors.append("disk full")
    return errors
```

সংশোধিত কোড:

```
def add_error(errors=None):
    if errors is None:
        errors = []
    errors.append("disk full")
    return errors
```

ব্যাখ্যা: Python-এ default list একবারই তৈরি হয় ও সব call-এ ভাগ হয় — ফলে প্রতিবার call করলে তালিকা বাড়তেই থাকে। `None` default দিয়ে ভেতরে নতুন list বানালে প্রতিটি call আলাদা তালিকা পায়। এটি Python-এর একটি বিখ্যাত ফাঁদ।

D.29 Script-Writing Challenges — স্ক্রিপ্ট লেখার চ্যালেঞ্জ

চ্যালেঞ্জ ১ (Beginner — Bash): Disk সতর্কতা স্ক্রিপ্ট। একটি Bash script লিখুন যা root filesystem-এর ব্যবহার শতাংশ পরীক্ষা করে: 80%-এর বেশি হলে WARNING, 90%-এর বেশি হলে CRITICAL বার্তা stderr-এ লিখে exit code 2 দিয়ে থামবে, নাহলে OK লিখে 0 দিয়ে থামবে।
সমাধানের রূপরেখা: `set -euo pipefail` দিয়ে শুরু করুন; `df --output=pcent / | tail -1 | tr -d ' %'`

দিয়ে সংখ্যা নিন; সংখ্যা কি না যাচাই করুন; `-ge 90` ও `-ge 80` দিয়ে if/elif লিখুন; বার্তা stderr-এ (`>&2`) লিখুন; `shellcheck` দিয়ে যাচাই করুন।

চ্যালেঞ্জ ২ (Intermediate — Python): Log বিশ্লেষণ tool। একটি Python CLI লিখুন যা একটি log ফাইল পড়ে প্রতিটি log level (INFO, WARNING, ERROR) কতবার এসেছে, তা গোনে ও JSON-এ লিখে।
সমাধানের রূপরেখা: `argparse` দিয়ে input/output পথ নিন; `pathlib` দিয়ে ফাইল আছে কি না যাচাই করুন; `with open(..., encoding="utf-8")` দিয়ে লাইন ধরে পড়ুন; `collections.Counter` দিয়ে গুনুন; `json.dump()` দিয়ে ফল লিখুন; `logging` দিয়ে অগ্রগতি জানান; pytest দিয়ে একটি ছোট test লিখুন।

চ্যালেঞ্জ ৩ (Advanced — Bash + Python): Backup ও যাচাই। একটি tool বানান যা একটি directory-র `tar.gz` backup নেয়, SHA-256 checksum লেখে, checksum মিলিয়ে যাচাই করে এবং ফলাফল structured log-এ লেখে।
সমাধানের রূপরেখা: Bash wrapper-এ argument যাচাই, `mktemp -d` ও trap cleanup, `tar -czf`; Python অংশে `hashlib.sha256()` দিয়ে checksum গণনা, JSON log লেখা, exit code-ভিত্তিক সফল/ব্যর্থতা; dry-run flag যোগ করুন; idempotency রাখুন — একই backup আবার চালালে পুরনোটি নষ্ট হবে না।

D.30 Troubleshooting Case Studies — সমস্যা সমাধানের কেস স্টাডি

কেস ১: `git push` rejected — "non-fast-forward"।
লক্ষণ: push করলে Git বলে remote-এ আপনার local-এ নেই এমন commit আছে। নির্ণয়: অন্য কেউ (বা আপনি অন্য কম্পিউটার থেকে) main-এ push করেছে; আপনার local ইতিহাস পিছিয়ে আছে। সমাধান: প্রথমে `git fetch` করে দেখুন, তারপর `git pull --rebase` (নিজের branch-এ) বা `git pull` করে merge করুন; conflict থাকলে সমাধান করে commit করুন, তারপর push করুন। `--force` দিয়ে অন্যের কাজ মুছে ফেলবেন না।

কেস ২: Cron job চলে না, কিন্তু terminal-এ script চলে।
লক্ষণ: crontab-এ job আছে, কিন্তু log ফাইল তৈরি হয় না। নির্ণয়: (ক) cron-এর PATH-এ কমান্ড নেই — script-এ relative পথ বা `python` ব্যবহার করা হয়েছে; (খ) output redirect করা হয়নি, তাই ব্যর্থতা দেখা যাচ্ছে না; (গ) script-এ execute permission নেই বা shebang ভুল। সমাধান: সব পথ absolute করুন (`/usr/bin/python3`), crontab-এ `>> /var/log/job.log 2>&1` যোগ করুন, `chmod +x` ও shebang যাচাই করুন এবং `grep CRON /var/log/syslog` দিয়ে cron কী করেছে দেখুন।

কেস ৩: Python script-এ `ModuleNotFoundError: requests`, অথচ pip install করা হয়েছে।
লক্ষণ: terminal-এ `pip install requests` সফল, কিন্তু script চালালে module পাওয়া যায় না। নির্ণয়: pip ও python আলাদা পরিবেশের — system pip-এ install হয়েছে, কিন্তু script অন্য venv বা `python3`-এ চলছে। সমাধান: `which python3` ও `python3 -m pip --version` মিলিয়ে দেখুন; সঠিক venv activate করে `python -m pip install -r requirements.txt` চালান; script-এর shebang যাচাই করুন (`#!/usr/bin/env python3` ভুল venv ধরতে পারে)।

D.31 Practical Final Examination — ব্যবহারিক চূড়ান্ত পরীক্ষা

সময়: ৩ ঘণ্টা। মোট নম্বর: ১০০। পাস নম্বর: ৬০। পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। পরীক্ষার সময় ইন্টারনেট ব্যবহার করা যাবে, কিন্তু অন্যের কোড কপি করা যাবে না।

অংশ ক — MCQ পুনরালোচনা (২০ নম্বর)

অধ্যায় ১–২৪-এর ১২০টি MCQ থেকে ২০টি প্রশ্ন দেওয়া হবে (প্রতিটি ১ নম্বর)। প্রতিটি অধ্যায়-গোষ্ঠী (Git, Bash, Python, automation, workflow) থেকে প্রশ্ন থাকবে। উত্তরপত্রে শুধু সঠিক বিকল্প লিখলেই হবে, ব্যাখ্যা লাগবে না।

অংশ খ — হাতে-কলমে কাজ (৮০ নম্বর)

কাজ ১ — Git repository ও branch (২০ নম্বর): `exam-toolkit` নামে একটি repository তৈরি করুন; `README.md` ও `.gitignore` যোগ করে commit করুন; `feature/health-check` branch-এ একটি ফাইল যোগ করুন; main-এ একটি আলাদা commit করুন; branch-টি main-এ merge করুন; `v1.0.0` tag দিন; `git log --oneline --graph --all --decorate`-এর output দেখান।

কাজ ২ — Bash health script (২০ নম্বর): একটি Bash script লিখুন যা CPU load, memory ব্যবহার, disk ব্যবহার ও একটি service-এর status পরীক্ষা করে; ফলাফল পরিষ্কারভাবে ছাপে; কোনো পরীক্ষা ব্যর্থ হলে nonzero exit code দেয়; `set -euo pipefail`, quoting ও input validation ব্যবহার করে; `shellcheck`-এ কোনো error থাকে না।

কাজ ৩ — Python API ও logging tool (২০ নম্বর): একটি Python CLI লিখুন যা একটি config ফাইল (JSON/YAML) পড়ে, একটি public HTTP API-তে timeout ও error handling-সহ request পাঠায়, JSON উত্তর প্রক্রিয়া করে, ফলাফল ফাইলে লেখে এবং structured logging ব্যবহার করে। venv ও `requirements.txt` থাকতে হবে; secret code-এ লেখা যাবে না।

কাজ ৪ — Cron job সেটআপ (২০ নম্বর): কাজ ২-এর script-টি প্রতি ৩০ মিনিটে চালানোর জন্য crontab entry লিখুন (absolute পথসহ), output log ফাইলে redirect করুন, `flock` দিয়ে ওভারল্যাপ ঠেকান এবং cron expression-এর প্রতিটি field-এর অর্থ ব্যাখ্যা করুন।

মূল্যায়নের রুব্রিক

মানদণ্ড	চমৎকার (৯০–১০০%)	ভালো (৭০–৮৯%)	পাস (৬০–৬৯%)	অসম্পূর্ণ (৬০%-এর নিচে)
সঠিকতা	সব কাজ সঠিকভাবে চলে, output যাচাই করা যায়	ছোট ভুল আছে, মূল কাজ চলে	মূল কাজ আংশিক চলে	কাজ চলে না বা ভুল ফল দেয়
নিরাপত্তা	quoting, shell=False, secret বাইরে, permission সঠিক	ছোট একটি নিরাপত্তা ঘাটতি	একাধিক ঘাটতি, কিন্তু বিপজ্জনক নয়	injection বা secret ফাঁসের ঝুঁকি
Error handling	ব্যর্থতা log হয়, exit code সঠিক, cleanup হয়	বেশিরভাগ ব্যর্থতা ধরা হয়	আংশিক handling	ব্যর্থতা চুপচাপ পেরিয়ে যায়
গঠন ও নথি	modular কোড, README, পরিষ্কার নাম	মোটামুটি গঠন, কিছু নথি	এক ফাইলে কাজ, নথি কম	অগোছালো, নথি নেই
পরীক্ষা	pytest/ShellCheck চালানো হয়েছে, প্রমাণ আছে	কিছু পরীক্ষা আছে	হাতে যাচাইয়ের উল্লেখ আছে	কোনো যাচাই নেই

> **WARNING:** পরীক্ষায় কখনো অন্যের repository-তে `--force` push করবেন না, production সার্ভারে পরীক্ষামূলক script চালাবেন না এবং কোনো secret (token, password) উত্তরপত্রে লিখবেন না।

পরিশিষ্ট E • Glossary

Appendix E: Glossary — পরিভাষা (English–Bangla)

এই পরিশিষ্টে বইয়ে ব্যবহৃত ১৯৭টি গুরুত্বপূর্ণ পরিভাষা ইংরেজি বর্ণানুক্রমে দেওয়া হয়েছে। প্রতিটি শব্দের পাশে এক বা দুই লাইনে বাংলায় অর্থ লেখা হয়েছে, যাতে পড়ার সময় দ্রুত দেখে নেওয়া যায়।

Term	বাংলা অর্থ
API (Application Programming Interface)	দুটি প্রোগ্রামের মধ্যে তথ্য আদান-প্রদানের নিয়ম ও মাধ্যম; যেমন একটি সার্ভার থেকে তথ্য নেওয়ার নির্দিষ্ট পথ।
Argument	কমান্ড বা function-কে দেওয়া মান; যেমন `ls -l`-এ `-l` একটি argument।
Array	একটি নামের অধীনে সাজানো মানের তালিকা; Bash-এ `files=(a.txt b.txt)` একটি array।
Artifact	কোনো কাজের ফল হিসেবে তৈরি ফাইল বা প্যাকেজ; যেমন build-এর পর তৈরি হওয়া ফাইল।
Authentication	পরিচয় যাচাই করা — আপনি আসলেই সেই ব্যবহারকারী কি না, তা প্রমাণ করা।
Authorization	অনুমতি যাচাই করা — পরিচয় প্রমাণিত ব্যবহারকারী কী করতে পারবে, তা নির্ধারণ করা।
Automation	মানুষের বারবার হস্তক্ষেপ ছাড়া কাজ স্বয়ংক্রিয়ভাবে করানো।
awk	কলামভিত্তিক text প্রক্রিয়াকরণের শক্তিশালী Linux tool; field আলাদা করে হিসাব করা যায়।
Backup	তথ্য হারালে ফিরিয়ে আনার জন্য রাখা অনুলিপি।
Bash	Linux-এর জনপ্রিয় shell ও scripting ভাষা; এই বইয়ের প্রধান scripting ভাষা।
Base branch	যে branch থেকে নতুন branch তৈরি হয় বা যেখানে merge করা হয়; সাধারণত main।
Binary	শুধু ০ ও ১ দিয়ে প্রকাশিত তথ্য; মানুষের পড়ার মতো text নয় এমন ফাইল।
Blob	Git object, যা একটি ফাইলের বিষয়বস্তু সংরক্ষণ করে; নাম থাকে না, শুধু content থাকে।
Branch	একটি commit-কে নির্দেশ করা pointer; মূল কাজ থেকে আলাদা করে কাজ করার পথ।
Break	loop থেকে সঙ্গে সঙ্গে বেরিয়ে আসার নির্দেশ।
Bug	প্রোগ্রামের ভুল, যার ফলে ভুল ফলাফল বা অপ্রত্যাশিত আচরণ হয়।
Build	source code থেকে চালানোর উপযোগী প্রোগ্রাম বা প্যাকেজ তৈরির প্রক্রিয়া।
Cache	বারবার লাগে এমন তথ্য দ্রুত পাওয়ার জন্য রাখা সাময়িক সংরক্ষণ।
Callback	কোনো ঘটনা ঘটলে পরে চালানোর জন্য দেওয়া function।
Case statement	Bash-এ একই মানের অনেকগুলো pattern-এর মধ্যে একটিকে বেছে নেওয়ার গঠন।

Certificate	পরিচয় ও নিরাপত্তা প্রমাণের ডিজিটাল নথি; যেমন TLS certificate।
Cherry-pick	অন্য branch থেকে নির্দিষ্ট একটি commit বেছে এনে বর্তমান branch-এ প্রয়োগ করা।
CI/CD	Continuous Integration / Continuous Delivery — কোড যাচাই ও প্রকাশ স্বয়ংক্রিয় করার পদ্ধতি।
CLI (Command-Line Interface)	লেখা কমান্ড দিয়ে প্রোগ্রাম চালানোর মাধ্যম; GUI-এর বিপরীত।
Clone	remote repository-র সম্পূর্ণ অনুলিপি নিজের কম্পিউটারে নেওয়া।
Cloud	ইন্টারনেটের মাধ্যমে ভাড়া নেওয়া computing সেবা (সার্ভার, storage ইত্যাদি)।
Code review	merge-এর আগে অন্য developer-এর কোড পড়ে ভুল, শৈলী ও নিরাপত্তা যাচাই করা।
Commit	Git-এ staging area-র snapshot স্থায়ীভাবে ইতিহাসে সংরক্ষণ করা।
Commit message	commit-এর সঙ্গে লেখা সংক্ষিপ্ত বিবরণ — কী বদলেছে ও কেন।
Compiler	পুরো source code একবারে machine code-এ রূপান্তরকারী প্রোগ্রাম।
Condition	সত্য/মিথ্যা যাচাইয়ের শর্ত, যার উপর ভিত্তি করে if/while সিদ্ধান্ত নেয়।
Configuration	পরিবেশভেদে বদলায় এমন সেটিং (URL, পথ, সীমা) — code থেকে আলাদা রাখা হয়।
Conflict	একই জায়গায় দুটি ভিন্ন পরিবর্তন থাকায় Git-এর স্বয়ংক্রিয় merge না পারা।
Container	অ্যাপ্লিকেশন ও তার dependency-কে একসঙ্গে প্যাকেজ করার হালকা বিচ্ছিন্ন পরিবেশ।
Continue	loop-এর বর্তমান চক্র বাদ দিয়ে পরের চক্রে যাওয়ার নির্দেশ।
Cron	Linux-এ নির্দিষ্ট সময়ে কাজ চালানোর সময়সূচি ব্যবস্থা।
Crontab	cron job-এর তালিকা ধারণকারী ফাইল; প্রতিটি লাইনে সময় ও কমান্ড থাকে।
Curl	command line থেকে HTTP request পাঠানোর tool।
Daemon	পেছনে চলমান সেবা প্রক্রিয়া; যেমন web server বা cron daemon।
Dashboard	system-এর অবস্থা এক নজরে দেখার সারসংক্ষেপ পর্দা।
Database	কাঠামোবদ্ধভাবে তথ্য সংরক্ষণ ও খোঁজার ব্যবস্থা।
Debugging	প্রোগ্রামের ভুল খুঁজে বের করে ঠিক করার প্রক্রিয়া।
Default value	মান না দিলে স্বয়ংক্রিয়ভাবে ব্যবহৃত মান।
Dependency	আপনার প্রোগ্রাম চালাতে প্রয়োজনীয় অন্য library বা tool।
Deployment	কোড সার্ভারে বসিয়ে ব্যবহারকারীদের জন্য চালু করা।
Detached HEAD	HEAD যখন কোনো branch-এর দিকে নয়, সরাসরি কোনো commit-কে নির্দেশ করে — এমন অবস্থা।

Dictionary	Python-এ key–value জোড়ায় তথ্য রাখার কাঠামো; যেমন `{"host": "localhost"}`।
Directory	ফাইল সাজিয়ে রাখার ফোল্ডার।
Disk usage	disk-এর কতটুকু জায়গা ব্যবহৃত হয়েছে, তার পরিমাপ।
Distributed VCS	প্রতিটি অনুলিপিতে পূর্ণ ইতিহাস থাকে এমন version control ব্যবস্থা; যেমন Git।
Docker	container তৈরি ও চালানোর জনপ্রিয় platform।
Documentation	কোনো tool বা প্রজেক্ট কীভাবে ব্যবহার করতে হয়, তার লিখিত নির্দেশনা।
Dry run	আসল পরিবর্তন না করে কী করা হতো, তা দেখানো — নিরাপদ যাচাইয়ের পদ্ধতি।
Encryption	তথ্যকে এমনভাবে বদলে দেওয়া, যাতে শুধু অনুমোদিত পক্ষ পড়তে পারে।
Endpoint	API-র নির্দিষ্ট ঠিকানা, যেখানে request পাঠাতে হয়; যেমন `/api/users`।
Environment variable	operating system বা shell থেকে প্রোগ্রামে পৌঁছে দেওয়া সেটিং মান; যেমন `HOME`।
Error handling	ত্রুটি ঘটলে প্রোগ্রাম ভেঙে না পড়ে নিরাপদে সামলানোর ব্যবস্থা।
Exception	Python-এ চলার সময় ওঠা ত্রুটি, যা try/except দিয়ে ধরা যায়।
Exit code	প্রোগ্রাম শেষ হওয়ার সময় ফেরত দেওয়া সংখ্যা; 0 মানে সফল, nonzero মানে ব্যর্থতা।
Export	Bash-এ ভেরিয়েবলকে child process-এর কাছে পৌঁছে দেওয়ার নির্দেশ।
Fast-forward merge	main-এ নতুন commit না থাকলে pointer শুধু এগিয়ে দিয়ে merge করা; merge commit তৈরি হয় না।
Fetch	remote-এর পরিবর্তন ডাউনলোড করা, কিন্তু working directory না বদলানো।
Field separator	text-কে field-এ ভাগ করার চিহ্ন; যেমন space, comma বা colon।
File descriptor	খোলা ফাইল বা stream-এর সংখ্যাগত পরিচয়; 0 = stdin, 1 = stdout, 2 = stderr।
File permission	কে ফাইল পড়তে, লিখতে বা চালাতে পারবে, তার নিয়ম; যেমন 644, 755।
Fixture	pytest-এ test-এর জন্য আগে থেকে তৈরি করা তথ্য বা পরিবেশ।
Fork	GitHub-এ অন্যের repository-র নিজস্ব অনুলিপি তৈরি করা।
Function	নাম দিয়ে ডাকা যায় এমন পুনর্ব্যবহারযোগ্য কোডের অংশ।
Git	distributed version control ব্যবস্থা; পরিবর্তনের ইতিহাস সংরক্ষণ করে।
GitHub	Git repository হোস্ট করা ও collaboration-এর অনলাইন সেবা।
Gitignore	Git যে ফাইলগুলো track করবে না, তার তালিকা ধারণকারী `.gitignore` ফাইল।
Glob	ফাইলের নাম মেলানোর pattern; যেমন `*.log`।

grep	ফাইলে নির্দিষ্ট pattern খোঁজার Linux tool।
Hash	তথ্য থেকে তৈরি নির্দিষ্ট দৈর্ঘ্যের অনন্য মান; Git-এ প্রতিটি object-এর পরিচয়।
HEAD	বর্তমান branch বা commit-কে নির্দেশ করা Git pointer।
Health check	সার্ভার বা সেবা ঠিকমতো চলছে কি না, তা পরীক্ষা করা।
Host	নেটওয়ার্কে যুক্ত কম্পিউটার বা সার্ভার।
HTTP	ওয়েবে তথ্য আদান-প্রদানের protocol।
HTTPS	TLS দিয়ে encrypt করা HTTP; নিরাপদ ওয়েব যোগাযোগের মান।
Idempotency	একই কাজ একাধিকবার চালালেও একই ফল পাওয়ার গুণ।
Index (staging area)	পরের commit-এ যাবে এমন পরিবর্তন রাখার Git-এর অস্থায়ী জায়গা।
Infrastructure	সার্ভার, নেটওয়ার্ক ও storage-সহ কোনো ব্যবস্থা চালানোর ভিত্তিগত কাঠামো।
Input validation	ব্যবহারকারীর দেওয়া তথ্য ব্যবহারের আগে তা সঠিক ও নিরাপদ কি না যাচাই করা।
Integration test	একাধিক অংশ একসঙ্গে কাজ করে কি না, তা পরীক্ষা করা।
Interpreter	source code লাইন ধরে ধরে অনুবাদ করে চালানো প্রোগ্রাম; যেমন Python interpreter।
Issue	GitHub-এ bug, কাজ বা আলোচনা track করার item।
JSON	JavaScript Object Notation — হালকা, মানুষের পড়ার মতো তথ্য বিনিময়ের ফরম্যাট।
Key pair	public key ও private key-এর জোড়া; SSH authentication-এ ব্যবহৃত হয়।
Known hosts	SSH-এ আগে যাচাই করা সার্ভারের host key-এর তালিকা (`~/.ssh/known_hosts`)।
Library	পুনর্ব্যবহারযোগ্য function-এর সংগ্রহ; যেমন Python-এর requests library।
Linting	কোড চালানোর আগেই ভুল ও শৈলীর সমস্যা খুঁজে বের করা।
Linux	open-source operating system kernel ও তার উপর ভিত্তি করে তৈরি OS পরিবার।
Local variable	শুধু একটি function-এর ভেতরে কার্যকর ভেরিয়েবল।
Lock file	একই কাজ একসঙ্গে দুবার চলা ঠেকাতে ব্যবহৃত ফাইল।
Log level	log বার্তার গুরুত্বের স্তর; যেমন DEBUG, INFO, WARNING, ERROR, CRITICAL।
Logging	প্রোগ্রাম কী করছে, তার সময়সহ লিখিত রেকর্ড রাখা।
Loop	একই কাজ বারবার করার গঠন; যেমন for ও while।
Main branch	repository-র প্রধান branch; সাধারণত স্থিতিশীল ও deployable অবস্থায় রাখা হয়।
Man page	Linux কমান্ডের আনুষ্ঠানিক নির্দেশিকা; `man ls` দিয়ে দেখা যায়।

Merge	এক branch-এর পরিবর্তন অন্য branch-এ একত্র করা।
Merge conflict	একই লাইনে ভিন্ন পরিবর্তন থাকায় merge স্বয়ংক্রিয়ভাবে না হওয়া।
Mock	test-এ আসল বস্তুর বদলে ব্যবহৃত নকল বস্তু; বাইরের সেবা ছাড়া test চালানো যায়।
Module	Python-এ পুনর্ব্যবহারযোগ্য কোড ধারণকারী ফাইল।
Monitoring	system-এর অবস্থা নিয়মিত পর্যবেক্ষণ করা।
Mutable	তৈরির পর বদলানো যায় এমন; যেমন Python list।
Namespace	নামগুলোকে আলাদা গোষ্ঠীতে ভাগ করার ব্যবস্থা, যাতে নামের সংঘাত না হয়।
Networking	কম্পিউটারগুলোর মধ্যে সংযোগ ও তথ্য আদান-প্রদানের ব্যবস্থা।
Nullglob	Bash option, যাতে কোনো ফাইল না মিললে glob pattern খালি হয়ে যায়, আক্ষরিক থাকে না।

পরিশিষ্ট E (চলমান) • Glossary — O–Z

Glossary (চলমান): O–Z

Term	বাংলা অর্থ
Object database	Git-এর `.git/objects` — blob, tree ও commit object সংরক্ষণের জায়গা।
Observability	log, metric ও status দিয়ে system-এর ভেতরের অবস্থা বোঝার ক্ষমতা।
Open source	যে software-এর source code সবার জন্য উন্মুক্ত ও ব্যবহারযোগ্য।
Operating system	hardware ও প্রোগ্রামের মধ্যে সমন্বয়কারী software; যেমন Ubuntu।
Package	install করার উপযোগী software-এর বান্ডিল; Python-এ pip দিয়ে install করা হয়।
Pagination	বড় ফলাফলকে ছোট ছোট পৃষ্ঠায় ভাগ করে দেওয়া; API-তে সাধারণ।
Parameter	function বা কমান্ডে পাঠানো মানের নাম; argument হলো তার বাস্তব মান।
Parent commit	কোনো commit-এর ঠিক আগের commit; ইতিহাসের শৃঙ্খল তৈরি করে।
Password	পরিচয় প্রমাণের গোপন শব্দ; কখনো code বা Git-এ লেখা যাবে না।
Path	ফাইল বা directory-র অবস্থান; যেমন `/var/log/syslog`।
PATH variable	shell কমান্ড খুঁজতে যে directory-গুলো দেখে, তার তালিকা।
Permission	কে কী করতে পারবে, তার নিয়ম; ফাইল, directory ও সেবায় প্রযোজ্য।
Pipeline	এক কমান্ডের output অন্য কমান্ডের input হিসেবে দেওয়ার শৃঙ্খল; যেমন `ls | grep txt`।
Pipefail	Bash option, যাতে pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline ব্যর্থ ধরা হয়।
Port	নেটওয়ার্কে নির্দিষ্ট সেবা চেনার সংখ্যা; যেমন SSH-এর 22, HTTPS-এর 443।
Process	চলমান প্রোগ্রামের একটি instance; প্রতিটির নিজস্ব process ID থাকে।
Production	আসল ব্যবহারকারীরা ব্যবহার করে এমন live পরিবেশ।
Pull	remote-এর পরিবর্তন এনে local branch-এ মেলানো (fetch + merge)।
Pull request	এক branch-এর পরিবর্তন অন্য branch-এ merge করার প্রস্তাব ও review-এর জায়গা।
Push	local commit remote repository-তে পাঠানো।
pytest	Python-এর জনপ্রিয় testing framework; `test_` দিয়ে শুরু হওয়া function চালায়।
Python	সহজে পড়া যায় এমন জনপ্রিয় প্রোগ্রামিং ভাষা; এই বইয়ের automation ভাষা।
Rate limiting	নির্দিষ্ট সময়ে কতগুলো request করা যাবে, তার সীমা; বেশি হলে 429 status আসে।
Read-only	শুধু পড়া যায়, বদলানো যায় না এমন অবস্থা।

Rebase	commit-গুলোকে নতুন base-এর উপর আবার প্রয়োগ করে সরলরৈখিক ইতিহাস বানানো।
Recovery	ব্যর্থতা বা তথ্য হারানোর পর আগের সঠিক অবস্থায় ফেরা।
Redirect	কমান্ডের output বা error-কে পর্দার বদলে ফাইলে বা অন্য জায়গায় পাঠানো।
Reference (ref)	Git-এ commit-কে নির্দেশ করা নাম; যেমন branch ও tag।
Refactoring	আচরণ না বদলে কোডের গঠন পরিষ্কার ও উন্নত করা।
Regression test	নতুন পরিবর্তনে আগের কাজ ভেঙে গেছে কি না, তা যাচাই করার test।
Regular expression	text-এর pattern মেলানোর সংক্ষিপ্ত ভাষা; যেমন `^[0-9]+$`।
Release	ব্যবহারের জন্য প্রকাশিত software-এর নির্দিষ্ট সংস্করণ।
Remote	অন্য জায়গায় থাকা repository-র নাম; সাধারণত `origin`।
Repository	Git-এ প্রজেক্টের ফাইল ও পূর্ণ ইতিহাস ধারণকারী ভাণ্ডার।
Request	client থেকে সার্ভারের কাছে তথ্য চাওয়ার বার্তা।
Response	request-এর উত্তরে সার্ভারের পাঠানো বার্তা ও তথ্য।
Retry	ব্যর্থ কাজ নির্দিষ্ট নিয়মে আবার চেষ্টা করা।
Return code	exit code-এর আরেক নাম; function বা প্রোগ্রামের সমাপ্তির সংকেত।
Rollback	ব্যর্থ deployment বা পরিবর্তনের পর আগের সঠিক সংস্করণে ফেরা।
Root user	Linux-এর সর্বোচ্চ ক্ষমতাসম্পন্ন ব্যবহারকারী; অসতর্ক ব্যবহার বিপজ্জনক।
rsync	শুধু বদলে যাওয়া অংশ কপি করে ফাইল/ফোল্ডার sync করার tool।
Runtime	প্রোগ্রাম চলার সময়ের পরিবেশ ও অবস্থা।
scp	SSH দিয়ে নিরাপদে ফাইল কপি করার কমান্ড।
Script	কমান্ড বা নির্দেশনার লিখিত ফাইল, যা চালালে কাজগুলো একে একে হয়।
SDK	Software Development Kit — নির্দিষ্ট platform-এ কাজ করার tool ও library-র সংগ্রহ।
Secret	API key, token বা password-এর মতো গোপন তথ্য; code-এ লেখা যাবে না।
sed	stream editor — text-এর নির্দিষ্ট অংশ বদলানোর Linux tool।
Semantic versioning	major.minor.patch নিয়মে সংস্করণ দেওয়া; যেমন 2.3.1।
Server	অন্য কম্পিউটারকে সেবা দেওয়া কম্পিউটার বা প্রোগ্রাম।
Set -e	Bash option — কোনো কমান্ড ব্যর্থ হলে script থেমে যাবে।
Set -u	Bash option — অনির্ধারিত ভেরিয়েবল ব্যবহার করলে error হবে।
Shell	ব্যবহারকারীর কমান্ড পড়ে operating system-কে চালাতে বলা প্রোগ্রাম; যেমন Bash।

ShellCheck	Bash script-এর ভুল ধরার static analysis tool।
Shell injection	আক্রমণকারী input-এর মাধ্যমে shell-এ নিজের কমান্ড চালানোর আক্রমণ।
Signal	process-কে পাঠানো বার্তা; যেমন SIGTERM (থামতে বলা) বা SIGINT (Ctrl+C)।
Snapshot	কোনো নির্দিষ্ট সময়ে পুরো প্রজেক্টের অবস্থার অনুলিপি; Git commit-এর ভিত্তি।
Sorting	তথ্যকে নির্দিষ্ট ক্রমে সাজানো; যেমন `sort` কমান্ড।
Source code	মানুষের লেখা প্রোগ্রামের মূল লেখা।
SSH	Secure Shell — দূরের কম্পিউটারে encrypt করা নিরাপদ সংযোগের protocol।
SSH agent	private key মনে রেখে বারবার passphrase না চেয়ে SSH-এ সাহায্য করা প্রোগ্রাম।
Staging area	পরের commit-এ যাবে এমন পরিবর্তন রাখার Git-এর জায়গা; index নামেও পরিচিত।
Standard error (stderr)	ত্রুটি ও সতর্কবার্তা পাঠানোর stream; file descriptor 2।
Standard input (stdin)	প্রোগ্রামে তথ্য ঢোকানোর stream; file descriptor 0।
Standard output (stdout)	প্রোগ্রামের আসল ফলাফল পাঠানোর stream; file descriptor 1।
Status code	HTTP উত্তরের সংখ্যাগত সংকেত; যেমন 200 সফল, 404 পাওয়া যায়নি, 500 সার্ভার ত্রুটি।
String	অক্ষরের ক্রম; যেমন `"hello"`।
Subprocess	Python থেকে চালানো আলাদা process; `subprocess` module দিয়ে নিয়ন্ত্রণ করা হয়।
sudo	সাধারণ ব্যবহারকারীকে সাময়িকভাবে root ক্ষমতায় কমান্ড চালানোর অনুমতি দেওয়া।
systemd	আধুনিক Linux-এ সেবা ও system পরিচালনার ব্যবস্থা।
systemd timer	cron-এর বিকল্প হিসেবে systemd দিয়ে সময়মতো কাজ চালানোর ব্যবস্থা।
Tag	নির্দিষ্ট commit-কে চিহ্নিত করা স্থির নাম; যেমন `v1.0.0`।
Test	কোড ঠিকমতো কাজ করে কি না, তা যাচাই করার প্রক্রিয়া বা কোড।
Timeout	কোনো কাজের জন্য অপেক্ষার সর্বোচ্চ সময়; এরপর ব্যর্থ ধরা হয়।
Token	API-তে পরিচয় প্রমাণের গোপন string; password-এর মতোই সুরক্ষিত রাখতে হয়।
Traceback	Python-এ exception-এর সময় দেখানো ভুলের উৎসের ধারাবাহিক তালিকা।
Tree object	Git object, যা directory-র তালিকা ধারণ করে — কোন নামে কোন blob বা sub-tree আছে।
Troubleshooting	সমস্যার কারণ খুঁজে ধাপে ধাপে সমাধান করার প্রক্রিয়া।
Tuple	Python-এ অপরিবর্তনীয় (immutable) মানের ক্রম; যেমন `(1, 2)`।
Unit test	কোডের একটি ছোট অংশ একা পরীক্ষা করা।
Untracked file	Git এখনো track করে না এমন ফাইল; `git status`-এ দেখা যায়।

User input	ব্যবহারকারীর দেওয়া তথ্য; যাচাই ছাড়া ব্যবহার করা নিরাপদ নয়।
Variable	মান ধারণ করার নামযুক্ত জায়গা; যেমন `name="Nahid"`।
Virtual environment	Python প্রজেক্টের dependency আলাদা রাখার বিচ্ছিন্ন পরিবেশ (venv)।
Version control	ফাইলের পরিবর্তনের ইতিহাস সংরক্ষণ ও পরিচালনার ব্যবস্থা।
Webhook	কোনো ঘটনা ঘটলে সার্ভার থেকে অন্য সার্ভারে স্বয়ংক্রিয়ভাবে পাঠানো HTTP বার্তা।
While loop	শর্ত সত্য থাকা পর্যন্ত বারবার চলা loop।
Working directory	বর্তমানে কাজ করা directory; Git-এ ফাইল সম্পাদনার জায়গা।
YAML	মানুষের পড়ার উপযোগী structured data ফরম্যাট; config ফাইলে ব্যবহৃত হয়।
Zip archive	একাধিক ফাইল compress করে এক ফাইলে রাখার পদ্ধতি; যেমন `.zip` বা `.tar.gz`।

মোট পরিভাষা: এই পরিশিষ্টে ১৯৭টি term আছে — A–N অংশে ১০৮টি এবং O–Z অংশে ৮৯টি।

পরিশিষ্ট F • References

Appendix F: References — তথ্যসূত্র

এই বইয়ের সব কমান্ড, syntax ও আচরণ নিচের আনুষ্ঠানিক (official) নথিগুলোর সঙ্গে মিলিয়ে যাচাই করা হয়েছে। প্রকাশের সময় ব্যবহৃত পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। কোনো তথ্য নিয়ে সন্দেহ হলে প্রথমে এই উৎসগুলো দেখুন — কোনো ব্লগ বা ভিডিও নয়।

#	উৎস	URL	কী পাবেন
1	Git Documentation	https://git-scm.com/doc	Git-এর আনুষ্ঠানিক নথি ও সব কমান্ডের ব্যাখ্যা।
2	Pro Git Book	https://git-scm.com/book/en/v2	Git-এর ভেতরের কাজ, branching ও workflow-এর পূর্ণাঙ্গ বই।
3	GitHub Docs	https://docs.github.com	GitHub repository, pull request, issue ও নিরাপত্তার নথি।
4	GitHub — About Pull Requests	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests	Pull request কী ও কীভাবে review হয়, তার ব্যাখ্যা।
5	GitHub — SSH Keys	https://docs.github.com/en/authentication/connecting-to-github-with-ssh	SSH key তৈরি ও GitHub-এ যোগ করার ধাপ।
6	GNU Bash Manual	https://www.gnu.org/software/bash/manual/	Bash-এর আনুষ্ঠানিক manual — variable, loop, function ও option।
7	GNU Coreutils Manual	https://www.gnu.org/software/coreutils/manual/	ls, cp, grep, sort, cut, find-সহ মূল Linux utility-র নথি।
8	Bash Reference — Shell Expansions	https://www.gnu.org/software/bash/manual/html_node/Shell-Expansions.html	quoting, word splitting ও command substitution-এর নিয়ম।
9	Python Documentation (3.12)	https://docs.python.org/3/	Python ভাষা, library ও tutorial-এর আনুষ্ঠানিক নথি।
10	Python — subprocess	https://docs.python.org/3/library/subprocess.html	subprocess দিয়ে নিরাপদে কমান্ড চালানো ও output ধরা।
11	Python — venv	https://docs.python.org/3/library/venv.html	virtual environment তৈরি ও ব্যবহারের নথি।

12	Python — logging	https://docs.python.org/3/library/logging.html	logging module, level ও handler-এর ব্যাখ্যা।
13	Python — argparse	https://docs.python.org/3/library/argparse.html	command-line argument parsing-এর নথি।
14	Python — pathlib	https://docs.python.org/3/library/pathlib.html	pathlib দিয়ে ফাইল ও directory নিয়ে কাজের নথি।
15	Python — json	https://docs.python.org/3/library/json.html	JSON পড়া ও লেখার নথি।
16	Python — shutil	https://docs.python.org/3/library/shutil.html	ফাইল কপি, সরানো ও archive করার নথি।
17	requests Documentation	https://requests.readthedocs.io/	Python-এ HTTP request পাঠানো, timeout ও error handling।
18	PyYAML Documentation	https://pyyaml.org/wiki/PyYAMLDocumentation	YAML পড়া ও লেখার নথি।
19	pytest Documentation	https://docs.pytest.org/	pytest দিয়ে test লেখা, fixture ও mocking-এর নথি।
20	ShellCheck	https://www.shellcheck.net/	Bash script যাচাইয়ের tool ও তার সতর্কবার্তার ব্যাখ্যা।
21	ShellCheck (GitHub)	https://github.com/koalaman/shellcheck	ShellCheck-এর source code ও প্রতিটি সতর্কবার্তার তালিকা।
22	Ubuntu Server Documentation	https://ubuntu.com/server/docs	Ubuntu Server install, পরিচালনা ও সেবার নথি।
23	OpenSSH Manual	https://www.openssh.com/manual.html	SSH, sshd ও ssh-keygen-এর আনুষ্ঠানিক নথি।
24	man7.org — Linux man-pages	https://man7.org/linux/man-pages/	Linux man page-গুলোর অনলাইন সংগ্রহ; যেমন cron, bash, ssh।
25	man7.org — crontab(5)	https://man7.org/linux/man-pages/man5/crontab.5.html	crontab ফাইলের field ও syntax-এর আনুষ্ঠানিক ব্যাখ্যা।
26	systemd — Timers	https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html	systemd timer দিয়ে সময়মতো কাজ চালানোর নথি।
27	systemd — systemctl	https://www.freedesktop.org/software/systemd/man/latest/systemctl.html	সেবা দেখা, চালু ও বন্ধ করার systemctl-এর নথি।

28	Git — gitignore Documentation	https://git-scm.com/docs/gitignore	.gitignore pattern-এর নিয়ম।
29	Git — git-branch Documentation	https://git-scm.com/docs/git-branch	branch তৈরি, তালিকা ও মোছার নথি।
30	Git — git-rebase Documentation	https://git-scm.com/docs/git-rebase	rebase-এর ব্যবহার ও সতর্কতার নথি।

> **CONCEPT:** এই বইয়ের কোনো উদাহরণ চালাতে গিয়ে ভিন্ন ফল পেলে প্রথমে সংস্করণ মিলিয়ে দেখুন (Git 2.43, Bash 5.2, Python 3.12) — তারপর উপরের আনুষ্ঠানিক নথিতে সংশ্লিষ্ট পৃষ্ঠাটি খুলুন। সংস্করণ বদলালে আচরণ বদলাতে পারে, কিন্তু নীতিগুলো একই থাকে।

DEVOPS ENGINEERING MASTERY — Volume 03: Git, Scripting & Automation Engineering — First Edition, 8 November 2026। পরীক্ষিত পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12।